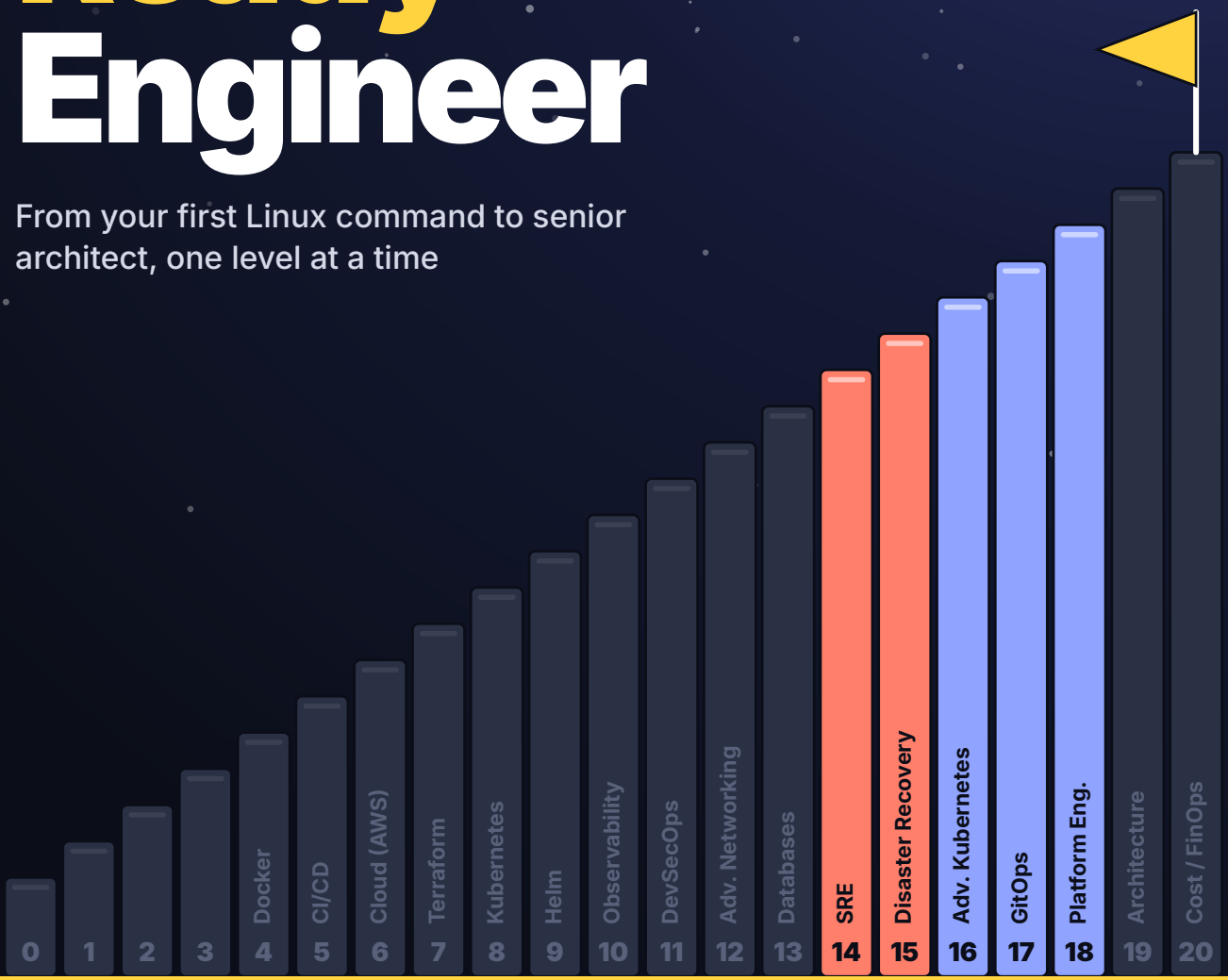
THE COMPLETE DEVOPS ROADMAP · VOLUME 5 OF 6

The Production-Ready Engineer

From your first Linux command to senior architect, one level at a time



Ubong Eduok

Reliability and Platforms at Scale

5 levels · **47** topics · **5** capstones

The Production-Ready Engineer

Volume 5: Reliability and Platforms at Scale

THE PRINT SET

The Production-Ready Engineer is published in six volumes, cut at Part boundaries so that each one stands on its own and binds cleanly. Page numbers run continuously through the set, and the index in Volume 6 covers all of it.

1	**Foundations** Base Camp and Part I: DevOps, Linux, Networking, Git	pp. 1–734
2	**Build, Ship and Cloud** Parts II and III: Docker, CI/CD, AWS, Terraform	pp. 735–1544
3	**Orchestrate, See and Secure** Parts IV and V: Kubernetes, Helm, Observability, DevSecOps	pp. 1545–2318
4	**Under the Hood** Part VI: Advanced Networking, Databases and Distributed Systems	pp. 2319–2814
5	**Reliability and Platforms at Scale** **Parts VII and VIII: SRE, Disaster Recovery, Advanced Kubernetes, GitOps, Platform Engineering**	**pp. 2815–3548**
6	**Senior Altitude and the Summit** Parts IX and X, Appendices and Index: Architecture, Cost, the Nine Projects	pp. 3549–3972

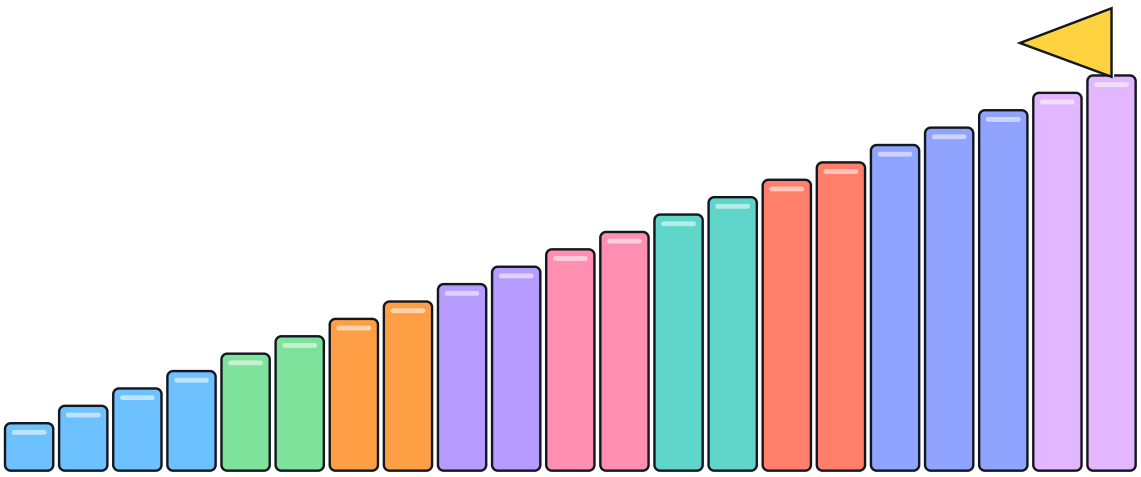
THE COMPLETE DEVOPS ROADMAP

The Production-Ready Engineer

The complete DevOps roadmap, from your first Linux command to senior architect

Volume 5 of 6: Reliability and Platforms at Scale

Ubong Eduok



github.com/mascot200/devops-roadmap

The Production-Ready Engineer
The complete DevOps roadmap, from your first Linux command to senior architect
Volume 5 of 6: Reliability and Platforms at Scale

Copyright © 2026 Ubong Eduok.

This book is released under the MIT License; the full text is printed in Appendix D, in Volume 6. You may use, copy, modify and share this material, including in your own courses and projects, as long as the copyright and license notice are kept.

Everything here is provided "as is", without warranty of any kind. Run the labs in sandboxes. Read the cost warnings before using a real cloud account, and finish each lab with its teardown steps. Commands that change systems can cause damage or charges; you are responsible for where you run them.

Product and company names are trademarks of their respective owners and are used here only to identify and teach their technology. Their use implies no affiliation or endorsement. Tool icons in the author portrait are from Devicon (MIT licence).

Tools change quickly. Command output in this book is illustrative, and version numbers will age. When in doubt, check the official documentation named in each chapter's further reading.

Source: github.com/mascot200/devops-roadmap, typeset from commit faf55fa.
First edition, 2026.

Do not aim to become the person who knows 50 DevOps tools.

Aim to become the engineer who can take a software application and independently make it production-ready.

Secure, observable, scalable, highly available, automatically deployable, recoverable, and cost-efficient.

How to Read This Book

The anatomy of a chapter

All 248 topic chapters follow the same eleven sections, which fall into three movements. Appendix A, in Volume 6, prints the template that every chapter was written against.

UNDERSTAND	1	**What it is**	A plain-English explanation, with every acronym defined on first use.
	2	**Why it matters**	What breaks, costs money, or wakes someone at 3 a.m. when this is misunderstood.
	3	**Real-world system**	How the topic shows up in a named, publicly documented system.
	4	**Diagram**	The real mechanism: a flow, a sequence, an architecture or a state machine.
	5	**How it works**	Key concepts, comparison tables and the commands that matter.
DO	6	**Step-by-step guide**	A lab you can type in a sandbox, with what to look for at each step.
	7	**Real-world scenario**	A short story under pressure: symptom, investigation, fix, lesson.
	8	**Mistakes and practices**	Two short lists: what not to do, and what to do.
PROVE	9	**Tasks**	Hands-on work, easiest first, each with a "done when" check. The last says Prove it.
	10	**Self-check**	Questions a senior engineer would expect you to answer without notes.
	11	**Further reading**	Official documentation and well-known books.

The eleven sections of every topic chapter

Each level opens with an overview of its goal, recommended time and topics, and closes with a **capstone project** marked with a star, which pulls all of that level's topics together.

Conventions used in print

- **Cross-references.** Where the repository has a hyperlink, this book prints a small tag after the words: Docker Compose §4.8 means Level 4, topic 8. Running heads carry the same numbers, so you can flip straight to it. In a PDF the tags are clickable when the target is in the same volume.
- **Check-boxes.** Every task has a box to tick with a pencil. Where the text mentions changing `- [ ]` to `- [x]`, that is the same idea for readers who keep their progress in a fork of the repository.

- **Answers.** On GitHub the self-check answers are folded away. In print they sit in a shaded box straight after the questions, so cover it with a card and answer first.
- **Code.** Long lines wrap, and the continuation is indented. Type a wrapped line as one line. Output is illustrative and yours may differ in the details.
- **Diagrams.** A few wide diagrams are printed sideways. Turn the book clockwise to read them.
- **Lab notes.** Chapters start on a right-hand page, and any left-hand page this leaves empty is ruled for your notes. Use them. For each task, write what you ran and what you saw. Explaining output is the skill.

Stay safe, and keep it cheap

Use disposable sandboxes: local virtual machines, Docker, kind or minikube, LocalStack, or a small VPS. Anything that can cost money says so and ends with teardown steps. Never practise on a system you cannot afford to lose.

The Whole Climb on One Page



PART I: THE GROUND BENEATH EVERYTHING				
0	What DevOps Actually Is	1 week (reading + mapping)	13 topics	35
1	Linux Fundamentals	2-4 weeks	13 topics	213
2	Networking	2-4 weeks	11 topics	397
3	Git & GitHub	2-3 weeks	13 topics	563
PART II: BUILD IT, SHIP IT				
4	Docker	2-3 weeks	14 topics	737
5	CI/CD	2-3 weeks	12 topics	931
PART III: CLOUD YOU CAN REBUILD				
6	Cloud (AWS)	4-6 weeks	18 topics	1137
7	Terraform / Infrastructure as Code	2-3 weeks	9 topics	1403
PART IV: ORCHESTRATION				
8	Kubernetes	6-10 weeks	19 topics	1547
9	Helm	1-2 weeks	6 topics	1823
PART V: SEE IT, SECURE IT				
10	Observability	3-4 weeks	11 topics	1919
11	Security / DevSecOps	3-4 weeks	13 topics	2123
PART VI: UNDER THE HOOD				
12	Advanced Networking	3-4 weeks	14 topics	2321
13	Databases & Distributed Systems	4-6 weeks	15 topics	2577
PART VII: KEEP IT RUNNING				
14	SRE	3-4 weeks	10 topics	2817
15	Disaster Recovery	2-3 weeks	10 topics	2983
PART VIII: PLATFORMS AT SCALE				
16	Advanced Kubernetes	4-6 weeks	10 topics	3137
17	GitOps	2-3 weeks	7 topics	3303
18	Platform Engineering	4-6 weeks	10 topics	3403
PART IX: SENIOR ALTITUDE				
19	Architecture	Ongoing	11 topics	3551
20	Cost Optimization	2-3 weeks	9 topics	3703

Level numbers follow the roadmap document; the recommended learning order differs slightly (see Base Camp). The nine Summit Projects in Part X then chain the levels into one system.

Contents of Volume 5

PART VII **Keep It Running** 2815

14	**SRE**	**2817**
14.1	Service Level Indicators (SLIs)	2819
14.2	Service Level Objectives (SLOs)	2831
14.3	Service Level Agreements (SLAs)	2841
14.4	Error Budgets	2851
14.5	Incident Management	2867
14.6	On-call Practices	2879
14.7	Postmortems	2893
14.8	Alert Fatigue Reduction	2907
14.9	Capacity Planning	2923
14.10	Reliability Engineering	2939
★	Capstone: Reliability for shop-api: SLOs, burn-rate alerts and a mock incident	2955
15	**Disaster Recovery**	**2983**
15.1	Backups	2985
15.2	Replication	2997
15.3	Snapshots	3007
15.4	Restore Procedures	3019
15.5	Failover	3033
15.6	RPO - Recovery Point Objective	3047
15.7	RTO - Recovery Time Objective	3059
15.8	Off-site Backups	3073
15.9	Testing Backup Restoration	3089
15.10	Failure Scenario Drills	3103
★	Capstone: Break It, Time It, Write the Plan	3121

PART VIII **Platforms at Scale** 3135

16	**Advanced Kubernetes**	**3137**
16.1	RBAC in Depth	3139
16.2	NetworkPolicies in Depth	3153
16.3	Pod Security	3167
16.4	Operators	3179
16.5	Custom Resource Definitions (CRDs)	3191
16.6	Admission Controllers and Webhooks	3205
16.7	HPA, VPA and Cluster Autoscaler	3221

16.8	Stateful Workloads	3237
16.9	Service Mesh with Istio or Linkerd	3251
16.10	Mesh: mTLS, Routing, Retries, Timeouts, Circuit Breaking and Observability	3265
★	Capstone: Harden a kind/EKS Cluster	3283
17	**GitOps**	3303
17.1	Desired State Management	3305
17.2	Argo CD and Flux	3317
17.3	Continuous Reconciliation	3329
17.4	Environment Promotion	3341
17.5	Application Synchronization	3351
17.6	Rollback Through Git	3365
17.7	The Full Flow: CI to GitOps	3375
★	Capstone: GitOps Delivery for shop-api	3389
18	**Platform Engineering**	3403
18.1	Internal Developer Platforms	3405
18.2	Developer Portals	3415
18.3	Self-service Infrastructure	3427
18.4	Golden Paths	3439
18.5	Service Catalogs	3451
18.6	Infrastructure Templates	3463
18.7	Policy as Code	3475
18.8	Backstage	3491
18.9	Crossplane	3505
18.10	Integrating Argo CD, Terraform and Kubernetes into a Platform	3521
★	Capstone: A Mini Internal Developer Platform	3535

PART VII

Keep It Running

Reliability as an engineering discipline, and the plans and drills that bring a system back when the worst happens.



IN THIS PART

14	SRE	2817
15	Disaster Recovery	2983

LAB NOTES What I ran, what I saw, what it means

LEVEL 14 · PART VII

SRE

14

Goal: Run services with measurable reliability: define SLOs, manage incidents and learn from failure.

RECOMMENDED TIME
3-4 weeks

TOPICS
10

PRACTICE / CAPSTONE
Define SLIs/SLOs for the sample app, build error-budget burn-rate alerts, run a mock incident and write a postmortem.

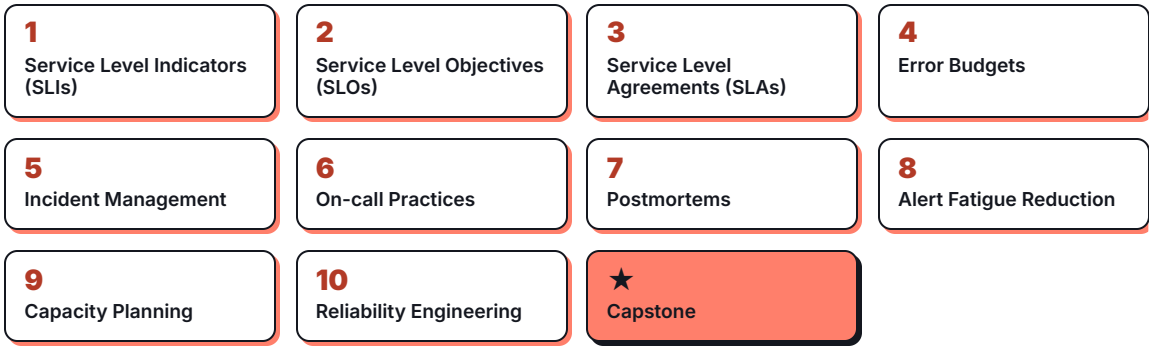
Topics

#	Topic	What you will learn
1	Service Level Indicators (SLIs) **p.2819**	choosing good SLIs, request-based vs window-based, measurement points
2	Service Level Objectives (SLOs) **p.2831**	setting targets, nines, rolling windows, SLO docs
3	Service Level Agreements (SLAs) **p.2841**	SLA vs SLO, credits, contractual vs internal
4	Error Budgets **p.2851**	budget math, burn rate alerts, release policy
5	Incident Management **p.2867**	roles (IC, comms, ops), severity levels, war rooms, status pages
6	On-call Practices **p.2879**	rotations, handoffs, runbooks, escalation, sustainable load
7	Postmortems **p.2893**	blameless culture, timeline, root cause vs contributing factors, action items
8	Alert Fatigue Reduction **p.2907**	symptom-based alerts, actionable alerts, deduplication, alert review
9	Capacity Planning **p.2923**	load testing, headroom, forecasting, saturation
10	Reliability Engineering **p.2939**	toil, automation, chaos engineering, game days, resilience patterns
★	Level capstone project **p.2955**	Define SLIs/SLOs for the sample app, build error-budget burn-rate alerts, run a mock incident and write a postmortem.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



14.1 Service Level Indicators (SLIs)

Topic 1 of 10 | ~3 hours | Difficulty: Intermediate

1 What it is

A **Service Level Indicator (SLI)** is a quantitative measure of one aspect of how well a service is doing its job *from the point of view of its users*. "CPU is at 70%" is not an SLI. "The proportion of checkout requests that returned a correct answer in under 300 ms" is.

The definition used throughout Google's SRE literature (see *Site Reliability Engineering*, the chapter on Service Level Objectives, and *The Site Reliability Workbook*, the chapter "Implementing SLOs") boils down to a single ratio:

```
SLI = good events / valid events        (a number between 0 % and 100 %)
```

"Events" can be HTTP requests, messages processed, jobs run, or one-minute time slices. "Valid" excludes things that are not the service's fault or not in scope (for example health checks, or requests rejected as `400` because the client sent garbage). Expressing every SLI as a percentage makes it comparable, graphable, and directly usable by the next two topics: SLOs §14.2 (the target for the SLI) and error budgets §14.4 (what is left of that target).

2 Why it matters in DevOps

- **It replaces opinions with numbers.** "The site felt slow yesterday" becomes "latency SLI was 97.1% against a 99% objective".
- **It points monitoring at users, not machines.** A server can be at 5% CPU and returning `500` to everyone; a server at 95% CPU can be perfectly healthy. Users only care about the outcome.
- **It is the foundation of the whole level.** Without a trustworthy SLI there is no SLO, no error budget, no burn-rate alert and no objective way to say "we are allowed to ship" or "we must stop and fix reliability".

- **Bad SLIs cause bad decisions.** An SLI that cannot see a failure mode (for example, measuring only at the app server while the load balancer is dropping connections) gives false confidence during real outages.

3 Real-world system example

Google and its published SRE practice. Google's SRE books describe SLIs as the starting point and recommend choosing a small number of them per user journey, grouped by the type of system: for *request-driven* services availability, latency and quality; for *data-processing pipelines* freshness, coverage, correctness and throughput; for *storage systems* latency, availability and durability. The same books distinguish the SLI *specification* (what we want to measure, in words) from the SLI *implementation* (the exact query and measurement point), a distinction worth copying.

Google Cloud Monitoring (a public product) exposes this model directly: you define an SLO on a **request-based** SLI (a ratio of good to total requests) or a **windows-based** SLI (the fraction of time windows in which a threshold was met).

AWS building blocks. An Application Load Balancer publishes `RequestCount`, `HTTPCode_Target_5XX_Count` and `TargetResponseTime` metrics to CloudWatch (see ALB §6.14). Those are exactly the raw material for an availability and latency SLI measured at the load balancer, which is why many teams choose that measurement point.

Open-source tools such as **Prometheus**, **Sloth** and **Pyrra** generate SLI recording rules and burn-rate alerts from a short SLO definition, which shows how mechanical the ratio becomes once the specification is clear.

4 Diagram

How a user request becomes an SLI value, and the places you can measure it:

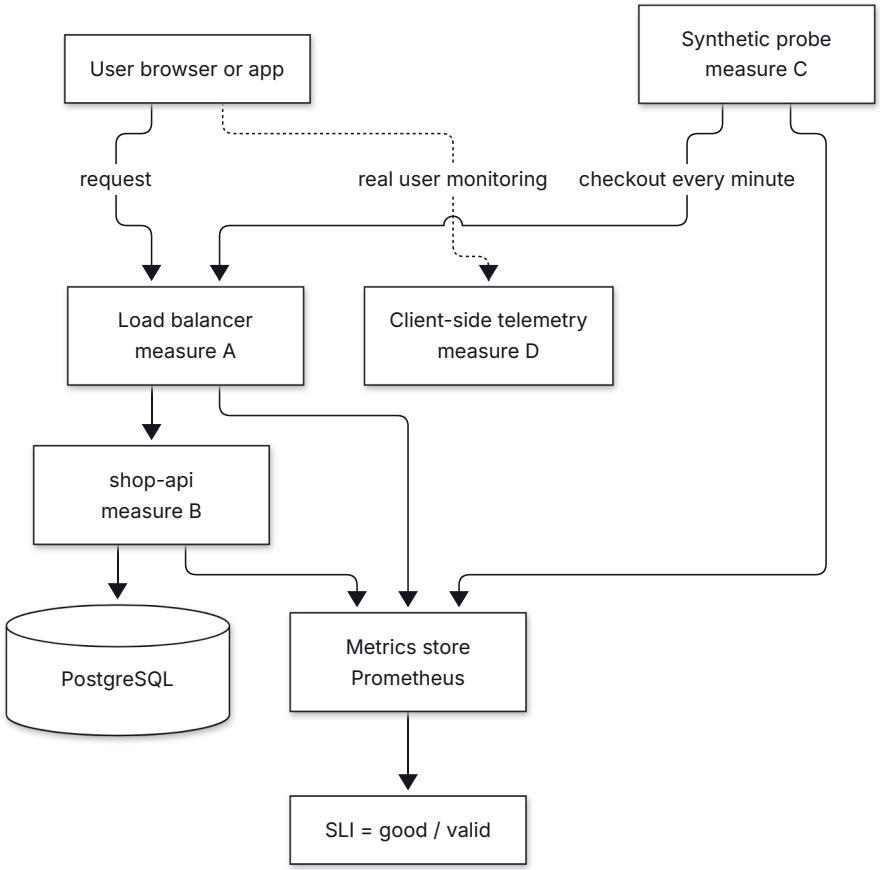


Figure 14.1-1

From raw events to a number:

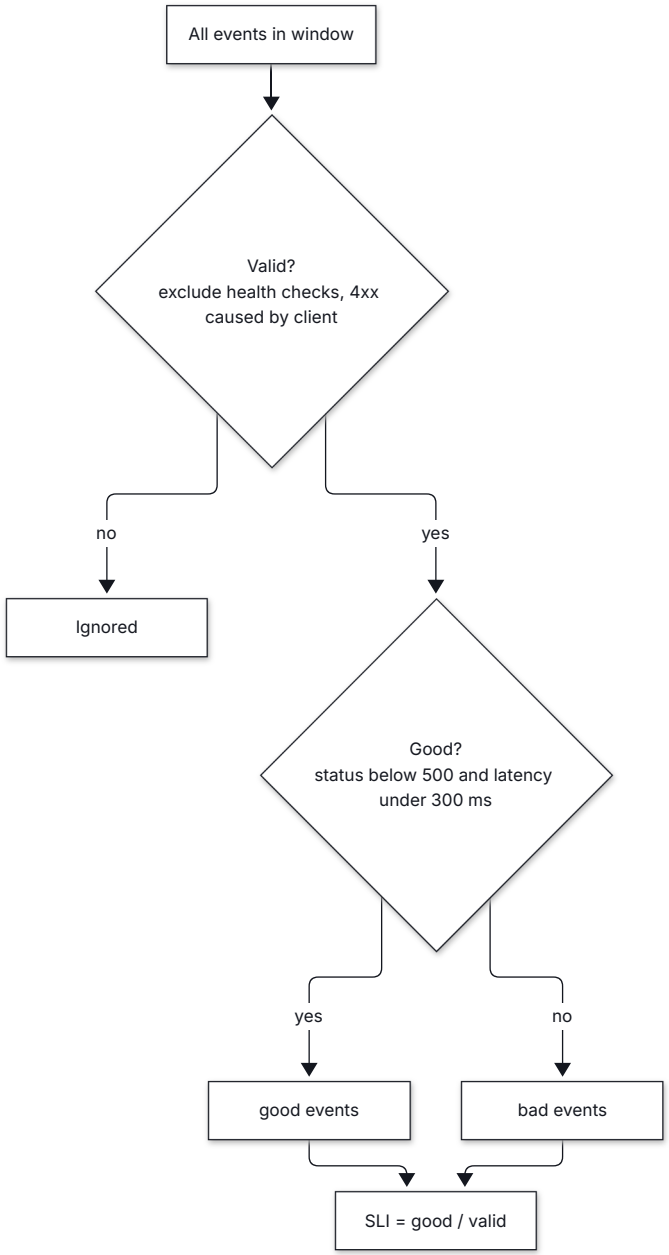


Figure 14.1-2

5 How it works

Choosing good SLIs

1. **List the user journeys**, not the components. For `shop-api`: browse catalogue, add to cart, checkout, view order.
2. **Pick 1-3 SLIs per journey** from the standard menu below. More than that and nobody remembers them.
3. **Write the specification in a sentence**, then the implementation as a query.
4. **Check it against a known bad day.** Replay a past incident through the SLI: would it have dropped? If not, it is the wrong SLI.

Category	Specification (example)	Typical measurement
Availability	Proportion of valid requests served successfully (not `5xx`)	LB or app counters by status code
Latency	Proportion of valid requests served faster than a threshold	histogram buckets, a threshold not an average
Quality	Proportion of responses that are not degraded (for example served from fallback)	app counter with a `degraded` label
Freshness	Proportion of data updated within X minutes	timestamp of last successful pipeline run
Correctness	Proportion of records processed with the right result	reconciliation job output
Durability	Proportion of stored objects that can still be read back	periodic restore or integrity check

Prefer **latency thresholds on a distribution** ("99% under 300 ms") over averages. An average hides the slow tail that users feel.

Request-based vs window-based SLIs

	Request-based	Window-based
Formula	good requests / total requests	good windows / total windows (for example 1-minute windows where p95 latency < 300 ms)
Strength	Weights by traffic: ten failed requests during a busy minute count more than ten during a quiet minute	Works when there is no natural "event" (a batch system, a probe) and is easy to explain
Weakness	A very quiet period carries little weight, so a 3 a.m. outage may barely dent the number	One failed request in an otherwise idle minute can make the whole window "bad"
Use for	HTTP APIs, queues	probes, uptime checks, low-traffic internal services

Measurement points (where to measure)

Point	Sees	Misses
Application metrics	rich detail, easy to instrument	crashes before the app records anything, LB or network drops
Load balancer or ingress	what the user-facing edge returned, including `502` / `503` / `504` when the app is dead	detail of why, client-side problems
Synthetic probe (blackbox)	end to end including DNS and TLS, works at zero traffic	only the paths you probe, not real user mix
Client-side or real user monitoring	what the user really experienced, including ISP and CDN problems	noisy, depends on the client being able to report
Logs	can be reprocessed retroactively	latency of the signal, cost

A common compromise: **load balancer metrics as the primary SLI, application metrics for diagnosis, a synthetic probe as a cross-check**. See also what to monitor §10.11 for the golden signals that these SLIs refine.

6 Step-by-step guide

Prerequisites: Docker and Docker Compose, about 1 GB RAM. We will create a tiny `shop-api` stand-in that exposes the metrics a real Prometheus client would, and compute SLIs from them. Everything is local and free.

1. **Create the sample service** (Python standard library plus `prometheus_client`).

```bash
mkdir -p ~/sre-lab/shop-api && cd ~/sre-lab/shop-api
python3 -m venv .venv && . .venv/bin/activate
pip install prometheus_client==0.20.0
cat > app.py <<'EOF'
import random, time
from http.server import BaseHTTPRequestHandler, HTTPServer
from prometheus_client import Counter, Histogram, start_http_server

REQS = Counter("http_requests_total", "Requests", ["code", "route"])
LAT = Histogram("http_request_duration_seconds", "Latency", ["route"],
                buckets=(0.05, 0.1, 0.3, 0.5, 1, 2))

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        start = time.time()
        time.sleep(random.choice([0.02] * 90 + [0.4] * 8 + [1.2] * 2))   # a slow tail
        code = 500 if random.random() < 0.02 else 200                    # 2 % errors
        self.send_response(code); self.end_headers(); self.wfile.write(b"ok")
        REQS.labels(code=str(code), route="/checkout").inc()
```

```
        LAT.labels(route="/checkout").observe(time.time() - start)
    def log_message(self, *a): pass

start_http_server(9100)           # metrics on :9100/metrics
HTTPServer(("", 8080), H).serve_forever()
EOF
python app.py &
```

Expect no output; the app listens on 8080 (traffic) and 9100 (metrics).

2. **Generate traffic.**

```bash
for i in $(seq 1 500); do curl -s -o /dev/null localhost:8080/checkout; done
curl -s localhost:9100/metrics | grep -E
  '^http_requests_total|^http_request_duration_seconds_(bucket|count)'
```

You should see `http_requests_total{code="200",...}` near 490 and `code="500"` near 10 (random, so similar to, not identical).

3. **Compute the availability SLI by hand** from the counters (this is exactly what PromQL will do):

```bash
curl -s localhost:9100/metrics | awk '
  /^http_requests_total/ { n=$NF; if ($0 ~ /code="5/) bad+=n; total+=n }
  END { printf "availability SLI = %.2f%% (%d good of %d)\n", 100*(total-bad)/total,
  total-bad, total }'
```

Expected: roughly `98%`.

4. **Compute the latency SLI** (fraction of requests under 300 ms) from histogram buckets:

```bash
curl -s localhost:9100/metrics | awk '
  /http_request_duration_seconds_bucket\{.*le="0.3"/ { fast=$NF }
  /http_request_duration_seconds_count/ { all=$NF }
  END { printf "latency SLI (<300ms) = %.2f%%\n", 100*fast/all }'
```

Expected: roughly `90%` (the app sleeps 400 ms or 1.2 s in 10% of requests).

5. **Write the SLIs as PromQL** (you will use these in a real Prometheus in topic 4 and the cap-

stone). The expressions below are the standard shape; `[5m]` is the window of the rate:

```promql
# Availability SLI: good = not 5xx
sum(rate(http_requests_total{route="/checkout", code!~"5.."}[5m]))
/
sum(rate(http_requests_total{route="/checkout"}[5m]))

# Latency SLI: good = served in 300 ms or less
sum(rate(http_request_duration_seconds_bucket{route="/checkout", le="0.3"}[5m]))
/
sum(rate(http_request_duration_seconds_count{route="/checkout"}[5m]))
```

A histogram bucket boundary must exist at your threshold (here `0.3`) or the SLI cannot be computed exactly.

6. **Stop the app.** `kill %1` (and `deactivate`).

7 Real-world scenario

Illustrative, not a documented incident. The `shop-api` team measures availability only at the application: "fraction of requests that returned `200`". For months it reads 99.97%. One Friday evening customers cannot check out, yet the dashboard is green. The cause: a bad Nginx config reload made the proxy answer `502` for 40% of requests when it could not reach upstream pods, and the app never saw those requests. The team replays the incident through their SLI and sees it would not have moved.

Fix: the primary SLI moves to the load balancer's response codes, plus a synthetic checkout probe every minute as a cross-check. The next time the proxy misbehaves the SLI drops within a minute. **Lesson:** an SLI is only as honest as its measurement point. Always ask "what failure modes can this number not see?"

8 Common mistakes and best practices

Don't

- Use resource metrics (CPU, memory, queue depth) as SLIs. They are causes or symptoms of causes, not user outcomes.
- Use averages for latency. Use a threshold on a percentile or histogram bucket instead.
- Count health checks, bots you intentionally block, or client `4xx` errors as "bad" (or as valid at all) without thinking.
- Define twenty SLIs. Nobody will watch them, and none will carry authority.

- Measure only where it is convenient. Check the measurement point against a real past outage.

Do

- Write each SLI as `good / valid` and keep the specification (words) separate from the implementation (query).
- Start with availability and latency for each critical user journey; add others when a real failure demands it.
- Measure at the edge (load balancer or ingress) and cross-check with a probe.
- Document exactly which requests are excluded and why.
- Keep the PromQL or query in version control next to the service.

9 Tasks

- [ ] **T1 — Compute an SLI from raw counters** *(≈30 min, beginner)*
  - **Goal:** see that an SLI is just good divided by valid.
  - **Steps:**
    1. Run the sample app from section 6 and send 500 requests.
    2. Run the `awk` availability command from step 3.
    3. Repeat after sending another 500 requests; note that the counters are cumulative.
  - **Done when:** you have two availability percentages and can explain why they differ slightly.
- [ ] **T2 — Specify SLIs for a user journey** *(≈30 min, beginner)*
  - **Goal:** practise writing a specification before an implementation.
  - **Steps:**
    1. Pick the `shop-api` journey "add to cart".
    2. Write two SLIs (availability and latency) as one-sentence specifications, listing valid and good events.
    3. State what is excluded from "valid" and why.
  - **Done when:** each SLI has a sentence of the form "the proportion of … that … within …".
- [ ] **T3 — Change the threshold** *(≈30 min, intermediate)*
  - **Goal:** feel how the threshold changes the SLI.
  - **Steps:**
    1. Compute the latency SLI for the `0.1`, `0.3`, `0.5` and `1` buckets using the awk pattern in step 4.

2. Tabulate the four results.
3. Decide which threshold matches "feels fast" for a checkout and justify it.

- **Done when:** you have a table with four percentages and a one-line justification.

- [ ] **T4 — Break the measurement point** *(≈45 min, intermediate)*
  - **Goal:** demonstrate an SLI that misses a failure.
  - **Steps:**
    1. Put Nginx in front of the app (`proxy_pass http://127.0.0.1:8080;` in a container or locally) and count requests in the app only.
    2. Stop the app and send 20 requests through Nginx; they return `502`.
    3. Compare the app-level SLI with the Nginx access log (`awk '{print $9}' access.log | sort | uniq -c`).
  - **Done when:** the app-level SLI is unchanged while the proxy log shows 20 `502`s, and you can explain which measurement point to trust.

- [ ] **T5 — Window-based version** *(≈45 min, advanced)*
  - **Goal:** compute a window-based SLI and compare it with the request-based one.
  - **Steps:**
    1. Write a script that curls `/checkout` once a second for 5 minutes, recording status and time.
    2. Group into 1-minute windows; a window is good if at least 95% of its requests were non-`5xx`.
    3. Compute good windows / total windows and the request-based SLI for the same data.
  - **Done when:** you have both numbers and a one-sentence explanation of why they differ.

- [ ] **T6 — Prove it: audit an SLI** *(≈45 min, advanced)*
  - **Goal:** critique an SLI without notes.
  - **Steps:**
    1. Take the SLI "average latency of `shop-api` is below 200 ms at the app server".
    2. In writing, list at least three failure modes it cannot see and a better SLI for each.
    3. Present it to a teammate or record yourself explaining it.
  - **Done when:** you have given three concrete failure modes and replacement specifications.

10 Self-check questions

1. Write the general form of an SLI. What goes in the denominator?
2. Why is "average latency under 200 ms" a poor SLI?
3. When would you choose a window-based SLI over a request-based one?
4. Name two measurement points and one failure each cannot see.
5. Why should CPU utilisation not be used as an SLI?

ANSWERS

1. `good events / valid events`. The denominator is valid events: everything in scope, excluding things like health checks or client-caused errors.
2. Averages hide the slow tail; 5% of users can have a terrible experience while the mean looks fine. Use a threshold on a percentile or histogram bucket.
3. When there is no natural event stream (batch jobs, probes, very low-traffic services), or when you want to express "fraction of time the service was OK".
4. Application metrics miss failures before the app sees the request (proxy `502`s, dropped connections). A synthetic probe misses paths it does not exercise and the real traffic mix.
5. CPU is a resource and a cause, not a user outcome: users can be failing at low CPU and happy at high CPU.

11 Further reading

- *Site Reliability Engineering* (Google, free online) — chapters "Service Level Objectives" and "Monitoring Distributed Systems".
- *The Site Reliability Workbook* (Google, free online) — chapter "Implementing SLOs", including SLI specification vs implementation.
- *Implementing Service Level Objectives* by Alex Hidalgo (O'Reilly).
- Prometheus documentation (prometheus.io/docs) — histograms and `rate()`.
- See also: Observability concepts §0.10 and Prometheus and PromQL §10.2.

LAB NOTES What I ran, what I saw, what it means

14.2 Service Level Objectives (SLOs)

Topic 2 of 10 | ~3 hours | Difficulty: Intermediate

1 What it is

A **Service Level Objective (SLO)** is a target value, or range of values, for an SLI §14.1 over a period of time. Example: "99.9% of valid `shop-api` checkout requests succeed, measured over a rolling 30 days." The SLI is the measurement; the SLO is the promise you make to yourself about that measurement.

The SLO also defines how much unreliability is *acceptable*. A 99.9% objective means 0.1% of events are allowed to be bad; that allowance is the error budget §14.4. Google's SRE books make the point that 100% is the wrong target for almost everything: it is unattainable (the user's own phone and ISP are not 100% reliable), and each extra "nine" costs disproportionately more in engineering effort and slows down feature delivery.

2 Why it matters in DevOps

- **It ends the "reliability vs features" argument.** Product and engineering agree on a number in advance, and the data decides when to slow down or speed up.
- **It sets alert thresholds.** Pages should fire when the SLO is genuinely at risk, not when a graph looks scary (see alert fatigue reduction §14.8).
- **It controls cost.** Going from 99.9% to 99.99% typically means multi-region redundancy, more on-call load and slower releases. An SLO that is too tight wastes money; one that is too loose loses customers.
- **It gives dependencies a contract.** Your service cannot be more available than the things it hard-depends on.

3 Real-world system example

Google popularised the practice: its SRE books describe setting SLOs per user journey, keeping them deliberately slightly looser than what users really need, and using internal SLOs that are tighter than any published agreement. *The Site Reliability Workbook* includes a

worked example of an SLO document for a fictional service, covering the SLIs, targets, rationale, error budget policy and reviewers; the template in this topic follows that idea.

Google Cloud Monitoring lets you define SLOs on a **calendar** period or a **rolling** period, and shows the remaining error budget, which is the same model described below.

Open-source tooling: Prometheus users commonly generate SLO recording rules and alerts with **Sloth** or **Pyrra**, which take a short SLO spec (objective, window, SLI queries) and emit the rules. **Kubernetes** itself publishes SLIs/SLOs for its API server and scheduler in its scalability documentation, as an example of an open project treating objectives as engineering artefacts rather than marketing.

4 Diagram

From SLI to action:

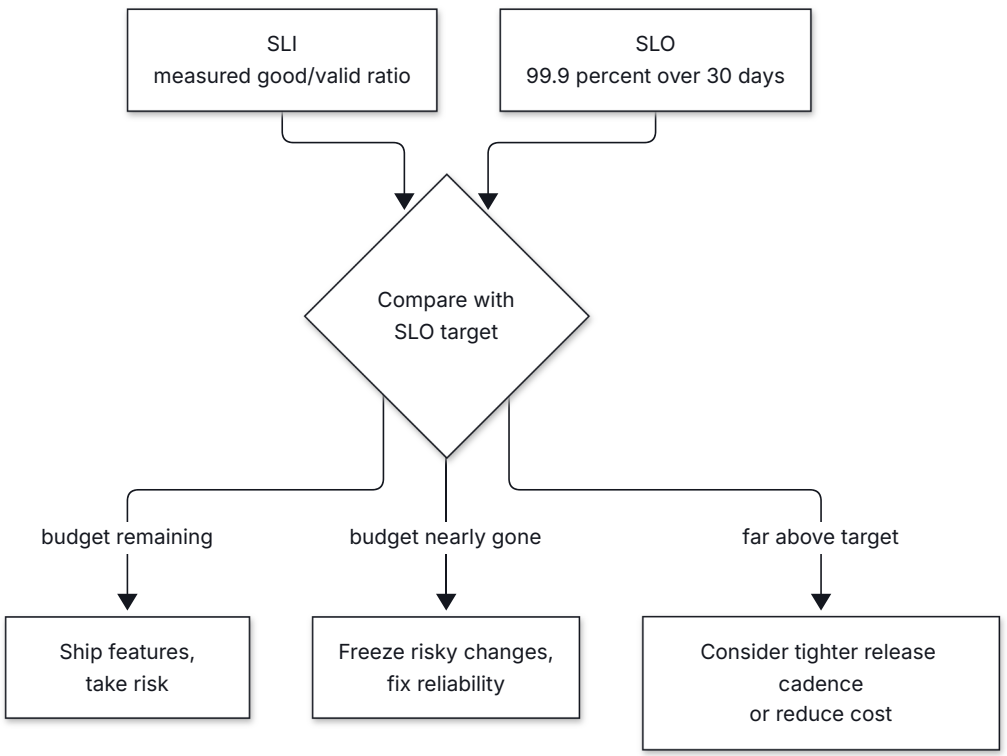


Figure 14.2-1

Rolling window versus calendar window:

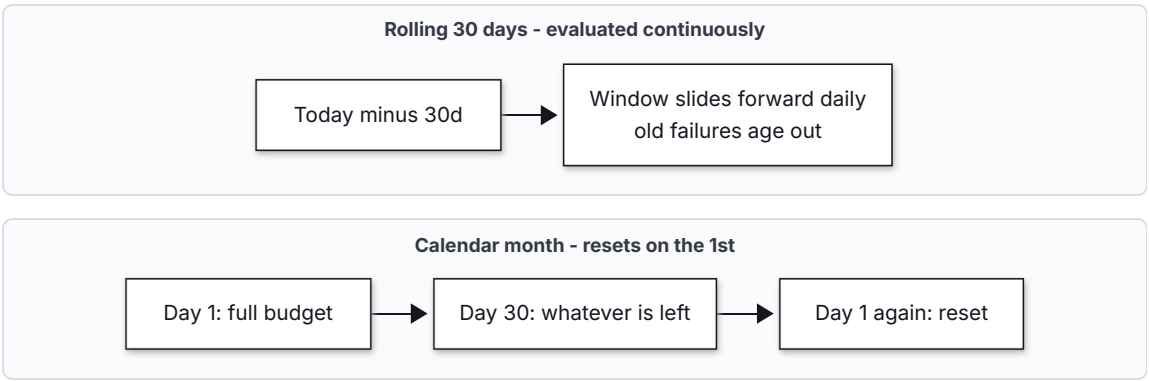


Figure 14.2-2

5 How it works

Setting targets

1. **Start from the user, not from what you currently achieve.** Ask: at what level of failure does the user complain, retry, or leave?
2. **Look at history.** Compute the SLI for the last 4-12 weeks. Set the first SLO a little *below* what you actually achieve, so it is achievable but meaningful. Tighten it later.
3. **Account for dependencies.** If a service calls dependencies serially, availabilities multiply (below).
4. **Keep it simple.** Few SLOs, round numbers, one per user journey and SLI type.
5. **Agree the consequence.** An SLO without a policy for what happens when it is missed is decoration (see error budgets §14.4).

The nines

Allowed bad time for common targets (period lengths exact; real traffic is not uniform, so request-based budgets differ):

SLO	Per 30 days	Per quarter (90 d)	Per year
99%	7.2 h	21.6 h	3.65 d
99.9% ("three nines")	43.2 min	2.16 h	8.76 h
99.95%	21.6 min	1.08 h	4.38 h
99.99% ("four nines")	4.32 min	12.96 min	52.6 min
99.999% ("five nines")	25.9 s	1.3 min	5.26 min

At four nines a single slow human response (waking up, logging in, diagnosing) can consume the whole month's budget, which is why that level normally requires automatic failover rather than heroic on-call.

Dependency math. If `shop-api` needs the database, Redis and a payment provider on the critical path and each is independent:

```
composite = 99.95 % (shop-api) x 99.95 % (PostgreSQL) x 99.9 % (Redis) x 99.9 % (payments)
  = about 99.70 %
```

So promising 99.9% for checkout on those parts is not realistic unless you add redundancy, caching, or make a dependency optional (degrade gracefully). The calculation is crude (failures are not always independent) but it exposes impossible promises quickly.

Windows: rolling vs calendar

	Rolling (for example last 30 days)	Calendar (month or quarter)
Behaviour	Old failures age out gradually	Budget resets on a fixed date
Pro	Matches user memory; no cliff; best for alerting	Easy to report and align with business reviews
Con	Hard to explain "when does it reset"	A bad day 1 can mean 29 days of freeze; a bad day 30 is "free"
Use for	operational decisions and burn-rate alerts	reporting and contracts (see SLAs §14.3)

Many teams alert on a rolling window and report on the calendar. The Google material discusses 28 or 30 day windows; four weeks has the nice property of always containing the same number of weekends.

SLO documents

An SLO that lives only in someone's head will be forgotten. Keep a short document in the service repository:

```
MARKDOWN
# SLO: shop-api checkout            Owner: team-shop    Reviewed: 2026-10-01 (quarterly)

## Service and user journey
Customers placing an order through the web and mobile apps.

## SLIs (specification → implementation)
| Name | Specification | Implementation |
|------|---------------|----------------|
| Availability | Valid checkout requests that do not return 5xx |
  sli:http_requests:ratio_good (recording rule, measured at ALB) |
```

```
| Latency | Valid checkout requests answered in 300 ms or less | histogram bucket le="0.3"
  / count |

## Objectives (rolling 28 days)
- Availability: 99.9 %      - Latency: 99 % of requests under 300 ms

## Rationale
History shows 99.95 %. Checkout dependencies (PostgreSQL, payments) compose to roughly
  99.7-99.9 %.

## Out of scope / excluded
Health checks, requests rejected with 4xx, planned maintenance announced 7 days ahead.

## Error budget policy
See error-budget-policy.md: freeze non-critical releases when budget is exhausted.

## Review
Reviewed every quarter by team-shop and product; changed only with a written reason.
```

6 Step-by-step guide

Prerequisites: Python 3 only. We will compute budgets and the nines table, then check a candidate SLO against measured history.

1. **Generate the nines table.**

```bash
mkdir -p ~/sre-lab && cd ~/sre-lab
cat > nines.py <<'EOF'
#!/usr/bin/env python3
"""Print allowed downtime for common SLO targets."""
WINDOWS = {"day": 1, "week": 7, "30 days": 30, "quarter (90d)": 90}
print(f"{'SLO':>8} | " + " | ".join(f"{w:>14}" for w in WINDOWS))
for slo in (99.0, 99.5, 99.9, 99.95, 99.99, 99.999):
    cells = []
    for days in WINDOWS.values():
        secs = days * 86400 * (1 - slo / 100)
        if secs ⩾ 3600:
            cells.append(f"{secs / 3600:.2f} h")
        elif secs ⩾ 60:
            cells.append(f"{secs / 60:.2f} min")
        else:
            cells.append(f"{secs:.1f} s")
    print(f"{slo:>7}% | " + " | ".join(f"{c:>14}" for c in cells))
EOF
python3 nines.py
```

Expect a table whose `99.9%` row shows `43.20 min` in the `30 days` column.

2. **Check the dependency math.**

```bash
python3 - <<'EOF'
deps = {"shop-api": 99.95, "postgres": 99.95, "redis": 99.9, "payments": 99.9}
total = 1
for v in deps.values():
    total *= v / 100
print(f"serial composite availability: {total*100:.3f}%")
EOF
```

   Expected output: `serial composite availability: 99.700%`.

3. **Test a candidate SLO against history.** Save a CSV of daily good/total counts (the sample below is made up) and see how often a 99.9% objective would have been missed.

```bash
cat > history.csv <<'EOF'
day,good,total
1,998800,1000000
2,999900,1000000
3,999950,1000000
4,997000,1000000
5,999990,1000000
6,999700,1000000
7,999980,1000000
EOF
awk -F, 'NR>1 { g+=$2; t+=$3; printf "day %s: %.4f%%\n", $1, 100*$2/$3 }
         END { printf "7-day SLI: %.4f%%  budget used: %.1f%%\n", 100*g/t, 100*(1-
  g/t)/0.001 }' history.csv
```

   Expected: `7-day SLI: 99.9331% budget used: 66.9%`. Over the week the 99.9% objective holds, even though day 1 and day 4 were individually below 99.9%: a window absorbs bad days as long as the total budget lasts. Change day 4 to `990000` and rerun to see the week fail.

4. **Write your SLO document.** Copy the template from section 5 into `slo-shop-checkout.md` and fill in your own numbers.

5. **Choose a window and say why.** Add one line to the document: "Operational window: rolling 28 days. Reporting: calendar quarter."

7 Real-world scenario

Illustrative, not a documented incident. The `shop-api` team promises "99.99%" in a sales deck without measuring. Their SLI history shows 99.93%. Three weeks into a quarter a single 25-minute database failover consumes the quarter's whole four-nines budget (12.96 minutes)

twice over, and the team is in permanent "SLO breached" mode. Nobody believes the dashboard any more, and feature work continues anyway, so the SLO has no effect on decisions.

Fix: they reset the objective to 99.9% on a rolling 28 days, document the dependency math (the database was on the critical path at 99.95%), and record the target for a later move to 99.95% once a read replica removes the single point of failure. The SLO now fires rarely and is trusted. **Lesson:** an SLO should be achievable with effort, tied to history and dependencies, and honest enough that the organisation will act on it.

8 Common mistakes and best practices

Don't

- Target 100%, or pick a number because it sounds impressive.
- Set the SLO equal to your current best performance with no margin; the first bad day breaches it.
- Create an SLO with no consequence for missing it.
- Change the SLO right after a miss to make the graph green.
- Forget that you cannot be more reliable than your hard dependencies.

Do

- Derive the first SLO from 4-12 weeks of real data, then tighten gradually.
- Keep one SLO per user journey and SLI type, in a reviewed document next to the code.
- Use a rolling window for operations and a calendar window for reporting.
- Review SLOs on a schedule (quarterly is common) and record why any change was made.
- Make SLOs for internal services stricter than any external SLA §14.3 you have signed.

9 Tasks

- [ ] **T1 — Build the nines table** *(≈20 min, beginner)*
  - **Goal:** know what each nine buys in minutes.
  - **Steps:**
    1. Run `nines.py` from section 6.
    2. Add `99.8` and `99.97` to the list of SLOs and rerun.
    3. Write down the 30-day allowance for each.
  - **Done when:** you can state the 30-day downtime allowance for 99%, 99.9% and 99.99% from memory.

- [ ] **T2 — Evaluate a candidate SLO against history** *(≈30 min, beginner)*
  - **Goal:** use data, not guesses, to choose a target.
  - **Steps:**
    1. Run the awk command from step 3 on the sample file.
    2. Rerun with the objective changed to 99.95% (replace `0.001` with `0.0005` in the awk), then change day 4 to `990000` and rerun with 99.9%.
    3. Report the 7-day SLI and budget used for each variant.
  - **Done when:** you have three SLI/budget results and can say which objective each satisfies (budget used above 100% means breached).
- [ ] **T3 — Dependency composition** *(≈30 min, intermediate)*
  - **Goal:** see why a promise can be impossible.
  - **Steps:**
    1. List the real dependencies on the `shop-api` checkout path (database, cache, payment provider, DNS, load balancer).
    2. Assign a plausible availability to each and compute the serial composite.
    3. Redo the calculation assuming the cache becomes optional (graceful degradation).
  - **Done when:** you have two composite numbers and can say what 99.9% requires of each dependency.
- [ ] **T4 — Write a real SLO document** *(≈45 min, intermediate)*
  - **Goal:** produce the artefact a team would review.
  - **Steps:**
    1. Copy the template from section 5.
    2. Fill in two SLIs and objectives for `shop-api`, with rationale and exclusions.
    3. Ask a peer to review it for ambiguity ("which requests are valid?").
  - **Done when:** the document has no TBD fields and a reviewer's comments are resolved.
- [ ] **T5 — Rolling vs calendar simulation** *(≈45 min, advanced)*
  - **Goal:** show the cliff effect of calendar windows.
  - **Steps:**
    1. Write a script that generates 60 days of daily error rates with one bad day on day 2 and another on day 29.
    2. Compute the remaining budget on day 30 for a calendar month and for a rolling 28-day window.
    3. Compute the same on day 31 and day 32.

- **Done when:** you can show that the calendar budget resets abruptly while the rolling budget recovers gradually, with numbers.

☐ **T6 — Prove it: defend a target** *(≈45 min, advanced)*

- **Goal:** justify an SLO to a product manager.
- **Steps:**
  1. Without the guide, choose an availability SLO for an internal admin tool and one for the customer-facing checkout.
  2. Explain in writing why they differ, referencing cost, dependencies and user impact.
  3. Say what you would do if the checkout budget were exhausted in week two.
- **Done when:** a teammate can restate your reasoning and your two numbers are different on purpose.

10 Self-check questions

1. What is the difference between an SLI and an SLO?
2. How many minutes of downtime does 99.9% allow over 30 days?
3. Why is 100% rarely the right SLO?
4. Why does a rolling window usually suit alerting better than a calendar window?
5. A service with three serial dependencies at 99.9% each: roughly what is its best-case availability?

ANSWERS

1. The SLI is the measured ratio; the SLO is the target for that ratio over a time window.
2. 43.2 minutes (0.1% of 43,200 minutes).
3. It is unattainable end to end, enormously expensive, and leaves no room for change, which causes most incidents. The budget is what makes change affordable.
4. The budget changes continuously and failures age out gradually, so there is no reset cliff where a bad day 30 is "free" and a bad day 1 punishes the whole month.
5. About 0.999 x 0.999 x 0.999 = 99.7%, before counting your own service's failures.

11 Further reading

- *Site Reliability Engineering* (Google) — "Service Level Objectives" and "Embracing Risk".

- *The Site Reliability Workbook* (Google) — "Implementing SLOs", which includes the SLO document example.
- *Implementing Service Level Objectives* by Alex Hidalgo (O'Reilly).
- Sloth and Pyrra project documentation for generating Prometheus SLO rules.
- See also: Site Reliability Engineering overview §0.11 and Availability architecture §19.2.

14.3 Service Level Agreements (SLAs)

Topic 3 of 10 | ~2 hours | Difficulty: Intermediate

1 What it is

A **Service Level Agreement (SLA)** is a *contract* between a service provider and a customer that states a level of service and what happens if it is not delivered, usually money back (**service credits**) or the right to terminate. It is built from the same ingredients as an SLO §14.2: an SLI §14.1, a target and a measurement period, but with a crucial addition: **consequences**. *Site Reliability Engineering* (Google) draws the line this way: if missing the target has no explicit consequence, you are looking at an SLO, not an SLA.

Three layers are worth keeping apart:

Layer	Audience	Missing it means	Typical value for the same service
SLI	engineers	nothing, it is a measurement	measured 99.96%
SLO (internal)	engineering and product	slow down releases, fix reliability	99.95%
SLA (external)	customers and lawyers	refunds, penalties, churn	99.9%

The SLA is deliberately the loosest of the three so that you notice and fix problems internally long before you owe a customer anything.

2 Why it matters in DevOps

- **Money and trust.** An SLA breach is a billing event and often a reputational one. Engineers who do not know their SLA's definitions can accidentally promise something the architecture cannot deliver.
- **Engineering decides what is measurable.** The legal text says "availability"; somebody must decide that means "ratio of non-5xx responses at the load balancer, per calendar month, excluding announced maintenance". That decision belongs in an engineering document that legal then reflects.

- **Dependencies cascade.** Your customer SLA cannot exceed what your own providers promise you (and often not even that). Knowing the cloud provider's SLA for the services you build on is a design input (see availability §19.2).
- **Evidence.** When a customer disputes uptime, you need metrics, status-page history and incident records to settle it.

3 Real-world system example

Cloud provider SLAs. AWS, Google Cloud and Microsoft Azure all publish a per-service SLA. The common pattern is a monthly (or sometimes quarterly) uptime percentage, a few tiers of **service credit** that grow as measured uptime falls (credits are usually a percentage of that month's bill for the affected service), and a requirement that the **customer files a claim** with evidence within a time limit. Credits are typically the sole remedy. The exact numbers and exclusions differ per service and change over time, so always read the current SLA text for the service you rely on. Notice also that many SLAs are tied to a specific architecture (for example, running instances in more than one Availability Zone), which means the *customer's own design* can void the promise.

SaaS and status pages. Many SaaS vendors publish a status page with historical uptime and an enterprise SLA for paying plans. The status page is the customer-visible evidence; see incident management §14.5.

Google. Its SRE books describe running with internal SLOs that are tighter than any SLA offered to Cloud customers, so that engineering reacts before a contractual line is crossed.

4 Diagram

How the three layers relate and where the money changes hands:

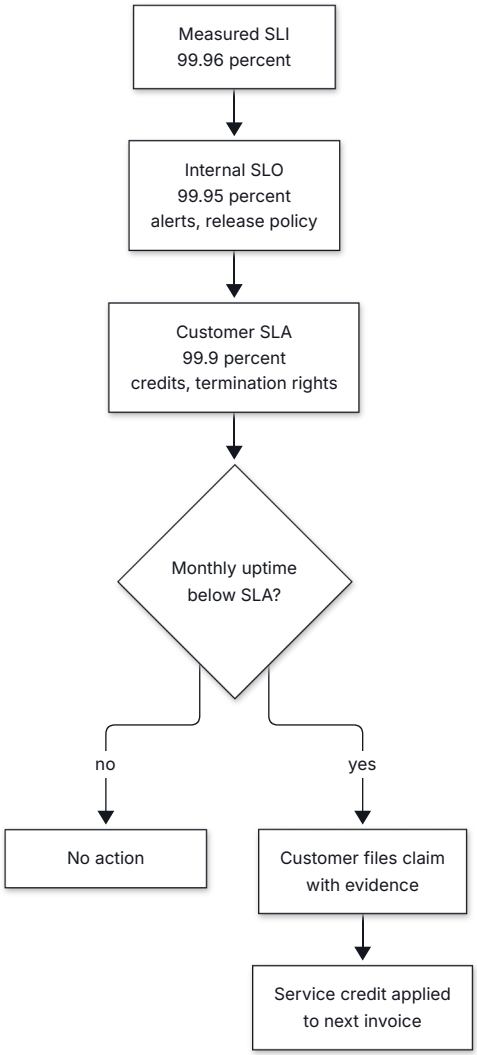


Figure 14.3-1

Dependency chain: your SLA must fit inside the SLAs you buy.

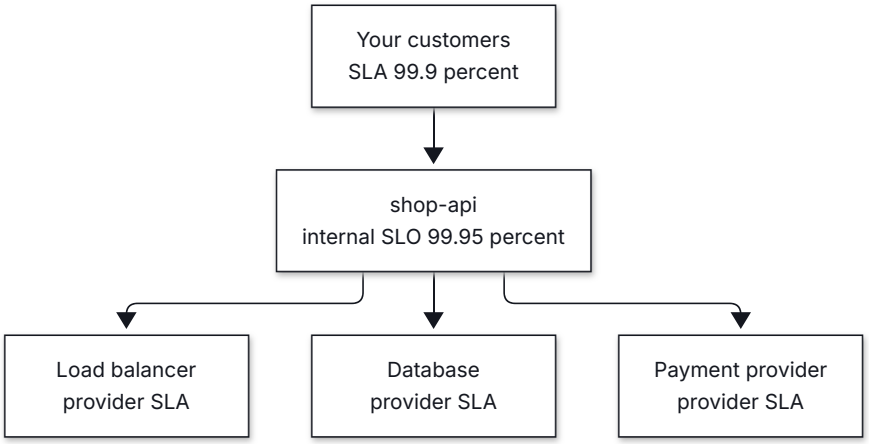


Figure 14.3-2

5 How it works

The parts of an SLA

Part	What to write	Example
Service scope	exactly what is covered	"Public HTTPS API at `api.example.com`"
Metric definition	the SLI, its measurement point and period	"Percentage of 1-minute intervals in the calendar month where the API error rate is below 5%"
Target	the number	99.9% monthly uptime
Exclusions	what does not count	announced maintenance, customer-caused issues, force majeure, third-party outages outside your control
Remedy	credits, tiers, caps	10% of monthly fee if uptime is 99.0-99.9%, higher below
Claims process	how and when the customer asks	within 30 days of the incident with request IDs
Reporting	where evidence lives	status page and monthly report

Contractual vs internal

	Contractual (SLA)	Internal (SLO)
Owner	legal, sales and engineering	engineering and product
Flexibility	changed by renegotiation	changed by a review meeting

	Contractual (SLA)	Internal (SLO)
Strictness	the loosest	tighter than the SLA
Missing it	credits, refunds, termination	feature freeze, reliability work
Window	usually calendar month for billing	usually rolling 28-30 days for operations

Choosing a margin

Rule of thumb used in practice: the internal SLO sits **one step tighter** than the SLA (for example SLA 99.9%, SLO 99.95%). The gap is the safety margin that turns "we missed our target" into an internal alert rather than a refund. A related rule: never sign an SLA for a number that you have not met in the measured history of the last several months.

6 Step-by-step guide

Prerequisites: Python 3. This guide builds a small SLA calculator for a hypothetical tiered credit table. The tiers are illustrative and are not any provider's real terms.

1. **Create the calculator.**

```bash
mkdir -p ~/sre-lab && cd ~/sre-lab
cat > sla_credit.py <<'EOF'
#!/usr/bin/env python3
"""Compute monthly uptime and the service credit owed under an example SLA.

usage: python3 sla_credit.py DOWNTIME_MINUTES [EXCLUDED_MINUTES] [MONTHLY_FEE]
Example tiers (illustrative, NOT any real provider's terms):
  uptime ⩾ 99.9 %          → 0 % credit
  99.0 % ⩽ uptime < 99.9 % → 10 % credit
  95.0 % ⩽ uptime < 99.0 % → 25 % credit
  uptime < 95.0 %          → 100 % credit
"""
import sys

if len(sys.argv) < 2:
    sys.exit(__doc__)
down = float(sys.argv[1])
excluded = float(sys.argv[2]) if len(sys.argv) > 2 else 0.0   # maintenance, force
  majeure
fee = float(sys.argv[3]) if len(sys.argv) > 3 else 2000.0
month_min = 30 * 24 * 60

counted = max(down - excluded, 0)
uptime = 100 * (month_min - counted) / month_min
tiers = [(99.9, 0), (99.0, 10), (95.0, 25), (0.0, 100)]
credit_pct = next(c for floor, c in tiers if uptime ⩾ floor)
```

```
print(f"month minutes          : {month_min}")
print(f"downtime reported      : {down:.1f} min, excluded {excluded:.1f} min, counted
  {counted:.1f} min")
print(f"monthly uptime         : {uptime:.4f} %")
print(f"credit tier            : {credit_pct} % of {fee:.2f} = {fee * credit_pct /
  100:.2f}")
EOF
```

2. **Run three scenarios.**

```bash
python3 sla_credit.py 90 30     # 90 min down, 30 of them in announced maintenance
python3 sla_credit.py 20        # 20 min down
python3 sla_credit.py 500       # a bad month
```

   Expected (the same on any machine):

   - first: `monthly uptime : 99.8611 %` and a `10 %` credit (`200.00`)
   - second: `99.9537 %` and `0 %`
   - third: `98.8426 %` and `25 %` (`500.00`)

3. **Express the SLA metric as a PromQL expression** so the claim can be answered with data. This is the "good 1-minute intervals" form; it requires the `sli:` recording rules built in Error Budgets §14.4:

```promql
# fraction of the last 30 days in which the 5-minute error ratio stayed below 5
  percent
avg_over_time((sli:http_requests:error_ratio_rate5m{job="shop-api"} < bool 0.05)
  [30d:1m])
```

   The `< bool` form returns 1 or 0 per evaluation and `avg_over_time` over a subquery turns it into a fraction of good minutes.

4. **Record the margin.** Add to your SLO document: "SLA 99.9% monthly; internal SLO 99.95% rolling 28 days; margin 0.05 points."

5. **Make a claims runbook.** Write a short checklist: who receives a claim, which dashboards and status-page entries prove or disprove it, who approves a credit, and how long you have to answer.

7 Real-world scenario

Illustrative, not a documented incident. A B2B customer of `shop-api` emails: "You were down for two hours on the 14th; we want a credit." The on-call lead opens the incident record: the outage lasted 38 minutes, of which 20 were an announced maintenance window (excluded by the contract). The SLO dashboard shows monthly uptime of 99.958%, which is above the 99.9% SLA, so no credit is owed. The customer's own logs, though, show 2 hours of failed checkouts: their integration retried against a deprecated endpoint that was not in the SLA scope.

The team replies politely with the evidence (incident timeline, status-page posts, SLI graph), and then opens a ticket to warn customers about the deprecated endpoint. **Lesson:** precise definitions of scope and measurement make disputes short and factual; the postmortem still lists "customer integrations on a deprecated path" as a contributing factor, because the customer experience was real even if the contract was not breached.

8 Common mistakes and best practices

Don't

- Copy a competitor's SLA number without measuring your own history.
- Set the SLA equal to the SLO; you would learn of trouble when the refund is already due.
- Promise availability that exceeds the compounded SLAs of your critical providers.
- Leave terms like "downtime" or "available" undefined. Define the measurement point and the interval.
- Hide the exclusions in fine print and then act surprised when a customer disagrees.

Do

- Keep the SLA looser than the internal SLO by a clear margin, and document both.
- Have engineering write the metric definition and legal wrap it in contract language.
- Publish status and history so customers can see the same evidence you do.
- Practise the claims process before the first claim arrives.
- Review SLAs when architecture changes (single-region vs multi-region, new dependency).

9 Tasks

- [ ] **T1 — Run the credit calculator** *(≈20 min, beginner)*
  - **Goal:** turn downtime into uptime and credit.

- **Steps:**
  1. Create `sla_credit.py` from section 6.
  2. Run the three scenarios.
  3. Add a fourth: `python3 sla_credit.py 130 0 5000`.
- **Done when:** you can explain each printed line and the credit for the fourth case (about `99.6991 %` uptime, a `10 %` credit of `500.00`).

☐ **T2 — Read a real SLA** *(≈45 min, beginner)*

- **Goal:** extract the structure of a provider SLA.
- **Steps:**
  1. Open the current SLA for one managed service you use (for example the AWS SLA page for your load balancer or database, from the provider's legal or SLA documentation).
  2. Fill in the seven rows of the table in section 5: scope, metric, target, exclusions, remedy, claims process, reporting.
  3. Note one architecture requirement that must hold for the SLA to apply.
- **Done when:** the table is complete with quotes or paraphrases from the document and no blanks.

☐ **T3 — Choose SLO and SLA together** *(≈30 min, intermediate)*

- **Goal:** place the margin sensibly.
- **Steps:**
  1. Use your SLI history (or the sample data from SLOs §14.2) for `shop-api`.
  2. Propose an internal SLO and an SLA that is looser by at least one step.
  3. Justify the choice by showing how many months in the history would have breached each.
- **Done when:** you show two numbers and a table of "months breached" for each.

☐ **T4 — Evidence for a claim** *(≈45 min, intermediate)*

- **Goal:** answer a customer claim with data.
- **Steps:**
  1. Using the sample app from SLIs §14.1, stop it for 3 minutes while a script requests it every 10 seconds.
  2. Compute the failed fraction of 1-minute intervals from the script's log.
  3. Write the reply to a hypothetical claim with the interval list as evidence.

- **Done when:** the reply states the number of bad minutes, the resulting uptime and whether a credit is owed under the section 6 tiers.

☐ **T5 — Tighten and loosen** *(≈30 min, advanced)*

- **Goal:** feel how exclusions change outcomes.
- **Steps:**
  1. Run `python3 sla_credit.py 70` and then `python3 sla_credit.py 70 40`.
  2. Explain why the second case owes nothing.
  3. Write one paragraph on the risk of generous exclusions for customer trust.
- **Done when:** you have both outputs and the paragraph states a trade-off.

☐ **T6 — Prove it: SLA vs SLO** *(≈30 min, advanced)*

- **Goal:** explain it without notes.
- **Steps:**
  1. Write a half-page note for a sales colleague on why we will not offer 99.99% on `shop-api`.
  2. Use dependency math and the margin rule.
  3. Include what it would take to offer it (architecture, cost, on-call).
- **Done when:** the note contains a number-based argument that a non-engineer can follow.

10 Self-check questions

1. What single feature separates an SLA from an SLO?
2. Why should the internal SLO be tighter than the SLA?
3. Name three common exclusions in an SLA.
4. Why can your SLA not exceed that of your critical dependencies?
5. What evidence do you want ready before a customer sends a claim?

ANSWERS

1. Explicit consequences (credits, penalties, termination rights) for missing it.
2. So engineering reacts, and fixes problems, before the contractual threshold is crossed.
3. Scheduled and announced maintenance, customer-caused issues, force majeure, and failures of third parties outside your control.
4. Serial dependencies multiply: if a required component is down, your service is down regardless of your own quality.

5. The SLI history for the period, incident records and timelines, status-page posts, and the precise metric definition from the contract.

11 Further reading

- *Site Reliability Engineering* (Google) — the section distinguishing SLOs and SLAs in "Service Level Objectives".
- *The Site Reliability Workbook* (Google) — "Implementing SLOs" for how SLOs relate to agreements.
- The published SLA pages of the cloud services you depend on (AWS, Google Cloud, Azure documentation roots); read the current text, as terms change.
- See also: Error Budgets §14.4 and Disaster Recovery §0.9.

LEVEL 14: SRE

14.4 Error Budgets

Topic 4 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

An **error budget** is the amount of unreliability an SLO §14.2 allows. If the objective is 99.9% of requests succeeding over 30 days, then 0.1% of requests may fail, and that 0.1% is a *budget you are allowed to spend*: on bugs, bad deploys, infrastructure failures, risky experiments and planned maintenance alike.

```
error budget = 1 - SLO target                       (99.9 %  →  0.1 % = 0.001)
budget spent = bad events / valid events in the window
budget left  = 1 - (budget spent / error budget)
```

The idea comes from Google's SRE practice (see *Site Reliability Engineering*, chapter "Embracing Risk", and *The Site Reliability Workbook*, chapters "Implementing SLOs" and "Alerting on SLOs"). It turns reliability from a vague virtue into a resource that product and engineering can plan with: while there is budget, ship faster; when it runs out, slow down and invest in reliability. A **burn rate** is how fast you are spending it: 1x means you would use exactly the whole budget by the end of the window; 10x means the budget is gone in a tenth of the window.

2 Why it matters in DevOps

- **Objective release decisions.** "Can we deploy on Friday?" becomes "how much budget is left, and what is the burn rate right now?".
- **Better alerts.** Alerting on budget burn pages you for failures that matter and stays quiet for blips that do not (see alert fatigue reduction §14.8).
- **Shared incentives.** Developers want velocity, operators want stability. A shared budget makes both sides optimise the same number.
- **Permission to take risk.** Teams that have never failed are probably too slow. Unspent budget is a sign you could be shipping more or testing in production more.

3 Real-world system example

Google is the origin of the policy approach: its SRE books describe an *error budget policy* in which, if the budget is exhausted, releases other than critical fixes pause until reliability is restored, and where postmortems of large budget consumers drive engineering priorities. *The Site Reliability Workbook* includes an example error budget policy and a full chapter, "Alerting on SLOs", that compares several alerting strategies and ends with the **multi-window, multi-burn-rate** alerts you will build below.

Prometheus ecosystem. Open-source generators such as **Sloth** and **Pyrra** emit exactly this family of recording rules and alerts from a short SLO definition, and Grafana's SLO features visualise remaining budget and burn rate. The rule shape in this topic is the same one those tools generate, which is why it is worth understanding by hand once.

4 Diagram

Budget as a running balance over a 30-day window:

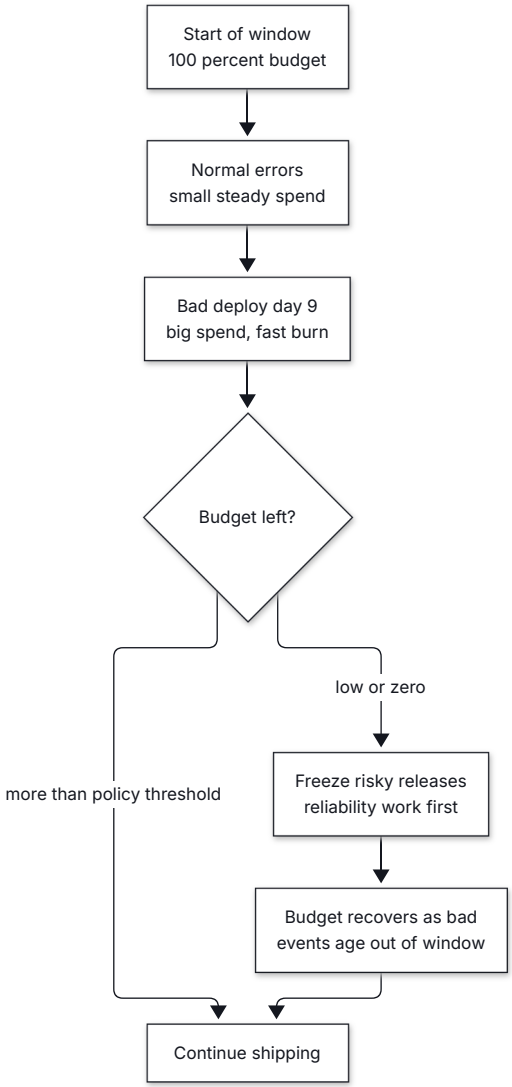


Figure 14.4-1

How the multi-window alert decides to page:

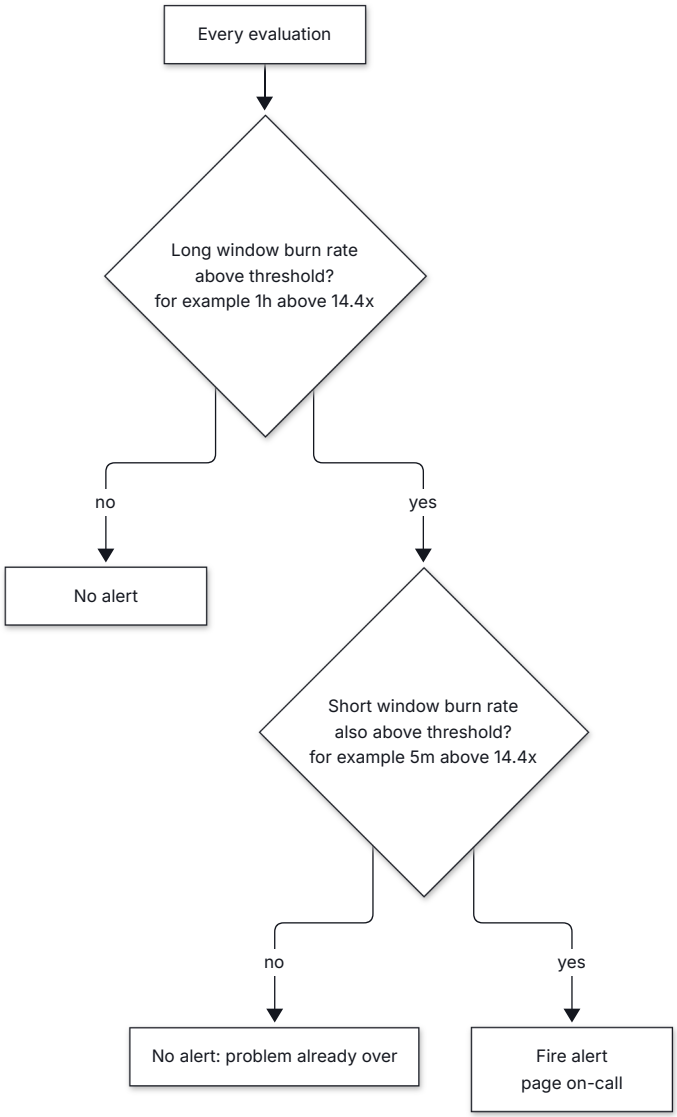


Figure 14.4-2

5 How it works

Budget math

Pick the SLO and the window, then everything else follows:

SLO	Budget	30-day time budget	Per 1,000,000 requests
99%	1%	7.2 h	10,000 may fail
99.9%	0.1%	43.2 min	1,000 may fail
99.99%	0.01%	4.32 min	100 may fail

For request-based SLOs the budget is counted in *requests*, not minutes: a total outage at 3 a.m. with ten requests per second burns far less than one at noon with a thousand. Relationship between burn rate, window and consumption:

```
burn rate     = observed error ratio / error budget ratio
budget used   = burn rate x (alert window / SLO window)
time to empty = SLO window / burn rate
```

Example, 99.9% SLO over 30 days (720 h), observed error ratio 1.44% for 1 hour: burn = 0.0144 / 0.001 = 14.4x; used = 14.4 x 1/720 = 2% of the budget; at that rate the budget lasts 720 / 14.4 = 50 hours.

Multi-window, multi-burn-rate alerts

A naive alert ("error rate above 0.1% for 5 minutes") either pages constantly on blips or reacts too late to slow burns. The Workbook chapter "Alerting on SLOs" develops the better pattern: **alert on burn rate over a long window, and require a short window to agree**.

Severity	Long window	Short window	Burn rate	Budget consumed when it fires	Action
Page	1 h	5 m	14.4x	2%	wake someone, act now
Page	6 h	30 m	6x	5%	wake someone, act soon
Ticket	3 d	6 h	1x	10%	triage next working day

- The **long window** gives *precision*: it filters blips.
- The **short window** (about 1/12 of the long one) gives a fast *reset*: once the problem is fixed, the alert clears in minutes instead of staying red for an hour.
- Fast burns page quickly; slow burns that would still empty the budget get a ticket.

Release policy (the error budget policy)

Write down, in advance and with product sign-off, what happens at each budget level. For example:

Budget remaining	Policy
above 50%	normal pace; experiments and risky changes allowed
25-50%	extra review for risky changes; consider canary-only rollouts (see canary deployments §5.12)
below 25%	no risky changes; reliability items are first in the sprint
exhausted	freeze all non-critical releases until the budget is back above a floor or the cause is fixed; post-mortem required

Include an escape hatch (for example, security fixes and fixes that *improve* reliability can still ship) and a named person who can override with a written reason. A policy without consequences will be ignored.

6 Step-by-step guide

Prerequisites: Python 3 for the calculators; either a local `promtool` (shipped in the Prometheus release tarball) or Docker for validating rules. All free and local.

1. **Compute budgets for your SLO.**

```bash
mkdir -p ~/sre-lab && cd ~/sre-lab
cat > budget.py <<'EOF'
#!/usr/bin/env python3
"""Error budget calculator: python3 budget.py 99.9 [window_days]"""
import sys

slo = float(sys.argv[1]) if len(sys.argv) > 1 else 99.9
days = float(sys.argv[2]) if len(sys.argv) > 2 else 30
allowed = 1 - slo / 100
minutes = days * 24 * 60
print(f"SLO {slo}% over {days:g} days")
print(f"  error budget      : {allowed * 100:.4f}% of requests/time")
print(f"  allowed downtime  : {allowed * minutes:.2f} minutes ({allowed * minutes *
  60:.0f} s)")
print(f"  1M requests →     : {allowed * 1_000_000:,.0f} may fail")
EOF
python3 budget.py 99.9 30
```

Expected:

```
SLO 99.9% over 30 days
  error budget      : 0.1000% of requests/time
  allowed downtime  : 43.20 minutes (2592 s)
  1M requests →     : 1,000 may fail
```

2. **Compute burn rate, consumption and alert thresholds.**

```bash
cat > burn.py <<'EOF'
#!/usr/bin/env python3
"""Burn-rate calculator.

usage: python3 burn.py SLO_PERCENT ERROR_RATIO WINDOW_HOURS [PERIOD_DAYS]
  e.g. python3 burn.py 99.9 0.0144 1
"""
import sys

if len(sys.argv) < 4:
    sys.exit(__doc__)
slo = float(sys.argv[1])
err = float(sys.argv[2])            # observed error ratio, 0.0144 = 1.44 %
win_h = float(sys.argv[3])
period_h = (float(sys.argv[4]) if len(sys.argv) > 4 else 30) * 24

budget = 1 - slo / 100              # allowed error ratio
burn = err / budget                 # 1.0 = exactly on budget
consumed = burn * win_h / period_h  # fraction of whole-period budget burned
exhaust_h = period_h / burn if burn else float("inf")

print(f"error budget ratio     : {budget:.6f}")
print(f"burn rate              : {burn:.2f}x")
print(f"budget consumed in {win_h:g}h : {consumed * 100:.2f}% of the period budget")
print(f"time to exhaustion     : {exhaust_h:.1f} h ({exhaust_h / 24:.1f} days) at this
  rate")

print("\nThresholds for this SLO (error ratio that trips each alert):")
print(f"{'severity':8} {'long':>5} {'short':>6} {'burn':>6} {'budget spent':>13}
  {'error ratio >':>14}")
for sev, long_h, short_h, rate in [("page", 1, "5m", 14.4), ("page", 6, "30m", 6.0),
  ("ticket", 72, "6h", 1.0)]:
    spent = rate * long_h / period_h * 100
    print(f"{sev:8} {long_h:>4}h {short_h:>6} {rate:>6} {spent:>12.1f}% {rate *
  budget:>14.6f}")
EOF
python3 burn.py 99.9 0.0144 1
```

Expected:

```
error budget ratio     : 0.001000
burn rate              : 14.40x
budget consumed in 1h : 2.00% of the period budget
time to exhaustion     : 50.0 h (2.1 days) at this rate

Thresholds for this SLO (error ratio that trips each alert):
severity  long  short   burn  budget spent  error ratio >
page        1h     5m   14.4          2.0%       0.014400
page        6h    30m    6.0          5.0%       0.006000
ticket     72h     6h    1.0         10.0%       0.001000
```

Try `python3 burn.py 99.9 0.002 6` (a 2x burn). It does not trip any alert yet, but would empty the budget in 15 days.

3. **Create the Prometheus rules file** `shop-api-slo-rules.yml`: recording rules for the error ratio at each window, then the three alerts.

YAML

```yaml
# SLO: 99.9 % of valid shop-api requests succeed, rolling 30 days.
# Error budget ratio = 1 - 0.999 = 0.001
groups:
  - name: shop-api-sli-recording
    interval: 30s
    rules:
      - record: sli:http_requests:error_ratio_rate5m
        expr: |
          sum by (job) (rate(http_requests_total{job="shop-api", code=~"5.."}[5m]))
          /
          sum by (job) (rate(http_requests_total{job="shop-api"}[5m]))
      - record: sli:http_requests:error_ratio_rate30m
        expr: |
          sum by (job) (rate(http_requests_total{job="shop-api", code=~"5.."}[30m]))
          /
          sum by (job) (rate(http_requests_total{job="shop-api"}[30m]))
      - record: sli:http_requests:error_ratio_rate1h
        expr: |
          sum by (job) (rate(http_requests_total{job="shop-api", code=~"5.."}[1h]))
          /
          sum by (job) (rate(http_requests_total{job="shop-api"}[1h]))
      - record: sli:http_requests:error_ratio_rate6h
        expr: |
          sum by (job) (rate(http_requests_total{job="shop-api", code=~"5.."}[6h]))
          /
          sum by (job) (rate(http_requests_total{job="shop-api"}[6h]))
      - record: sli:http_requests:error_ratio_rate3d
        expr: |
          sum by (job) (rate(http_requests_total{job="shop-api", code=~"5.."}[3d]))
          /
```

```
        sum by (job) (rate(http_requests_total{job="shop-api"}[3d]))
    - record: sli:http_requests:error_ratio_rate30d
      expr: |
        sum by (job) (rate(http_requests_total{job="shop-api", code=~"5.."}[30d]))
        /
        sum by (job) (rate(http_requests_total{job="shop-api"}[30d]))
    # fraction of the 30-day error budget still unspent (negative = overspent)
    - record: slo:error_budget_remaining:ratio
      expr: 1 - (sli:http_requests:error_ratio_rate30d / 0.001)

- name: shop-api-burn-rate-alerts
  rules:
    # 14.4x burn: 2 % of a 30-day budget gone in 1 hour. Page.
    - alert: ShopApiErrorBudgetBurnFast
      expr: |
        sli:http_requests:error_ratio_rate1h{job="shop-api"} > (14.4 * 0.001)
        and
        sli:http_requests:error_ratio_rate5m{job="shop-api"} > (14.4 * 0.001)
      for: 2m
      labels:
        severity: page
        slo: shop-api-availability
      annotations:
        summary: "shop-api is burning its error budget 14.4x too fast"
        description: "1h error ratio {{ $value | humanizePercentage }}. At this rate
the 30-day budget is gone in about 2 days."
        runbook_url: "https://runbooks.example.com/shop-api/error-budget-burn"

    # 6x burn: 5 % of the budget gone in 6 hours. Page.
    - alert: ShopApiErrorBudgetBurnSlow
      expr: |
        sli:http_requests:error_ratio_rate6h{job="shop-api"} > (6 * 0.001)
        and
        sli:http_requests:error_ratio_rate30m{job="shop-api"} > (6 * 0.001)
      for: 5m
      labels:
        severity: page
        slo: shop-api-availability
      annotations:
        summary: "shop-api is burning its error budget 6x too fast"
        description: "6h error ratio {{ $value | humanizePercentage }}. The 30-day
budget would be gone in about 5 days."
        runbook_url: "https://runbooks.example.com/shop-api/error-budget-burn"

    # 1x burn: 10 % of the budget gone in 3 days. Ticket, not a page.
    - alert: ShopApiErrorBudgetBurnTicket
      expr: |
        sli:http_requests:error_ratio_rate3d{job="shop-api"} > (1 * 0.001)
        and
        sli:http_requests:error_ratio_rate6h{job="shop-api"} > (1 * 0.001)
      labels:
        severity: ticket
        slo: shop-api-availability
```

```
        annotations:
          summary: "shop-api is spending its error budget at or above the sustainable
  rate"
          description: "3d error ratio {{ $value | humanizePercentage }}. Triage in
  working hours."
          runbook_url: "https://runbooks.example.com/shop-api/error-budget-burn"
```

The `{{ $value }}` in `description` is the value of the left-hand side of the `and`, i.e. the long-window ratio. The 30-day rule only sees as much history as Prometheus keeps; the default retention is 15 days, so start Prometheus with `--storage.tsdb.retention.time=35d` (or accept a shorter effective window while the lab is young).

4. **Validate the syntax.**

```bash
promtool check rules shop-api-slo-rules.yml
# no promtool installed? use the one inside the Prometheus image:
docker run --rm -v "$PWD":/w --entrypoint promtool prom/prometheus:v2.53.0 check rules
  /w/shop-api-slo-rules.yml
```

Expected: `SUCCESS: 10 rules found`.

5. **Unit-test the alerts** with synthetic data. `promtool test rules` feeds fake time series and asserts which alerts fire. Save as `shop-api-slo-rules.test.yml`:

```yaml
rule_files:
  - shop-api-slo-rules.yml
evaluation_interval: 30s

tests:
  # 20 req/s overall, 5 % errors = 50x burn: both fast windows exceed 14.4x
  - interval: 30s
    input_series:
      - series: 'http_requests_total{job="shop-api", code="200"}'
        values: '0+570x400'
      - series: 'http_requests_total{job="shop-api", code="500"}'
        values: '0+30x400'
    alert_rule_test:
      - eval_time: 3h
        alertname: ShopApiErrorBudgetBurnFast
        exp_alerts:
          - exp_labels:
              severity: page
              slo: shop-api-availability
              job: shop-api
            exp_annotations:
              summary: "shop-api is burning its error budget 14.4x too fast"
```

```
            description: "1h error ratio 5%. At this rate the 30-day budget is gone
in about 2 days."
            runbook_url: "https://runbooks.example.com/shop-api/error-budget-burn"

  # healthy: 0.05 % errors = 0.5x burn, nothing fires
  - interval: 30s
    input_series:
      - series: 'http_requests_total{job="shop-api", code="200"}'
        values: '0+599.7x400'
      - series: 'http_requests_total{job="shop-api", code="500"}'
        values: '0+0.3x400'
    alert_rule_test:
      - eval_time: 3h
        alertname: ShopApiErrorBudgetBurnFast
        exp_alerts: []
      - eval_time: 3h
        alertname: ShopApiErrorBudgetBurnSlow
        exp_alerts: []
```

BASH

```
promtool test rules shop-api-slo-rules.test.yml
```

Expected: `SUCCESS`. Now break it on purpose: change `0+30x400` to `0+0.3x400`, rerun, and read the failure message.

6. **Check the thresholds are consistent with the calculator.** `14.4 * 0.001 = 0.0144`, the first row of the `burn.py` table. If you change the SLO to 99.95%, replace the `0.001` literals with `0.0005`. A cleaner production approach is a recording rule such as `slo:shop_api_availability:error_budget` holding the number once.
7. **Write the error budget policy** (half a page) using the table in section 5 and have the product owner initial it.

7 Real-world scenario

Illustrative, not a documented incident. On a Tuesday afternoon `shop-api` ships a change that adds an unindexed query to the checkout path. Error ratio climbs to 10% as the database saturates and requests time out. With the old alert ("5xx above 1% for 10 minutes") the on-call would have been paged late and again whenever traffic dipped. With the new alerts, `ShopApiErrorBudgetBurnFast` fires roughly 11 minutes in: the 1-hour average passes 1.44% after about 9 minutes of 10% errors, the 5-minute ratio is already above it, and the 2-minute `for` clause is satisfied. The responder sees the deploy marker, rolls back (see rollback §5.7), and within about five minutes the 5-minute ratio falls under the threshold so the page auto-resolves, even though the 1-hour ratio is still elevated.

The incident consumed about 4% of the month's budget (roughly 17 minutes at 10% errors). With the month's earlier spending, about 70% remains, which the policy table treats as normal pace, so no freeze, but the postmortem action "add a query-plan check to CI" is accepted as top priority. **Lesson:** the short window made the alert clear promptly, the long window kept it from firing on noise, and the budget put a number on how bad it was.

8 Common mistakes and best practices

Don't

- Page on every error spike. Page on budget burn; let the ticket alert handle slow bleeds.
- Use only a long window (alert stays on for an hour after the fix) or only a short one (pages on every blip).
- Define an error budget policy and then let the first exhausted budget pass with no consequence.
- Compute burn rates on a metric with no traffic and divide by zero. Add a minimum-traffic guard or rely on a probe for quiet services.
- Treat the budget as a target to hit exactly. Unspent budget is fine; it means room to move faster.

Do

- Keep the recording rules and thresholds in version control, unit-test them with `promtool test rules`, and review them with each SLO change.
- Derive thresholds from the SLO, not from gut feel: `burn rate x (1 - SLO)`.
- Include a runbook link in every page-severity alert.
- Report remaining budget in a weekly review and in release discussions.
- Spend budget deliberately (chaos experiments, risky migrations) when you have plenty.

9 Tasks

- [ ] **T1 — Budget math by hand** *(≈20 min, beginner)*
  - **Goal:** be fluent in budget and burn calculations.
  - **Steps:**
    1. Run `budget.py` for 99%, 99.9%, 99.95% and 99.99% over 30 days.
    2. For 99.9% compute by hand the burn rate when the error ratio is 0.5%, then check with `burn.py 99.9 0.005 1`.
    3. State how long the budget lasts at that rate.

- **Done when:** your hand calculation (5x burn, budget gone in 6 days) matches the script.

- [ ] **T2 — Install and validate the rules** *(≈30 min, beginner)*
  - **Goal:** have a syntactically valid multi-burn-rate rule file.
  - **Steps:**
    1. Create `shop-api-slo-rules.yml` from section 6.
    2. Run `promtool check rules` (local or Docker).
    3. Count the recording rules and alerts it reports.
  - **Done when:** the output says `SUCCESS: 10 rules found`.

- [ ] **T3 — Change the SLO** *(≈30 min, intermediate)*
  - **Goal:** modify thresholds correctly.
  - **Steps:**
    1. Change the objective to 99.5% (budget 0.005) in the rules file, including the `slo:error_budget_remaining:ratio` rule.
    2. Update the unit test so that a 10% error input still fires (the fast threshold is now 14.4 x 0.005 = 7.2%) and a 0.2% input does not.
    3. Run `promtool test rules`.
  - **Done when:** `promtool test rules` reports `SUCCESS` for the new SLO.

- [ ] **T4 — Break the alert and fix it** *(≈45 min, intermediate)*
  - **Goal:** understand why both windows are needed.
  - **Steps:**
    1. Remove the short-window condition from `ShopApiErrorBudgetBurnFast`.
    2. Write a test where errors stop at minute 70 and check at minute 100: the broken alert still fires.
    3. Restore the short window and show the alert no longer fires at minute 100.
  - **Done when:** you have a failing test for the broken rule and a passing test for the fixed one, and you can explain the difference in one sentence.

- [ ] **T5 — Live demo with Prometheus** *(≈60 min, advanced)*
  - **Goal:** watch real data trip the alert.
  - **Steps:**
    1. Run the sample app from SLIs §14.1 with the error rate raised from 2% to 20% and scrape it with a local Prometheus (`docker run -p 9090:9090 -v ...prometheus.yml prom/prometheus`) loading your rules file.

2. Use a short evaluation interval and temporarily scale the windows down (for example 1h to 10m, 5m to 1m) so you do not wait an hour.
3. Open the Prometheus Alerts page and watch `ShopApiErrorBudgetBurnFast` go from pending to firing, then stop the errors and watch it resolve.

- **Done when:** you have seen the alert fire and resolve, and can name which window cleared first.

☐ **T6 — Prove it: write the policy** *(≈45 min, advanced)*

- **Goal:** produce a policy a product owner could sign.
- **Steps:**
  1. Without the guide, write an error budget policy for `shop-api` with at least four budget levels and an exceptions clause.
  2. Pick the three alert thresholds for an SLO of 99.95% and show the arithmetic.
  3. Explain to a teammate why the 6-hour alert uses a 30-minute short window.
- **Done when:** the thresholds you computed are 0.0072, 0.003 and 0.0005 (14.4x, 6x and 1x of 0.0005) and the policy has a named override owner.

10 Self-check questions

1. What is the error budget of a 99.95% SLO in minutes per 30 days?
2. What does a burn rate of 6 mean, and how long does the budget last at that rate?
3. Why do multi-window alerts require both a long and a short window?
4. Which of the three standard alerts should create a ticket rather than a page, and why?
5. What should happen to releases when the budget is exhausted?

ANSWERS

1. 0.05% of 43,200 minutes = 21.6 minutes.
2. You are spending budget six times faster than the sustainable rate; the 30-day budget is gone in 5 days.
3. The long window avoids paging on short blips (precision); the short window makes the alert reset quickly once the problem is fixed (fast recovery).
4. The 1x burn over 3 days: the budget is being used at the sustainable rate or slightly above, so it is urgent to look at but not at 3 a.m.
5. Per the policy: freeze non-critical, risky releases and prioritise reliability work until the budget recovers or the cause is fixed, with explicit exceptions for security and reliability fixes.

11 Further reading

- *The Site Reliability Workbook* (Google) — "Alerting on SLOs" (the multi-window, multi-burn-rate approach) and the example error budget policy appendix.
- *Site Reliability Engineering* (Google) — "Embracing Risk" and "Service Level Objectives".
- Prometheus documentation (prometheus.io/docs) — recording rules, alerting rules and `promtool test rules`.
- Sloth and Pyrra documentation for generated SLO rules.
- See also: Service Level Objectives §14.2, Alert Fatigue Reduction §14.8 and Prometheus and PromQL §10.2.

LAB NOTES What I ran, what I saw, what it means

LEVEL 14: SRE

14.5 Incident Management

Topic 5 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

Incident management is the structured way an organisation responds when a service is broken or about to break: detect it, decide how bad it is, assign clear roles, restore service as fast as possible, tell the people affected, and capture what happened so you can learn. An **incident** is any unplanned event that degrades the service enough that a human must respond now.

Two ideas dominate the practice. First, **mitigate first, understand later**: the goal during an incident is to stop the user pain (roll back, fail over, shed load, disable a feature), not to find the root cause. Second, **roles over heroics**: even a three-person team does better when one person coordinates, one does the hands-on work and one communicates. Google's SRE books ("Managing Incidents" in *Site Reliability Engineering* and "Incident Response" in *The Site Reliability Workbook*) borrow this from the emergency-services **Incident Command System (ICS)**.

2 Why it matters in DevOps

- **Minutes matter, confusion costs minutes.** Without roles, five engineers debug the same thing, nobody updates the customers and nobody notices the fix made things worse.
- **Burn rate pays for it.** Each minute of impact spends error budget §14.4; good process shortens time to mitigate.
- **Trust.** Customers forgive outages more readily than silence. A calm, regular status update is part of reliability.
- **People.** A chaotic 3 a.m. incident is how engineers burn out. A rehearsed process lowers stress (see on-call practices §14.6).
- **Learning.** Every incident is an unplanned experiment; the record feeds post-mortems §14.7.

3 Real-world system example

Google describes an incident command structure with three core roles, usually called the **Incident Commander (IC)**, the **Operations Lead** (the only people changing the system) and the **Communications Lead**, with a **Planning Lead** added for large incidents. The IC holds the high-level picture and delegates; they do not type commands.

PagerDuty publishes its internal incident response documentation openly (the "PagerDuty Incident Response" site). It describes the roles of Incident Commander, Deputy, Scribe, Communications Liaison and Subject Matter Experts, and the principle of a clear handoff and an explicit "who is in charge now".

Atlassian publishes an incident management handbook with severity levels, roles and communication templates.

Status pages and public postmortems. Companies such as GitHub, Cloudflare and AWS post incident updates on public status pages and, for major events, publish detailed postmortems. This transparency is itself an operational practice: customers can see the timeline, and the engineers who write the report are the ones who fix the root causes.

4 Diagram

Incident lifecycle as a state machine:

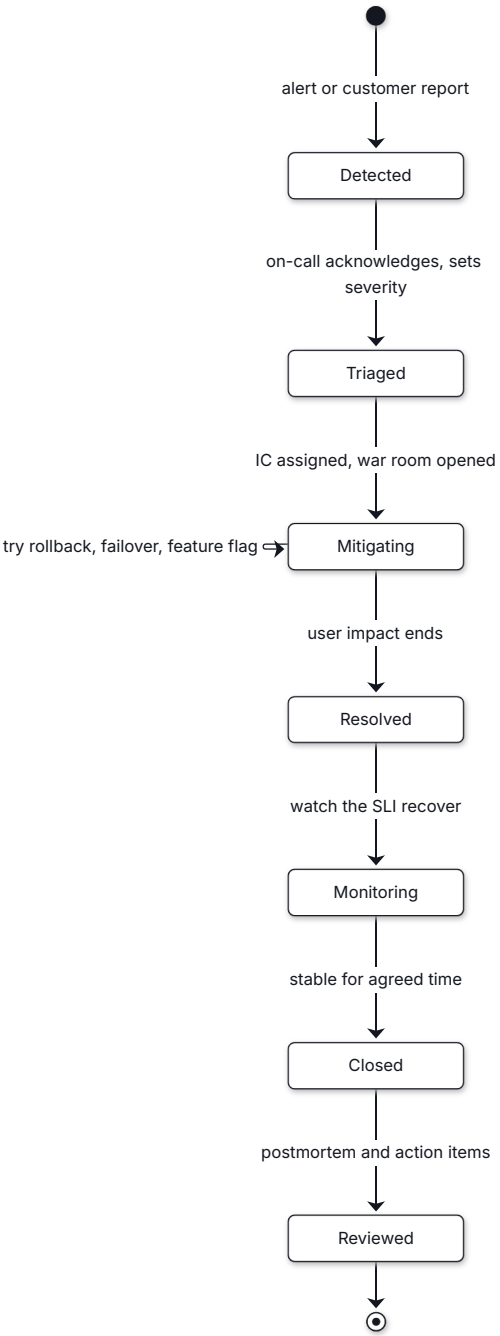

alert or customer report
Detected
on-call acknowledges, sets severity
Triaged
IC assigned, war room opened
try rollback, failover, feature flag
Mitigating
user impact ends
Resolved
watch the SLI recover
Monitoring
stable for agreed time
Closed
postmortem and action items
Reviewed

Figure 14.5-1

Who talks to whom during a large incident:

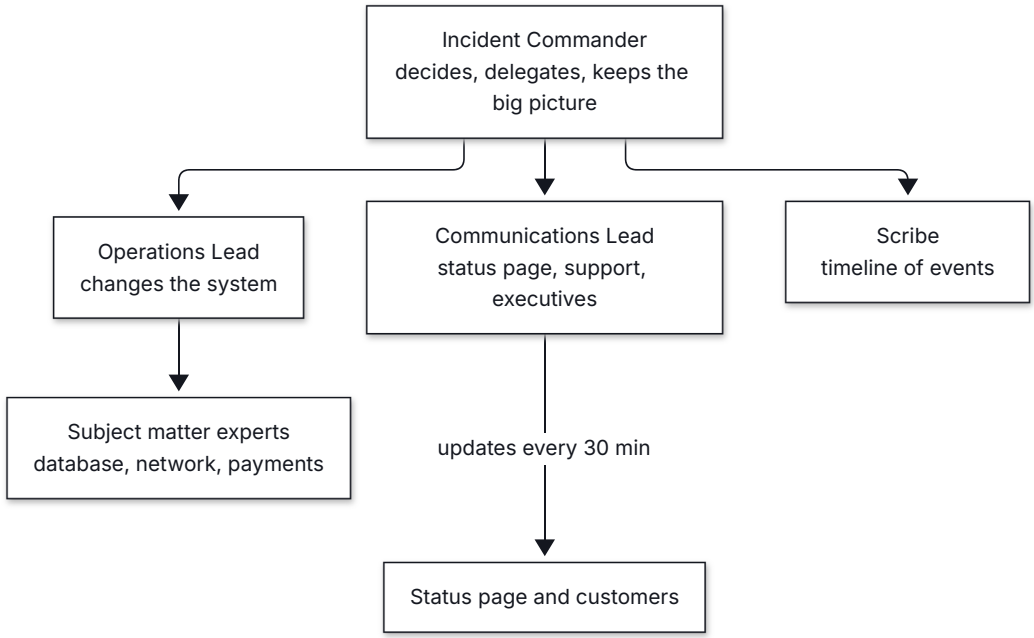


Figure 14.5-2

5 How it works

Roles (the incident-roles card)

Role	Responsibilities	Does NOT
Incident Commander (IC)	Declares the incident, sets severity, assigns roles, decides priorities, calls for help, decides when it is over, hands off clearly	type commands into production
Operations Lead (Ops)	Investigates and applies mitigations; the single point that changes the system; reports findings to the IC in short sentences	decide severity or talk to customers
Communications Lead (Comms)	Posts status-page and internal updates on a fixed cadence, handles support and executive questions, shields responders from interruptions	debug
Scribe	Keeps the timestamped timeline in the incident channel or document	take decisions
Subject matter expert (SME)	Answers specific questions for Ops	self-assign work without telling the IC

In a small team one person may hold two roles, but **the IC and the hands-on operator must be different people** as soon as the incident is more than trivial.

Severity levels

Define severity by *impact*, not by cause. Example scale (adapt the numbers to your SLOs):

Severity	Typical impact	Response	Update cadence
SEV1	Major outage: checkout or login down for most users, data loss or security breach	page immediately, IC + Comms, war room, leadership notified	every 15-30 min
SEV2	Significant degradation: a major feature broken or a large subset of users affected, fast budget burn	page, IC, war room	every 30-60 min
SEV3	Minor: limited users, workaround exists	work in hours, ticket, no war room	on resolution
SEV4	Cosmetic or no user impact	normal backlog	none

Rules that save arguments: **when in doubt, declare the higher severity** (downgrading is free), and anyone may declare an incident. Tie the top severities to your SLO burn-rate alerts from error budgets §14.4: the fast-burn page is usually SEV2 or higher.

War rooms

A **war room** is a single place where the response happens: a dedicated chat channel (`#inc-2026-10-09-checkout`) and, for severe cases, a voice bridge. Rules: one room per incident, the IC announces their name and the severity at the top, the scribe pins the timeline, side conversations move to threads, and observers stay quiet (or read only). Nobody makes a change in production without saying so in the channel first.

Status pages and communication

Audience	Channel	Content
Customers	status page	what is affected, since when, what we are doing, next update time
Support	internal channel	wording to use with customers, known workarounds
Executives	short summary	impact, ETA if known, customer or revenue exposure
Responders	war room	facts and decisions

Good status updates have a fixed shape: **Investigating -> Identified -> Monitoring -> Resolved**, each with a timestamp and the time of the next update. Say what users experience, not internal jargon, and never guess a cause publicly.

Mitigation toolbox

Rollback, feature flag off, fail over to a replica or another region, scale up, shed load or rate-limit, block a bad client, restore from backup (last resort). Prepare these as **tested runbooks** so the on-call is choosing from a menu rather than inventing.

6 Step-by-step guide

Prerequisites: a terminal and, optionally, a chat tool or a shared document. We will build the scribe tool, a declaration message and a status template, then use them in the capstone mock incident.

1. **Create a tiny incident scribe** that timestamps everything in UTC (the timeline is the raw material of the postmortem).

```bash
mkdir -p ~/sre-lab && cd ~/sre-lab
cat > inc.sh <<'EOF'
#!/usr/bin/env bash
# Tiny incident scribe: appends UTC-timestamped lines to ./incident-<id>.log
# usage: ./inc.sh start "<title>" | note "<text>" | who "<role>=<name>" | show
set -euo pipefail
ID="${INCIDENT_ID:-$(date -u +%Y%m%d)-01}"
LOG="incident-${ID}.log"
ts() { date -u +%Y-%m-%dT%H:%M:%SZ; }
case "${1:-}" in
  start) echo "$(ts) START  ${2:?title required}" >> "$LOG" ;;
  note)  echo "$(ts) NOTE   ${2:?text required}" >> "$LOG" ;;
  who)   echo "$(ts) ROLE   ${2:?role=name required}" >> "$LOG" ;;
  show)  cat "$LOG" ;;
  *)     echo "usage: $0 start|note|who|show ..." >&2; exit 2 ;;
esac
EOF
chmod +x inc.sh
```

2. **Run a three-line practice incident.**

```bash
export INCIDENT_ID=practice-01
./inc.sh start "Checkout returns 502 for 30% of users (SEV2)"
./inc.sh who "IC=alex"
./inc.sh note "Deploy 4821 correlates with start of errors; Ops proposes rollback"
./inc.sh show
```

Expected output (timestamps will differ):

```
2026-10-09T10:15:02Z START  Checkout returns 502 for 30% of users (SEV2)
2026-10-09T10:15:02Z ROLE   IC=alex
2026-10-09T10:15:03Z NOTE   Deploy 4821 correlates with start of errors; Ops proposes
  rollback
```

3. **Write the declaration message** you paste in the war-room channel in the first minute:

```
INCIDENT DECLARED  SEV2  Checkout errors (shop-api)
IC: @alex   Ops: @priya   Comms: @sam   Scribe: @jo
Impact: ~30% of checkout requests return 502 since 10:07 UTC
Next update: 10:45 UTC     Channel: #inc-20261009-checkout
Rule: say your intent before you change anything in production.
```

4. **Prepare the status-page templates** (save them somewhere your Comms Lead can paste from):

```
Investigating - We are investigating errors when placing orders. Some customers may
  see failures at checkout. Next update by <TIME UTC>.
Identified     - The cause has been identified and we are applying a fix. Next update
  by <TIME UTC>.
Monitoring     - A fix has been applied. We are watching to confirm that checkout has
  recovered. Next update by <TIME UTC>.
Resolved       - Checkout has been working normally since <TIME UTC>. A summary will
  follow. We apologise for the disruption.
```

5. **Keep the incident-roles card** from the capstone (`99-capstone-project.md`) printed or pinned where responders can find it, and agree who may declare which severity.
6. **Hand off cleanly.** At a shift change, the outgoing IC says: "I am handing IC to Priya at 11:00 UTC. State: impact is X, mitigation Y is in progress, next decision Z." The incoming IC repeats it back; the scribe logs it with `./inc.sh note`.

7 Real-world scenario

Illustrative, not a documented incident. At 02:12 UTC the fast-burn alert pages Priya. She sees 502s from the load balancer and, because the burn rate is high, declares SEV2 and opens `#inc-20261009-checkout`. Five more engineers join within minutes and start four parallel investigations, one of which restarts the API pods without telling anyone; errors briefly worsen. Alex, the engineering manager, joins, announces "I am IC", and asks Priya to be Ops and everyone else to stay quiet unless asked. Jo takes the scribe role, Sam posts the first status

update. Within ten minutes the timeline shows a deploy at 02:05, and Ops rolls it back; the 5-minute error ratio recovers in five minutes. Sam moves the status page to "Monitoring", then "Resolved" after 30 stable minutes.

Lesson: the technical fix was a rollback; the time saved came from one voice coordinating, announced changes, and a written timeline. The uncoordinated restart cost several minutes and became a contributing factor in the postmortem.

8 Common mistakes and best practices

Don't

- Let the most senior engineer be both IC and the person typing commands.
- Chase the root cause before the user pain is mitigated.
- Make production changes silently or in parallel without telling the room.
- Go quiet on customers. No update for an hour looks like no progress.
- Guess publicly about the cause, or assign blame in the channel.

Do

- Rehearse: run game days and mock incidents so roles are familiar before the real thing.
- Define severity by impact and link it to SLO burn; declare early, downgrade later.
- Use a dedicated channel, a scribe, UTC timestamps and a fixed update cadence.
- Prefer reversible mitigations (rollback, flag off, fail over) and keep the runbook for them current.
- Close every SEV1-SEV2 incident with a blameless postmortem §14.7 and tracked action items.

9 Tasks

- [ ] **T1 — Run the scribe** *(≈20 min, beginner)*
  - **Goal:** produce a timestamped incident log.
  - **Steps:**
    1. Create `inc.sh` and run the practice incident from section 6.
    2. Add two more `note` lines for a mitigation attempt and the result.
    3. Run `./inc.sh show`.
  - **Done when:** the log shows five or more timestamped lines in UTC order.
- [ ] **T2 — Classify five scenarios** *(≈30 min, beginner)*

- **Goal:** practise severity assignment.
- **Steps:**
  1. For each: (a) checkout fully down, (b) admin report page slow, (c) 2% of logins fail, (d) customer data exposed in a log, (e) one pod crash-looping with no user impact, assign SEV1-SEV4.
  2. Justify each in one sentence by impact.
  3. Compare with a teammate and resolve differences.
- **Done when:** you have five assignments with impact-based reasons and noted any disagreement.

☐ **T3 — Write your own severity matrix** *(≈45 min, intermediate)*

- **Goal:** adapt the scale to `shop-api`.
- **Steps:**
  1. Using your SLOs from SLOs §14.2, define what burn rate or error ratio maps to SEV1, SEV2 and SEV3.
  2. Add response expectations: who is paged, whether a war room opens, update cadence.
  3. Put it in the repo as `docs/severity.md`.
- **Done when:** the matrix references measurable thresholds, not adjectives.

☐ **T4 — Tabletop exercise** *(≈60 min, intermediate)*

- **Goal:** rehearse roles without breaking anything.
- **Steps:**
  1. With one or more teammates, read this scenario aloud: "Checkout is returning 502 for 30% of users since 10:07 UTC. A deploy happened at 10:05."
  2. Assign IC, Ops, Comms and Scribe, and talk through each person's first five actions.
  3. Write the first status-page post and the declaration message.
- **Done when:** the declaration, a status post and a timeline of at least six entries exist, and the IC did not suggest typing a single command.

☐ **T5 — Break and recover** *(≈60 min, advanced)*

- **Goal:** practise a real mitigation.
- **Steps:**
  1. Run the `shop-api` sample (or any container) and deploy a "bad version" that returns 500 half the time.

2. Detect it with a simple loop (`watch -n2 'curl -s -o /dev/null -w "%{http_code}\n" localhost:8080'`), declare an incident with the scribe tool, and roll back to the good version.
3. Log detection, decision and recovery times.

- **Done when:** the log shows time to detect, time to mitigate and the rollback command used.

- [ ] **T6 — Prove it: explain the roles** *(≈30 min, advanced)*
  - **Goal:** teach it without notes.
  - **Steps:**
    1. Explain to a teammate, from memory, why the IC must not be the operator.
    2. Draw the roles diagram and the status-page state sequence on paper.
    3. Say what you do when you are the only person on call.
  - **Done when:** your teammate can name the four roles and the first message to post, and your solo answer covers declaring, logging and communicating.

10 Self-check questions

1. What is the single most important goal during an incident?
2. Why should the Incident Commander not be the person executing changes?
3. When unsure between SEV2 and SEV3, which do you pick and why?
4. What are the four typical status-page states?
5. What is the purpose of a scribe?

ANSWERS

1. To stop user impact (mitigate), not to find the root cause.
2. The IC needs the whole picture and must coordinate and decide; typing commands narrows their focus and removes the independent check on what Ops is doing.
3. SEV2: downgrading later is free, while under-reacting costs minutes and trust.
4. Investigating, Identified, Monitoring, Resolved.
5. To keep an accurate timestamped record of events and decisions, which feeds the post-mortem timeline and allows clean handoffs.

11 Further reading

- *Site Reliability Engineering* (Google) — "Managing Incidents" and "Emergency Response".

- *The Site Reliability Workbook* (Google) — "Incident Response".
- PagerDuty Incident Response documentation (response.pagerduty.com) — open roles, handoff and communication guides.
- Atlassian Incident Management Handbook.
- See also: Postmortems §14.7, On-call Practices §14.6 and Rollback §5.7.

14.6 On-call Practices

Topic 6 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

On-call means that, for a defined period, a named engineer is reachable and ready to respond to pages about a production service, inside an agreed response time (for example "acknowledge within 5 minutes"). A **page** is an urgent notification, usually a phone push, SMS or phone call, that interrupts whatever the person is doing. The set of rules that decides *who* is on-call, *when* the duty changes hands, *what* they do when paged and *who* they call when they are stuck is the on-call practice.

A good practice has five parts, which are also the five things this topic teaches: a **rotation** (who and when), a **handoff** (how knowledge moves from one shift to the next), **runbooks** (what to do for each known alert), **escalation** (what happens when the first responder cannot fix it or does not answer), and **sustainable load** (how much interruption a human can absorb before quality and health collapse). Google's SRE books treat on-call as a core duty of the role, and the chapter "Being On-Call" in *Site Reliability Engineering* is the standard reference for the balance between responsiveness and burnout.

2 Why it matters in DevOps

- **Reliability is a people system.** Your SLOs §14.2 are only as good as the speed and clarity with which a tired human can respond at 03:00.
- **Burnout is a leading cause of attrition.** Teams with constant night pages lose their best engineers, and the knowledge goes with them.
- **Bad alerts waste the budget twice.** A noisy pager both interrupts people and trains them to ignore it, which delays response to real incidents (see alert fatigue reduction §14.8).
- **Developers who carry the pager write better software.** "You build it, you run it" closes the feedback loop: the person who ships a flaky retry loop is the one it wakes.
- **Incidents need a first responder.** The roles in incident management §14.5 assume that someone is on-call to start the process.

3 Real-world system example

Google SRE describes primary and secondary on-call, with the secondary acting as a backup and, in some teams, handling non-urgent tickets so the primary can focus. The SRE book discusses keeping the interrupt load low enough that engineers have time to investigate properly, and it recommends sizing a rotation so on-call does not come round too often (it gives figures for the minimum team size for single-site and dual-site rotations; treat them as guidance, not law). It also describes teams in two time zones doing **follow-the-sun** shifts so nobody works nights routinely.

PagerDuty publishes its incident response documentation openly (the "PagerDuty Incident Response" site), including a "Being On-Call" guide that sets expectations such as being reachable, having a working laptop and connectivity, and being ready to escalate rather than struggle alone.

Atlassian publishes an incident management handbook that covers on-call, escalation and ownership; the common thread across these public sources is that every service has a named owning team, layered escalation, and a runbook behind each paging alert.

Tools: Prometheus **Alertmanager** routes alerts to receivers; **PagerDuty**, **Opsgenie** and **Grafana OnCall** manage schedules, escalation policies and overrides. The concepts below apply to all of them.

4 Diagram

Escalation path when the first responder does not respond:

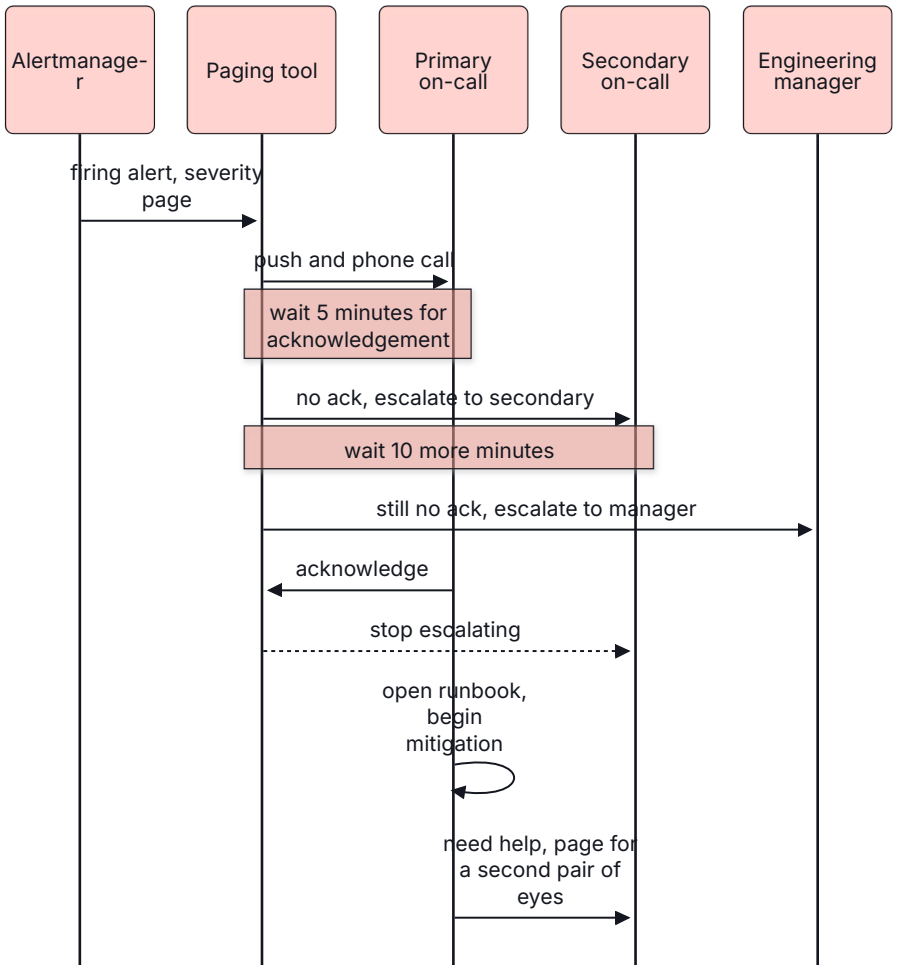


Figure 14.6-1

Weekly rotation and handoff:

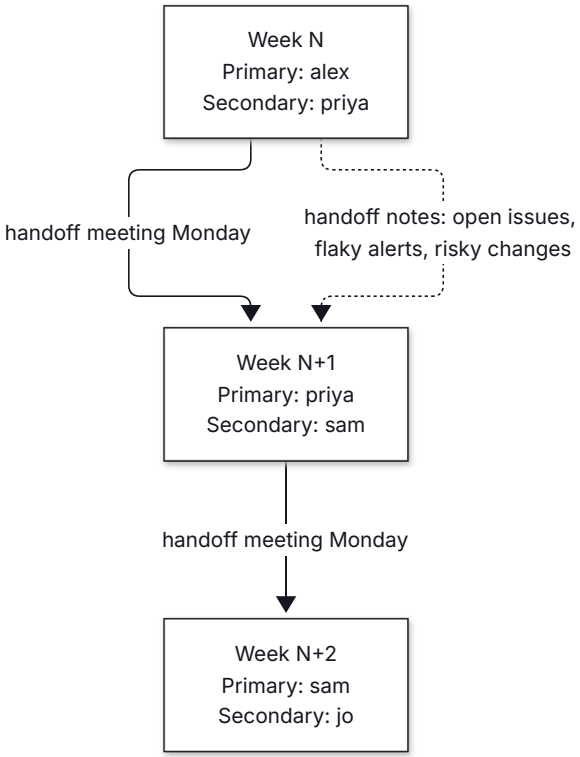


Figure 14.6-2

5 How it works

Rotation design

Choice	Options	Guidance
Shift length	1 week, 12 hours day/night, 24 hours	One week keeps context together but is tiring if pages are frequent. Shorter shifts suit busy services.
Layers	primary only, primary + secondary	Always have a backup. The secondary is who gets paged when the primary misses a page or needs help.
Coverage	single site, follow-the-sun	If you have engineers in two or more regions, keep night work out of anyone's routine.
Team size	minimum viable rotation	Fewer than about six people means on-call returns too often. Google's SRE book gives a minimum; use it as a sanity check.
Eligibility	owners of the service	Shadow first: new members sit as secondary before they go primary.

Choice	Options	Guidance
Compensation	pay, time off in lieu, or both	Out-of-hours work must be recognised. Unpaid, unrecognised on-call is a retention risk.

Fairness rules worth writing down: overrides for holidays and illness are easy and visible in the tool, swaps do not require manager approval, nobody is primary two weeks in a row, and the person who worked a night page starts later the next day.

Handoff

A handoff is a short written and spoken transfer at the start of each shift. It should cover:

1. **Open incidents and tickets** and who owns them.
2. **Noisy or flaky alerts** seen this week and what was done about them.
3. **Risky changes** planned for the coming week (a migration, a certificate rotation, a big launch).
4. **Quirks** that are not yet in a runbook ("the nightly job spikes CPU at 02:00, ignore").

The outgoing engineer writes it; the incoming one reads it and asks questions. Keep it in the same place every time (a pinned doc or a ticket).

Runbooks

A **runbook** is the page linked from an alert that tells the responder what the alert means, how to confirm it, and how to mitigate it. The rule: *every paging alert has a runbook URL in its annotations*, and a runbook is reviewed after each incident that used it. A good one has: what the alert means in user terms, a dashboard link, three or four diagnostic commands with expected output, the safe mitigations in order (rollback, flag off, scale up, fail over), who to escalate to and when, and a "last reviewed" date. A runbook that says "investigate" is not a runbook.

Escalation

An **escalation policy** is the ordered list of who is notified and the delay before the next layer. Typical shape:

Step	Who	Delay before next step
1	Primary on-call	5 min to acknowledge
2	Secondary on-call	10 min
3	Team lead or engineering manager	15 min
4	Director or incident-response lead (SEV1 only)	n/a

Separately, **responders can escalate for help at any time**: "I am out of my depth, page the database owner" is a skill, not a failure. Make it safe to wake someone up when the cost of not doing so is user impact.

Sustainable load

Google's SRE material suggests an on-call engineer should handle only a small number of distinct events per shift and that time spent on operational work (on-call, tickets, manual tasks) should be capped, so engineers keep time for engineering. Concretely, track these numbers every month:

- **Pages per shift** and **out-of-hours pages per shift**, ideally with a target such as "fewer than two incidents per 12-hour shift".
- **Actionable ratio**: the share of pages that required a human action. Aim for nearly all.
- **Time to acknowledge** and **time to mitigate**.
- **Repeat pages**: the same alert firing again and again means a bug or a bad threshold, not a heroic responder.

If the load exceeds the target, the next sprint's work is reducing it: fix the root cause, automate the mitigation or delete the alert. This links directly to error budgets [§14.4]: budget burn that keeps paging people is the signal to stop shipping features.

6 Step-by-step guide

Prerequisites: Python 3 only. Optional: a free Grafana Cloud or PagerDuty trial account if you want to see a real schedule UI. No cost for the lab itself.

1. **Generate a six-week rotation with primary and secondary.**

```bash
mkdir -p ~/sre-lab && cd ~/sre-lab
cat > rotation.py <<'EOF'
#!/usr/bin/env python3
"""Generate a weekly primary/secondary on-call rotation.

usage: python3 rotation.py START_DATE WEEKS NAME [NAME ...]
  e.g. python3 rotation.py 2026-10-12 6 alex priya sam jo kim lee
The secondary is always the NEXT person in the list, so every engineer shadows
before they carry the pager alone.
"""
import sys
from datetime import date, timedelta

if len(sys.argv) < 5:
    sys.exit(__doc__)
```

```
start = date.fromisoformat(sys.argv[1])
weeks = int(sys.argv[2])
people = sys.argv[3:]
if len(people) < 4:
    sys.exit("need at least 4 people for a primary + secondary rotation")

print(f"{'week starting':14} {'primary':10} {'secondary':10} handoff")
for w in range(weeks):
    day = start + timedelta(weeks=w)
    primary = people[w % len(people)]
    secondary = people[(w + 1) % len(people)]
    handoff = (day + timedelta(weeks=1)).isoformat()
    print(f"{day.isoformat():14} {primary:10} {secondary:10} Mon {handoff} 10:00
  local")
print(f"\nEach person is primary 1 week in {len(people)} "
      f"({100 / len(people):.1f}% of the time) and secondary 1 week in
  {len(people)}.")
EOF
python3 rotation.py 2026-10-12 6 alex priya sam jo kim lee
```

Expected:

```
week starting  primary    secondary  handoff
2026-10-12     alex       priya      Mon 2026-10-19 10:00 local
2026-10-19     priya      sam        Mon 2026-10-26 10:00 local
2026-10-26     sam        jo         Mon 2026-11-02 10:00 local
2026-11-02     jo         kim        Mon 2026-11-09 10:00 local
2026-11-09     kim        lee        Mon 2026-11-16 10:00 local
2026-11-16     lee        alex       Mon 2026-11-23 10:00 local

Each person is primary 1 week in 6 (16.7% of the time) and secondary 1 week in 6.
```

2. **Estimate whether your pager load is sustainable.**

```BASH
cat > oncall_load.py <<'EOF'
#!/usr/bin/env python3
"""On-call load estimator.

usage: python3 oncall_load.py TEAM_SIZE PAGES_PER_WEEK [OUT_OF_HOURS_FRACTION]
  [MIN_PER_PAGE]
  e.g. python3 oncall_load.py 6 21 0.4 45
PAGES_PER_WEEK is the total pages the whole rotation receives per week.
"""
import sys

if len(sys.argv) < 3:
    sys.exit(__doc__)
team = int(sys.argv[1])
pages_week = float(sys.argv[2])
```

```
ooh = float(sys.argv[3]) if len(sys.argv) > 3 else 0.4
mins = float(sys.argv[4]) if len(sys.argv) > 4 else 45

shifts_per_year = 52 / team            # one-week primary shifts
pages_per_shift = pages_week           # the primary takes every page that week
ooh_pages = pages_per_shift * ooh
per_day = pages_per_shift / 7
hours_on_pages = pages_per_shift * mins / 60

print(f"team size                : {team}")
print(f"one-week shifts per year : {shifts_per_year:.1f} per engineer")
print(f"pages per shift          : {pages_per_shift:.1f}  ({per_day:.1f} per day)")
print(f"out-of-hours pages/shift : {ooh_pages:.1f}  (each can wake someone up)")
print(f"hands-on time per shift  : {hours_on_pages:.1f} h at {mins:g} min per page")

verdict = []
if per_day > 2:
    verdict.append("more than 2 pages per day: not sustainable, fix the alerts first")
if ooh_pages > 7:
    verdict.append("more than one night-time page per night on average: sleep debt")
if team < 6:
    verdict.append("small rotation: on-call comes round too often, add people or share
  across teams")
print("verdict                 :", "; ".join(verdict) if verdict else "within a
  sustainable envelope")
EOF
python3 oncall_load.py 6 21 0.4 45
```

Expected:

```
team size                : 6
one-week shifts per year : 8.7 per engineer
pages per shift          : 21.0  (3.0 per day)
out-of-hours pages/shift : 8.4  (each can wake someone up)
hands-on time per shift  : 15.8 h at 45 min per page
verdict                  : more than 2 pages per day: not sustainable, fix the alerts
  first; more than one night-time page per night on average: sleep debt
```

Now try `python3 oncall_load.py 6 5 0.2 30` and compare. The thresholds in the script are illustrative; tune them to your team's agreement.

3. **Write the escalation policy as code** so it is reviewed like any other change. The exact format depends on your tool; this vendor-neutral YAML is what you would translate into

PagerDuty, Opsgenie or Grafana OnCall:

```yaml
# escalation-policy.yml  (illustrative, tool-neutral)
service: shop-api
urgency_routes:
  page:    # severity=page alerts from Alertmanager
    steps:
      - notify: schedule/shop-api-primary
        wait_minutes: 5
      - notify: schedule/shop-api-secondary
        wait_minutes: 10
      - notify: user/engineering-manager
        wait_minutes: 15
  ticket:  # severity=ticket alerts: never wake anyone
    steps:
      - notify: queue/shop-api-triage
```

Check it parses: `python3 -c "import yaml,sys; print(list(yaml.safe_load(open('escalation-policy.yml'))['urgency_routes']))"` should print `['page', 'ticket']` (needs `pip install pyyaml` if missing).

4. **Route alerts by severity in Alertmanager** so only `severity: page` reaches the phone. This is a minimal, syntactically valid `alertmanager.yml`; replace the placeholders with your own endpoints and keys:

```yaml
route:
  receiver: ticket-queue
  group_by: [alertname, job]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - matchers:
        - severity = "page"
      receiver: oncall-pager
      repeat_interval: 1h
receivers:
  - name: oncall-pager
    pagerduty_configs:
      - routing_key: <YOUR_PAGERDUTY_INTEGRATION_KEY>
  - name: ticket-queue
    webhook_configs:
      - url: https://tickets.example.com/hooks/alertmanager
```

If you have `amtool`, `amtool check-config alertmanager.yml` validates it.

5. **Write the runbook for your first paging alert.** Save as `runbooks/shop-api-error-budget-burn.md`:

```markdown
# Runbook: ShopApiErrorBudgetBurnFast
Last reviewed: 2026-10-09   Owner: team-shop
## What it means
Checkout errors are burning the 30-day budget 14.4x too fast (about 2% in an hour).
## Confirm
- Dashboard: https://grafana.example.com/d/shop-api-slo
- `kubectl -n shop get pods` (look for CrashLoopBackOff)
- `kubectl -n shop rollout history deploy/shop-api` (was there a deploy in the last
  hour?)
## Mitigate, in order
1. Recent deploy? `kubectl -n shop rollout undo deploy/shop-api`
2. Database saturated? Disable the heavy feature flag `reports_v2`.
3. Dependency down? Fail over per the dependency runbook.
## Escalate
After 15 minutes without improvement, page the secondary and declare a SEV2 (see
  incident management).
```

6. **Use a handoff template.** Paste into a pinned ticket and copy it each week:

```
ON-CALL HANDOFF  week of <DATE>    from: <name>    to: <name>
Open incidents / tickets : <links>
Noisy alerts this week   : <alert name, count, action taken>
Risky changes next week  : <what, when, owner>
Not yet in a runbook     : <quirks>
Pages: <n total>, <n out of hours>, <n actionable>
```

7 Real-world scenario

Illustrative, not a documented incident. The `shop-api` team of four has a weekly rotation with no secondary. In one week the primary, Jo, receives 31 pages, 14 out of hours, most from a disk-usage alert on a log volume that fixes itself when logrotate runs. On Thursday at 03:20 a genuine fast-burn page arrives; Jo, after four nights of broken sleep, acknowledges and falls asleep again. Nobody else is paged, and the incident runs 40 minutes before a customer email wakes the team lead.

The review changes three things: add a **secondary** and an escalation step after 5 minutes; delete the disk alert and replace it with a `predict_linear` ticket-severity alert; and grow the rotation to six by borrowing two engineers from the platform team. The next month's report shows 4 pages in the week, 1 out of hours. **Lesson:** the fix for a missed page was not "try harder", it was reducing noise, adding a backup layer and spreading the load.

8 Common mistakes and best practices

Don't

- Page on things that do not need a human now (disk at 80%, a single pod restart). Send those to a ticket queue.
- Run a rotation with one person and no escalation. A missed page then becomes an outage.
- Let on-call be unpaid, unrecognised or a punishment. Do not schedule someone for back-to-back weeks.
- Keep runbooks in someone's head, or link alerts to wiki pages nobody has opened in a year.
- Treat repeat pages as normal. If the same alert fires weekly, the alert or the system is broken.

Do

- Put a runbook link in every page-severity alert and review the runbook after each use.
- Have a primary and a secondary, a documented escalation policy and an easy override process.
- Do a written handoff every shift, including the noisy alerts and upcoming risky changes.
- Track pages per shift, actionable ratio and out-of-hours pages, and reduce them every sprint.
- Let new engineers shadow before going primary, and run mock pages so the first real one is not the first.

9 Tasks

- [ ] **T1 — Build a rotation** *(≈20 min, beginner)*
  - **Goal:** generate a fair primary/secondary schedule.
  - **Steps:**
    1. Create `rotation.py` and run it for eight people over eight weeks.
    2. Check that nobody is primary two weeks in a row.
    3. Add a holiday: edit the output by hand so that one person swaps a week with another, and note it as an override.
  - **Done when:** you have an eight-week schedule with one documented override and each person primary exactly once.
- [ ] **T2 — Judge a load** *(≈20 min, beginner)*

- **Goal:** use numbers to decide whether a rotation is sustainable.
- **Steps:**
  1. Run `oncall_load.py` for these cases: `4 8`, `6 21 0.4 45`, `8 6 0.1 30`.
  2. For each, write one line saying whether it is sustainable and what you would change first.
  3. Change one threshold in the script to match a target your team might agree on.
- **Done when:** you have three verdicts and the script reflects your own threshold.

☐ **T3 — Write a real runbook** *(≈45 min, intermediate)*

- **Goal:** produce a runbook a stranger could follow at 3 a.m.
- **Steps:**
  1. Pick `ShopApiErrorBudgetBurnFast` from error budgets §14.4 or an alert from your own lab.
  2. Write meaning, confirm commands with expected output, ordered mitigations and escalation.
  3. Ask a teammate to follow it cold and note where they got stuck.
- **Done when:** a teammate completed the confirm and mitigate steps without asking you a question, and you fixed every gap they found.

☐ **T4 — Wire severity routing** *(≈45 min, intermediate)*

- **Goal:** ensure only urgent alerts page.
- **Steps:**
  1. Create the `alertmanager.yml` from section 6 (use a local webhook receiver such as a small `python3 -m http.server` handler instead of PagerDuty if you want to see the delivery).
  2. Run `amtool check-config alertmanager.yml` or start Alertmanager and open its status page.
  3. Send a test alert with `curl -XPOST localhost:9093/api/v2/alerts` carrying `severity=page`, then one with `severity=ticket`.
- **Done when:** the page alert reaches the pager receiver and the ticket alert reaches only the ticket receiver.

☐ **T5 — Run a missed-page drill** *(≈45 min, advanced)*

- **Goal:** feel the escalation path before it matters.
- **Steps:**
  1. With a teammate as secondary, simulate an alert and have the primary deliberately not acknowledge.

2. Time how long until the secondary is notified and until a human starts working.
3. Compare the measured times with the policy in section 5 and fix any gap (wrong phone number, muted app, wrong schedule).

- **Done when:** you have a timed log from alert to first human action and a list of at least one improvement made.

- [ ] **T6 — Prove it: audit an on-call setup** *(≈60 min, advanced)*
  - **Goal:** assess a rotation without a checklist.
  - **Steps:**
    1. From memory, list the five parts of an on-call practice and one metric for each.
    2. Review a real or invented `shop-api` setup (team of five, 18 pages a week, 60% out of hours, three runbooks for twelve alerts) and write the three highest-value changes.
    3. Present the reasoning in under five minutes.
  - **Done when:** your three changes address noise, runbook coverage and rotation size, each with a measurable target.

10 Self-check questions

1. Why is a secondary on-call layer necessary?
2. What must be in the annotations of every paging alert?
3. Name four things that belong in a shift handoff.
4. What is the difference between a page and a ticket, and who decides?
5. What do you do if the same alert pages three times in a week?

ANSWERS

1. People miss pages (asleep, no signal, in the middle of another incident). The secondary prevents a single failure from becoming an unanswered outage and provides a second pair of eyes.
2. A runbook URL (plus a summary that explains impact in user terms and ideally a dashboard link).
3. Open incidents and tickets, noisy or flaky alerts, risky changes coming up, and quirks not yet in a runbook (also the page counts for the week).
4. A page requires immediate human action and wakes someone; a ticket can wait until working hours. The team decides together, using the rule "if no human action is needed now, it is not a page".

5. Treat it as a defect: find the root cause or fix the threshold, add automation for the mitigation, or delete the alert. Track it as a work item, do not normalise it.

11 Further reading

- *Site Reliability Engineering* (Google) — "Being On-Call", "Dealing with Interrupts" and "Eliminating Toil".
- *The Site Reliability Workbook* (Google) — "On-Call" and "Incident Response".
- PagerDuty Incident Response documentation (response.pagerduty.com) — "Being On-Call".
- Prometheus Alertmanager documentation (prometheus.io/docs/alerting) — routing and receivers.
- See also: Incident Management §14.5, Alert Fatigue Reduction §14.8 and Observability §0.10.

14.7 Postmortems

Topic 7 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

A **postmortem** (also called an incident review or retrospective) is a written record of an incident: what happened, what the impact was, how it was detected and fixed, *why it was possible*, and what the organisation will change so it is less likely or less damaging next time. It is written after the incident is resolved, reviewed by people outside the immediate response, and shared widely.

The defining principle is **blamelessness**. A blameless postmortem assumes that everyone involved acted reasonably with the information, tools and incentives they had at the time, and asks what about the *system* made the failure possible: missing guardrails, confusing tooling, absent tests, unclear ownership, time pressure. It does not ask "who messed up?". Google's SRE books ("Postmortem Culture: Learning from Failure" in *Site Reliability Engineering* and the matching chapter in *The Site Reliability Workbook*) are the standard reference, and the idea of a "just culture" in operations was popularised in the industry by writing from Etsy's engineering team.

2 Why it matters in DevOps

- **Incidents are the cheapest education you will ever get.** You have already paid the price (downtime, error budget §14.4); the postmortem is how you collect the value.
- **Blame hides information.** If admitting a mistake gets you punished, the next person will hide theirs, and the next incident will repeat the same failure with less evidence.
- **Action items are the output.** Without tracked follow-up, a postmortem is a story. With it, it is a reliability investment that competes fairly with feature work.
- **Shared knowledge.** A well-written postmortem teaches the whole company how a subsystem really fails, including people who were not on the call.
- **Feeds the loop.** Findings change alerts §14.8, runbooks §14.6, capacity assumptions and the error budget policy §14.4.

3 Real-world system example

Google publishes its approach in the SRE books: postmortems are expected after significant incidents, are blameless, are reviewed, and are kept somewhere searchable so patterns across incidents become visible. The books also describe recognising good postmortem writing, and its on-call training material describes "Wheel of Misfortune" role-play exercises that replay past incidents.

AWS publishes "post-event summaries" for major service events. The widely read summary of the February 2017 Amazon S3 event in US-EAST-1 describes an operator running an established playbook command with an input that removed more capacity than intended, and lists changes such as making the tool remove capacity more slowly and adding safeguards against going below minimum capacity. It is a public example of focusing on tooling and guardrails rather than on the individual.

GitHub publishes monthly availability reports and detailed write-ups of significant incidents; **Cloudflare** is known for long technical post-incident blog posts with timelines. **Atlassian** publishes its incident-management handbook, including a postmortem process with priority action items and agreed completion deadlines. The shared pattern: a timeline, an honest account of what went wrong, and concrete follow-ups.

4 Diagram

The postmortem process:

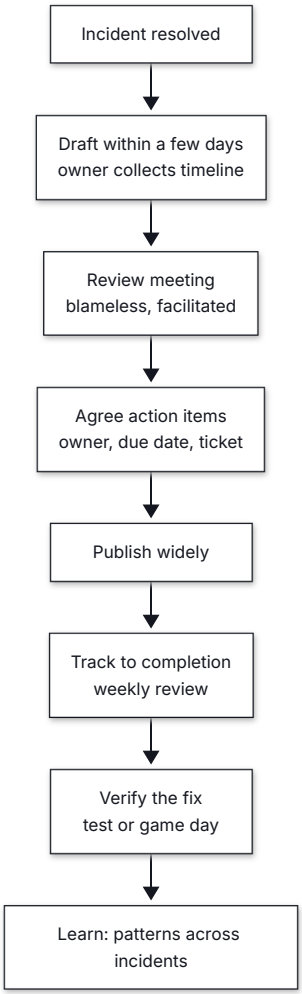


Figure 14.7-1

Why one incident has many causes:

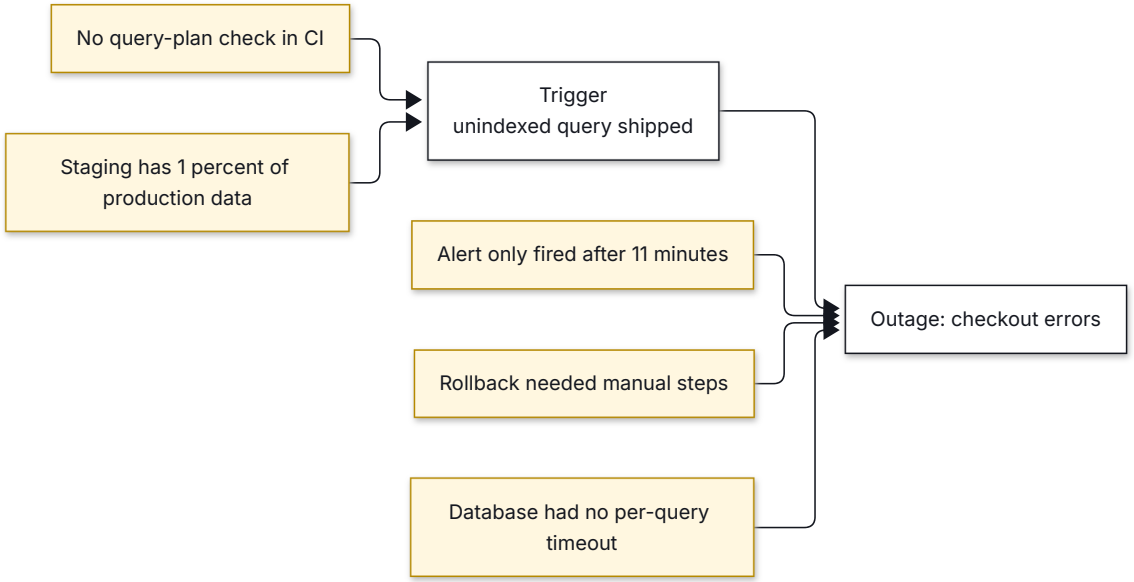


Figure 14.7-2

5 How it works

Blameless culture, in practice

Blameful phrasing	Blameless phrasing
"Alex pushed a bad query."	"A change containing an unindexed query passed review and CI."
"The on-call should have noticed."	"The alert gave no indication of which deploy to suspect."
"Human error."	"The tool allowed an unsafe input without confirmation."

"Human error" is where analysis starts, not where it ends: ask what made the error easy to make and hard to catch. Blameless does *not* mean unaccountable: people still own action items and still explain their decisions, but the question is "what did you see, and what did you expect?", not "why were you careless?".

The timeline

The **timeline** is a list of timestamped facts in UTC, written from the incident log (see the scribe tool in incident management §14.5). Include the trigger, when impact started, when it was detected, each significant decision and why, each mitigation attempt (including the fail-

ures), when impact ended and when the incident was closed. Record what people *knew* at the time, not what is known now. From it you compute the key durations:

Metric	Meaning
Time to detect (TTD)	impact start to first alert or report
Time to mitigate (TTM)	detection to the moment user impact stops
Time to resolve (TTR)	impact start to full resolution
Budget consumed	bad events during the incident divided by the 30-day budget

Root cause versus contributing factors

Complex systems rarely fail for a single reason. The **trigger** is the event that started this particular failure (the bad deploy). The **root cause** is the deepest underlying condition that, if fixed, would have prevented the trigger from causing harm (for example, "there is no automated check that stops unindexed queries reaching production"). **Contributing factors** are the other conditions that made it worse or longer: slow detection, a missing runbook, a confusing dashboard, no timeout. Practical guidance:

- Use techniques like **Five Whys** to get beyond the trigger, but stop when the next "why" leaves your control or becomes "because humans are fallible".
- It is fine to name *several* causes. Do not force a single root cause to make the story tidy.
- Separate **what caused it** from **what made it long**: the first drives prevention, the second drives detection and mitigation.
- Include things that went *well* (the alert fired, the rollback worked); these are controls you want to keep.

Action items

An action item is only useful if it is specific, owned and tracked. Classify each by what it improves:

Type	Question it answers	Example
Prevent	How do we stop this class of failure?	Add a query-plan check to CI
Detect	How do we find it faster?	Alert on p99 query time per route
Mitigate	How do we shorten the impact?	One-command rollback; per-query timeout
Process	How do we respond better?	Add the missing step to the runbook

Each item needs: an ID, a clear verb-first description, a type, a named **owner**, a **due date**, a **ticket** in the normal work tracker, and a priority. Keep the list short (a handful of high-value items beats thirty vague ones) and review completion in a recurring meeting. "Be more careful" and "improve monitoring" are not action items.

6 Step-by-step guide

Prerequisites: Python 3 and the `inc.sh` scribe from incident management §14.5 (or just a text file). All free and local.

1. **Compute incident timings from the scribe log.** Mark key moments by starting the note text with a tag (`[impact-start]` , `[detected]` , `[mitigated]` , `[resolved]`).

```bash
mkdir -p ~/sre-lab && cd ~/sre-lab
cat > pm_times.py <<'EOF'
#!/usr/bin/env python3
"""Compute incident timings from an inc.sh log.

usage: python3 pm_times.py incident-<id>.log

Mark key moments in the log with these tags at the start of the note text:
  [impact-start] [detected] [mitigated] [resolved]
"""
import re
import sys
from datetime import datetime

if len(sys.argv) != 2:
    sys.exit(__doc__)

TAGS = ("impact-start", "detected", "mitigated", "resolved")
marks = {}
for line in open(sys.argv[1]):
    m = re.match(r"(\S+Z)\s+\w+\s+\[([a-z-]+)\]\s*(.*)", line)
    if m and m.group(2) in TAGS:
        marks[m.group(2)] = datetime.strptime(m.group(1), "%Y-%m-%dT%H:%M:%SZ")

missing = [t for t in TAGS if t not in marks]
if missing:
    sys.exit(f"missing tags in log: {', '.join(missing)}")

def mins(a, b):
    return (marks[b] - marks[a]).total_seconds() / 60

print(f"time to detect   (impact-start → detected) : {mins('impact-start',
  'detected'):6.1f} min")
print(f"time to mitigate (detected → mitigated)    : {mins('detected',
  'mitigated'):6.1f} min")
```

```
print(f"time to resolve  (impact-start → resolved) : {mins('impact-start',
  'resolved'):6.1f} min")
print(f"total user impact (impact-start → mitigated): {mins('impact-start',
  'mitigated'):6.1f} min")
EOF
cat > sample.log <<'EOF'
2026-10-09T10:05:00Z NOTE   [impact-start] Deploy 4821 reaches 100% of pods, checkout
  errors begin
2026-10-09T10:12:10Z NOTE   [detected] ShopApiErrorBudgetBurnFast pages priya
2026-10-09T10:15:02Z START  Checkout returns 502 for 30% of users (SEV2)
2026-10-09T10:15:02Z ROLE   IC=alex
2026-10-09T10:21:30Z NOTE   Ops proposes rollback of deploy 4821
2026-10-09T10:24:45Z NOTE   [mitigated] Rollback complete, 5m error ratio falling
2026-10-09T10:55:00Z NOTE   [resolved] Error ratio below 0.1% for 30 minutes
EOF
python3 pm_times.py sample.log
```

Expected:

```
time to detect   (impact-start → detected) :    7.2 min
time to mitigate (detected → mitigated)    :   12.6 min
time to resolve  (impact-start → resolved) :   50.0 min
total user impact (impact-start → mitigated):   19.8 min
```

2. **Create the postmortem template** `postmortem-template.md`. This is the document you copy for every incident:

```
BASH
cat > postmortem-template.md <<'EOF'
# Postmortem: <short title>

| | |
|---|---|
| Incident | INC-<id>, SEV<n> |
| Date | <YYYY-MM-DD> |
| Authors | <names> |
| Status | Draft / In review / Final |
| Reviewers | <names> |

## Summary
Two or three sentences a stranger can read: what broke, who was affected, how long,
  how it was fixed.

## Impact
- User impact: <what users saw, how many, which feature>
- Duration: <minutes> (TTD <m>, TTM <m>, TTR <m>)
- Error budget consumed: <percent of the 30-day budget>
- Other: <revenue, support tickets, data loss, SLA exposure>

## Timeline (all times UTC)
```

```
| Time | Event |
|------|-------|
| HH:MM | Trigger: <change or event> |
| HH:MM | Impact begins |
| HH:MM | Detected by <alert or person> |
| HH:MM | Incident declared SEV<n>, IC <name> |
| HH:MM | Decision: <what, why, what was known> |
| HH:MM | Mitigated |
| HH:MM | Resolved |

## Root cause
The deepest condition that, if fixed, would have prevented this failure.

## Contributing factors
- <factor that made it worse or longer>

## What went well
- <controls and behaviours to keep>

## What went poorly
- <gaps, confusion, luck>

## Where we got lucky
- <things that could have made it worse>

## Action items
| ID | Action | Type | Owner | Due | Ticket |
|----|--------|------|-------|-----|--------|
| AI-1 | <verb-first, specific> | prevent/detect/mitigate/process | @name | YYYY-MM-DD
  | <TICKET-ID> |

## Lessons
What other teams should learn from this.
EOF
```

3. **Lint a postmortem** before it goes to review. The checker enforces that the sections exist and that every action item has an owner, a due date and a ticket.

BASH

```
cat > pm_lint.py <<'EOF'
#!/usr/bin/env python3
"""Lint a postmortem Markdown file.

usage: python3 pm_lint.py postmortem.md
Checks: required headings, at least one action item row, every action item has an
owner (@name), a due date (YYYY-MM-DD) and a ticket link or ID.
"""
import re
import sys

if len(sys.argv) != 2:
```

```
    sys.exit(__doc__)
text = open(sys.argv[1], encoding="utf-8").read()
problems = []

for h in ["Summary", "Impact", "Timeline", "Root cause", "Contributing factors",
          "What went well", "What went poorly", "Action items", "Lessons"]:
    if not re.search(rf"^##\s+{re.escape(h)}", text, re.M | re.I):
        problems.append(f"missing section: {h}")

rows = [r for r in text.splitlines() if re.match(r"\|\s*AI-\d+", r)]
if not rows:
    problems.append("no action items (rows starting with | AI-n)")
for r in rows:
    cells = [c.strip() for c in r.strip().strip("|").split("|")]
    ident = cells[0]
    if not re.search(r"@\w+", r):
        problems.append(f"{ident}: no owner (@name)")
    if not re.search(r"\d{4}-\d{2}-\d{2}", r):
        problems.append(f"{ident}: no due date")
    if not re.search(r"\b[A-Z]{2,}-\d+\b|https?://", " ".join(cells[1:])):
        problems.append(f"{ident}: no ticket reference")

for p in problems:
    print("PROBLEM:", p)
print(f"{len(problems)} problems, {len(rows)} action items")
sys.exit(1 if problems else 0)
EOF
python3 pm_lint.py postmortem-template.md
```

The raw template contains one placeholder row (`AI-1 | <verb-first, specific> | ... | @name | YYYY-MM-DD | <TICKET-ID>`), so it reports problems for the date and ticket. That is the point: a filled-in postmortem must pass with `0 problems`.

4. **Fill in a postmortem for the sample incident** (the unindexed query from error budgets §14.4). Start from the template and write the Root cause and Contributing factors honestly. A reasonable pair of examples:

```
MARKDOWN
## Root cause
Nothing in the delivery pipeline checks query plans, so a query that scans the whole
  orders table reached production.

## Contributing factors
- Staging holds roughly 1% of production data, so the query was fast there.
- The database has no statement timeout, so slow queries piled up and exhausted the
  connection pool.
- The fast-burn alert fired about 7 minutes after impact began and carried no link to
  the recent deploy.
- Rollback required three manual steps.
```

And action items that pass the linter:

```markdown
| AI-1 | Add EXPLAIN-based plan check for new queries to CI | prevent | @priya | 2026-10-30 | SHOP-412 |
| AI-2 | Set statement_timeout of 2s on the API role | mitigate | @jo | 2026-10-16 | SHOP-413 |
| AI-3 | Add deploy annotation and rollback link to the burn-rate alert | detect | @sam | 2026-10-23 | SHOP-414 |
| AI-4 | Make rollback a single pipeline button | mitigate | @alex | 2026-11-06 | SHOP-415 |
```

5. **Run the review meeting** (30-45 minutes): a facilitator who was not in the incident walks the timeline, asks "what did you see at that point?", records new facts, and then agrees action items with owners. The facilitator's job is to redirect any "who" question into a "what in the system" question.
6. **Track to completion.** Review open postmortem action items in a weekly or fortnightly meeting. An item that is overdue twice is either not important (close it explicitly with a reason) or not resourced (escalate).

7 Real-world scenario

Illustrative, not a documented incident. After a 20-minute checkout outage the first draft of the `shop-api` postmortem reads "Root cause: engineer deployed an inefficient query". In review the facilitator asks the author what Alex saw before shipping. Alex explains the query was reviewed by two people, passed tests, and ran in 4 ms in staging. The team realises the actual gaps: staging data is too small, there is no plan check, and no timeout protects the pool. The root cause is rewritten as a missing control, and "Alex" disappears from the document except as the person who tried to roll back first.

Six weeks later a different engineer ships a different slow query. CI blocks it with a plan-check failure. **Lesson:** a blameless review changed the system, and the system, not a reminder email, prevented the repeat. The tracked action items were what turned the lesson into a control.

8 Common mistakes and best practices

Don't

- Write "human error" or a person's name as the root cause.
- Stop at the trigger. Keep asking how it got past review, tests, staging and alerts.
- Produce a postmortem with no owners or deadlines, then never look at it again.

- Hide postmortems. Restricting readership hides patterns and teaches nothing.
- Skip the postmortem for "small" incidents that repeat. Repeats are what budgets are lost to.

Do

- Write a draft within a few days while memory is fresh, working from the incident log.
- Put times in UTC and record what people knew at the time.
- List contributing factors, what went well and where you got lucky, not only what failed.
- Keep action items few, specific, owned, ticketed and dated, and review them regularly.
- Share broadly and search past postmortems before designing similar systems.

9 Tasks

- [ ] **T1 — Compute the timings** *(≈20 min, beginner)*
  - **Goal:** turn a log into TTD, TTM and TTR.
  - **Steps:**
    1. Create `pm_times.py` and `sample.log` as in section 6.
    2. Run the script and check the four numbers.
    3. Edit the log so the rollback happens 10 minutes later and rerun.
  - **Done when:** time to mitigate increases by 10 minutes and you can say what each metric measures.
- [ ] **T2 — Rewrite the blame** *(≈20 min, beginner)*
  - **Goal:** practise blameless wording.
  - **Steps:**
    1. Rewrite these in system terms: "The intern deleted the wrong table", "Ops ignored the alert", "QA missed the bug", "Sam should have read the docs".
    2. For each, name one control (guardrail, alert, check, doc) that would have helped.
  - **Done when:** four rewritten sentences contain no person as the subject and each names a concrete control.
- [ ] **T3 — Lint and fix** *(≈30 min, intermediate)*
  - **Goal:** use the checker to enforce action-item quality.
  - **Steps:**
    1. Create `pm_lint.py` and a postmortem with two action items, one missing an owner and one missing a ticket.

2. Run the linter and read the problems.
3. Fix them until the script prints `0 problems`.

- **Done when:** the final run exits with status 0 (`echo $?` prints `0`).

- [ ] **T4 — Separate trigger, root cause and contributing factors** *(≈45 min, intermediate)*
  - **Goal:** analyse an incident beyond the trigger.
  - **Steps:**
    1. Take the unindexed-query incident (or a real incident from your work) and write a Five Whys chain.
    2. State the trigger, the root cause and at least three contributing factors, tagging each as "caused it" or "made it longer".
    3. Write one action item of each type: prevent, detect, mitigate, process.
  - **Done when:** you have four action items with owner, due date and ticket, and none says "be more careful".

- [ ] **T5 — Run a mock review** *(≈60 min, advanced)*
  - **Goal:** practise facilitation.
  - **Steps:**
    1. After the capstone mock incident (or any practice incident), gather two or more people.
    2. Have a facilitator who was not the IC walk the timeline and convert every "who" question into a "what in the system" question.
    3. Agree action items live and record them in the template.
  - **Done when:** the postmortem passes `pm_lint.py` and participants confirm the discussion never assigned fault to a person.

- [ ] **T6 — Prove it: read a public postmortem** *(≈60 min, advanced)*
  - **Goal:** analyse an outside incident without notes.
  - **Steps:**
    1. Read a public post-incident report from a cloud provider or a company that publishes them (for example AWS post-event summaries or GitHub availability reports).
    2. Summarise it in the template format: summary, timeline, trigger, root cause, contributing factors, and the fixes they announced.
    3. Classify each fix as prevent, detect, mitigate or process and say which you think was highest value.

- **Done when:** your one-page summary distinguishes trigger from contributing factors and justifies the highest-value fix.

10 Self-check questions

1. What does "blameless" mean, and what does it not mean?
2. What is the difference between a trigger, a root cause and a contributing factor?
3. What four fields must every action item have?
4. Why do postmortems use UTC timestamps and record what people knew at the time?
5. How do you tell a good action item from a bad one?

ANSWERS

1. It means analysing how the system and its incentives made the failure possible, assuming people acted reasonably with what they knew. It does not mean no accountability: owners still deliver action items and explain decisions.
2. The trigger starts the failure (the bad deploy). The root cause is the underlying condition that, if fixed, prevents the trigger from causing harm. Contributing factors are conditions that made impact larger or longer (slow detection, no timeout, no runbook).
3. An owner, a due date, a ticket in the real tracker, and a specific description (type and priority are also recommended).
4. UTC avoids time-zone confusion across responders and logs; recording what was known at the time avoids hindsight bias and shows which information was missing.
5. A good one is specific, verb-first, testable, owned, dated and tied to a cause ("add EXPLAIN check to CI"). A bad one is vague ("improve monitoring", "be more careful").

11 Further reading

- *Site Reliability Engineering* (Google) — "Postmortem Culture: Learning from Failure" and the appendix with an example postmortem.
- *The Site Reliability Workbook* (Google) — "Postmortem Culture" chapter and its postmortem template material.
- John Allspaw, "Blameless PostMortems and a Just Culture" (Etsy engineering blog).
- AWS "post-event summaries" and GitHub availability reports, as examples of public postmortems.
- Atlassian Incident Management Handbook — postmortem process.
- See also: Incident Management §14.5, Error Budgets §14.4 and Disaster Recovery §0.9.

LAB NOTES What I ran, what I saw, what it means

14.8 Alert Fatigue Reduction

Topic 8 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

Alert fatigue is what happens when people receive so many alerts, or so many low-value ones, that they stop trusting them. They acknowledge without looking, mute channels, delay responding, and eventually miss the one alert that mattered. It is a property of the *alerting system*, not of the people: a pager that cries wolf teaches its audience to ignore wolves.

Alert fatigue reduction is the continuous work of making every alert earn its interruption. Four ideas do most of the work and make up this topic: **symptom-based alerts** (page on what users feel, not on every internal cause), **actionable alerts** (each one tells a human something to do right now), **deduplication and grouping** (one problem produces one notification, not forty), and a regular **alert review** (measure, then delete or tune). The foundation is the "Monitoring Distributed Systems" chapter in Google's *Site Reliability Engineering* book, which separates symptoms ("what is broken") from causes ("why") and argues that pages should be rare, urgent and actionable.

2 Why it matters in DevOps

- **Missed incidents.** Fatigued responders miss the real page in the noise. Every unanswered alert is a hidden outage.
- **Burnout and attrition.** Night-time pages that need no action are the fastest way to lose good on-call engineers (see on-call practices §14.6).
- **Slower response.** Time to acknowledge grows when people expect false alarms, which burns error budget §14.4 during real incidents.
- **Wasted effort.** Investigating a symptom-less alert, such as CPU at 85% with healthy latency, takes time away from engineering that would improve reliability.
- **Credibility.** When every page is meaningful, leaders and developers take on-call seriously and fund the fixes.

3 Real-world system example

Google SRE states the guiding rules in *Site Reliability Engineering* ("Monitoring Distributed Systems", and the section on symptoms versus causes): pages should be about conditions that are urgent, actionable and visible to users; every page should require intelligence, not a robotic response; and alerts that produce no action should be removed or turned into tickets or dashboards. The *Workbook* chapter "Alerting on SLOs" operationalises this with burn-rate alerts that replace many cause-based thresholds. Rob Ewaschuk's widely circulated "My Philosophy on Alerting" (written while at Google) is a short companion that states the same ideas bluntly.

Prometheus Alertmanager is the open-source implementation of grouping, inhibition and silencing: it groups alerts with shared labels into one notification, suppresses dependent alerts when a more fundamental one is firing, and mutes alerts during maintenance. **PagerDuty** and **Opsgenie** provide similar grouping, deduplication keys and noise-reduction features, plus analytics for "which alerts wake people most".

Public examples of the symptom/cause split exist in most mature monitoring guides, including the Prometheus documentation's alerting practices: alert on user-visible symptoms such as error rate and latency, and keep cause-level metrics for dashboards and diagnosis.

4 Diagram

Deciding whether something deserves a page:

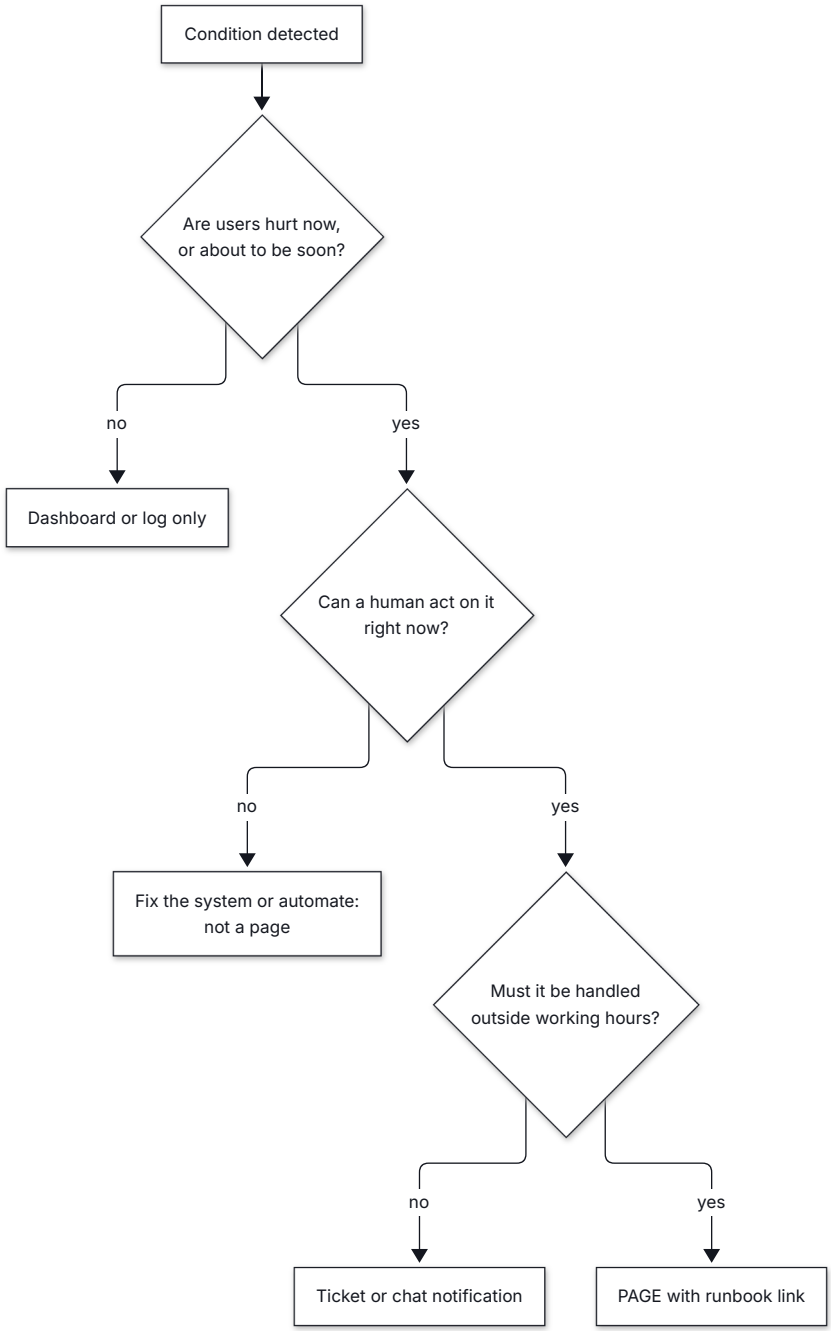

Condition detected
Are users hurt now, or about to be soon?
no
yes
Dashboard or log only
Can a human act on it right now?
no
yes
Fix the system or automate: not a page
Must it be handled outside working hours?
no
yes
Ticket or chat notification
PAGE with runbook link

Figure 14.8-1

What Alertmanager does between a firing alert and your phone:

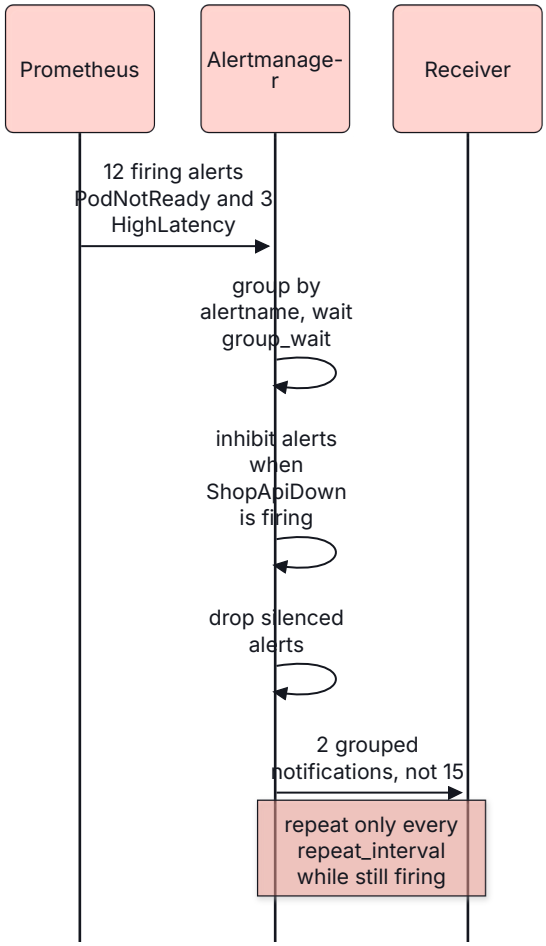


Figure 14.8-2

5 How it works

Symptom-based versus cause-based

	Cause-based (noisy)	Symptom-based (quiet)
Examples	CPU above 80%, one pod restarted, disk at 80%, memory above 90%	5xx ratio above threshold, p99 latency above SLO, SLO burn rate, checkout probe failing
What it tells you	A resource is busy; might or might not matter	Users are failing now
False positives	Many: busy is often fine	Few

	Cause-based (noisy)	Symptom-based (quiet)
Maintenance	Thresholds drift as traffic and hardware change	Tied to the SLO, which changes rarely
Where it belongs	Dashboards, tickets, diagnosis	The page

Cause metrics are still useful, but as **diagnostic context** after the symptom alert fires, and as **ticket-level** alerts for slow-moving risks (a disk predicted to fill in 3 days). The multi-window, multi-burn-rate alerts from error budgets §14.4 are the canonical symptom alerts: one set per SLO replaces dozens of threshold alerts.

What makes an alert actionable

Before an alert is allowed to page, it must pass these checks:

1. **Urgent:** waiting until morning makes things materially worse.
2. **Actionable:** there is something a human can do (not "watch it").
3. **Real:** it is almost never a false positive; if it flaps, fix the rule.
4. **Novel:** it is not a duplicate of another alert that will fire at the same time.
5. **Explained:** it carries a summary in user terms, severity, a dashboard link and a runbook URL.
6. **Owned:** it routes to the team that can fix it.

An alert that fails any check is moved down: to a ticket, a dashboard annotation or deleted.

Deduplication, grouping, inhibition, silences

Mechanism	What it does	Alertmanager setting
Deduplication	Identical alerts (same label set) from several Prometheus replicas become one	automatic by label fingerprint
Grouping	Alerts that share labels such as `alertname` or `job` are sent as one notification	`group_by`, `group_wait`, `group_interval`
Inhibition	A more fundamental alert suppresses its consequences ("service down" suppresses "high latency")	`inhibit_rules`
Silencing	Temporary mute for planned work, with an expiry and a reason	UI, `amtool silence add`
Throttling repeats	Stop re-notifying for the same unresolved problem	`repeat_interval`
`for:` **duration**	The condition must hold for N minutes before firing	in the alerting rule

Alert review

Treat the alert set like code: review it on a schedule. In a weekly (or per-rotation) 30-minute meeting, look at every page from the last period and for each alert ask: did it lead to action? Was it urgent? Could it be a ticket? Was it a duplicate? Then **delete, tune, demote or keep**, and record the decision. Typical metrics to bring: pages per alert, **actionable ratio** (pages that needed a human action divided by pages), out-of-hours pages, time to acknowledge, and repeat offenders.

6 Step-by-step guide

Prerequisites: Python 3. For the rules and Alertmanager config, `promtool` / `amtool` (from the Prometheus and Alertmanager release tarballs) are optional for validation; every snippet below is also valid YAML you can check with a YAML parser. All free and local.

1. **Review a week of pages.** Create a CSV that mimics your pager export, then analyse it.

```bash
mkdir -p ~/sre-lab && cd ~/sre-lab
cat > pages.csv <<'EOF'
timestamp,alert,action_taken
2026-10-01T02:10:00Z,DiskUsageHigh,no
2026-10-02T03:30:00Z,DiskUsageHigh,no
2026-10-03T01:05:00Z,DiskUsageHigh,no
2026-10-04T14:00:00Z,DiskUsageHigh,no
2026-10-05T04:20:00Z,DiskUsageHigh,no
2026-10-06T02:45:00Z,DiskUsageHigh,no
2026-10-01T09:00:00Z,HighCPU,no
2026-10-01T09:30:00Z,HighCPU,no
2026-10-02T11:00:00Z,HighCPU,no
2026-10-03T23:30:00Z,HighCPU,yes
2026-10-04T10:15:00Z,HighCPU,no
2026-10-03T15:00:00Z,PodRestarted,no
2026-10-03T15:05:00Z,PodRestarted,no
2026-10-04T09:00:00Z,PodRestarted,no
2026-10-05T16:00:00Z,PodRestarted,yes
2026-10-02T10:12:00Z,ShopApiErrorBudgetBurnFast,yes
2026-10-05T03:40:00Z,ShopApiErrorBudgetBurnFast,yes
2026-10-04T21:00:00Z,PostgresReplicaLag,yes
2026-10-05T22:00:00Z,PostgresReplicaLag,yes
2026-10-06T21:30:00Z,PostgresReplicaLag,no
EOF
cat > alert_review.py <<'EOF'
#!/usr/bin/env python3
"""Weekly alert review: which pages earn their place?

usage: python3 alert_review.py pages.csv
CSV columns: timestamp (ISO, UTC), alert, action_taken (yes/no)
"""
```

```
import csv
import sys
from collections import defaultdict
from datetime import datetime

if len(sys.argv) != 2:
    sys.exit(__doc__)

stats = defaultdict(lambda: {"n": 0, "act": 0, "ooh": 0})
with open(sys.argv[1], newline="") as f:
    for row in csv.DictReader(f):
        hour = datetime.strptime(row["timestamp"], "%Y-%m-%dT%H:%M:%SZ").hour
        s = stats[row["alert"]]
        s["n"] += 1
        s["act"] += row["action_taken"].strip().lower() == "yes"
        s["ooh"] += hour < 8 or hour >= 20     # treat 20:00-08:00 UTC as out of hours

def advice(s):
    ratio = s["act"] / s["n"]
    if ratio == 0:
        return "DELETE or turn into a ticket: no page led to action"
    if ratio < 0.5:
        return "TUNE: threshold, duration or symptom-based rewrite"
    return "KEEP: mostly actionable"

total = sum(s["n"] for s in stats.values())
total_act = sum(s["act"] for s in stats.values())
print(f"{'alert':30} {'pages':>5} {'actionable':>10} {'out of hrs':>10}  advice")
for name, s in sorted(stats.items(), key=lambda kv: -kv[1]["n"]):
    print(f"{name:30} {s['n']:>5} {100 * s['act'] / s['n']:>9.0f}% {s['ooh']:>10}
  {advice(s)}")
print(f"\ntotal pages {total}, actionable {100 * total_act / total:.0f}%")
EOF
python3 alert_review.py pages.csv
```

Expected:

```
alert                          pages actionable out of hrs  advice
DiskUsageHigh                      6         0%          5  DELETE or turn into a
  ticket: no page led to action
HighCPU                            5        20%          1  TUNE: threshold, duration
  or symptom-based rewrite
PodRestarted                       4        25%          0  TUNE: threshold, duration
  or symptom-based rewrite
PostgresReplicaLag                 3        67%          3  KEEP: mostly actionable
ShopApiErrorBudgetBurnFast         2       100%          1  KEEP: mostly actionable

total pages 20, actionable 30%
```

Seventy percent of this week's pages were noise, and the noisiest alert (`DiskUsageHigh`) woke someone five times without any action.

2. **See what grouping does to notification volume.**

```bash
cat > dedupe.py <<'EOF'
#!/usr/bin/env python3
"""Show what grouping does to notification volume.

usage: python3 dedupe.py alerts.txt [GROUP_WINDOW_MINUTES]
Each line: ISO_TIMESTAMP alertname instance
Compares: one notification per alert vs one per (alertname) group per window.
"""
import sys
from datetime import datetime, timedelta

if len(sys.argv) < 2:
    sys.exit(__doc__)
window = timedelta(minutes=float(sys.argv[2]) if len(sys.argv) > 2 else 5)
events = []
for line in open(sys.argv[1]):
    ts, name, inst = line.split()
    events.append((datetime.strptime(ts, "%Y-%m-%dT%H:%M:%SZ"), name, inst))
events.sort()

open_group = {}        # alertname -> window start
grouped = 0
for ts, name, inst in events:
    start = open_group.get(name)
    if start is None or ts - start > window:
        open_group[name] = ts
        grouped += 1
print(f"raw alerts           : {len(events)}")
print(f"notifications grouped: {grouped}  (group_by alertname, window {window.seconds
  // 60} min)")
print(f"reduction            : {100 * (1 - grouped / len(events)):.0f}%")
EOF
{ for i in $(seq 0 11); do printf '2026-10-09T10:0%d:%02dZ PodNotReady shop-api-%d\n'
  $((i%5)) $((10+i)) $i; done
  for i in 0 1 2; do printf '2026-10-09T10:02:4%dZ HighLatency shop-api-%d\n' $i $i;
  done
  echo "2026-10-09T11:30:00Z PodNotReady shop-api-1"; } > alerts.txt
python3 dedupe.py alerts.txt
```

Expected:

```
raw alerts           : 16
notifications grouped: 3  (group_by alertname, window 5 min)
reduction            : 81%
```

Run `python3 dedupe.py alerts.txt 1` to see a 1-minute window produce more notifications (7); the window is a trade-off between latency and noise. This is a simplified model of Alertmanager's `group_by` behaviour, not a re-implementation.

3. **Replace cause alerts with symptom alerts.** Save as `shop-api-alerts.yml`:

```yaml
groups:
  - name: shop-api-noisy-before
    rules:
      # BAD as a page: CPU is a cause, not a symptom.
      - alert: HighCPU
        expr: avg by (instance) (rate(node_cpu_seconds_total{mode≠"idle"}[5m])) > 0.8
        labels:
          severity: page

  - name: shop-api-quiet-after
    rules:
      # GOOD page: users are failing. (The full multi-window version lives in the
  error budgets topic.)
      - alert: ShopApiHighErrorRatio
        expr: |
          sum by (job) (rate(http_requests_total{job="shop-api", code=~"5.."}[5m]))
          /
          sum by (job) (rate(http_requests_total{job="shop-api"}[5m])) > 0.05
        for: 5m
        labels:
          severity: page
        annotations:
          summary: "More than 5% of shop-api requests are failing"
          runbook_url: "https://runbooks.example.com/shop-api/error-ratio"
      # GOOD ticket: a slow, predictable risk. Wakes nobody.
      - alert: ShopApiDiskWillFillIn3Days
        expr: predict_linear(node_filesystem_avail_bytes{mountpoint="/var/lib/docker"}
  [6h], 3 * 24 * 3600) < 0
        for: 1h
        labels:
          severity: ticket
        annotations:
          summary: "Docker volume predicted to fill within 3 days"
          runbook_url: "https://runbooks.example.com/shop-api/disk"
      # GOOD ticket: repeated restarts are worth a look in working hours.
      - alert: ShopApiCrashLooping
        expr: increase(kube_pod_container_status_restarts_total{namespace="shop"}[1h])
  > 5
        labels:
          severity: ticket
        annotations:
          summary: "A shop container restarted more than 5 times in an hour"
          runbook_url: "https://runbooks.example.com/shop-api/crashloop"
```

The disk and restart alerts keep the information but remove the 03:00 interruption. Validate with `promtool check rules shop-api-alerts.yml` if you have it (expected `SUCCESS: 4 rules found`, or 5 once you add `ShopApiDown` in step 4).

4. **Add grouping and inhibition to Alertmanager.** Save as `alertmanager.yml`:

```yaml
route:
  receiver: ticket-queue
  group_by: [alertname, job]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - matchers:
        - severity = "page"
      receiver: oncall-pager
      repeat_interval: 1h
inhibit_rules:
  # If the whole service is down, do not also page for its latency or error ratio.
  - source_matchers:
      - alertname = "ShopApiDown"
    target_matchers:
      - severity =~ "page|ticket"
    equal: [job]
receivers:
  - name: oncall-pager
    webhook_configs:
      - url: http://localhost:9099/page
  - name: ticket-queue
    webhook_configs:
      - url: http://localhost:9099/ticket
```

Check with `amtool check-config alertmanager.yml` if available. Add the rule that the inhibition refers to, then re-read the config to see the pieces fit:

```yaml
# append to shop-api-quiet-after in shop-api-alerts.yml
- alert: ShopApiDown
  expr: sum by (job) (up{job="shop-api"}) == 0
  for: 2m
  labels:
    severity: page
  annotations:
    summary: "No shop-api instance is up"
    runbook_url: "https://runbooks.example.com/shop-api/down"
```

An alert never inhibits itself, so `ShopApiDown` still pages. Inhibition only works when the `equal` label (`job` here) is present on both alerts, which is why the rules above aggregate with `sum by (job)`.

5. **Silence for planned work, with an expiry.** With Alertmanager running on `localhost:9093`:

```bash
amtool --alertmanager.url=http://localhost:9093 silence add
  alertname=ShopApiCrashLooping \
  --duration=2h --author="$USER" --comment="Planned node upgrade, ticket SHOP-420"
amtool --alertmanager.url=http://localhost:9093 silence query
```

   Expect a silence ID and the comment in the query output. A silence without an expiry and a reason is how real alerts get hidden for months.

6. **Schedule the alert review.** Put a recurring 30-minute event in the calendar, bring the output of `alert_review.py` on the real export, and record decisions in a table (alert, action, owner). For each DELETE or TUNE, open a ticket and close it within the sprint.

7 Real-world scenario

Illustrative, not a documented incident. The `shop-api` team's pager produces 140 pages a month; the on-call dashboard shows an actionable ratio of 22%. Leading offenders are `HighCPU`, `PodRestarted` and `DiskUsageHigh`. After a Friday-night outage the team realises the fast-burn alert did fire, but the engineer on call had been woken by CPU alerts 11 times that week and silenced the pager app for two hours.

The team spends a sprint on noise: it deletes `HighCPU`, converts disk and restarts to ticket alerts, adds grouping and an inhibition rule for `ShopApiDown`, and replaces six threshold alerts with the three burn-rate alerts. A weekly alert review is added to the rotation handoff. A month later: 31 pages, actionable ratio 84%, out-of-hours pages down from 52 to 6. **Lesson:** the answer to fatigue was not a better phone; it was fewer, better alerts, and a recurring habit of checking them.

8 Common mistakes and best practices

Don't

- Page on resource thresholds (CPU, memory, disk percentage) that users cannot feel.
- Send everything to one channel, one severity and one receiver.
- Silence an alert forever instead of deciding whether it should exist.

- Alert on a metric nobody knows how to act on, or without a runbook.
- Copy a vendor's default alert pack into production without reviewing it.

Do

- Page on SLO burn and clear user symptoms; ticket slow-moving risks; dashboard the rest.
- Group, inhibit and set sensible `for:` and `repeat_interval` values.
- Give every alert an owner, severity, summary, dashboard and runbook.
- Measure actionable ratio and out-of-hours pages, and review them on a schedule.
- Treat each non-actionable page as a bug with an owner and a due date.

9 Tasks

- [ ] **T1 — Review the pages** *(≈20 min, beginner)*
  - **Goal:** classify alerts by value.
  - **Steps:**
    1. Create `pages.csv` and `alert_review.py` and run the review.
    2. For each alert, write delete, tune, demote to ticket or keep, with one reason.
    3. Compute how many of the 20 pages disappear if you apply your decisions.
  - **Done when:** you have a decision per alert and a projected reduction you can defend (for example 15 of 20).
- [ ] **T2 — Measure grouping** *(≈20 min, beginner)*
  - **Goal:** see the effect of grouping.
  - **Steps:**
    1. Create `alerts.txt` and run `dedupe.py` with windows of 1, 5 and 15 minutes.
    2. Record raw alerts, notifications and reduction for each.
    3. Say which window you would choose for a page route and why.
  - **Done when:** you have a three-row table and a one-sentence justification mentioning the latency trade-off.
- [ ] **T3 — Rewrite cause alerts as symptoms** *(≈45 min, intermediate)*
  - **Goal:** convert noisy alerts to user-focused ones.
  - **Steps:**
    1. List five cause-based alerts from a real or imaginary setup (CPU, memory, restarts, queue depth, disk).

2. For each, write the user-visible symptom it should map to and the PromQL for that symptom or a ticket-level alternative.
3. Create `shop-api-alerts.yml` with your versions and, if possible, validate it with `promtool check rules`.

- **Done when:** every alert in the file has a severity, summary and runbook_url, and at most one of them pages.

☐ **T4 — Add inhibition and test it** *(≈45 min, intermediate)*

- **Goal:** prove that a root alert suppresses its consequences.
- **Steps:**
  1. Run Alertmanager with the config from section 6 and a tiny webhook receiver (`python3 -m http.server 9099` is enough to see connection attempts, or any request-logging script).
  2. Post a `ShopApiDown` alert and a `ShopApiHighErrorRatio` alert with the same `job` label to `/api/v2/alerts`.
  3. Check the Alertmanager UI to see the second alert shown as inhibited.
- **Done when:** the UI lists `ShopApiHighErrorRatio` as inhibited and only one notification is delivered.

☐ **T5 — Break and fix a flapping alert** *(≈45 min, advanced)*

- **Goal:** understand `for:` and thresholds.
- **Steps:**
  1. Write a unit test with `promtool test rules` where the error ratio oscillates above and below the threshold every minute and observe the alert states.
  2. Add `for: 5m` and re-test so that it no longer fires on the oscillation.
  3. Add a second test with a sustained 10-minute breach and confirm it still fires.
- **Done when:** the flapping test passes with no alert, the sustained test passes with the alert, and you can explain the delay you introduced.

☐ **T6 — Prove it: run an alert review** *(≈60 min, advanced)*

- **Goal:** lead a review without the guide.
- **Steps:**
  1. With a teammate, take a week of real or invented pages (at least 30 rows) and run the analysis.
  2. Facilitate a 30-minute review that ends with a decision, owner and date for each alert.
  3. Define the three numbers you will track next month to prove it worked.

- **Done when:** you have a written decision table and targets for actionable ratio, pages per week and out-of-hours pages.

10 Self-check questions

1. Why is "CPU above 80%" usually a bad paging alert?
2. List four properties a paging alert must have.
3. What is the difference between grouping, inhibition and silencing?
4. What does the `for:` clause in a Prometheus alerting rule do?
5. What would you measure to prove an alert-reduction effort worked?

ANSWERS

1. CPU is a cause, not a user-visible symptom; high CPU is often normal and harmless, so the alert pages people without anything to do. Page on error rate, latency or SLO burn instead and use CPU for diagnosis.
2. Urgent, actionable, real (low false-positive rate), not a duplicate, owned, and linked to a runbook with a user-impact summary.
3. Grouping merges related alerts into one notification; inhibition suppresses dependent alerts while a more fundamental alert is firing; silencing temporarily mutes matching alerts for a fixed time with a reason.
4. It requires the expression to remain true for the stated duration before the alert fires, which filters short blips and flapping.
5. Pages per week, out-of-hours pages, actionable ratio, and time to acknowledge, compared before and after; ideally also that no incidents were missed.

11 Further reading

- *Site Reliability Engineering* (Google) — "Monitoring Distributed Systems".
- *The Site Reliability Workbook* (Google) — "Alerting on SLOs".
- Rob Ewaschuk, "My Philosophy on Alerting" (a Google SRE essay widely shared online).
- Prometheus documentation (prometheus.io/docs) — "Alerting" and "Alertmanager" (grouping, inhibition, silences) and best practices on alerting.
- See also: Error Budgets §14.4, On-call Practices §14.6 and What to Monitor §10.11.

LAB NOTES What I ran, what I saw, what it means

14.9 Capacity Planning

Topic 9 of 10 | ~5 hours | Difficulty: Intermediate

1 What it is

Capacity planning is the practice of making sure a service always has enough resources (CPU, memory, disk, network, database connections, instances, money) to serve the load users will put on it, with margin to spare, without paying for far more than it needs. It answers three questions: how much can we handle *today* (measured by **load testing**), how much do we use at peak and how much slack remains (**headroom**), and when will we run out (**forecasting**). A fourth idea, **saturation**, explains why the answer is never "we can run at 100%": as a resource approaches full utilisation, queues build, latency explodes and errors start long before the hardware is technically maxed out.

Google's SRE books treat capacity as part of reliability: *Site Reliability Engineering* discusses capacity planning in its chapters on software engineering in SRE and on handling overload and cascading failures, and the **Four Golden Signals** (latency, traffic, errors, saturation) put saturation next to the signals your SLIs §14.1 are built from.

2 Why it matters in DevOps

- **Outages from success.** A marketing campaign, a launch or a seasonal peak can double traffic. If the system was only ever tested at today's load, the first discovery of its limit happens in production, in public.
- **Error budget.** Overload burns error budget §14.4 fast: latency SLOs fail first, then availability.
- **Cost.** Over-provisioning "to be safe" wastes real money every month. Planning replaces fear with numbers (see also autoscaling for cost §20.4).
- **Lead times.** Some capacity takes weeks to arrive: database instance class changes, reserved capacity, hardware, quota increases. You must forecast further ahead than the lead time.
- **Resilience maths.** Losing one node or one zone shifts its load to the survivors; without headroom a small failure cascades into a large one.

3 Real-world system example

Retail peaks. Large e-commerce platforms plan for predictable peak events. Shopify has written publicly about load-testing its platform ahead of Black Friday and Cyber Monday with in-house tooling, and Amazon publishes guidance in the AWS Well-Architected Framework (Reliability pillar) to test that scaling works and to design for demand changes. The consistent pattern: rehearse the peak at or above expected traffic well before it happens, and fix whatever breaks.

Netflix wrote publicly about predictive autoscaling (an internal system named Scryer in its technology blog) that uses historical daily patterns to scale before the evening peak rather than reacting after latency rises; the idea of combining reactive and predictive scaling is now offered by cloud providers as a managed feature.

Google describes in the SRE book how overload is handled: load shedding, graceful degradation and per-client limits protect the system when demand exceeds capacity, while capacity planning aims to make that the exception.

Kubernetes gives the primitives: resource requests and limits (requests and limits §8.14), the Horizontal Pod Autoscaler (HPA §8.16), and cluster autoscaling. Planning decides the numbers that go into them.

4 Diagram

The capacity planning loop:

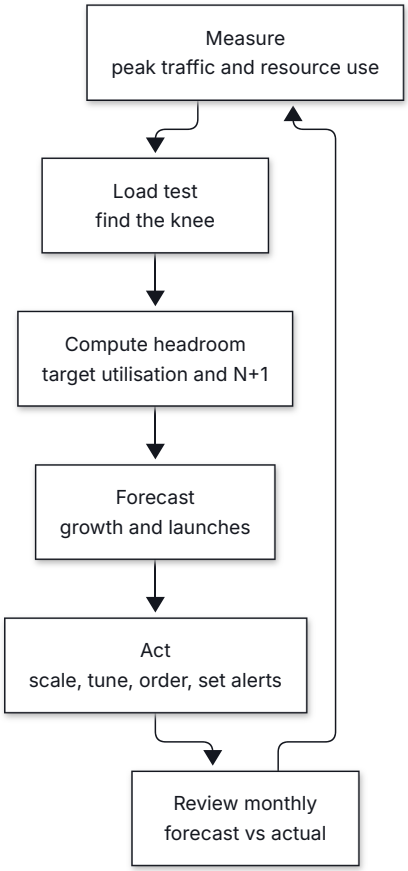


Figure 14.9-1

Why saturation hurts before you reach 100 percent (mean response time grows as service time divided by one minus utilisation):

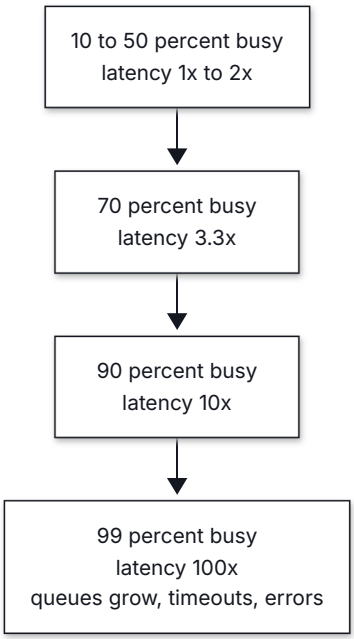


Figure 14.9-2

5 How it works

Load testing

A **load test** applies controlled, increasing traffic to a copy of the system and records throughput, latency percentiles and error rate. The goal is to find the **knee**: the load at which latency or errors stop being acceptable. Types:

Test	Question it answers	Shape
Load	Does it meet the SLO at expected peak?	ramp to expected peak and hold
Stress	Where does it break and how does it fail?	ramp past peak until errors rise
Soak	Does it leak memory, connections or disk over hours?	steady load for hours
Spike	Can it absorb a sudden burst and recover?	sudden jump up and back
Capacity (step)	What is the max sustainable throughput?	stepped increases, one level at a time

Rules for honest results: use production-like data volume and configuration, keep dependencies realistic (a mocked database that answers in 1 ms proves nothing), drive the test from a separate machine, use **open-loop** load (requests start on schedule even if earlier ones are slow, like real users) and watch percentiles rather than averages. Do not load-test production dependencies you do not own, and tell people first.

Headroom

Headroom is the unused capacity you deliberately keep. Define it from the knee of your load test and the failures you must survive:

```
safe capacity per instance  = knee throughput x target utilisation   (for example 80 rps x
  0.6 = 48 rps)
instances needed            = ceil(forecast peak / safe capacity per instance)
fleet size                  = instances needed + redundancy           (N+1, or one zone's
  worth)
```

Typical choices: plan to run at **50-70%** of the tested knee at peak, keep **N+1** so the loss of one instance (or N+2 / one availability zone for critical tiers) still leaves you under the knee, and include autoscaling lead time: if new capacity takes 5 minutes to become ready, headroom must cover the traffic that arrives in those 5 minutes.

Saturation

Saturation is the degree to which a resource has more work than it can process promptly, shown by queues. Measure it per resource using the **USE method** (Utilization, Saturation, Errors) and the golden signals:

Resource	Utilization	Saturation signal
CPU	`1 - idle` fraction	run queue length, load average per core, container CPU throttling
Memory	used vs available	swapping, page faults, OOM kills
Disk	busy time, space used	I/O wait, queue depth
Network	bandwidth used	drops, retransmits
Connection pool	connections in use / max	requests waiting for a connection
Thread or worker pool	busy workers	queue length, rejected tasks

Saturation is usually a better alert and a better capacity signal than raw utilisation, because it says the resource is already costing users time. See what to monitor §10.11.

Forecasting

A **forecast** projects demand forward so you can order capacity before you need it. Start simple and honest:

- **Organic growth:** fit a trend to weekly peak traffic (linear or compound) and extrapolate. Re-fit regularly; do not trust a forecast longer than a few quarters.
- **Seasonality:** daily, weekly and yearly cycles. Compare peak to peak, not average to average.
- **Known events:** launches, campaigns and migrations get explicit multipliers from the business, added on top of the trend.
- **Resource-per-request changes:** a new feature may double CPU per request; track cost per request as well as request volume.
- **Lead time:** order or provision when the forecast crosses (safe limit minus lead time).

Always state the **forecast, the assumptions and the date you will check it**, then compare actual with forecast each month. The error in the forecast teaches you more than the forecast.

6 Step-by-step guide

Prerequisites: Python 3.10 or newer (for `statistics.linear_regression`). All free and local; the toy service runs on `127.0.0.1:8081`. The numbers are similar to, not identical to, what you will see: they depend on your machine.

1. **Start a toy service with a known capacity.** It has 4 worker slots and each request takes 50 ms, so its theoretical capacity is 4 / 0.05 = 80 requests per second; beyond a small queue it sheds load with `503`.

```bash
mkdir -p ~/sre-lab/capacity && cd ~/sre-lab/capacity
cat > toyapi.py <<'EOF'
#!/usr/bin/env python3
"""A toy 'shop-api' with a known capacity: WORKERS slots x 1/SERVICE_S requests per
  second."""
import threading
import time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

WORKERS = 4
SERVICE_S = 0.05          # 50 ms of 'work' per request  →  capacity = 4 / 0.05 = 80
  req/s
MAX_QUEUE = 20            # more than this many waiting → 503 (load shedding)

slots = threading.Semaphore(WORKERS)
waiting = 0
```

```
lock = threading.Lock()

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        global waiting
        with lock:
            if waiting ⩾ MAX_QUEUE:
                self.send_response(503); self.end_headers(); return
            waiting += 1
        with slots:
            with lock:
                waiting -= 1
            time.sleep(SERVICE_S)
        self.send_response(200); self.end_headers(); self.wfile.write(b"ok")
    def log_message(self, *a):
        pass

ThreadingHTTPServer(("127.0.0.1", 8081), H).serve_forever()
EOF
python3 toyapi.py &
sleep 1 && curl -s -o /dev/null -w "%{http_code}\n" localhost:8081/
```

Expected: `200`.

2. **Write an open-loop step load tester** (standard library only, so it works anywhere; for real systems prefer `k6`, `hey`, `wrk` or `vegeta`).

```bash
cat > loadtest.py <<'EOF'
#!/usr/bin/env python3
"""Step load test: python3 loadtest.py URL RATE1 RATE2 ... [--secs N]

Open-loop: requests are started on schedule whether or not earlier ones finished,
which is how real users behave (they do not wait for the server to catch up).
"""
import http.client
import sys
import time
from concurrent.futures import ThreadPoolExecutor
from urllib.parse import urlparse

args = sys.argv[1:]
secs = 5
if "--secs" in args:
    i = args.index("--secs")
    secs = float(args[i + 1])
    del args[i:i + 2]
if len(args) < 2:
    sys.exit(__doc__)
u = urlparse(args[0])
rates = [float(r) for r in args[1:]]
```

```
def hit():
    t0 = time.perf_counter()
    try:
        c = http.client.HTTPConnection(u.hostname, u.port, timeout=10)
        c.request("GET", u.path or "/")
        code = c.getresponse().status
        c.close()
    except Exception:
        code = 0
    return code, time.perf_counter() - t0

def pct(xs, p):
    xs = sorted(xs)
    return xs[min(len(xs) - 1, int(p * len(xs)))]

print(f"{'target rps':>10} {'achieved':>9} {'errors':>7} {'p50 ms':>7} {'p99 ms':>7}")
with ThreadPoolExecutor(max_workers=200) as pool:
    for rate in rates:
        futs = []
        start = time.perf_counter()
        n = int(rate * secs)
        for k in range(n):
            wait = start + k / rate - time.perf_counter()
            if wait > 0:
                time.sleep(wait)
            futs.append(pool.submit(hit))
        res = [f.result() for f in futs]
        dur = time.perf_counter() - start
        ok = [t for c, t in res if c == 200]
        err = sum(1 for c, _ in res if c != 200)
        print(f"{rate:>10.0f} {len(ok) / dur:>9.1f} {100 * err / len(res):>6.1f}% "
              f"{pct([t for _, t in res], 0.5) * 1000:>7.0f} {pct([t for _, t in res],
  0.99) * 1000:>7.0f}")
EOF
python3 loadtest.py http://127.0.0.1:8081/ 20 40 60 80 100 140 --secs 5
```

Expected output, similar to:

```
target rps  achieved  errors  p50 ms  p99 ms
        20      20.0    0.0%      52      64
        40      39.8    0.0%      51      60
        60      59.6    0.0%      51      56
        80      78.9    0.0%      72      95
       100      78.9   16.8%     292     313
       140      65.4   41.1%     279     629
```

Read it like an SRE: up to 60 rps latency is flat at about 50 ms; at 80 rps (the theoretical capacity) the median starts rising as queues form; at 100 rps throughput is stuck near 79 rps, latency is six times higher and 17% of requests are shed. The **knee is around 70-80**

rps. Stop the service when done: `kill %1`.

3. **Turn the knee into a headroom plan.** With a knee of 80 rps per instance, a peak of 240 rps, a 60% target utilisation, and 8% monthly growth:

```bash
cat > headroom.py <<'EOF'
#!/usr/bin/env python3
"""Headroom calculator.

usage: python3 headroom.py PEAK_RPS RPS_PER_INSTANCE [TARGET_UTIL]
  [GROWTH_PCT_PER_MONTH] [MONTHS]
  e.g. python3 headroom.py 240 80 0.6 8 6
TARGET_UTIL is the busiest you want instances to run at peak (0.6 = 60 %).
Adds N+1: the fleet must still carry the peak with one instance lost.
"""
import math
import sys

if len(sys.argv) < 3:
    sys.exit(__doc__)
peak = float(sys.argv[1])
per = float(sys.argv[2])
util = float(sys.argv[3]) if len(sys.argv) > 3 else 0.6
growth = float(sys.argv[4]) / 100 if len(sys.argv) > 4 else 0.0
months = int(sys.argv[5]) if len(sys.argv) > 5 else 6

print(f"{'month':>5} {'peak rps':>9} {'instances':>9} {'with N+1':>8} {'util at
  peak':>13} {'util if 1 lost':>15}")
for m in range(months + 1):
    demand = peak * (1 + growth) ** m
    need = math.ceil(demand / (per * util))
    fleet = need + 1
    print(f"{m:>5} {demand:>9.0f} {need:>9} {fleet:>8} "
          f"{100 * demand / (fleet * per):>12.0f}% {100 * demand / ((fleet - 1) *
  per):>14.0f}%")
EOF
python3 headroom.py 240 80 0.6 8 6
```

Expected:

```
month  peak rps instances with N+1  util at peak  util if 1 lost
    0       240         5        6           50%             60%
    1       259         6        7           46%             54%
    2       280         6        7           50%             58%
    3       302         7        8           47%             54%
    4       327         7        8           51%             58%
    5       353         8        9           49%             55%
    6       381         8        9           53%             60%
```

Even with one instance lost, utilisation stays at or below the 60% target. Without the extra instance, losing one of five would push the remaining four to 75% (240 / 320), above your target.

4. **See why you cannot run hot.**

```bash
cat > saturation.py <<'EOF'
#!/usr/bin/env python3
"""Why latency explodes near saturation (M/M/1 queue model).

usage: python3 saturation.py [SERVICE_TIME_MS]
Mean response time = S / (1 - utilization).  A model, not a measurement, but it
shows the shape every real service follows.
"""
import sys

s = float(sys.argv[1]) if len(sys.argv) > 1 else 50.0
print(f"service time {s:g} ms")
print(f"{'utilization':>11} {'mean response ms':>17} {'x service time':>15}")
for u in (0.10, 0.30, 0.50, 0.60, 0.70, 0.80, 0.90, 0.95, 0.99):
    r = s / (1 - u)
    print(f"{u * 100:>10.0f}% {r:>17.0f} {r / s:>14.1f}x")
EOF
python3 saturation.py
```

Expected: `50%` busy gives `100` ms (2.0x), `90%` gives `500` ms (10.0x), `99%` gives `5000` ms (100.0x). Real services differ in detail, but the hockey-stick shape is universal.

5. **Forecast when you run out.** Twelve weeks of weekly peak traffic and a tested ceiling of 400 rps for the whole fleet:

```bash
cat > forecast.py <<'EOF'
#!/usr/bin/env python3
"""Linear capacity forecast.

usage: python3 forecast.py peaks.csv CAPACITY_RPS [SAFE_FRACTION]
peaks.csv: one 'week,peak_rps' row per line (header allowed).
Fits a straight line, projects forward and reports when demand crosses the
safe limit (CAPACITY_RPS x SAFE_FRACTION, default 0.7).
"""
import csv
import sys
from statistics import linear_regression

if len(sys.argv) < 3:
    sys.exit(__doc__)
xs, ys = [], []
for row in csv.reader(open(sys.argv[1])):
```

```
    try:
        xs.append(float(row[0]))
        ys.append(float(row[1]))
    except ValueError:
        continue                        # header line
cap = float(sys.argv[2])
safe = cap * (float(sys.argv[3]) if len(sys.argv) > 3 else 0.7)

slope, intercept = linear_regression(xs, ys)
print(f"trend: peak rps = {intercept:.1f} + {slope:.2f} x week   ({len(xs)} points)")
last = xs[-1]
print(f"latest fitted peak : {intercept + slope * last:.0f} rps  (safe limit
  {safe:.0f}, hard limit {cap:.0f})")
if slope <= 0:
    print("no growth trend: no exhaustion date")
else:
    wk = (safe - intercept) / slope
    hard = (cap - intercept) / slope
    print(f"safe limit crossed : week {wk:.1f}  ({max(wk - last, 0):.1f} weeks from
  now)")
    print(f"hard limit crossed : week {hard:.1f}  ({max(hard - last, 0):.1f} weeks
  from now)")
EOF
printf
  'week,peak_rps\n1,100\n2,104\n3,111\n4,113\n5,121\n6,124\n7,131\n8,138\n9,141\n10,14
  9\n11,153\n12,160\n' > peaks.csv
python3 forecast.py peaks.csv 400
```

Expected:

```
trend: peak rps = 93.2 + 5.47 x week   (12 points)
latest fitted peak : 159 rps  (safe limit 280, hard limit 400)
safe limit crossed : week 34.1  (22.1 weeks from now)
hard limit crossed : week 56.1  (44.1 weeks from now)
```

If ordering new capacity takes 8 weeks, you must start by week 26. Add a known event (for example, a 2x launch spike in week 20) by hand and see how much earlier the date moves.

6. **Alert on saturation and on the runway, not just utilisation.** In Prometheus (rule syntax; adjust metric names to your exporters):

```yaml
groups:
  - name: shop-api-capacity
    rules:
      # symptom of saturation: requests are waiting for a database connection
      - alert: ShopApiDbPoolSaturated
        expr: avg_over_time(db_pool_waiting_requests{job="shop-api"}[5m]) > 5
```

```
        for: 10m
        labels:
          severity: ticket
        annotations:
          summary: "shop-api requests are queueing for database connections"
          runbook_url: "https://runbooks.example.com/shop-api/db-pool"
      # runway: the disk will be full within a week
      - alert: ShopApiDiskRunway7d
        expr:
  predict_linear(node_filesystem_avail_bytes{mountpoint="/var/lib/postgresql"}[24h], 7
  * 24 * 3600) < 0
        for: 2h
        labels:
          severity: ticket
        annotations:
          summary: "Postgres volume predicted to fill within 7 days"
          runbook_url: "https://runbooks.example.com/shop-api/disk"
```

Both are tickets: they give days of warning, not a 3 a.m. page (see alert fatigue reduction §14.8).

7. **Put the plan in version control.** One page per service: knee (with the date and test setup), target utilisation, redundancy rule, forecast with assumptions, lead times, and the next review date.

7 Real-world scenario

Illustrative, not a documented incident. The `shop-api` team launches a "flash sale" email at 12:00. Normal peak is 240 rps; the email lifts traffic to 520 rps within three minutes. The service has five instances with a 80 rps knee each (400 rps total) and an autoscaler that takes about six minutes to add capacity. At 12:03 median latency climbs from 50 ms to 300 ms, then database connection waits appear, retries amplify the load, and the fast-burn alert pages the on-call at 12:08. The team scales manually to 12 instances and the service recovers at 12:15; a day's worth of error budget is gone.

The postmortem finds three planning gaps: nobody told engineering about the email, the load test had only gone to 300 rps, and scale-out lead time exceeded the ramp. Actions: a launch calendar that feeds the forecast, a quarterly step test to 2x expected peak, pre-scaling before announced events, a request rate limit with graceful degradation for non-checkout routes, and a saturation ticket alert on the database pool. **Lesson:** capacity failures are usually communication and rehearsal failures before they are technical ones.

8 Common mistakes and best practices

Don't

- Plan for the average. Plan for the peak, including its shape and speed.
- Run a load test against a tiny dataset or with mocked dependencies and trust the result.
- Aim for 100% utilisation. Queueing makes latency explode long before that.
- Forecast far ahead from a few weeks of data, or forget known events like launches.
- Load-test production dependencies or third-party APIs without permission.

Do

- Find the knee with a step test, then run at 50-70% of it at peak, with N+1 or better.
- Measure saturation (queues, waits, throttling) as well as utilisation.
- Account for lead times: autoscaler warm-up, instance class changes, quota requests, purchases.
- Re-run the load test after major changes and at least quarterly; store results with the date.
- Compare forecast to actuals each month and publish both the forecast and its assumptions.
- Keep a release valve: rate limiting, load shedding and degraded modes (see rate limiting §12.13).

9 Tasks

- [ ] **T1 — Find the knee** *(≈30 min, beginner)*
  - **Goal:** locate the capacity of a service empirically.
  - **Steps:**
    1. Run the toy service and `loadtest.py` as in section 6.
    2. Identify the first step where p50 rises noticeably and the first step with errors.
    3. State your knee in requests per second.
  - **Done when:** you have a table like the one above and a written knee (around 70-80 rps) with the evidence.
- [ ] **T2 — Change the capacity** *(≈30 min, beginner)*
  - **Goal:** predict, then confirm, the effect of a change.
  - **Steps:**

1. Predict the new capacity if `WORKERS` becomes 8 and then if `SERVICE_S` becomes 0.025.
2. Restart the service with each change and rerun the step test up to 200 rps.
3. Compare prediction with result.

- **Done when:** both predictions (160 rps each) are within about 25% of the measured knee (the Python client itself becomes a limit at high rates, so treat the result as approximate).

☐ **T3 — Size a fleet with N+1** *(≈30 min, intermediate)*

- **Goal:** convert a knee and a forecast into an instance count.
- **Steps:**
  1. Run `python3 headroom.py 200 80 0.6 5 12` (peak 200 rps, 80 rps per instance, 60% target, 5% monthly growth, 12 months).
  2. Find the month in which the instances needed (before adding N+1) first reach 7, then work out by hand how many instances you would need to survive losing one of three equal zones.
  3. Record the assumptions.
- **Done when:** you report month 8 for the seventh instance (needed 7, fleet 8 with N+1) and can explain the "util if 1 lost" column.

☐ **T4 — Forecast with an event** *(≈45 min, intermediate)*

- **Goal:** combine trend and a known spike.
- **Steps:**
  1. Generate a `peaks.csv` for your own invented service (at least 10 weeks) and run `forecast.py`.
  2. Add a planned launch that doubles traffic in week 20 and decide when you must order capacity given a 6-week lead time.
  3. Write the date and the assumptions in a half-page plan.
- **Done when:** the plan states the order date, which is earlier than the pure-trend date.

☐ **T5 — Break it and watch saturation** *(≈45 min, advanced)*

- **Goal:** observe queueing and load shedding live.
- **Steps:**
  1. Run the toy service and, in a second terminal, a step test that goes to 200 rps.
  2. Capture the latency and error columns, then reduce `MAX_QUEUE` to 5 and repeat.
  3. Explain what the smaller queue changes about latency and errors, and when each setting is preferable.

- **Done when:** you have two result tables and can explain the trade-off between longer queues (higher latency, fewer errors) and shorter queues (lower latency, more rejections).

☐ **T6 — Prove it: capacity plan for shop-api** *(≈90 min, advanced)*

- **Goal:** produce a one-page capacity plan without the guide.
- **Steps:**
  1. Load-test any service you can run locally (the toy service, or `shop-api` in Docker) with a tool of your choice and record the knee.
  2. Choose target utilisation and redundancy, compute fleet sizes for six months of growth, and define two saturation alerts.
  3. Present it, including what would make the plan wrong.
- **Done when:** the plan has a measured knee, a justified target utilisation, a forecast with stated assumptions, a review date and two alert definitions.

10 Self-check questions

1. Why shouldn't you run a service at 100% of its tested capacity?
2. What is the difference between utilisation and saturation?
3. What does N+1 mean for capacity?
4. Name three kinds of load test and what each tests.
5. Why do you forecast peak rather than average traffic?

ANSWERS

1. Queueing delay grows sharply as utilisation approaches 100%, so latency and errors rise before the resource is "full"; you also need slack for failures, deploys and bursts.
2. Utilisation is how busy a resource is; saturation is how much work is waiting because the resource cannot keep up (queues, throttling, waits). Saturation means users are already paying.
3. The fleet is sized so that if one instance (or one zone) fails, the rest can still carry the peak under the target utilisation.
4. For example: load (meets SLO at expected peak), stress (where and how it breaks), soak (leaks over long periods), spike (absorbs and recovers from bursts), step/capacity (maximum sustainable throughput).
5. Failures happen at the peak, not on average; an average hides the short bursts that saturate the system and burn error budget.

11 Further reading

- *Site Reliability Engineering* (Google) — chapters on "Handling Overload", "Addressing Cascading Failures" and "Software Engineering in SRE" (capacity planning).
- *The Site Reliability Workbook* (Google) — chapters on SLO engineering and on non-abstract large system design.
- AWS Well-Architected Framework — Reliability pillar (testing scaling and managing demand).
- Brendan Gregg, "The USE Method" (utilisation, saturation, errors) and *Systems Performance*.
- Documentation for `k6`, `vegeta`, `wrk` or `hey` for real load generation.
- See also: Horizontal and Vertical Scaling §13.10, Auto Scaling §6.2 and Error Budgets §14.4.

14.10 Reliability Engineering

Topic 10 of 10 | ~5 hours | Difficulty: Advanced

1 What it is

Reliability engineering is the proactive half of SRE: instead of only responding to failures, you remove the work that keeps you busy and you build, then test, the system's ability to survive failure. This topic joins four connected practices:

- **Toil reduction and automation.** *Toil* is operational work that is manual, repetitive, automatable, tactical and has no lasting value, and that grows in proportion to the size of the service. Automating it frees engineers to improve the system.
- **Resilience patterns.** Design techniques such as timeouts, retries with backoff, circuit breakers, bulkheads, load shedding and graceful degradation that let a service keep working when a dependency or a component does not.
- **Chaos engineering.** The discipline of experimenting on a system on purpose, with a hypothesis and a limited blast radius, to find weaknesses before they find you.
- **Game days.** Scheduled, rehearsed exercises where a team injects failure (or replays an incident) and practises detection, response and recovery together.

Google's *Site Reliability Engineering* book defines toil and the goal of capping operational work in its chapter "Eliminating Toil", and *The Site Reliability Workbook* has a chapter of the same name; the *Principles of Chaos Engineering* manifesto defines the experiment loop used below.

2 Why it matters in DevOps

- **Toil does not scale.** If every new customer means more manual steps, the team's time disappears into operations and reliability work never happens. Capping toil protects engineering time.
- **Failure is normal.** Disks fill, nodes disappear, dependencies slow down. A system that has never been tested against these is only assumed to be resilient.

- **Resilience patterns can backfire.** Retries without backoff turn a small outage into a self-inflicted overload; poorly configured timeouts hold resources open. Patterns must be understood and tested together.
- **Practice shortens incidents.** Teams that rehearse incident response §14.5 and on-call §14.6 find the gaps in runbooks, alerts and permissions in a calm exercise instead of at 3 a.m.
- **Budget use.** Healthy error budget §14.4 is permission to run experiments; exhausted budget is a reason to prioritise this work.

3 Real-world system example

Google describes in the SRE book that SRE teams should spend no more than about half their time on operational work, keeping the rest for engineering projects that reduce future toil; it also runs company-wide disaster-recovery exercises (known publicly as DiRT, Disaster Recovery Testing) that deliberately take services away to test whether teams and systems cope.

Netflix popularised chaos engineering: Chaos Monkey, which randomly terminates instances in production so that services must tolerate instance loss, was created during its move to AWS and later joined by other tools in a "Simian Army"; Netflix engineers co-authored the *Principles of Chaos Engineering*. The open-source Chaos Monkey is now part of the Spinnaker project.

Amazon is associated with "GameDay" exercises (credited publicly to Jesse Robbins) in which failures are simulated against production-like systems. **AWS** now offers **Fault Injection Service (FIS)** as a managed way to run experiments such as stopping instances or adding latency. In the Kubernetes world, **Chaos Mesh** and **LitmusChaos** (both CNCF projects) provide declarative fault injection, and **Gremlin** is a commercial platform.

Resilience libraries and proxies such as Resilience4j (JVM), Polly (.NET), Envoy and Istio (service mesh §12.9) package timeouts, retries and circuit breaking so each team does not reinvent them.

4 Diagram

The chaos experiment loop:

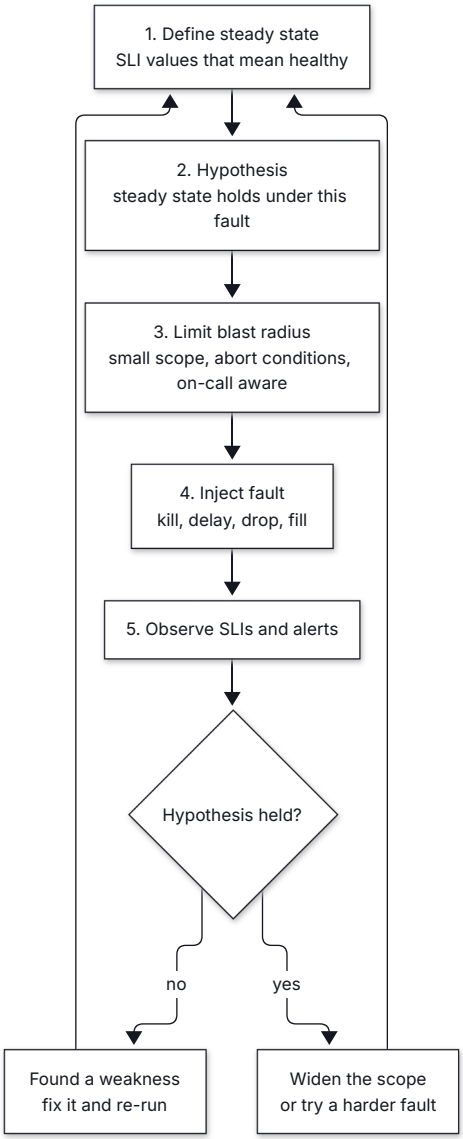


Figure 14.10-1

The circuit breaker state machine:

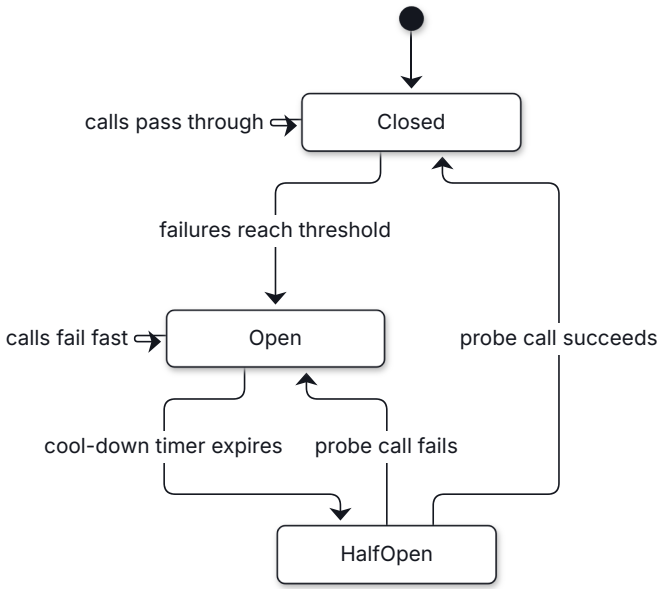


Figure 14.10-2

5 How it works

Toil

The SRE book characterises toil as work that is **manual**, **repetitive**, **automatable**, **tactical** (interrupt-driven rather than strategic), has **no enduring value**, and **scales linearly** with the size of the service. Not all operational work is toil: designing a capacity plan or writing a postmortem is overhead but not toil, because it has lasting value. Process for tackling it:

1. **Inventory.** Everyone logs recurring manual tasks for two to four weeks: what, how often, how long.
2. **Rank by payback.** Hours saved per month versus hours to automate. Start with short payback periods.
3. **Automate or eliminate.** Often the best fix is to remove the need (a self-healing component, a safer default), not to script the task.
4. **Measure.** Track toil as a percentage of team time and review it quarterly.

Automation levels: documented steps, then a script run by a human, then a script run by a pipeline, then self-service, then a fully automatic system. Each step reduces both effort and error. Automation is software: version it, review it, test it, and give it the same alerting as

production code (see infrastructure automation §0.6).

Resilience patterns

Pattern	What it does	Watch out for
Timeouts	Never wait forever for a dependency; every network call has a deadline	Set below the caller's own deadline so total time is bounded
Retries with exponential backoff and jitter	Retry transient failures with growing, randomised delays	Retries multiply load; cap attempts; retry only idempotent operations (see idempotency §13.14)
Circuit breaker	After repeated failures stop calling the dependency for a while and fail fast	Choose thresholds and cool-down carefully; see circuit breakers §12.14
Bulkhead	Isolate resources (thread pools, connection pools, queues) per dependency so one failure cannot starve everything	Sizing the compartments
Load shedding and rate limiting	Reject excess work early to protect the work you can do	Decide what to drop first (see rate limiting §12.13)
Graceful degradation / fallback	Serve a reduced answer (cached, default, partial) when a dependency fails	A degraded answer must be safe and clearly acceptable
Redundancy and failover	N+1 instances, multiple zones, replicas	Test the failover; untested failover is a hope
Health checks and self-healing	Detect unhealthy instances and replace them	See liveness and readiness probes §8.15
Safe deploys	Canary, progressive rollout, instant rollback	See canary deployments §5.12

The combination matters. A retry policy without a circuit breaker amplifies an outage; a breaker without a fallback just fails faster. The simulation in section 6 shows the numbers.

Chaos engineering

A good experiment follows the loop in the diagram. Practical rules:

- **Start in a staging or test environment**, then graduate to production for services with good observability and rollback, and only when the error budget §14.4 is healthy.
- **Define the steady state in SLI terms** (error ratio, latency percentile), not in "the pods are running".
- **Keep the blast radius small** (one instance, 1% of traffic, one zone) and write **abort conditions** ("stop if the 5-minute error ratio exceeds 2%").
- **Tell people**: the on-call and anyone whose service is in scope. Chaos is not a surprise attack.
- **Fix what you find**, then repeat the experiment as an automated regression check.

Typical faults: kill an instance or pod, add latency or packet loss to a dependency, fail DNS, fill a disk, exhaust a connection pool, expire a certificate, take a zone away, return errors from a downstream.

Game days

A **game day** is a scheduled exercise with a plan: scenario, scope, roles, abort criteria, observers and a debrief. Variants: **tabletop** (talk through a scenario, no systems touched), **wheel of misfortune** (an experienced engineer plays the system and a trainee debugs it, described in Google's SRE material), **fault injection game day** (real faults in staging or controlled production), and **disaster recovery drill** (restore from backup, fail over a region; see failure scenario drills §15.10). The output is the same as a postmortem: findings with owners and due dates.

6 Step-by-step guide

Prerequisites: Linux or macOS shell, Python 3, `curl`. No Docker or cloud account is needed, and everything is free. (Section 3 mentions Chaos Mesh, Litmus and AWS FIS; apply them later on a throwaway `kind` cluster or a sandbox account.) Run the long-lived commands in separate terminals.

1. **Inventory your toil and rank it by payback.**

```bash
mkdir -p ~/sre-lab/reliability && cd ~/sre-lab/reliability
cat > toil.csv <<'EOF'
task,times_per_month,minutes_each,build_hours
Restart stuck queue worker,12,15,6
Rotate TLS certificate by hand,2,60,10
Create customer sandbox environment,20,30,16
Clean full disk on build runner,8,20,3
Copy logs for support requests,15,10,12
Manual database failover test,1,120,40
EOF
cat > toil.py <<'EOF'
#!/usr/bin/env python3
"""Toil inventory: how much time does manual work eat, and what is worth automating?

usage: python3 toil.py toil.csv [ENGINEERS] [HOURS_PER_ENGINEER_MONTH]
CSV columns: task, times_per_month, minutes_each, build_hours (to automate)
"""
import csv
import sys

if len(sys.argv) < 2:
    sys.exit(__doc__)
eng = int(sys.argv[2]) if len(sys.argv) > 2 else 4
```

```
cap = eng * (float(sys.argv[3]) if len(sys.argv) > 3 else 140)

rows = []
with open(sys.argv[1], newline="") as f:
    for r in csv.DictReader(f):
        monthly_h = float(r["times_per_month"]) * float(r["minutes_each"]) / 60
        build = float(r["build_hours"])
        payback = build / monthly_h if monthly_h else float("inf")
        rows.append((r["task"], monthly_h, build, payback))
rows.sort(key=lambda x: x[3])

total = sum(r[1] for r in rows)
print(f"{'task':32} {'hours/month':>11} {'build h':>8} {'payback (months)':>17}")
for task, m, b, p in rows:
    print(f"{task:32} {m:>11.1f} {b:>8.0f} {p:>17.1f}")
print(f"\ntotal toil: {total:.1f} h/month = {100 * total / cap:.1f}% of {eng}
  engineers' time ({cap:.0f} h)")
print("Google's SRE guidance is to keep toil under 50% of an SRE's time; aim far
  lower.")
EOF
python3 toil.py toil.csv 4
```

Expected:

```
task                             hours/month  build h  payback (months)
Clean full disk on build runner          2.7        3               1.1
Create customer sandbox environment     10.0       16               1.6
Restart stuck queue worker               3.0        6               2.0
Copy logs for support requests           2.5       12               4.8
Rotate TLS certificate by hand           2.0       10               5.0
Manual database failover test            2.0       40              20.0

total toil: 22.2 h/month = 4.0% of 4 engineers' time (560 h)
Google's SRE guidance is to keep toil under 50% of an SRE's time; aim far lower.
```

The sandbox environment is the best target: it costs 10 hours a month and repays an automation project in under two months. Real inventories are usually much worse than this sample.

2. **Compare resilience strategies with a simulation.** The script models a dependency that fails 20% of calls at random and is completely down for the middle third of 600 requests.

```bash
cat > resilience_demo.py <<'EOF'
#!/usr/bin/env python3
"""Compare client strategies against a flaky dependency (pure simulation, no network).

usage: python3 resilience_demo.py [SEED]
Scenario: 600 requests. The dependency is healthy for the first 200, completely
```

```
down for the next 200 and healthy again for the last 200. 20 % of healthy calls
also fail at random (a transient error).
"""
import random
import sys

random.seed(int(sys.argv[1]) if len(sys.argv) > 1 else 7)
N, TRANSIENT = 600, 0.20

def dependency_ok(i):
    if 200 ⩽ i < 400:
        return False                       # hard outage
    return random.random() > TRANSIENT     # otherwise 20 % transient failures

class Breaker:
    """closed → open after 5 straight failures → half-open probe after 20
  requests."""
    def __init__(self):
        self.fails, self.opened_at = 0, None
    def allow(self, i):
        return self.opened_at is None or i - self.opened_at ⩾ 20
    def record(self, ok, i):
        if ok:
            self.fails, self.opened_at = 0, None
        else:
            self.fails += 1
            if self.fails ⩾ 5:
                self.opened_at = i

def run(name, retries, use_breaker, fallback):
    calls = good = served = 0
    br = Breaker()
    for i in range(N):
        ok = False
        for attempt in range(retries + 1):
            if use_breaker and not br.allow(i):
                break                      # fail fast: do not call the dependency
            calls += 1
            ok = dependency_ok(i)
            if use_breaker:
                br.record(ok, i)
            if ok:
                break
        if ok:
            good += 1
            served += 1
        elif fallback:
            served += 1                    # degraded answer (cached or default)
    print(f"{name:34} dependency calls {calls:>5} ({calls / N:4.2f}/request)  "
          f"fully correct {100 * good / N:5.1f}%  user got an answer {100 * served /
  N:5.1f}%")

run("1 naive: no retry", 0, False, False)
```

```
run("2 retry x3", 3, False, False)
run("3 retry x3 + circuit breaker", 3, True, False)
run("4 retry x3 + breaker + fallback", 3, True, True)
EOF
python3 resilience_demo.py
```

Output similar to (exact figures vary with the seed):

```
1 naive: no retry                 dependency calls   600 (1.00/request)  fully
  correct  51.2%  user got an answer  51.2%
2 retry x3                        dependency calls  1299 (2.17/request)  fully
  correct  66.7%  user got an answer  66.7%
3 retry x3 + circuit breaker      dependency calls   520 (0.87/request)  fully
  correct  66.5%  user got an answer  66.5%
4 retry x3 + breaker + fallback   dependency calls   519 (0.86/request)  fully
  correct  66.5%  user got an answer 100.0%
```

Read it: retries fix transient errors (51% to 67%) but more than double the calls to a dependency that is already struggling; the breaker removes that amplification during the outage; the fallback is what keeps users served at all. Try seeds 1 to 5.

3. **Build a chaos lab: a web server, a supervisor and a probe.** The supervisor restarts the server if it dies, which is a stand-in for systemd or a Kubernetes Deployment.

```bash
mkdir -p site && echo hello > site/index.html
cat > supervise.sh <<'EOF'
#!/usr/bin/env bash
# Restart the web server whenever it dies (a very small stand-in for systemd or
  Kubernetes).
# usage: ./supervise.sh [restart|norestart]
cd site
while true; do
  python3 -m http.server 8082 --bind 127.0.0.1 >/dev/null 2>&1 &
  echo $! > ../server.pid
  wait $!
  [ "${1:-restart}" = "norestart" ] && break
  sleep 0.5
done
EOF
cat > probe.sh <<'EOF'
#!/usr/bin/env bash
# usage: ./probe.sh SECONDS   - hit the service 5 times a second and report
  availability
end=$((SECONDS + ${1:-20})); ok=0; total=0
while [ $SECONDS -lt $end ]; do
  code=$(curl -m 0.4 -s -o /dev/null -w '%{http_code}' http://127.0.0.1:8082/ || true)
  total=$((total + 1)); [ "$code" = "200" ] && ok=$((ok + 1))
  sleep 0.2
```

```
done
awk -v ok=$ok -v t=$total 'BEGIN { printf "availability during experiment: %.1f%% (%d
  of %d probes)\n", 100*ok/t, ok, t }'
EOF
cat > chaos.sh <<'EOF'
#!/usr/bin/env bash
# usage: ./chaos.sh SECONDS INTERVAL - kill the web server every INTERVAL seconds
end=$((SECONDS + ${1:-20}))
while [ $SECONDS -lt $end ]; do
  sleep "${2:-5}"
  kill "$(cat server.pid)" 2>/dev/null && echo "chaos: killed server pid $(cat
  server.pid) at +${SECONDS}s"
done
EOF
chmod +x supervise.sh probe.sh chaos.sh
```

4. **Run the experiment.** Hypothesis: *with a supervisor, killing the server once every 10 seconds keeps availability at or above 95%.* Blast radius: a local toy server. Abort condition: none needed.

```bash
# terminal 1
./supervise.sh restart
# terminal 2: start the probe first, then immediately start chaos in terminal 3
./probe.sh 20
# terminal 3
./chaos.sh 20 10
```

   Expected, similar to: two `chaos: killed server ...` lines, and `availability during experiment: 97.8% (89 of 91 probes)`. The hypothesis holds: the system recovers from the fault by itself, and the few failed probes are the half-second restart gap.

5. **Break the hypothesis, learn something.** Run again with the interval shortened to 3 seconds (`./chaos.sh 20 3`) and you will see roughly 85% availability: the restart gap now matters because failures come more often. Then stop the supervisor and run `./supervise.sh norestart` instead; after the first kill, availability collapses (about 26% over a 12-second run). That is what "no self-healing" costs, in a number. Press `Ctrl-C` in terminal 1 to finish; run `kill "$(cat server.pid)"` if a stray server remains.

6. **Plan a game day.** Fill in this page and run it as a tabletop first:

```
GAME DAY PLAN  shop-api  <DATE>  duration 90 min
Scenario     : primary database becomes unreachable for 10 minutes during peak
Hypothesis   : checkout error ratio stays below 2% because of read replica + cached
  prices
Scope        : staging environment, then 5% of production if staging passes
Roles        : Game master <name> (injects, can abort), IC <name>, Ops <name>, Scribe
  <name>, Observer <name>
Abort when   : 5m error ratio above 5% in production, or any data-loss signal
Comms        : #gameday channel, on-call notified, status page not used
Success      : detected by alert within 5 min, mitigated within 15 min, runbook was
  enough
Debrief      : blameless review within 2 days; action items with owners
```

7 Real-world scenario

Illustrative, not a documented incident. The `shop-api` team notices that every few weeks an overloaded recommendations service makes checkout slow. The checkout code retries recommendations three times with a 10-second timeout and no breaker. In a game day against staging they inject 2 seconds of latency into the recommendations service. The hypothesis, "checkout p99 stays under 500 ms", fails within a minute: threads pile up waiting, the connection pool exhausts and checkout times out.

Fixes, in order of value: make recommendations optional (a fallback of "popular items" from cache), add a 300 ms timeout, one retry with jitter, and a breaker; put recommendations in their own bulkhead (separate pool). They re-run the experiment and the hypothesis now holds; they add the experiment to the weekly CI-triggered test. Two months later the real recommendations service has an outage; checkout error ratio does not move and nobody is paged. **Lesson:** the chaos experiment converted a vague worry into a specific defect, and the fix was a combination of patterns, each necessary.

8 Common mistakes and best practices

Don't

- Confuse all operational work with toil. Capacity planning and postmortems are engineering, not toil.
- Add retries everywhere with no backoff, no cap and no idempotency. That is how outages grow.
- Run chaos in production without observability, rollback, a hypothesis, abort conditions or telling the on-call.

- Treat a game day as a test the team can fail. It is a rehearsal designed to find gaps.
- Automate a broken process. Fix or remove the process first, then automate.

Do

- Track toil as a percentage of time and spend a fixed share of each sprint removing it.
- Set a timeout on every network call, and make retries bounded, jittered and limited to idempotent operations.
- Combine patterns: timeout, retry, breaker, bulkhead, fallback; test them together.
- Define steady state with SLIs, start small, and widen the blast radius as confidence grows.
- Turn every finding into a ticket with an owner, and re-run the experiment as a regression test.

9 Tasks

- [ ] **T1 — Rank your toil** *(≈30 min, beginner)*
  - **Goal:** quantify manual work and pick the best automation target.
  - **Steps:**
    1. Create `toil.csv` with at least eight real or realistic recurring tasks from your own work.
    2. Run `toil.py` and read the payback column.
    3. Choose the top target and justify it in two sentences.
  - **Done when:** you have a ranked table and a written justification that cites payback time.
- [ ] **T2 — Experiment with resilience parameters** *(≈45 min, intermediate)*
  - **Goal:** see how settings change outcomes.
  - **Steps:**
    1. Run `resilience_demo.py` with five different seeds and note the ranges.
    2. Change retries to 1, then 5, and the breaker threshold to 3 and 10.
    3. Record dependency calls per request and the "user got an answer" percentage for each combination.
  - **Done when:** you can state which parameters raised load on the failing dependency and which protected it.
- [ ] **T3 — Run the kill experiment** *(≈30 min, intermediate)*
  - **Goal:** observe self-healing in a number.

- **Steps:**
  1. Create the three scripts and run the experiment in section 6 with a 10-second kill interval.
  2. Record availability.
  3. Repeat with a 3-second interval and with `norestart`.
- **Done when:** you have three availability numbers (about 98%, about 85%, much lower) and one sentence on what each teaches.

☐ **T4 — Improve the system under test** *(≈60 min, advanced)*

- **Goal:** fix a weakness found by chaos.
- **Steps:**
  1. Modify `supervise.sh` to run two servers on ports 8082 and 8083, and modify `probe.sh` to try 8082 first and fall back to 8083 (a stand-in for a load balancer).
  2. Update `chaos.sh` to kill one server at random.
  3. Re-run the 3-second-interval experiment.
- **Done when:** availability under the 3-second chaos rises clearly above the single-server result (for example above 95%) and you can explain why redundancy beat faster restarts.

☐ **T5 — Write and run a tabletop game day** *(≈60 min, advanced)*

- **Goal:** rehearse a failure scenario with others.
- **Steps:**
  1. Fill in the game day plan from section 6 for a scenario relevant to `shop-api` (database failover, expired certificate, bad deploy).
  2. Gather at least two people, assign the roles, and talk through detection, mitigation and communication.
  3. Write findings as postmortem-style action items (see postmortems §14.7).
- **Done when:** the plan and at least three findings with owners and due dates are written down.

☐ **T6 — Prove it: a resilience review** *(≈60 min, advanced)*

- **Goal:** assess a design without notes.
- **Steps:**
  1. For a service that calls three dependencies (payments, inventory, recommendations), list for each: timeout, retry policy, breaker, fallback and bulkhead.
  2. Say which dependency may degrade gracefully and which must fail the request.

3. Design one chaos experiment per dependency with a hypothesis and abort condition.

- **Done when:** you have a table of nine decisions, three experiments, and can explain why retries can make an outage worse.

10 Self-check questions

1. List four characteristics of toil. Give an example of work that is overhead but not toil.
2. Why can retries make an outage worse, and what two mechanisms limit the damage?
3. What are the steps of a chaos experiment?
4. What is a game day, and how does it differ from a chaos experiment?
5. What is the difference between a circuit breaker and a bulkhead?

ANSWERS

1. Manual, repetitive, automatable, tactical (interrupt-driven), no enduring value, scales with service size. Overhead but not toil: writing a postmortem, planning capacity, interviewing.
2. Each retry adds load to a dependency that is already struggling, multiplying traffic when it can least cope. Limit with backoff plus jitter and a retry cap, and with a circuit breaker that stops calls during a sustained failure (also retry only idempotent operations).
3. Define steady state, form a hypothesis, limit the blast radius and set abort conditions, inject the fault, observe, then either widen scope or fix the weakness and repeat.
4. A game day is a scheduled team exercise (scenario, roles, debrief) that practises people and process as well as systems; a chaos experiment is the technical hypothesis test. A game day often contains chaos experiments.
5. A breaker stops calling a failing dependency for a while (fail fast); a bulkhead isolates resource pools so one failing dependency cannot consume the resources needed by others.

11 Further reading

- *Site Reliability Engineering* (Google) — "Eliminating Toil", "Handling Overload", "Addressing Cascading Failures", "Testing for Reliability".
- *The Site Reliability Workbook* (Google) — "Eliminating Toil" and "Engagement Model" chapters.
- *Principles of Chaos Engineering* (principlesofchaos.org) and *Chaos Engineering* (Rosenthal and Jones, O'Reilly).
- Netflix Technology Blog posts on Chaos Monkey and the Simian Army; Chaos Mesh, LitmusChaos and AWS Fault Injection Service documentation.

- *Release It!* by Michael Nygard — stability patterns including timeouts, circuit breakers and bulkheads.
- See also: Site Reliability Engineering §0.11, Incident Management §14.5 and Failure Scenario Drills §15.10.

LAB NOTES What I ran, what I saw, what it means

LEVEL 14: SRE

Reliability for shop-api: SLOs, burn-rate alerts and a mock incident

Level 14 capstone ~12 hours

1 Goal and outcome

Take the `shop-api` sample app from "we think it is fine" to "we can prove how reliable it is and we know what to do when it is not". On your laptop you will:

- Define **SLIs and SLOs** for the checkout journey (availability and latency), document them, and compare them with a hypothetical customer **SLA**.
- Turn the SLOs into an **error budget policy** and **multi-window, multi-burn-rate alerts** in Prometheus and Alertmanager, with unit tests.
- Route pages to a fake pager, write the **on-call runbook** for the alert, and check every alert against the "actionable" test.
- Run a **mock incident (a game day)**: inject a bad-deploy failure, get paged, run the response with named roles and a scribe log, mitigate, and watch the alerts clear.
- Write a blameless **postmortem** with a timeline, impact in error budget, contributing factors and tracked action items.

You finish with a repeatable lab, an SLO document, a rules file, a runbook, an incident log and a postmortem: the artifacts a real SRE team keeps.

Topics exercised	Where
01 SLIs §14.1	Milestone 1
02 SLOs §14.2, 03 SLAs §14.3	Milestone 2
04 Error budgets §14.4, 08 Alert fatigue reduction §14.8	Milestone 3
06 On-call practices §14.6	Milestone 4
05 Incident management §14.5, 09 Capacity planning §14.9, 10 Reliability engineering §14.10	Milestone 5
07 Postmortems §14.7	Milestone 6

The approach follows the ideas in Google's *Site Reliability Engineering* book and *The Site Reliability Workbook* (the chapters on service level objectives, "Alerting on SLOs", incident response and postmortem culture).

2 Architecture

Everything runs in five containers. The app exposes Prometheus-format metrics and has a

fault-injection endpoint so you can create incidents on demand.

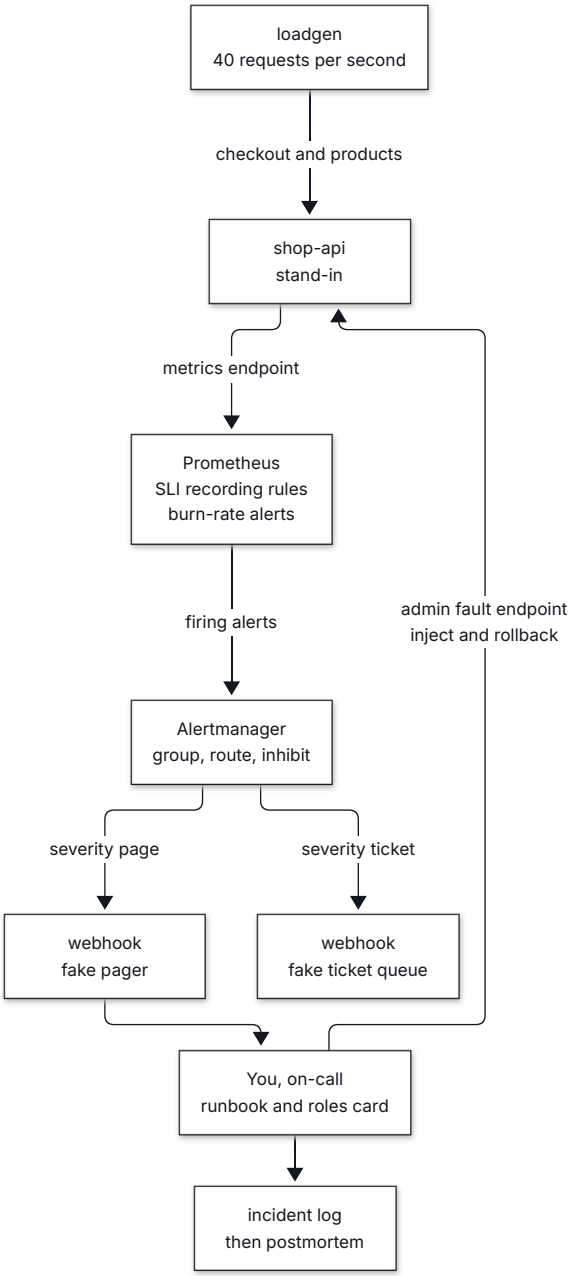


Figure 14-1

The mock incident, end to end (lab time is about 20 times faster than production time, see

Milestone 3):

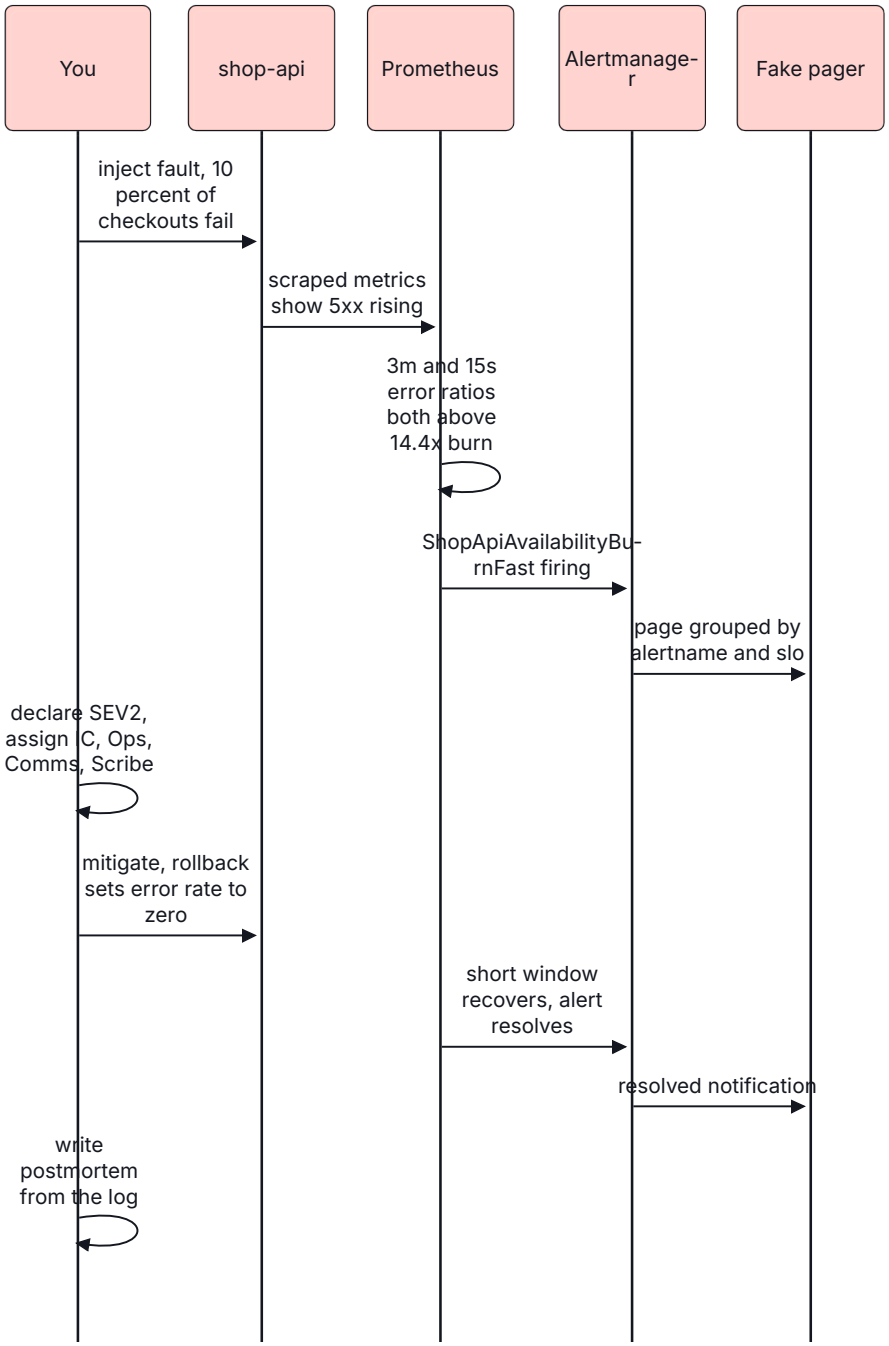


Figure 14-2

3 Prerequisites and cost

- **Cost:** free. Everything is local containers; no cloud account.
- **Machine:** about 2 GB RAM free, 1 GB disk, Docker with Compose v2, Python 3.9+ on the host (for the calculators, no packages needed), `curl`.
- **Time shape:** the live parts take minutes, not days, because the lab rules use windows scaled down by 20 (Milestone 3). The real-window rules are checked with `promtool` and the simulator.
- **Prior reading:** the topic files linked in section 1. You can do the capstone with them open.
- **Verification note:** the Python scripts (app, load generator, rule generator, simulator, webhook) were run in the authoring sandbox, the generated rules were parsed with a PromQL parser and the YAML files were loaded, and the simulator output below is real. The authoring sandbox had no Docker daemon, so the Compose stack, `promtool`, `amtool` and Alertmanager routing were **not run there**: ports, timings and alert states in sample output are "similar to" what you will see, and you should run the `promtool` and `amtool` checks yourself. All URLs, keys and addresses are placeholders.

4 Build it step by step

Milestone 0 — Start the lab

Create a working directory and the five lab files. The app is a standard-library stand-in for `shop-api`: it serves `/checkout` and `/products`, exposes `http_requests_total` and a latency histogram on `/metrics`, returns about 0.05% server errors and 1% client errors as a baseline, and lets you inject faults with `/admin/fault`.

```bash
mkdir l14-lab && cd l14-lab

cat > shop_api.py <<'EOF'
#!/usr/bin/env python3
"""shop-api stand-in: /checkout, /products, /healthz, /metrics (Prometheus text format),
  /admin/fault."""
import random, threading, time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlparse, parse_qs

BUCKETS = (0.05, 0.1, 0.3, 0.5, 1.0, 2.5)
lock = threading.Lock()
reqs = {}                       # (route, code) -> count
hist = {}                       # route -> {"b": [per-bucket counts], "sum": s, "count": n}
fault = {"error_rate": 0.0, "slow_rate": 0.0}   # injected by /admin/fault
```

```
def observe(route, code, secs):
    with lock:
        reqs[(route, code)] = reqs.get((route, code), 0) + 1
        h = hist.setdefault(route, {"b": [0] * len(BUCKETS), "sum": 0.0, "count": 0})
        for i, le in enumerate(BUCKETS):
            if secs ⩽ le:
                h["b"][i] += 1
        h["sum"] += secs
        h["count"] += 1

def render():
    out = ["# TYPE http_requests_total counter"]
    with lock:
        for (route, code), n in sorted(reqs.items()):
            out.append(f'http_requests_total{{route="{route}",code="{code}"}} {n}')
        out.append("# TYPE http_request_duration_seconds histogram")
        for route, h in sorted(hist.items()):
            for le, n in zip(BUCKETS, h["b"]):
                out.append(f'http_request_duration_seconds_bucket{{route="{route}",le="
  {le}"}} {n}')
            out.append(f'http_request_duration_seconds_bucket{{route="{route}",le="+Inf"}}
  {h["count"]}')
            out.append(f'http_request_duration_seconds_sum{{route="{route}"}}
  {h["sum"]:.6f}')
            out.append(f'http_request_duration_seconds_count{{route="{route}"}}
  {h["count"]}')
    return "\n".join(out) + "\n"

class H(BaseHTTPRequestHandler):
    def reply(self, code, body=b"ok", ctype="text/plain"):
        self.send_response(code); self.send_header("Content-Type", ctype);
  self.end_headers()
        try:
            self.wfile.write(body)
        except BrokenPipeError:
            pass

    def do_GET(self):
        u = urlparse(self.path)
        if u.path == "/metrics":
            return self.reply(200, render().encode(), "text/plain; version=0.0.4")
        if u.path == "/healthz":
            return self.reply(200)
        if u.path == "/admin/fault":                        # e.g. /admin/fault?
  error_rate=0.3&slow_rate=0
            q = parse_qs(u.query)
            for k in fault:
                if k in q:
                    fault[k] = float(q[k][0])
            return self.reply(200, f"{fault}\n".encode())
        if u.path in ("/checkout", "/products"):
            start = time.time()
```

```
            slow = random.random() < fault["slow_rate"]
            time.sleep(0.6 if slow else random.choice([0.02] * 95 + [0.15] * 5))
            r = random.random()
            code = 500 if r < fault["error_rate"] + 0.0005 else 400 if r > 0.99 else 200
  # 0.05 % baseline errors, 1 % client errors
            observe(u.path, code, time.time() - start)
            return self.reply(code)
        self.reply(404)

    def log_message(self, *a):
        pass

if __name__ == "__main__":
    ThreadingHTTPServer(("0.0.0.0", 8080), H).serve_forever()
EOF

cat > loadgen.py <<'EOF'
#!/usr/bin/env python3
"""Steady open-loop traffic: python3 loadgen.py [rps] [url]"""
import sys, threading, time, urllib.request

rps = float(sys.argv[1]) if len(sys.argv) > 1 else 20
base = sys.argv[2] if len(sys.argv) > 2 else "http://localhost:8080"

def hit(path):
    try:
        urllib.request.urlopen(base + path, timeout=5).read()
    except Exception:
        pass            # 4xx/5xx raise HTTPError; the server already counted them

n = 0
while True:
    path = "/checkout" if n % 2 == 0 else "/products"
    threading.Thread(target=hit, args=(path,), daemon=True).start()
    n += 1
    time.sleep(1 / rps)
EOF

cat > webhook.py <<'EOF'
#!/usr/bin/env python3
"""Fake pager: prints every Alertmanager webhook it receives."""
import json
from http.server import BaseHTTPRequestHandler, HTTPServer

class H(BaseHTTPRequestHandler):
    def do_POST(self):
        body = json.loads(self.rfile.read(int(self.headers["Content-Length"])))
        for al in body["alerts"]:
            print(f'{self.path:8} {al["status"]:9} {al["labels"]["alertname"]}
  {al["annotations"].get("summary", "")}', flush=True)
        self.send_response(200); self.end_headers()
    def log_message(self, *a):
        pass
```

```
HTTPServer(("0.0.0.0", 9000), H).serve_forever()
EOF
```

Prometheus scrapes every 5 seconds (fast for the lab) and sends alerts to Alertmanager:

BASH

```
cat > prometheus.yml <<'EOF'
global:
  scrape_interval: 5s          # fast for the lab; use 15s to 60s in production
  evaluation_interval: 15s
rule_files:
  - /etc/prometheus/rules.yml
alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]
scrape_configs:
  - job_name: shop-api
    static_configs:
      - targets: ["shop-api:8080"]
EOF

cat > alertmanager.yml <<'EOF'
route:
  receiver: tickets
  group_by: [alertname, slo]
  group_wait: 10s
  group_interval: 1m
  repeat_interval: 4h
  routes:
    - matchers: [severity="page"]
      receiver: pager
    - matchers: [severity="ticket"]
      receiver: tickets
receivers:
  - name: pager
    webhook_configs:
      - url: http://webhook:9000/page
  - name: tickets
    webhook_configs:
      - url: http://webhook:9000/ticket
inhibit_rules:
  # while the fast-burn page is firing, do not also page for the slow burn of the same SLO
  - source_matchers: [alertname="ShopApiAvailabilityBurnFast"]
    target_matchers: [alertname="ShopApiAvailabilityBurnSlow"]
    equal: [slo]
  - source_matchers: [alertname="ShopApiLatencyBurnFast"]
    target_matchers: [alertname="ShopApiLatencyBurnSlow"]
    equal: [slo]
EOF
```

```
cat > compose.yaml <<'EOF'
name: l14
services:
  shop-api:
    image: python:3.12-slim
    command: ["python", "/app/shop_api.py"]
    volumes: ["./shop_api.py:/app/shop_api.py:ro"]
    ports: ["127.0.0.1:8080:8080"]
  loadgen:
    image: python:3.12-slim
    command: ["python", "/app/loadgen.py", "40", "http://shop-api:8080"]
    volumes: ["./loadgen.py:/app/loadgen.py:ro"]
    depends_on: [shop-api]
  webhook:
    image: python:3.12-slim
    command: ["python", "/app/webhook.py"]
    volumes: ["./webhook.py:/app/webhook.py:ro"]
  alertmanager:
    image: prom/alertmanager:v0.27.0
    volumes: ["./alertmanager.yml:/etc/alertmanager/alertmanager.yml:ro"]
    ports: ["127.0.0.1:9093:9093"]
  prometheus:
    image: prom/prometheus:v2.53.0
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.retention.time=35d
      - --web.enable-lifecycle
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./rules-lab.yml:/etc/prometheus/rules.yml:ro
    ports: ["127.0.0.1:9090:9090"]
EOF

echo "groups: []" > rules-lab.yml        # filled in Milestone 3
docker compose up -d
docker compose ps
```

Expected: five services (`shop-api` , `loadgen` , `webhook` , `alertmanager` , `prometheus`) in state `running` . Add a small helper to query Prometheus from the shell:

BASH

```
q() { curl -s localhost:9090/api/v1/query --data-urlencode "query=$1" \
  | python3 -c 'import sys,json; [print(r["metric"], r["value"][1]) for r in
  json.load(sys.stdin)["data"]["result"]]'; }
curl -s localhost:8080/healthz; echo
q 'up{job="shop-api"}'
```

Expected: `ok`, then a line similar to `{'__name__': 'up', 'instance': 'shop-api:8080', 'job': 'shop-api'} 1`. If host port 8080 is already used, change the left side of the `ports:` mapping in `compose.yaml` and use that port in the `curl` commands.

Milestone 1 — Define the SLIs (topic 01)

Choose the user journey (placing an order) and specify each SLI as "good events / valid events", then implement it as PromQL.

SLI	Specification	Good events	Valid events
Availability	checkout requests that do not fail on the server	`/checkout`, code 2xx or 3xx	`/checkout`, every code except 4xx
Latency	checkout requests answered fast enough	`/checkout` served in 0.3 s or less (histogram bucket `le="0.3"`)	all `/checkout` requests

4xx responses are the client's mistakes, so they are excluded from the valid events; `/healthz` and `/products` are out of scope for the checkout SLO.

```bash
q 'sum(rate(http_requests_total{route="/checkout",code=~"2..|3.."}[1m])) /
  sum(rate(http_requests_total{route="/checkout",code!~"4.."}[1m]))'
q 'sum(rate(http_request_duration_seconds_bucket{route="/checkout",le="0.3"}[1m])) /
  sum(rate(http_request_duration_seconds_count{route="/checkout"}[1m]))'
```

Expected after a minute of traffic: availability close to `0.9995` (the baseline is 0.05% errors) and latency close to `1`. The histogram has a bucket boundary exactly at 0.3, which the latency SLI needs. Cross-check against the raw counters once, as in SLIs §14.1: `curl -s localhost:8080/metrics | grep 'route="/checkout"'`.

Milestone 2 — Set the SLOs and compare them with an SLA (topics 02 and 03)

Write the objective down where the team keeps its code. Save as `slo-shop-checkout.md`:

```markdown
# SLO: shop-api checkout            Owner: team-shop    Reviewed: 2026-10-09 (quarterly)

## Service and user journey
Customers placing an order through the web and mobile apps (the `/checkout` route in the
  lab).

## SLIs
Availability: good = 2xx/3xx, valid = all but 4xx. Latency: good = answered in 0.3 s or
  less. Measured at the application in the lab (at the load balancer in production).

## Objectives (rolling 30 days)
- Availability: 99.9 %  (error budget 0.1 %, 43.2 minutes of full outage per 30 days)
```

```
- Latency: 99 % of requests under 300 ms  (error budget 1 %)

## Rationale
Baseline in the lab is 99.95 % availability. Dependencies (PostgreSQL, Redis, payments)
  compose to roughly 99.7-99.9 %, so a tighter target is not credible yet.

## Exclusions
Health checks, 4xx responses, announced maintenance (7 days notice).

## Error budget policy
Above 50 % remaining: normal pace. 25-50 %: extra review, canary only. Below 25 %: no
  risky changes, reliability work first.
Exhausted: freeze non-critical releases until the cause is fixed or budget recovers;
  security and reliability fixes are exempt. Override owner: the engineering manager, in
  writing.

## Review
Quarterly, by team-shop and product. Changed only with a written reason.
```

The example SLO document in topic 2 uses a 28-day window and topic 4 uses 30; this capstone standardises on **30 days** because the burn-rate arithmetic in the rules assumes it.

Now compare with a **hypothetical** customer SLA (illustrative, not any provider's terms): "99.5% monthly availability, 10% credit below it". The internal SLO must be tighter, and the gap is your safety margin:

```bash
python3 -c "
slo, sla, minutes = 99.9, 99.5, 30*24*60
print(f'SLO downtime budget : {(100-slo)/100*minutes:6.1f} min / 30 days')
print(f'SLA downtime allowed: {(100-sla)/100*minutes:6.1f} min / 30 days')
print(f'margin before a credit is owed: {(slo-sla)/100*minutes:6.1f} min')"
```

Expected: `43.2`, `216.0` and `172.8` minutes. Write one paragraph in `slo-shop-checkout.md` saying what happens at each line: SLO missed means internal freeze, SLA missed means credits and a customer conversation (see SLAs §14.3).

Milestone 3 — Error budget policy and burn-rate alerts (topics 04 and 08)

Generate the rules from the SLO targets instead of hand-copying thresholds. The generator emits, for each SLI, recording rules for the error ratio at every window, a `budget_remaining` rule over the 30-day window, and three alerts (fast page, slow page, ticket) using the long-window-and-short-window pattern from error budgets §14.4.

```bash
cat > gen_rules.py <<'EOF'
#!/usr/bin/env python3
```

```
"""Generate multi-window, multi-burn-rate Prometheus rules for the shop-api SLOs.

usage: python3 gen_rules.py [--scale 1] > shop-api-slo-rules.yml
  --scale 1     real windows (5m/1h, 30m/6h, 6h/3d, 30d budget window)
  --scale 0.05  lab windows, 20x faster (15s/3m, 90s/18m, 18m/3.6h, 36h) for watching
  alerts fire in minutes
"""
import argparse

ap = argparse.ArgumentParser()
ap.add_argument("--scale", type=float, default=1.0)
scale = ap.parse_args().scale

def dur(seconds):
    s = round(seconds)
    for unit, n in (("d", 86400), ("h", 3600), ("m", 60), ("s", 1)):
        if s % n == 0:
            return f"{s // n}{unit}"
    return f"{s}s"

# (alert suffix, long window s, short window s, burn rate, severity, for)
ALERTS = [("Fast", 3600, 300, 14.4, "page", 120),
          ("Slow", 21600, 1800, 6.0, "page", 300),
          ("Ticket", 259200, 21600, 1.0, "ticket", None)]
BUDGET_WINDOW = 30 * 86400

SLOS = {  # name → (target, bad-ratio PromQL template with {w} for the range window)
    "availability": (0.999,
        '(sum(rate(http_requests_total{{route="/checkout",code=~"5.."}}[{w}])) or
  vector(0))\n'
        '/\nsum(rate(http_requests_total{{route="/checkout",code!~"4.."}}[{w}]))'),
    "latency": (0.99,
        '1 - (\n
  sum(rate(http_request_duration_seconds_bucket{{route="/checkout",le="0.3"}}[{w}]))\n'
        '  /\n  sum(rate(http_request_duration_seconds_count{{route="/checkout"}}
  [{w}]))\n)'),
}

def block(text, indent):
    return "\n".join(" " * indent + line for line in text.split("\n"))

out = ["# Generated by gen_rules.py --scale %g  (do not edit by hand)" % scale, "groups:"]
for name, (target, tmpl) in SLOS.items():
    budget = round(1 - target, 6)
    windows = sorted({int(w * scale) for a in ALERTS for w in a[1:3]} | {int(BUDGET_WINDOW
  * scale)})
    rec = f"sli:shop_api_{name}:error_ratio_rate"
    out += [f"  - name: shop-api-{name}-recording", "    interval: 15s", "    rules:"]
    for w in windows:
        out += [f"      - record: {rec}{dur(w)}", "        expr: |",
  block(tmpl.format(w=dur(w)), 10)]
    out += [f"      - record: slo:shop_api_{name}:error_budget_remaining:ratio",
            f"        expr: 1 - ({rec}{dur(BUDGET_WINDOW * scale)} / {budget})",
```

```
            f"  - name: shop-api-{name}-burn-rate-alerts", "    rules:"]
    for suffix, long_s, short_s, rate, sev, wait in ALERTS:
        lw, sw = dur(long_s * scale), dur(short_s * scale)
        thr = f"{rate:g} * {budget}"
        out += [f"      - alert: ShopApi{name.capitalize()}Burn{suffix}",
                "        expr: |",
                f"          {rec}{lw} > ({thr})",
                "          and",
                f"          {rec}{sw} > ({thr})"]
        if wait:
            out.append(f"        for: {dur(wait * scale)}")
        out += ["        labels:", f"          severity: {sev}", f"          slo: shop-
  api-{name}",
                "        annotations:",
                f'          summary: "shop-api {name} SLO burning {rate:g}x too fast ({lw}
  and {sw} windows)"',
                '          runbook_url: "https://runbooks.example.com/shop-api/error-
  budget-burn"']
print("\n".join(out))
EOF
python3 gen_rules.py > shop-api-slo-rules.yml              # production windows
python3 gen_rules.py --scale 0.05 > rules-lab.yml          # 20x faster, for the lab
grep -E "record:|alert:" shop-api-slo-rules.yml | head -8
```

Two details worth reading in the output. The availability numerator is wrapped in `(... or vector(0))`, so the ratio is `0` rather than "no data" when there are no 5xx series at all. The thresholds are written as `burn rate * budget` (for example `14.4 * 0.001`) so a reviewer can see where the numbers come from.

Validate the files (Docker images contain the tools):

BASH

```
docker run --rm -v "$PWD":/w --entrypoint promtool prom/prometheus:v2.53.0 check rules
  /w/shop-api-slo-rules.yml
docker run --rm -v "$PWD":/w --entrypoint promtool prom/prometheus:v2.53.0 check rules
  /w/rules-lab.yml
docker run --rm -v "$PWD":/w --entrypoint amtool prom/alertmanager:v0.27.0 check-config
  /w/alertmanager.yml
```

Expected: `SUCCESS: 20 rules found` twice and an Alertmanager config summary ending without errors.

Unit-test the production rules with synthetic traffic. Save as `shop-api-slo-rules.test.yml`:

YAML

```
rule_files:
  - shop-api-slo-rules.yml
evaluation_interval: 15s
```

```
tests:
  # 20 req/s, 5 % errors = 50x burn: the fast alert fires
  - interval: 30s
    input_series:
      - series: 'http_requests_total{route="/checkout", code="200"}'
        values: '0+570x400'
      - series: 'http_requests_total{route="/checkout", code="500"}'
        values: '0+30x400'
    alert_rule_test:
      - eval_time: 3h
        alertname: ShopApiAvailabilityBurnFast
        exp_alerts:
          - exp_labels:
              severity: page
              slo: shop-api-availability
            exp_annotations:
              summary: "shop-api availability SLO burning 14.4x too fast (1h and 5m
  windows)"
              runbook_url: "https://runbooks.example.com/shop-api/error-budget-burn"

  # errors stop after 70 minutes: the 5m window recovers, so the alert has cleared at
  minute 100 although the 1h average is still high
  - interval: 30s
    input_series:
      - series: 'http_requests_total{route="/checkout", code="200"}'
        values: '0+570x140 80400+600x259'
      - series: 'http_requests_total{route="/checkout", code="500"}'
        values: '0+30x140 4200+0x259'
    alert_rule_test:
      - eval_time: 69m
        alertname: ShopApiAvailabilityBurnFast
        exp_alerts:
          - exp_labels:
              severity: page
              slo: shop-api-availability
            exp_annotations:
              summary: "shop-api availability SLO burning 14.4x too fast (1h and 5m
  windows)"
              runbook_url: "https://runbooks.example.com/shop-api/error-budget-burn"
      - eval_time: 100m
        alertname: ShopApiAvailabilityBurnFast
        exp_alerts: []

  # healthy: 0.05 % errors = 0.5x burn, nothing fires
  - interval: 30s
    input_series:
      - series: 'http_requests_total{route="/checkout", code="200"}'
        values: '0+599.7x400'
      - series: 'http_requests_total{route="/checkout", code="500"}'
        values: '0+0.3x400'
    alert_rule_test:
      - eval_time: 3h
        alertname: ShopApiAvailabilityBurnFast
```

```
        exp_alerts: []
      - eval_time: 3h
        alertname: ShopApiAvailabilityBurnSlow
        exp_alerts: []
```

BASH

```
docker run --rm -v "$PWD":/w -w /w --entrypoint promtool prom/prometheus:v2.53.0 test
  rules shop-api-slo-rules.test.yml
```

Expected: `SUCCESS`. If a test fails, read the diff `promtool` prints: it shows expected versus actual alerts and labels.

Now a runnable calculator that tells you **when each alert would fire** for a failure you describe, with production windows. It replays a constant error ratio minute by minute:

BASH

```
cat > burn_sim.py <<'EOF'
#!/usr/bin/env python3
"""When would the shop-api burn-rate alerts fire?  Minute-by-minute replay of a constant-
  rate failure.

usage: python3 burn_sim.py [--slo 99.9] [--err 0.10] [--fault-min 25] [--minutes 480] [--
  baseline 0.0005]
  --err        error ratio while the fault is active (0.10 = 10 %)
  --fault-min  minutes the fault lasts before it is mitigated
"""
import argparse

ap = argparse.ArgumentParser()
ap.add_argument("--slo", type=float, default=99.9)
ap.add_argument("--err", type=float, default=0.10)
ap.add_argument("--fault-min", type=int, default=25)
ap.add_argument("--minutes", type=int, default=480)
ap.add_argument("--baseline", type=float, default=0.0005)   # normal error ratio, 0.05 %
a = ap.parse_args()

budget = 1 - a.slo / 100
period = 30 * 24 * 60                                       # 30-day window in minutes
# (name, long window min, short window min, burn rate, for-minutes)
ALERTS = [("Fast  14.4x 1h/5m", 60, 5, 14.4, 2), ("Slow  6x 6h/30m", 360, 30, 6.0, 5),
  ("Ticket 1x 3d/6h", 4320, 360, 1.0, 0)]

err = [a.err if t < a.fault_min else a.baseline for t in range(a.minutes)]
def mean(t, w):                                             # average error ratio over
  the last w minutes, healthy before t=0
    vals = [err[i] if i >= 0 else a.baseline for i in range(t - w + 1, t + 1)]
    return sum(vals) / w
```

```
print(f"SLO {a.slo}%  budget {budget:.4%}  fault: {a.err:.1%} errors for {a.fault_min} min
  (burn {a.err / budget:.0f}x)")
for name, lw, sw, rate, wait in ALERTS:
    thr, run, fired, cleared = rate * budget, 0, None, None
    for t in range(a.minutes):
        cond = mean(t, lw) > thr and mean(t, sw) > thr
        run = run + 1 if cond else 0
        if fired is None and run > wait:                    # condition held for the
  whole `for` period
            fired = t + 1
        if fired is not None and cleared is None and not cond:
            cleared = t + 1
    f = f"fires at minute {fired}" if fired else "never fires"
    c = f", clears at minute {cleared}" if cleared else (", still firing at end" if fired
  else "")
    print(f"  {name:20} threshold {thr:.4%}  {f}{c}")

spent = sum(e - a.baseline for e in err[:a.fault_min]) / (budget * period)
print(f"The incident itself consumed {spent:.1%} of the 30-day budget (baseline noise
  excluded).")
EOF
python3 burn_sim.py --err 0.10 --fault-min 25
```

Expected (this is real output):

```
SLO 99.9%  budget 0.1000%  fault: 10.0% errors for 25 min (burn 100x)
  Fast  14.4x 1h/5m    threshold 1.4400%  fires at minute 11, clears at minute 30
  Slow  6x 6h/30m      threshold 0.6000%  fires at minute 25, clears at minute 54
  Ticket 1x 3d/6h      threshold 0.1000%  fires at minute 22, clears at minute 384
The incident itself consumed 5.8% of the 30-day budget (baseline noise excluded).
```

Read it carefully: the fast alert pages after about 11 minutes and clears 5 minutes after the fix (the short window at work); the slow alert is still true for roughly half an hour after the fix because its short window is 30 minutes. Try the other shapes:

BASH

```
python3 burn_sim.py --err 0.02 --fault-min 120 --minutes 600      # a 20x burn for two
  hours
python3 burn_sim.py --err 0.003 --fault-min 4000 --minutes 4000   # a quiet 3x burn: only
  the ticket fires, after about 14 hours
python3 burn_sim.py --slo 99 --err 0.30 --fault-min 10 --baseline 0   # a 30x burn of a 99
  percent SLO for 10 minutes: no alert yet
```

The last one is a lesson: a short, intense failure against a loose SLO may not trip any alert, because it spends only 0.7% of the budget. That is the policy working as designed, not a gap, but you should know it.

Load the lab rules into the running Prometheus and check them (the lab stack scrapes every 5 seconds and the lab windows are 15s/3m, 90s/18m and 18m/3.6h):

```bash
curl -s -X POST localhost:9090/-/reload
q 'slo:shop_api_availability:error_budget_remaining:ratio'
q 'slo:shop_api_latency:error_budget_remaining:ratio'
```

Expected after a couple of minutes: availability budget remaining close to `0.5` (the app's baseline 0.05% errors use half of a 0.1% budget by design; with minutes of history in a 36h window the number is noisy) and latency close to `1`. Open `http://localhost:9090/alerts` and confirm six alerts, all `inactive`.

Alert quality (topic 08). Check each alert against the six tests: urgent, actionable, real, novel, explained, owned.

Alert	Severity	Urgent	Actionable	Duplicates?	Explained
Availability fast and slow burn	page	yes, budget gone in hours or days	yes: runbook step 1 is "was there a deploy"	slow inhibited while fast fires	summary in user terms, runbook link
Latency fast and slow burn	page	yes	yes	slow inhibited while fast fires	same
Availability and latency ticket	ticket	no, next working day	yes	no	same

The `inhibit_rules` in `alertmanager.yml` already suppress the slow-burn notification while the fast-burn alert for the same SLO is firing.

Milestone 4 — On-call: route, runbook, handoff (topic 06)

Pages go to the fake pager, tickets to the ticket queue. Watch both streams in a spare terminal: `docker compose logs -f webhook`. Then write the runbook the pager link points to. Save as `runbooks/shop-api-error-budget-burn.md`:

```
MARKDOWN
# Runbook: shop-api error budget burn (ShopApi*Burn*)

**Meaning:** checkout is failing or slow fast enough to empty the 30-day budget in hours
  (page) or days (ticket).
**First 5 minutes**
1. Acknowledge the page. Open the dashboard / Prometheus: `sum by (code)
  (rate(http_requests_total{route="/checkout"}[5m]))`.
2. Which SLI? 5xx rising = availability. `le="0.3"` share falling = latency.
3. Did anything change in the last hour (deploy, config, flag, traffic spike)? If yes and
  the timing matches: roll it back first, investigate after.
4. Declare an incident if impact is user-visible and not fixed in 15 minutes (SEV2 for
  fast burn). Use the roles card.
**Mitigations:** rollback, feature flag off, scale out, shed load, fail over. **Escalate**
  to the secondary after 15 minutes without progress.
**After:** note the times in the incident log, open a postmortem for any page that
  consumed more than 5 percent of the budget.
```

Write a handoff note for the end of your pretend shift (open incidents, noisy alerts, risky changes, quirks), as in on-call practices §14.6. For sustainability, run `python3 burn_sim.py --err 0.10 --fault-min 25` three times in your head as "three incidents this week": each produces at most two pages, which is well inside what a single on-call can absorb.

Milestone 5 — Run the mock incident (topics 05, 09 and 10)

This is a **game day**: a planned failure with a hypothesis, a blast radius (a local container) and an abort condition (`docker compose restart shop-api` resets the faults).

Hypothesis: if 10% of checkouts start failing, `ShopApiAvailabilityBurnFast` pages within about two minutes of lab time, the response restores service within five minutes of the page, and the alert resolves by itself afterwards.

First print the card and decide who plays each role (if you are alone, play them in turn and write the role name before every log line):

```
INCIDENT ROLES CARD
IC (Incident Commander)  declares, sets severity, assigns roles, decides, calls the end.
  Does NOT type in production.
Ops (Operations Lead)    investigates and changes the system, says intent before acting.
  Does NOT decide severity or talk to customers.
Comms (Communications)   status updates on a fixed cadence, answers support and
  leadership. Does NOT debug.
Scribe                   keeps the UTC timeline.
  Does NOT take decisions.
SME                      answers questions for Ops when asked.
Severity: SEV1 core journey down for most users or data loss.  SEV2 major degradation,
  fast burn.  SEV3 minor, workaround exists.  When in doubt, go higher.
Status updates: Investigating → Identified → Monitoring → Resolved, each with the time
  of the next update.
Rule: no change in production without saying so in the channel first.
```

Before the failure: a capacity check (topic 09). Know how much load you carry and what margin you have, because a rollback that sends users back in a burst needs headroom:

```bash
q 'sum(rate(http_requests_total{route="/checkout"}[1m]))'
python3 -c "
import math
peak, knee, util = 20, 40, 0.6      # checkout rps now, an assumed tested knee per
  instance, target utilisation
print('safe per instance:', knee*util, 'rps; instances needed for 3x peak with N+1:',
  math.ceil(peak*3/(knee*util))+1)"
```

Expected: about `20` requests per second (the loadgen alternates two routes at 40 per second) and `instances needed ... 4`. The knee here is an assumption; in a real service you measure it with a load test as in capacity planning §14.9.

Start the scribe log and the failure:

```bash
note() { echo "$(date -u +%Y-%m-%dT%H:%M:%SZ) NOTE   $*" >> incident.log; }
note "[impact-start] Bad deploy 4821 begins failing 10 percent of checkouts"
curl -s 'localhost:8080/admin/fault?error_rate=0.10'
```

Expected response: `{'error_rate': 0.1, 'slow_rate': 0.0}`. Now respond as if you did not know the cause:

1. Wait for the page: `docker compose logs -f webhook`. Expected within roughly one to two minutes: a line similar to `/page firing ShopApiAvailabilityBurnFast shop-api availability SLO burning 14.4x too fast (3m and 15s windows)`. Log it: `note "[detected] ShopApiAvailabilityBurnFast paged"`.
2. **Declare** (IC), and log it:

   ```
   INCIDENT DECLARED  SEV2  Checkout errors (shop-api)
   IC: <name>  Ops: <name>  Comms: <name>  Scribe: <name>
   Impact: about 10% of checkout requests return 5xx since <time> UTC
   Next update: <time + 15 min>   Channel: #inc-<date>-checkout
   ```

   Post the first status update: "Investigating: some customers see errors when placing orders. Next update by ...".
3. **Investigate** (Ops, using the runbook). Confirm the symptom and scope:

   ```bash
   q 'sum by (code)(rate(http_requests_total{route="/checkout"}[1m]))'
   q 'sum by (route)(rate(http_requests_total{code="500"}[1m]))'
   q 'slo:shop_api_availability:error_budget_remaining:ratio'
   ```

   You should see `500` at roughly 10% of checkout traffic and the budget-remaining ratio falling. Note what you see and what you decide: `note "Ops: 5xx at 10 percent, started with deploy 4821, proposing rollback"`.
4. **Mitigate** (Ops announces, then acts). In the lab, "rolling back" is switching the fault off:

   ```bash
   curl -s 'localhost:8080/admin/fault?error_rate=0'
   note "[mitigated] Rollback of deploy 4821 done, 5xx ratio falling"
   ```

   Post "Monitoring: a fix has been applied, we are watching checkout."
5. **Watch it recover.** The fast alert should resolve within a minute or so (watch `docker compose logs -f webhook` for a `resolved` line, and the `alerts` page). You may also see the slow-burn alert go pending or firing during the incident: Alertmanager suppresses its notification while the fast alert is firing. Check what happens to it once the fast alert has resolved, and compare with the simulator's "slow clears later" line. Whether to keep that behaviour is a design decision for the postmortem. When the 5xx ratio is back to baseline: `note "[resolved] Error ratio at baseline for 2 minutes"` and post "Resolved".
6. Keep the log: `cat incident.log`. It is the raw material for Milestone 6.

Variation B (reliability engineering, topic 10). Same hypothesis format, different fault: `curl -s 'localhost:8080/admin/fault?slow_rate=0.3'` makes 30% of requests take 0.6 s. Predict first: which alert fires (latency, not availability) and why the availability SLI stays green. Run it, then reset with `curl -s 'localhost:8080/admin/fault?slow_rate=0'`.

Milestone 6 — Write the postmortem (topic 07)

First compute the timings from your log with the `pm_times.py` script from postmortems §14.7 (step 1 there): `python3 pm_times.py incident.log` prints time to detect, mitigate and resolve. The lab runs about 20 times faster than production, so quote the lab numbers and say so. Then copy this template to `postmortem-2026-10-09-checkout.md` and fill it in. The linter from the same topic (`pm_lint.py`) checks that every action item has an owner, a due date and a ticket.

```markdown
# Postmortem: <short title>

| | |
|---|---|
| Incident | INC-<id>, SEV<n> |
| Date | <YYYY-MM-DD> |
| Authors | <names> |
| Status | Draft / In review / Final |

## Summary
Two or three sentences a stranger can read: what broke, who was affected, how long, how it
  was fixed.

## Impact
- User impact: <what users saw, how many, which feature>
- Duration: <minutes> (time to detect <m>, time to mitigate <m>, time to resolve <m>)
- Error budget consumed: <percent of the 30-day budget> (from
  slo:shop_api_availability:error_budget_remaining:ratio before and after)
- Other: <support tickets, revenue, SLA exposure>

## Timeline (all times UTC)
| Time | Event |
|---|---|
| HH:MM | Trigger: <change or event> |
| HH:MM | Impact begins |
| HH:MM | Detected by <alert or person> |
| HH:MM | Incident declared SEV<n>, IC <name> |
| HH:MM | Decision: <what, why, what was known at the time> |
| HH:MM | Mitigated |
| HH:MM | Resolved |

## Root cause
The deepest condition that, if fixed, would have prevented this failure. A system or
  process, never a person.
```

```
## Contributing factors
- <what made it worse or longer>

## What went well
- <controls and behaviours to keep>

## What went poorly
- <gaps, confusion, luck>

## Where we got lucky
- <things that could have made it worse>

## Action items
| ID | Action | Type | Owner | Due | Ticket |
|----|--------|------|-------|-----|--------|
| AI-1 | <verb-first, specific> | prevent/detect/mitigate/process | @name | YYYY-MM-DD |
  SHOP-000 |

## Lessons
What other teams should learn from this.
```

For the lab incident, a believable set of findings (these are illustrative; use what you actually observed): root cause "nothing in the release pipeline verifies error rate after a deploy, so a faulty version served all traffic until a human rolled it back"; contributing factors "rollback is a manual command", "the page did not link to the deploy that preceded it", "the slow-burn alert became active after the fast one resolved"; action items such as "Add automatic post-deploy error-rate check with rollback to the pipeline" (prevent), "Add deploy annotation and rollback link to the alert" (detect), "Decide whether slow-burn pages are wanted after mitigation" (process), each with an owner, date and ticket. Hold a 30-minute review with a facilitator who was not the IC, and agree the action items, as in postmortems §14.7.

5 Tasks

- [ ] **T1 — Bring up the lab and measure the SLIs** *(≈45 min, beginner; topic 01, Milestones 0-1)*
  - **Goal:** a running `shop-api` stack whose availability and latency SLIs you can read from Prometheus.
  - **Steps:**
    1. Create the lab files and run `docker compose up -d`.
    2. Run the two SLI queries and the raw-counter cross-check.
    3. Change the latency threshold in the query from `0.3` to `0.1` and explain why the answer drops.

- **Done when:** `q` returns availability near `0.9995` and latency near `1` for 0.3 s, and a lower value for 0.1 s with a one-sentence explanation (about 5% of requests take 150 ms).

- [ ] **T2 — Write the SLO document and the SLA margin** *(≈45 min, beginner; topics 02-03, Milestone 2)*
  - **Goal:** an SLO document a product owner could sign, and a quantified margin to a customer SLA.
  - **Steps:**
    1. Save `slo-shop-checkout.md` with SLIs, objectives, rationale, exclusions and policy.
    2. Run the margin calculation for an SLO of 99.9% and an SLA of 99.5%.
    3. Change the SLA to 99.9% and explain what is wrong with the result.
  - **Done when:** the margin prints `172.8` for the first case, and for the second you state that there is no safety margin so every SLO miss is an SLA miss.

- [ ] **T3 — Generate, test and understand the burn-rate rules** *(≈90 min, intermediate; topics 04 and 08, Milestone 3)*
  - **Goal:** valid multi-window, multi-burn-rate alerts, validated and unit-tested.
  - **Steps:**
    1. Generate both rules files and run `promtool check rules` on each.
    2. Save and run the unit test file with `promtool test rules`.
    3. Run `burn_sim.py` for the three shapes in the guide and write down which alert fires for each.
    4. Change the SLO to 99.5% in `gen_rules.py` (`0.995`), regenerate, and update the test so a 10% error rate still fires but 0.2% does not.
  - **Done when:** `promtool` reports `SUCCESS: 20 rules found` and `SUCCESS` for the tests (before and after the SLO change), and your notes explain why a 0.3% error ratio never pages at 99.9% but eventually tickets.

- [ ] **T4 — Break the alert and fix it** *(≈60 min, intermediate; topics 04 and 08, Milestone 3)*
  - **Goal:** see why each alert needs both a long and a short window.
  - **Steps:**
    1. In the generated production rules, delete the short-window condition (the second line after `and`) from `ShopApiAvailabilityBurnFast`.
    2. Re-run the second unit test (errors stop at minute 70, check at minute 100).
    3. Observe the failure, then restore the line and re-run until the test passes.

- **Done when:** you have output of a failing test for the broken rule and `SUCCESS` for the restored one, and you can say in one sentence what the short window buys you.

- [ ] **T5 — Route, runbook and handoff** *(≈60 min, intermediate; topic 06, Milestone 4)*
  - **Goal:** pages reach the right place and a stranger could act on them.
  - **Steps:**
    1. Start the stack with the lab rules loaded and confirm all six alerts are `inactive`.
    2. Save the runbook to `runbooks/shop-api-error-budget-burn.md` and add its real path to the `runbook_url` in `gen_rules.py` (placeholders are fine for the lab).
    3. Fill in the six-point alert quality table for all six alerts and delete or downgrade any that fail it.
    4. Write a handoff note for the next on-call.
  - **Done when:** `amtool check-config` passes, the alerts page shows six inactive alerts, and a teammate can follow the runbook's first five minutes without asking you a question.

- [ ] **T6 — Run the mock incident and keep the log** *(≈90 min, advanced; topics 05, 09 and 10, Milestone 5)*
  - **Goal:** a complete, role-based response with evidence.
  - **Steps:**
    1. Do the capacity check, write the hypothesis, then inject the 10% failure.
    2. Declare the incident with the template, run the response, mitigate and wait for the alert to resolve.
    3. Keep `incident.log` with the tags `[impact-start]`, `[detected]`, `[mitigated]` and `[resolved]`.
    4. Run Variation B (slow requests) and record which alert fired.
  - **Done when:** `docker compose logs webhook` shows a firing and a resolved line for the availability alert, `incident.log` has all four tags, and your hypothesis is marked confirmed or refuted with the numbers.

- [ ] **T7 — Write and lint the postmortem** *(≈90 min, advanced; topic 07, Milestone 6)*
  - **Goal:** a blameless postmortem that passes the linter.
  - **Steps:**
    1. Compute timings with `pm_times.py incident.log`.
    2. Fill in the template with a timeline from the log, a root cause that is a system condition, at least three contributing factors and four action items with owner, due date and ticket.

3. Run `pm_lint.py` on it and fix every problem.

- **Done when:** `python3 pm_lint.py` prints `0 problems`, the impact section quotes the budget consumed, and no sentence names a person as the cause.

- [ ] **T8 — Prove it: SLO to postmortem from a blank directory** *(≈2 hours, advanced; all topics)*
  - **Goal:** show you can do the whole loop without the guide, and explain it.
  - **Steps:**
    1. In a new empty directory, with only the scripts and your notes, define one new SLO (for example 99.95% availability), regenerate the rules, run `promtool check rules` and `promtool test rules`.
    2. Predict with `burn_sim.py` when the fast alert will fire for a 5% failure, then verify in the lab and compare.
    3. Run a short mock incident as IC with a second person as Ops if possible, and write a half-page postmortem with three action items.
    4. Explain in writing (one page) why you page on budget burn rather than on raw error rate, what the error budget policy would have done after this incident, and what you would change first.
  - **Done when:** the predicted and observed firing times agree within a scheduling interval or two, the one-page explanation covers burn rate, windows, policy and blamelessness, and a reviewer can follow every claim to a command output.

6 Verification checklist

- [ ] `docker compose ps` shows five running services and `q 'up{job="shop-api"}'` returns `1`.
- [ ] The availability and latency SLIs return values you can reproduce from the raw `/metrics` counters.
- [ ] `slo-shop-checkout.md` states both objectives, exclusions, rationale and an error budget policy with an override owner.
- [ ] `promtool check rules` passes on both generated files (`SUCCESS: 20 rules found`) and `promtool test rules` passes.
- [ ] `amtool check-config alertmanager.yml` passes; pages and tickets go to different webhook paths.
- [ ] `burn_sim.py` output for the 10% scenario matches the numbers in this capstone and you can explain each line.

- [ ] The mock incident produced a firing and a resolved notification, and `incident.log` contains the four tagged moments.
- [ ] The postmortem has a timeline, root cause, contributing factors, a budget impact figure and four action items, and `pm_lint.py` reports `0 problems`.
- [ ] You can explain, without notes, what a 14.4x burn rate means and why both a long and a short window are needed.

7 Teardown

Nothing here costs money. Clean up the containers, volumes and files:

```bash
docker compose down -v          # stops the stack and deletes Prometheus data
cd .. && rm -rf l14-lab         # only after copying out anything you want to keep: SLO
  doc, rules, runbook, postmortem
```

Keep `slo-shop-checkout.md`, `gen_rules.py`, the unit tests, the runbook and the postmortem in a Git repository: they are the deliverables of this level.

8 Stretch goals and what to learn next

- **Use a generator or a platform.** Reproduce the rules with Sloth or Pyrra from a short SLO definition and diff the output against yours. Add a Grafana dashboard for budget remaining and burn rate (Grafana dashboards §10.3).
- **Measure at the edge.** Move the availability SLI from the application to the load balancer metrics, and add a synthetic probe as a cross-check (what to monitor §10.11).
- **Close the loop with deploys.** Add an error-rate gate to a canary deployment that rolls back automatically (canary deployments §5.12, rollback §5.7), and take that postmortem action item to done.
- **Add resilience patterns.** Add timeouts, retries with backoff and a circuit breaker to the load generator and rerun Variation B (circuit breakers §12.14).
- **Run a real game day.** Gather three people, assign roles from the card, and have someone you do not sit near choose the failure.
- **Next level:** reliability includes surviving the failures you cannot prevent. Continue with Level 15 and start with backups §15.1, then define recovery targets in RPO §15.6 and RTO §15.7.

LEVEL 15 · PART VII

Disaster Recovery

15

Goal: Design, document and rehearse recovery so that failures are survivable and recovery times are known, not guessed.

RECOMMENDED TIME
2-3 weeks

TOPICS
10

PRACTICE / CAPSTONE
Practice failure scenarios: database failure, server failure, pod failure, node failure and region failure, with timed recovery and a written DR plan.

Topics

#	Topic	What you will learn
1	Backups **p.2985**	3-2-1 rule, full/incremental, logical vs physical, encryption, retention
2	Replication **p.2997**	sync/async, cross-AZ/region, replication is not backup
3	Snapshots **p.3007**	EBS/RDS snapshots, consistency, lifecycle, cross-region copy
4	Restore Procedures **p.3019**	documented runbooks, order of restore, validation after restore
5	Failover **p.3033**	automatic vs manual, DNS/LB failover, split-brain, failback
6	RPO - Recovery Point Objective **p.3047**	data-loss tolerance, deriving backup frequency, cost of lower RPO
7	RTO - Recovery Time Objective **p.3059**	downtime tolerance, DR tiers (backup/restore, pilot light, warm standby, active-active)
8	Off-site Backups **p.3073**	cross-region/cross-account, immutability, object lock, ransomware protection
9	Testing Backup Restoration **p.3089**	automated restore tests, checksums, restore timing, evidence
10	Failure Scenario Drills **p.3103**	database, server, pod, node, region failure drills and game days
★	Level capstone project **p.3121**	Practice failure scenarios: database failure, server failure, pod failure, node failure and region failure, with timed recovery and a written DR plan.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



15.1 Backups

Topic 1 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

A **backup** is an independent, point-in-time copy of data that you can use to rebuild a system after the original is lost, corrupted or deleted. The key word is *independent*: if the copy shares a failure with the original (same disk, same account, same admin credentials, same replication stream) it is not a backup, it is a second copy of the same risk.

Backups differ along a few axes you must be able to choose between on purpose:

- **Full vs incremental vs differential.** A *full* backup copies everything. An *incremental* copies only what changed since the previous backup (full or incremental). A *differential* copies what changed since the last *full*.
- **Logical vs physical.** A *logical* backup exports data as SQL or rows (`pg_dump`); it is portable across versions but slow to restore. A *physical* backup copies the database's files plus the write-ahead log (`pg_basebackup`, volume snapshots); it restores fast and enables point-in-time recovery but is tied to the major version and CPU architecture.
- **Where it lives.** The **3-2-1 rule**: keep **3** copies of your data, on **2** different kinds of media or systems, with **1** copy off-site (modern wording adds: and 1 copy offline or immutable, and 0 errors after a restore test).

2 Why it matters in DevOps

- **Every other safety net has a hole.** High availability and replication protect against a *machine* failing; they happily replicate `DROP TABLE` and ransomware to every replica (see Replication §15.2). Only a backup protects against *humans, bugs and attackers*.
- **A backup that was never restored is a hope, not a backup.** The most common DR failure is discovering at 3 a.m. that the backup job has been silently failing for months. Topic 9 turns this into a routine test.
- **Backups define your worst case.** Your backup frequency *is* your RPO (RPO §15.6); your restore speed *is* a large part of your RTO (RTO §15.7). You cannot promise either without knowing how you back up.

- **Compliance and legal.** Retention periods, encryption at rest and proof of recoverability show up in SOC 2, ISO 27001, HIPAA and GDPR audits.

3 Real-world system example

GitLab.com, January 2017. A widely documented public postmortem describes an engineer, while fighting replication problems on the production PostgreSQL setup, deleting a data directory on the wrong database host. GitLab then found that several of its backup and replication mechanisms were not usable in practice, and recovered from a snapshot that happened to be about six hours old, losing some hours of user-generated data (git repositories themselves were stored separately). GitLab published its timeline openly and streamed the recovery. The lesson the industry took away: having "five backup methods" means nothing until at least one is monitored and restored on a schedule.

Managed databases as a reference design. Amazon RDS takes daily automated snapshots *and* continuously ships transaction logs to S3, which is what lets you restore to a chosen second inside the retention window (1-35 days, configurable). Self-managed PostgreSQL shops usually reproduce the same design with open-source tools such as **pgBackRest** or **WAL-G**: a base backup plus continuous WAL archiving to object storage, with encryption and retention managed by the tool.

4 Diagram

The 3-2-1 rule applied to the `shop-api` PostgreSQL database:

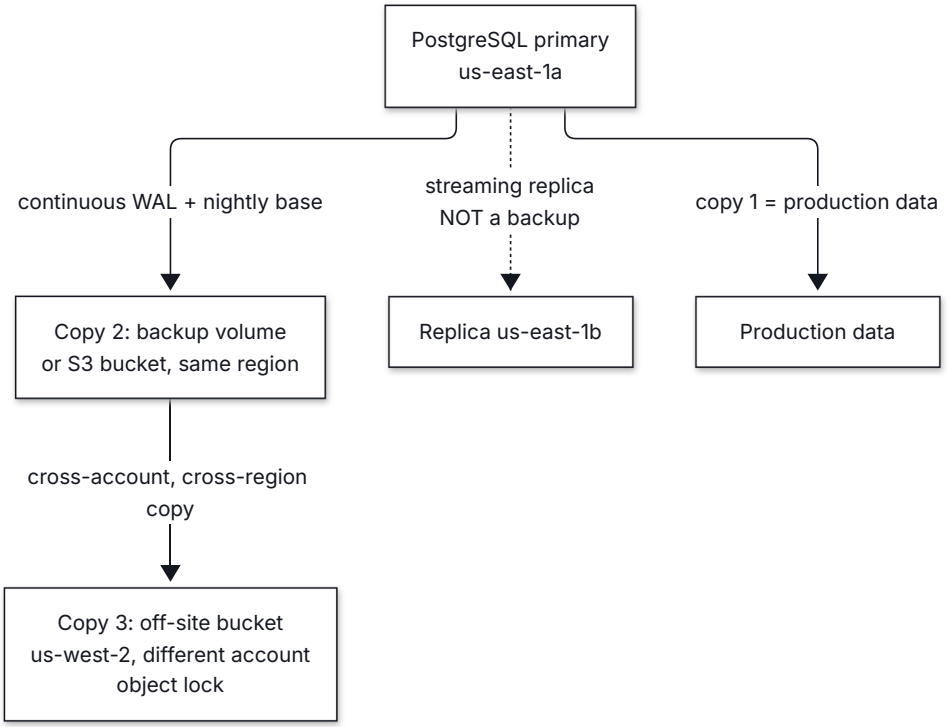


Figure 15.1-1

A backup chain, and why restore order matters:

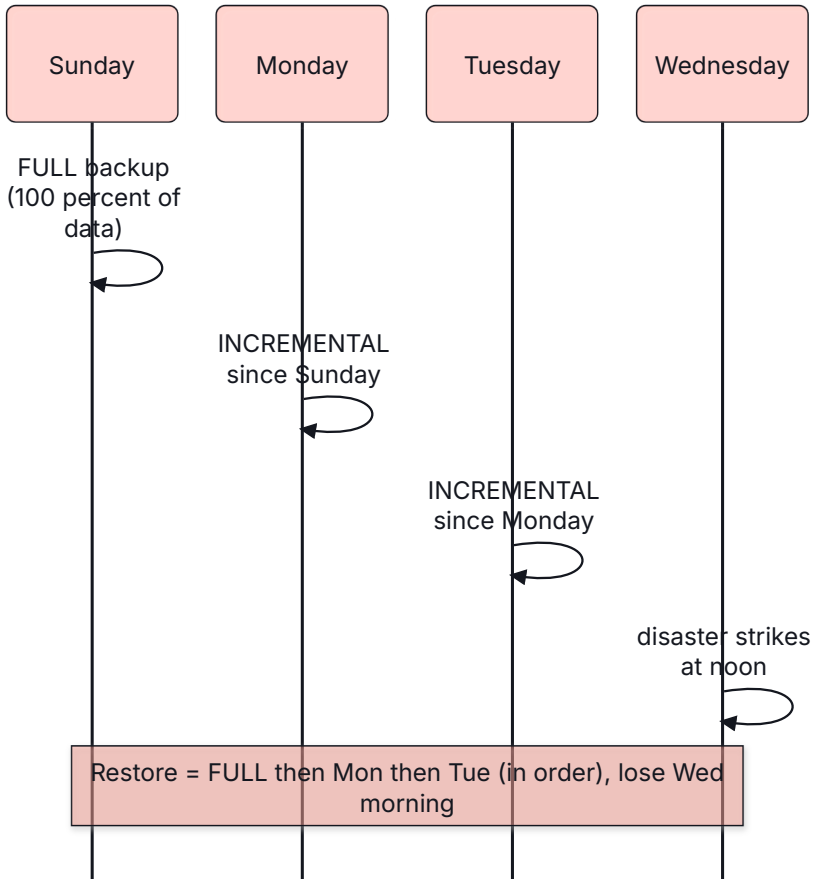


Figure 15.1-2

5 How it works

Type	Copies	Backup speed / size	Restore speed	Needs to restore
Full	everything	slowest, largest	fastest	itself only
Incremental	changes since last backup of any kind	fastest, smallest	slowest	full + every incremental in order
Differential	changes since last full	middle, grows daily	middle	full + latest differential

Approach	Tool examples	Strengths	Weaknesses
Logical	`pg_dump`, `mysqldump`, `mongodump`	portable across versions, can restore one table	slow on large DBs, only a single point in time, no PITR
Physical	`pg_basebackup`, `xtrabackup`, volume snapshots	fast restore, enables PITR with WAL archive	same major version and architecture, restores everything
Continuous (WAL / binlog archive)	`archive_command`, pgBackRest, WAL-G	RPO of seconds to minutes	more moving parts, archive gaps break recovery
File-level	`tar`, `rsync`, `restic`, `borg`	simple for configs and uploads	not consistent for a running database unless quiesced

Encryption. Encrypt *before* data leaves the host (client-side), keep keys somewhere that survives the disaster (a password manager or KMS in a different account), and test decryption in the restore drill. Tools: `age`, `gpg`, `restic` / `borg` (built-in), S3 SSE-KMS (server-side, protects the disk, not against anyone who can read the bucket). Losing the key equals losing the backup.

Retention. Choose a policy that matches the threats: recent copies for operator errors (days), older copies for slow-burn corruption nobody noticed (weeks to months), and compliance copies (years). A common scheme is "grandfather-father-son": daily for a week, weekly for a month, monthly for a year. Retention must never be shorter than the time you need to *notice* a problem.

Consistency. A backup of a running database must be transactionally consistent: use the database's own tool (`pg_dump` uses a snapshot transaction; `pg_basebackup` brackets the copy with WAL) rather than `cp -r` on a live data directory.

6 Step-by-step guide

Prerequisites: a Linux shell with PostgreSQL 16 server binaries (`sudo apt-get install -y postgresql-16`; the binaries live in `/usr/lib/postgresql/16/bin`), plus `tar`, `sha256sum`, `age` (`sudo apt-get install -y age`) and `python3`. Everything runs locally, for free, as your normal user in a scratch directory. We create a **throwaway** cluster on port 55433 and never touch any existing database.

1. **Start a disposable PostgreSQL cluster and load sample data.**

```bash
export PATH=/usr/lib/postgresql/16/bin:$PATH
export W=$HOME/dr-lab/backups-demo && rm -rf "$W" && mkdir -p "$W/backups"
initdb -D "$W/pg" >/dev/null
printf "port = 55433\nunix_socket_directories = '/tmp'\nlisten_addresses = ''\n" >>
  "$W/pg/postgresql.conf"
pg_ctl -D "$W/pg" -l "$W/pg.log" -w start
export PGHOST=/tmp PGPORT=55433
createdb shop
psql -X -q -d shop -c "CREATE TABLE orders(id serial primary key, item text, qty int)"
  \
  -c "INSERT INTO orders(item,qty) SELECT 'item-'||g, g%10 FROM
  generate_series(1,200000) g"
```

   Expected: `waiting for server to start.... done` / `server started`.

2. **Take a logical backup (compressed custom format) and a plain-SQL one; compare.**

```bash
time pg_dump -Fc -d shop -f "$W/backups/shop.dump"
pg_dump -d shop -f "$W/backups/shop.sql"
ls -lh "$W/backups"
```

   Similar to: the custom-format dump is ~1 MB, the plain SQL ~4 MB, and the dump takes well under a second for 200k rows. `pg_restore -l "$W/backups/shop.dump" | head` lists the table of contents, which lets you restore a single table later.

3. **Take a physical backup and verify it.**

```bash
time pg_basebackup -D "$W/backups/base" -Ft -z -X stream -c fast
pg_basebackup -D "$W/backups/plain" -X stream -c fast
pg_verifybackup "$W/backups/plain"
```

   Expected last line: `backup successfully verified`. PostgreSQL writes a `backup_manifest` with a checksum per file; `pg_verifybackup` checks it. This is your first automated integrity check.

4. **Restore the logical backup into a fresh database (proof that it works).**

```bash
createdb shop_restored
time pg_restore -d shop_restored --no-owner "$W/backups/shop.dump"
psql -X -At -d shop_restored -c "SELECT count(*) FROM orders"
```

Expected: `200000`.

5. **Add a checksum and client-side encryption, then prove you can decrypt.**

```bash
cd "$W/backups"
age-keygen -o backup-key.txt                          # prints: Public key: age1...
PUB=$(grep -o 'age1[a-z0-9]*' backup-key.txt | head -1)
age -r "$PUB" -o shop.dump.age shop.dump
sha256sum shop.dump.age > shop.dump.age.sha256
sha256sum -c shop.dump.age.sha256                     # shop.dump.age: OK
age -d -i backup-key.txt shop.dump.age | sha256sum    # must equal: sha256sum
  shop.dump
sha256sum shop.dump
```

Keep `backup-key.txt` somewhere that is **not** the machine being backed up. In production you would store only the public key on the server.

6. **Try incremental backups of plain files with GNU tar.**

```bash
mkdir -p ~/dr-lab/files && cd ~/dr-lab/files && rm -rf data snap.snar *.tar.gz &&
  mkdir data
for i in 1 2 3; do echo "file $i" > data/f$i.txt; done
tar --listed-incremental=snap.snar -czf full-0.tar.gz data
echo changed > data/f2.txt; echo new > data/f4.txt
tar --listed-incremental=snap.snar -czf incr-1.tar.gz data
tar -tzf incr-1.tar.gz        # only data/, data/f2.txt, data/f4.txt
mv data data.lost
tar --listed-incremental=/dev/null -xzf full-0.tar.gz
tar --listed-incremental=/dev/null -xzf incr-1.tar.gz
cat data/f2.txt               # changed
```

Note the restore applies full first, then each incremental in order.

7. **Back up a SQLite file safely (never `cp` an open database).**

```bash
python3 - <<'EOF'
import sqlite3
src = sqlite3.connect('shop.db')
src.execute('create table if not exists t(a)')
src.executemany('insert into t values(?)', [(i,) for i in range(100)]); src.commit()
dst = sqlite3.connect('shop-backup.db'); src.backup(dst); dst.close()   # online,
  consistent backup API
c = sqlite3.connect('shop-backup.db')
print(c.execute('select count(*) from t').fetchone(), c.execute('pragma
  integrity_check').fetchone())
EOF
```

Expected: `(100,) ('ok',)`.

8. **Implement retention: keep 7 daily and 5 weekly (Sunday) backups.**

```bash
cat > ~/dr-lab/prune.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
DIR=${1:?backup dir}
for f in "$DIR"/shop-*.dump.age; do
  d=$(basename "$f" | sed -E 's/shop-([0-9-]{10}).*/\1/')
  age_days=$(( ( $(date +%s) - $(date -d "$d" +%s) ) / 86400 ))
  dow=$(date -d "$d" +%u)                    # 7 = Sunday
  if [ "$age_days" -le 7 ] || { [ "$dow" = 7 ] && [ "$age_days" -le 35 ]; }; then
  continue; fi
  echo "prune $f"; rm -- "$f"
done
EOF
chmod +x ~/dr-lab/prune.sh
mkdir -p ~/dr-lab/ret && cd ~/dr-lab/ret
for d in $(seq 0 40); do touch -d "-$d day" "shop-$(date -d "-$d day" +%F).dump.age";
  done
~/dr-lab/prune.sh . | wc -l; ls | wc -l
```

Expected with 41 fake files: 29 pruned, 12 kept (7-8 daily plus the Sundays inside 35 days; exact counts depend on today's weekday).

9. **Clean up.**

```bash
pg_ctl -D "$W/pg" -m fast stop
```

7 Real-world scenario

Illustrative. The `shop-api` team runs a nightly `pg_dump` from cron to a disk attached to the same VM. A developer ships a migration with a missing `WHERE` clause and zeroes every customer's `email` column. Nobody notices for three days. The nightly dumps have a retention of 2 days, so every copy already contains the damage. The team can only recover by reconstructing emails from the replica's old WAL and a vendor's mailing list export. Afterwards they change three things: retention becomes 7 daily + 5 weekly + 12 monthly, dumps are copied off the host to a second account, and a weekly job restores the latest dump to a scratch database and compares row counts with production. **Lesson:** retention must exceed your *detection time*, and a backup on the same machine is a copy, not a backup.

8 Common mistakes and best practices

Don't

- Store backups on the same disk, host, cloud account or credentials as the original.
- Run `cp -r /var/lib/postgresql` on a live database and call it a backup.
- Treat "the cron job exited 0" as proof. Empty files and truncated dumps also exit 0 if you pipe carelessly (`set -o pipefail`!).
- Keep the encryption key only on the server it protects.
- Count a replica as a backup.

Do

- Follow 3-2-1 (and make one copy immutable or offline, see Off-site Backups §15.8).
- Alert on *backup age* (time since last successful backup), not only on job failure.
- Record the backup size, duration and checksum every run; sudden drops in size are an early warning.
- Restore regularly (Testing Backup Restoration §15.9); see also Backups and PITR in the databases level §13.5.
- Define retention from your detection time and compliance needs, then automate pruning.

9 Tasks

- [ ] **T1 — Logical vs physical, measured** *(≈30 min, beginner)*
  - **Goal:** see the size and time difference with your own numbers.
  - **Steps:**
    1. Follow steps 1-3 of the guide.
    2. Record dump size, base backup size and elapsed `time` for each in a small table.
    3. Restore the dump into `shop_restored` and note the restore time.
  - **Done when:** you have a 3-row table (dump.sql, dump.dump, base backup) with size and duration, and `count(*)` on the restored database is `200000`.
- [ ] **T2 — Build an encrypted, checksummed backup script** *(≈45 min, intermediate)*
  - **Goal:** one script that produces `shop-YYYY-MM-DD.dump.age` plus a `.sha256` file.
  - **Steps:**
    1. Write `backup.sh` with `set -euo pipefail` that runs `pg_dump -Fc | age -r "$PUB" > file.tmp`, then `mv`s it to the final name only on success.

2. Write the sha256 file next to it.
3. Run it twice on different days (use `faketime` or just rename) and run `prune.sh`.

- **Done when:** `sha256sum -c` says OK and decrypting then `pg_restore -l` lists the table of contents.

- [ ] **T3 — Incremental chain** *(≈30 min, intermediate)*
  - **Goal:** prove the restore order for incrementals.
  - **Steps:**
    1. Repeat step 6 and add a second incremental `incr-2.tar.gz` that deletes `f1.txt`.
    2. Restore with `--listed-incremental=/dev/null` applying full, incr-1, incr-2 in order.
    3. Then try applying incr-2 before incr-1 on a fresh directory and observe the difference.
  - **Done when:** `ls data` after the correct order shows `f2.txt f3.txt f4.txt`, and you can explain why the wrong order produces a different result.

- [ ] **T4 — Break it: silent failure** *(≈30 min, intermediate)*
  - **Goal:** see why `pipefail` and backup-age alerts matter.
  - **Steps:**
    1. Run `pg_dump -d nonexistent | age -r "$PUB" > bad.age` **without** `pipefail` and look at `echo $?` and the size of `bad.age`.
    2. Re-run with `set -o pipefail` and compare.
    3. Write a 5-line check that fails if the newest backup is older than 26 hours or smaller than 50 percent of the previous one.
  - **Done when:** your check script exits non-zero when you `touch -d "-2 days"` the newest backup.

- [ ] **T5 — Automate on a schedule** *(≈30 min, intermediate)*
  - **Goal:** run the backup nightly and record results.
  - **Steps:**
    1. Create a systemd timer or cron entry that runs `backup.sh`, then `prune.sh`, and appends one line (`date, size, sha256, seconds`) to `backup.log`.
    2. Trigger it manually twice.
  - **Done when:** `backup.log` has two lines with different timestamps and a non-zero size.

- [ ] **T6 — Prove it: design a 3-2-1 plan** *(≈45 min, advanced)*
  - **Goal:** justify every copy to a skeptical reviewer.
  - **Steps:**

1. Without the guide, write a one-page backup policy for `shop-api` listing what is backed up (database, uploads, Terraform state, secrets inventory), the type (logical/physical), frequency, retention, encryption and where each of the 3 copies lives.
2. State which threat each copy defends against (disk loss, operator error, account compromise, region loss).

- **Done when:** every threat in that list maps to at least one copy, and you can explain out loud why the replica is not one of the three.

10 Self-check questions

1. What does 3-2-1 stand for, and which failure does the "1 off-site" part protect against?
2. Why is an incremental backup faster to take but slower to restore than a full one?
3. When would you choose `pg_dump` over `pg_basebackup`, and vice versa?
4. Your backups are encrypted and the server was destroyed in a fire. What must you have stored elsewhere?
5. A bad migration corrupted data 10 days ago. Your retention is 7 days. What are your options now, and what do you change?

ANSWERS

1. Three copies, two media or systems, one off-site. It protects against site-level loss: fire, region outage, or an account compromise that wipes everything in one place.
2. It only reads and writes changed data, but a restore needs the full backup plus every incremental in sequence, so more files and more steps.
3. `pg_dump` for portability, upgrades across major versions and restoring a single table; `pg_basebackup` plus WAL archive for fast whole-cluster restore and point-in-time recovery.
4. The decryption key (or the KMS key and access to it), and the documented procedure and credentials to reach the backup storage.
5. Probably only partial repair (reconstruct from logs, replicas, exports). Lengthen retention to exceed your detection time and add a data-quality check that catches corruption sooner.

11 Further reading

- PostgreSQL documentation, *Backup and Restore* chapter (SQL dump, file system level backup, continuous archiving and PITR).
- pgBackRest and WAL-G project documentation.
- AWS documentation: *Backing up and restoring an Amazon RDS DB instance*.

- GitLab's public postmortem of the 31 January 2017 database incident.
- *Site Reliability Engineering* (Google), the chapter on data integrity.
- Related: Disaster Recovery overview §0.9, Backups and PITR §13.5.

15.2 Replication

Topic 2 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

Replication continuously copies changes from one system (the *primary*, or leader) to one or more others (*replicas*, or followers/standbys), so that more than one machine holds a current copy of the data. It exists to give you **availability** (a copy ready to take over) and **read scale** (replicas can serve queries).

Two properties define a replication setup:

- **Synchronous vs asynchronous.** In *synchronous* replication the primary does not tell the client "committed" until at least one replica has confirmed it received (or applied) the change. Data loss on primary failure is zero, but every write pays a network round trip and an unreachable replica can stall writes. In *asynchronous* replication the primary commits immediately and ships changes in the background: writes are fast, but anything not yet shipped is lost if the primary dies. That lag is your potential data loss (RPO §15.6).
- **Distance.** Replicating across **availability zones (AZs)** in one region has low latency (typically single-digit milliseconds) and is often synchronous. Replicating across **regions** has tens to hundreds of milliseconds of latency, so it is almost always asynchronous.

The sentence to tattoo on your runbook: **replication is not a backup.** A replica faithfully copies `DROP TABLE`, a buggy `UPDATE`, ransomware encryption and silent corruption. It protects against *hardware and site failure*, not against *mistakes and attacks*.

2 Why it matters in DevOps

- Replication is the mechanism behind fast recovery: promoting a warm replica takes seconds to minutes, restoring from backup takes minutes to hours (RTO §15.7).
- Choosing sync vs async is choosing between latency and data loss; you must be able to explain that trade-off to a product owner.

- Misconfigured replication causes the nastiest incidents: stalled writes because the sync replica vanished, replicas that fell silently hours behind, and "failover" to a replica that was missing the last 20 minutes of orders.
- Teams that believe "we have a replica, so we are backed up" learn the difference the hard way (Backups §15.1).

3 Real-world system example

Amazon RDS and Aurora. RDS *Multi-AZ* deployments keep a standby copy in a different AZ and replicate to it synchronously; on failure of the primary, RDS flips the DNS endpoint to the standby (AWS documents failover typically completing within a minute or two). RDS *read replicas*, including cross-region ones, use the engine's native asynchronous replication, and AWS documents that replica lag can occur. Amazon Aurora separates compute from a distributed storage layer that keeps six copies of the data across three AZs, and *Aurora Global Database* adds asynchronous cross-region replication for disaster recovery. S3 offers cross-region replication that is asynchronous; with Replication Time Control, AWS publishes a 15-minute target for the vast majority of objects, which tells you it is not instant.

PostgreSQL itself. The open-source default is *streaming replication* of the write-ahead log (WAL), asynchronous unless you set `synchronous_standby_names`. Large PostgreSQL deployments typically layer a failover manager (such as Patroni) on top and still keep separate WAL-archive backups, precisely because streaming replication copies mistakes too.

4 Diagram

Synchronous versus asynchronous commit:

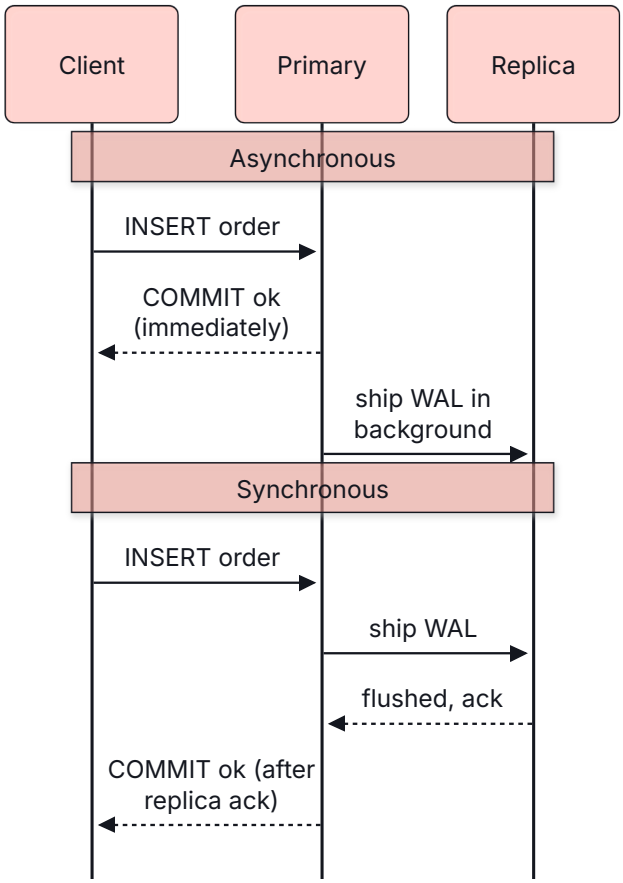


Figure 15.2-1

Why a replica is not a backup:

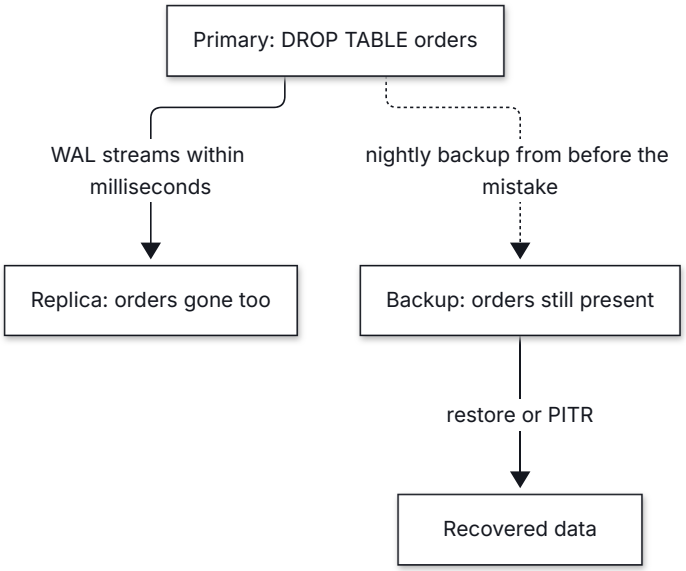


Figure 15.2-2

5 How it works

Mode	Data loss on primary failure	Write latency	Failure mode to watch
Asynchronous	up to the replication lag (RPO greater than 0)	no extra	silent lag growth, lost tail of transactions
Synchronous (1 standby)	none for acknowledged commits	+1 network round trip	writes stall if the sync standby is down
Quorum sync (any 1 of 2)	none for acknowledged commits	+round trip to the fastest replica	needs 2+ replicas, more cost
Semi-sync (MySQL style)	near zero, falls back to async on timeout	small	silently degrades to async

Scope	Typical latency	Typical mode	Protects against
Same AZ	sub-millisecond	either	server or process failure only
Cross-AZ, same region	low milliseconds	sync or async	AZ outage (power, network, fire in one data center)
Cross-region	tens to hundreds of ms	async	regional outage, large-scale disaster

PostgreSQL specifics worth knowing:

- Replica state is visible on the primary in `pg_stat_replication` (`state`, `sync_state`, `replay_lag`) and on the replica via `pg_is_in_recovery()` and `pg_last_wal_replay_lsn()`.
- A **replication slot** keeps WAL until the replica has consumed it; this avoids replica breakage but a dead replica makes the primary's disk fill with WAL. Monitor slot lag.
- Logical replication (publications/subscriptions) replicates row changes per table and allows different versions, but does not carry DDL or sequences by default.
- Replica **delayed replication** (`recovery_min_apply_delay`) deliberately lags a standby, giving you a short "oops window", but it is a convenience, not a substitute for backups.

6 Step-by-step guide

Prerequisites: PostgreSQL 16 server binaries (`sudo apt-get install -y postgresql-16`, binaries in `/usr/lib/postgresql/16/bin`). Everything runs on one machine with two throwaway clusters on ports 55440 (primary) and 55441 (replica), unix sockets only, free.

1. **Create the primary.**

```bash
export PATH=/usr/lib/postgresql/16/bin:$PATH
export W=$HOME/dr-lab/repl && rm -rf "$W" && mkdir -p "$W" && export PGHOST=/tmp
initdb -D "$W/primary" >/dev/null
cat >> "$W/primary/postgresql.conf" <<'CONF'
port = 55440
unix_socket_directories = '/tmp'
listen_addresses = ''
wal_level = replica
max_wal_senders = 5
hot_standby = on
CONF
echo "local replication all trust" >> "$W/primary/pg_hba.conf"
pg_ctl -D "$W/primary" -l "$W/primary.log" -w start
psql -X -q -p 55440 -d postgres -c "CREATE DATABASE shop"
psql -X -q -p 55440 -d shop -c "CREATE TABLE orders(id serial primary key, item text)"
  \
  -c "INSERT INTO orders(item) SELECT 'i'||g FROM generate_series(1,1000) g"
```

2. **Clone a replica with** `pg_basebackup -R` (`-R` writes `standby.signal` and the connection

string).

```bash
pg_basebackup -h /tmp -p 55440 -D "$W/replica" -R -X stream -c fast
sed -i 's/^port = 55440/port = 55441/' "$W/replica/postgresql.conf"
pg_ctl -D "$W/replica" -l "$W/replica.log" -w start
```

3. **Observe the replication state from both sides.**

```bash
psql -X -p 55440 -d postgres -x -c "SELECT application_name, state, sync_state,
  replay_lag FROM pg_stat_replication"
psql -X -p 55441 -d shop -At -c "SELECT pg_is_in_recovery(), count(*) FROM orders"
```

Expected: `state | streaming`, `sync_state | async`, a tiny `replay_lag` (microseconds), and `t|1000` on the replica.

4. **Prove the replica is read-only.**

```bash
psql -X -p 55441 -d shop -c "INSERT INTO orders(item) VALUES ('x')"
```

Expected: `ERROR: cannot execute INSERT in a read-only transaction`.

5. **Switch to synchronous replication.**

```bash
psql -X -p 55440 -d postgres -c "ALTER SYSTEM SET synchronous_standby_names =
  'walreceiver'" -c "SELECT pg_reload_conf()"
sleep 1
psql -X -p 55440 -d postgres -At -c "SELECT application_name, sync_state FROM
  pg_stat_replication"
```

Expected: `walreceiver|sync`. (`walreceiver` is the default application name of a standby created with `-R`; in production set an explicit `application_name` in `primary_conninfo`.)

6. **See the cost of synchronous replication: stop the replica and try to write.**

```bash
psql -X -q -p 55440 -d shop -c "CREATE TABLE t(a int)"
pg_ctl -D "$W/replica" -m fast stop
time timeout 5 psql -X -p 55440 -d shop -c "INSERT INTO t VALUES (1)"; echo "exit=$?"
```

Expected: the insert **hangs** until the timeout (`exit=124`, about 5 seconds) because no synchronous standby can acknowledge. This is the availability price of zero data loss. Restart the replica and the write completes: `pg_ctl -D "$W/replica" -l "$W/replica.log" -w start`. (Use `Ctrl+C` instead of the timeout if you like; rows only become visible after replication confirms.)

7. **Prove "replication is not a backup".**

```bash
psql -X -p 55440 -d shop -c "DROP TABLE orders"
sleep 1
psql -X -p 55441 -d shop -c "SELECT count(*) FROM orders"
```

Expected: `ERROR: relation "orders" does not exist` on the replica too. The mistake travelled in milliseconds.

8. **Clean up.**

```bash
pg_ctl -D "$W/replica" -m fast stop; pg_ctl -D "$W/primary" -m fast stop
```

AWS equivalent (read-only reference, costs money if you actually create it): `aws rds create-db-instance-read-replica --source-db-instance-identifier shop-api-db --db-instance-identifier shop-api-db-replica` creates an asynchronous replica; adding `--multi-az` on the primary gives the synchronous standby. A cross-region replica needs `--region` pointing at the target and `--source-region`. Always set a billing alarm and delete the instances afterwards.

7 Real-world scenario

Illustrative. `shop-api` runs a PostgreSQL primary in `us-east-1a` with an asynchronous replica in `us-east-1b`. During a deploy, a script truncates the `payments` table in production instead of staging. Monitoring shows the replica has `payments` empty within a second. The on-call engineer's first instinct, "fail over to the replica", would change nothing. They instead restore a base backup to a scratch instance, replay WAL up to one minute before the truncate (point-in-time recovery, see Restore Procedures §15.4), and copy the table back. A second, different night, the primary's host dies. The replica was 40 seconds behind, so about 40 seconds of orders are lost; the team now knows exactly what their RPO is in practice and decides whether that is acceptable or whether they need synchronous replication. **Lesson:** replication answers "the machine died"; backups answer "we did something wrong". You need both.

8 Common mistakes and best practices

Don't

- Call a replica your backup or skip backups because you "have HA".
- Use synchronous replication with a single standby and no plan for what happens when that standby is down.
- Ignore replica lag until failover time. A lagging replica fails over to stale data.
- Let a dead replica's replication slot pin WAL until the primary disk fills.
- Replicate to a node with the same credentials, same account and same blast radius if you want it to survive attacks.

Do

- Alert on replica lag in seconds and bytes, and on `state ≠ streaming`.
- Decide per data class: money and orders may justify sync, analytics and sessions usually do not.
- Keep a separate backup path with PITR (Backups §15.1) and replicate across a failure domain you actually fear (AZ, region).
- Use quorum sync (`ANY 1 (a, b)`) when you need zero data loss without a single-replica stall.
- Practice promotion regularly (Failover §15.5); see also Replication in the databases level §13.4.

9 Tasks

- [ ] **T1 — Build and inspect a replica** *(≈30 min, beginner)*
  - **Goal:** a working primary and streaming replica on your machine.
  - **Steps:** follow guide steps 1-3.
  - **Done when:** `pg_stat_replication` shows `streaming`, and the replica returns the same `count(*)` as the primary.
- [ ] **T2 — Measure lag under load** *(≈30 min, intermediate)*
  - **Goal:** see asynchronous lag as a number.
  - **Steps:**
    1. Run `pgbench -i -s 20 -h /tmp -p 55440 shop` then `pgbench -T 20 -c 8 -h /tmp -p 55440 shop` (pgbench ships with the PostgreSQL packages).
    2. During the run, poll `SELECT application_name, write_lag, flush_lag, replay_lag FROM pg_stat_replication` every second on the primary.

- **Done when:** you captured a peak `replay_lag` value and can say how much data (in transactions per second times seconds) you could have lost at that instant.

- [ ] **T3 — Sync versus async cost** *(≈30 min, intermediate)*
  - **Goal:** quantify what zero data loss costs.
  - **Steps:**
    1. Run `pgbench -T 15 -c 4 -h /tmp -p 55440 shop` with async, record transactions per second (TPS) and latency average.
    2. Enable `synchronous_standby_names = 'walreceiver'`, reload, repeat.
  - **Done when:** you have a two-row table (async vs sync, TPS, latency) and a sentence explaining the difference.

- [ ] **T4 — Break it: the stalled sync write** *(≈20 min, intermediate)*
  - **Goal:** experience and fix the sync stall.
  - **Steps:** follow guide step 6; then recover without restarting the replica by running `ALTER SYSTEM RESET synchronous_standby_names` and `SELECT pg_reload_conf()` on the primary.
  - **Done when:** the blocked insert completes and you can state the availability trade-off of the setting you just reverted.

- [ ] **T5 — Disprove "replica equals backup"** *(≈30 min, intermediate)*
  - **Goal:** evidence you can show a skeptical manager.
  - **Steps:**
    1. Take a base backup (`pg_basebackup -D "$W/b" -Ft -z`) first.
    2. Run `DELETE FROM orders WHERE id % 2 = 0` on the primary, wait one second, and count rows on the replica.
    3. Write down what you would restore from, and from where.
  - **Done when:** the replica shows half the rows, and your note names the backup as the recovery source.

- [ ] **T6 — Prove it: choose a replication design** *(≈40 min, advanced)*
  - **Goal:** justify a design for `shop-api`.
  - **Steps:** Without the guide, write half a page covering sync vs async for orders, cross-AZ vs cross-region, expected lag, what happens when the replica is down, and what *else* you still need (backups).
  - **Done when:** each claim has a stated failure it addresses, and you can explain it to a teammate in under five minutes.

10 Self-check questions

1. What exactly do you lose on an asynchronous primary failure, and how do you measure it beforehand?
2. Why can a synchronous standby reduce availability?
3. Why is cross-region replication almost always asynchronous?
4. A developer runs `DELETE FROM users` by mistake. Does your replica help? Why or why not?
5. What can go wrong with replication slots if a replica disappears?

ANSWERS

1. Transactions committed on the primary but not yet received/applied by the replica. Measure replication lag (`replay_lag` , bytes behind) and its peak under load.
2. The primary waits for the standby's acknowledgement; if the standby is down or partitioned, commits block (unless you use quorum or fall back).
3. The speed of light: tens to hundreds of ms per round trip on every commit would make writes slow; so you accept some lag.
4. No. The delete is replicated almost instantly. You need a backup or PITR to recover.
5. The primary retains WAL for the slot indefinitely, which can fill the disk and take the primary down. Monitor and cap slot retention.

11 Further reading

- PostgreSQL documentation, *High Availability, Load Balancing, and Replication* (streaming replication, synchronous replication, replication slots).
- Amazon RDS documentation on Multi-AZ deployments and read replicas; Aurora Global Database documentation.
- *Designing Data-Intensive Applications* by Martin Kleppmann, the chapter on replication.
- Related: High Availability and Scalability §0.8, Replication in the databases level §13.4.

15.3 Snapshots

Topic 3 of 10 | ~3 hours | Difficulty: Intermediate

1 What it is

A **snapshot** is a point-in-time image of a storage volume (or a whole managed database), taken by the storage layer itself rather than by reading every file through the application. Because the storage system only has to remember "the state of the blocks as of this instant", a snapshot is created in seconds regardless of volume size, and on cloud platforms it is stored incrementally: only blocks that changed since the previous snapshot cost extra space.

Two properties decide whether a snapshot is trustworthy:

- **Consistency level.** A *crash-consistent* snapshot captures what is on disk at that instant, exactly as if the power had been pulled. Journaling filesystems and databases with a write-ahead log (PostgreSQL, MySQL/InnoDB) recover from this on start-up. An *application-consistent* snapshot additionally asks the application to flush and pause first (for example with a database backup mode, `fsfreeze`, or VSS on Windows), so recovery needs no crash replay.
- **Where it lives.** A snapshot that sits next to the volume in the same region and account protects against deleting the volume, not against losing the region or the account. Copying it elsewhere is a separate step (Off-site Backups §15.8).

2 Why it matters in DevOps

- Snapshots are the cheapest, fastest "backup" you can automate on a cloud platform, and they make restores fast because you attach a new volume instead of replaying data.
- They are also easy to get wrong: a snapshot of a volume under heavy writes with several disks (data + WAL on separate volumes) can be internally inconsistent.
- Lifecycle management matters: snapshots that are never deleted quietly add to the bill; snapshots that are deleted too soon leave you without a restore point.
- Encrypted snapshots tie your recovery to a KMS key. If the key (or its permissions) is lost, so is the snapshot.

3 Real-world system example

Amazon EBS and RDS. EBS snapshots are incremental, are stored durably by AWS (in S3-backed storage you do not manage directly), and can be copied across regions and shared across accounts. A volume restored from a snapshot is usable immediately but blocks are loaded lazily from storage in the background, so first reads can be slower unless you use Fast Snapshot Restore (an extra-cost feature). EBS snapshots of a single volume are crash-consistent; AWS offers a multi-volume snapshot call so that all volumes attached to an instance are captured at the same moment. Amazon Data Lifecycle Manager (DLM) automates creation, retention and cross-region copy on a schedule. RDS builds on the same idea: *automated backups* (daily snapshot plus transaction logs for point-in-time recovery, retention 1-35 days) and *manual snapshots*, which you control and which are kept until you delete them. Restoring either type always creates a **new** DB instance with a **new endpoint**; nothing is rolled back in place.

Kubernetes. The CSI snapshot API (`VolumeSnapshot`, `VolumeSnapshotClass`) exposes the same storage-layer snapshots to cluster operators, and tools such as Velero use them to back up persistent volumes alongside cluster objects. A CSI driver must support snapshots and the snapshot controller must be installed; a plain kind cluster does not provide this out of the box.

4 Diagram

How an incremental snapshot chain looks, and why deleting an old snapshot is safe:

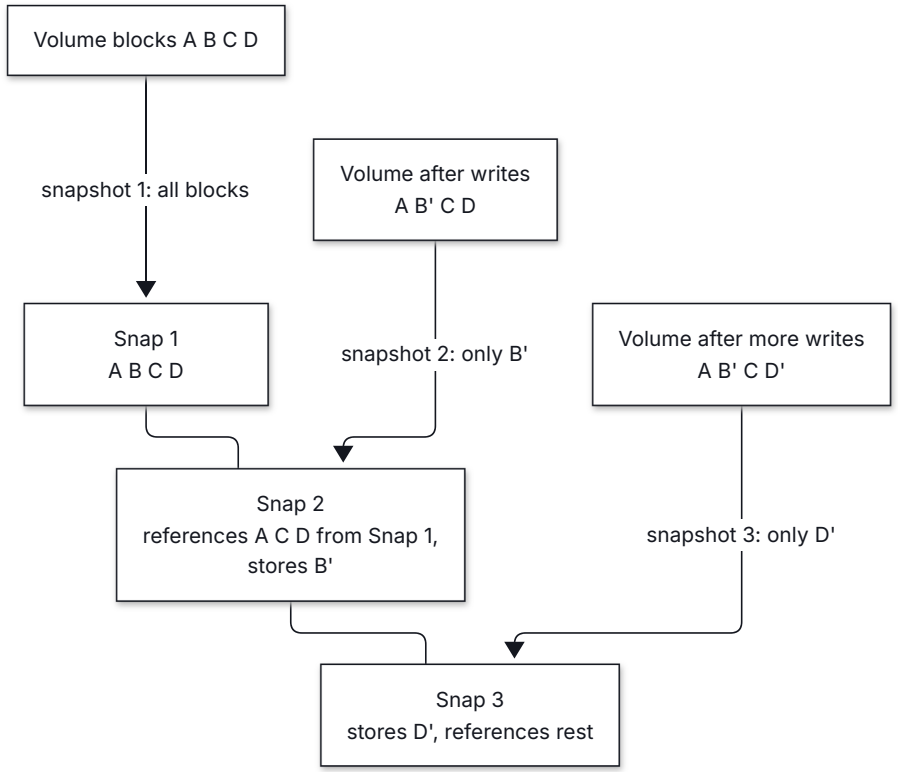


Figure 15.3-1

Cross-region and cross-account protection of a snapshot:

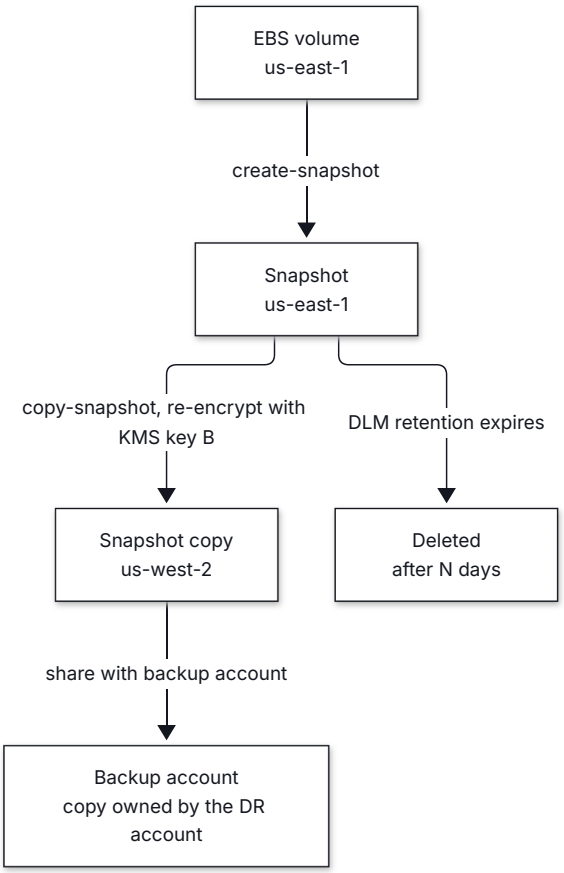


Figure 15.3-2

5 How it works

Question	Crash-consistent	Application-consistent
What is captured	disk state at the instant	disk state after the app flushed/quiesced
Recovery on restore	filesystem journal and database WAL replay	usually none or minimal
Risk	multi-volume skew; apps without a log (some caches, plain files being written)	brief write pause or backup-mode complexity
Typical method	EBS snapshot, `fsfreeze`-less cloud snapshot	`pg_backup_start` / `pg_backup_stop` around the snapshot, `fsfreeze`, VSS

Rules of thumb for databases:

- **One volume for everything** (data + WAL) gives a crash-consistent snapshot you can usually trust. If data and WAL live on different volumes, use a multi-volume snapshot call or bracket the snapshots with the database's backup mode.
- A snapshot gives you **one point in time**. For anything finer, combine it with continuous log archiving (point-in-time recovery). RDS does this for you.
- **Test the restore**: attach the volume to a scratch instance and start the database; check the logs for recovery messages and run integrity queries (Testing Backup Restoration §15.9).

Lifecycle concern	What to decide
Frequency	derived from your RPO (RPO §15.6)
Retention	count or days; separate rules for hourly, daily, monthly
Cross-region copy	which snapshots, to which region, with which KMS key
Cross-account copy	a second account the production credentials cannot delete from
Deletion protection	recycle bin / retention lock on snapshots, tags that mark "do not delete"
Cost	incremental storage per GB-month; copies are billed separately; data transfer for cross-region copies

6 Step-by-step guide

Part A: a local, free crash-consistent snapshot of a live database. We use a loop-mounted ext4 volume as a stand-in for an EBS volume, freeze the filesystem for a moment, copy the block image (our "snapshot"), and prove that PostgreSQL started from the copy recovers cleanly.

Prerequisites: Linux with root (`sudo`), PostgreSQL 16 server binaries (`sudo apt-get install -y postgresql-16`), `fsfreeze` (util-linux, preinstalled). This needs loop-device support, which a normal VM or VPS has; some restricted containers do not. Uses ~300 MB of disk and port 55450.

1. **Create and mount a 300 MB "volume".**

```bash
sudo -i
export PATH=/usr/lib/postgresql/16/bin:$PATH
D=/tmp/snaplab; mkdir -p $D/vol $D/snapvol; chmod 777 $D
truncate -s 300M $D/vol.img && mkfs.ext4 -q $D/vol.img
mount -o loop $D/vol.img $D/vol
mkdir $D/vol/pgdata && chown postgres:postgres $D/vol/pgdata && chmod 700
  $D/vol/pgdata
```

2. **Initialise PostgreSQL on that volume and generate write load.**

```bash
as() { su postgres -s /bin/bash -c "export PATH=$PATH PGHOST=/tmp; $*"; }
as "initdb -D $D/vol/pgdata >/dev/null"
as "printf \"port = 55450\nunix_socket_directories = '/tmp'\nlisten_addresses = ''\n\"
  >> $D/vol/pgdata/postgresql.conf"
as "pg_ctl -D $D/vol/pgdata -l $D/pg.log -w start"
as "createdb -p 55450 shop; pgbench -i -s 5 -p 55450 shop"
as "pgbench -T 8 -c 4 -p 55450 shop" > $D/bench.out 2>&1 &
sleep 3
```

`pgbench` keeps inserting into `pgbench_history` for 8 seconds, so the snapshot is taken **while the database is busy**.

3. **Take the snapshot: freeze, copy, thaw.**

```bash
time ( fsfreeze -f $D/vol; cp --sparse=always $D/vol.img $D/snap.img; fsfreeze -u
  $D/vol )
wait
as "psql -p 55450 -At -d shop -c 'select count(*) from pgbench_history'"
```

Similar to: about a second of freeze (limited by copying a small image; a real cloud snapshot returns immediately), and a final live row count such as `15214`. Always run `fsfreeze -u` even on error; a frozen filesystem blocks every write on it.

4. **Stop production, mount the snapshot, and start PostgreSQL from it.**

```bash
as "pg_ctl -D $D/vol/pgdata -m fast stop"
mount -o loop $D/snap.img $D/snapvol
rm -f $D/snapvol/pgdata/postmaster.pid
sed -i 's/^port = 55450/port = 55451/' $D/snapvol/pgdata/postgresql.conf
as "pg_ctl -D $D/snapvol/pgdata -l $D/snap.log -w start"
as "psql -p 55451 -At -d shop -c 'select count(*) from pgbench_history'"
grep -E "not properly shut down|automatic recovery|redo done|ready to accept"
  $D/snap.log
```

   Similar to: a **smaller** count than production (for example `7844`, the state at the moment of the snapshot) and log lines `database system was not properly shut down; automatic recovery in progress`, `redo done at ...`, `database system is ready to accept connections`. That is exactly what a crash-consistent snapshot looks like: valid, internally consistent, but frozen at one instant, with WAL replay on start.

5. **Clean up.**

```bash
as "pg_ctl -D $D/snapvol/pgdata -m fast stop"
umount $D/snapvol $D/vol && rm -rf $D && exit
```

Part B: the same on AWS (read-only reference; creating these costs money). Replace placeholders; set a billing alarm first, and run the cleanup at the end.

```bash
# Snapshot every volume on an instance at the same instant (crash-consistent across
  volumes)
aws ec2 create-snapshots \
  --instance-specification InstanceId=i-0123456789abcdef0,ExcludeBootVolume=true \
  --description "shop-api db 2026-10-09" \
  --tag-specifications 'ResourceType=snapshot,Tags=[{Key=app,Value=shop-api},
  {Key=retention,Value=daily}]'

# Copy to another region (for an encrypted snapshot, pass a KMS key that exists in the
  destination region)
aws ec2 copy-snapshot --region us-west-2 --source-region us-east-1 \
  --source-snapshot-id snap-0123456789abcdef0 --encrypted --kms-key-id alias/shop-dr

# RDS: manual snapshot, cross-region copy, restore as a NEW instance
aws rds create-db-snapshot --db-instance-identifier shop-api-db --db-snapshot-identifier
  shop-api-db-2026-10-09
aws rds copy-db-snapshot --region us-west-2 --source-region us-east-1 \
  --source-db-snapshot-identifier arn:aws:rds:us-east-1:111122223333:snapshot:shop-api-db-
  2026-10-09 \
  --target-db-snapshot-identifier shop-api-db-2026-10-09-dr --kms-key-id alias/shop-dr
```

```
aws rds restore-db-instance-from-db-snapshot --region us-west-2 \
  --db-instance-identifier shop-api-db-drill --db-snapshot-identifier shop-api-db-2026-10-
  09-dr \
  --db-instance-class db.t3.micro
```

Illustrative: output is JSON describing the new resource; exact fields vary. Automate retention with a Data Lifecycle Manager policy (`aws dlm create-lifecycle-policy ...`) or AWS Backup, then **clean up**: `aws rds delete-db-instance --db-instance-identifier shop-api-db-drill --skip-final-snapshot`, `aws rds delete-db-snapshot ...`, `aws ec2 delete-snapshot --snapshot-id ...` in both regions. Snapshots keep billing until deleted.

7 Real-world scenario

Illustrative. The `shop-api` team snapshots the database volume nightly with a cron-triggered CLI call and keeps seven. PostgreSQL's data directory is on `/dev/nvme1n1` and its WAL on `/dev/nvme2n1`, snapshotted by two separate calls a few seconds apart. During a drill the team restores both snapshots to a new instance: PostgreSQL refuses to start with `invalid checkpoint record` because the WAL snapshot is newer than the data snapshot. They switch to the multi-volume `create-snapshots` call, add a restore check that starts PostgreSQL from the snapshot and runs `SELECT count(*)`, and add cross-region copy for the weekly snapshot. **Lesson:** a snapshot you have not restored is a guess, and multi-volume consistency is a design decision, not an afterthought.

8 Common mistakes and best practices

Don't

- Snapshot a multi-volume database with independent calls at different times.
- Leave snapshots only in the account and region that production lives in.
- Forget that snapshots of encrypted volumes need access to the KMS key at restore time.
- Treat a snapshot as an in-place undo: on RDS and EBS a restore creates a new resource, and you must repoint the application.
- Let snapshots accumulate forever (cost) or expire after fewer days than your detection time (risk).

Do

- Use lifecycle automation (DLM, AWS Backup, or an equivalent) with explicit retention and tags.
- Combine snapshots with log archiving when your RPO is smaller than the snapshot interval.

- Copy critical snapshots to a second region *and* a second account; protect them from deletion (Off-site Backups §15.8).
- Restore a snapshot on a schedule and record the time it took (RTO §15.7).
- Related reading: EBS and EFS §6.9, RDS and Aurora §6.10, KMS §6.15.

9 Tasks

- [ ] **T1 — Take and inspect a crash-consistent snapshot** *(≈40 min, intermediate)*
  - **Goal:** reproduce Part A and read the recovery log.
  - **Steps:** run guide steps 1-4 with `sudo`.
  - **Done when:** the restored count is lower than production's final count and `snap.log` contains `automatic recovery in progress`.
- [ ] **T2 — Snapshot without freezing** *(≈30 min, intermediate)*
  - **Goal:** see what freezing buys you.
  - **Steps:**
    1. Repeat the exercise but copy the image while the filesystem is **not** frozen (`cp` while `pgbench` runs).
    2. Try to start PostgreSQL from the copy; run `pg_amcheck -p 55451 shop` or just the count query.
  - **Done when:** you report whether it recovered, and explain why a non-atomic copy of a busy volume is not a snapshot. (It may happen to work; the point is that nothing guarantees it.)
- [ ] **T3 — Application-consistent variant** *(≈30 min, advanced)*
  - **Goal:** use the database's backup mode around the copy.
  - **Steps:**
    1. In one `psql` session run `SELECT pg_backup_start('snap', true);` and keep the session open.
    2. Freeze and copy the image from another terminal, thaw, then run `SELECT * FROM pg_backup_stop();`.
    3. Start the copy and compare the log with T1.
  - **Done when:** you can describe the difference in the startup messages and why backup mode requires the WAL generated during the copy to be kept.
- [ ] **T4 — Write the lifecycle policy** *(≈30 min, intermediate)*
  - **Goal:** a policy file others can review.

- **Steps:** Write a table for `shop-api` with columns: snapshot, frequency, retention, copy destination (region, account), KMS key, deletion protection. Include hourly, daily, weekly and monthly rows, derive frequency from an RPO of 1 hour.
- **Done when:** every row has a retention number and a copy destination, and the monthly cost estimate is stated (use the AWS pricing page; mark it an estimate).

☐ **T5 — AWS: snapshot, copy, restore, delete** *(≈60 min, intermediate, **costs money**)*

- **Goal:** run the Part B flow once on a tiny resource.
- **Steps:**
  1. Create a billing alarm. Create a 1 GiB gp3 volume (or a `db.t3.micro` RDS instance in the free tier if eligible).
  2. Snapshot it, copy the snapshot to `us-west-2`, create a volume from the copy.
  3. Delete the volume, both snapshots, and any instance.
- **Done when:** `aws ec2 describe-snapshots --owner-ids self` in both regions returns an empty list.

☐ **T6 — Prove it: explain consistency** *(≈30 min, advanced)*

- **Goal:** teach crash-consistent versus application-consistent.
- **Steps:** Without the guide, explain to a teammate (or in writing) what is on the disk at snapshot time, why PostgreSQL recovers from a crash-consistent snapshot, when it would not (multi-volume skew), and what you would do about it.
- **Done when:** your explanation includes one concrete failure message or log line and one mitigation.

10 Self-check questions

1. Why is an EBS snapshot fast and cheap even for a 2 TB volume?
2. What is the difference between crash-consistent and application-consistent?
3. Why does a database with data and WAL on separate volumes need extra care?
4. You restored an RDS snapshot. What is different about the resulting instance?
5. A snapshot cannot be restored in another region. Name two likely causes.

ANSWERS

1. It is incremental and copy-on-write at the storage layer: only blocks changed since the previous snapshot are stored, and creation just records a point in time.
2. Crash-consistent equals what the disk holds at that instant (like a power cut); application-consistent means the app flushed and quiesced first, so no recovery is required.

3. Two independent snapshots are taken at different instants, so data and WAL can disagree. Use a multi-volume snapshot or backup mode around the snapshots.
4. It is a new instance with a new endpoint, parameter and security group choices to reapply; the original is untouched. Applications must be repointed.
5. The KMS key for an encrypted snapshot does not exist or is not permitted in the destination region; or the snapshot was never copied there and the source region is down.

11 Further reading

- AWS documentation: *Amazon EBS snapshots*, *Amazon Data Lifecycle Manager*, *Backing up and restoring an Amazon RDS DB instance* and *AWS Backup*.
- Kubernetes documentation: *Volume Snapshots* and the CSI snapshot controller; Velero documentation.
- PostgreSQL documentation, *Continuous Archiving and Point-in-Time Recovery (PITR)*, section on low-level base backups and snapshots.
- Related: Backups §15.1, EBS and EFS §6.9, RDS and Aurora §6.10.

LAB NOTES What I ran, what I saw, what it means

15.4 Restore Procedures

Topic 4 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

A **restore procedure** is the written, rehearsed sequence of steps that turns "we have backups" into "the service is running again with correct data". It has three parts: a **runbook** (the documented steps, with owners, commands and decision points), an **order of restore** (what must come back first because other things depend on it) and **validation** (evidence that what came back is complete and correct, not merely "the process started").

A backup is only half of recovery. The other half is knowing, under stress and possibly at 3 a.m. with the usual tools unavailable, exactly which command to run next. A good runbook is boring: numbered steps, copy-pasteable commands, expected output after each step, and a clear "stop and escalate if you see X".

2 Why it matters in DevOps

- **Under pressure people skip steps and guess.** A runbook removes the need to remember. The person restoring may not be the person who built the system.
- **Your RTO is the sum of the steps** (RTO §15.7). You can only measure and shrink it if the steps are written down and timed.
- **Dependencies hide.** Restoring the database before the secrets store, the KMS key or the network means a restored database that nobody can reach or decrypt.
- **"Restored" is not "correct".** A restore that finishes with exit code 0 can still be missing hours of data, have broken constraints or point at the wrong environment. Validation is what lets you tell the business "we are back" with a straight face.
- **The runbook must survive the disaster.** If it lives only in the wiki that runs on the failed cluster, it is not available when needed.

3 Real-world system example

GitLab.com, January 2017. The public postmortem of the production database incident shows the restore side of the story as clearly as the backup side: after the data directory was removed on the wrong host, the team had to work out, live, which of several backup mechanisms was usable, copy a large amount of data from a staging server, and wait for it to transfer and load. GitLab published a running log and live-streamed the recovery. A frequently drawn lesson is that recovery time was dominated by copying and loading data on a path nobody had rehearsed, and by uncertainty about which copy to trust, not by the restore command itself.

Google SRE practice. The Google *Site Reliability Engineering* books describe treating restores as something you exercise, since data that cannot be recovered is effectively lost. The same books describe the discipline of playbooks (runbooks) linked from alerts so on-call engineers do not have to invent a response. Most mature operations teams follow this pattern: each page has a runbook, each runbook has been executed by someone other than its author.

AWS RDS. Restoring an RDS snapshot or point-in-time creates a *new* instance with a *new endpoint*; the application must be repointed (or DNS/Route 53 updated) and parameter groups, security groups and monitoring re-attached. Teams that only practise "click restore" discover this dependency list during the real event.

4 Diagram

Order of restore: things on the left must exist before things on the right can work.

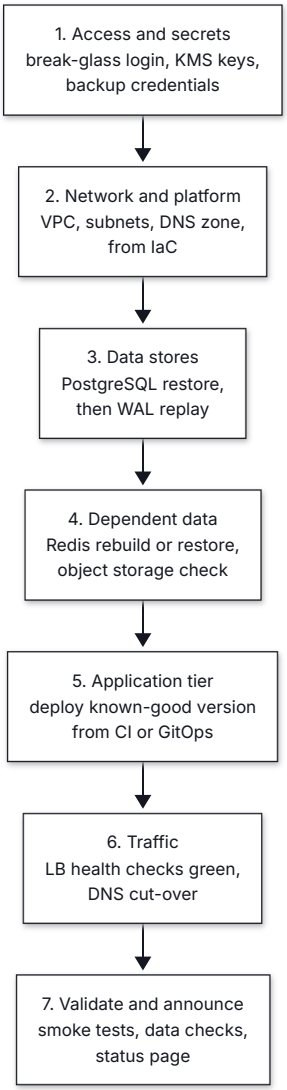


Figure 15.4-1

The restore-and-validate loop with a decision point:

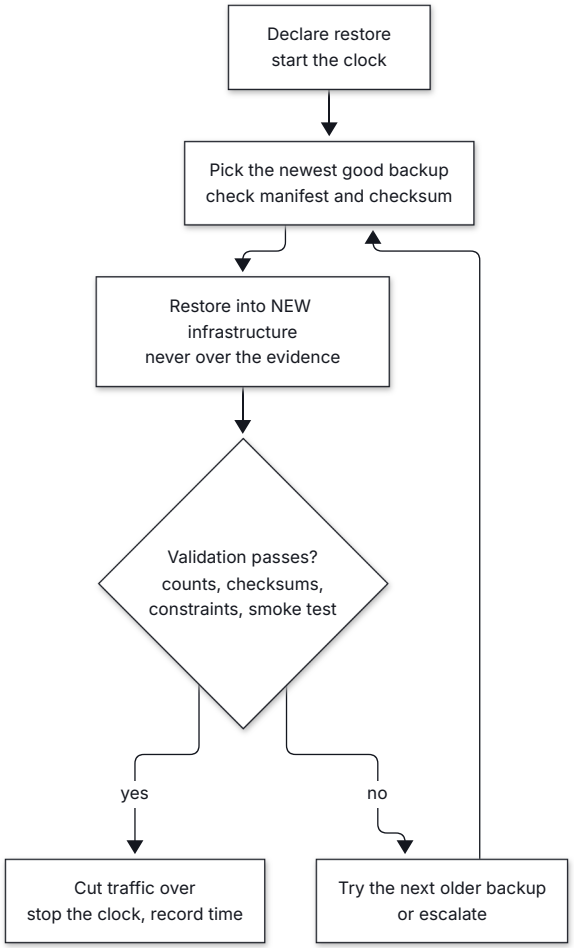


Figure 15.4-2

5 How it works

What a runbook contains

Element	Why
Trigger and scope	"Use this when: primary DB lost and replica unusable." Prevents using the wrong runbook.
Roles	Incident commander, restorer, validator, communicator. Restoring and deciding are different jobs.

Element	Why
Prerequisites	Where credentials, keys and the backup location are, and how to get them if SSO is down.
Numbered steps with expected output	Each command followed by "you should see ...".
Decision points	"If the newest backup fails the checksum, use the previous one; if both fail, escalate."
Validation checklist	Objective checks, not "looks fine".
Rollback / abort	How to back out of a half-finished restore.
Time estimates	Measured during drills, not guessed.
Last tested date and by whom	A runbook untested for a year is suspect.

Order of restore (typical). Restore in dependency order: (1) the ability to act (identity, break-glass accounts, key management, backup access), (2) the platform (network, DNS, compute, built from Infrastructure as Code, see Infrastructure as Code §0.3), (3) the system of record (the primary database), (4) derived or secondary data (caches, search indexes, which can often be rebuilt rather than restored), (5) stateless applications deployed from a known-good version, (6) traffic, (7) validation and communication. Restore the **system of record first**, rebuild the **derived data last**.

Restore into new infrastructure. Restoring over the damaged system destroys evidence and removes your fallback. Build the replacement beside it, validate, then switch traffic.

Validation layers

Layer	Example check
Backup integrity	`sha256sum -c`, `pg_verifybackup`, decryption succeeds
Completeness	row counts per table, `max(created_at)` close to the incident time, a stored "canary" total (sum of a column) recorded at backup time
Structural integrity	`pg_amcheck`, foreign keys valid (no orphans), unique constraints present, sequences ahead of data
Application	smoke test: log in, read an order, write an order and delete it
Business	a human spot-checks a known customer or yesterday's revenue figure
Security	restored environment has the right credentials, is not exposed publicly, and does not send real emails to real customers from a test restore

Common restore pitfalls: restoring a staging snapshot into production by name, sequences not advanced (duplicate key errors after cut-over), restored database accepting writes before validation completes, forgetting that replicas and backups must be re-established after the restore (your new primary has no backups until you start them again), and restored job queues re-sending old emails.

6 Step-by-step guide

Prerequisites: Linux with PostgreSQL 16 server binaries in `/usr/lib/postgresql/16/bin`. `initdb` refuses to run as root, so use your normal user, or `sudo -u postgres -i` on a server where you are root. Everything is local and free. We use port 55434 and a scratch directory; nothing existing is touched.

1. **Build a "production" database with a recorded canary value.**

```bash
export PATH=/usr/lib/postgresql/16/bin:$PATH
export W=$HOME/dr-lab/restore-demo && rm -rf "$W" && mkdir -p "$W/backups"
initdb -D "$W/pg" >/dev/null
printf "port = 55434\nunix_socket_directories = '/tmp'\nlisten_addresses = ''\n" >>
  "$W/pg/postgresql.conf"
pg_ctl -D "$W/pg" -l "$W/pg.log" -w start
export PGHOST=/tmp PGPORT=55434
createdb shop
psql -X -q -d shop <<'SQL'
CREATE TABLE customers(id serial PRIMARY KEY, email text UNIQUE NOT NULL);
CREATE TABLE orders(id serial PRIMARY KEY, customer_id int NOT NULL REFERENCES
  customers(id),
                    total_cents int NOT NULL, created_at timestamptz DEFAULT now());
INSERT INTO customers(email) SELECT 'user'||g||'@example.com' FROM
  generate_series(1,5000) g;
INSERT INTO orders(customer_id,total_cents) SELECT 1+g%5000, 100+g%9000 FROM
  generate_series(1,100000) g;
SQL
```

   Expected: `server started`, then no errors.

2. **Take the backup and record what "correct" looks like.** The canary is stored *with the backup*, not in the database that may be lost.

```bash
pg_dump -Fc -d shop -f "$W/backups/shop.dump"
psql -X -At -d shop -c "SELECT count(*), sum(total_cents) FROM orders" | tee
  "$W/backups/expected.txt"
sha256sum "$W/backups/shop.dump" > "$W/backups/shop.dump.sha256"
```

Expected: `100000|455951000`.

3. **Write the restore runbook as a script (executable documentation).**

```bash
cat > "$W/restore.sh" <<'EOF'
#!/usr/bin/env bash
# restore.sh - runbook steps for the shop database. Usage: restore.sh <work-dir>
set -euo pipefail
export PATH=/usr/lib/postgresql/16/bin:$PATH
W=${1:?work dir}
export PGHOST=/tmp PGPORT=55434
log() { printf '%s  %s\n' "$(date +%T)" "$*" | tee -a "$W/restore.log"; }
T0=$(date +%s)
log "STEP 1 verify backup checksum"
(cd "$W/backups" && sha256sum -c shop.dump.sha256)
log "STEP 2 initialise a fresh cluster"
initdb -D "$W/pg" >/dev/null
printf "port = 55434\nunix_socket_directories = '/tmp'\nlisten_addresses = ''\n" >>
  "$W/pg/postgresql.conf"
pg_ctl -D "$W/pg" -l "$W/pg.log" -w start >/dev/null
log "STEP 3 create database and restore data, indexes and constraints"
createdb shop
pg_restore -d shop --no-owner --exit-on-error "$W/backups/shop.dump"
log "STEP 4 refresh planner statistics"
psql -X -q -d shop -c "ANALYZE"
log "DONE restore in $(( $(date +%s) - T0 )) seconds"
EOF
chmod +x "$W/restore.sh"
```

4. **Write the validation script.** It encodes the validation layers from section 5 and exits non-zero on the first failure.

```bash
cat > "$W/validate.sh" <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
export PATH=/usr/lib/postgresql/16/bin:$PATH
W=${1:?work dir}
export PGHOST=/tmp PGPORT=55434
q() { psql -X -At -d shop -c "$1"; }
fail() { echo "FAIL: $*"; exit 1; }

want=$(cat "$W/backups/expected.txt")
got=$(q "SELECT count(*), sum(total_cents) FROM orders")
[ "$got" = "$want" ] && echo "PASS row count and checksum: $got" || fail "orders $got
  ≠ $want"

orphans=$(q "SELECT count(*) FROM orders o LEFT JOIN customers c ON c.id=o.customer_id
  WHERE c.id IS NULL")
[ "$orphans" = 0 ] && echo "PASS no orphan orders" || fail "$orphans orphan orders"
```

```
seq_ok=$(q "SELECT (SELECT last_value FROM orders_id_seq) ⩾ (SELECT max(id) FROM
  orders)")
[ "$seq_ok" = t ] && echo "PASS sequence ahead of max(id)" || fail "sequence behind
  data"

q "CREATE EXTENSION IF NOT EXISTS amcheck" >/dev/null
pg_amcheck -d shop >/dev/null && echo "PASS pg_amcheck (heap and btree structure)" ||
  fail "pg_amcheck"

q "INSERT INTO customers(email) VALUES ('smoke-test@example.com')" >/dev/null
q "DELETE FROM customers WHERE email='smoke-test@example.com'" >/dev/null && echo
  "PASS write smoke test"
echo "ALL CHECKS PASSED"
EOF
chmod +x "$W/validate.sh"
```

5. **Cause the disaster, then restore against the clock.**

```bash
pg_ctl -D "$W/pg" -m immediate stop
rm -rf "$W/pg"                                  # the server's disk is gone
time "$W/restore.sh" "$W" && "$W/validate.sh" "$W"
```

   Similar to: `shop.dump: OK`, the four STEP lines, `DONE restore in 1 seconds` (a few seconds at most for this tiny data set), then five `PASS` lines and `ALL CHECKS PASSED`. The point is the *shape*: every step logged with a timestamp, and validation as a separate, objective gate.

6. **Prove the validation can fail.** Corrupt the restored data and re-run the gate.

```bash
psql -X -q -d shop -c "DELETE FROM orders WHERE id % 1000 = 0"
"$W/validate.sh" "$W" || echo "gate correctly refused (exit $?)"
```

   Expected: `FAIL: orders 99900|... ≠ 100000|455951000` and `gate correctly refused (exit 1)`. A check that never fails proves nothing.

7. **Write the runbook document.** Copy this skeleton into your wiki *and* into the repository

(so it exists when the wiki does not):

```markdown
# Runbook: restore shop-api database from backup
Owner: @db-team   Last drilled: YYYY-MM-DD by @name   Measured duration: NN min
Use when: primary PostgreSQL lost and replica unusable or also corrupted.
Do not use when: a replica is healthy (see failover runbook).
Roles: IC, restorer, validator, communicator.
1. Declare incident, start timer, post to #incidents.
2. Fetch break-glass credentials from <location>. Expected: can run `aws sts get-
   caller-identity`.
3. Pick newest backup, verify checksum. If fail, try previous. If two fail, escalate
   to <person>.
4. Provision replacement DB host from Terraform module <path>. Expected: instance
   healthy.
5. Run restore.sh. Expected: DONE restore in <time>.
6. Run validate.sh. All PASS required. Any FAIL: stop, do not cut over.
7. Repoint app config / DNS. Expected: /healthz returns 200 from all pods.
8. Re-enable backups and replication on the new primary. Expected: first backup
   success logged.
9. Stop timer, record RPO (newest data timestamp) and RTO, announce, schedule
   postmortem.
```

8. **Clean up.**

```bash
pg_ctl -D "$W/pg" -m fast stop
```

7 Real-world scenario

Illustrative. At 02:10 the `shop-api` primary database host in `us-east-1` dies and the replica is lagging by 40 minutes with replay paused (a maintenance setting nobody reverted). The on-call engineer opens the runbook. Step 2 (break-glass credentials) fails: the credentials are in a vault that requires the SSO provider, which authenticates through the same company VPN as the dead host's network. The restore is delayed 25 minutes until a second engineer retrieves the offline break-glass copy from the security team. The restore itself takes 18 minutes, but validation catches that the `orders_id_seq` sequence had been restored behind the data (a plain-SQL script had been used in an old runbook), which would have caused duplicate-key errors on the first checkout. They advance the sequence and cut over at 03:21. **Lessons:** the runbook must list and test *access* as step one, validation found a real defect, and the measured steps (25 + 18 + 15 minutes) become the honest RTO number for the next planning round.

8 Common mistakes and best practices

Don't

- Keep the only copy of the runbook in a system that depends on what failed.
- Restore over the damaged system. Build beside it and switch.
- Declare success when the restore command exits 0.
- Let the author be the only person who has ever followed the runbook.
- Forget to restart backups and replication on the restored primary.

Do

- Write runbooks as numbered steps with expected output and decision points; link them from the alert that pages.
- Restore in dependency order: access, platform, system of record, derived data, apps, traffic, validation.
- Store a canary (count, sum, checksum) with each backup and compare after restore.
- Time every step during drills and put the measured duration at the top of the runbook.
- Re-test after every significant change (new database version, new region, new backup tool). See Testing Backup Restoration §15.9 and Failure Scenario Drills §15.10.
- Keep a printed or offline copy of the first three steps (who to call, where the break-glass credentials are).

9 Tasks

- [ ] **T1 — Run the timed restore** *(≈30 min, beginner)*
  - **Goal:** execute the restore and validation scripts and record a baseline.
  - **Steps:**
    1. Follow steps 1-5 of the guide.
    2. Open `restore.log` and note the time of each STEP.
    3. Record the total in a small table (step, seconds).
  - **Done when:** `ALL CHECKS PASSED` is printed and you have a table with the duration of each step.
- [ ] **T2 — Scale the data and re-time it** *(≈30 min, intermediate)*
  - **Goal:** see how restore time grows with data.
  - **Steps:**

1. Increase the `orders` insert to 5,000,000 rows (update `expected.txt` accordingly), back up and restore again.
2. Compare the STEP durations with T1. Note which step grew most (usually `pg_restore`, mostly index builds).
3. Try `pg_restore -j 4` and compare.

- **Done when:** you can state, with numbers, how many MB/s your restore achieved and what parallelism bought you.

- [ ] **T3 — Break it: sequence and orphan traps** *(≈30 min, intermediate)*
  - **Goal:** make the validation script catch three different faults.
  - **Steps:**
    1. After a restore, run `SELECT setval('orders_id_seq', 1)` and run `validate.sh`.
    2. Drop the foreign key, delete a customer that has orders, re-run.
    3. Truncate the last 1,000 orders, re-run.
  - **Done when:** each fault produces a distinct `FAIL:` line and you restored the healthy state afterwards.

- [ ] **T4 — Add a layer: application-level smoke test** *(≈45 min, intermediate)*
  - **Goal:** extend validation beyond SQL.
  - **Steps:**
    1. Add a check to `validate.sh` that confirms the most recent `created_at` is within a tolerance you choose, and fails otherwise.
    2. Add a check that prints (not asserts) the number of rows per table, for the incident report.
    3. If you have the sample app, add a `curl` to its health endpoint.
  - **Done when:** the script fails when you restore an artificially old backup (create one earlier in the day and compare).

- [ ] **T5 — Turn it into a real runbook and swap roles** *(≈45 min, advanced)*
  - **Goal:** prove the runbook works for someone who did not write it.
  - **Steps:**
    1. Fill in the skeleton from step 7 for your own setup, including a "Do not use when" line.
    2. Give it (and only it) to a colleague or to yourself a week later; they delete the lab and restore from it.
    3. Write down every question they had to ask.

- **Done when:** the restorer succeeded with at most two clarifying questions and you updated the runbook to answer them.

☐ **T6 — Prove it: ordered restore plan for the whole stack** *(≈45 min, advanced)*

- **Goal:** explain the order of restore for `shop-api` without notes.
- **Steps:**
  1. List every component (Terraform state, KMS keys, secrets, PostgreSQL, Redis, Node.js API, Nginx, DNS, monitoring).
  2. Draw the dependency graph and number the restore order; mark which items are restored and which are rebuilt.
  3. For each, write the validation check that proves it is back.
- **Done when:** a reviewer can pick any component and you can say what must exist before it, how you will know it is healthy, and who owns it.

10 Self-check questions

1. Why restore into new infrastructure instead of in place?
2. What belongs in a runbook besides the commands?
3. In what order would you restore access, network, database, cache, application and DNS, and why?
4. Give four validation checks that go beyond "the restore command succeeded".
5. After a successful restore, what must you do before you can call the system protected again?

ANSWERS

1. In-place restore destroys evidence for the postmortem and removes your fallback if the restore fails. New infrastructure lets you validate before switching traffic.
2. Trigger and scope, roles, prerequisites (how to get access), expected output after each step, decision points, validation checklist, abort path, measured timings and the date it was last drilled.
3. Access (break-glass and keys), network/platform, database (system of record), cache (often rebuilt), application, then traffic/DNS and finally validation and announcement. Each step depends on the previous; restoring the app before its data creates inconsistent state.
4. Row counts and a stored checksum, no orphan foreign keys, sequences ahead of max IDs, `pg_amcheck` or equivalent structural check, a write smoke test, freshness of the newest record.

5. Re-establish backups and replication on the new primary, confirm monitoring and alerts point to it, and record the measured RPO and RTO.

11 Further reading

- PostgreSQL documentation: *pg_restore*, *Continuous Archiving and Point-in-Time Recovery* and the `amcheck` module.
- *Site Reliability Engineering* and *The Site Reliability Workbook* (Google): chapters on data integrity, incident response and on-call playbooks.
- GitLab's public postmortem of the 31 January 2017 database incident.
- AWS documentation: *Restoring a DB instance to a specified time* (RDS).
- Related: Backups §15.1, Snapshots §15.3, Disaster Recovery overview §0.9, Backups and PITR §13.5.

LAB NOTES What I ran, what I saw, what it means

15.5 Failover

Topic 5 of 10 | ~4 hours | Difficulty: Advanced

1 What it is

Failover is moving work from a failed (or failing) component to a standby that can take over: a load balancer stops sending requests to a dead web server, a database replica is **promoted** to primary, or traffic for a whole region is redirected to another region. **Failback** is the later, deliberate return to the original arrangement once the failed component is healthy again.

Two decisions define a failover design. First, **who decides**: software (*automatic* failover) or a human following a runbook (*manual* failover). Second, **how clients find the new active node**: through a load balancer or proxy, a virtual IP, or a DNS change. Both choices trade speed against the risk of the worst failover outcome, **split-brain**: two nodes both believing they are the primary and both accepting writes, so the data diverges and one side's writes must eventually be thrown away.

2 Why it matters in DevOps

- **Failover is where HA either works or makes things worse.** A badly tuned automatic failover can turn a 5-second network blip into a 30-minute outage plus data loss.
- **It sets part of your RTO** (RTO §15.7): detection time + decision time + promotion time + client re-routing time. Each term can be measured and shrunk.
- **It sets part of your RPO** (RPO §15.6): promoting an asynchronous replica loses whatever had not replicated (Replication §15.2).
- **Untested failover does not work.** Configuration drift, expired credentials, a firewall rule that only the old primary had and an application that caches DNS for an hour are the usual reasons.
- **Failback is a second outage waiting to happen** if it is not planned: it needs the old node to be re-synchronised, not just switched on.

3 Real-world system example

Amazon RDS Multi-AZ. AWS keeps a synchronous standby in another Availability Zone and, on failure of the primary instance, automatically promotes the standby and flips the DB instance endpoint's DNS record to it. Clients must reconnect and must honour DNS changes (short TTL, no endless connection reuse); AWS documents failover as typically taking on the order of a minute or two, so check the current documentation for your engine. The application's job is to retry.

Patroni (PostgreSQL). The open-source Patroni project, used by many self-managed PostgreSQL platforms and by the Zalando Postgres Operator for Kubernetes, runs an agent beside each PostgreSQL node and uses a distributed consensus store (etcd, Consul or ZooKeeper, or the Kubernetes API) holding a **leader key with a lease**. The node that holds the lease is primary; if it cannot renew, it demotes itself, and a replica acquires the key and promotes. Because only one holder can own the lease, this is a defence against split-brain. Clients reach the primary through HAProxy health checks on Patroni's REST API or a Kubernetes service.

DNS and global load balancing. AWS Route 53 offers *failover routing* backed by health checks, and most large CDN and edge providers can steer traffic away from unhealthy origins. GitHub's 2018 outage is a widely cited public example of failover side effects: a brief network partition caused an automated system to promote database nodes in another data center, writes then existed on both sides, and reconciling them took many hours (GitHub published a detailed postmortem). The lesson is not "avoid automation" but "automatic failover must be designed around split-brain".

4 Diagram

Health-check failover through a load balancer, and the detection window where requests can still fail:

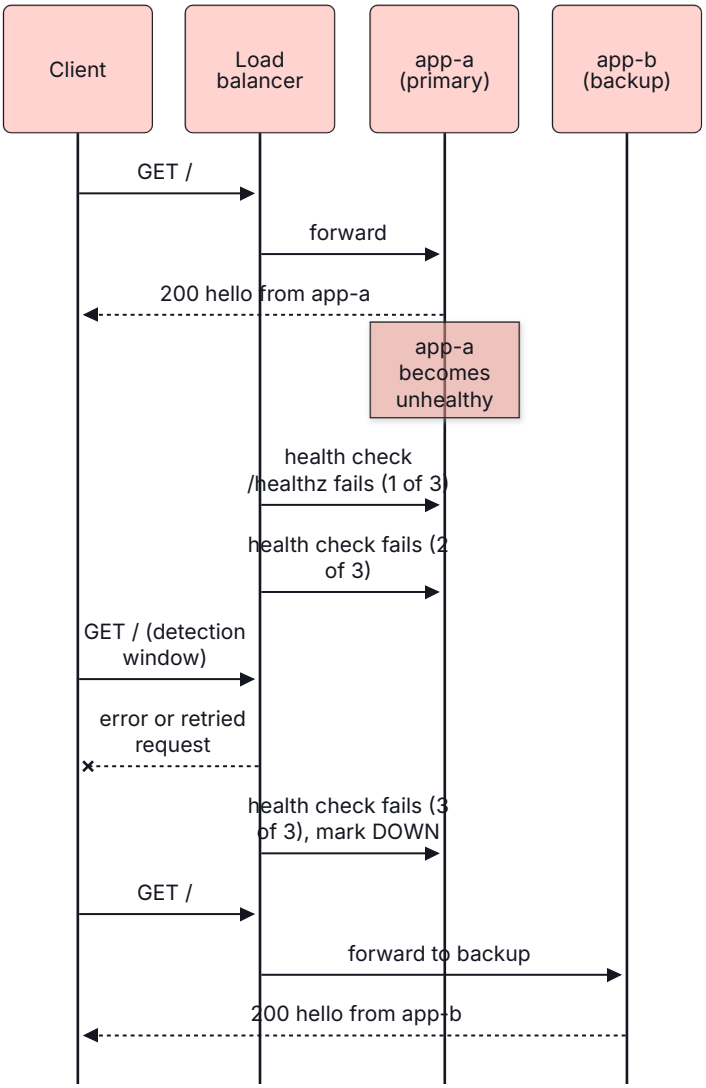


Figure 15.5-1

Split-brain, and how fencing prevents it:

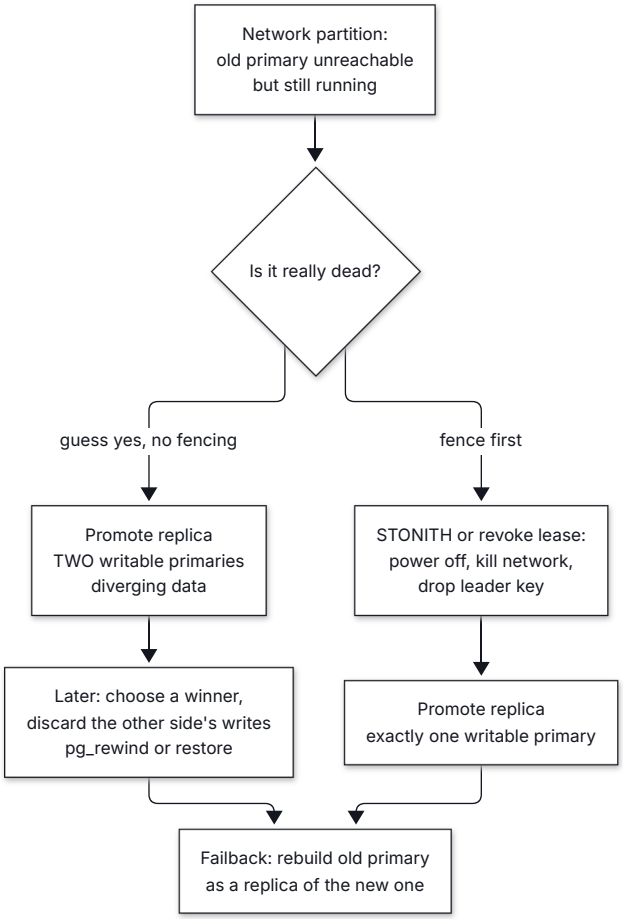


Figure 15.5-2

5 How it works

Automatic vs manual

	Automatic	Manual (human-triggered, scripted)
Detection to action	seconds	minutes (page, assess, run)
Typical tools	RDS Multi-AZ, Patroni, Aurora, Kubernetes controllers, cloud LB health checks	runbook plus a promote script, `pg_ctl promote`, Route 53 record edit
Main risk	false positives, flapping, split-brain, cascading failover	slow RTO, human error under stress

	Automatic	Manual (human-triggered, scripted)
Good fit	stateless tiers, well-understood data tiers with fencing	cross-region failover, data tiers without reliable fencing, decisions with data-loss trade-offs
Safeguards	quorum, leader lease, fencing, a minimum time between failovers, `fall` / `rise` thresholds	explicit "go" from the incident commander, pre-flight checks in the script

A common compromise: **automatic within a region (stateless tiers and the database across AZs), manual between regions**, because a regional failover usually implies accepting some data loss and takes a business decision.

Ways to route clients to the new active node

Mechanism	Speed	Caveat
Load balancer or proxy with health checks (ALB, NLB, HAProxy, Nginx)	seconds	the LB itself must be redundant; passive checks (Nginx open source) only notice failures on real requests
Virtual IP moved by keepalived or VRRP	seconds	needs a shared L2 network, not available in most clouds
DNS record change (Route 53 failover routing, manual edit)	TTL plus client caching: tens of seconds to hours	some resolvers, JVMs and connection pools cache beyond the TTL; keep TTL low (for example 30-60 s) *before* you need it
Database endpoint managed by the platform (RDS endpoint, Kubernetes Service)	tens of seconds	clients must reconnect and retry
Global anycast or CDN steering	seconds	typically HTTP-level only, not database traffic

Health checks decide everything. A shallow check (TCP port open) passes while the app is wedged; a deep check (query the database) can take the whole fleet out when the database is slow and cause a cascading outage. A reasonable default: a *liveness-style* check on each instance (can this process serve) plus separate dependency alerts. Use thresholds (`fall 3`, `rise 2`) to avoid flapping.

Split-brain defences. (1) A **quorum or lease** from a consensus system so only one leader can exist. (2) **Fencing** ("shoot the other node in the head", STONITH): make sure the old primary can no longer write before promoting, by powering it off through an out-of-band API, revoking its storage or network access. (3) Make the old primary demote itself if it loses contact with the quorum. (4) **Witness/odd number** of voters, spread over failure domains, so a partition leaves one side with a majority.

Failback. After failover, the old primary may contain writes the new primary never saw. You cannot just restart it as primary. Either rebuild it from scratch as a replica of the new primary, or use `pg_rewind`, which rolls back the diverged changes and re-synchronises. Then decide: stay on the new primary (usually simplest) or perform a *planned switchover* back during a quiet window (clean stop of the current primary, wait for the standby to catch up, promote, re-attach the other as replica).

6 Step-by-step guide

Prerequisites: Linux, `nginx`, `haproxy`, `python3`, `curl` and PostgreSQL 16 server binaries (`sudo apt-get install -y nginx haproxy postgresql-16`; if a package starts a system service you do not want, run `sudo systemctl disable --now nginx haproxy`). Everything uses non-root processes, loopback only and unusual ports (18101-18191, 55450-55451), so nothing existing is touched and nothing is exposed. No cost. Run as a normal user (`initdb` refuses root; on a root-only box use `sudo -u postgres -i`).

1. **Create two toy backends that can be made "sick".** Touching `app-a.sick` makes only the `/healthz` endpoint return 503, simulating a hung dependency while the process stays up.

```bash
export W=$HOME/dr-lab/failover && rm -rf "$W" && mkdir -p "$W/logs" "$W/tmp"
cat > "$W/backend.py" <<'PY'
import os, sys
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
name, port, here = sys.argv[1], int(sys.argv[2]),
  os.path.dirname(os.path.abspath(__file__))
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        sick = os.path.exists(os.path.join(here, name + ".sick"))
        code = 503 if (sick and self.path == "/healthz") else 200
        body = f"hello from {name}\n".encode()
        self.send_response(code); self.send_header("Content-Length", str(len(body)));
  self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a): pass
class S(ThreadingHTTPServer):
    def handle_error(self, *a): pass
S(("127.0.0.1", port), H).serve_forever()
PY
python3 "$W/backend.py" app-a 18101 & echo $! > "$W/a.pid"
python3 "$W/backend.py" app-b 18102 & echo $! > "$W/b.pid"
curl -s localhost:18101/ localhost:18102/
```

Expected: `hello from app-a` and `hello from app-b`.

2. **Nginx failover (passive health checks, backup server).** Nginx open source marks a server failed after real requests fail (`max_fails`) and sends traffic to the `backup` only then.

```bash
cat > "$W/nginx.conf" <<CONF
worker_processes 1;
pid $W/nginx.pid;
error_log $W/logs/error.log;
events { worker_connections 64; }
http {
  access_log off;
  client_body_temp_path $W/tmp/body; proxy_temp_path $W/tmp/proxy; fastcgi_temp_path
  $W/tmp/fcgi;
  uwsgi_temp_path $W/tmp/uwsgi; scgi_temp_path $W/tmp/scgi;
  upstream shop {
    server 127.0.0.1:18101 max_fails=1 fail_timeout=10s;
    server 127.0.0.1:18102 backup;
  }
  server {
    listen 127.0.0.1:18180;
    location / {
      proxy_pass http://shop;
      proxy_connect_timeout 1s;
      proxy_next_upstream error timeout http_502 http_503;
    }
  }
}
CONF
nginx -c "$W/nginx.conf" -e "$W/logs/error.log" -p "$W"
for i in 1 2 3; do curl -s localhost:18180/; done
kill "$(cat "$W/a.pid")"                      # simulate app-a crash
for i in 1 2 3; do curl -s -w '  (HTTP %{http_code})\n' localhost:18180/; done
```

Expected: three `hello from app-a` , then after the kill three `hello from app-b (HTTP 200)` . No client saw an error because `proxy_next_upstream` retried the first failed request on the backup. Note this works for idempotent requests; Nginx will not retry a `POST` by default, which is the right default for orders and payments. Restart app-a and stop Nginx: `python3 "$W/backend.py" app-a 18101 & echo $! > "$W/a.pid"; kill "$(cat "$W/nginx.pid")"` .

3. **HAProxy failover with active health checks, and measure the detection time.**

```bash
cat > "$W/haproxy.cfg" <<CONF
global
    maxconn 256
defaults
    mode http
```

```
    timeout connect 1s
    timeout client 5s
    timeout server 5s
frontend fe
    bind 127.0.0.1:18190
    default_backend shop
backend shop
    option httpchk GET /healthz
    default-server inter 1s fall 3 rise 2
    server app-a 127.0.0.1:18101 check
    server app-b 127.0.0.1:18102 check backup
listen stats
    bind 127.0.0.1:18191
    stats enable
    stats uri /stats
CONF
haproxy -c -f "$W/haproxy.cfg"
haproxy -f "$W/haproxy.cfg" -D -p "$W/haproxy.pid"; sleep 3
curl -s localhost:18190/                                    # hello from app-a
touch "$W/app-a.sick"; T=$(date +%s.%N)
until curl -s -m 1 localhost:18190/ | grep -q app-b; do sleep 0.2; done
echo "switched after $(echo "$(date +%s.%N) - $T" | bc) s"
curl -s 'localhost:18191/stats;csv' | cut -d, -f1,2,18 | grep app
```

Similar to: `Configuration file is valid`, then `switched after 3.1 s` (about `fall x inter` = 3 x 1 s) and `shop,app-a,DOWN` / `shop,app-b,UP`. During those 3 seconds clients got errors or timeouts. That detection window is a direct input to your RTO. Now fail back and time it:

```bash
rm "$W/app-a.sick"; T=$(date +%s.%N)
until curl -s -m 1 localhost:18190/ | grep -q app-a; do sleep 0.2; done
echo "failed back after $(echo "$(date +%s.%N) - $T" | bc) s"     # about rise x inter
  = 2 s
```

Try `fall 1` and `fall 6` and observe the trade-off: faster detection, more flapping risk, versus slower detection.

4. **Database failover: a primary and a streaming replica.** Settings needed for a safe fail-back later: `wal_log_hints` (required by `pg_rewind` unless data checksums are on) and `wal_keep_size` (so the WAL needed to rewind still exists).

```bash
export PATH=/usr/lib/postgresql/16/bin:$PATH PGHOST=/tmp
P=$W/pg && mkdir -p "$P"
initdb -D "$P/node1" >/dev/null
cat >> "$P/node1/postgresql.conf" <<'CONF'
unix_socket_directories = '/tmp'
```

```
listen_addresses = ''
wal_log_hints = on
wal_keep_size = 128MB
max_wal_senders = 5
CONF
echo "local replication all trust" >> "$P/node1/pg_hba.conf"
pg_ctl -D "$P/node1" -o "-p 55450" -l "$P/node1.log" -w start
psql -X -q -p 55450 -d postgres -c "CREATE TABLE orders(id serial primary key, item
  text)" \
  -c "INSERT INTO orders(item) SELECT 'i'||g FROM generate_series(1,1000) g"
pg_basebackup -h /tmp -p 55450 -D "$P/node2" -R -X stream -c fast
pg_ctl -D "$P/node2" -o "-p 55451" -l "$P/node2.log" -w start
psql -X -At -p 55451 -d postgres -c "SELECT pg_is_in_recovery()"       # t
```

Ports are passed on the command line (`-o "-p ..."`) rather than in `postgresql.conf` because `pg_rewind` later copies configuration from the source node, which would silently change a port.

5. **Cause a split-brain on purpose.** Promote the replica while the old primary is still alive and writable, as an over-eager automatic failover would after a network partition.

```bash
pg_ctl -D "$P/node2" promote -w
psql -X -At -p 55450 -d postgres -c "INSERT INTO orders(item) VALUES ('written-to-OLD-
  primary')"
psql -X -At -p 55451 -d postgres -c "INSERT INTO orders(item) VALUES ('written-to-NEW-
  primary')"
psql -X -At -p 55450 -d postgres -c "SELECT id,item FROM orders WHERE id>1000"
psql -X -At -p 55451 -d postgres -c "SELECT id,item FROM orders WHERE id>1000"
```

Expected: both nodes accepted id 1001 with different contents (`1001|written-to-OLD-primary` and `1001|written-to-NEW-primary`). Two histories now exist; no merge is possible without choosing a winner. This is the exact failure the fencing rules prevent.

6. **Resolve it: fence, rewind, re-attach (failback of the old primary as a replica).** We choose node2 as the winner; the old-primary write will be lost.

```bash
pg_ctl -D "$P/node1" -m fast stop                        # fencing: the old primary
  must not run
pg_rewind --target-pgdata="$P/node1" --source-server="host=/tmp port=55451
  dbname=postgres" -R
pg_ctl -D "$P/node1" -o "-p 55450" -l "$P/node1.log" -w start
psql -X -At -p 55450 -d postgres -c "SELECT pg_is_in_recovery()" -c "SELECT id,item
  FROM orders WHERE id>1000"
```

Expected: `pg_rewind: servers diverged at WAL location ...`, `Done!`, then `t` and `1001|written-to-NEW-primary`. The old write is gone, which is why fencing (preventing the second writer) is cheaper than rewinding. If `pg_rewind` complains that it cannot find a WAL file, `wal_keep_size` was too small or WAL archiving is needed (`restore_command`); that is a real-world failure mode.

7. **Planned switchover back to node1 (failback).**

```bash
psql -X -At -p 55451 -d postgres -c "INSERT INTO orders(item) VALUES ('written-while-failed-over')"
sleep 1
pg_ctl -D "$P/node2" -m fast stop                          # clean stop flushes all WAL to the standby
pg_ctl -D "$P/node1" promote -w
pg_rewind --target-pgdata="$P/node2" --source-server="host=/tmp port=55450 dbname=postgres" -R
pg_ctl -D "$P/node2" -o "-p 55451" -l "$P/node2.log" -w start
sleep 2
psql -X -At -p 55450 -d postgres -c "SELECT id,item FROM orders WHERE id>1000"
psql -X -At -p 55451 -d postgres -c "SELECT pg_is_in_recovery(), count(*) FROM orders"
```

Expected: node1 shows both rows (`1001|written-to-NEW-primary` and `1002|written-while-failed-over`) and node2 reports `t|1002`. `pg_rewind: no rewind required` is normal after a clean shutdown.

8. **DNS failover (read-only reference).** In Route 53 a *failover routing policy* uses two records for `api.example.com`: PRIMARY with a health check, SECONDARY pointing at the standby region. Keep TTL at 60 seconds or less. Cost and safety: health checks and hosted zones cost money; use a throwaway domain and delete them afterwards. A quick local feel for TTL effects: `dig +noall +answer example.com` shows the remaining TTL counting down in your resolver's cache.
9. **Clean up.**

```bash
pg_ctl -D "$P/node1" -m fast stop; pg_ctl -D "$P/node2" -m fast stop
kill "$(cat "$W/haproxy.pid")" "$(cat "$W/a.pid")" "$(cat "$W/b.pid")"
```

7 Real-world scenario

Illustrative. `shop-api` runs Patroni-style automatic failover inside `us-east-1`. A flaky switch makes the primary unable to reach the consensus store for 20 seconds. The leader lease expires and a replica is promoted; the old primary, correctly, demotes itself. But two application pods keep a pooled connection to the old primary for several minutes and receive er-

rors ("read-only transaction"). The on-call engineer sees elevated 5xx for 4 minutes: 25 seconds for failover, the remainder for connection pools that had no retry or reconnect logic. The fix is in the application: validate connections on borrow, retry idempotent writes with backoff, and use the cluster endpoint, not an instance hostname. In the postmortem they also turn on a minimum interval between automatic failovers, to prevent a flapping network from promoting replicas in a loop. **Lesson:** failover time is database promotion plus client recovery; the second term is often larger.

8 Common mistakes and best practices

Don't

- Promote a replica without making sure the old primary can no longer accept writes.
- Set health-check thresholds so tight that a single slow response triggers failover.
- Put a database dependency check in every instance's health check (a slow database then removes every web server at once).
- Rely on DNS failover with a 1-hour TTL, or assume applications honour TTLs.
- Forget failback: restarting a failed primary as primary is how split-brain is created the second time.

Do

- Decide per tier: automatic for stateless and for in-region databases with fencing, manual (scripted) for cross-region.
- Use a consensus-based leader election or the platform's managed failover rather than rolling your own scripts.
- Measure every term: detection, decision, promotion, client recovery. Then put them in the DR plan (RTO §15.7).
- Build clients that retry with backoff and jitter, validate pooled connections and re-resolve DNS (Connection Pooling §12.10).
- Drill it regularly (Failure Scenario Drills §15.10); see also High Availability and Scalability §0.8 and Load Balancers §2.8.

9 Tasks

- [ ] **T1 — Passive and active load balancer failover** *(≈45 min, beginner)*
  - **Goal:** watch traffic move away from a dead and from a sick backend.
  - **Steps:**
    1. Follow guide steps 1-3.

2. Record in a table: failure type (process killed vs `/healthz` 503), balancer (Nginx vs HAProxy), seconds until traffic moved, whether any client request failed.

- **Done when:** the table has four rows with measured values and a sentence explaining each difference.

☐ **T2 — Tune detection time** *(≈30 min, intermediate)*

- **Goal:** understand the speed versus flapping trade-off.
- **Steps:**
  1. Change `fall` to 1 and then 6, `inter` to 500ms and 2s, reload HAProxy (`haproxy -f ... -sf $(cat "$W/haproxy.pid")` for a graceful restart) and re-measure.
  2. Make `/healthz` flap: loop `touch` / `rm` of `app-a.sick` every 1.5 seconds and watch the HAProxy stats page.
- **Done when:** you can show one setting that detects fast but flaps under the loop and one that is stable but slow, with numbers.

☐ **T3 — Database failover and rewind** *(≈45 min, intermediate)*

- **Goal:** promote, fence and rejoin a database node.
- **Steps:** follow guide steps 4, 6 (skip 5's split-brain at first: stop node1 first, then promote node2) and 7.
- **Done when:** both nodes are up, the original primary is a replica that has no writes the new primary lacked, and `pg_stat_replication` on the current primary shows it streaming.

☐ **T4 — Break it: reproduce split-brain and describe the damage** *(≈30 min, advanced)*

- **Goal:** experience divergence and quantify the lost writes.
- **Steps:**
  1. Follow guide step 5, then insert 10 different rows on each side.
  2. Run `pg_rewind` in the direction that discards the *other* side and list exactly which rows are gone.
  3. Write the three-line rule your team would adopt to make this impossible (fencing, lease, who may promote).
- **Done when:** you can list the discarded rows and name the control that would have prevented them.

☐ **T5 — Script a safe manual promote** *(≈60 min, advanced)*

- **Goal:** automate the manual failover with guard rails.
- **Steps:**

1. Write `promote.sh` that refuses to run unless: the old primary does not answer on its port (or `--force-fence` stops it), the replica's `pg_last_wal_replay_lsn()` is printed with its lag, and the operator types the node name to confirm.
2. After promotion it prints the new primary and the exact `pg_rewind` command for the old one.
3. Test it against your lab with the old primary up and down.

- **Done when:** the script aborts with a clear message when the old primary is still writable, and succeeds when it is not.

☐ **T6 — Prove it: failover design review** *(≈45 min, advanced)*

- **Goal:** justify a failover design for `shop-api` out loud.
- **Steps:**
  1. Without notes, draw the web tier, database and DNS path and mark for each: automatic or manual, how it is detected, how clients find the new node, what prevents split-brain, and how failback works.
  2. Write the total expected unavailability as detection + decision + promotion + client recovery with numbers you measured in T1-T3.
- **Done when:** each term has a number from your lab or a stated assumption, and a reviewer cannot find a path that allows two writable primaries.

10 Self-check questions

1. What is split-brain and what are two independent defences against it?
2. When would you choose manual failover over automatic?
3. Why is DNS-based failover usually slower than a load balancer, and what must you do before the incident to improve it?
4. Why can a deep health check (querying the database) be dangerous on every web instance?
5. After failover the old primary comes back. Why can you not just start it as primary, and what are your options?

ANSWERS

1. Two nodes both acting as primary and accepting writes, so histories diverge. Defences: quorum/leader lease so only one leader exists, and fencing (power off, revoke storage or network) before promoting; also self-demotion on losing quorum.

2. When failover implies a business decision or acceptable data loss (cross-region, asynchronous replication), when there is no reliable fencing, or when false positives would be costlier than a few extra minutes.
3. Resolvers, applications and connection pools cache answers beyond or up to the TTL, and propagation is not instantaneous. Lower the TTL in advance, make clients re-resolve and retry, and test with real clients.
4. A shared dependency failure (slow database) marks every instance unhealthy simultaneously, so the load balancer removes all of them and turns a degradation into a full outage, and recovery can stampede.
5. It may contain writes the new primary never received, so the histories diverged. Rebuild it as a replica from a fresh base backup, or use `pg_rewind` to roll back divergence and re-attach it, then optionally do a planned switchover.

11 Further reading

- PostgreSQL documentation: *High Availability, Load Balancing, and Replication*, `pg_rewind` and `pg_ctl promote`.
- Patroni project documentation and the Zalando Postgres Operator documentation.
- HAProxy and Nginx documentation on health checks (`option httpchk`, `max_fails`, `proxy_next_upstream`).
- AWS documentation: *Amazon RDS Multi-AZ deployments* and *Route 53 DNS failover*.
- GitHub engineering's published postmortem of the October 2018 incident.
- Related: Replication §15.2, Restore Procedures §15.4, Replication in the databases level §13.4.

15.6 RPO - Recovery Point Objective

Topic 6 of 10 | ~3 hours | Difficulty: Intermediate

1 What it is

The **Recovery Point Objective (RPO)** is the maximum amount of data loss, measured in *time*, that the business accepts after a disaster. "RPO = 15 minutes" means: after we recover, the newest data we are guaranteed to have is at most 15 minutes older than the moment of the failure. It is a *requirement* the business sets ("we can tolerate losing at most this much"), not a property you discover afterwards.

Do not confuse it with three neighbours. **RTO** (RTO §15.7) is how long recovery takes, not how much data is lost. The **SLA** (SLAs §14.3) is a promise to customers about availability. The **actual recovery point** is what a real incident or drill *measured*: if the measured value is worse than the objective, the design (or the money) is wrong. RPO is set **per data class**: orders and payments may need seconds; analytics events, search indexes and sessions can often be rebuilt or lost.

2 Why it matters in DevOps

- **It converts a vague wish ("don't lose data") into engineering choices.** RPO decides whether nightly dumps are enough, whether you need continuous WAL archiving, or whether you must pay for synchronous replication.
- **Every minute of RPO has a price, in both directions.** Tight RPO costs money (more storage, network, standby capacity, write latency). Loose RPO costs business (lost orders, lost trust, regulatory exposure).
- **It tells you what to monitor.** The honest RPO of an asynchronous system is its current replication lag or the age of the newest archived WAL file. If you do not alert on those, your real RPO is "unknown".
- **It exposes hidden dependencies.** The database may have RPO 1 minute, but if uploaded files sync to a second region nightly, the *system's* RPO is 24 hours.
- **It drives contracts.** Customers and auditors ask for it in security reviews and disaster-recovery clauses.

3 Real-world system example

GitLab.com, January 2017. After the production database incident described in GitLab's public postmortem, service was restored from a snapshot taken roughly six hours earlier, and some hours of user-generated database content (issues, merge requests, comments) were lost. That outcome is a textbook measured RPO: it was far worse than the continuous protection people assumed they had, because the mechanisms they believed in were not working.

Amazon RDS and Aurora. RDS continuously uploads transaction logs, and its documentation describes the latest restorable time as typically within about five minutes of the present, which is the practical RPO of its point-in-time recovery for a single-region restore. Aurora stores data across multiple Availability Zones within a region and offers continuous backup; **Aurora Global Database** and RDS cross-region read replicas replicate asynchronously between regions, so a regional disaster implies loss of whatever had not yet reached the other region (AWS publishes typical lag figures for Global Database; check the current documentation, because they depend on workload).

Object storage. Amazon S3 Cross-Region Replication is asynchronous by default; *S3 Replication Time Control* adds an SLA that most objects replicate within 15 minutes and metrics to watch. A design that depends on uploaded files being in the second region should state which of these RPOs it assumes.

4 Diagram

RPO on a timeline. Data written after the last recovery point is the data you lose:

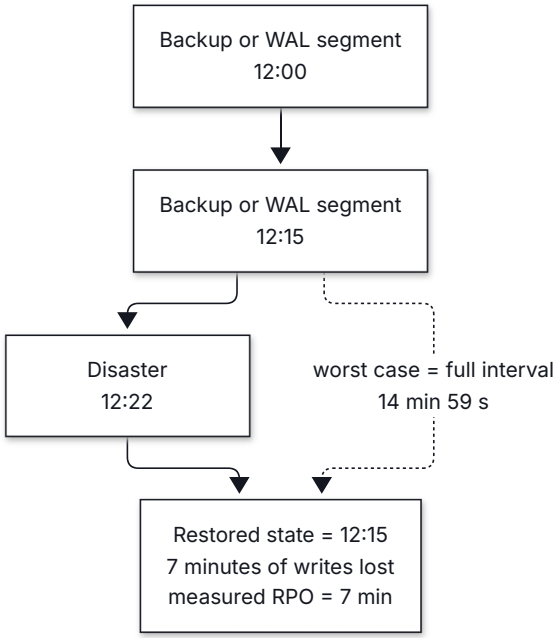


Figure 15.6-1

Cost versus RPO: each step down needs a more expensive mechanism.

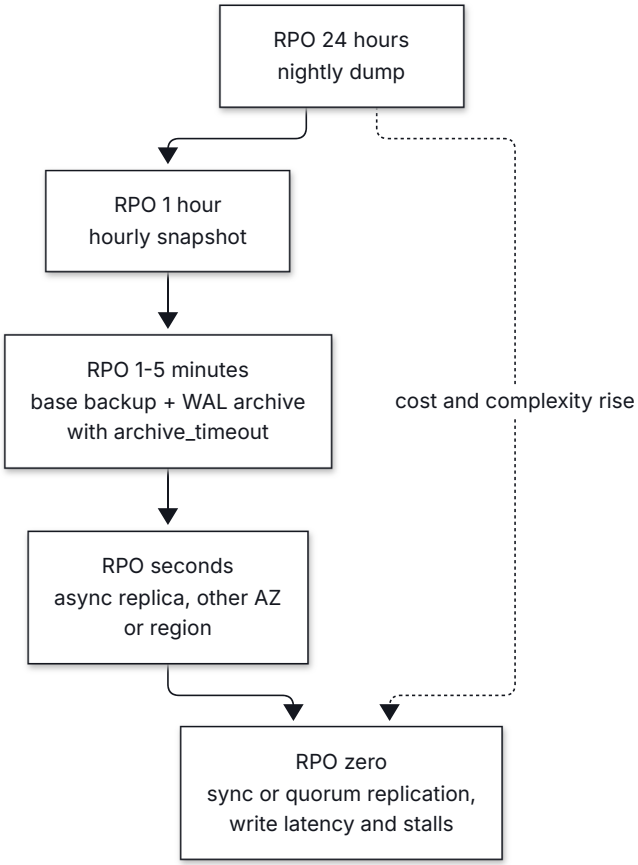


Figure 15.6-2

5 How it works

Deriving backup frequency from RPO. The worst-case data loss of any periodic mechanism is *one full interval plus the time the job takes to finish and ship*. So:

```
interval + job duration + shipping delay ⩽ RPO
```

Leave headroom, because one job can fail and still leave you inside the objective: a common rule is to schedule at no more than **half the RPO**. Worked example for `shop-api` (illustrative numbers): the business says "at most 15 minutes of orders". Nightly `pg_dump` gives RPO 24 hours, fails. Hourly snapshots give 1 hour, fails. Continuous WAL archiving with `archive_timeout = 60s` (segments are forced out at least every minute) plus a base backup

gives an RPO of about one minute plus shipping delay, passes. Adding an asynchronous replica gives seconds of exposure for a *server* failure (lag), while WAL archive to object storage covers a *site* failure.

Mechanism	Worst-case loss	What it protects against	Cost drivers
Nightly logical dump	up to 24 h	operator error, small data sets	cheap; slow restore
Hourly volume snapshot	up to 1 h	volume loss, bad deploy	snapshot storage, scheduling
Base backup + WAL archive	`archive_timeout` + ship delay (seconds to minutes)	corruption, bad migration (with PITR), site loss if copied off-site	object storage, a bit of CPU and network
Async replica, other AZ	replication lag (usually sub-second to seconds)	instance or AZ failure	a second database instance (roughly 2x compute)
Async replica, other region	lag across regions (seconds, can grow)	region failure	instance cost plus cross-region transfer
Sync or quorum replication	zero for acknowledged commits	node failure, with no loss	write latency, writes can stall if standbys are unreachable (Replication §15.2)

Cost of a lower RPO, in practice. Going from 24 hours to 1 hour is a scheduling change. Going from 1 hour to 1 minute adds a log-shipping pipeline and monitoring. Going from minutes to seconds requires a running standby (double compute) and sometimes a cross-region link. Going to zero requires synchronous commit across failure domains, which adds a network round trip to every write and can halt writes when the standby is unreachable. Teams therefore set tiers: *Tier A* data (payments) tight RPO, *Tier C* data (cache, analytics) loose.

Backups do not protect the last mile by themselves. Replication copies mistakes (Backups §15.1); the RPO you can claim for *logical corruption* is the time of the last good backup or the PITR window, not the replication lag. You need both numbers in the plan: RPO for infrastructure failure and RPO for data corruption (which PITR can bring to "seconds before the mistake").

How to measure the real RPO continuously. For WAL archiving, alert when `now() - pg_stat_archiver.last_archived_time` exceeds your target. For replicas, alert on replay lag in seconds. For snapshots, alert on the age of the newest completed snapshot. For object replication, use the replication metrics. Write the measured values into your DR drills.

6 Step-by-step guide

Prerequisites: Linux with PostgreSQL 16 server binaries in `/usr/lib/postgresql/16/bin`, a normal (non-root) user, `python3`. Free, local, uses ports 55460 and 55461 and a scratch directory. Nothing existing is touched.

1. **Compute what an RPO implies with a small calculator.**

```bash
mkdir -p ~/dr-lab/rpo && cd ~/dr-lab/rpo
cat > rpo_calc.py <<'EOF'
#!/usr/bin/env python3
"""rpo_calc.py - what does a given RPO imply? Usage: rpo_calc.py RPO_SECONDS
  [orders_per_hour] [avg_order_usd]"""
import sys
rpo = float(sys.argv[1])
orders_per_hour = float(sys.argv[2]) if len(sys.argv) > 2 else 600
avg_order = float(sys.argv[3]) if len(sys.argv) > 3 else 40.0

orders_lost = orders_per_hour / 3600 * rpo          # worst case: one full interval of
  orders
print(f"RPO target            : {rpo:>10.0f} s ({rpo/3600:.2f} h)")
print(f"worst-case orders lost: {orders_lost:>10.0f}  (about
  ${orders_lost*avg_order:,.0f} revenue at risk)")

if rpo ⩾ 86400:   method = "nightly logical or physical backup"
elif rpo ⩾ 3600:  method = "hourly snapshots or incremental backups"
elif rpo ⩾ 60:    method = "base backup + continuous WAL archiving (archive_timeout
  ⩽ RPO/2)"
elif rpo > 0:     method = "streaming replica (asynchronous), lag alerting"
else:             method = "synchronous or quorum replication (write latency and
  availability cost)"
print(f"minimum mechanism     : {method}")
if rpo > 0:
    print(f"schedule with headroom: every {rpo/2:.0f} s ({172800/rpo:.1f} runs per
  day), because the job itself takes time and can fail once")
EOF
python3 rpo_calc.py 900 1200 25
```

Expected:

```
RPO target            :        900 s (0.25 h)
worst-case orders lost:        300  (about $7,500 revenue at risk)
minimum mechanism     : base backup + continuous WAL archiving (archive_timeout ⩽
  RPO/2)
schedule with headroom: every 450 s (192.0 runs per day), because the job itself takes
  time and can fail once
```

Try `86400`, `30` and `0` and read how the mechanism changes. The order rate and price are placeholders; use your own business numbers.

2. **Write the RPO drill script.** It builds a database with WAL archiving, runs a continuous writer, destroys the server *violently* (`kill -9`, then deletes the data directory) after 25 seconds, restores from the base backup plus the archive, and reports how many seconds of data were lost.

```bash
cat > rpo-drill.sh <<'EOF'
#!/usr/bin/env bash
# Usage: rpo-drill.sh <archive_timeout_seconds>
set -eu
export PATH=/usr/lib/postgresql/16/bin:$PATH
export W=$HOME/dr-lab/rpo/lab && rm -rf "$W" && mkdir -p "$W/archive"
export PGHOST=/tmp PGPORT=55460
TIMEOUT=${1:-10}
initdb -D "$W/pg" >/dev/null
cat >> "$W/pg/postgresql.conf" <<CONF
port = 55460
unix_socket_directories = '/tmp'
listen_addresses = ''
archive_mode = on
archive_command = 'test ! -f $W/archive/%f && cp %p $W/archive/%f'
archive_timeout = ${TIMEOUT}s
CONF
pg_ctl -D "$W/pg" -l "$W/pg.log" -w start >/dev/null
createdb shop
psql -X -q -d shop -c "CREATE TABLE ticks(id serial primary key, ts timestamptz not
  null)"
pg_basebackup -D "$W/base" -X stream -c fast
# the "orders": one row every 0.2 s, each stamped with the time it committed
( while true; do psql -X -q -d shop -c "INSERT INTO ticks(ts) VALUES
  (clock_timestamp())"; sleep 0.2; done ) >/dev/null 2>&1 &
WR=$!
sleep 25
CRASH=$(date +%s.%N)                                  # the disaster
kill -9 "$(head -1 "$W/pg/postmaster.pid")"; kill $WR 2>/dev/null || true
sleep 1; rm -rf "$W/pg"                               # the disk is gone; only base/
  and archive/ survive
echo "archived WAL segments: $(ls "$W/archive" | wc -l)"
START=$(date +%s)
cp -r "$W/base" "$W/restored"; chmod 700 "$W/restored"
export PGPORT=55461
cat >> "$W/restored/postgresql.conf" <<CONF
port = 55461
unix_socket_directories = '/tmp'
listen_addresses = ''
restore_command = 'cp $W/archive/%f %p'
CONF
touch "$W/restored/recovery.signal"
```

```
pg_ctl -D "$W/restored" -l "$W/restored.log" -w start >/dev/null
until [ "$(psql -X -At -d shop -c 'SELECT pg_is_in_recovery()')" = f ]; do sleep 0.2;
  done
echo "recovery took $(( $(date +%s) - START )) s"
psql -X -At -d shop -c "SELECT count(*) AS rows_recovered, round(extract(epoch from
  (to_timestamp($CRASH) - max(ts)))::numeric,1) AS seconds_lost FROM ticks"
pg_ctl -D "$W/restored" -m fast stop >/dev/null
EOF
chmod +x rpo-drill.sh
```

3. **Run the drill with a loose and a tight `archive_timeout` and compare the measured RPO.**

```
BASH
./rpo-drill.sh 10
./rpo-drill.sh 2
```

Similar to (your numbers will differ run to run):

```
archived WAL segments: 5
recovery took 1 s
91|5.5          ← 91 rows recovered, about 5.5 seconds of data lost
...
archived WAL segments: 15
recovery took 1 s
114|0.3         ← far fewer seconds lost, but 3x more WAL files (each is 16 MB on
  disk)
```

The measured loss is bounded by `archive_timeout` (the open WAL segment that was never shipped). Tighter means more files shipped and stored: that is the "cost of lower RPO", demonstrated. (Each forced segment switch writes a full 16 MB file even when nearly empty, which compresses well, so real systems compress archived WAL.)

4. **Compare with a replica's exposure.** Reuse the two-node lab from Replication §15.2, run `pgbench` against the primary, and watch `replay_lag` in `pg_stat_replication`. A replica's RPO for a server crash is that lag; a replica's RPO for a `DROP TABLE` is zero protection.
5. **Set up the alert that tells you the real RPO.**

```
BASH
psql -X -At -h /tmp -p 55460 -d postgres -c "SELECT now() - last_archived_time AS
  archive_age, failed_count FROM pg_stat_archiver" 2>/dev/null || echo "start the lab
  cluster first"
```

In production, export `archive_age` as a metric and page when it exceeds half your RPO. See What to Monitor §10.11.

6. **Clean up.**

```bash
rm -rf ~/dr-lab/rpo/lab
```

7 Real-world scenario

Illustrative. `shop-api` is told "we can lose at most 5 minutes of orders" and configures an asynchronous replica plus nightly dumps. The team believes the replica gives them seconds. During a quarterly drill they kill the primary's VM and promote the replica: 11 seconds of lag, fine. Then they run the *corruption* drill: a faulty migration deletes rows at 14:02, discovered at 14:40. The replica has the same deletion; the newest dump is from 02:00, so the honest recovery point for that scenario is **12 hours**, not 5 minutes. They add base backups plus WAL archiving with `archive_timeout = 60s`, which allows point-in-time recovery to 14:01:59. Their DR plan now states two RPOs: 15 seconds for infrastructure failure (replica lag, alerted) and about one minute for logical corruption (PITR). **Lesson:** one RPO number per scenario, backed by a measured drill, not one optimistic number for everything.

8 Common mistakes and best practices

Don't

- Quote a single RPO for the whole system; stateful dependencies (uploads, queues, third-party data) often have a worse one.
- Assume replication lag is your RPO for data corruption or human error.
- Choose "zero data loss" without agreeing to its availability and latency cost.
- Set RPO from what the current tooling happens to do ("our dumps are nightly so RPO is 24 hours" is a result, not an objective).
- Forget the time the backup job takes and the delay shipping it off-site.

Do

- Ask the business to price data loss (orders per minute, revenue per order, regulatory exposure) and derive the RPO from it.
- Tier data: tight RPO for money and customer records, loose for derived or rebuildable data.
- Schedule jobs at half the RPO or better, and monitor *backup age* and *replication lag* against it.

- Measure the actual recovery point in every drill and record the gap (Failure Scenario Drills §15.10).
- Keep an off-site copy that meets the RPO too (Off-site Backups §15.8).

9 Tasks

- [ ] **T1 — Run the RPO drill and chart the trade-off** *(≈45 min, beginner)*
  - **Goal:** measure loss versus `archive_timeout` with your own numbers.
  - **Steps:**
    1. Follow guide steps 1-3 and also run with timeouts 1, 5, 30 and 60.
    2. Record seconds lost, number of archived segments and total archive size (`du -sh lab/archive`) for each run.
  - **Done when:** you have a 5-row table showing loss rising with the timeout and archive volume rising as it shrinks, plus one sentence on the trade-off.
- [ ] **T2 — Derive a schedule from a business statement** *(≈20 min, beginner)*
  - **Goal:** turn a requirement into a backup schedule.
  - **Steps:**
    1. For three data classes in `shop-api` (orders, product images, session cache) write an RPO the business could accept and the mechanism that meets it, using `rpo_calc.py`.
    2. State the schedule with headroom for each.
  - **Done when:** each row names RPO, mechanism, interval and how you will alert if the interval is missed.
- [ ] **T3 — Break it: RPO does not cover corruption** *(≈30 min, intermediate)*
  - **Goal:** show that a replica's low RPO is irrelevant to logical errors.
  - **Steps:**
    1. Using the replication lab, `DELETE` from a table on the primary and show the replica agrees.
    2. Using the archive from the RPO drill, restore to a point just before the delete using `recovery_target_time` and `recovery_target_action = 'promote'` in `postgresql.conf` of the restored copy (use a timestamp from your `ticks` table; the drill script deletes `lab/pg` but keeps `lab/base` and `lab/archive`).
    3. Compare the data loss of the two approaches.
  - **Done when:** the PITR copy contains the rows the replica no longer has, and you can state both RPOs for this scenario.

- [ ] **T4 — Alert on the real RPO** *(≈45 min, intermediate)*
  - **Goal:** make the RPO observable.
  - **Steps:**
    1. Write a script or Prometheus query exporter that reports `now() - last_archived_time` from `pg_stat_archiver` and the age of your newest backup file.
    2. Make it exit non-zero (or alert) when either exceeds half your chosen RPO.
    3. Stop the archive by breaking `archive_command` (for example point it at a non-existent directory) and watch the alert fire.
  - **Done when:** the alert fires within one `archive_timeout` of the break and clears after you fix the command and the backlog drains.

- [ ] **T5 — Cost model** *(≈45 min, advanced)*
  - **Goal:** price each RPO tier for `shop-api`.
  - **Steps:**
    1. Using your cloud provider's public pricing calculator (read-only, no resources created), estimate monthly cost for: nightly snapshot, hourly snapshot, WAL archive to object storage, a same-region replica, a cross-region replica.
    2. Add the expected revenue at risk per incident from `rpo_calc.py`.
  - **Done when:** a table lists monthly cost, RPO and "break-even incident frequency" for each tier, and you recommend one.

- [ ] **T6 — Prove it: defend an RPO** *(≈30 min, advanced)*
  - **Goal:** explain an RPO decision to a non-engineer.
  - **Steps:**
    1. Without notes, write one page that states the RPO for orders, the mechanism, how it is monitored, the measured value from your drill and the cost of making it tighter by 10x.
    2. Explain the difference between this RPO and the RPO for accidental deletion.
  - **Done when:** the page contains both numbers (infrastructure failure and corruption) with evidence from your drills.

10 Self-check questions

1. Define RPO in one sentence and say what it is not.
2. Your backup runs every 6 hours and takes 30 minutes to complete and upload. What is the worst-case data loss, and what would you change to reach a 1-hour RPO?

3. Why is an asynchronous replica not enough for an RPO of 5 minutes against accidental `DROP TABLE`?
4. What does synchronous replication buy you and what does it cost?
5. How would you detect, before a disaster, that your actual RPO has silently drifted above target?

ANSWERS

1. The maximum tolerated data loss measured in time. It is not recovery time (RTO), nor an availability promise (SLA), nor what the tooling happens to achieve.
2. Roughly 6.5 hours (an interval plus the job and upload time, and if one run fails, double that). Move to continuous WAL archiving or at least hourly snapshots/incrementals with off-site shipping, and alert on backup age.
3. Replication faithfully copies the drop within moments, so the replica has no pre-drop state. You need PITR or backups with retention beyond your detection time.
4. Zero loss of acknowledged commits across the synchronous failure domain; it costs write latency (a network round trip) and can stall writes when the standby is unreachable.
5. Alert on replication lag, `pg_stat_archiver.last_archived_time` age, newest backup age and replication metrics for object storage, and measure the real value in drills.

11 Further reading

- PostgreSQL documentation: *Continuous Archiving and Point-in-Time Recovery (PITR)* and the `pg_stat_archiver` view.
- AWS documentation: *Amazon RDS backups and point-in-time recovery*, *Aurora Global Database*, and *S3 Replication Time Control*.
- *Site Reliability Engineering* (Google), chapters on data integrity; AWS Well-Architected Framework, Reliability Pillar (disaster recovery objectives).
- GitLab's public postmortem of the 31 January 2017 database incident.
- Related: Backups §15.1, Replication §15.2, Disaster Recovery overview §0.9, Backups and PITR §13.5.

15.7 RTO - Recovery Time Objective

Topic 7 of 10 | ~3 hours | Difficulty: Intermediate

1 What it is

The **Recovery Time Objective (RTO)** is the maximum acceptable length of time between the moment a service goes down and the moment it is usable again. "RTO = 30 minutes" means: from the failure, the service must be back (possibly degraded, and as agreed with the business) within half an hour. The clock covers the *whole* outage, not just the technical restore: detecting the problem, waking someone up, deciding what to do, restoring or failing over, validating, and switching traffic.

RTO is a requirement the business sets and the architecture must meet; the **achieved RTO** is what a drill or incident measured. RTO is paired with the RPO (RPO §15.6): RPO is "how much data may we lose", RTO is "how long may we be down". A related number, the **Maximum Tolerable Downtime (MTD)**, is the point at which the business itself is damaged; RTO must sit comfortably inside it. Work that follows recovery (reconciling records, rebuilding caches) is often forgotten but also takes time, so agree what "recovered" means up front.

2 Why it matters in DevOps

- **It decides how much you spend.** Recovering in four hours from backups costs very little; recovering in four minutes means a second environment, running, tested and paid for. RTO is the single biggest driver of DR cost.
- **It shapes the architecture.** A tight RTO cannot be met by "rebuild everything from scratch"; it requires automation (Infrastructure as Code, scripted promotion), pre-provisioned capacity, and in some cases active-active design.
- **Your RTO is the sum of the slowest steps**, usually human ones: noticing, deciding, finding credentials. Technical restore time is often not the biggest term.
- **It is a contract in disguise.** If sales promised a customer a 99.95% SLA (SLAs §14.3), the budget for downtime per month is about 22 minutes; an RTO of two hours blows through it in a single incident.

- **Untested RTOs are fiction.** Teams routinely discover their "30 minute RTO" is 3 hours the first time they time it (Testing Backup Restoration §15.9).

3 Real-world system example

AWS's DR strategies. The AWS Well-Architected Reliability Pillar and the AWS whitepaper *Disaster Recovery of Workloads on AWS* describe four strategies on a cost-versus-recovery-time scale: **backup and restore**, **pilot light**, **warm standby** and **multi-site active/active**. They map directly onto RTO and RPO targets (see the table in section 5). The whitepaper is a good reference for picking a tier per workload.

Netflix. Netflix has publicly described running its streaming service from multiple AWS regions and practising evacuating a region, rehearsed with the "Chaos Kong" exercise, which simulates a regional failure by shifting traffic away from a region. The point for RTO is that an active-active design with a rehearsed traffic shift can recover in minutes because nothing has to be restored: capacity and data are already live elsewhere. (Netflix engineers have written about this on their tech blog; details such as exact timings change over time, so treat the pattern, not the numbers, as the lesson.)

OVHcloud Strasbourg, March 2021. A fire destroyed one data center (SBG2) of OVHcloud's Strasbourg site and caused an outage of other buildings on the site. It is a public, widely reported event. Customers with backups in the same data center or only on the same site lost data, while those with copies and an environment elsewhere could restart there. It is a reminder that both the RTO and the RPO of a "region failure" depend on where the second copy and the second environment physically are. We keep the facts high-level here; OVHcloud published its own updates and a postmortem.

4 Diagram

What the clock covers. Each segment is a term in your RTO, and each can be measured:

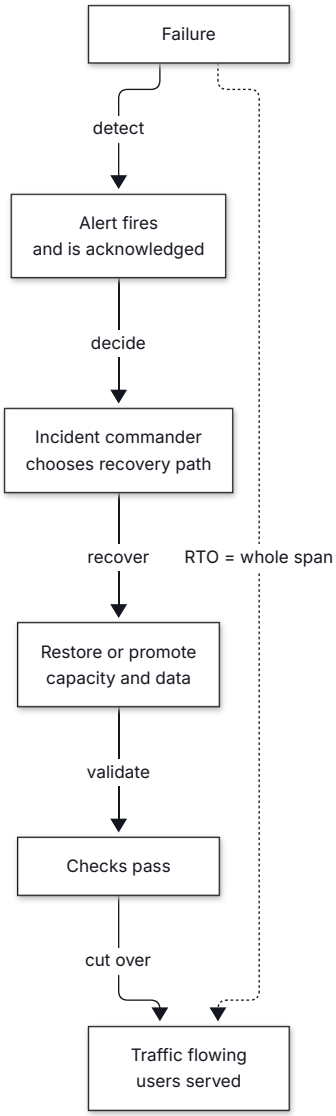


Figure 15.7-1

The four DR tiers trade money for time:

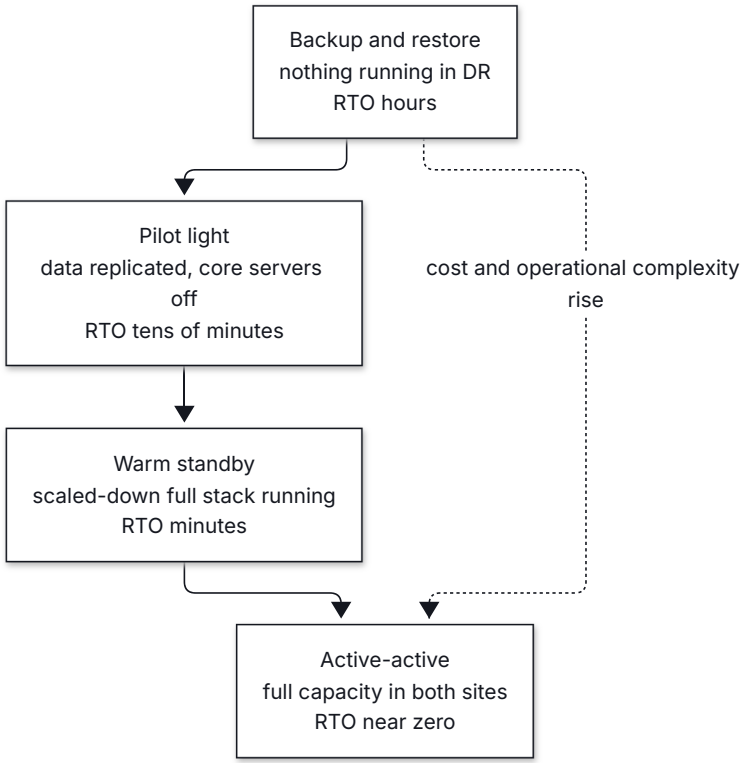


Figure 15.7-2

5 How it works

Choosing an RTO. Start from the business impact of downtime: revenue per minute, contractual penalties, safety or legal obligations, reputational cost. Then ask what is *degraded but acceptable* (read-only mode, no search, queued checkouts). Different services get different RTOs; the dependency graph decides the real value, because a service cannot recover faster than the things it depends on. A common approach is to assign each system a **tier**:

Tier	Example systems in `shop-api`	Typical RTO	Typical RPO
0	identity, secrets, DNS, networking	minutes (everything else waits for it)	near zero (config in Git)
1	checkout API, orders database	15-60 minutes	1-5 minutes
2	search, recommendations, emails	4-8 hours	hours
3	internal reporting, analytics	24-72 hours	a day

The four DR strategies (names from the AWS DR whitepaper)

Strategy	What runs in the DR site	Typical RTO	Typical RPO	Relative cost	Main work to recover
Backup and restore	nothing, only backups copied off-site	hours	hours (backup interval)	lowest	create infrastructure from code, restore data, switch DNS
Pilot light	data stores replicated and kept up to date, application servers defined but stopped or at minimum size	tens of minutes	minutes	low-medium	start and scale the application tier, promote the database, switch DNS
Warm standby	a complete but scaled-down copy running and serving health checks	minutes	seconds to minutes	medium-high	scale up, promote, shift traffic
Multi-site active-active	full capacity in two or more sites, serving traffic	seconds to minutes	near zero to seconds	highest, and hard (data conflicts, consistency)	remove the failed site from rotation

Indicative ranges only; the real values depend on your data size, automation and drills.

RTO budget. `RTO = detection + decision + provisioning + data restore (or promote) + validation + traffic cut-over`. Every term can be attacked:

Term	How to shrink it
Detection	symptom-based alerts on user-visible errors (not only host metrics), synthetic checks, short evaluation windows
Decision	runbooks with pre-authorised actions, clear incident roles (Incident Management §14.5), a rule such as "if X is not back in 10 minutes, fail over"
Provisioning	Infrastructure as Code, pre-built machine images and container images, pilot light or warm capacity
Data restore	smaller restores (snapshots, PITR with fewer WAL files), parallel restore, restore from a recent base backup in the same region, or replace restore with a promote of a replica
Validation	automated checks (Restore Procedures §15.4) instead of manual spot-checks
Cut-over	low DNS TTL set in advance, load balancer switch, pre-warmed caches and connections

Restore time scales with data size, but not linearly. Copying bytes is bounded by network and disk throughput; loading a logical dump adds index builds; a physical restore adds WAL replay up to the target time. The only reliable estimate is to measure with realistic data. Restoring a cloud snapshot is fast to start but the volume may read slowly until blocks are pulled in from object storage (Snapshots §15.3). When data restore dominates, change architecture rather than tune: replicas, smaller shards, or a warmer tier.

RTO must include people. The 3 a.m. factor (finding the laptop, VPN, MFA), unavailability of the one person who knows the system, and decision paralysis are real. Track **time to acknowledge** and **time to decision** as part of drills (On-call Practices §14.6).

6 Step-by-step guide

Prerequisites: Linux with PostgreSQL 16 server binaries in `/usr/lib/postgresql/16/bin`, `python3`, `bc`, a normal (non-root) user. Free and local; uses port 55470/55471 and a scratch directory. Nothing existing is touched.

1. **Create a script that turns an incident timeline into an achieved RTO.** This is the instrument you will use in every drill.

```bash
mkdir -p ~/dr-lab/rto && cd ~/dr-lab/rto
cat > rto_timeline.py <<'EOF'
#!/usr/bin/env python3
"""rto_timeline.py - turn 'HH:MM:SS event text' lines into per-phase durations and the
  achieved RTO.
Usage: rto_timeline.py timeline.txt [target_minutes]"""
import sys
from datetime import datetime

def t(s): return datetime.strptime(s, "%H:%M:%S")

rows = []
for line in open(sys.argv[1]):
    line = line.strip()
    if line and not line.startswith("#"):
        ts, text = line.split(None, 1)
        rows.append((t(ts), text))
if len(rows) < 2:
    sys.exit("need at least two events")
print(f"{'phase (ends at event)':<46}{'seconds':>8}")
for (a, _), (b, text) in zip(rows, rows[1:]):
    print(f"{text:<46}{(b - a).total_seconds():>8.0f}")
total = (rows[-1][0] - rows[0][0]).total_seconds()
print(f"{'TOTAL (achieved RTO)':<46}{total:>8.0f}  = {total/60:.1f} min")
if len(sys.argv) > 2:
    target = float(sys.argv[2]) * 60
    verdict = "MET" if total <= target else "MISSED"
    print(f"target {sys.argv[2]} min → {verdict} (margin {(target - total)/60:+.1f}
  min)")
EOF
cat > timeline.txt <<'EOF'
# first line is the moment of failure
12:00:00 failure injected
12:02:10 alert fired and acknowledged
12:05:40 decision made to restore
```

```
12:21:30 database restored on new host
12:24:00 validation passed
12:27:15 traffic switched and verified
EOF
python3 rto_timeline.py timeline.txt 30
```

Expected (the example timeline above is made up):

```
phase (ends at event)                          seconds
alert fired and acknowledged                       130
decision made to restore                           210
database restored on new host                      950
validation passed                                  150
traffic switched and verified                      195
TOTAL (achieved RTO)                              1635  = 27.2 min
target 30 min → MET (margin +2.8 min)
```

Notice that restoring the database is 58 percent of the total, but detection plus decision is another 21 percent that no backup tool can fix.

2. **Measure restore time against data size (cold "backup and restore" tier), then compare with promoting a warm standby.**

```bash
cat > rto-bench.sh <<'EOF'
#!/usr/bin/env bash
set -eu
export PATH=/usr/lib/postgresql/16/bin:$PATH
export W=$HOME/dr-lab/rto/lab && rm -rf "$W" && mkdir -p "$W" && export PGHOST=/tmp
  PGPORT=55470
initdb -D "$W/pg" >/dev/null
cat >> "$W/pg/postgresql.conf" <<CONF
port = 55470
unix_socket_directories = '/tmp'
listen_addresses = ''
max_wal_senders = 5
CONF
echo "local replication all trust" >> "$W/pg/pg_hba.conf"
pg_ctl -D "$W/pg" -l "$W/pg.log" -w start >/dev/null
createdb shop
for N in 200000 1000000 3000000; do
  psql -X -q -d shop -c "DROP TABLE IF EXISTS orders" \
    -c "CREATE TABLE orders(id serial primary key, customer_id int, item text,
  total_cents int, created_at timestamptz default now())" \
    -c "INSERT INTO orders(customer_id,item,total_cents) SELECT g%50000, md5(g::text),
  g%9000 FROM generate_series(1,$N) g" \
    -c "CREATE INDEX ON orders(customer_id)" -c "CREATE INDEX ON orders(created_at)"
  pg_dump -Fc -d shop -f "$W/shop.dump"
  SIZE=$(du -m "$W/shop.dump" | cut -f1)
  for J in 1 4; do
```

```
    dropdb --if-exists shop_r 2>/dev/null; createdb shop_r
    S=$(date +%s.%N); pg_restore -j $J -d shop_r "$W/shop.dump"; E=$(date +%s.%N)
    printf "rows=%s dump=%sMB jobs=%s restore=%.1fs\n" $N $SIZE $J "$(echo "$E - $S" |
  bc)"
  done
done
pg_basebackup -D "$W/standby" -R -X stream -c fast
pg_ctl -D "$W/standby" -o "-p 55471" -l "$W/standby.log" -w start >/dev/null
S=$(date +%s.%N); pg_ctl -D "$W/standby" promote -w >/dev/null; E=$(date +%s.%N)
printf "promote warm standby = %.2fs\n" "$(echo "$E - $S" | bc)"
pg_ctl -D "$W/standby" -m fast stop >/dev/null; pg_ctl -D "$W/pg" -m fast stop
  >/dev/null
EOF
chmod +x rto-bench.sh && ./rto-bench.sh 2>&1 | grep -v NOTICE
```

Similar to (numbers depend on your disk and CPU):

```
rows=200000 dump=6MB jobs=1 restore=0.5s
rows=200000 dump=6MB jobs=4 restore=0.3s
rows=1000000 dump=26MB jobs=1 restore=3.5s
rows=1000000 dump=26MB jobs=4 restore=3.3s
rows=3000000 dump=79MB jobs=1 restore=6.3s
rows=3000000 dump=79MB jobs=4 restore=4.8s
promote warm standby = 0.10s
```

The restore grows with data; the promote does not. The big difference is the lesson of the tier table: a warm standby converts a data-size-dependent restore into a near-constant promote, at the price of running a second database.

3. **Extrapolate to production size, and be honest about it.** Divide your measured MB/s into your real database size and add the non-restore terms.

```BASH
python3 - <<'EOF'
dump_mb, secs = 79, 4.8          # replace with your measured values
prod_gb = 400                    # replace with your real (compressed) dump or base-
  backup size
est = prod_gb * 1024 / (dump_mb / secs)
print(f"naive restore estimate for {prod_gb} GB: {est/60:.0f} min ({est/3600:.1f} h)")
print("then add: detection + decision + provisioning + validation + cut-over, and
  measure at real scale before promising it")
EOF
```

Expected: roughly `415 min (6.9 h)`. The naive straight-line number is a lower bound for planning, not a promise; index builds, WAL replay and network egress can change it a lot. This is why you need an at-scale drill.

4. **Time the "people" terms in a tabletop.** With a colleague (or alone with a stopwatch), simulate the page: how long to get from "alert fires" to "restore command running"? Write each event with a timestamp into `timeline.txt` and rerun `rto_timeline.py`.
5. **Clean up.**

```bash
rm -rf ~/dr-lab/rto/lab
```

7 Real-world scenario

Illustrative. `shop-api` promises a 1-hour RTO for checkout. The team tests it once: a full restore of the 250 GB production database from snapshot in the same region takes 38 minutes in a quiet drill, and the team declares victory. In the real region-level incident six months later, the snapshot copy in the second region is only 14 hours old (the cross-region copy ran nightly), the Terraform state is in the failed region's S3 bucket, and the second region has never run the application so quotas for the required instances are too low. Detection to decision takes 25 minutes, provisioning needs a quota increase (another 40 minutes), and the data restore starts only at 01:10, nearly two hours after the failure. Total outage: about 3 hours 20 minutes. In the postmortem they adopt a **pilot light** design: continuous WAL shipping to the second region, the Terraform state replicated, pre-requested quotas, and an AMI/container images present in both regions. Their next drill measures a 38-minute RTO for the region scenario, and they publish the timeline. **Lesson:** RTO is a property of the *whole path*, including quotas, state, people and cross-region copies, not of the restore command alone.

8 Common mistakes and best practices

Don't

- Quote an RTO derived from the restore command's runtime.
- Forget dependencies: DNS, identity, secrets, Terraform state and container registries must be available in the recovery site.
- Choose active-active because it sounds safe; it introduces data-conflict and operational complexity that must be justified by the business.
- Assume service quotas and capacity are available in the DR region at the moment you need them.
- Set one RTO for everything; tier your services.

Do

- Tie each tier of service to a documented RTO and RPO and review them with the business annually.
- Break down the RTO into terms, measure each in a drill, and attack the largest.
- Pre-provision what is slow: quotas, images, state, DNS TTLs, IAM roles, certificates (Infrastructure as Code §0.3).
- Replace restore with promote where the RTO demands it (Failover §15.5).
- Re-run the timed drill after major changes and keep the evidence (Failure Scenario Drills §15.10).

9 Tasks

- [ ] **T1 — Build the RTO timer and use it** *(≈30 min, beginner)*
  - **Goal:** produce a per-phase breakdown from a real run.
  - **Steps:**
    1. Create `rto_timeline.py` (guide step 1).
    2. Run the restore lab from Restore Procedures §15.4, writing the time of each event (failure, alert, decision, restore done, validation, cut-over) by hand.
    3. Feed it to the script with a target.
  - **Done when:** the script prints the per-phase table and a MET or MISSED verdict from your own timestamps.
- [ ] **T2 — Restore time versus size** *(≈45 min, intermediate)*
  - **Goal:** estimate production restore time with evidence.
  - **Steps:**
    1. Run `rto-bench.sh` and add a larger data set (for example 10 million rows) if your disk allows.
    2. Plot or tabulate MB/s versus size and the effect of `-j`.
    3. Extrapolate to a 100 GB and 1 TB database and note the uncertainty.
  - **Done when:** you have a table with measured rows plus extrapolated rows clearly marked as estimates.
- [ ] **T3 — Compare two tiers on the same workload** *(≈45 min, intermediate)*
  - **Goal:** put numbers on "backup and restore" versus "warm standby".
  - **Steps:**
    1. Time a full restore (cold) of your data set.
    2. Build a streaming replica (Replication §15.2) and time `pg_ctl promote`.

3. Estimate the monthly cost difference using your cloud's public pricing calculator (compute, storage, transfer; no resources created).

- **Done when:** a 2-row table shows RTO, RPO, extra monthly cost and what the business gets for the money.

- [ ] **T4 — Break it: hidden dependency** *(≈45 min, advanced)*
  - **Goal:** find the dependency that would blow your RTO.
  - **Steps:**
    1. List what the `shop-api` restore needs that is not the data (credentials, images, DNS zone, Terraform state, certificates, quotas, people).
    2. For each, ask "where does it live, and does it survive the failure we are planning for?".
    3. Simulate one failing: for example restore with the encryption key unavailable, or without network access to your registry, and time how long it takes to unblock.
  - **Done when:** you have a list of at least three dependencies with their location and one measured delay caused by a missing one.

- [ ] **T5 — Write a tiered RTO/RPO table and wire in alerts** *(≈60 min, advanced)*
  - **Goal:** make the targets operational.
  - **Steps:**
    1. Classify the services of `shop-api` into tiers 0-3 with RTO, RPO and DR strategy for each, and justify with revenue per minute.
    2. For tier 1, write the alert that would page you early enough to still meet the RTO (for example 5xx rate over 2 minutes, with the page routed to a runbook).
  - **Done when:** the table is complete, every tier-1 item names its runbook, and the page-to-decision time budget fits inside the RTO.

- [ ] **T6 — Prove it: the budget conversation** *(≈45 min, advanced)*
  - **Goal:** defend an RTO and its cost in plain language.
  - **Steps:**
    1. Without notes, explain to a non-engineer why a 15-minute RTO costs more than a 4-hour one, naming the four strategies and what each leaves running.
    2. Present your measured timeline from T1 and say which term you would shorten first and how.
  - **Done when:** you can state your achieved RTO for one scenario with evidence, and the single change that would cut it the most.

10 Self-check questions

1. Name the terms that add up to an RTO. Which is usually the biggest?
2. What distinguishes pilot light from warm standby?
3. Why is your database restore not your RTO?
4. A customer SLA allows about 22 minutes of downtime a month. Is a 2-hour RTO acceptable? What should you do?
5. Which DR strategy would you choose for a tier-1 checkout service with an RTO of 15 minutes, and what would worry you?

ANSWERS

1. Detection, decision, provisioning, data restore or promote, validation and traffic cut-over. Depending on the system, data restore or the human terms (detection and decision) dominate; measure to know.
2. Pilot light keeps data replicated and the core pieces ready but application capacity off or minimal, so you must start and scale it. Warm standby runs a complete scaled-down stack that can serve traffic and only needs scaling up.
3. The RTO includes everything from the failure until users are served: detecting, deciding, building infrastructure, validating and switching traffic. The restore is one term.
4. No. One incident would consume several months of downtime budget. Either improve the design (warm standby or active-active for that service) or renegotiate the SLA to match what you can really do, and document it.
5. Warm standby (or active-active) with an automated or one-command promote and a low DNS TTL. Concerns: data lag at promote time (RPO), capacity quotas in the second region, shared dependencies, and whether the team has actually rehearsed the cut-over.

11 Further reading

- AWS whitepaper *Disaster Recovery of Workloads on AWS: Recovery in the Cloud* and the Well-Architected Framework Reliability Pillar.
- *Site Reliability Engineering* and *The Site Reliability Workbook* (Google): chapters on managing incidents and disaster recovery.
- PostgreSQL documentation: *pg_restore* (parallel restore with `-j`) and *High Availability, Load Balancing, and Replication*.
- Netflix Technology Blog posts on regional failover and chaos engineering.
- OVHcloud's public communications and postmortem about the March 2021 Strasbourg fire.

- Related: Backups §15.1, Failover §15.5, High Availability and Scalability §0.8, Disaster Recovery Architecture §19.9.

LAB NOTES What I ran, what I saw, what it means

15.8 Off-site Backups

Topic 8 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

An **off-site backup** is a copy of your backup that lives in a different failure domain from the system it protects: another building, another cloud **region**, and, most importantly for modern attacks, another **account** with different credentials. In the 3-2-1 rule (Backups §15.1) it is the "1". "Off-site" originally meant a tape in a vault across town; in the cloud it means *a copy that a fire, a region outage, a deleted account or a stolen administrator password in the production environment cannot reach*.

Two properties make an off-site copy trustworthy. **Isolation:** the production account's credentials cannot delete or overwrite it (cross-account, write-only or append-only access). **Immutability:** even someone with the backup account's own credentials cannot delete or modify it until a retention period ends. On AWS S3 the immutability feature is **Object Lock**, with *governance* mode (privileged users can bypass), *compliance* mode (nobody can, not even the root user, until the retention date) and *legal hold* (an indefinite flag you can remove). Equivalents exist elsewhere: Azure immutable blob storage, Google Cloud bucket retention policies and Bucket Lock, AWS Backup Vault Lock, and the append-only modes of backup tools.

2 Why it matters in DevOps

- **Ransomware and malicious insiders target backups first.** An attacker who gets production admin rights deletes snapshots and backup buckets before encrypting data. If your only backups sit in the same account, the attacker owns them.
- **Region and site failures happen.** A backup in the same region as the primary dies with it (OVHcloud example).
- **Accidents look like attacks.** A mistaken `terraform destroy` or an expired payment method that suspends an account removes everything in that account, backups included.

- **Replication is not off-site backup.** Cross-region replication copies deletes and corruption quickly; combined with versioning and Object Lock it becomes a safe copy, but on its own it is only a faster way to spread a mistake.
- **Compliance.** Regulations and cyber-insurance questionnaires increasingly ask for offline or immutable backups and tested restores.

3 Real-world system example

OVHcloud Strasbourg fire, March 2021. A fire destroyed one data center (SBG2) and caused outages in neighbouring buildings of the same site. It was widely reported that some customers lost data permanently because their backups, or the snapshots they relied on, were kept at the same site. OVHcloud published updates and a postmortem; the practical lesson, stated by many engineers afterwards, is that an "off-site" copy must be in a different *physical* site, not merely a different server in the same campus.

AWS S3 Object Lock and AWS Backup. S3 Object Lock provides write-once-read-many (WORM) storage on versioned buckets, with retention modes described above. AWS Backup offers **Backup Vault Lock**, which can enforce a minimum retention for recovery points in a vault; in compliance mode, after a short cooling-off period it can no longer be changed or removed. AWS also supports copying recovery points to a vault in another region or another account, and "logically air-gapped" vaults. AWS's Well-Architected guidance recommends separating backups into a dedicated account.

Ransomware-resistant backups in the wider industry. Backup vendors and cloud providers document the same pattern: immutable object storage, a separate security domain, and tested restores. Tools such as restic and Borg offer append-only repository modes for the same purpose.

4 Diagram

A defence-in-depth layout for `shop-api` backups. The production account can write, but cannot delete; the backup account is a separate trust domain:

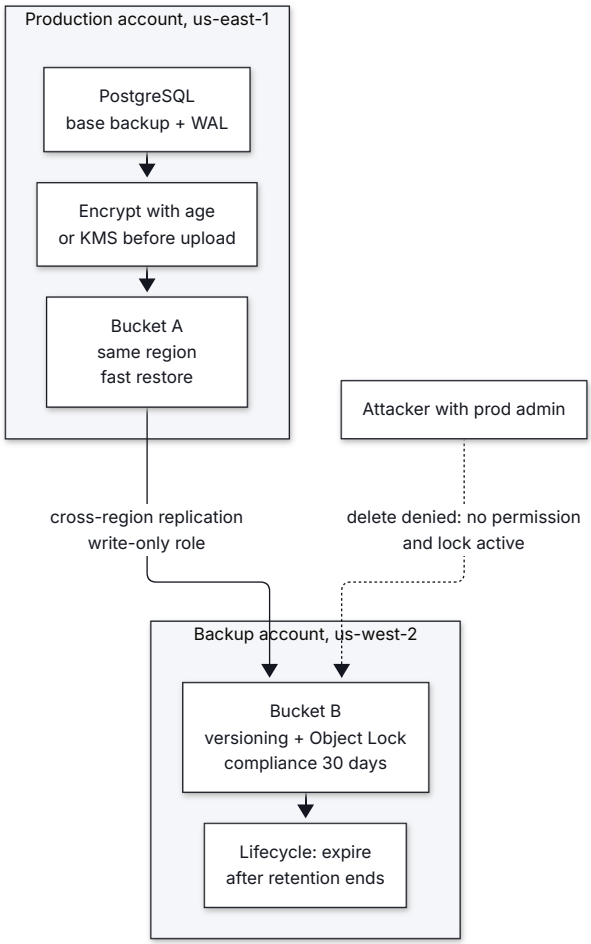


Figure 15.8-1

The ransomware attempt, step by step:

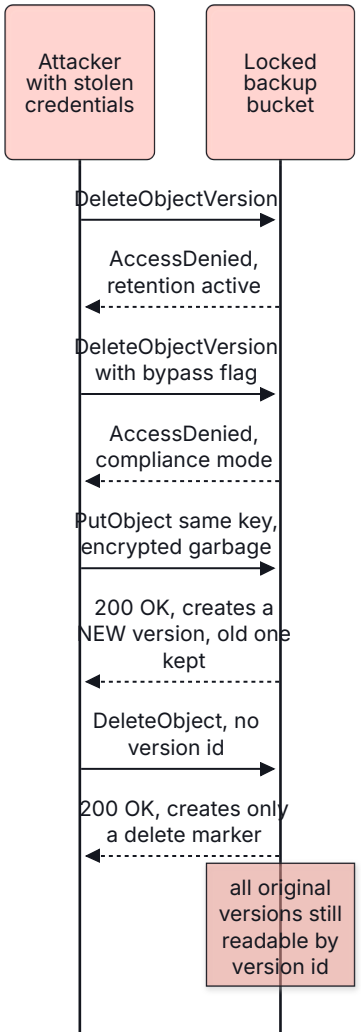


Figure 15.8-2

5 How it works

Where to put the second copy (increasing protection):

Copy	Protects against	Does not protect against
Other server, same rack or AZ	disk, host failure	fire, AZ outage, account compromise
Other AZ, same region and account	AZ outage	region outage, account compromise

Copy	Protects against	Does not protect against
Other region, same account	region outage	account compromise, admin mistakes
Other region, **other account**	region outage, account compromise	a bug that corrupts data *before* upload (needs restore tests), loss of the encryption key
Other region, other account, **immutable**	all of the above including ransomware	the retention window expiring before you notice, key loss
Provider-independent (other cloud, or offline tape or disk)	provider-wide failure or account termination	slow restore, operational overhead

Object Lock modes

Mode	Who can delete before the date	Typical use
Governance	principals with `s3:BypassGovernanceRetention` permission who pass the bypass flag	internal protection against accidents, easy to undo
Compliance	nobody, including the root user; retention cannot be shortened	ransomware defence, regulatory WORM; **cost cannot be avoided by deleting**, choose retention carefully
Legal hold	anyone with `s3:PutObjectLegalHold` can remove the hold	investigations, indefinite hold

Object Lock requires **versioning**. A plain delete on a versioned bucket just adds a *delete marker*; an overwrite adds a new version. The old versions stay until their retention ends. Remember that with compliance mode you pay for stored bytes until the date passes even if you made a mistake.

Cross-account design checklist

1. A dedicated **backup account** in the AWS Organization, with few humans and no day-to-day workloads; protect the management account and enable MFA everywhere.
2. The production role gets only `s3:PutObject` (and replication permissions) on the backup bucket. A bucket policy denies `s3:DeleteObjectVersion`, `s3:PutBucketObjectLockConfiguration`, lifecycle changes and `s3:BypassGovernanceRetention` for everyone except a break-glass role that requires MFA.
3. Encrypt before upload (age, GPG, or a KMS key whose policy lives in the backup account). Keep the key somewhere that survives (see Backups §15.1); store a **sha256** next to every backup.
4. Replicate in the background. S3 replication copies new objects, needs versioning on both buckets, and does not replicate delete markers or version deletions by default for the safety you want here; existing objects need a one-time batch replication.

5. Alert on anomalies: backup age, size drops, any `Delete*` or `PutBucketLifecycle*` API call in the backup account (CloudTrail, see CloudTrail §6.17).
6. Test restoring *from the off-site copy*, in the DR region, with only the backup account's access (Testing Backup Restoration §15.9).

Costs. Storage in a second region, cross-region data transfer (typically charged per GB transferred out of the source region), request charges, and the retention you cannot shorten in compliance mode. Use lifecycle rules to move older backups to cheaper storage classes after the lock expires; see Storage Lifecycle Policies §20.6.

6 Step-by-step guide

This guide uses a **local S3-compatible emulator (moto)** so you can practise Object Lock for free, with no AWS account and no risk. The emulator reproduces S3 API behaviour for the cases below; it does not replace testing against real S3 (real IAM, replication and Vault Lock are not emulated).

Prerequisites: Linux or macOS, Python 3.10+, `age` (`sudo apt-get install -y age`), a normal user. Ports used: 18500 on loopback only.

1. **Install the emulator and a client in a virtual environment, and start it.** boto3 is pinned to an older release because newer client defaults send checksum headers that this emulator version rejects on locked buckets; real AWS accepts both.

```bash
mkdir -p ~/dr-lab/offsite && cd ~/dr-lab/offsite
python3 -m venv venv && ./venv/bin/pip install -q "moto[server]==5.0.28"
  "boto3==1.35.99" "botocore==1.35.99"
./venv/bin/moto_server -H 127.0.0.1 -p 18500 > moto.log 2>&1 &
echo $! > moto.pid; sleep 3
export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test        # dummy credentials,
  accepted by the emulator only
```

2. **Create an encrypted backup file and its keys.** The random bytes stand in for the `pg_dump` from Backups §15.1; the pipeline is the same.

```bash
age-keygen -o key.txt 2>/dev/null
PUB=$(grep -o 'age1[a-z0-9]*' key.txt | head -1)
head -c 3000000 /dev/urandom | age -r "$PUB" -o shop-2026-10-09.dump.age
ls -l shop-2026-10-09.dump.age
```

3. **Write the off-site tool.**

BASH

```
cat > offsite.py <<'EOF'
#!/usr/bin/env python3
"""offsite.py - upload encrypted backups to an Object Lock bucket, verify them, and
  simulate an attacker.
Usage: offsite.py init | upload FILE | verify KEY | attack KEY | recover KEY"""
import datetime, hashlib, os, sys
import boto3
from botocore.exceptions import ClientError

BUCKET = "shop-api-backups-offsite"
s3 = boto3.client("s3", endpoint_url=os.environ.get("S3_ENDPOINT",
  "http://127.0.0.1:18500"),
                  region_name="us-east-1")   # credentials come from AWS_ACCESS_KEY_ID
  / AWS_SECRET_ACCESS_KEY

def sha256(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()

def init():
    s3.create_bucket(Bucket=BUCKET, ObjectLockEnabledForBucket=True)   # lock must be
  enabled at creation; enables versioning too
    s3.put_object_lock_configuration(Bucket=BUCKET, ObjectLockConfiguration={
        "ObjectLockEnabled": "Enabled",
        "Rule": {"DefaultRetention": {"Mode": "COMPLIANCE", "Days": 7}}})
    print("bucket ready: versioning on, default retention COMPLIANCE 7 days")

def upload(path):
    key = os.path.basename(path)
    digest = sha256(path)
    until = datetime.datetime.now(datetime.timezone.utc) + datetime.timedelta(days=7)
    with open(path, "rb") as f:
        r = s3.put_object(Bucket=BUCKET, Key=key, Body=f, Metadata={"sha256": digest},
                          ObjectLockMode="COMPLIANCE",
  ObjectLockRetainUntilDate=until)
    print(f"uploaded {key} version={r['VersionId']} sha256={digest[:16]}... locked
  until {until.date()}")

def verify(key):
    obj = s3.get_object(Bucket=BUCKET, Key=key)
    want = obj["Metadata"]["sha256"]
    got = hashlib.sha256(obj["Body"].read()).hexdigest()
    print("verify:", "OK" if got == want else f"MISMATCH {got} ≠ {want}")
    sys.exit(0 if got == want else 1)

def attack(key):
    """What a thief holding full credentials can and cannot do."""
```

```
    vers = [v["VersionId"] for v in s3.list_object_versions(Bucket=BUCKET,
  Prefix=key).get("Versions", [])]
    for vid in vers:
        for bypass in (False, True):
            try:
                s3.delete_object(Bucket=BUCKET, Key=key, VersionId=vid,
  BypassGovernanceRetention=bypass)
                print(f"  delete version {vid[:8]} bypass={bypass}: DELETED  ←— lock
  failed")
            except ClientError as e:
                print(f"  delete version {vid[:8]} bypass={bypass}:
  {e.response['Error']['Code']}")
    s3.put_object(Bucket=BUCKET, Key=key, Body=b"ENCRYPTED BY RANSOMWARE")   #
  overwrite attempt = a new version
    s3.delete_object(Bucket=BUCKET, Key=key)                                 # plain
  delete = a delete marker only
    n = len(s3.list_object_versions(Bucket=BUCKET, Prefix=key).get("Versions", []))
    print(f"  after overwrite and delete the bucket still holds {n} versions of
  {key}")

def recover(key):
    """Find the version whose bytes match the sha256 recorded at upload time and
  download it."""
    for v in s3.list_object_versions(Bucket=BUCKET, Prefix=key).get("Versions", []):
        obj = s3.get_object(Bucket=BUCKET, Key=key, VersionId=v["VersionId"])
        body, want = obj["Body"].read(), obj["Metadata"].get("sha256")
        ok = want == hashlib.sha256(body).hexdigest()
        print(f"  version {v['VersionId'][:8]} modified {v['LastModified']:%H:%M:%S}
  checksum {'GOOD' if ok else 'bad'}")
        if ok:
            out = "recovered-" + key
            open(out, "wb").write(body)
            print(f"  saved the good version to {out}")
            return
    sys.exit("no good version found")

{"init": lambda: init(), "upload": lambda: upload(sys.argv[2]), "verify": lambda:
  verify(sys.argv[2]),
 "attack": lambda: attack(sys.argv[2]), "recover": lambda: recover(sys.argv[2])}
  [sys.argv[1]]()
EOF
```

4. **Create the locked bucket, upload and verify.**

```bash
./venv/bin/python offsite.py init
./venv/bin/python offsite.py upload shop-2026-10-09.dump.age
./venv/bin/python offsite.py verify shop-2026-10-09.dump.age
```

Similar to: `bucket ready: versioning on, default retention COMPLIANCE 7 days`, `uploaded ... locked until <date 7 days from now>`, `verify: OK`.

5. **Play the ransomware actor.**

```bash
./venv/bin/python offsite.py attack shop-2026-10-09.dump.age
./venv/bin/python offsite.py verify shop-2026-10-09.dump.age 2>&1 | tail -1   #
  NoSuchKey: hidden by a delete marker
```

Expected: both deletes `AccessDenied`, then `after overwrite and delete the bucket still holds 2 versions` (the original plus the attacker's garbage). The last command prints a `NoSuchKey` error: the normal "latest object" view is now broken, exactly what the attacker wanted; the data is not gone.

6. **Recover: find the good version by checksum and prove it decrypts.**

```bash
./venv/bin/python offsite.py recover shop-2026-10-09.dump.age
sha256sum shop-2026-10-09.dump.age recovered-shop-2026-10-09.dump.age
age -d -i key.txt recovered-shop-2026-10-09.dump.age | wc -c
```

Expected: one version `GOOD`, one `bad`; both sha256 lines identical; `3000000` bytes after decryption. This is the recovery you would script in a real incident, using the stored checksum to choose between versions.

7. **Compare governance and legal hold, and see what bypass means.**

```bash
cat > modes.py <<'EOF'
import datetime, boto3
from botocore.exceptions import ClientError
s3 = boto3.client("s3", endpoint_url="http://127.0.0.1:18500", region_name="us-east-
  1")
B = "gov-test"
s3.create_bucket(Bucket=B, ObjectLockEnabledForBucket=True)
until = datetime.datetime.now(datetime.timezone.utc) + datetime.timedelta(days=7)
v1 = s3.put_object(Bucket=B, Key="k", Body=b"v1", ObjectLockMode="GOVERNANCE",
  ObjectLockRetainUntilDate=until)["VersionId"]
try: s3.delete_object(Bucket=B, Key="k", VersionId=v1)
except ClientError as e: print("governance delete →", e.response["Error"]["Code"])
s3.delete_object(Bucket=B, Key="k", VersionId=v1, BypassGovernanceRetention=True)
print("governance + bypass flag → deleted")
v3 = s3.put_object(Bucket=B, Key="h", Body=b"x", ObjectLockLegalHoldStatus="ON")
  ["VersionId"]
try: s3.delete_object(Bucket=B, Key="h", VersionId=v3)
except ClientError as e: print("legal hold delete →", e.response["Error"]["Code"])
s3.put_object_legal_hold(Bucket=B, Key="h", VersionId=v3, LegalHold={"Status": "OFF"})
```

```
s3.delete_object(Bucket=B, Key="h", VersionId=v3); print("hold released, then
  deleted")
EOF
./venv/bin/python modes.py
```

Expected: `governance delete → AccessDenied`, `governance + bypass flag → deleted`, `legal hold delete → AccessDenied`, `hold released, then deleted`. Governance stops accidents and unprivileged users, but anyone with the bypass permission (including a stolen admin role) can still delete. Compliance has no bypass. That difference is why ransomware defence uses compliance mode or a separate account whose admin credentials the attacker does not have. (The emulator does not enforce IAM; on real S3 the bypass flag also needs the `s3:BypassGovernanceRetention` permission.)

8. **Immutability on a plain Linux file system (single-host, weaker).** The `chattr +i` flag makes a file undeletable even by root until the flag is cleared.

```bash
mkdir -p imm && cp shop-2026-10-09.dump.age imm/
sudo chattr +i imm/shop-2026-10-09.dump.age
rm imm/shop-2026-10-09.dump.age; lsattr imm/
sudo chattr -i imm/shop-2026-10-09.dump.age && rm -r imm
```

Expected: `rm: cannot remove ...: Operation not permitted` and `----i---` in the `lsattr` listing. It stops accidents and simple malware, but anyone with root can run `chattr -i`, which is exactly why a locked remote bucket under *different* credentials is stronger. (Works on ext4 and xfs; some file systems and containers do not support it.)

9. **Cross-region and cross-account on real AWS: read-only reference. Not run here, illustrative, untested; costs money if applied.** The shape of the Terraform for the backup account:

```hcl
resource "aws_s3_bucket" "offsite" {
  bucket              = "example-shop-api-backups-offsite"
  object_lock_enabled = true
}
resource "aws_s3_bucket_versioning" "offsite" {
  bucket = aws_s3_bucket.offsite.id
  versioning_configuration { status = "Enabled" }
}
resource "aws_s3_bucket_object_lock_configuration" "offsite" {
  bucket = aws_s3_bucket.offsite.id
  rule {
    default_retention {
      mode = "COMPLIANCE"
      days = 30
    }
```

```
  }
}
# plus: a bucket policy that allows only the production replication role to write,
# denies s3:DeleteObjectVersion and s3:PutBucketObjectLockConfiguration to everyone,
# and an aws_s3_bucket_replication_configuration on the source bucket.
```

Cost and safety warning: a COMPLIANCE lock cannot be removed, so you pay for the stored data for the full retention. Test with 1 day, a tiny bucket and a billing alarm, and in a sandbox account. Teardown: after the retention date, empty all versions and delete markers, then delete the bucket (`aws s3api list-object-versions` shows what remains).

10. **Clean up the emulator.**

```bash
kill "$(cat moto.pid)"; cd ~ && rm -rf ~/dr-lab/offsite
```

7 Real-world scenario

Illustrative. An attacker phishes a `shop-api` engineer's cloud console session, creates a new access key on a production admin role, and over 40 minutes: deletes the RDS automated backups and manual snapshots, empties the backup bucket in the production account, and finally encrypts the database volume. The on-call engineer notices through a CloudTrail alert on `DeleteDBSnapshot`. The production backups are gone. The cross-account copy in the backup account in `us-west-2` is intact: the replication role in production could write but not delete, and the objects are locked for 30 days. The team recovers the previous night's base backup plus WAL from the locked bucket, using break-glass access to the *backup account* and the encryption key stored in the password manager. Restore into a fresh VPC with Terraform takes 2 hours 10 minutes, RPO about 4 minutes (the replication lag at the time). **Lesson:** the production credentials could do anything in production, and that is exactly why the backup account existed; the drill had already proven the restore from the copy.

8 Common mistakes and best practices

Don't

- Put backups in the same account, region or credential domain as production.
- Rely on replication alone; it copies deletions and corruption.
- Use governance-mode locks and give wide `BypassGovernanceRetention` permissions, then call it ransomware-proof.
- Set a COMPLIANCE retention longer than you can afford before testing with a short one.

- Forget the key: encrypted backups with the key only in the lost account are unrecoverable.

Do

- Use a dedicated backup account with write-only cross-account access from production and MFA-protected break-glass access.
- Turn on versioning and Object Lock (or Vault Lock) on the final copy, and keep a checksum with every backup.
- Alert on any destructive API calls in the backup account and on backup age and size anomalies.
- Keep one copy that is independent of your cloud provider if the business cannot survive provider-level loss.
- Restore from the off-site copy on a schedule (Testing Backup Restoration §15.9); see also S3 §6.8, KMS §6.15 and IAM §6.7.

9 Tasks

- [ ] **T1 — Lock a bucket and try to break it** *(≈45 min, beginner)*
  - **Goal:** observe Object Lock behaviour in practice.
  - **Steps:** follow guide steps 1-5.
  - **Done when:** both delete attempts print `AccessDenied` and the bucket still contains the original version after the "attack".
- [ ] **T2 — Recover by checksum** *(≈30 min, intermediate)*
  - **Goal:** pick the right version after tampering.
  - **Steps:**
    1. Run `recover`, then decrypt the result with your `age` key.
    2. Modify the script to download *all* versions and print their sizes and checksum status.
  - **Done when:** the recovered file's sha256 equals the original and you can explain how the tool decided which version was good.
- [ ] **T3 — Governance versus compliance versus legal hold** *(≈30 min, intermediate)*
  - **Goal:** explain which mode defends against which actor.
  - **Steps:**
    1. Run `modes.py`.

2. Extend it to attempt to *shorten* a compliance retention and to delete the bucket while versions exist; record the errors.
3. Write a table: actor (accident, junior admin, stolen admin credentials, root of the account) versus mode (governance, compliance, hold) with "protected" or "not protected".

- **Done when:** your table is filled in and you can defend each cell from the output or the documentation.

☐ **T4 — Break it: unencrypted or key-less backup** *(≈30 min, intermediate)*

- **Goal:** feel what losing the key means.
- **Steps:**
  1. Create an encrypted backup and delete `key.txt`; try to decrypt.
  2. Restore the key from where your plan says it is stored (password manager, second location), or conclude the backup is lost.
  3. Add a "key location and recovery" section to your backup policy.
- **Done when:** you proved the file is undecryptable without the key and your policy names two independent places for the key.

☐ **T5 — Automate off-site copy with monitoring** *(≈60 min, advanced)*

- **Goal:** a nightly pipeline: dump, encrypt, upload with checksum, verify, alert.
- **Steps:**
  1. Combine the `pg_dump | age` pipeline from Backups §15.1 with `offsite.py upload` in a script with `set -euo pipefail`.
  2. Add a check that fails when the newest object in the bucket is older than 26 hours or its checksum does not verify.
  3. Run it twice and break it (remove credentials) to see the alert.
- **Done when:** the check exits non-zero for a stale backup and zero for a fresh one, and the log shows the two uploads.

☐ **T6 — Prove it: design the isolation** *(≈45 min, advanced)*

- **Goal:** explain a ransomware-proof backup design without notes.
- **Steps:**
  1. Draw the accounts, regions, roles and buckets for `shop-api`; mark who can write, who can read, who can delete and what is locked.
  2. Walk through three attacks (stolen prod admin, deleted production account, insider with backup-account admin) and say what survives each.

3. State the retention and the monthly cost of keeping it (use a public pricing calculator, create nothing).

- **Done when:** every attack has a surviving copy, you have named the residual risk (for example key loss or retention shorter than detection time), and the cost is written down.

10 Self-check questions

1. Why is "cross-region, same account" not enough against a ransomware attack?
2. What is the difference between governance and compliance mode in S3 Object Lock?
3. A plain delete is issued on a versioned bucket. What happens to the data?
4. Why must Object Lock be paired with a checksum and a restore test?
5. What costs money in an off-site design, and what is the risk of a very long compliance retention?

ANSWERS

1. An attacker with credentials for the account can delete the replicated copies as easily as the originals; only a separate trust domain (another account with different credentials) and/or immutability prevents that.
2. In governance mode a principal with the bypass permission can delete or change the lock; in compliance mode nobody can, including the account root user, until the retention date.
3. A delete marker is added and the object appears gone in the normal view, but earlier versions remain and can be listed and retrieved by version id (and locked versions cannot be removed).
4. Lock guarantees that bytes are not changed after write, not that they were correct when written or that you can use them. A checksum proves integrity and a restore test proves usability, including keys and procedure.
5. Storage in a second region, cross-region transfer, requests and the locked retention window; a long compliance lock means you pay for mistakes (for example uploading huge wrong data) until it expires and cannot delete them early.

11 Further reading

- AWS documentation: *Amazon S3 Object Lock*, *S3 Replication*, *AWS Backup Vault Lock* and *Logically air-gapped vault*.
- AWS Well-Architected Framework: Reliability Pillar and Security Pillar guidance on backups and account separation.

- Azure documentation on immutable blob storage; Google Cloud documentation on retention policies and Bucket Lock.
- restic and Borg documentation on append-only repositories; the NIST guidance on data integrity and ransomware recovery.
- OVHcloud's public communications and postmortem on the March 2021 Strasbourg fire.
- Related: Backups §15.1, Snapshots §15.3, Disaster Recovery overview §0.9, Secrets Management and Encryption §11.4.

LAB NOTES What I ran, what I saw, what it means

15.9 Testing Backup Restoration

Topic 9 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

Testing backup restoration means regularly, and automatically, doing what you would do in a disaster: take the newest backup, decrypt it, restore it into a throwaway environment, check that the result is complete and correct, measure how long it took, and keep the result as evidence. The principle is old and blunt: **a backup you have never restored is a hope, not a backup** (often attributed to the sysadmin saying "Schrödinger's backup": its state is unknown until you look).

A restore test checks four different things that a green "backup job succeeded" message does not:

1. **Integrity:** the file is intact (a **checksum** such as SHA-256 matches the one recorded at backup time) and decryptable with the key you would have in a disaster.
2. **Usability:** the database software accepts it and restores without errors, on the version and hardware you would use in recovery.
3. **Completeness and correctness:** the restored data matches what was in production when the backup was taken (row counts, sums, a manifest of expected content).
4. **Speed:** the restore finishes within the time budget; the measured duration feeds your RTO (RTO §15.7).

2 Why it matters in DevOps

- **Backups fail silently.** Full disks, expired credentials, changed schemas, a new table excluded by a well-meaning optimisation, an upgrade that changes dump format: the job exits 0 and the file is useless.
- **It is the only way to know your RTO.** Estimates from restore throughput are guesses until you time a restore on production-sized data (RTO §15.7).
- **It tests the people and the procedure too.** Missing keys, wrong runbook steps and expired access are found in drills, not at 3 a.m. (Restore Procedures §15.4).

- **It produces evidence.** Auditors (SOC 2, ISO 27001) and insurers want proof that recoverability is tested; a dated log of automated restores is exactly that.
- **It catches slow-burn corruption early,** while an older, good backup still exists within retention (Backups §15.1).

3 Real-world system example

GitLab.com, January 2017. The public postmortem lists a set of recovery mechanisms (regular backups, disk snapshots, replication and others) and states that when the team needed them, several were not working or not restorable in practice: for instance backup jobs that had failed without anyone noticing. The follow-up actions in the postmortem included monitoring backups and testing restores. It is the most-cited industry example of why restoring, not backing up, is the test.

Managed services still need your validation. With a managed database such as Amazon RDS, the provider operates the backup mechanism, but restoring still creates a new instance whose *contents* you must check, and restoring from your own copies is entirely your responsibility. Many engineering organisations run a scheduled "restore the latest production backup into a scratch environment and run checks" pipeline, sometimes also used to seed staging with realistic data (with sensitive fields masked, see Secrets and Encryption §11.4). PostgreSQL itself provides `pg_verifybackup` for physical backups and the `amcheck` extension for structural verification.

Site Reliability Engineering practice. Google's SRE books describe periodic restores of backups, and "recovery testing" as part of the data-integrity strategy, because recoverability is a property that decays unless it is exercised.

4 Diagram

An automated restore test as a pipeline. Every box can fail the run, and every outcome is

recorded:

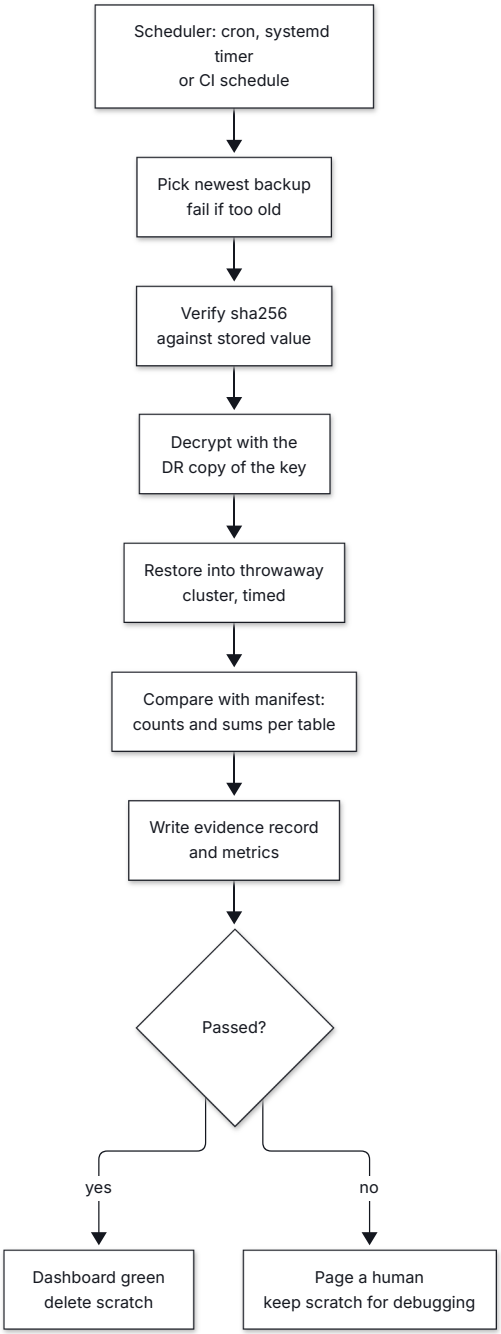


Figure 15.9-1

What a green checkmark means at each level of testing, from weak to strong:

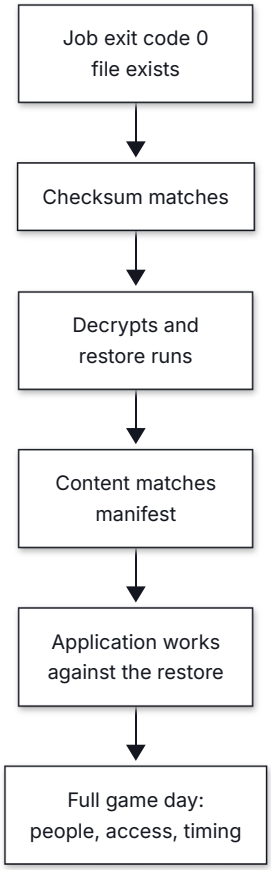


Figure 15.9-2

5 How it works

What to record at backup time (so the restore can be judged).

Artifact	Why
`sha256` of the final (encrypted) file	detects corruption and truncation in storage or transfer
Manifest: per-table row counts and a checksum column (e.g. `sum(total_cents)`) taken from the *live* database at dump time	the restore must reproduce it; catches missing tables, truncated data and silent exclusions
Size and duration	sudden size drop is an early warning
Tool and database version	so the restore environment matches

Artifact	Why
Backup timestamp	gives the real RPO of this backup

What the restore test does. It runs on a schedule, in an isolated environment with no route to production and no ability to send real emails or webhooks. Steps: choose newest backup, check age, verify checksum, decrypt using the key from the DR location, restore, compare to manifest, run structural checks (`pg_amcheck`), run a smoke query or even the application's integration tests, record duration and result, destroy the scratch environment. Failure pages someone; success updates a metric ("last successful restore test timestamp") that is itself alerted on.

Levels of rigor

Level	What you check	Catches	Cost
Job success	exit code	crashes	free, weakest
Checksum	file unchanged since backup	storage corruption, truncation	trivial
Restore runs	restore finishes without errors	wrong version, missing key, corrupt dump	a scratch host for minutes
Content check	manifest comparison	excluded tables, partial dumps	needs a manifest
Application check	smoke test or integration tests against the restored DB	schema and permission issues	more setup
Full drill	humans follow the runbook under time pressure	process, access and communication gaps	quarterly effort (Failure Scenario Drills §15.10)

Frequency. Automated restore test: after every backup or at least daily for critical data, weekly for the rest. Full human drill: quarterly or after major changes (new region, database upgrade, new backup tool). Test restores from *each* storage location (local, replicated, off-site copy) because they have different failure modes (Off-site Backups §15.8).

Evidence. Keep an append-only log (a JSON line per test: time, backup, status, duration), export metrics, and store drill reports. Evidence answers "when did we last prove this works, and how long did it take?" without anyone having to remember.

6 Step-by-step guide

Prerequisites: Linux with PostgreSQL 16 server binaries in `/usr/lib/postgresql/16/bin` , `age` (`sudo apt-get install -y age`), a normal non-root user. Free and local; uses ports 55480 ("production") and 55481 (the throwaway restore target) and a scratch directory.

Nothing existing is touched.

1. **Create a "production" database and an encryption key.**

```bash
export PATH=/usr/lib/postgresql/16/bin:$PATH
export W=$HOME/dr-lab/restore-test && rm -rf "$W" && mkdir -p "$W/backups"
  "$W/evidence"
export PGHOST=/tmp PGPORT=55480
initdb -D "$W/prod" >/dev/null
printf "port = 55480\nunix_socket_directories = '/tmp'\nlisten_addresses = ''\n" >>
  "$W/prod/postgresql.conf"
pg_ctl -D "$W/prod" -l "$W/prod.log" -w start
createdb shop
psql -X -q -d shop <<'SQL'
CREATE TABLE customers(id serial PRIMARY KEY, email text UNIQUE NOT NULL);
CREATE TABLE orders(id serial PRIMARY KEY, customer_id int NOT NULL REFERENCES
  customers(id),
                    total_cents int NOT NULL, created_at timestamptz DEFAULT now());
INSERT INTO customers(email) SELECT 'user'||g||'@example.com' FROM
  generate_series(1,5000) g;
INSERT INTO orders(customer_id,total_cents) SELECT 1+g%5000, 100+g%9000 FROM
  generate_series(1,200000) g;
SQL
age-keygen -o "$W/key.txt" 2>/dev/null
```

   In real life `key.txt` would **not** live next to the backups; the test uses it from the "DR copy" location.

2. **Write the backup script: encrypt, checksum, manifest.**

```bash
cat > "$W/backup.sh" <<'EOF'
#!/usr/bin/env bash
# backup.sh - encrypted dump + sha256 + manifest of expected content. Usage: backup.sh
  <work-dir> <age-public-key>
set -euo pipefail
export PATH=/usr/lib/postgresql/16/bin:$PATH PGHOST=/tmp PGPORT=55480
W=${1:?work dir}; PUB=${2:?age public key}
STAMP=$(date +%Y-%m-%dT%H%M%S)
OUT="$W/backups/shop-$STAMP.dump.age"
pg_dump -Fc -d shop | age -r "$PUB" > "$OUT.tmp"
mv "$OUT.tmp" "$OUT"
( cd "$W/backups" && sha256sum "$(basename "$OUT")" > "$(basename "$OUT").sha256" )
# the manifest records what a correct restore must contain, taken from the live
  database
psql -X -At -d shop -F, -c "SELECT 'customers', count(*), coalesce(sum(id),0) FROM
  customers UNION ALL SELECT 'orders', count(*), coalesce(sum(total_cents),0) FROM
  orders" > "$OUT.manifest"
echo "$STAMP $(stat -c %s "$OUT") bytes"
EOF
```

```
chmod +x "$W/backup.sh"
PUB=$(grep -o 'age1[a-z0-9]*' "$W/key.txt" | head -1)
"$W/backup.sh" "$W" "$PUB"
cat "$W"/backups/*.manifest
```

Expected: a line like `2026-10-09T221500 1311259 bytes` (size varies), then `customers,5000,12502500` and `orders,200000,912902000`.

3. **Write the automated restore test.** It never touches production: it creates its own scratch cluster on another port, destroys it on exit, and writes one JSON evidence line plus a Prometheus-style metrics file.

BASH

```
cat > "$W/restore-test.sh" <<'EOF'
#!/usr/bin/env bash
# restore-test.sh - restore the newest backup into a throwaway cluster and prove it.
# Usage: restore-test.sh <work-dir> [max_age_hours]
set -uo pipefail
export PATH=/usr/lib/postgresql/16/bin:$PATH
W=${1:?work dir}; MAXAGE=${2:-26}
export PGHOST=/tmp PGPORT=55481
SCRATCH=$(mktemp -d "$W/scratch.XXXXXX")
START=$(date +%s); STATUS=pass; REASON=""
fail() { STATUS=fail; REASON="$1"; }
cleanup() { pg_ctl -D "$SCRATCH/pg" -m immediate stop >/dev/null 2>&1; rm -rf
  "$SCRATCH"; }
trap cleanup EXIT

BK=$(ls -1t "$W"/backups/shop-*.dump.age 2>/dev/null | head -1)
[ -n "$BK" ] || fail "no backup found"
if [ "$STATUS" = pass ]; then
  AGE_H=$(( ( $(date +%s) - $(stat -c %Y "$BK") ) / 3600 ))
  [ "$AGE_H" -le "$MAXAGE" ] || fail "newest backup is ${AGE_H}h old (limit
  ${MAXAGE}h)"
fi
if [ "$STATUS" = pass ]; then ( cd "$W/backups" && sha256sum -c "$(basename
  "$BK").sha256" >/dev/null 2>&1 ) || fail "sha256 mismatch"; fi
if [ "$STATUS" = pass ]; then age -d -i "$W/key.txt" "$BK" > "$SCRATCH/shop.dump"
  2>/dev/null || fail "decryption failed"; fi
T_DECRYPT=$(date +%s)
if [ "$STATUS" = pass ]; then
  initdb -D "$SCRATCH/pg" >/dev/null 2>&1
  printf "port = 55481\nunix_socket_directories = '/tmp'\nlisten_addresses = ''\nfsync
  = off\n" >> "$SCRATCH/pg/postgresql.conf"
  pg_ctl -D "$SCRATCH/pg" -l "$SCRATCH/pg.log" -w start >/dev/null 2>&1 || fail
  "scratch cluster did not start"
fi
if [ "$STATUS" = pass ]; then
  createdb shop && pg_restore -j 2 --exit-on-error -d shop "$SCRATCH/shop.dump"
  2>"$SCRATCH/restore.err" || fail "pg_restore failed: $(head -1
  "$SCRATCH/restore.err")"
```

```
fi
T_RESTORE=$(date +%s)
if [ "$STATUS" = pass ]; then
  GOT=$(psql -X -At -d shop -F, -c "SELECT 'customers', count(*), coalesce(sum(id),0)
  FROM customers UNION ALL SELECT 'orders', count(*), coalesce(sum(total_cents),0)
  FROM orders" 2>/dev/null)
  [ "$GOT" = "$(cat "$BK.manifest")" ] || fail "content differs from manifest"
fi
END=$(date +%s)
printf
  '{"time":"%s","backup":"%s","status":"%s","reason":"%s","restore_seconds":%d,"total_
  seconds":%d}\n' \
  "$(date -u +%FT%TZ)" "$(basename "${BK:-none}")" "$STATUS" "$REASON" "$(( T_RESTORE
  - T_DECRYPT ))" "$(( END - START ))" | tee -a "$W/evidence/restore-tests.jsonl"
# metrics for node_exporter's textfile collector: alert if the success timestamp gets
  old
{ echo "backup_restore_test_success $([ "$STATUS" = pass ] && echo 1 || echo 0)"
  echo "backup_restore_test_duration_seconds $(( END - START ))"
  [ "$STATUS" = pass ] && echo "backup_restore_test_last_success_timestamp_seconds
  $END"
} > "$W/evidence/restore_test.prom"
[ "$STATUS" = pass ]
EOF
chmod +x "$W/restore-test.sh"
"$W/restore-test.sh" "$W"; echo "exit=$?"
```

Similar to: `{"time":"...","backup":"shop-2026-10-09T221500.dump.age","status":"pass","reason":"","restore_seconds":1,"total_seconds":1}` and `exit=0`. (`fsync = off` on the scratch cluster speeds up the test; never use it for production.)

4. **Prove the test can fail: corrupt one byte of the stored backup.**

```bash
F=$(ls -1t "$W"/backups/*.age | head -1)
cp "$F" "$F.good"; printf '\x00' | dd of="$F" bs=1 seek=1000 conv=notrunc 2>/dev/null
"$W/restore-test.sh" "$W"; echo "exit=$?"
mv "$F.good" "$F"
```

Expected: `"status":"fail","reason":"sha256 mismatch"` and `exit=1`. Restore the good copy afterwards.

5. **Prove it catches the nastiest failure: a backup that restores fine but is missing data.**

Simulate a well-meaning "optimisation" that excludes the biggest table.

```bash
sed 's/pg_dump -Fc -d shop/pg_dump -Fc -T orders -d shop/' "$W/backup.sh" >
  "$W/backup-broken.sh" && chmod +x "$W/backup-broken.sh"
sleep 1; "$W/backup-broken.sh" "$W" "$PUB"
"$W/restore-test.sh" "$W"; echo "exit=$?"
```

Expected: `"status":"fail","reason":"content differs from manifest"` and `exit=1` (a psql error about the missing `orders` table may print above it). The checksum passed, decryption passed and `pg_restore` exited 0: only the content check saw the problem. Remove the broken backup set (the file and its `.sha256` and `.manifest` companions) and take a fresh good one before continuing: `rm "$(ls -1t "$W"/backups/*.age | head -1)"* && "$W/backup.sh" "$W" "$PUB"`.

6. **Test staleness detection.**

```bash
touch -d '-30 hours' "$(ls -1t "$W"/backups/*.age | head -1)"
"$W/restore-test.sh" "$W"; echo "exit=$?"
```

Expected: `"reason":"newest backup is 30h old (limit 26h)"`. A job that silently stopped yesterday is detected today.

7. **Schedule it and alert on the absence of success.** Cron example (edit with `crontab -e`; adjust the paths):

```
17 3 * * *  /home/you/dr-lab/restore-test/backup.sh /home/you/dr-lab/restore-test
  <AGE_PUBLIC_KEY> >> /home/you/dr-lab/restore-test/cron.log 2>&1
47 3 * * *  /home/you/dr-lab/restore-test/restore-test.sh /home/you/dr-lab/restore-
  test >> /home/you/dr-lab/restore-test/cron.log 2>&1
```

The metrics file `evidence/restore_test.prom` can be picked up by Prometheus `node_exporter`'s textfile collector. A Prometheus alert such as `time() - backup_restore_test_last_success_timestamp_seconds > 129600` (36 hours) pages you when the restore test has not *succeeded* recently, which also covers the case where the test job itself stopped running (Prometheus §10.2). Run the restore test in CI too, as a scheduled workflow (illustrative, not run here): a GitHub Actions workflow with `on: schedule: - cron: "47 3 * * *"` that pulls the newest off-site object using a read-only role, runs the script, and uploads the JSON line as an artifact (GitHub Actions §3.11).

8. **Structural check on the restored copy (optional extra layer).** Add to the script after `pg_restore`: `psql -X -d shop -c "CREATE EXTENSION IF NOT EXISTS amcheck"` and `pg_amcheck -d shop`, as in Restore Procedures §15.4. For physical backups use

`pg_verifybackup <backup-dir>` before restoring.

9. **Clean up.**

```bash
pg_ctl -D "$W/prod" -m fast stop
```

7 Real-world scenario

Illustrative. `shop-api` has a nightly dump job that reports success for eight months. The weekly automated restore test starts failing with `content differs from manifest: orders` has 4.1 million rows in the manifest but the restore has 3.9 million. Investigation shows a schema change created `orders_2026` as a partitioned table; the dump job's exclusion pattern (`-T 'orders_*'`, added to skip temporary tables) silently omitted the new partition, so every backup since the migration was missing the newest orders. Because the restore test caught it on the first weekly run after the migration, the team lost nothing: they fixed the exclude pattern, took a full backup that night and kept the evidence file as a postmortem exhibit. Without the test, the next real restore would have returned a database with the last two months of orders missing. **Lesson:** the manifest comparison caught an error that checksum, decryption and `pg_restore` all passed.

8 Common mistakes and best practices

Don't

- Treat "backup job succeeded" as the test.
- Restore only to the same host or only from the local copy: test the off-site copy and a clean environment.
- Keep the decryption key only in the system being backed up, and forget to use the DR copy of the key in the test.
- Let the restore test touch production data stores, send email, or call external APIs.
- Hide failures: a test that fails and no one is paged is the same as no test.

Do

- Record expected content (manifest, checksums) at backup time and compare after restore.
- Test automatically on every backup or at least daily, and also quarterly with humans following the runbook.
- Time the restore at realistic data size and publish the number (RTO §15.7).

- Alert on the *absence of recent success* as well as on failure.
- Keep evidence: JSON lines, metrics, drill reports (Failure Scenario Drills §15.10).
- Re-run after every change to the backup tooling, the database version, or the schema layout.

9 Tasks

- [ ] **T1 — Build and run the automated restore test** *(≈45 min, beginner)*
  - **Goal:** a restore test that exits 0 on a good backup and writes evidence.
  - **Steps:** follow guide steps 1-3.
  - **Done when:** `evidence/restore-tests.jsonl` has a `"status":"pass"` line with a non-zero `total_seconds`.
- [ ] **T2 — Make it fail three different ways** *(≈45 min, intermediate)*
  - **Goal:** confirm the test detects corruption, staleness and missing data.
  - **Steps:** follow guide steps 4-6, then add a fourth case: delete `key.txt` (or use the wrong key) and observe the failure reason.
  - **Done when:** the evidence file contains four distinct `fail` reasons and you restored the healthy state.
- [ ] **T3 — Add an application-level check** *(≈45 min, intermediate)*
  - **Goal:** go beyond counts.
  - **Steps:**
    1. Extend the script to run `pg_amcheck` and a query that validates a business invariant (for example "no order has a customer_id that does not exist" and "no negative totals").
    2. Break the invariant in a scratch copy and confirm the test notices.
  - **Done when:** the script fails when you restore a dump in which you deliberately removed one customer with orders (use `session_replication_role = replica` to bypass the foreign key when creating the bad data).
- [ ] **T4 — Metrics and alert** *(≈45 min, advanced)*
  - **Goal:** page on missing restore evidence.
  - **Steps:**
    1. Serve `restore_test.prom` through a textfile collector or a tiny HTTP endpoint, scrape it with Prometheus (see Prometheus §10.2).
    2. Write the alert rule `time() - backup_restore_test_last_success_timestamp_seconds > 129600`.

3. Stop running the test and wait (or temporarily shorten the threshold) to see the alert fire.

- **Done when:** an alert is firing in Prometheus (or Alertmanager) for a stale restore test and clears after a successful run.

- [ ] **T5 — Time it at scale** *(≈45 min, advanced)*
  - **Goal:** a restore-time number you can put into the DR plan.
  - **Steps:**
    1. Re-create the data with 10 million orders (if your disk allows), run backup and restore test, record `restore_seconds`.
    2. Compute MB/s and extrapolate to the real database size (RTO §15.7).
  - **Done when:** your evidence file shows the large run and a table gives the estimated production restore time with the assumptions written next to it.

- [ ] **T6 — Prove it: audit pack** *(≈45 min, advanced)*
  - **Goal:** prepare what an auditor or a new CTO asks for.
  - **Steps:**
    1. Without notes, write a one-page "backup verification" statement: what is backed up, how often it is restore-tested, from which location, what is checked, the last result and the last measured time.
    2. Attach the evidence file lines and a dashboard screenshot description.
    3. List the three most likely ways your test could still give a false pass, and one mitigation each.
  - **Done when:** the page cites real dated evidence from your lab and the false-pass list includes at least "data was already wrong at backup time" and "the test ran against a different copy than the one you would use".

10 Self-check questions

1. Why is a matching checksum not enough to call a backup good?
2. What is a manifest, and what failure does it catch that `pg_restore` exit code 0 does not?
3. How often should you automate restore tests, and how often run a human drill?
4. Your restore test passes every night, but the alert for "no successful restore test in 36 hours" is also useful. Why?
5. What can a restore test *not* prove?

ANSWERS

1. A checksum proves the file is unchanged since it was written, not that it was complete or correct when written, that it can be decrypted, or that the database accepts it.
2. A record of what the restored data must contain (row counts and sums per table, taken from the live database at backup time). It catches backups that restore cleanly but are missing tables or rows.
3. Automate after every backup (or daily for critical data); run a human, timed drill at least quarterly and after major changes.
4. If the test job itself stops (cron broken, runner gone), no failure is ever reported. Alerting on the absence of a recent success covers a dead tester.
5. That data was correct before it was backed up (garbage in, garbage out), that your whole organisation can execute the process under pressure, or that a different copy (such as the offsite one) is also good unless you test that copy.

11 Further reading

- PostgreSQL documentation: `pg_verifybackup`, `pg_amcheck`, `pg_restore` and the `amcheck` module.
- GitLab's public postmortem of the 31 January 2017 database incident, and their later documentation on backup verification.
- *Site Reliability Engineering* (Google), the data integrity chapter.
- Prometheus documentation: node_exporter textfile collector and alerting rules.
- Related: Backups §15.1, Restore Procedures §15.4, Backups and PITR §13.5, Disaster Recovery overview §0.9.

15.10 Failure Scenario Drills

Topic 10 of 10 | ~6 hours | Difficulty: Advanced

1 What it is

A **failure scenario drill** is a planned, controlled rehearsal of a specific failure: you break something on purpose (or pretend you did), the team recovers it using the real runbooks, and you measure how long it took and how much data was lost. A **game day** is the team-level version: a scheduled exercise with a scenario, roles, a facilitator, observers, a timeline and a debrief. **Chaos engineering** is the continuous, automated cousin that injects small failures regularly to prove the system tolerates them.

This topic works through the five failure classes that the level's capstone asks for, from smallest to largest blast radius: **database**, **server**, **pod**, **node** and **region**. For each you will cause the failure, watch the system react, and write down three numbers: the **detection time**, the **recovery time** (your measured RTO, RTO §15.7) and the **data loss** (your measured RPO, RPO §15.6).

2 Why it matters in DevOps

- **Plans decay.** Every deploy, schema change, credential rotation and new dependency can silently break a runbook. A drill is the only test that includes people, access and timing, not just software (Testing Backup Restoration §15.9).
- **It converts RTO and RPO from promises into measurements.** "We can recover in an hour" becomes "last quarter we recovered in 41 minutes with 12 seconds of data loss".
- **Failures are cheaper on Tuesday afternoon.** Finding that the on-call engineer has no access to the DR region is a ticket during a drill and a business-threatening outage during a real event.
- **It builds muscle memory and psychological safety.** Teams that rehearse communicate better under stress and escalate earlier (Incident Management §14.5).
- **It feeds the postmortem loop.** Drill findings are tracked as action items exactly like incident findings (Postmortems §14.7).

3 Real-world system example

Netflix and chaos engineering. Netflix publicly described Chaos Monkey, which randomly terminates production instances so engineers build for instance failure, and larger exercises in the Simian Army family, including "Chaos Kong", which simulates the loss of an entire AWS region by moving traffic out of it. The pattern is: failures are routine, rehearsed and automated, so that the response is a practiced one.

Google DiRT. Google has described its Disaster Recovery Testing (DiRT) program in public talks and in the *Site Reliability Engineering* books: scheduled, company-wide exercises where teams deliberately simulate large failures (including loss of whole locations and unavailability of key people or systems) to find the weaknesses of their plans.

AWS. AWS Fault Injection Service (FIS) lets customers run controlled experiments such as stopping instances, injecting network latency or interrupting an Availability Zone's connectivity, with stop conditions tied to CloudWatch alarms; the Well-Architected Reliability Pillar recommends game days and testing resilience. **Kubernetes** projects such as Chaos Mesh and LitmusChaos (CNCF projects) provide pod, node and network faults for clusters.

Learning from real data-loss incidents. The GitLab.com database incident of January 2017 and the OVHcloud Strasbourg data center fire of March 2021 (both publicly documented, see Backups §15.1 and Off-site Backups §15.8) are standard case studies: in both, the most valuable lesson was that assumptions about backups and site independence had not been exercised before they were needed.

4 Diagram

The shape of every drill, from hypothesis to fixed runbook:

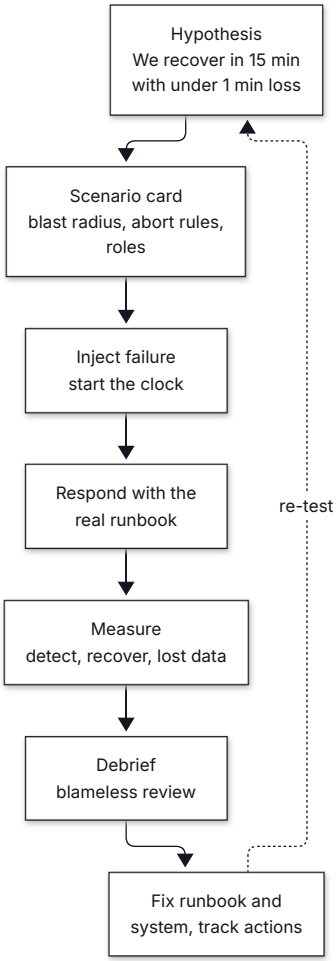


Figure 15.10-1

Blast radius grows, and so does the recovery mechanism:

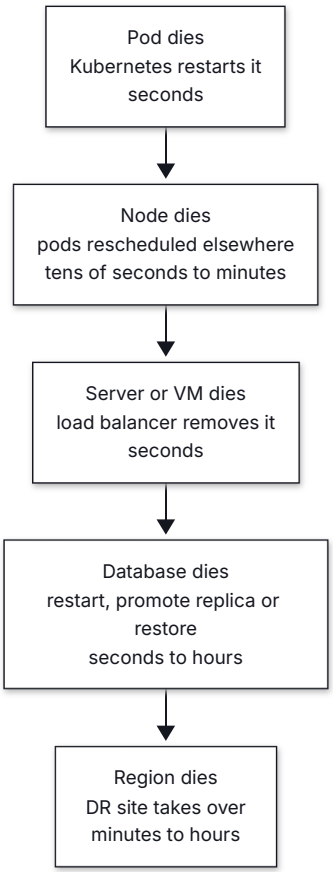


Figure 15.10-2

5 How it works

Scenario card. Every drill starts with a one-page card:

Field	Example
Name	DB-01: primary database host lost
Hypothesis	A standby is promoted and traffic resumes within 5 minutes with less than 30 s of data loss
Injection	`kill -9` on the primary's postmaster (or stop the VM)
Blast radius	staging, or a canary slice of production; never an unannounced production-wide failure
Abort criteria	customer-visible error rate above X, or any data-loss signal; the facilitator can stop at any time

Field	Example
Roles	facilitator, incident commander, operator(s), scribe, observers (customer-support liaison)
Runbook under test	link; the operator gets only this
Measurements	time to detect, time to decide, time to recover, requests failed, data lost
Success	hypothesis met and runbook executed without improvisation

Drill types (increasing realism and risk):

Type	Description	Risk
Tabletop	talk through the scenario on a whiteboard, no systems touched	none; finds gaps in plans and ownership
Walkthrough in a sandbox	execute the runbook against a lab or staging	low
Simulation in production-like environment	inject failure in staging with real traffic replay	low-medium
Game day in production	controlled failure with real users, small blast radius, tight abort rules	medium; needs mature observability
Continuous chaos	automated small faults daily	requires strong guardrails

The five failure classes and what "good" looks like:

Failure	Typical mechanism	What recovery should look like	Measure
Pod	container crash, OOM kill, eviction	Deployment/ReplicaSet recreates it; readiness probe keeps traffic away until ready; other replicas absorb load; no failed requests with 2+ replicas	failed requests, time to ready
Node	VM or machine lost	node marked NotReady, pods evicted after a toleration timeout, scheduled to other nodes; spread constraints keep replicas apart	time to detect node, time to replacement pods ready
Server (VM / app host)	host crash, kernel panic	load balancer health check removes it; autoscaling replaces it	detection window (`fall` x `inter`), failed requests
Database	process crash, disk loss, corruption	restart with crash recovery (seconds), promote a replica (tens of seconds), or restore from backup/PITR (minutes to hours)	outage, rows lost, validation passed (Restore Procedures §15.4)
Region	provider outage, network partition, disaster	traffic moved to the DR region, database promoted there, stateless tier already running (warm) or started (pilot light)	RTO, RPO, manual steps count, quota/access surprises

Rules for safe drills. Announce the window; have a rollback; keep humans out of the injection unless the scenario says so; never run an unannounced production drill; keep a "stop everything" word; make sure monitoring and communications channels work from outside the failure domain; do not test what you cannot undo.

After the drill. Write the report immediately (template in the capstone Capstone 15): timeline, measured values vs targets, what surprised the team, what you will change, owners and dates. Track the actions to completion and schedule the re-test.

6 Step-by-step guide

The lab simulates **two regions on one machine**. Region A runs a PostgreSQL primary and two copies of an app; region B runs an asynchronous streaming standby and one app. HAProxy fronts all three apps and treats region B as the backup site. The app's `/healthz` returns 200 only if its database accepts writes, so region B serves traffic only after its database is promoted: a manual-failover design. A load generator sends five order requests per second through HAProxy; each order is an acknowledged write we can later look for. Everything is free and local. **Prerequisites:** Linux, PostgreSQL 16 server binaries (`/usr/lib/postgresql/16/bin`), `haproxy`, `python3`, `curl`, run as a normal non-root user. Ports 56190, 56191, 18201-18203, 18290 on loopback only; Unix sockets in `/tmp`. Pod and node drills need Docker and `kind` (free) and are in steps 7-8.

1. **Create the lab (database pair, three app instances, load balancer, load generator, analyser).**

```bash
export PATH=/usr/lib/postgresql/16/bin:$PATH
export W=$HOME/dr-lab/region PGHOST=/tmp
rm -rf "$W" && mkdir -p "$W" && cd "$W"          # fresh start; run the teardown (step
  10) first if a lab is still running
cat > app.py <<'PY'
#!/usr/bin/env python3
"""app.py NAME PORT DBSOCKDIR DBPORT - a tiny stand-in for shop-api.
GET /healthz → 200 only if the local database accepts writes (not a standby)
GET /order   → inserts an order, returns its id"""
import subprocess, sys
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
name, port, sock, dbport = sys.argv[1], int(sys.argv[2]), sys.argv[3], sys.argv[4]

def psql(sql):
    r = subprocess.run(["psql", "-X", "-At", "-h", sock, "-p", dbport, "-d", "shop",
  "-c", sql],
                       capture_output=True, text=True, timeout=3)
    if r.returncode ≠ 0:
        raise RuntimeError(r.stderr.strip())
    return r.stdout.strip().split("\n")[0]
```

```
class H(BaseHTTPRequestHandler):
    def reply(self, code, text):
        body = (text + "\n").encode()
        self.send_response(code); self.send_header("Content-Length", str(len(body)));
  self.end_headers()
        self.wfile.write(body)
    def do_GET(self):
        try:
            if self.path == "/healthz":
                ok = psql("SELECT NOT pg_is_in_recovery()") == "t"
                self.reply(200 if ok else 503, "writable" if ok else "standby")
            else:
                self.reply(200, f"{name} order {psql('INSERT INTO orders(item) VALUES
  (' + chr(39) + name + chr(39) + ') RETURNING id')}")
        except Exception as e:
            self.reply(500, f"{name} error: {e}"[:200])
    def log_message(self, *a): pass
class S(ThreadingHTTPServer):
    def handle_error(self, *a): pass
S(("127.0.0.1", port), H).serve_forever()
PY
cat > loadgen.sh <<'SH'
#!/usr/bin/env bash
# loadgen.sh <seconds> <logfile> - one order request every 0.2 s through the load
  balancer
END=$(( $(date +%s) + $1 ))
while [ "$(date +%s)" -lt "$END" ]; do
  R=$(curl -s -m 1 -w ' %{http_code}' localhost:18290/order 2>/dev/null | tr '\n' ' ')
  R=${R:- 000}; R=${R% }
  echo "$(date +%s.%N) ${R##* } ${R% *}" >> "$2"
  sleep 0.2
done
SH
cat > analyze.py <<'PY'
#!/usr/bin/env python3
"""analyze.py <loadgen.log> <fail_epoch> [survivor_ids_file] - measured outage and
  data loss"""
import sys
rows = [(l.split(None, 2) + [""])[:3] for l in open(sys.argv[1]) if l.strip()]
fail = float(sys.argv[2])
good = [(float(t), body) for t, code, body in rows if code == "200"]
bad = [float(t) for t, code, _ in rows if code ≠ "200" and float(t) ⩾ fail - 0.5]
print(f"requests sent={len(rows)} failed={len(bad)}")
if bad:
    last_ok_before = max(t for t, _ in good if t < bad[0])
    first_ok_after = min(t for t, _ in good if t > bad[0])
    served_by = [b.split()[0] for t, b in good if t == first_ok_after][0]
    print(f"outage as seen by users: {first_ok_after - last_ok_before:.1f} s, traffic
  resumed on {served_by}")
else:
    print("outage as seen by users: 0.0 s")
if len(sys.argv) > 3:
```

```
    survivors = {int(x) for x in open(sys.argv[3]).read().split()}
    acked = [int(b.split()[-1]) for t, b in good if b.startswith("app-a")]
    lost = [i for i in acked if i not in survivors]
    print(f"orders acknowledged by region A: {len(acked)}, present after recovery:
  {len(acked) - len(lost)}, LOST: {len(lost)}")
PY
chmod +x loadgen.sh analyze.py
```

2. **Start region A (primary), region B (standby), the apps and HAProxy.**

```bash
initdb -D "$W/region-a" >/dev/null
cat >> "$W/region-a/postgresql.conf" <<CONF
unix_socket_directories = '/tmp'
listen_addresses = ''
wal_keep_size = 128MB
max_wal_senders = 5
CONF
echo "local replication all trust" >> "$W/region-a/pg_hba.conf"
pg_ctl -D "$W/region-a" -o "-p 56190" -l "$W/pg-a.log" -w start >/dev/null
psql -X -q -p 56190 -d postgres -c "CREATE DATABASE shop"
psql -X -q -p 56190 -d shop -c "CREATE TABLE orders(id serial primary key, item text,
  created_at timestamptz default now())"
pg_basebackup -h /tmp -p 56190 -D "$W/region-b" -R -X stream -c fast
pg_ctl -D "$W/region-b" -o "-p 56191" -l "$W/pg-b.log" -w start >/dev/null
nohup python3 "$W/app.py" app-a  18201 /tmp 56190 > "$W/app-a.log"  2>&1 & echo $! >
  "$W/app-a.pid"
nohup python3 "$W/app.py" app-a2 18203 /tmp 56190 > "$W/app-a2.log" 2>&1 & echo $! >
  "$W/app-a2.pid"
nohup python3 "$W/app.py" app-b  18202 /tmp 56191 > "$W/app-b.log"  2>&1 & echo $! >
  "$W/app-b.pid"
cat > "$W/haproxy.cfg" <<CONF
global
    maxconn 256
defaults
    mode http
    timeout connect 1s
    timeout client 5s
    timeout server 5s
frontend fe
    bind 127.0.0.1:18290
    default_backend shop
backend shop
    option httpchk GET /healthz
    default-server inter 1s fall 3 rise 2
    server app-a  127.0.0.1:18201 check
    server app-a2 127.0.0.1:18203 check
    server app-b  127.0.0.1:18202 check backup
CONF
haproxy -f "$W/haproxy.cfg" -D -p "$W/haproxy.pid"
sleep 3
```

```
for i in 1 2 3; do curl -s localhost:18290/order; done
psql -X -At -p 56190 -d postgres -c "SELECT application_name, state FROM
  pg_stat_replication"
curl -s -o /dev/null -w 'region B healthz: %{http_code}\n' localhost:18202/healthz
```

Expected: three lines like `app-a order 1`, `app-a2 order 2`, `app-a order 3` (HAProxy round-robins the two primary-site apps), `walreceiver|streaming`, and `region B healthz: 503` (the standby correctly refuses to take traffic).

3. **Write the drill runner.** One script, three scenarios, one measured result.

BASH

```
cat > "$W/drill.sh" <<'EOF'
#!/usr/bin/env bash
# drill.sh <server|db|region> [decision_delay_seconds] [partition_seconds]
#   server : kill -9 one app instance (automatic recovery by the load balancer)
#   db     : kill -9 the region A database, a human restarts it (crash recovery)
#   region : (optionally cut replication first) kill all of region A, a human promotes
  region B
set -u
export PATH=/usr/lib/postgresql/16/bin:$PATH PGHOST=/tmp
W=$HOME/dr-lab/region; SCEN=${1:?server|db|region}; DECIDE=${2:-10}; PART=${3:-0}
rm -f "$W/loadgen.log"
"$W/loadgen.sh" 40 "$W/loadgen.log" &
sleep 8
case $SCEN in
  server)
    T_FAIL=$(date +%s.%N); echo "$(date +%T) app-a killed"
    kill -9 "$(cat "$W/app-a.pid")"; SURV=""
    ;;
  db)
    T_FAIL=$(date +%s.%N); echo "$(date +%T) region A database killed (kill -9)"
    kill -9 "$(head -1 "$W/region-a/postmaster.pid")"
    sleep "$DECIDE"; echo "$(date +%T) human restarts the database"
    pg_ctl -D "$W/region-a" -o "-p 56190" -l "$W/pg-a.log" -w start >/dev/null
    SURV=56190
    ;;
  region)
    if [ "$PART" -gt 0 ]; then
      echo "$(date +%T) inter-region link breaks (replication stops) ${PART}s before
  the disaster"
      psql -X -q -p 56191 -d postgres -c "ALTER SYSTEM SET primary_conninfo =
  'host=/nonexistent port=1'" -c "SELECT pg_reload_conf()" >/dev/null
      sleep "$PART"
    fi
    T_FAIL=$(date +%s.%N); echo "$(date +%T) REGION A FAILS (database and both app
  instances killed)"
    kill -9 "$(head -1 "$W/region-a/postmaster.pid")" "$(cat "$W/app-a.pid")" "$(cat
  "$W/app-a2.pid")"
    echo "$(date +%T) humans are deciding for ${DECIDE}s ..."; sleep "$DECIDE"
    echo "$(date +%T) PROMOTE region B"; pg_ctl -D "$W/region-b" promote -w >/dev/null
```

```
    SURV=56191
    ;;
esac
wait
if [ -n "$SURV" ]; then
  psql -X -At -p "$SURV" -d shop -c "SELECT id FROM orders WHERE item LIKE 'app-a%'" >
  "$W/survivors.txt"
  python3 "$W/analyze.py" "$W/loadgen.log" "$T_FAIL" "$W/survivors.txt"
else
  python3 "$W/analyze.py" "$W/loadgen.log" "$T_FAIL"
fi
if [ "$SCEN" = server ]; then   # repair so the lab is reusable
  nohup python3 "$W/app.py" app-a 18201 /tmp 56190 > "$W/app-a.log" 2>&1 & echo $! >
  "$W/app-a.pid"
fi
EOF
chmod +x "$W/drill.sh"
```

4. **Drill SERVER-01: an app host dies.** Recovery is automatic, so the only human job is to watch.

```bash
"$W/drill.sh" server
```

   Similar to:

```
22:36:17 app-a killed
requests sent=164 failed=2
outage as seen by users: 1.4 s, traffic resumed on app-a2
```

   Only the requests that happened to be routed to the dead instance before the health check noticed failed. The sibling instance carried on. Record: detection about 3 s (`fall 3` x `inter 1s`), user-visible failures 2 of 164, recovery automatic. Hypothesis "no human needed and under 5 seconds of errors" confirmed.

5. **Drill DB-01: the primary database process dies, the human restarts it.** `kill -9` on the postmaster is an unclean crash; PostgreSQL recovers from its write-ahead log on start.

```bash
"$W/drill.sh" db 8
```

Similar to:

```
22:41:17 region A database killed (kill -9)
22:41:25 human restarts the database
pg_ctl: another server might be running; trying to start server anyway
requests sent=175 failed=44
outage as seen by users: 9.8 s, traffic resumed on app-a2
orders acknowledged by region A: 131, present after recovery: 131, LOST: 0
```

The warning about "another server might be running" is the stale `postmaster.pid` left by the kill and is expected. Outage is approximately the 8 s "human decision" you chose plus start-up and the health check's `rise` time; crash recovery itself took a fraction of a second here. Zero acknowledged orders lost, because committed transactions are flushed to the WAL before they are acknowledged. Re-run with `drill.sh db 30` and see how the decision time dominates; this is why detection and decision belong in your RTO (RTO §15.7).

6. **Drill REGION-01: all of region A is lost; promote region B.** First with healthy replication, then with a replication link that broke 4 seconds before the disaster. You must rebuild the lab (steps 1-2, after the teardown in step 9) between runs because the promoted standby is now a primary.

```bash
"$W/drill.sh" region 12 4
```

Similar to:

```
22:42:01 inter-region link breaks (replication stops) 4s before the disaster
22:42:01 REGION A FAILS (database and both app instances killed)
22:42:01 humans are deciding for 12s ...
22:42:13 PROMOTE region B
requests sent=166 failed=52
outage as seen by users: 14.4 s, traffic resumed on app-b
orders acknowledged by region A: 53, present after recovery: 36, LOST: 17
```

Measured: **RTO about 14 s** (12 s decision + promotion + HAProxy `rise`), **RPO: 17 orders lost**, about 3-4 seconds of traffic at five requests per second, exactly the replication gap. Run `drill.sh region 12 0` on a fresh lab and expect LOST to be 0 or very small. In the real world the 12 seconds would be minutes of paging, assessing and approving, and the partition of seconds would be replication lag across regions (Replication §15.2, Failover §15.5).

7. **Drill POD-01 on Kubernetes (kind).** Requires Docker and `kind`. Free, local. *Output below is illustrative: it was not run in the sandbox that built this guide; run it yourself and record your numbers.*

```bash
cat > kind.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: worker
- role: worker
EOF
kind create cluster --name dr-drill --config kind.yaml
cat > shop-api.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata: { name: shop-api }
spec:
  replicas: 3
  selector: { matchLabels: { app: shop-api } }
  template:
    metadata: { labels: { app: shop-api } }
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: ScheduleAnyway
        labelSelector: { matchLabels: { app: shop-api } }
      tolerations:                    # evict pods quickly from a dead node (default
  is 300 s)
      - { key: node.kubernetes.io/unreachable, operator: Exists, effect: NoExecute,
  tolerationSeconds: 20 }
      - { key: node.kubernetes.io/not-ready,   operator: Exists, effect: NoExecute,
  tolerationSeconds: 20 }
      containers:
      - name: web
        image: nginx:1.27
        ports: [ { containerPort: 80 } ]
        readinessProbe: { httpGet: { path: /, port: 80 }, periodSeconds: 2 }
---
apiVersion: v1
kind: Service
metadata: { name: shop-api }
spec:
  selector: { app: shop-api }
  ports: [ { port: 80, targetPort: 80 } ]
EOF
kubectl apply -f shop-api.yaml && kubectl rollout status deploy/shop-api
kubectl run load --image=curlimages/curl:8.10.1 --restart=Never -- \
  sh -c 'while true; do curl -s -o /dev/null -m 1 -w "%{http_code}\n" http://shop-
  api/; sleep 0.2; done'
kubectl get pods -o wide
```

Now kill a pod and count errors seen by the load generator:

```bash
POD=$(kubectl get pods -l app=shop-api -o name | head -1)
kubectl delete "$POD" --wait=false
kubectl get pods -l app=shop-api -w        # Ctrl+C after the replacement is Running
  and 1/1 Ready
kubectl logs load | sort | uniq -c
```

Illustrative expectation: a replacement pod is Running within a few seconds and Ready after its first readiness probe; the log shows only `200` with 3 replicas. Repeat with `kubectl scale deploy/shop-api --replicas=1` first: you should see a short burst of `000` or `503` while the single pod restarts. The lesson is replicas plus readiness probes, not luck (Pods §8.2, Liveness and Readiness Probes §8.15).

8. **Drill NODE-01 on Kubernetes: a worker node dies.** Use the same cluster. *Illustrative output; verify on your machine.*

```bash
kubectl get pod load -o wide      # note the node the load generator is on; do NOT
  stop that one
kubectl get pods -l app=shop-api -o wide
docker stop dr-drill-worker2      # pick the worker that is NOT hosting "load"; if
  "load" is on worker2 stop dr-drill-worker
date +%T; kubectl get nodes -w    # Ctrl+C once the node shows NotReady
```

Timeline to expect (defaults, rounded): the node controller marks the node `NotReady` after roughly 40-50 seconds without a heartbeat (`node-monitor-grace-period`); the taint `node.kubernetes.io/unreachable:NoExecute` is added; pods tolerating it for 20 s (as configured above) are then evicted and recreated on the surviving worker; without the custom tolerations the default `tolerationSeconds` is 300, so recovery would take about six minutes. During the first minute requests routed to pods on the dead node fail until the endpoint is removed. Count them:

```bash
kubectl logs load | sort | uniq -c
kubectl get pods -l app=shop-api -o wide   # all Running on the surviving worker
docker start dr-drill-worker2              # bring the node back; wait for Ready
```

Record: detection (~45 s), eviction (+20 s), new pods Ready (+ a few seconds), failed requests, and the knobs that move the numbers (probe period, tolerations, replica spread, PodDisruptionBudget). Clean up: `kind delete cluster --name dr-drill`.

9. **Region failure on real cloud: read-only reference.** *Not run here; costs real money; use a sandbox account, a billing alarm and a tag on everything.* On AWS a region drill for `shop-api` typically combines: a pilot-light or warm-standby stack in a second region (for example `us-west-2`), an RDS cross-region read replica or Aurora Global Database, ECR/S3 replication, and Route 53 failover records. The drill is: (1) announce, (2) simulate loss of `us-east-1` by removing it from rotation (set the Route 53 health check to fail, or weight 0) rather than by breaking it, (3) promote the replica in the DR region, (4) scale the application tier, (5) validate (Restore Procedures §15.4), (6) measure, (7) fail back deliberately. Teardown: delete the DR stack, replica and hosted-zone records afterwards; a warm-standby environment left running is a monthly bill. AWS Fault Injection Service can automate steps such as stopping instances, with stop conditions bound to alarms.
10. **Tear down the local lab (and run it before every re-run of step 2).**

```bash
kill "$(cat "$W/haproxy.pid")" "$(cat "$W/app-a.pid")" "$(cat "$W/app-a2.pid")" "$(cat
  "$W/app-b.pid")" 2>/dev/null
pg_ctl -D "$W/region-a" -m immediate stop 2>/dev/null; pg_ctl -D "$W/region-b" -m
  immediate stop 2>/dev/null
```

Remove `~/dr-lab/region` when you are done. If a start fails with "address in use", another process on your machine uses one of the ports; change the numbers in all scripts consistently.

7 Real-world scenario

Illustrative. The `shop-api` team runs its first quarterly game day on staging with production-sized data. The scenario card says: "primary region lost; hypothesis: back in 30 minutes with less than 1 minute of data loss". At 10:00 the facilitator cuts the staging VPC's route to the primary region. The alert fires at 10:03 and the incident commander is paged at 10:05, but the engineer who owns the database is on holiday and the runbook step "get the break-glass credential" requires a laptop with a hardware key that only he holds. A deputy gets a replacement at 10:27. The replica is promoted at 10:31, but the application pods in the DR region have a 4-vCPU quota instead of 40, so only a tenth of the capacity can start; the quota request is approved at 11:04. Traffic is fully served at 11:12. The measured RTO is 72 minutes against a 30-minute hypothesis, and measured data loss is 6 seconds. The debrief produces five actions: a second key holder, quotas set to match production in the DR region (verified by a weekly check), a promote script with guard rails, a runbook update, and a re-test in six weeks. **Lesson:** the technical failover worked; the plan failed on people and limits. Only a drill finds that.

8 Common mistakes and best practices

Don't

- Run an unannounced destructive test in production, or without an abort rule and a rollback.
- Treat a tabletop as sufficient; at least once, execute the runbook for real in a sandbox.
- Let the person who wrote the runbook run the drill; use someone else.
- Skip measurement: "it worked" without timestamps and counts cannot be compared next quarter.
- Punish people for drill findings; make them blameless or nobody will report the next gap.

Do

- Start small: pod, then node, then server, then database, then region; graduate when each is boring.
- Write a scenario card with hypothesis, abort criteria and roles before every drill.
- Measure detect, decide, recover and data lost, and record them in a drill report (capstone templates Capstone 15).
- Include dependencies and people: access, quotas, DNS TTLs, communication channels that live outside the failure domain.
- Track every finding as an action item with an owner and a date, and re-test (Postmortems §14.7).
- Automate repeatable drills (scripts, chaos tools) and keep humans for the ones that test judgement.

9 Tasks

- [ ] **T1 — Build the lab and run the server and database drills** *(≈60 min, intermediate)*
  - **Goal:** produce measured results for two failure classes.
  - **Steps:** follow guide steps 1-5; run `drill.sh db` with decision delays of 5, 15 and 30 seconds.
  - **Done when:** you have a table (delay, outage seconds, failed requests, lost orders) and can explain why the outage grows with the decision delay while lost orders stay zero.
- [ ] **T2 — Region failure with and without replication lag** *(≈60 min, advanced)*
  - **Goal:** separate RTO from RPO in a region failure.
  - **Steps:**

1. Rebuild the lab and run `drill.sh region 5 0`; record outage and lost orders.
2. Rebuild and run `drill.sh region 5 8`; record again.
3. Compute the data loss in seconds from the lost orders divided by the request rate.

- **Done when:** you can show that the outage depends mostly on the decision delay and the health-check timing, while the data loss depends on the replication gap, with your two result sets.

☐ **T3 — Break it: improve the design, re-drill** *(≈90 min, advanced)*

- **Goal:** cut the measured RTO.
- **Steps:**
  1. Change HAProxy to `fall 2 inter 500ms` and the human decision to a script (`promote.sh` from Failover §15.5) that runs automatically 3 seconds after the failure is detected.
  2. Re-run the region drill and compare outage and lost orders with T2.
  3. Write down the new risk you introduced (false positive promotion, split-brain) and one guard rail for it.
- **Done when:** your report shows the before and after outage numbers and names the risk you traded for the speed.

☐ **T4 — Pod and node drills on kind** *(≈90 min, intermediate)*

- **Goal:** measure Kubernetes self-healing.
- **Steps:** follow guide steps 7-8, then repeat the node drill without the custom tolerations and observe how much longer pods take to move.
- **Done when:** you have measured numbers for pod replacement time, node-detection time, eviction time and failed requests, with and without the tolerations, and you have deleted the cluster.

☐ **T5 — Write a scenario card and run a tabletop** *(≈60 min, intermediate)*

- **Goal:** practice the human side.
- **Steps:**
  1. Use the scenario-card fields from section 5 to write a card for "primary region lost" for `shop-api`.
  2. Walk through it with at least one other person using only the runbooks, speaking each step and who does it.
  3. Write down every question that no document answers.
- **Done when:** you have a completed card and a list of at least five gaps with owners.

- [ ] **T6 — Prove it: full drill and report** *(≈120 min, advanced)*
  - **Goal:** run a complete drill without the guide and report on it.
  - **Steps:**
    1. Choose one scenario (database or region) and write the hypothesis, abort criteria and roles.
    2. Run it with a stopwatch and the load generator; capture the timeline.
    3. Produce a one-page drill report using the template in the capstone [Capstone 15], including measured RTO and RPO and at least three action items.
  - **Done when:** the report compares measured values with targets, lists what surprised you, and each action has an owner and a due date.

10 Self-check questions

1. What is the difference between a tabletop, a game day and chaos engineering?
2. Why does the outage in the database drill grow with the decision delay but the data loss does not?
3. In a region failover with asynchronous replication, which quantity determines data loss and which determine downtime?
4. Why set `tolerationSeconds` on pods, and what is the default behavior without it?
5. What must a scenario card contain before you run a drill in production?

ANSWERS

1. A tabletop is a discussion with no systems touched; a game day is a scheduled, facilitated exercise where failure is actually injected and the team responds; chaos engineering is automated, continuous injection of small failures to prove tolerance.
2. Time passes while people decide, and requests fail in the meantime; the data was already safely committed in the write-ahead log, so a restart recovers it. Data loss depends on what had not been durably stored or replicated at the moment of failure.
3. Data loss is determined by the replication gap at the time of failure (RPO). Downtime is determined by detection, decision, promotion, scaling the DR tier and traffic cut-over (RTO).
4. A node that stops reporting gets a NoExecute taint; pods tolerate it for `tolerationSeconds` before eviction. The default added to pods is 300 seconds, so without tuning recovery from a dead node takes about five minutes on top of detection.
5. A hypothesis, blast radius, abort criteria, rollback, roles, the runbook under test, what will be measured, and an announcement window with the facilitator able to stop the drill at any time.

11 Further reading

- *Site Reliability Engineering* and *The Site Reliability Workbook* (Google): chapters on managing incidents, postmortems and disaster-recovery testing.
- *Chaos Engineering: System Resiliency in Practice* (Rosenthal, Jones) and the Principles of Chaos Engineering site.
- AWS Well-Architected Framework, Reliability Pillar (testing resilience, game days) and AWS Fault Injection Service documentation.
- Kubernetes documentation: *Taints and Tolerations*, *Pod Disruption Budgets*, *Node-pressure eviction* and kind (Kubernetes in Docker) quick start.
- Chaos Mesh and LitmusChaos project documentation.
- Related: Failover §15.5, Testing Backup Restoration §15.9, Incident Management §14.5, Reliability Engineering §14.10.

LEVEL 15: DISASTER RECOVERY

★ Break It, Time It, Write the Plan

Level 15 capstone ~16 hours

1 Goal and outcome

Practise five failure scenarios against the `shop-api` stack (Node.js API + PostgreSQL + Redis + Nginx, domain `example.com`) and finish with **measured recovery numbers and a written DR plan**:

1. **Database failure** (process crash, total loss of the primary, and logical corruption needing point-in-time recovery).
2. **Server failure** (an application host dies behind a load balancer).
3. **Pod failure** (a Kubernetes pod is killed).
4. **Node failure** (a Kubernetes worker node is lost).
5. **Region failure** (the primary site disappears; the standby site is promoted).

Outcome: for each scenario a dated drill report with detect/decide/recover times and data loss compared with targets you set up front; an automated, evidence-producing restore test; an immutable off-site backup copy; and a DR plan document a stranger could follow. You are done when the five reports exist, every miss against a target has an action item, and you can explain RPO, RTO, failover, restore order and off-site design without notes.

2 Architecture

Everything runs on one laptop (two simulated regions plus a kind cluster); cloud equivalents

are noted in the stretch goals.

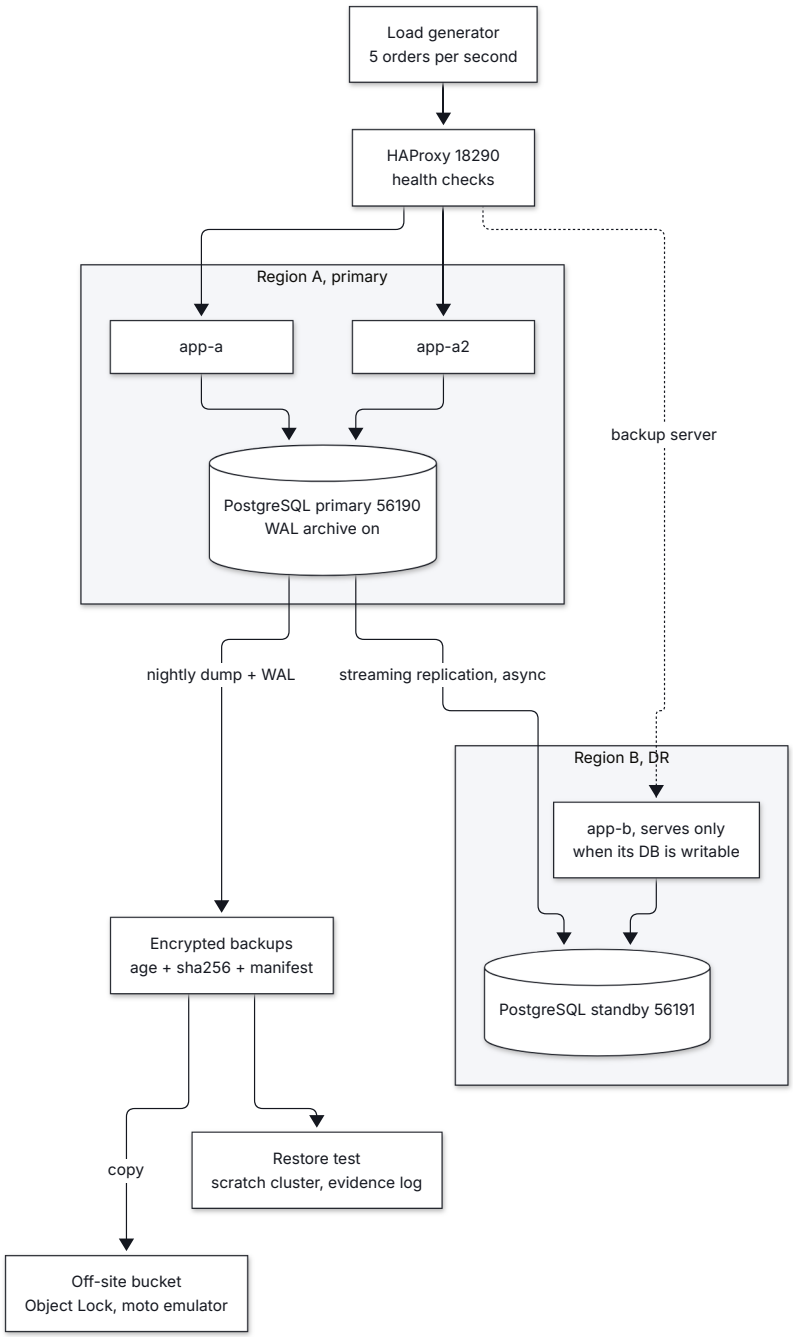


Figure 15-1

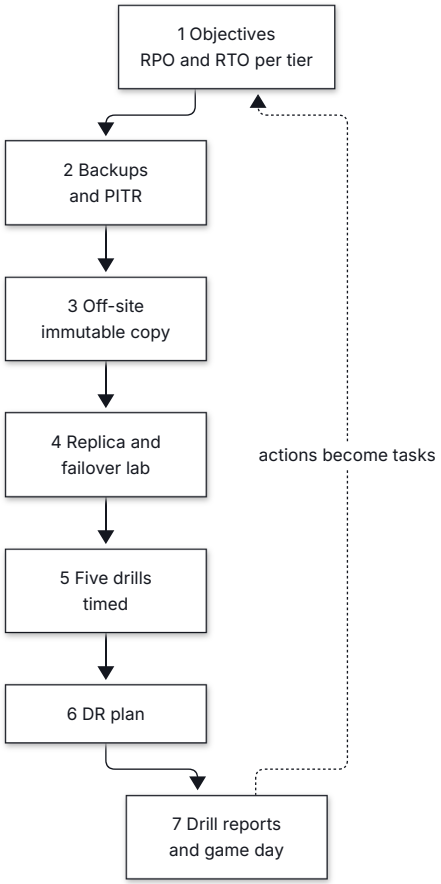


Figure 15-2

3 Prerequisites and cost

- **Cost:** free. Everything uses local processes, loopback ports and a local S3 emulator. The kind pod/node drills need Docker and `kind` (free). Cloud parts are optional stretch goals and **cost real money**: if you do them, set a billing alarm first, tag everything `dr-capstone` and run the teardown in section 7.
- **Machine:** Linux, 8 GB RAM, 10 GB disk; PostgreSQL 16 server binaries (`/usr/lib/postgresql/16/bin`), `haproxy`, `python3`, `curl`, `age`, a Python venv for `moto`; Docker + `kind` + `kubectl` for drills 3 and 4. Use a normal non-root user.
- **Knowledge:** the ten topics of this level: Backups §15.1, Replication §15.2, Snapshots §15.3, Restore Procedures §15.4, Failover §15.5, RPO §15.6, RTO §15.7, Off-site Backups §15.8, Testing Backup Restoration §15.9, Failure Scenario Drills §15.10.

- **Ports used (loopback only):** 56190-56191 (regions), 56200-56201 (PITR), 55480-55481 (restore test), 18201-18203 and 18290 (apps and balancer), 18500 (S3 emulator). Pick other numbers consistently if any clash.
- **Evidence directory:** `mkdir -p ~/dr-lab/capstone/{evidence,reports}`; every number you quote in the DR plan must come from a file here.

4 Build it step by step

Milestone 1 — Objectives first (topics 6 and 7)

Before touching anything, decide targets. Create `~/dr-lab/capstone/objectives.md`:

Tier	Component	RPO	RTO	Strategy	Rationale (revenue per minute, contract)
0	access, secrets, DNS	0 (in Git/vault)	15 min	warm	everything else waits for it
1	orders database + checkout	30 s	15 min	warm standby	revenue
2	Redis cache, search	rebuild	4 h	rebuild	derived data
3	reports	24 h	72 h	backup/restore	internal

Run `python3 rpo_calc.py 30 1200 25` from RPO §15.6 to show the revenue at risk, and write the **scenario targets** you will test against, for example: DB crash RTO 60 s; server RTO 10 s; pod 0 failed requests; node RTO 2 min; region RTO 5 min with RPO 30 s; logical corruption RPO 1 min via PITR.

Milestone 2 — Backups, restore test and PITR (topics 1, 3, 4, 9)

1. Build the encrypted backup + automated restore test exactly as in Testing Backup Restoration §15.9 steps 1-3 (`backup.sh`, `restore-test.sh`, JSON evidence). Run both, then copy `restore-tests.jsonl` into `capstone/evidence/`.
2. Snapshots: a base backup is the local analogue of a volume snapshot (a point-in-time physical copy plus WAL). Take one with `pg_basebackup`, verify it with `pg_verifybackup`, and write one paragraph on the cloud equivalent (EBS snapshot + DLM, RDS snapshot) from Snapshots §15.3.
3. Write the logical-corruption drill (a bad `DELETE`, recovered by PITR). Save as `pitr-drill.sh` and run it:

```bash
cat > ~/dr-lab/capstone/pitr-drill.sh <<'EOF'
#!/usr/bin/env bash
```

```
# pitr-drill.sh - logical corruption: a bad DELETE at a known time, recovered with
  point-in-time recovery
set -eu
export PATH=/usr/lib/postgresql/16/bin:$PATH
export W=$HOME/dr-lab/pitr && rm -rf "$W" && mkdir -p "$W/archive" && export
  PGHOST=/tmp PGPORT=56200
initdb -D "$W/pg" >/dev/null 2>&1
cat >> "$W/pg/postgresql.conf" <<CONF
port = 56200
unix_socket_directories = '/tmp'
listen_addresses = ''
archive_mode = on
archive_command = 'test ! -f $W/archive/%f && cp %p $W/archive/%f'
archive_timeout = 5s
CONF
pg_ctl -D "$W/pg" -l "$W/pg.log" -w start >/dev/null
createdb shop
psql -X -q -d shop -c "CREATE TABLE orders(id serial primary key, item text)" \
  -c "INSERT INTO orders(item) SELECT 'i'||g FROM generate_series(1,50000) g"
pg_basebackup -D "$W/base" -X stream -c fast
psql -X -q -d shop -c "INSERT INTO orders(item) SELECT 'late'||g FROM
  generate_series(1,1000) g"   # work after the backup
sleep 2
GOOD=$(psql -X -At -d shop -c "SELECT clock_timestamp()")
  # the last good moment
ROWS_BEFORE=$(psql -X -At -d shop -c "SELECT count(*) FROM orders")
sleep 1
echo "$(date +%T) THE MISTAKE: DELETE FROM orders"
psql -X -q -d shop -c "DELETE FROM orders"
psql -X -q -d shop -c "SELECT pg_switch_wal()" >/dev/null; sleep 7
  # let the WAL reach the archive
START=$(date +%s)
echo "$(date +%T) restoring base backup + WAL up to: $GOOD"
cp -r "$W/base" "$W/restored"; chmod 700 "$W/restored"; export PGPORT=56201
cat >> "$W/restored/postgresql.conf" <<CONF
port = 56201
unix_socket_directories = '/tmp'
listen_addresses = ''
restore_command = 'cp $W/archive/%f %p'
recovery_target_time = '$GOOD'
recovery_target_action = 'promote'
CONF
touch "$W/restored/recovery.signal"
pg_ctl -D "$W/restored" -l "$W/restored.log" -w start >/dev/null
until [ "$(psql -X -At -d shop -c 'SELECT pg_is_in_recovery()' 2>/dev/null)" = f ]; do
  sleep 0.2; done
echo "PITR took $(( $(date +%s) - START )) s; rows before the mistake: $ROWS_BEFORE;
  rows after PITR: $(psql -X -At -d shop -c 'SELECT count(*) FROM orders')"
pg_ctl -D "$W/restored" -m fast stop >/dev/null; pg_ctl -D "$W/pg" -m immediate stop
  >/dev/null
EOF
chmod +x ~/dr-lab/capstone/pitr-drill.sh && ~/dr-lab/capstone/pitr-drill.sh
```

Similar to: `PITR took 1 s; rows before the mistake: 51000; rows after PITR: 51000`. The RPO for this scenario is the gap between `GOOD` and the mistake (here about 1 second), not the replication lag.

Milestone 3 — Off-site, immutable copy (topic 8)

Follow Off-site Backups §15.8 steps 1-6 with the newest `*.dump.age` from milestone 2: start the emulator, `offsite.py init`, `upload`, `attack`, `recover`. Save the output as `capstone/evidence/offsite.txt`. Add one step to your restore test: after the local restore passes, also download the off-site object, verify its sha256 and decrypt it.

Milestone 4 — Replica and failover lab (topics 2 and 5)

Build the two-region lab from Failure Scenario Drills §15.10 steps 1-3 (ports 56190/56191, HAProxy, load generator, `drill.sh`). Confirm `walreceiver|streaming` and `region B healthz: 503`. Then answer, in writing: which tier of RTO §15.7 is this (warm standby), what decides whether it fails over automatically (HAProxy health checks) and what is manual (the database promote), and what prevents two writable primaries (Failover §15.5: fencing by killing region A, and never auto-promoting).

Milestone 5 — The five drills (topic 10)

Run each drill, keep the terminal output as evidence, and fill the numbers into a `capstone/evidence/results.csv` with the header `scenario,detect_s,decide_s,recover_s,failed_requests,data_lost,target_met`.

ID	Scenario	How	What to record
DB-1	database crash	`~/dr-lab/region/drill.sh db 8`	outage seconds, failed requests, lost orders (expect 0)
DB-2	primary host lost, promote standby	covered by REGION-1 with `drill.sh region 12 0`	RTO, RPO
DB-3	logical corruption	`pitr-drill.sh` from milestone 2	restore seconds, rows recovered, RPO to the mistake
SRV-1	server dies	`~/dr-lab/region/drill.sh server`	detection (about 3 s), failed requests
POD-1	pod killed	topic 10 step 7 on kind	failed requests with 3 replicas and with 1 replica
NODE-1	node lost	topic 10 step 8 on kind	time to NotReady, eviction, pods Ready, failed requests
REGION-1	region lost	`drill.sh region 12 4` on a fresh lab (teardown, then steps 1-2)	outage, lost orders vs the 4 s replication gap

After each drill write one sentence stating whether the target from milestone 1 was met, and what you would change. A helper to keep the CSV honest:

```bash
rec() { echo "$1,$2,$3,$4,$5,$6,$7" >> ~/dr-lab/capstone/evidence/results.csv; }
echo "scenario,detect_s,decide_s,recover_s,failed_requests,data_lost,target_met" > ~/dr-lab/capstone/evidence/results.csv
rec REGION-1 3 12 14 54 17 no          # example row; use your measured values
```

Milestone 6 — The written DR plan

Create `~/dr-lab/capstone/reports/DR-PLAN.md` from this template. Every number must be one you measured; if you did not measure it, write "not tested" instead.

```markdown
# Disaster Recovery Plan — shop-api
Version: 1.0   Date: YYYY-MM-DD   Owner: <name>   Reviewers: <names>   Next review: <date + 6 months>

## 1. Purpose and scope
What this plan covers (services, environments, regions) and what it explicitly does not.

## 2. Objectives
| Tier | Service | RPO target | RPO measured (date) | RTO target | RTO measured (date) | Strategy |
|------|---------|-----------|---------------------|-----------|---------------------|----------|

## 3. Scenarios and responses
| Scenario | Detection | Decision rule | Recovery action | Runbook link | Last drilled |
|----------|-----------|---------------|-----------------|--------------|--------------|
| Pod / node / server / database / logical corruption / region | | | | | |

## 4. Backups and recovery data
| Data | Method | Frequency | Retention | Encrypted | Location 1 | Off-site (account, region, immutable) | Key location | Last restore test |
|------|--------|-----------|-----------|-----------|------------|---------------------------------------|--------------|-------------------|

## 5. Order of restore
Numbered dependency list (access, network, database, cache, apps, traffic, validation) with owners.

## 6. Roles and contacts
Incident commander, communications lead, database owner, deputies, escalation path, vendor support numbers.
Out-of-band communication channel that survives the primary failure.

## 7. Access and break-glass
```

```
Where emergency credentials and encryption keys live, who can retrieve them, how it is
  tested.

## 8. Communication
Internal and customer templates, status page owner, update cadence (for example every 30
  minutes).

## 9. Failback
Criteria and steps to return to normal after the event.

## 10. Testing schedule and evidence
| Test | Frequency | Owner | Evidence location |
|------|-----------|-------|-------------------|
| Automated restore test | daily | | |
| Pod/node/server drill | monthly | | |
| Database and region game day | quarterly | | |

## 11. Known gaps and accepted risks
Each with owner, due date or explicit risk acceptance signed by the business owner.

## 12. Change log
```

Milestone 7 — Drill reports and a game day

For every drill create `reports/DRILL-<id>-<date>.md` from this template, then run one **game day** combining DB-3 and REGION-1 with a second person acting as incident commander using only the DR plan.

MARKDOWN

```
# Drill report DRILL-<ID> — <scenario>
Date/time window: ...   Facilitator: ...   Incident commander: ...   Operators: ...
  Scribe: ...   Environment: lab / staging

## Hypothesis and targets
RPO target ..., RTO target ..., success criteria ...

## Scenario card
Injection ..., blast radius ..., abort criteria ..., runbook under test ...

## Timeline (from the scribe)
| Time | Event | Elapsed |
|------|-------|---------|
| hh:mm:ss | failure injected | 0 |
| | detected / alert | |
| | decision made | |
| | recovery started / finished | |
| | validation passed / traffic restored | |

## Results
| Metric | Target | Measured | Met? |
|--------|--------|----------|------|
```

```
| Detection time | | | |
| Recovery time (RTO) | | | |
| Data loss (RPO) | | | |
| Requests failed | | | |

## What went well / what went badly / what surprised us

## Action items
| # | Action | Owner | Due | Ticket |
|---|--------|-------|-----|--------|

## Evidence
Links to logs, CSV rows, screenshots, terminal output.
Sign-off: <name, date>
```

Finish by turning the action items into tickets, fixing at least one, and re-running that drill to show the improvement.

5 Tasks

- [ ] **T1 — Set targets and the evidence skeleton** *(≈60 min, beginner; milestone 1; topics 6, 7)*
  - **Goal:** objectives and scenario targets written before any drill.
  - **Steps:**
    1. Create the evidence directory, `objectives.md` and the scenario targets list.
    2. Run `rpo_calc.py` for each tier and put the revenue-at-risk figure next to it.
  - **Done when:** `objectives.md` has RPO, RTO, strategy and rationale for four tiers and six scenario targets.
- [ ] **T2 — Automated, evidence-producing restore test plus PITR** *(≈150 min, intermediate; milestone 2; topics 1, 3, 4, 9)*
  - **Goal:** a backup you have restored, timed and can prove.
  - **Steps:** milestone 2, steps 1-3; add the three failure injections from topic 9 (corrupt byte, missing table, stale backup).
  - **Done when:** `restore-tests.jsonl` has passes and three distinct failures with reasons, and `pitr-drill.sh` shows equal row counts before the mistake and after PITR.
- [ ] **T3 — Immutable off-site copy and a ransomware attempt** *(≈90 min, intermediate; milestone 3; topic 8)*
  - **Goal:** prove the off-site copy survives deletion attempts and can be restored.
  - **Steps:** milestone 3; then restore from the recovered object through the restore test.

- **Done when:** `offsite.txt` shows `AccessDenied` on both delete attempts, the recovered file's sha256 matches, and the restore test passes from that copy.

- [ ] **T4 — Build the two-region failover lab and explain it** *(≈90 min, intermediate; milestone 4; topics 2, 5)*
  - **Goal:** a working primary, standby, health-checked balancer and load generator.
  - **Steps:** milestone 4; run `drill.sh server` once as a smoke test.
  - **Done when:** the smoke test reports 1-3 failed requests and a half-page note explains what is automatic, what is manual and how split-brain is prevented.

- [ ] **T5 — Run all five drills and record results** *(≈240 min, advanced; milestone 5; topics 5, 10, 7, 6)*
  - **Goal:** measured detect/decide/recover/loss for database, server, pod, node and region.
  - **Steps:** milestone 5 table; fill `results.csv`; for POD-1 and NODE-1 also run the "worse" variants (1 replica; default tolerations) and record the difference.
  - **Done when:** `results.csv` has all seven rows plus the two variants, each with a `target_met` value, and you can explain every miss.

- [ ] **T6 — Write the DR plan and one drill report per scenario** *(≈180 min, advanced; milestones 6-7; topics 4, 7, 10)*
  - **Goal:** documents a stranger can execute.
  - **Steps:** fill both templates from your evidence; have a colleague follow the plan for DB-3 without help (or leave it a week and do it yourself).
  - **Done when:** five reports and the DR plan exist, the colleague completed DB-3 using only the plan with at most two questions, and every question is now answered in the plan.

- [ ] **T7 — Improve one result and re-test** *(≈120 min, advanced; milestones 5-7; topics 5, 7, 9)*
  - **Goal:** show that drills lead to improvement.
  - **Steps:**
    1. Pick the worst miss in `results.csv`.
    2. Change one thing (for example automate the promote with a guarded script, shorten `fall` / `inter`, or add the second replica) and re-run the drill.
    3. Update the DR plan and the report with before/after numbers and the new risk you introduced.
  - **Done when:** the before/after measurements are in the report and the plan reflects the new design.

- [ ] **T8 — Prove it: game day without notes** *(≈180 min, advanced; all milestones; all topics)*

- **Goal:** demonstrate mastery under time pressure.
- **Steps:**
  1. Have a second person pick a scenario you have not told them in advance (database loss, corruption or region loss) and inject it into a fresh lab.
  2. Recover using only the DR plan and your scripts, narrating each step and why.
  3. Hold a 20-minute blameless debrief and write the report.
- **Done when:** recovery meets or misses the targets with measured evidence, you can explain the RPO/RTO tier, the failover decision, the restore order and where the off-site copy lives, and the report lists at least three action items.

6 Verification checklist

- [ ] `objectives.md` states RPO and RTO per tier with a business rationale.
- [ ] `restore-tests.jsonl` contains at least one pass and failures for corruption, missing data and staleness.
- [ ] `pitr-drill.sh` output shows row counts equal before the mistake and after recovery.
- [ ] The off-site bucket refused both delete attempts and a version was recovered by checksum.
- [ ] The standby shows `streaming` before each region drill and HAProxy marks region B only after promotion.
- [ ] `results.csv` has database, server, pod, node and region rows, each with measured numbers.
- [ ] Every region drill record separates RTO (seconds) from RPO (lost orders) and links them to the replication gap and decision time.
- [ ] `DR-PLAN.md` has no number that is not backed by an evidence file; untested items say "not tested".
- [ ] Five drill reports exist with timelines, targets vs measured, and action items with owners.
- [ ] You re-ran at least one drill after a fix and the measured numbers improved (or you explained why not).
- [ ] No lab process is still running and no secret or key from the lab is committed anywhere.

7 Teardown

```bash
# local region lab and any app/haproxy processes
W=$HOME/dr-lab/region
kill "$(cat "$W/haproxy.pid")" "$(cat "$W/app-a.pid")" "$(cat "$W/app-a2.pid")" "$(cat
  "$W/app-b.pid")" 2>/dev/null
for d in region-a region-b; do /usr/lib/postgresql/16/bin/pg_ctl -D "$W/$d" -m immediate
  stop 2>/dev/null; done
# restore-test and PITR labs
/usr/lib/postgresql/16/bin/pg_ctl -D ~/dr-lab/restore-test/prod -m immediate stop
  2>/dev/null
# S3 emulator
kill "$(cat ~/dr-lab/offsite/moto.pid)" 2>/dev/null
# kind cluster
kind delete cluster --name dr-drill 2>/dev/null
# keep evidence and reports; delete the heavy lab data
rm -rf ~/dr-lab/region ~/dr-lab/pitr ~/dr-lab/restore-test ~/dr-lab/offsite ~/dr-lab/rpo
  ~/dr-lab/rto
```

If you ran any cloud stretch goals: delete DR stacks, replicas, snapshots, hosted-zone records and health checks, empty and delete buckets (Object Lock in compliance mode cannot be removed before its retention date, so use 1 day when testing), and check the billing console the next day.

8 Stretch goals and what to learn next

- Repeat the region drill on real infrastructure with a pilot-light stack in a second AWS region (Terraform for the DR region, RDS cross-region replica or Aurora Global Database, Route 53 failover records), and compare the measured RTO with the local one. See RDS and Aurora §6.10 and Route 53 §6.12. Set a billing alarm and tear down afterwards.
- Replace the hand-run promote with Patroni (etcd leader lease) or a managed failover and measure the false-positive risk with network-partition tests.
- Add Velero or CSI snapshots for the Kubernetes volumes and drill a persistent-volume restore (Volumes, PV and PVC §8.9).
- Run chaos experiments continuously (Chaos Mesh, LitmusChaos, AWS Fault Injection Service) and alert on drill staleness.
- Feed the results into SLOs and error budgets: how much downtime does one regional failover consume (Error Budgets §14.4)?
- Study the architectural side in Disaster Recovery Architecture §19.9 and the cost side in Storage Lifecycle Policies §20.6.

- Next level: Level 16 — Advanced Kubernetes §16.1, where stateful workloads and operators bring the backup and failover patterns of this level into the cluster (Stateful Workloads §16.8).

PART VIII

Platforms at Scale

Advanced Kubernetes, GitOps, and the craft of building an internal platform that other engineers want to use.



IN THIS PART

16	Advanced Kubernetes	3137
17	GitOps	3303
18	Platform Engineering	3403

LAB NOTES What I ran, what I saw, what it means

LEVEL 16 · PART VIII

Advanced Kubernetes 16

Goal: Secure, extend and scale Kubernetes for multi-team production use.

RECOMMENDED TIME	TOPICS
4-6 weeks	10

PRACTICE / CAPSTONE

Harden a kind/EKS cluster: Pod Security Standards, default-deny NetworkPolicies, an admission policy, autoscaling, and a mesh with mTLS and canary routing.

Topics

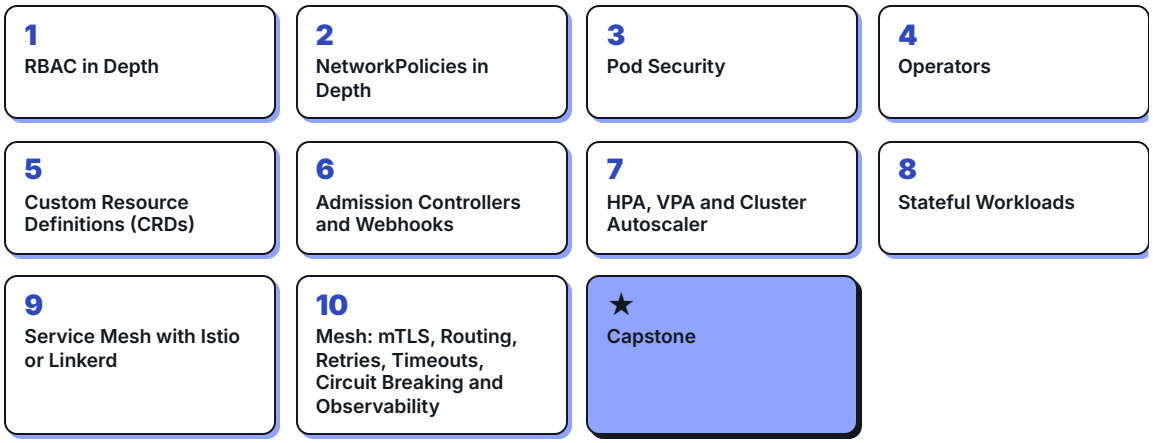
#	Topic	What you will learn
1	RBAC in Depth **p.3139**	least-privilege roles, aggregated roles, audit, impersonation, common escalations
2	NetworkPolicies in Depth **p.3153**	default-deny patterns, egress control, Cilium/Calico, policy testing
3	Pod Security **p.3167**	Pod Security Standards/Admission, securityContext, seccomp, AppArmor
4	Operators **p.3179**	controller pattern, reconcile loops, real operators (Prometheus, Postgres), Operator SDK/kubebuilder
5	Custom Resource Definitions (CRDs) **p.3191**	defining CRDs, schemas, versions, writing a custom resource
6	Admission Controllers and Webhooks **p.3205**	mutating/validating, ValidatingAdmissionPolicy, Kyverno/OPA Gatekeeper
7	HPA, VPA and Cluster Autoscaler **p.3221**	pod vs node scaling, Karpenter, interactions and pitfalls
8	Stateful Workloads **p.3237**	running databases on K8s, operators, storage, backup, when not to
9	Service Mesh with Istio or Linkerd **p.3251**	architecture, sidecar injection, install and onboard an app

#	Topic	What you will learn
10	Mesh: mTLS, Routing, Retries, Timeouts, Circuit Breaking and Observability **p.3265**	VirtualService/DestinationRule or Linkerd equivalents, traffic shifting, golden metrics from the mesh
★	Level capstone project **p.3283**	Harden a kind/EKS cluster: Pod Security Standards, default-deny NetworkPolicies, an admission policy, autoscaling, and a mesh with mTLS and canary routing.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



16.1 RBAC in Depth

Topic 1 of 10 | ~5 hours | Difficulty: Advanced

1 What it is

You already know the four RBAC objects (Role, ClusterRole, RoleBinding, ClusterRoleBinding) from RBAC §8.18. This topic is about what happens when a cluster is shared by many teams, many CI pipelines and many controllers: how to design roles that stay **least-privilege**, how to compose them with **aggregated ClusterRoles**, how to **audit** who did what, how **impersonation** lets you test and delegate safely, and which permission combinations are **privilege escalations in disguise**.

The key mental model: RBAC is **additive and deny-by-default**, and the API server authorizes the *verb* + *resource* + *subresource* + *namespace* + *(optionally) resource name* of every request. Because there are no deny rules, the only way to be safe is to never grant a permission that quietly implies a bigger one.

2 Why it matters in DevOps

- **Most real cluster takeovers are RBAC over-grants,** not kernel exploits: a pipeline ServiceAccount that can `create pods` can run a pod that mounts the node's filesystem, and then it effectively owns the node.
- `list secrets` **is** `get secrets`**.** A role that can *list* Secrets in a namespace receives every Secret value in the response. Auditors find this constantly.
- **Platform teams need composable roles.** Writing one giant ClusterRole per team does not scale; aggregation lets each CRD (see Custom Resource Definitions §16.5) add its own permissions to the standard `view` / `edit` / `admin` roles.
- **You cannot answer "who did that?" without audit logs.** Without an audit policy, a deleted Deployment has no author.
- **Compliance:** SOC 2 and ISO 27001 reviews ask "who can read production Secrets?" and expect a query, not a guess.

3 Real-world system example

Kubernetes' own built-in roles. The default `view`, `edit` and `admin` ClusterRoles are not hand-maintained lists. They are **aggregated ClusterRoles**: each has an `aggregationRule` that selects other ClusterRoles carrying labels such as `rbac.authorization.k8s.io/aggregate-to-view: "true"`. When you install a CRD-based product (for example the Prometheus Operator or cert-manager), its manifests typically include small ClusterRoles with those aggregation labels so that anyone who already has `edit` in a namespace can also manage that product's custom resources, with no extra bindings. The controller-manager fills in the combined rules automatically.

Managed clouds map their own identity systems onto RBAC groups: Amazon EKS maps IAM principals to Kubernetes users/groups through access entries (the older `aws-auth` ConfigMap is being superseded), GKE combines IAM and RBAC, and AKS binds Microsoft Entra ID groups. The Kubernetes project's RBAC good-practices documentation lists the same escalation paths covered below (wildcards, `escalate`, `bind`, `impersonate`, access to Secrets, `pods/exec`, and `create pods`).

4 Diagram

How a request is judged, and where audit fits:

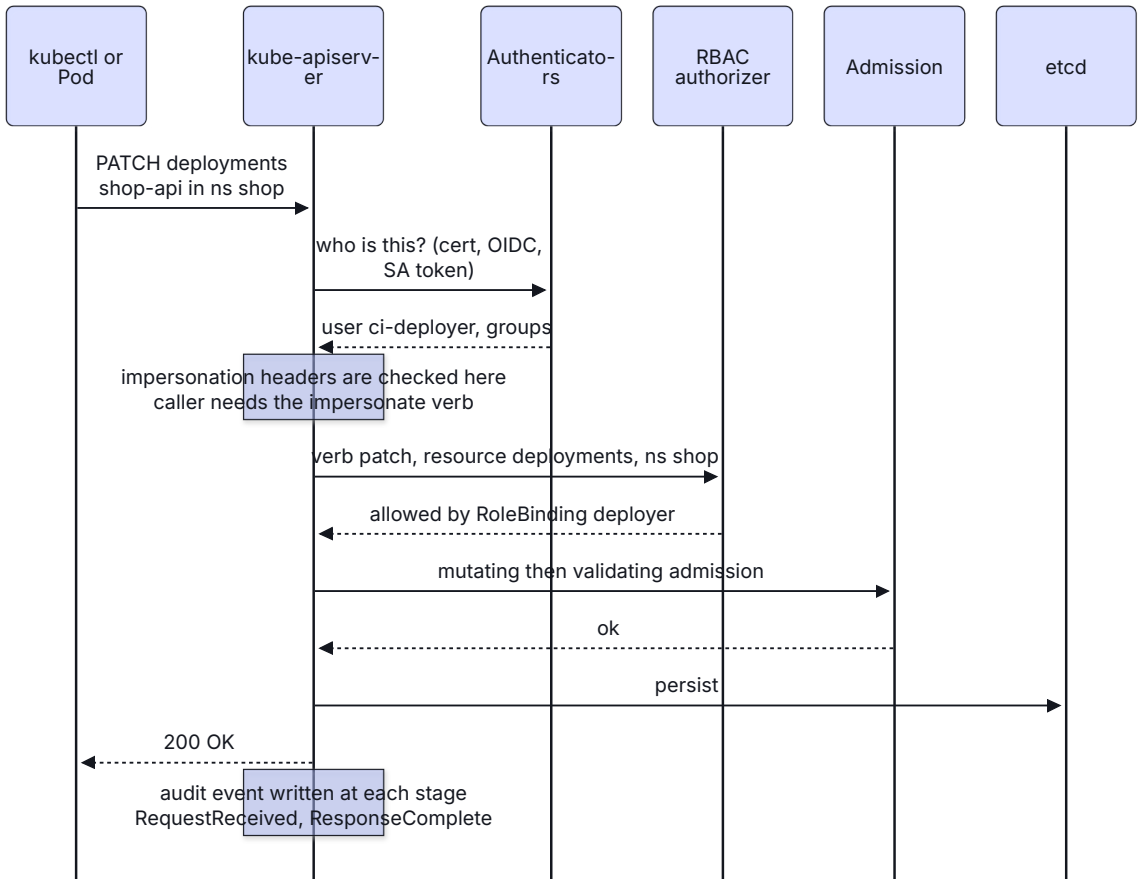


Figure 16.1-1

How aggregation composes roles:

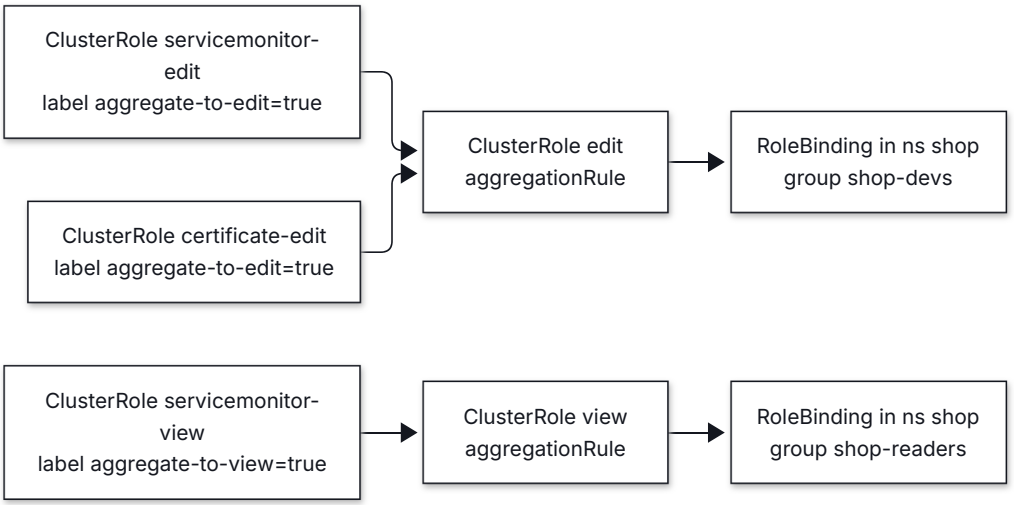


Figure 16.1-2

5 How it works

Designing least-privilege roles

Dimension	Use it to narrow	Example
`verbs`	Only what the task needs	`get`, `list`, `watch`, `patch` — not `*`
`resources`	Exact kinds	`deployments`, not `*`
`resourceNames`	Specific objects	`resourceNames: ["shop-api"]` for `patch` (works for `get`, `update`, `patch`, `delete`; **not** for `list`, `watch` or `create`)
Namespace	Role + RoleBinding, not ClusterRole + ClusterRoleBinding	the pipeline for `shop` gets a Role in `shop` only
Subresources	`pods/log` and `pods/exec` are separate	grant `pods/log`, withhold `pods/exec`

Prefer **groups** (from OIDC or cloud IAM) over individual users in bindings, and bind **one reusable ClusterRole through many namespaced RoleBindings** instead of cloning Roles.

Aggregated roles

A ClusterRole with `aggregationRule.clusterRoleSelectors` has its `rules` **overwritten** by the controller with the union of all matching ClusterRoles. Do not hand-edit `rules` on an aggregated role; create a small labelled ClusterRole instead.

Impersonation

A caller with the `impersonate` verb on `users`, `groups`, `serviceaccounts` (or `userextras`) can act as someone else: `kubectl --as=jane --as-group=shop-devs get pods`. It is invaluable for **testing** a role ("what would this ServiceAccount see?") and for support tooling, and dangerous if granted broadly: `impersonate` on `groups` lets you become `system:masters` and so cluster-admin. Impersonated calls appear in audit logs with both the real and impersonated identity.

Common escalations (permissions that imply more)

Permission	Why it is dangerous
`create pods` (or any workload controller) in a namespace	Run a pod with any ServiceAccount in that namespace, mount Secrets, or (without Pod Security §16.3) mount `hostPath` and take the node
`get/list/watch secrets`	Reads every Secret, including ServiceAccount tokens in older clusters and cloud credentials
`create serviceaccounts/token`	Mints tokens for any ServiceAccount in the namespace
`escalate` on `roles` / `clusterroles`	Lets you write a role with permissions you do not hold
`bind` on `roles` / `clusterroles`	Lets you create a binding to a role you do not hold (including `cluster-admin`)
`impersonate`	Become any user or group
`pods/exec`, `pods/attach`, `pods/portforward`	Shell inside workloads that hold credentials
`nodes/proxy`	Reaches the kubelet API, often a path to exec in any pod on the node
`create/patch` on admission webhook configurations	Intercept or break every API request (Admission Controllers §16.6)
Wildcards `*` in verbs, resources or API groups	Silently includes future resources and CRDs

By default the API server **blocks privilege escalation through RBAC objects**: you can only create a role or binding if you already hold every permission it contains, unless you have `escalate` / `bind`. That protection is only as strong as who holds those two verbs.

Audit

The API server can write an audit log following an **audit policy** file (levels `None`, `Metadata`, `Request`, `RequestResponse`). A good baseline: `Metadata` for Secrets and ConfigMaps (never log their bodies), `RequestResponse` for RBAC and webhook objects, `None` for noisy read-only system traffic. On managed clusters you enable audit logs through the provider (for example EKS control-plane logging to CloudWatch Logs), and you do not edit the policy yourself.

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`, `jq`. Cost: none (local). Commands were written against a recent kind/kubectl; outputs below are illustrative ("similar to").

1. **Create a cluster with auditing enabled.** Save the policy first:

```bash
mkdir -p ~/l16 && cd ~/l16
cat > audit-policy.yaml <<'EOF'
apiVersion: audit.k8s.io/v1
kind: Policy
omitStages: ["RequestReceived"]
rules:
  - level: None
    users: ["system:kube-proxy"]
  - level: Metadata
    resources:
      - group: ""
        resources: ["secrets", "configmaps"]
  - level: RequestResponse
    resources:
      - group: "rbac.authorization.k8s.io"
      - group: "admissionregistration.k8s.io"
  - level: Metadata
EOF
cat > kind-audit.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
kubeadmConfigPatches:
  - |
    kind: ClusterConfiguration
    apiServer:
      extraArgs:
        audit-log-path: /var/log/kubernetes/audit.log
        audit-policy-file: /etc/kubernetes/policies/audit-policy.yaml
      extraVolumes:
        - name: audit-policies
          hostPath: /etc/kubernetes/policies
          mountPath: /etc/kubernetes/policies
          readOnly: true
```

```
          pathType: DirectoryOrCreate
        - name: audit-logs
          hostPath: /var/log/kubernetes
          mountPath: /var/log/kubernetes
          readOnly: false
          pathType: DirectoryOrCreate
nodes:
  - role: control-plane
    extraMounts:
      - hostPath: ./audit-policy.yaml
        containerPath: /etc/kubernetes/policies/audit-policy.yaml
        readOnly: true
EOF
kind create cluster --name rbac-lab --config kind-audit.yaml
```

Expect `Set kubectl context to "kind-rbac-lab"`. *Note:* newer kubeadm config versions express `extraArgs` as a list of `name`/`value` pairs; if kind reports a patch/schema error, consult the kind "Auditing" docs for the form matching your kind version.

2. **Create a namespace, a ServiceAccount and a least-privilege Role for a pipeline.**

```bash
kubectl create namespace shop
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ci-deployer
  namespace: shop
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: shop-api-deployer
  namespace: shop
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["apps"]
    resources: ["deployments"]
    resourceNames: ["shop-api"]
    verbs: ["update", "patch"]
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ci-deployer-binding
```

```
  namespace: shop
subjects:
  - kind: ServiceAccount
    name: ci-deployer
    namespace: shop
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: shop-api-deployer
EOF
```

3. **Test the role with `can-i` and impersonation.**

```bash
SA=system:serviceaccount:shop:ci-deployer
kubectl auth can-i patch deployments/shop-api -n shop --as=$SA     # yes
kubectl auth can-i patch deployments/other    -n shop --as=$SA     # no
kubectl auth can-i list secrets               -n shop --as=$SA     # no
kubectl auth can-i create pods                -n shop --as=$SA     # no
kubectl auth can-i --list -n shop --as=$SA
```

   Note that `resourceNames` limits `patch` to `shop-api`. `--list` prints a table of every verb the identity has in that namespace. Your own user (cluster-admin in kind) can impersonate; a normal user cannot unless granted `impersonate`.

4. **Build an aggregated role.** Create a role that aggregates everything labelled `shop-reader`, plus teach the built-in `view` about a custom resource:

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: shop-reader
aggregationRule:
  clusterRoleSelectors:
    - matchLabels:
        shop.example.com/aggregate-to-shop-reader: "true"
rules: []
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: shop-reader-core
  labels:
    shop.example.com/aggregate-to-shop-reader: "true"
rules:
  - apiGroups: ["", "apps"]
    resources: ["pods", "services", "deployments"]
```

```
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: shop-reader-extra
  labels:
    shop.example.com/aggregate-to-shop-reader: "true"
rules:
  - apiGroups: ["autoscaling"]
    resources: ["horizontalpodautoscalers"]
    verbs: ["get", "list", "watch"]
EOF
kubectl get clusterrole shop-reader -o jsonpath='{.rules[*].resources}'
```

Expect the union of both (`pods services deployments horizontalpodautoscalers` or similar). Add a third labelled ClusterRole and re-run: the rules update within seconds.

5. **Demonstrate the escalation guard, then the `escalate` bypass.** Let a user manage RoleBindings but not hold what they grant:

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: binding-admin, namespace: shop}
rules:
  - apiGroups: ["rbac.authorization.k8s.io"]
    resources: ["rolebindings"]
    verbs: ["create", "get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: binding-admin, namespace: shop}
subjects: [{kind: User, name: mallory, apiGroup: rbac.authorization.k8s.io}]
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: Role, name: binding-admin}
EOF
kubectl create rolebinding grab-admin --clusterrole=admin --user=mallory -n shop --
  as=mallory
```

Similar to: `forbidden: user "mallory" (groups=...) is attempting to grant RBAC permissions not currently held`. Mallory has `create rolebindings`, but the API server refuses because she lacks the `admin` permissions. Now add `{verbs: ["bind"], resources: ["clusterroles"], resourceNames: ["admin"], apiGroups: ["rbac.authorization.k8s.io"]}` to her role and the same command succeeds: `bind` is a deliberate escape hatch, so grant it rarely and always with `resourceNames`.

6. **Read the audit log.** Make some changes, then inspect the log inside the control-plane node:

```bash
kubectl delete rolebinding grab-admin -n shop
docker exec rbac-lab-control-plane sh -c 'tail -n 200 /var/log/kubernetes/audit.log' \
  | jq -c 'select(.objectRef.resource=="rolebindings") | {verb, user: .user.username,
  ns: .objectRef.namespace, name: .objectRef.name, code: .responseStatus.code}'
```

   You should see `create` / `delete` events on `rolebindings` with the acting user, including the `403` for Mallory's first attempt.

7. **Hunt for dangerous grants in a live cluster.**

```bash
# who is bound to cluster-admin?
kubectl get clusterrolebindings -o json | jq -r '.items[] |
  select(.roleRef.name=="cluster-admin") | .metadata.name + " → " + ([.subjects[]? |
  .kind + "/" + .name] | join(","))'
# which roles contain a wildcard verb or resource?
kubectl get clusterroles,roles -A -o json | jq -r '.items[] | select(any(.rules[]?;
  (.verbs|index("*")) or (.resources|index("*")))) | .kind + " " +
  (.metadata.namespace // "-") + "/" + .metadata.name'
```

   In a fresh kind cluster expect only built-in `system:` roles and the `cluster-admin` binding for `system:masters` and kubeadm admins. Dedicated tools such as `kubectl-who-can`, `rbac-tool` and `rakkess` automate these queries.

8. **Clean up:** `kind delete cluster --name rbac-lab`.

7 Real-world scenario

(Illustrative, not a documented incident.) A `shop-api` team's CI pipeline used a ServiceAccount with a ClusterRole that granted `get, list, create, patch` on `*` resources in the `apps` and core API groups "so deploys never fail". A leaked CI token turned up in a public build log. The security team ran `kubectl auth can-i --list --as=system:serviceaccount:ci:deployer -n kube-system` and found the token could `list secrets` cluster-wide. They rotated the ServiceAccount token and replaced the ClusterRole with a namespaced Role limited to `deployments` (with `resourceNames`) in `shop`, added an audit rule recording RBAC changes at `RequestResponse`, and put a weekly job on the wildcard-hunting query from step 7. **Lesson:** convenience wildcards turn one leaked token into a cluster-wide incident; least privilege is cheap to apply on day one and expensive to retrofit.

8 Common mistakes and best practices

Don't

- Bind humans or pipelines to `cluster-admin` "temporarily". Temporary grants become permanent.
- Grant `list` or `watch` on Secrets to anything that does not need the values.
- Use `*` for verbs, resources or API groups.
- Hand out `escalate`, `bind` or `impersonate` without `resourceNames` limits and a documented reason.
- Edit the `rules` of an aggregated ClusterRole directly; the controller will overwrite your change.
- Rely on the default ServiceAccount in a namespace; pods get it automatically.

Do

- Start from zero and add verbs until the task works; verify with `kubectl auth can-i --as=...`.
- Use groups from your identity provider, and review bindings on a schedule.
- Disable token auto-mount (`automountServiceAccountToken: false`) on pods that never call the API.
- Turn on audit logging (control-plane logs on managed clusters) and alert on RBAC object changes and `cluster-admin` use.
- Keep roles in Git and review them as code (see Policy as Code §18.7).

9 Tasks

- [ ] **T1 — Least-privilege pipeline role** *(≈30 min, beginner)*
  - **Goal:** give a ServiceAccount exactly the access a deploy needs.
  - **Steps:**
    1. Follow guide steps 1-3 to create `ci-deployer` and the `shop-api-deployer` Role.
    2. Run the four `can-i` checks and record yes/no for each.
  - **Done when:** `patch deployments/shop-api` is `yes`, while `patch deployments/other`, `list secrets` and `create pods` are all `no`.
- [ ] **T2 — Aggregate a role** *(≈30 min, intermediate)*
  - **Goal:** compose a role from labelled pieces.
  - **Steps:**

1. Create `shop-reader` and the two labelled ClusterRoles from step 4.
2. Add a third labelled ClusterRole granting `get, list, watch` on `networkpolicies` (group `networking.k8s.io`).
3. Bind `shop-reader` to a group in `shop` with a RoleBinding and test with `--as=anna --as-group=shop-readers`.

- **Done when:** `kubectl get clusterrole shop-reader -o yaml` shows networkpolicies in its rules and `kubectl auth can-i list networkpolicies -n shop --as=anna --as-group=shop-readers` prints `yes`.

- [ ] **T3 — Safe impersonation for support** *(≈30 min, intermediate)*
  - **Goal:** let a support identity impersonate only one ServiceAccount.
  - **Steps:**
    1. Create ServiceAccount `support-bot` in `shop`, plus a ClusterRole with verb `impersonate` on `serviceaccounts` with `resourceNames: ["ci-deployer"]`, bound to `support-bot` with a ClusterRoleBinding.
    2. Mint a token: `TOKEN=$(kubectl create token support-bot -n shop)`.
    3. Run `kubectl --token=$TOKEN get pods -n shop --as=system:serviceaccount:shop:ci-deployer` (should work), then the same with `--as=system:serviceaccount:shop:default` (should fail). Note that `impersonate` on serviceaccounts also needs the matching namespace to be allowed, which the ClusterRole grants cluster-wide for that one name.
    4. Find the impersonated call in the audit log (`.impersonatedUser`).
  - **Done when:** impersonating `ci-deployer` succeeds, impersonating any other ServiceAccount returns `forbidden`, and the audit event shows both `user.username` and `impersonatedUser.username`.

- [ ] **T4 — Break it: trigger and bypass the escalation guard** *(≈40 min, advanced)*
  - **Goal:** see exactly what `bind` and `escalate` change.
  - **Steps:**
    1. Reproduce step 5 and capture the `403` message.
    2. Add `bind` scoped to `resourceNames: ["admin"]`, retry and succeed.
    3. Remove the `bind` rule and confirm the RoleBinding can no longer be created.
  - **Done when:** you have the failing and succeeding commands plus an audit-log excerpt showing both, and you can explain why `bind` is safer with `resourceNames`.

- [ ] **T5 — Automate an RBAC audit** *(≈45 min, advanced)*
  - **Goal:** a script that flags risky grants.

- **Steps:**
  1. Write a shell or Python script around the step 7 queries that also flags any role with `secrets` + `list`, `escalate`, `bind`, `impersonate`, or `pods/exec`.
  2. Run it against your kind cluster after creating one deliberately bad Role.
  3. Make it exit non-zero when findings exist so CI can run it.
- **Done when:** the script exits `1` and names your bad Role, and exits `0` after you delete it.

☐ **T6 — Prove it: explain the takeover path** *(≈45 min, advanced)*

- **Goal:** show a namespace-level permission becoming node-level control, and how to prevent it.
- **Steps:**
  1. Without the guide, give a ServiceAccount only `create pods` in a namespace and write down (not necessarily run) the pod spec an attacker would use.
  2. Write the three controls that stop it: RBAC narrowing, Pod Security (`restricted`), and admission policy.
  3. Explain it to a teammate or in a one-page note.
- **Done when:** your note names the permission, the abused feature (`hostPath`, privileged, other SA), and one concrete mitigation from each of the three layers.

10 Self-check questions

1. Why is granting `list` on Secrets as dangerous as granting `get`?
2. What do `escalate` and `bind` do, and why does RBAC refuse to let you create a role you could not already exercise?
3. Which RBAC verb on which resource lets a user become `system:masters`?
4. How does an aggregated ClusterRole get its rules, and why should you not edit them?
5. Why can `resourceNames` not restrict `list` or `create`?
6. What audit level should Secrets use, and why?

ANSWERS

1. A list response includes the full objects, so every Secret value is returned.
2. They bypass the escalation check: `escalate` lets you write roles containing permissions you lack; `bind` lets you bind a role you lack. The default check stops a low-privileged user from granting themselves more.
3. `impersonate` on `groups` (or `users`), which lets the caller act as any group, including `system:masters`.

4. A controller replaces `rules` with the union of ClusterRoles matched by `clusterRoleSelectors`; manual edits are overwritten at the next sync.
5. At `list` / `create` time the request does not target a specific existing object name (a list covers many objects; the create has no name in the URL). Use `resourceNames` with `get`, `update`, `patch`, `delete`.
6. `Metadata`, so the log shows who touched which Secret without storing the secret value in the log.

11 Further reading

- Kubernetes documentation: *Using RBAC Authorization* and *Role Based Access Control Good Practices* (kubernetes.io/docs).
- Kubernetes documentation: *Auditing* and *User Impersonation*.
- *Kubernetes Security and Observability* by Brendan Creane and Amit Gupta (O'Reilly).
- Tools: `kubectl-who-can` (Aqua Security), `rbac-tool` (Insight Engineering), `rakkess`.
- Related topics: Pod Security §16.3, Admission Controllers and Webhooks §16.6, Least-privilege IAM §11.2.

16.2 NetworkPolicies in Depth

Topic 2 of 10 | ~5 hours | Difficulty: Advanced

1 What it is

By default every Pod in a Kubernetes cluster can open a connection to every other Pod, in every namespace, and out to the internet. A **NetworkPolicy** is a namespaced object that selects a set of Pods and declares which **ingress** (incoming) and **egress** (outgoing) traffic is allowed for them. The moment *any* policy selects a Pod for a direction, that direction flips from "allow all" to "deny everything except what some policy allows". Policies are **additive**: there are no deny rules, only allows, and the union of all policies selecting a Pod is what is permitted.

Crucially, a NetworkPolicy is only an API object. **Enforcement is done by the CNI plugin** (Container Network Interface, the networking plugin). If your CNI does not implement NetworkPolicy, the objects are accepted and silently do nothing. This topic covers the **default-deny patterns**, **egress control** (including DNS and the internet), the **richer policy languages of Cilium and Calico**, and how to **test** policies instead of trusting them. You met the basics in NetworkPolicies §8.19.

2 Why it matters in DevOps

- **Lateral movement.** In a flat network, one compromised front-end Pod can reach the database, the metadata service and other teams' workloads. Default-deny shrinks the blast radius.
- **Compliance.** PCI DSS-style segmentation requirements are usually answered with namespace-level default-deny plus explicit allows.
- **Silent failure.** A policy on a CNI that does not enforce it gives *false confidence*; an over-tight policy that forgets DNS gives 3 a.m. "everything times out" incidents.
- **Egress is the exfiltration path.** Most teams only restrict ingress; attackers and malware need egress to call home. See Cloud Security Groups and Policies §11.3 for the same layered idea at the VPC level.

3 Real-world system example

Cilium and Calico, the two most widely deployed policy-capable CNIs. Both are open source CNCF-hosted or CNCF-adjacent projects used on managed and self-managed clusters (Amazon EKS documents installing Calico or Cilium for network policy; GKE's "Dataplane V2" is built on Cilium and enforces NetworkPolicy; AKS offers Azure Network Policy, Calico and Cilium options). Cilium uses eBPF (extended Berkeley Packet Filter, in-kernel programmable hooks) to enforce policy by *workload identity* derived from labels, adds its own `CiliumNetworkPolicy` with L7 (HTTP) rules and DNS-name (FQDN) egress rules, and ships **Hubble** for observing allowed and dropped flows. Calico enforces with iptables or eBPF and adds `GlobalNetworkPolicy` for cluster-wide rules and ordered, deny-capable policies. The Kubernetes SIG Network is also developing cluster-scoped `AdminNetworkPolicy` and `BaselineAdminNetworkPolicy` APIs (still maturing, so check your cluster's version before relying on them) for platform teams to set rules that namespace owners cannot override.

4 Diagram

How a policy decision is made and where it is enforced:

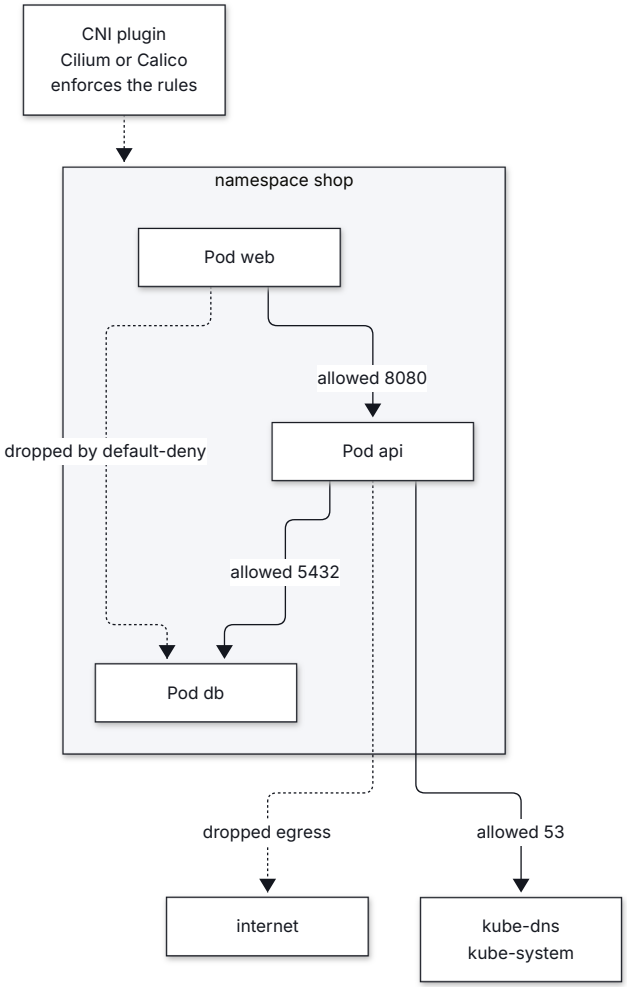


Figure 16.2-1

The rule that surprises everyone, AND versus OR inside `from`:

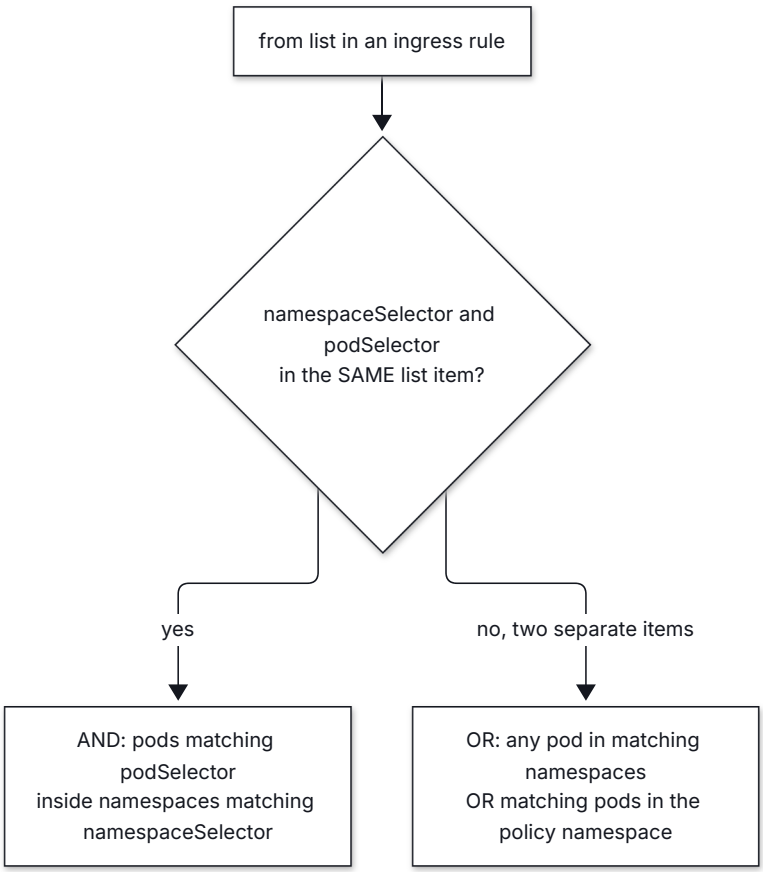


Figure 16.2-2

5 How it works

Anatomy

```yaml
spec:
  podSelector: {}            # {} = every Pod in this namespace
  policyTypes: [Ingress, Egress]
  ingress:
    - from: [ ... ]          # peers: podSelector, namespaceSelector, ipBlock
      ports: [ ... ]         # protocol + port (or port + endPort range)
  egress:
    - to: [ ... ]
      ports: [ ... ]
```

- **Empty** `podSelector` selects all Pods; **empty rule list** with a `policyTypes` entry means "allow nothing" for that direction.
- **Peers** are `podSelector`, `namespaceSelector` (use the automatic label `kubernetes.io/metadata.name: <ns>`), or `ipBlock` (CIDR with optional `except`). There are no hostname peers in the standard API.
- **Policies are stateful:** reply packets for an allowed connection are allowed automatically.
- **Both ends must agree:** traffic from A to B must be allowed by egress on A (if A is egress-restricted) **and** ingress on B (if B is ingress-restricted).
- NetworkPolicy does not apply to traffic from a Pod to its own node's host network in a consistent way across CNIs, and cannot filter traffic to the node's own kubelet/IMDS reliably; use cloud-level controls (for example blocking the instance metadata address `169.254.169.254` via `ipBlock except`, plus IMDSv2 hop limit) for that.

The default-deny pattern

1. Apply a `default-deny-all` policy (ingress and egress) to every application namespace.
2. Immediately add `allow-dns` egress (otherwise service names stop resolving).
3. Add one small allow policy per real flow: ingress controller to web, web to api, api to db, api to the allowed external CIDRs.

Egress control

Need	Standard NetworkPolicy	Cilium / Calico
Allow DNS	egress to `kube-dns` on 53 UDP and TCP	same
Allow a CIDR (e.g. a payment provider range)	`ipBlock`	same

Need	Standard NetworkPolicy	Cilium / Calico
Allow `api.stripe.com` by name	not possible (IPs change)	`toFQDNs` (Cilium), domain-based rules in Calico (feature availability depends on edition and version)
Allow only `GET /health`	not possible	`CiliumNetworkPolicy` L7 HTTP rules
Cluster-wide baseline	repeat in every namespace	`CiliumClusterwideNetworkPolicy`, Calico `GlobalNetworkPolicy`
Explicit deny rules, ordering	no	Calico and Cilium (deny policies)

Policy testing

Method	What it tells you
`kubectl run` a throwaway client and `wget` / `curl` with a short timeout	End-to-end allowed or dropped, from a chosen label set
Hubble (`hubble observe --verdict DROPPED`)	Which flow was dropped and by which policy (Cilium)
`cilium connectivity test`	Built-in conformance run for Cilium
Static analysis in CI (for example, NetworkPolicy analyzers and `kubectl` dry-runs)	Catches "policy selects nothing" and overly permissive rules before merge
Calico flow logs / policy audit mode	Shows what *would* be denied before you enforce

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`, the `cilium` CLI (and optionally the `hubble` CLI). Cost: none. Outputs are illustrative.

1. **Create a kind cluster with no default CNI** (kind's default CNI does not provide the richer features used here, so we install Cilium):

```bash
cat > kind-cni.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
networking:
  disableDefaultCNI: true
nodes:
  - role: control-plane
  - role: worker
EOF
kind create cluster --name netpol-lab --config kind-cni.yaml
kubectl get nodes     # NotReady until a CNI is installed
```

2. **Install Cilium and wait for it.**

```bash
cilium install
cilium status --wait
kubectl get nodes     # now Ready
```

Expect `Cilium: OK` and `Operator: OK`. (Calico is an equally valid choice; its install uses the Tigera operator manifests from the Calico docs.)

3. **Deploy the stand-in `shop` workloads.** We use an unprivileged nginx listening on 8080 as a stand-in for web, api and db (in real life the db would be PostgreSQL on 5432):

```bash
kubectl create namespace shop
for app in web api db; do
cat <<EOF | kubectl apply -n shop -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: $app}
spec:
  replicas: 1
  selector: {matchLabels: {app: $app}}
  template:
    metadata: {labels: {app: $app}}
    spec:
      containers:
        - name: $app
          image: nginxinc/nginx-unprivileged:1.27-alpine
          ports: [{containerPort: 8080}]
---
apiVersion: v1
kind: Service
metadata: {name: $app}
spec:
  selector: {app: $app}
  ports: [{port: 8080, targetPort: 8080}]
EOF
done
kubectl -n shop rollout status deploy/web deploy/api deploy/db 2>/dev/null || kubectl
  -n shop get pods
```

4. **Prove the network is flat first.**

```bash
kubectl -n shop run probe --rm -it --restart=Never --image=busybox:1.36 --
  labels=app=probe -- \
  wget -qO- -T 3 http://db:8080 | head -3
```

You get the nginx welcome HTML. Everything can reach everything.

5. **Apply default-deny and watch things break.**

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: shop
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
EOF
kubectl -n shop run probe --rm -it --restart=Never --image=busybox:1.36 -- wget -qO- -
  T 3 http://db:8080
```

Similar to `wget: bad address 'db'` (DNS egress blocked) — the classic first failure.

6. **Allow DNS egress, then the real flows.**

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: allow-dns, namespace: shop}
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels: {kubernetes.io/metadata.name: kube-system}
          podSelector:
            matchLabels: {k8s-app: kube-dns}
      ports:
        - {protocol: UDP, port: 53}
        - {protocol: TCP, port: 53}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: web-to-api, namespace: shop}
spec:
  podSelector: {matchLabels: {app: api}}
  policyTypes: [Ingress]
  ingress:
    - from: [{podSelector: {matchLabels: {app: web}}}]
      ports: [{protocol: TCP, port: 8080}]
---
apiVersion: networking.k8s.io/v1
```

```
kind: NetworkPolicy
metadata: {name: web-egress-to-api, namespace: shop}
spec:
  podSelector: {matchLabels: {app: web}}
  policyTypes: [Egress]
  egress:
    - to: [{podSelector: {matchLabels: {app: api}}}]
      ports: [{protocol: TCP, port: 8080}]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: api-to-db, namespace: shop}
spec:
  podSelector: {matchLabels: {app: db}}
  policyTypes: [Ingress]
  ingress:
    - from: [{podSelector: {matchLabels: {app: api}}}]
      ports: [{protocol: TCP, port: 8080}]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: api-egress-to-db, namespace: shop}
spec:
  podSelector: {matchLabels: {app: api}}
  policyTypes: [Egress]
  egress:
    - to: [{podSelector: {matchLabels: {app: db}}}]
      ports: [{protocol: TCP, port: 8080}]
EOF
```

7. **Test allowed and denied paths from the right identities.**

```bash
# allowed: web → api
kubectl -n shop exec deploy/web -- wget -qO- -T 3 http://api:8080 | head -2
# denied: web → db (no policy allows it)
kubectl -n shop exec deploy/web -- wget -qO- -T 3 http://db:8080 ; echo "exit=$?"
# allowed: api → db
kubectl -n shop exec deploy/api -- wget -qO- -T 3 http://db:8080 | head -2
```

The first and third print HTML; the second times out (`wget: download timed out`, `exit=1`). `nginx-unprivileged` images include BusyBox `wget`; if yours lacks it, use a probe pod with matching labels instead.

8. **Observe drops with Hubble (Cilium).**

```bash
cilium hubble enable
cilium status --wait
cilium hubble port-forward &
hubble observe --namespace shop --verdict DROPPED --last 10
```

Similar to `shop/web:41234 (ID:...) <> shop/db:8080 (ID:...) Policy denied DROPPED (TCP Flags: SYN)`.

9. **Add an egress rule with a CIDR and a Cilium FQDN rule.** Standard API, with the metadata address excluded:

```yaml
# api may call a payment provider's documented range, but never the IMDS address
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: api-egress-payments, namespace: shop}
spec:
  podSelector: {matchLabels: {app: api}}
  policyTypes: [Egress]
  egress:
    - to:
        - ipBlock:
            cidr: 203.0.113.0/24
            except: [203.0.113.99/32]
      ports: [{protocol: TCP, port: 443}]
```

And the name-based version, which only Cilium understands:

```yaml
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata: {name: api-egress-fqdn, namespace: shop}
spec:
  endpointSelector: {matchLabels: {app: api}}
  egress:
    - toEndpoints:
        - matchLabels:
            io.kubernetes.pod.namespace: kube-system
            k8s-app: kube-dns
      toPorts:
        - ports: [{port: "53", protocol: ANY}]
          rules:
            dns: [{matchPattern: "*"}]
    - toFQDNs: [{matchName: "api.payments.example.com"}]
      toPorts: [{ports: [{port: "443", protocol: TCP}]}]
```

The DNS rule is required so Cilium can observe lookups and learn the IPs behind the name.

10. **Clean up:** `kind delete cluster --name netpol-lab`.

7 Real-world scenario

(Illustrative.) After a security review the `shop-api` team enabled default-deny in the `shop` namespace on a Friday. Within minutes the checkout page returned 502s: the web Pod could not resolve `api`. Investigation: `kubectl exec` into web, `nslookup api` timed out; the cluster CNI was enforcing, `allow-dns` had been left out of the change. They applied the DNS policy and the site recovered in two minutes. The post-incident fixes were to ship `default-deny-all` and `allow-dns` as **one** kustomize base so they can never be applied separately, to test every policy change in staging with a probe-pod script from this topic, and to alert on Hubble drop spikes. **Lesson:** default-deny is correct, but it must be rolled out with DNS and a test.

8 Common mistakes and best practices

Don't

- Assume policies work: verify that your CNI enforces NetworkPolicy by testing a deny.
- Forget DNS egress (UDP *and* TCP 53) when you restrict egress.
- Mix up AND/OR in `from` / `to`: a `namespaceSelector` and `podSelector` in the same list item is AND; in separate items it is OR.
- Use `0.0.0.0/0` egress "just for now", or leave the instance metadata address reachable from workloads.
- Write policies keyed to labels that anyone can change (a developer who can relabel a Pod can change which policy applies; restrict who can patch Pods).

Do

- Start every namespace with `default-deny-all` plus `allow-dns`, then add one policy per flow with a descriptive name (`web-to-api`).
- Select peers by labels you control and use `kubernetes.io/metadata.name` to select namespaces.
- Test allow *and* deny paths in CI, and watch drops in Hubble or flow logs.
- Use FQDN or cluster-wide policy features (Cilium/Calico) for egress to SaaS APIs and for platform-owned baselines.

- Combine with Pod Security §16.3 and a service mesh for mTLS (see Service Mesh §16.9); NetworkPolicy is L3/L4, a mesh adds identity and L7.

9 Tasks

- [ ] **T1 — Show the flat network, then end it** *(≈30 min, beginner)*
  - **Goal:** prove default-deny changes behaviour.
  - **Steps:**
    1. Build the lab (guide steps 1-3).
    2. Show `web` reaching `db` with `wget`.
    3. Apply `default-deny-all` and repeat.
  - **Done when:** the same `wget` succeeds before and fails (`bad address` or timeout) after, and you can say why.
- [ ] **T2 — Rebuild the allowed paths** *(≈40 min, intermediate)*
  - **Goal:** least-privilege flows only.
  - **Steps:**
    1. Apply `allow-dns`, `web-to-api`, `web-egress-to-api`, `api-to-db`, `api-egress-to-db` from step 6.
    2. Run the three tests in step 7.
  - **Done when:** web to api and api to db succeed, web to db times out.
- [ ] **T3 — Allow the ingress controller and Prometheus** *(≈40 min, intermediate)*
  - **Goal:** add cross-namespace allows correctly.
  - **Steps:**
    1. Create a namespace `monitoring` with a probe Pod labelled `app=prometheus`.
    2. Write an ingress policy on `api` allowing TCP 8080 only from Pods labelled `app=prometheus` **in** `monitoring` (namespaceSelector AND podSelector in one item).
    3. Test from a Pod with the same label in another namespace and confirm it is denied.
  - **Done when:** the monitoring probe connects and the look-alike in another namespace does not.
- [ ] **T4 — Break it: the OR trap** *(≈30 min, advanced)*
  - **Goal:** experience the AND/OR mistake.
  - **Steps:**

1. Rewrite the T3 policy with `namespaceSelector` and `podSelector` as two separate list items.
2. Re-run the look-alike test and capture what is now allowed.
3. Fix it and write one sentence on the YAML difference (a single leading `-`).

- **Done when:** you have a failing-test and a passing-test transcript and the corrected manifest.

- [ ] **T5 — Observe and automate** *(≈60 min, advanced)*
  - **Goal:** turn manual probing into a repeatable test.
  - **Steps:**
    1. Write a script `netpol-test.sh` with a table of `source-label, destination, expected` rows that runs `kubectl exec ... wget -T 3` and compares the result.
    2. Use `hubble observe --verdict DROPPED` to confirm drops appear for the expected-deny rows.
    3. Make the script exit non-zero on any mismatch and add it to a GitHub Actions workflow that runs against a kind cluster (see GitHub Actions Workflows §3.11).
  - **Done when:** the script passes on your policies and fails when you delete `api-to-db`.

- [ ] **T6 — Prove it: egress lockdown design** *(≈45 min, advanced)*
  - **Goal:** design, from scratch and without the guide, egress control for `shop-api`.
  - **Steps:**
    1. List every external dependency (DNS, a payments API, package mirrors, IMDS) and decide how each is allowed or blocked.
    2. Write the policies (standard and Cilium FQDN versions where needed).
    3. Explain to a teammate why AND/OR matters, why DNS is needed, and what NetworkPolicy cannot do.
  - **Done when:** the policies apply cleanly, the allowed calls work, a call to an unlisted address fails, and your explanation covers all three points.

10 Self-check questions

1. What happens to a Pod's egress when a policy with `policyTypes: [Egress]` and no `egress` rules selects it?
2. Why can a NetworkPolicy be applied to a cluster and do nothing?
3. How do you allow traffic from Pods labelled `app=prometheus` in the `monitoring` namespace only?

4. Why can standard NetworkPolicy not allow "api.stripe.com", and what are your options?
5. A policy allows A to B on the B side, but A is egress-restricted with no rule to B. Does traffic flow?
6. Name two ways to test that a deny actually works.

ANSWERS

1. All egress is denied, including DNS.
2. Enforcement belongs to the CNI plugin; if it does not implement NetworkPolicy the objects are ignored.
3. One `from` item containing both `namespaceSelector` (`kubernetes.io/metadata.name: monitoring`) and `podSelector` (`app: prometheus`).
4. The API only supports IP-based peers. Use a CNI feature such as Cilium `toFQDNs` or Calico domain rules, an egress proxy/gateway, or a cloud firewall.
5. No. Both the egress on A and the ingress on B must allow it.
6. A probe Pod with a short-timeout `wget` / `curl` from the denied identity, and Hubble or flow logs showing a `DROPPED` verdict (also `cilium connectivity test`, policy audit mode, or CI analyzers).

11 Further reading

- Kubernetes documentation: *Network Policies* (kubernetes.io/docs/concepts/services-networking/network-policies).
- Cilium documentation: *Network Policy*, *Hubble* and *DNS-based policies*.
- Calico documentation: *Network policy* and *Global network policy*.
- Kubernetes Network Policy API working group material on `AdminNetworkPolicy` (network-policy-api.sigs.k8s.io).
- Related: Admission Controllers and Webhooks §16.6 for enforcing "every namespace must have a default-deny", and mTLS §12.12.

16.3 Pod Security

Topic 3 of 10 | ~5 hours | Difficulty: Advanced

1 What it is

A container is "just a process", and a Pod spec decides how much power that process gets over the node: may it run as root, gain extra Linux capabilities, mount the host's filesystem, share the host network, or call any system call? **Pod security** is the set of controls that constrain this.

There are two layers:

1. **Per-workload hardening with** `securityContext` — fields in the Pod and container spec (`runAsNonRoot`, `allowPrivilegeEscalation: false`, `capabilities.drop`, `seccompProfile`, `appArmorProfile`, `readOnlyRootFilesystem`) that you set so the process runs with minimum power.
2. **Cluster-level enforcement with Pod Security Admission (PSA)** — a built-in admission controller that checks every Pod against one of three **Pod Security Standards (PSS)**: `privileged` (unrestricted), `baseline` (blocks known privilege escalations) and `restricted` (current hardening best practice). You turn it on per namespace with labels.

PodSecurityPolicy (PSP) was removed in Kubernetes 1.25. If you read an old tutorial that creates a `PodSecurityPolicy`, skip it: use PSA for the built-in standards, and use ValidatingAdmissionPolicy, Kyverno or Gatekeeper §16.6 for anything custom.

2 Why it matters in DevOps

- **Container escape is the path from "one bug" to "whole node".** Privileged containers, `hostPath` mounts, `hostPID` and added capabilities such as `SYS_ADMIN` are how a compromised application becomes a compromised node and then a compromised cluster (see how RBAC `create pods` feeds into this in RBAC in Depth §16.1).
- **Root in a container is risky by default.** The first line of defence is to not run as root (Non-root Containers §4.11, Rootless and Minimal Containers §11.12).

- **Benchmarks and audits expect it.** The CIS Kubernetes Benchmark and most compliance frameworks ask for a pod-level security control on every namespace.
- **Retrofitting is painful.** Turning on `restricted` in a namespace full of root-running images breaks deployments; the `warn` / `audit` modes exist so you can find that out safely first.

3 Real-world system example

Kubernetes itself and the managed distributions. The Pod Security Standards are defined in the Kubernetes docs, and PSA has been a built-in admission plugin since it went beta in v1.23 and stable in v1.25 as the PSP replacement. Many popular Helm charts and operators (for example cert-manager, the Prometheus stack and ingress controllers) document or ship `securityContext` defaults that satisfy `restricted`, with node-level agents such as CNI plugins and node exporters being the typical workloads that need `privileged` or `baseline` and therefore live in separately labelled namespaces (the `kube-system` namespace is normally exempted or left `privileged`). Kubernetes' own `kind`, `minikube` and managed clusters ship with PSA enabled and, by default, enforce nothing: namespaces without labels behave as `privileged`. Cloud providers publish hardening guides (for example the EKS best-practices guide and GKE hardening guide) that recommend `restricted` / `baseline` on application namespaces, seccomp `RuntimeDefault` and non-root users.

4 Diagram

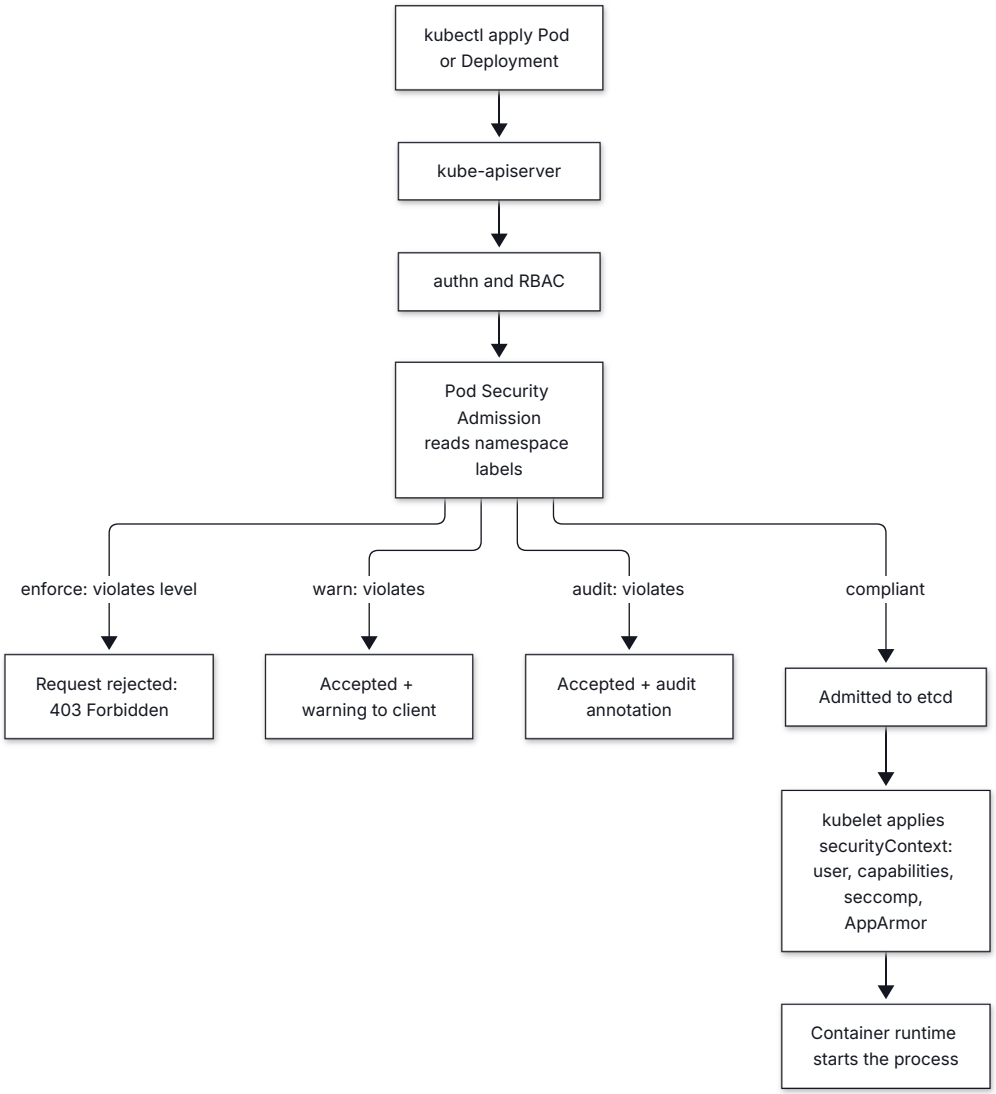


Figure 16.3-1

The three levels, nested:

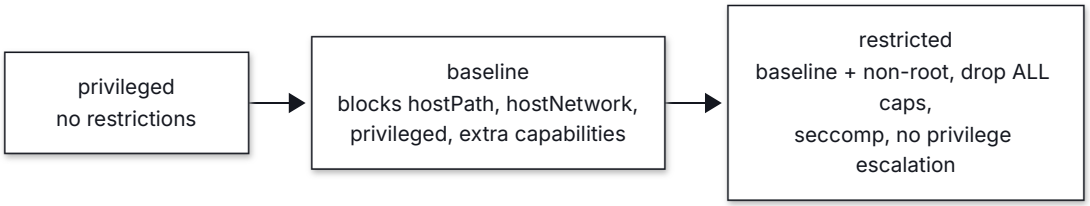


Figure 16.3-2

5 How it works

Pod Security Standards

Level	Intent	Representative rules
`privileged`	Unrestricted; for system and infrastructure workloads	Anything goes
`baseline`	Prevent *known* privilege escalations; easy to adopt	No `privileged: true`; no `hostNetwork`, `hostPID`, `hostIPC`; no `hostPath` volumes; no host ports; only a default capability set; no custom AppArmor or SELinux types; limited `sysctls`
`restricted`	Heavily restricted, hardening best practice	Everything in baseline, plus `allowPrivilegeEscalation: false`, `runAsNonRoot: true` (and no UID 0), `capabilities.drop: [ALL]` (adding back only `NET_BIND_SERVICE`), `seccompProfile` of `RuntimeDefault` or `Localhost`, and only a limited set of volume types

Pod Security Admission labels

Applied to a **namespace**, with three **modes** and an optional version pin:

```yaml
metadata:
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest   # or v1.xx to pin
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/audit: restricted
```

Mode	Effect on violation
`enforce`	Pod is **rejected**
`warn`	Pod accepted; the user sees a warning from `kubectl` (also applies to Deployments, Jobs and other Pod templates)

Mode	Effect on violation
`audit`	Pod accepted; a violation annotation is added to the audit log event

Important details:

- `enforce` only evaluates **Pods** (and checks updates that change security fields). Creating a Deployment succeeds, and the failure shows up as the ReplicaSet being unable to create Pods (`kubectl describe rs`). `warn` and `audit` do evaluate Pod templates in workload resources.
- Namespaces can be **exempted** (users, runtime classes, namespaces) and given **cluster-wide defaults** through an `AdmissionConfiguration` for the `PodSecurity` plugin, but managed control planes such as EKS and GKE generally do not let you edit it, so rely on namespace labels there.
- A safe rollout recipe: set `warn` and `audit` to `restricted` first, read the output for a week, fix the workloads, then switch on `enforce`.

`securityContext` essentials

```yaml
spec:
  securityContext:                  # pod level
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    fsGroup: 10001                  # group ownership for mounted volumes
    seccompProfile: {type: RuntimeDefault}
  containers:
    - name: api
      securityContext:              # container level
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: {drop: ["ALL"]}
```

- **Seccomp** (secure computing mode) filters which system calls the process may make. `RuntimeDefault` applies the container runtime's default profile (containerd, CRI-O), blocking many rarely-needed, dangerous calls with low compatibility risk. `Localhost` loads a custom profile from the node (`/var/lib/kubelet/seccomp/...`). `Unconfined` disables it and is rejected by `baseline` and `restricted`. Older clusters needed a kubelet `--seccomp-default` flag to make `RuntimeDefault` the default; setting it in the manifest works everywhere.

- **AppArmor** is a Linux Security Module (LSM) that confines programs by path-based profiles. On current Kubernetes it is set with `securityContext.appArmorProfile.type: RuntimeDefault | Localhost | Unconfined` (stable in v1.30; older clusters used the annotation `container.apparmor.security.beta.kubernetes.io/<container>`). It only works on nodes whose OS has AppArmor enabled (Ubuntu and Debian yes, many other distributions use **SELinux** instead, configured through `seLinuxOptions`).
- `readOnlyRootFilesystem: true` is not required by `restricted` but is a strong, cheap addition; mount an `emptyDir` at paths the app must write (`/tmp`).
- **User namespaces** (`hostUsers: false` in the Pod spec) map container root to an unprivileged host UID on runtimes and kernels that support it; check your Kubernetes version for its feature status before depending on it.

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`. Cost: none. Outputs are illustrative ("similar to").

1. **Create a cluster and a namespace in `warn` + `audit` mode.**

```bash
kind create cluster --name pss-lab
kubectl create namespace shop
kubectl label namespace shop \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/audit=restricted
```

2. **Deploy a typical "naive" workload and read the warnings.**

```bash
kubectl -n shop create deployment web --image=nginx:1.27
```

Similar to:

```
Warning: would violate PodSecurity "restricted:latest": allowPrivilegeEscalation ≠
  false (container "nginx" must set securityContext.allowPrivilegeEscalation=false),
  unrestricted capabilities (container "nginx" must set
  securityContext.capabilities.drop=["ALL"]), runAsNonRoot ≠ true (pod or container
  "nginx" must set securityContext.runAsNonRoot=true), seccompProfile (pod or
  container "nginx" must set securityContext.seccompProfile.type to "RuntimeDefault"
  or "Localhost")
deployment.apps/web created
```

The Deployment was *created* (warn mode); the warning is your to-do list.

3. **Dry-run the effect of `enforce` before turning it on.** The server evaluates all existing Pods in the namespace:

```bash
kubectl label --dry-run=server --overwrite namespace shop \
  pod-security.kubernetes.io/enforce=restricted
```

   You get the same warnings for each existing Pod that would be rejected on re-creation; nothing is changed.

4. **Fix the workload so it meets `restricted`.** The stock `nginx` image runs as root and listens on 80, so use the unprivileged image:

```bash
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 1
  selector: {matchLabels: {app: web}}
  template:
    metadata: {labels: {app: web}}
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 101
        seccompProfile: {type: RuntimeDefault}
      containers:
        - name: web
          image: nginxinc/nginx-unprivileged:1.27-alpine
          ports: [{containerPort: 8080}]
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: {drop: ["ALL"]}
          volumeMounts:
            - {name: tmp, mountPath: /tmp}
      volumes:
        - {name: tmp, emptyDir: {}}
EOF
kubectl -n shop rollout status deploy/web
```

   No warning this time. (The nginx-unprivileged image also needs writable cache/pid paths; if the Pod crash-loops with a read-only error, check `kubectl logs` and add an `emptyDir` for the path named, or temporarily drop `readOnlyRootFilesystem`.)

5. **Turn on `enforce` and try to sneak a bad Pod in.**

```bash
kubectl label --overwrite namespace shop pod-security.kubernetes.io/enforce=restricted
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: v1
kind: Pod
metadata: {name: bad}
spec:
  hostNetwork: true
  containers:
    - name: c
      image: busybox:1.36
      command: ["sleep", "3600"]
      securityContext: {privileged: true}
      volumeMounts: [{name: host, mountPath: /host}]
  volumes:
    - name: host
      hostPath: {path: /}
EOF
```

Similar to: `Error from server (Forbidden): error when creating "STDIN": pods "bad" is forbidden: violates PodSecurity "restricted:latest": host namespaces (hostNetwork=true), privileged (container "c" must not set securityContext.privileged=true), hostPath volumes (volume "host"), ...`.

6. **See the Deployment-versus-Pod subtlety.** Create a violating Deployment under `enforce`:

```bash
kubectl -n shop create deployment naive --image=nginx:1.27
kubectl -n shop get deploy naive            # READY 0/1
kubectl -n shop describe rs -l app=naive | grep -A2 -i "forbidden"
```

The Deployment exists but its ReplicaSet reports `FailedCreate ... violates PodSecurity`. Monitoring should alert on ReplicaSets that cannot create Pods.

7. **Check what the runtime actually applied.** Confirm seccomp and the user:

```bash
kubectl -n shop get pod -l app=web -o jsonpath='{.items[0].spec.securityContext}
  {"\n"}'
kubectl -n shop exec deploy/web -- id
kubectl -n shop exec deploy/web -- grep -E 'Seccomp|CapEff|NoNewPrivs' /proc/1/status
```

Expect `uid=101`, `Seccomp: 2` (filter mode), `CapEff: 0000000000000000` and `NoNewPrivs: 1`.

8. **Optional: AppArmor.** On a node/VM where AppArmor is enabled (for example an Ubuntu host with a multi-node kind cluster, not Docker Desktop), add this to the container's `securityContext` and verify with `kubectl exec ... cat /proc/1/attr/current`:

   ```yaml
   appArmorProfile:
     type: RuntimeDefault
   ```

   If the node cannot enforce AppArmor the Pod may fail to start; that is correct behaviour for a profile you asked for.
9. **Clean up:** `kind delete cluster --name pss-lab`.

7 Real-world scenario

(Illustrative.) A platform team rolled out PSA `restricted` on all team namespaces after an audit finding. They started with `warn` and `audit` only. The audit log annotations showed that 14 of 60 Deployments lacked `runAsNonRoot` and that a legacy reporting service mounted `hostPath: /var/log` to read node logs. The team fixed the 14 images and moved the log reader into a separate `observability-agents` namespace labelled `baseline`, with the exception documented. Two weeks later they flipped `enforce`. Months afterwards a compromised dependency in one app attempted to spawn a privileged pod through a leaked token; the API server returned `violates PodSecurity "restricted:latest"`, and the attempt appeared in the audit log. **Lesson:** `warn` / `audit` first, exceptions in dedicated namespaces, and `enforce` as a backstop that limits what a stolen token can do.

8 Common mistakes and best practices

Don't

- Write new `PodSecurityPolicy` objects; the API no longer exists.
- Flip `enforce` straight to `restricted` in production without `warn` / `audit` and a server-side dry run.
- Exempt a whole namespace "because it is easier" or give application teams `privileged`.
- Assume `Deployment` creation failing is how PSA reports; check ReplicaSet events too.
- Rely on image `USER` alone; set `runAsNonRoot` so the kubelet verifies it.

Do

- Use `restricted` for application namespaces, `baseline` or `privileged` only for narrowly justified infrastructure namespaces, and pin the `-version` for predictable upgrades.

- Set `seccompProfile: RuntimeDefault`, `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]` and `readOnlyRootFilesystem: true` in every chart and base manifest.
- Write the namespace labels in Git and add an admission policy that rejects namespaces without them (Admission Controllers §16.6).
- Scan images and keep them minimal (Image Scanning and Container Security §4.14).

9 Tasks

- [ ] **T1 — Read the warnings** *(≈20 min, beginner)*
  - **Goal:** see what `restricted` objects to.
  - **Steps:**
    1. Do guide steps 1-2.
    2. Copy the warning text and map every item to the `securityContext` field that fixes it.
  - **Done when:** you have a table of 4+ violations and the field that resolves each.
- [ ] **T2 — Make it compliant** *(≈40 min, intermediate)*
  - **Goal:** run `shop-api`-style workloads under `restricted`.
  - **Steps:**
    1. Apply the hardened Deployment from step 4.
    2. Confirm with `kubectl label --dry-run=server --overwrite ns shop pod-security.kubernetes.io/enforce=restricted` that no warnings appear.
    3. Run step 7 and record the `id` and `/proc/1/status` lines.
  - **Done when:** the dry run is silent and `exec ... id` shows a non-zero UID with `NoNewPrivs: 1`.
- [ ] **T3 — Enforce and attack** *(≈30 min, intermediate)*
  - **Goal:** watch enforcement block an escalation.
  - **Steps:**
    1. Enable `enforce=restricted` (step 5) and try a Pod with `privileged: true`, then one with `hostPath`, then one with `capabilities.add: ["NET_ADMIN"]`.
    2. Capture each rejection message.
  - **Done when:** all three are rejected and you can name the PSS rule that caught each.
- [ ] **T4 — Break it: the silent Deployment failure** *(≈30 min, advanced)*
  - **Goal:** diagnose a "0/1 ready" caused by PSA.

- **Steps:**
  1. Create the `naive` Deployment under `enforce`.
  2. Find the cause using only `kubectl get/describe` (no external tools).
  3. Fix it by patching the Deployment's Pod template.
- **Done when:** `naive` reaches `1/1` and you wrote down the exact event message that revealed the cause.

☐ **T5 — Automate namespace policy** *(≈45 min, advanced)*

- **Goal:** make unlabelled namespaces impossible.
- **Steps:**
  1. Write a short script (or a kustomize base) that creates a namespace with `enforce`, `warn` and `audit` labels all set to `restricted` and pinned `-version`.
  2. Add a CI check that fails if any `Namespace` manifest in the repo lacks `pod-security.kubernetes.io/enforce`.
  3. (Stretch) implement the same check as an admission policy after Admission Controllers and Webhooks §16.6.
- **Done when:** CI fails on a namespace manifest without labels and passes with them.

☐ **T6 — Prove it: harden and explain** *(≈45 min, advanced)*

- **Goal:** from a plain `nginx` Deployment, reach `restricted` without the guide.
- **Steps:**
  1. Take `kubectl create deployment web --image=nginx:1.27` and make it pass `restricted` enforcement.
  2. Explain in writing what each added field defends against (privilege escalation, kernel attack surface, container escape) and why PSP is not an option.
- **Done when:** the Pod runs in an `enforce=restricted` namespace and your note covers `runAsNonRoot`, capabilities, seccomp and `allowPrivilegeEscalation`.

10 Self-check questions

1. What replaced PodSecurityPolicy and in which release was PSP removed?
2. Which three modes does PSA have and in what order would you roll them out?
3. Why does `kubectl create deployment` succeed in an `enforce` namespace even though the Pod will not run?
4. Which four settings are the core of the `restricted` profile for a container?

5. What does `seccompProfile: RuntimeDefault` do, and what is AppArmor's role compared to it?
6. How do you apply a different level to a node-agent namespace safely?

ANSWERS

1. Pod Security Admission with the Pod Security Standards; PSP was removed in Kubernetes 1.25.
2. `audit` and `warn` first (observe), then `enforce`. Use the server-side dry-run label check before enforcing.
3. PSA enforcement checks Pods, not Deployments. The ReplicaSet controller's attempt to create the Pod is what is rejected, so you see `FailedCreate` events on the ReplicaSet.
4. Non-root user, no privilege escalation, drop ALL capabilities, and seccomp `RuntimeDefault` (or `Localhost`).
5. Seccomp filters which system calls the process may invoke. AppArmor (or SELinux) is a separate Linux Security Module that restricts file, network and capability access by profile.
6. Put those workloads in their own namespace labelled `baseline` or `privileged`, restrict who can deploy there with RBAC, and document the exception.

11 Further reading

- Kubernetes documentation: *Pod Security Standards*, *Pod Security Admission* and *Configure a Security Context for a Pod or Container*.
- Kubernetes documentation: *Migrate from PodSecurityPolicy to the Built-In PodSecurity Admission Controller*.
- Kubernetes documentation: *Restrict a Container's Syscalls with seccomp* and *Restrict a Container's Access to Resources with AppArmor*.
- CIS Kubernetes Benchmark (Center for Internet Security) and the NSA/CISA *Kubernetes Hardening Guide*.
- Related: Admission Controllers and Webhooks §16.6, RBAC in Depth §16.1.

16.4 Operators

Topic 4 of 10 | ~5 hours | Difficulty: Advanced

1 What it is

Kubernetes is built from **controllers**: loops that watch the *desired state* you declared (a Deployment with 3 replicas) and the *observed state* of the world (2 Pods running), and act to close the gap. A **reconcile loop** is that act: "look at what is, compare to what should be, make one step toward it, repeat".

An **Operator** is a controller that encodes the operational knowledge of a specific application. Instead of a runbook that says "to add a PostgreSQL replica, create a volume, run `pg_basebackup`, edit `postgresql.conf`, register the standby", you apply a small declarative object (`Cluster` with `instances: 3`) and the operator performs those steps, and keeps performing them when things fail. The pattern is **Custom Resource Definition (CRD) + custom controller**: the CRD teaches the API server a new kind (covered next in Custom Resource Definitions §16.5), and the controller is a Pod that reconciles objects of that kind.

2 Why it matters in DevOps

- **Day-2 operations as code.** Backups, failover, version upgrades, certificate renewal and scaling become reconciled behaviour, not 3 a.m. manual procedures.
- **It is how the ecosystem ships software.** Prometheus, cert-manager, Strimzi (Kafka), the major Postgres operators, Argo CD, Istio and Crossplane are all controllers; to run or debug them you must understand reconcile loops, finalizers and status conditions.
- **Platform APIs.** Platform teams use operators and CRDs to expose "give me a database" as one YAML object (Platform Engineering §0.13).
- **Risk.** An operator runs with broad RBAC and manages your data. A buggy or unmaintained operator can delete things. Choosing and vetting operators is a senior skill.

3 Real-world system example

The Prometheus Operator and Postgres operators.

- The **Prometheus Operator** (the basis of the `kube-prometheus-stack` Helm chart) introduces CRDs such as `Prometheus`, `Alertmanager`, `ServiceMonitor`, `PodMonitor` and `PrometheusRule`. You declare *what to scrape* with a `ServiceMonitor` selecting Services by label; the operator generates the Prometheus configuration, writes it to a Secret, and reloads Prometheus. Application teams never edit `prometheus.yml` (see Prometheus and PromQL §10.2).
- **PostgreSQL operators** such as **CloudNativePG** (a CNCF project), Zalando's postgres-operator, Crunchy Data's PGO and Percona's operator let you declare a Postgres cluster (instance count, storage, backup schedule) and handle replication, primary election, failover, rolling minor upgrades and backups to object storage. They differ in design: CloudNativePG, for example, manages Pods directly rather than through StatefulSets.
- **cert-manager** reconciles `Certificate` objects into issued TLS Secrets and renews them before expiry; **Strimzi** does the same for Kafka clusters.

Kubernetes' own built-in controllers (Deployment, ReplicaSet, Job, StatefulSet) use exactly the same pattern, so operators are not a foreign mechanism, just more domain-specific controllers.

4 Diagram

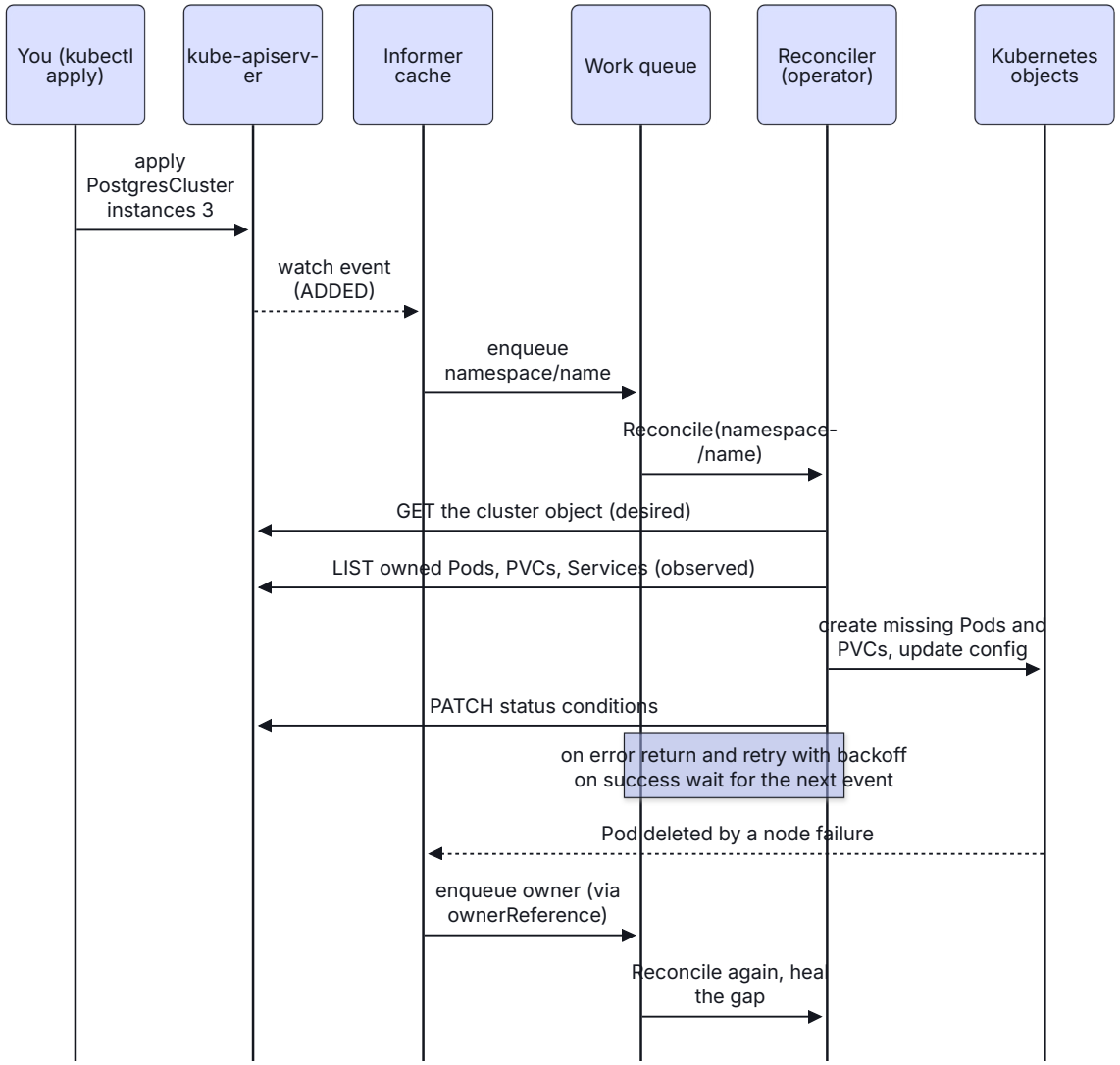


Figure 16.4-1

Controller anatomy:

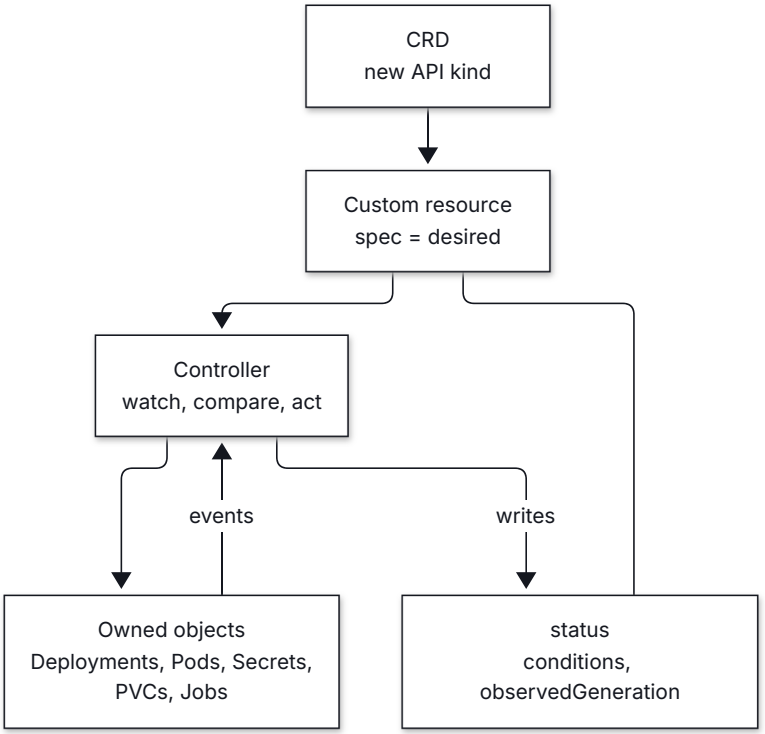


Figure 16.4-2

5 How it works

Key ideas

Concept	Meaning
Level-triggered	The loop reacts to *state*, not to individual events. If an event is missed, the next reconcile still converges. Never write reconcile code that assumes "this runs once per change".
Idempotent	Running reconcile twice with the same input gives the same result. Use create-or-update, not blind create.
Informer / cache	Controllers read from a local watch-backed cache to avoid hammering the API server; writes still go to the API.
Work queue and backoff	Keys (`namespace/name`) are queued, deduplicated and retried with exponential backoff on error.
ownerReferences	Child objects point to the parent CR, so deleting the parent garbage-collects the children and child events trigger the parent's reconcile.

Concept	Meaning
Finalizers	A marker that blocks deletion until the controller has cleaned up external resources (a cloud disk, a DNS record). A stuck finalizer is a classic "namespace stuck in Terminating" cause.
Status subresource	`spec` is what the user wants, `status` is what the controller observed; conditions (`Ready`, `Progressing`) and `observedGeneration` tell users whether it is up to date.
Leader election	Multiple operator replicas run but only one reconciles, via a Lease object.

Operator capability levels

The Operator Framework describes five capability levels: (1) basic install, (2) seamless upgrades, (3) full lifecycle (backup, failure recovery), (4) deep insights (metrics, alerts), (5) auto-pilot (auto-scaling, tuning). Use them to judge how much an operator really automates before trusting it with data.

Tooling to write your own

Tool	Notes
kubebuilder	The Kubernetes SIG project's framework and scaffolder (Go, built on `controller-runtime`). Generates the project, CRD manifests from Go types, RBAC from markers, and tests with `envtest`.
Operator SDK	From the Operator Framework (Red Hat); in Go it builds on kubebuilder, and it also supports **Ansible** and **Helm**-based operators when you do not want to write Go.
controller-runtime	The library under both: `Manager`, `Reconciler`, `client`, `builder`.
Operator Lifecycle Manager (OLM) and **OperatorHub.io**	Catalog and install/upgrade operators (more common on OpenShift; elsewhere Helm and Argo CD installs dominate).
Kopf (Python), Metacontroller, shell-operator, Crossplane compositions	Lighter or non-Go options for simple cases (Crossplane §18.9).

A minimal reconcile in Go (a sketch of what kubebuilder scaffolds; not complete code):

```go
func (r *ShopCacheReconciler) Reconcile(ctx context.Context, req ctrl.Request)
  (ctrl.Result, error) {
  var cache shopv1alpha1.ShopCache
  if err := r.Get(ctx, req.NamespacedName, &cache); err ≠ nil {
          return ctrl.Result{}, client.IgnoreNotFound(err) // deleted: nothing to do
  }

  dep := &appsv1.Deployment{ObjectMeta: metav1.ObjectMeta{Name: cache.Name, Namespace:
  cache.Namespace}}
  _, err := controllerutil.CreateOrUpdate(ctx, r.Client, dep, func() error {
          dep.Spec.Replicas = &cache.Spec.Replicas
```

```
            // ... set selector, pod template (redis image from cache.Spec.Version) ...
            return controllerutil.SetControllerReference(&cache, dep, r.Scheme)
    })
    if err ≠ nil {
            return ctrl.Result{}, err // controller-runtime requeues with backoff
    }

    cache.Status.ReadyReplicas = dep.Status.ReadyReplicas
    return ctrl.Result{}, r.Status().Update(ctx, &cache)
}
```

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`, `helm`. Optional: Go and `kubebuilder` for step 7. Cost: none. Outputs are illustrative.

1. **Start a cluster and watch a built-in controller reconcile.**

   BASH
   ```
   kind create cluster --name operators-lab
   kubectl create namespace shop
   kubectl -n shop create deployment shop-api --image=nginx:1.27 --replicas=3
   kubectl -n shop get pods -w &           # watch in the background
   kubectl -n shop delete pod -l app=shop-api --wait=false
   ```

   Similar to: new Pods appear within seconds as old ones terminate. The ReplicaSet controller saw observed (0) differ from desired (3) and reconciled. Stop the watch with `kill %1`.

2. **Write a "poor man's operator" to feel the loop.** It reads desired replicas from a ConfigMap and drives the Deployment:

   BASH
   ```
   kubectl -n shop create configmap desired --from-literal=replicas=2
   cat > mini-operator.sh <<'EOF'
   #!/usr/bin/env bash
   # level-triggered loop: compare desired (ConfigMap) with observed (Deployment) every
     5s
   while true; do
     want=$(kubectl -n shop get cm desired -o jsonpath='{.data.replicas}')
     have=$(kubectl -n shop get deploy shop-api -o jsonpath='{.spec.replicas}')
     if [ "$want" ≠ "$have" ]; then
       echo "$(date +%T) drift: have=$have want=$want → scaling"
       kubectl -n shop scale deploy/shop-api --replicas="$want"
     fi
     sleep 5
   done
   EOF
   ```

```
chmod +x mini-operator.sh && ./mini-operator.sh &
```

Within 5 seconds expect `drift: have=3 want=2 → scaling`. Now try `kubectl -n shop scale deploy/shop-api --replicas=5` by hand: the loop undoes your change. Edit the ConfigMap (`kubectl -n shop patch cm desired --type merge -p '{"data":{"replicas":"4"}}'`) and it converges to 4. Real operators avoid polling by using watches, and handle errors, conflicts and deletion, but the logic is the same. Stop it with `kill %2` (or `fg` then Ctrl+C).

3. **Install a real operator: CloudNativePG** (an open source Postgres operator) with Helm:

```bash
helm repo add cnpg https://cloudnative-pg.github.io/charts
helm repo update
helm upgrade --install cnpg cnpg/cloudnative-pg --namespace cnpg-system --create-
  namespace
kubectl -n cnpg-system rollout status deploy -l app.kubernetes.io/name=cloudnative-pg
kubectl get crd | grep cnpg.io
```

Expect CRDs such as `clusters.postgresql.cnpg.io`, `backups.postgresql.cnpg.io` and `scheduledbackups.postgresql.cnpg.io`. The operator is just a Deployment; the new API kinds are the product.

4. **Declare a database; do not build one.**

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: shop-db
  namespace: shop
spec:
  instances: 3
  storage:
    size: 1Gi
EOF
kubectl -n shop get cluster shop-db -w
```

Similar to `STATUS: Cluster in healthy state` after a minute or two, with Pods `shop-db-1`, `shop-db-2`, `shop-db-3` (one primary, two replicas) and Services `shop-db-rw`, `shop-db-ro`, `shop-db-r`.

5. **Break it and watch the operator heal.**

```bash
kubectl -n shop get cluster shop-db -o jsonpath='{.status.currentPrimary}{"\n"}'
kubectl -n shop delete pod $(kubectl -n shop get cluster shop-db -o
  jsonpath='{.status.currentPrimary}')
kubectl -n shop get cluster shop-db -w
```

   Expect a short failover: a replica is promoted, `currentPrimary` changes and the deleted instance returns as a replica. You wrote no failover code.

6. **Read the operator's logs and the object's status.**

```bash
kubectl -n cnpg-system logs deploy/cnpg-cloudnative-pg --tail=20
kubectl -n shop describe cluster shop-db | sed -n '/Status:/,$p' | head -30
```

   Look for reconcile messages and the `Conditions` list. This is where you debug an operator that is "stuck".

7. **Scaffold your own operator (optional, needs Go and kubebuilder).** Commands, not run in this repo's sandbox:

```bash
mkdir shop-operator && cd shop-operator
kubebuilder init --domain example.com --repo github.com/example/shop-operator
kubebuilder create api --group shop --version v1alpha1 --kind ShopCache --resource --
  controller
# edit api/v1alpha1/shopcache_types.go (Spec: Replicas, Version) and
  internal/controller/shopcache_controller.go
make manifests generate        # regenerate CRD YAML and deepcopy code
make install                   # apply the CRD to the current cluster
make run                       # run the controller locally against your kube context
kubectl apply -f config/samples/shop_v1alpha1_shopcache.yaml
```

   `make deploy IMG=<registry>/shop-operator:0.1.0` builds and installs it in-cluster after `make docker-build docker-push IMG=...`. Operator SDK equivalents are `operator-sdk init` and `operator-sdk create api`.

8. **Clean up:** `kill %1 %2 2>/dev/null; kind delete cluster --name operators-lab`.

7 Real-world scenario

(Illustrative.) The `shop-api` team ran PostgreSQL on a hand-built StatefulSet. When the primary's node was drained for a kernel update, the Pod restarted but the replica's replication slot was stale, and an engineer spent two hours repairing replication while the checkout API served errors. After the incident they moved to a Postgres operator: they declared `instances: 3`, a backup schedule to object storage and a `PodDisruptionBudget`. The next node drain triggered an automatic switchover in seconds, and the status condition and operator logs told the on-call exactly what happened. The follow-up risk review added an upgrade policy for the operator itself (pinned versions, staged rollout, restore-from-backup drill). **Lesson:** operators turn tribal knowledge into reconciled behaviour, but you now operate the operator.

8 Common mistakes and best practices

Don't

- Treat reconcile as event-driven ("on create do X"). Always compare current and desired state and tolerate re-runs.
- Block the reconcile with long sleeps; return `RequeueAfter` instead.
- Forget finalizers and ownerReferences, leaving orphaned cloud resources or undeletable objects.
- Install operators from random manifests with `cluster-admin` and no review.
- Write an operator when a Helm chart or a Deployment suffices.

Do

- Report progress in `status.conditions` and `observedGeneration`; users and GitOps tools read them.
- Pin operator and CRD versions, upgrade in staging, and read upgrade notes (CRD changes are cluster-wide).
- Run operators with narrow RBAC, resource limits, replicas with leader election and metrics scraped by Prometheus.
- Prefer a well-maintained community operator for databases and brokers over writing your own.
- Back up the *data*, not just the manifests, and test restores (Testing Backup Restoration §15.9).

9 Tasks

- [ ] **T1 — Watch a controller reconcile** *(≈20 min, beginner)*
  - **Goal:** see level-triggered convergence.
  - **Steps:**
    1. Do guide step 1.
    2. Scale to 5, then delete two Pods, then `kubectl cordon` a node and delete a Pod.
    3. Watch the ReplicaSet recreate Pods each time.
  - **Done when:** you can describe, for each action, what the observed and desired counts were.
- [ ] **T2 — Run the mini operator** *(≈30 min, intermediate)*
  - **Goal:** implement drift correction yourself.
  - **Steps:**
    1. Run `mini-operator.sh` from step 2.
    2. Change replicas by hand and via the ConfigMap, and observe the difference.
  - **Done when:** the Deployment always converges to the ConfigMap value within ~5 s and you can explain why this is level-triggered.
- [ ] **T3 — Improve the mini operator** *(≈45 min, intermediate)*
  - **Goal:** add the missing production behaviours.
  - **Steps:**
    1. Add validation (reject a non-integer or a value above 10 and keep the old value).
    2. Write the result to the ConfigMap's annotation `status` (for example `reconciled at <time>`), mimicking `status`.
    3. Make the script exit cleanly on SIGTERM using `trap`.
  - **Done when:** invalid input is ignored with a log line, the annotation is updated after each successful reconcile and `kill -TERM` stops it.
- [ ] **T4 — Install a real operator and fail it over** *(≈45 min, advanced)*
  - **Goal:** prove an operator heals a stateful app.
  - **Steps:**
    1. Install CloudNativePG and create the `shop-db` cluster (steps 3-4).
    2. Delete the primary Pod (step 5).
    3. Record the old and new `currentPrimary`, and the time to recover.

- **Done when:** the cluster returns to the healthy state with a different primary and you can show it in the status and logs.

- [ ] **T5 — Break it: the stuck finalizer** *(≈30 min, advanced)*
  - **Goal:** understand why deletions hang.
  - **Steps:**
    1. Scale the operator Deployment in `cnpg-system` to 0.
    2. Delete the `shop-db` Cluster object and observe it stuck in terminating or its children lingering.
    3. Scale the operator back to 1 and watch cleanup complete.
  - **Done when:** you can explain (with `kubectl get cluster shop-db -o yaml | grep -A3 finalizers`) why the operator must be running for cleanup, and the object is gone after scaling up.

- [ ] **T6 — Prove it: scaffold and explain** *(≈90 min, advanced)*
  - **Goal:** build and explain a minimal operator.
  - **Steps:**
    1. Using kubebuilder (or Operator SDK), scaffold `ShopCache` and implement a reconcile that keeps a Redis Deployment at `spec.replicas`.
    2. Run it with `make run` and show self-healing when you delete the Deployment.
    3. Explain `ownerReferences`, `status` and requeue behaviour out loud or in writing.
  - **Done when:** deleting the owned Deployment results in it being recreated, and your explanation covers idempotency and level-triggering.

10 Self-check questions

1. What is the difference between an event-driven and a level-triggered controller, and why does Kubernetes use the second?
2. What do a CRD and a controller each contribute to an operator?
3. Why do operators set `ownerReferences` on the objects they create?
4. What is a finalizer and what happens if the operator that owns it is gone?
5. When would you not write an operator?
6. How do kubebuilder and Operator SDK relate?

ANSWERS

1. Event-driven reacts once per change and can miss events; level-triggered compares full current and desired state each time, so a missed event or restart still converges.

2. The CRD adds the new API type (desired-state schema); the controller makes reality match it.
3. For garbage collection when the parent is deleted and to map child events back to the parent's reconcile.
4. A key in `metadata.finalizers` that blocks deletion until cleanup is done. Without the operator, the object stays in terminating until the finalizer is removed manually (carefully).
5. When a Deployment, a Helm chart or a managed cloud service does the job; operators add code to maintain and permissions to trust.
6. Operator SDK (Go) builds on kubebuilder and controller-runtime and adds Ansible/Helm operator types and OLM integration.

11 Further reading

- Kubernetes documentation: *Operator pattern* and *Controllers* (kubernetes.io/docs/concepts).
- The Kubebuilder Book (book.kubebuilder.io) and the Operator SDK documentation (sdk.operatorframework.io).
- Prometheus Operator and CloudNativePG project documentation.
- *Kubernetes Operators* by Jason Dobies and Joshua Wood (O'Reilly); *Programming Kubernetes* by Michael Hausenblas and Stefan Schimanski (O'Reilly).
- Related: Stateful Workloads §16.8, StatefulSets §8.10.

16.5 Custom Resource Definitions (CRDs)

Topic 5 of 10 | ~4 hours | Difficulty: Advanced

1 What it is

The Kubernetes API is a set of **resource types** (Pod, Service, Deployment, ...) served by the API server and stored in etcd. A **CustomResourceDefinition (CRD)** lets you add your *own* resource type to that API at runtime, with no changes to Kubernetes itself and no extra API server. Once the CRD exists you can `kubectl apply` objects of the new kind (called **custom resources**, CRs), and `kubectl get`, RBAC, audit logging, watches, labels and GitOps tools all work on them exactly as they do on built-in objects.

A CRD alone is only **data with validation**: the API server will store a `ShopCache` object, but nothing happens until a controller acts on it. The CRD plus a controller is the Operator §16.4 pattern.

2 Why it matters in DevOps

- **Almost every cloud-native tool is a CRD set.** Argo CD (`Application`), cert-manager (`Certificate`), Istio (`VirtualService`), Prometheus Operator (`ServiceMonitor`), Crossplane and Karpenter (`NodePool`) all extend Kubernetes this way. Reading and debugging them is daily work.
- **Schema design is API design.** A sloppy schema becomes a permanent contract: field names, defaults and validation bugs live for years, and CRD upgrades are cluster-wide.
- **Validation at the door.** A good schema (types, enums, ranges, CEL rules) rejects bad YAML at `kubectl apply` time instead of at 3 a.m. inside a controller.
- **Platform engineering.** Teams expose "give me a cache" as a simple CR and hide the Deployments, Services and PodDisruptionBudgets behind it (Self-service Infrastructure §18.3).

3 Real-world system example

Karpenter, cert-manager and Argo CD. Karpenter's node-provisioning API is two CRDs in the `karpenter.sh` group (`NodePool`, `NodeClaim`) plus a cloud-specific `EC2NodeClass` on AWS; you tune cluster scaling by editing those objects. cert-manager defines `Issuer`, `ClusterIssuer` and `Certificate`; a `Certificate` is a few lines of YAML while the controller talks to Let's Encrypt and writes a TLS Secret. Argo CD defines `Application` and `AppProject`, so "what is deployed where" is itself Kubernetes data. Kubernetes also ships its own API extensions this way: the Gateway API (`Gateway`, `HTTPRoute`) is delivered as CRDs from the Kubernetes SIG Network project, installed independently of the cluster version, and the Cluster API project models whole clusters as CRs.

4 Diagram

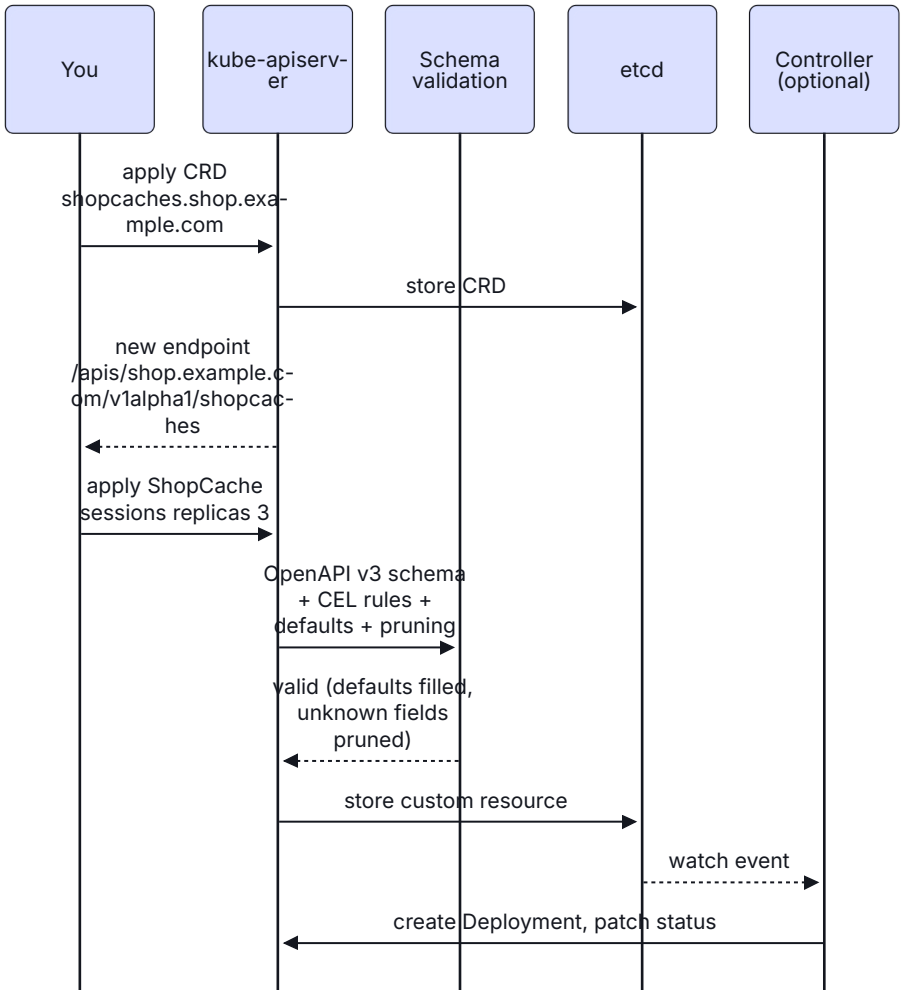


Figure 16.5-1

Multiple versions and conversion:

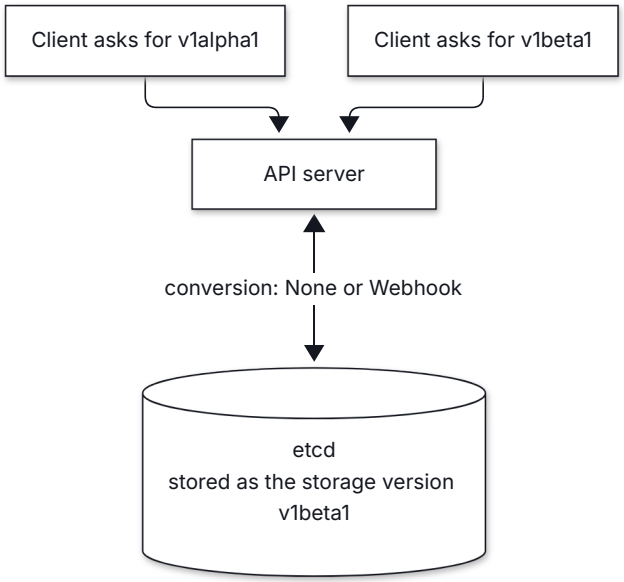


Figure 16.5-2

5 How it works

Anatomy of a CRD

Field	Purpose
`metadata.name`	Must be `<plural>.<group>`
`spec.group`	API group, a DNS name you control (`shop.example.com`)
`spec.scope`	`Namespaced` or `Cluster`
`spec.names`	`kind` , `plural` , `singular` , optional `shortNames` and `categories` (`kubectl get all` style groups)
`spec.versions[]`	Each with `name` , `served` , `storage` , `schema` , `subresources` , `additionalPrinterColumns` , `deprecated`
`schema.openAPIV3Schema`	A **structural** schema (every field typed) used for validation, defaulting and pruning
`subresources.status`	Separate `/status` endpoint so users edit `spec` and the controller edits `status` (with different RBAC)
`subresources.scale`	Makes `kubectl scale` and HPA work on your kind

Field	Purpose
`conversion`	`None` (only `apiVersion` changes) or `Webhook` (a service you run converts between versions)

Schema features worth knowing

- **Types and validation:** `type`, `enum`, `minimum`, `maximum`, `pattern`, `maxLength`, `required`, `format`.
- **Defaulting:** `default:` fills missing fields on create and on read from etcd.
- **Pruning:** fields not in the schema are dropped (unless you mark `x-kubernetes-preserve-unknown-fields: true`). Typos in field names are silently removed, so use `kubectl apply --validate=strict` or `kubectl explain` to catch them.
- **CEL validation rules** (`x-kubernetes-validations`) express cross-field and transition rules without a webhook, for example immutable fields via `self == oldSelf` (Admission Controllers and Webhooks §16.6 uses the same expression language).
- **Lists:** `x-kubernetes-list-type: map` with `x-kubernetes-list-map-keys` makes `conditions` mergeable by `type`.

Versioning rules of thumb

- Exactly **one version is the storage version**. Other served versions are converted on the fly.
- Start at `v1alpha1`, promote to `v1beta1` and `v1`. Keep old versions `served: true` until clients have migrated, mark them `deprecated: true`, then stop serving.
- With `conversion: None`, versions must be structurally compatible (a renamed field needs a **conversion webhook**).
- Changing the storage version does not rewrite old objects; use a storage-version migration (for example the Kube Storage Version Migrator or re-applying each object) before you remove an old version from the CRD's `status.storedVersions`.
- Deleting a CRD **deletes every custom resource of that kind**. Treat CRDs like database schemas.

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`. Cost: none. Outputs are illustrative.

1. **Create a cluster.**

```bash
kind create cluster --name crd-lab
```

2. **Define the `ShopCache` CRD (version `v1alpha1`).**

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: shopcaches.shop.example.com
spec:
  group: shop.example.com
  scope: Namespaced
  names:
    plural: shopcaches
    singular: shopcache
    kind: ShopCache
    shortNames: [sc]
    categories: [shop]
  versions:
    - name: v1alpha1
      served: true
      storage: true
      subresources:
        status: {}
        scale:
          specReplicasPath: .spec.replicas
          statusReplicasPath: .status.readyReplicas
      additionalPrinterColumns:
        - {name: Version, type: string, jsonPath: .spec.version}
        - {name: Replicas, type: integer, jsonPath: .spec.replicas}
        - {name: Ready, type: integer, jsonPath: .status.readyReplicas}
        - {name: Age, type: date, jsonPath: .metadata.creationTimestamp}
      schema:
        openAPIV3Schema:
          type: object
          required: [spec]
          properties:
            spec:
              type: object
              required: [version]
              properties:
                replicas:
                  type: integer
                  minimum: 1
                  maximum: 10
                  default: 1
                version:
                  type: string
                  pattern: '^[0-9]+\.[0-9]+(\.[0-9]+)?$'
                evictionPolicy:
                  type: string
                  enum: [noeviction, allkeys-lru, volatile-lru]
                  default: allkeys-lru
```

```
                  x-kubernetes-validations:
                    - rule: self == oldSelf
                      message: evictionPolicy is immutable once set
              x-kubernetes-validations:
                - rule: "!(self.evictionPolicy == 'noeviction' && self.replicas > 3)"
                  message: noeviction caches are limited to 3 replicas
            status:
              type: object
              properties:
                readyReplicas:
                  type: integer
                conditions:
                  type: array
                  x-kubernetes-list-type: map
                  x-kubernetes-list-map-keys: [type]
                  items:
                    type: object
                    required: [type, status]
                    properties:
                      type: {type: string}
                      status: {type: string}
                      reason: {type: string}
                      message: {type: string}
                      lastTransitionTime: {type: string, format: date-time}
EOF
kubectl get crd shopcaches.shop.example.com
kubectl api-resources --api-group=shop.example.com
```

Expect the CRD with `ESTABLISHED` shortly after, and `shopcaches sc shop.example.com/v1alpha1 true ShopCache`.

3. **Let the cluster document your type.**

```bash
kubectl explain shopcache.spec
kubectl explain shopcache.spec.replicas
```

`kubectl explain` reads your OpenAPI schema, so a good schema is your API documentation.

4. **Write a custom resource.**

```bash
kubectl create namespace shop
cat <<'EOF' | kubectl apply -f -
apiVersion: shop.example.com/v1alpha1
kind: ShopCache
metadata:
  name: sessions
  namespace: shop
spec:
  version: "7.2"
EOF
kubectl -n shop get shopcaches
kubectl -n shop get sc sessions -o yaml | sed -n '/^spec:/,/^status:/p'
```

The defaults appear (`replicas: 1`, `evictionPolicy: allkeys-lru`) even though you did not write them. The table shows the printer columns. Nothing runs: no controller exists yet.

5. **Trigger each validation rule.**

```bash
# out of range
kubectl -n shop patch sc sessions --type merge -p '{"spec":{"replicas":50}}'
# bad version format
kubectl -n shop patch sc sessions --type merge -p '{"spec":{"version":"latest"}}'
# CEL transition rule: immutable field
kubectl -n shop patch sc sessions --type merge -p '{"spec":
  {"evictionPolicy":"noeviction"}}'
```

Similar to `spec.replicas: Invalid value: 50: spec.replicas in body should be less than or equal to 10`, `spec.version in body should match '^[0-9]+\.[0-9]+(\.[0-9]+)?$'`, and `evictionPolicy is immutable once set`.

6. **See pruning silently remove a typo.**

```bash
cat <<'EOF' | kubectl apply -f - --validate=false
apiVersion: shop.example.com/v1alpha1
kind: ShopCache
metadata: {name: typo, namespace: shop}
spec: {version: "7.2", replicaz: 3}
EOF
kubectl -n shop get sc typo -o jsonpath='{.spec}{"\n"}'
```

`replicaz` is gone from the stored object. Without `--validate=false`, `kubectl` (strict field validation) would have warned or errored. Make strict validation your default.

7. **Use the scale and status subresources.**

```bash
kubectl -n shop scale sc sessions --replicas=3
kubectl -n shop patch sc sessions --subresource=status --type merge \
  -p '{"status":{"readyReplicas":3}}'
kubectl -n shop get sc sessions
```

`kubectl scale` works because you declared the scale subresource; the status patch simulates what a controller would do, and RBAC can allow `shopcaches/status` only to the controller.

8. **Add a second version.** Introduce `v1beta1` with a new optional field `tier`, make it the storage version and keep `v1alpha1` served and deprecated. Apply the full CRD again with both entries (shown condensed; keep the full schema from step 2 for `v1alpha1`):

```yaml
# excerpt of spec.versions after the change
versions:
  - name: v1alpha1
    served: true
    storage: false
    deprecated: true
    deprecationWarning: "shop.example.com/v1alpha1 ShopCache is deprecated; use
  v1beta1"
    # ... same schema as before ...
  - name: v1beta1
    served: true
    storage: true
    # ... same schema, plus under spec.properties:
    #   tier: {type: string, enum: [standard, premium], default: standard}
```

Because there is no `conversion` stanza the strategy is `None`, and the two versions are structurally compatible apart from the optional field. Check with `kubectl get sc sessions -o jsonpath='{.apiVersion}'` and `kubectl get crd shopcaches.shop.example.com -o jsonpath='{.status.storedVersions}'` (expect both `["v1alpha1","v1beta1"]` until you migrate stored objects). Reading through `v1alpha1` prunes `tier`; reading through `v1beta1` shows it.

9. **Validate in CI before applying.** The API server is the source of truth, so use a server-side

dry run:

```bash
kubectl apply --dry-run=server -f my-cache.yaml
```

Offline tools such as `kubeconform` can validate CRs against schemas exported from CRDs.

10. **Clean up:** `kind delete cluster --name crd-lab`. (Never delete a CRD in a real cluster casually: it deletes all its custom resources.)

7 Real-world scenario

(Illustrative.) The platform team shipped a `ShopCache` CRD where `spec.size` was a free-form string. Teams wrote `"2GB"`, `"2gb"`, `"2048"` and `"two"`. The controller parsed only the first form, and a Friday deploy of a typo'd size crash-looped the cache for the checkout flow. In the post-mortem they added a `pattern` on `size`, an enum for tiers and a CEL rule requiring `replicas ⩾ 2` for `tier: premium`, introduced `v1beta1` with the corrected field and a conversion webhook mapping the old field, then deprecated `v1alpha1` with a warning and migrated stored objects before dropping it. A server-side dry run in CI now blocks bad CRs at pull request time. **Lesson:** the schema is the first line of defence and the version plan is part of the design.

8 Common mistakes and best practices

Don't

- Leave the schema loose (`x-kubernetes-preserve-unknown-fields: true` everywhere): you lose validation, defaulting and documentation.
- Rename fields inside a version. Add a new version and convert.
- Delete or replace a CRD that has live objects without a backup.
- Put controller-written data in `spec`, or user input in `status`.
- Invent a group like `k8s.io` or `kubernetes.io`; use a domain you own.

Do

- Make every field typed and bounded, add `required`, `enum`, `pattern`, and CEL rules for cross-field logic.
- Enable the `status` subresource and report `conditions` plus `observedGeneration`.
- Version from the start, mark deprecations and publish a migration path.

- Generate CRD YAML from Go types with kubebuilder markers (`+kubebuilder:validation:Minimum=1`) so code and schema cannot drift.
- Install CRDs separately from their controller (Helm `crds/` or a dedicated chart) so upgrades and rollbacks are deliberate; GitOps tools treat CRDs as a first-class ordering concern (Argo CD and Flux §17.2).

9 Tasks

- [ ] **T1 — Define and use a CRD** *(≈30 min, beginner)*
  - **Goal:** extend the API and create an object.
  - **Steps:**
    1. Apply the CRD from step 2 and the `sessions` CR from step 4.
    2. Run `kubectl explain shopcache.spec` and `kubectl get sc -n shop`.
  - **Done when:** `kubectl get sc -n shop` shows your printer columns with the defaulted `Replicas` value.
- [ ] **T2 — Add validation** *(≈40 min, intermediate)*
  - **Goal:** reject bad input at the API.
  - **Steps:**
    1. Add `maxMemoryMB` (integer, 16-4096) to the schema and make it required.
    2. Add a CEL rule that `maxMemoryMB ⩾ 256` when `replicas > 1`.
    3. Re-apply and test one passing and two failing CRs.
  - **Done when:** the failing CRs return your messages and the passing one is stored.
- [ ] **T3 — Break it: pruning and strict validation** *(≈30 min, intermediate)*
  - **Goal:** understand silent field loss.
  - **Steps:**
    1. Apply a CR with a misspelled field using `--validate=false` and show it was pruned.
    2. Apply it again with default strict validation and capture the error or warning.
  - **Done when:** you can show both outcomes and say which `kubectl` flag or default prevents the loss.
- [ ] **T4 — Version it** *(≈60 min, advanced)*
  - **Goal:** serve two versions safely.
  - **Steps:**

1. Add `v1beta1` with the optional `tier` field, as in step 8, and make it the storage version.
2. Show `storedVersions` before and after re-saving each object (`kubectl get sc -A -o json | kubectl replace -f -`).
3. Mark `v1alpha1` deprecated and observe the warning on `kubectl apply`.

- **Done when:** `storedVersions` contains only `v1beta1` after migration and the old version still serves with a deprecation warning.

☐ **T5 — Wire the CRD to a controller and RBAC** *(≈60 min, advanced)*

- **Goal:** make the CR do something, with least privilege.
- **Steps:**
  1. Adapt `mini-operator.sh` from Operators §16.4 to read `.spec.replicas` from the `sessions` CR (instead of a ConfigMap) and create/scale a `redis` Deployment.
  2. Create a ServiceAccount with a Role allowing `get,list,watch` on `shopcaches` and `update,patch` on `shopcaches/status` only.
  3. Write `readyReplicas` back through the status subresource.
- **Done when:** `kubectl scale sc sessions --replicas=2` results in two Redis Pods and `kubectl get sc` shows `Ready 2`.

☐ **T6 — Prove it: design a CRD** *(≈60 min, advanced)*

- **Goal:** design an API without the guide.
- **Steps:**
  1. Design a `Backup` CRD for `shop-db` with `schedule` (cron), `retentionDays` (1-365) and `target` enum, a status with conditions, printer columns and one CEL rule.
  2. Apply it, create a valid and an invalid CR, and run `kubectl explain` on it.
  3. Explain in writing why deleting the CRD is dangerous and how you would version a rename.
- **Done when:** the CRD is accepted, the invalid CR is rejected with your message, and your note covers storage versions and conversion.

10 Self-check questions

1. What does a CRD give you and what does it not do by itself?
2. What is a structural schema and why is pruning dangerous for typos?
3. Which version of a multi-version CRD is stored in etcd, and how do other versions work?
4. When do you need a conversion webhook?

5. Why enable the `status` subresource?
6. What happens to custom resources if you delete the CRD?

ANSWERS

1. A new API type with schema validation, defaulting, RBAC and `kubectl` support; it does not implement behaviour. A controller is needed.
2. A schema where all fields are typed so the API server can validate, default and prune. Unknown fields are dropped, so a misspelled field disappears silently unless strict validation catches it.
3. The single version with `storage: true`; others are converted at request time (via `None` or a webhook).
4. When versions differ in more than the `apiVersion` string, for example a renamed or restructured field.
5. It separates user-writable `spec` from controller-written `status`, with separate RBAC, and prevents status writes from bumping `metadata.generation`.
6. They are all deleted.

11 Further reading

- Kubernetes documentation: *Custom Resources*, *Extend the Kubernetes API with CustomResourceDefinitions* and *Versions in CustomResourceDefinitions*.
- Kubernetes documentation: *Validation rules (CEL)* for CRDs.
- The Kubebuilder Book, section on CRD markers and multi-version APIs.
- *Kubernetes API conventions* (kubernetes/community design docs) for `spec`, `status` and conditions.
- Related: Operators §16.4, Admission Controllers and Webhooks §16.6.

LAB NOTES What I ran, what I saw, what it means

16.6 Admission Controllers and Webhooks

Topic 6 of 10 | ~6 hours | Difficulty: Advanced

1 What it is

After the API server has authenticated and authorized a request (see RBAC in Depth §16.1), it passes any request that **creates, updates or deletes** an object through **admission control**. Admission controllers can **mutate** the object (add a default, inject a sidecar) and **validate** it (reject a Pod that runs as root). RBAC answers "may this user touch Deployments?"; admission answers "is *this particular Deployment* acceptable?". RBAC cannot express "images must come from our registry" or "every namespace needs a `team` label". Admission can.

There are three ways to add your own admission logic:

1. **Dynamic admission webhooks** (`MutatingWebhookConfiguration`, `ValidatingWebhookConfiguration`) — the API server calls an HTTPS service you run, sending an `AdmissionReview` and expecting an allow/deny (and, for mutating, a JSON patch). Maximum flexibility, maximum operational risk.
2. **ValidatingAdmissionPolicy (VAP)** — validation rules written in **CEL** (Common Expression Language) and evaluated *inside* the API server, with no webhook to run. Stable (GA) since Kubernetes 1.30. A newer **MutatingAdmissionPolicy** brings the same in-process idea to mutation; it is a more recent feature, so check its status in your Kubernetes version before relying on it.
3. **Policy engines — Kyverno** and **OPA Gatekeeper** are webhook-based engines (Kyverno policies are YAML, Gatekeeper uses Rego) that add reporting, background scanning and libraries of ready-made policies.

2 Why it matters in DevOps

- **Guardrails for self-service.** When 30 teams deploy to one cluster you cannot review every manifest; admission enforces org rules (labels, registries, resource limits, no `latest`) automatically (Policy as Code §18.7).

- **Security enforcement.** Pod Security Admission is itself an admission controller (Pod Security §16.3); policy engines cover what it does not, such as signed images or required network policies.
- **It sits on the critical path.** A broken admission webhook with `failurePolicy: Fail` can stop *every* matching create/update in the cluster, including the fix. Many production incidents trace to this.
- **Consistency.** Mutating policies add the same labels, sidecars, tolerations or default `securityContext` everywhere, so developers do not have to remember.

3 Real-world system example

Istio, cert-manager and Kubernetes itself. Istio's automatic sidecar injection is a **mutating admission webhook**: when a Pod is created in a namespace labelled for injection, the webhook patches in the Envoy proxy container and init container (Service Mesh §16.9). cert-manager and the Prometheus Operator ship validating webhooks that reject malformed `Certificate` and `PrometheusRule` objects at apply time. Kubernetes' own built-ins are admission plugins too: `LimitRanger`, `ResourceQuota`, `ServiceAccount`, `PodSecurity` and `DefaultStorageClass` are compiled into the API server and are what fill in your default StorageClass or reject a Pod over quota. **Kyverno** and **Gatekeeper** are CNCF projects widely used to implement "no `latest` tag", "must have owner label" and "only approved registries" policies; both can also scan existing resources in the background and report violations, so you learn about drift as well as new requests. VAP is the in-tree option designed to cover the common validation cases without running a webhook.

4 Diagram

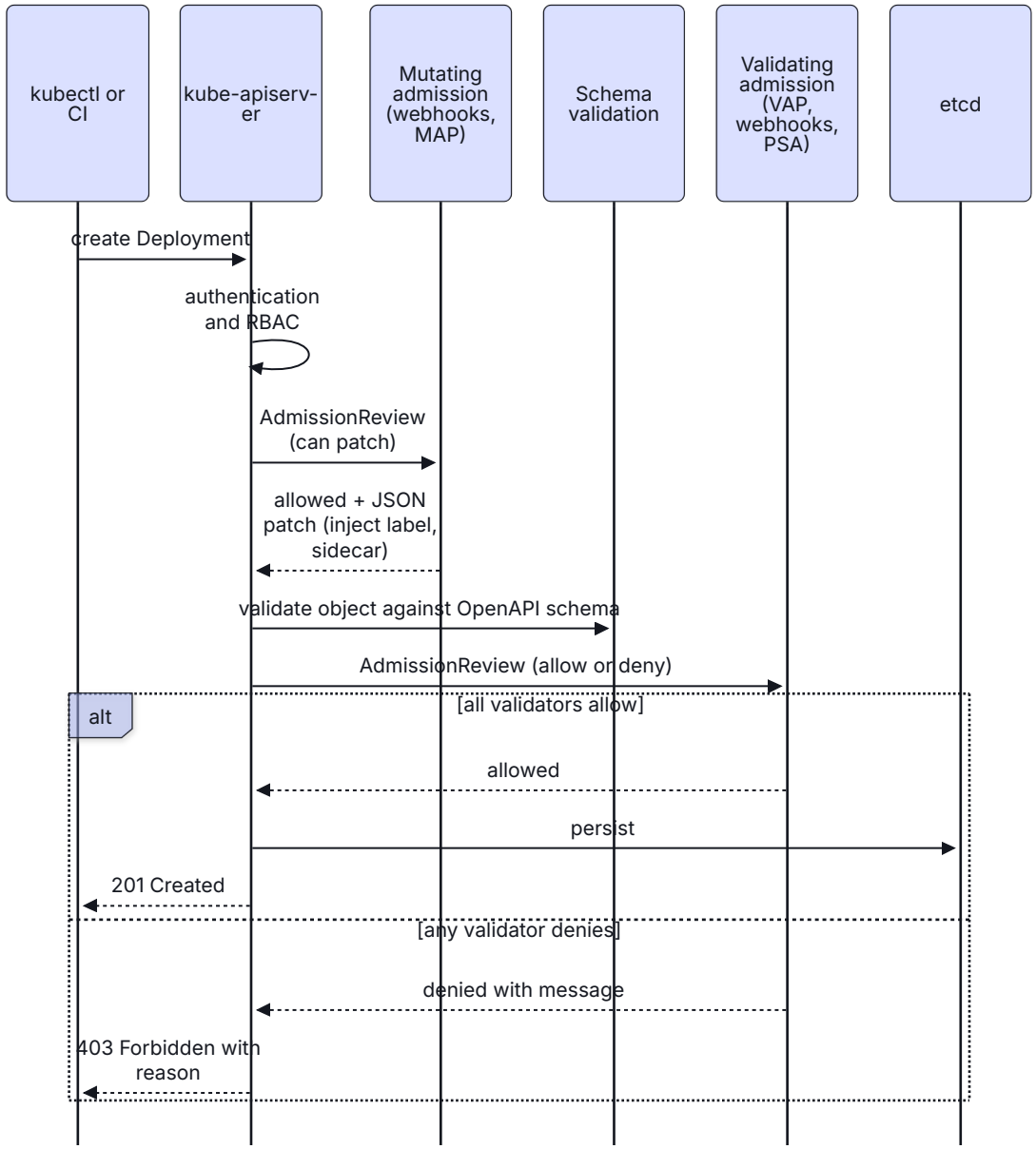


Figure 16.6-1

Choosing a tool:

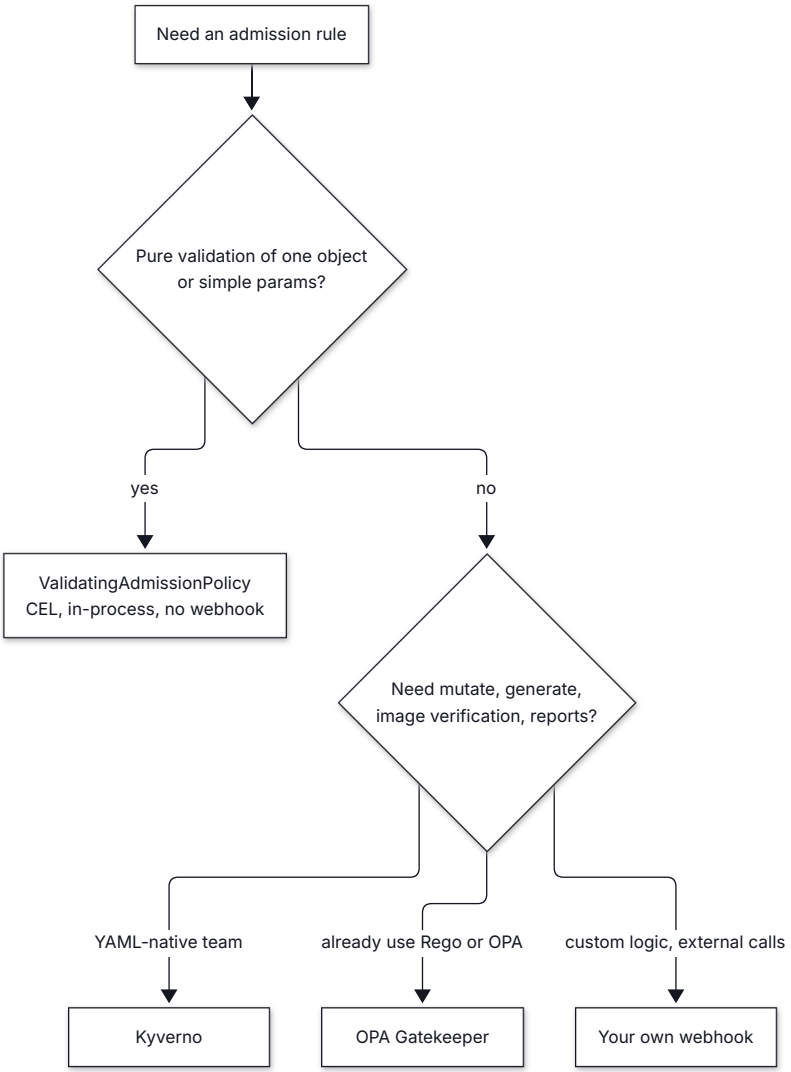


Figure 16.6-2

5 How it works

Order and rules

- Order of evaluation: **mutating** admission (in series, may be re-invoked if a later mutator changes the object), then **object schema validation**, then **validating** admission (in parallel). Any single denial rejects the request.

- Admission applies to writes (`CREATE`, `UPDATE`, `DELETE`, `CONNECT` such as `exec`), never to reads.
- Webhooks match on `rules` (apiGroups, apiVersions, resources, operations, scope), and can be narrowed with `namespaceSelector`, `objectSelector` and `matchConditions` (CEL pre-filter).

Dynamic webhook settings that decide outages

Setting	Meaning	Guidance
`failurePolicy`	`Fail` rejects requests if the webhook errors or times out; `Ignore` lets them through	`Fail` for security-critical policies, but only with an HA webhook and sane exclusions
`timeoutSeconds`	1-30 s, default 10	Keep short; each call adds latency to every matching write
`namespaceSelector`	Skip namespaces	Always exclude `kube-system` and the webhook's own namespace to avoid deadlock
`sideEffects`	`None` or `NoneOnDryRun`	Required; webhooks must be dry-run safe
`admissionReviewVersions`	e.g. `["v1"]`	Required
`reinvocationPolicy` (mutating)	`IfNeeded` re-runs after other mutators	Use when other mutators may change fields you care about
TLS `caBundle`	CA that signed the webhook's serving cert	Automate with cert-manager; expiry breaks the cluster

ValidatingAdmissionPolicy

Two objects: a **ValidatingAdmissionPolicy** (the rules, in CEL) and a **ValidatingAdmissionPolicyBinding** (where to apply it, and which actions to take). Optional **parameters** (a ConfigMap or your own CRD) let one policy serve many bindings with different values. `validationActions` can be `Deny`, `Warn` and `Audit` in any combination, so you roll out as `[Warn, Audit]` first and then `[Deny]`. CEL gives you `object`, `oldObject`, `params`, `request`, `namespaceObject` and `variables`.

Kyverno vs Gatekeeper vs VAP

	VAP (CEL)	Kyverno	OPA Gatekeeper
Runs as	In the API server	Webhook controller	Webhook controller (plus audit)
Language	CEL	YAML with patterns, JMESPath, and CEL	Rego (via ConstraintTemplates)
Validate	Yes	Yes	Yes

	VAP (CEL)	Kyverno	OPA Gatekeeper
Mutate	Newer MutatingAdmissionPolicy	Yes (patches, `generate` rules too)	Yes (mutation CRDs)
Generate resources	No	Yes (for example default NetworkPolicy per namespace)	No
Image signature verification	No	Yes (`verifyImages`)	Via external data/providers
Existing-object reports	Audit actions only	Background scan and PolicyReports	Audit controller
Extra moving part	None	Controller deployment	Controller deployment

Rule of thumb: use VAP first for plain validation; reach for Kyverno when you need mutate/generate/image verification with low learning curve; use Gatekeeper when your organization already writes Rego for OPA.

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`, `helm`. Cost: none. Outputs are illustrative. VAP needs Kubernetes 1.30 or newer (a current kind node image is fine).

1. **Create a cluster and namespaces.**

```bash
kind create cluster --name admission-lab
kubectl create namespace shop
kubectl create namespace sandbox
```

2. **Write a ValidatingAdmissionPolicy: memory limit required, no `latest` tag.**

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: workload-hygiene
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - apiGroups: ["apps"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["deployments"]
  variables:
```

```
    - name: containers
      expression: "object.spec.template.spec.containers"
  validations:
    - expression: "variables.containers.all(c, has(c.resources) &&
  has(c.resources.limits) && 'memory' in c.resources.limits)"
      message: "every container must set resources.limits.memory"
    - expression: "variables.containers.all(c, c.image.contains(':') &&
  !c.image.endsWith(':latest'))"
      message: "images must be pinned to a tag other than latest"
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata:
  name: workload-hygiene-shop
spec:
  policyName: workload-hygiene
  validationActions: [Deny]
  matchResources:
    namespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: shop
EOF
kubectl get validatingadmissionpolicy workload-hygiene -o
  jsonpath='{.status.typeChecking}{"\n"}'
```

The API server type-checks your CEL against the Deployment schema; an empty or absent `typeChecking.expressionWarnings` means no problems found.

3. **Try a bad and a good Deployment.**

```bash
kubectl -n shop create deployment bad --image=nginx:latest
kubectl -n shop create deployment good --image=nginx:1.27 --dry-run=client -o yaml \
  | kubectl set resources --local -f - --limits=memory=128Mi -o yaml \
  | kubectl apply -f -
```

The first fails, similar to: `Error from server (Invalid): deployments.apps "bad" is invalid: ... ValidatingAdmissionPolicy 'workload-hygiene' with binding 'workload-hygiene-shop' denied request: every container must set resources.limits.memory`. The second is admitted. The `sandbox` namespace is not bound, so the same bad Deployment is allowed there; that is how you scope rollouts.

4. **Add parameters and a staged rollout.** One policy, values from a ConfigMap, warn-only first:

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
```

```
metadata: {name: replica-limits, namespace: default}
data:
  maxReplicas: "5"
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata: {name: max-replicas}
spec:
  failurePolicy: Fail
  paramKind: {apiVersion: v1, kind: ConfigMap}
  matchConstraints:
    resourceRules:
      - apiGroups: ["apps"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["deployments"]
  validations:
    - expression: "object.spec.replicas <= int(params.data.maxReplicas)"
      messageExpression: "'replicas must be <= ' + params.data.maxReplicas"
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata: {name: max-replicas-shop}
spec:
  policyName: max-replicas
  validationActions: [Warn, Audit]
  paramRef:
    name: replica-limits
    namespace: default
    parameterNotFoundAction: Deny
  matchResources:
    namespaceSelector:
      matchLabels: {kubernetes.io/metadata.name: shop}
EOF
kubectl -n shop patch deployment good --type merge -p '{"spec":{"replicas":8}}'
```

(Use `patch`, not `kubectl scale`: scaling goes through the `deployments/scale` subresource, which the policy's `deployments` rule does not match.) You get `Warning: Validation failed for ValidatingAdmissionPolicy 'max-replicas' ... replicas must be <= 5` but the scale succeeds. Later change `validationActions` to `[Deny]` once the warnings are clean.

5. **Install Kyverno and write mutate and validate policies.**

```bash
helm repo add kyverno https://kyverno.github.io/kyverno/
helm repo update
helm install kyverno kyverno/kyverno -n kyverno --create-namespace
kubectl -n kyverno get pods
```

Wait until the Kyverno Pods are `Running`. Then:

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: team-label-and-defaults
spec:
  validationFailureAction: Enforce
  background: true
  rules:
    - name: require-team-label
      match:
        any:
          - resources:
              kinds: [Deployment]
              namespaces: [shop]
      validate:
        message: "label 'team' is required on Deployments in shop"
        pattern:
          metadata:
            labels:
              team: "?*"
    - name: add-managed-by
      match:
        any:
          - resources:
              kinds: [Deployment]
              namespaces: [shop]
      mutate:
        patchStrategicMerge:
          metadata:
            labels:
              +(managed-by): platform
EOF
kubectl -n shop create deployment nolabel --image=nginx:1.27
```

The create is denied with a message naming the policy and rule. *Note:* newer Kyverno releases move the enforce/audit setting to the rule's `validate.failureAction` and add CEL-based policy kinds; the older `spec.validationFailureAction` form above still works but check the Kyverno docs for your installed version.

6. **See a mutation happen.** Create a compliant Deployment and inspect it:

```bash
kubectl -n shop create deployment labelled --image=nginx:1.27 --dry-run=client -o yaml
  \
  | kubectl label --local -f - team=checkout -o yaml \
  | kubectl set resources --local -f - --limits=memory=128Mi -o yaml \
  | kubectl apply -f -
kubectl -n shop get deploy labelled --show-labels
```

   Expect `managed-by=platform` in the labels even though you never wrote it. (If the earlier VAP also applies, you need the memory limit and a pinned tag, which this command satisfies.) The `team` label on the Deployment object is what the validate rule checks; adjust the pattern to `spec.template.metadata.labels` if you want Pods labelled.

7. **Reproduce the webhook outage, safely.** Create a webhook that points at a Service that does not exist, scoped to one labelled namespace so it cannot hurt anything else:

```bash
kubectl create namespace webhook-test
kubectl label namespace webhook-test webhook-test=true
cat <<'EOF' | kubectl apply -f -
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: broken-webhook
webhooks:
  - name: broken.shop.example.com
    admissionReviewVersions: ["v1"]
    sideEffects: None
    failurePolicy: Fail
    timeoutSeconds: 5
    namespaceSelector:
      matchLabels: {webhook-test: "true"}
    rules:
      - apiGroups: [""]
        apiVersions: ["v1"]
        operations: ["CREATE"]
        resources: ["configmaps"]
    clientConfig:
      service:
        name: does-not-exist
        namespace: default
        path: /validate
EOF
kubectl -n webhook-test create configmap x --from-literal=a=b
```

Similar to: `Error from server (InternalError): Internal error occurred: failed calling webhook "broken.shop.example.com": ... service "does-not-exist" not found`. This is the shape of a real outage: nothing in that namespace can create ConfigMaps. Fix it by deleting the configuration (`kubectl delete validatingwebhookconfiguration broken-webhook`) or changing `failurePolicy` to `Ignore`.

8. **Know what the webhook receives.** An `AdmissionReview` request (abridged) and a denial response:

```json
{"apiVersion":"admission.k8s.io/v1","kind":"AdmissionReview",
 "request":{"uid":"705ab4f5-6393-11e8-b7cc-42010a800002","operation":"CREATE",
   "kind":{"group":"apps","version":"v1","kind":"Deployment"},
   "namespace":"shop","userInfo":{"username":"ci-deployer"},
   "object":{"metadata":{"name":"bad"},"spec":{"replicas":1}}}}
```

```json
{"apiVersion":"admission.k8s.io/v1","kind":"AdmissionReview",
 "response":{"uid":"705ab4f5-6393-11e8-b7cc-42010a800002","allowed":false,
   "status":{"code":403,"message":"images must come from registry.example.com"}}}
```

A mutating response adds `"patchType":"JSONPatch"` and a base64-encoded JSON Patch.

9. **(Optional) Gatekeeper.** The same "required labels" rule in Rego, shown for comparison. Install with `helm repo add gatekeeper https://open-policy-agent.github.io/gatekeeper/charts` then `helm install gatekeeper gatekeeper/gatekeeper -n gatekeeper-system --create-namespace`, and apply:

```yaml
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8srequiredlabels
spec:
  crd:
    spec:
      names:
        kind: K8sRequiredLabels
      validation:
        openAPIV3Schema:
          type: object
          properties:
            labels:
              type: array
              items:
                type: string
  targets:
```

```
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8srequiredlabels

        violation[{"msg": msg}] {
          provided := {l | input.review.object.metadata.labels[l]}
          required := {l | l := input.parameters.labels[_]}
          missing := required - provided
          count(missing) > 0
          msg := sprintf("missing required labels: %v", [missing])
        }
---
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: deployments-need-team
spec:
  enforcementAction: dryrun
  match:
    kinds:
      - apiGroups: ["apps"]
        kinds: ["Deployment"]
  parameters:
    labels: ["team"]
```

`enforcementAction: dryrun` records violations in the constraint's `status` without blocking, the Gatekeeper equivalent of `[Warn, Audit]`. Newer Gatekeeper releases accept Rego v1 syntax (`violation contains {...} if {...}`); follow the docs for your version.

10. **Clean up:** `kind delete cluster --name admission-lab`.

7 Real-world scenario

(Illustrative.) The platform team installed a policy-engine webhook with `failurePolicy: Fail` matching all Pods. During a routine node upgrade the engine's only Pod was evicted. New Pods, including the replacement engine Pod, were rejected because the webhook it needed was unreachable: a deadlock. The API server kept returning `failed calling webhook ... connection refused` and the on-call could not schedule anything until they deleted the `ValidatingWebhookConfiguration` by hand. The fixes: three replicas with a PodDisruptionBudget and topology spread; `namespaceSelector` excluding `kube-system` and the engine's own namespace; `failurePolicy: Ignore` for non-critical policies; migrating simple rules to ValidatingAdmissionPolicy (no webhook to be down); alerts on webhook latency and error rate from the API server metrics. **Lesson:** an admission webhook is a dependency of every write; design it like a control-plane component.

8 Common mistakes and best practices

Don't

- Match `*` resources in `*` namespaces with `failurePolicy: Fail`.
- Let your webhook's namespace or `kube-system` be subject to its own policies.
- Make webhooks slow or dependent on external services; timeouts block every write.
- Roll out `Deny` on day one; you will break teams' pipelines.
- Write mutations that depend on ordering of other mutators without `reinvocationPolicy`.

Do

- Prefer ValidatingAdmissionPolicy for simple validation, a policy engine for mutate/generate/verify, and your own webhook only when you must.
- Roll out as `Warn` / `Audit` (VAP) or `dryrun` /audit mode (Gatekeeper, Kyverno `Audit`), fix violations, then enforce.
- Run webhooks with 2+ replicas, PDB, resource requests, and automated certificate rotation (cert-manager).
- Test policies in CI with the engines' CLIs (Kyverno CLI, `gator` for Gatekeeper) or `kubectl apply --dry-run=server`.
- Keep policies in Git and deploy them with GitOps (Argo CD and Flux §17.2).

9 Tasks

- [ ] **T1 — First ValidatingAdmissionPolicy** *(≈40 min, intermediate)*
  - **Goal:** block bad Deployments without a webhook.
  - **Steps:**
    1. Apply the `workload-hygiene` policy and binding (step 2).
    2. Try a Deployment without limits and one using `:latest`; then a compliant one.
  - **Done when:** both bad Deployments are rejected with your messages and the good one is created.

- [ ] **T2 — Stage a rollout** *(≈30 min, intermediate)*
  - **Goal:** use `Warn` / `Audit` before `Deny`.
  - **Steps:**
    1. Change the binding's `validationActions` to `[Warn, Audit]` and create a violating Deployment.
    2. Capture the warning, then switch back to `[Deny]`.

- **Done when:** the object is admitted with a warning in warn mode and rejected in deny mode.

☐ **T3 — Parameterize** *(≈45 min, advanced)*

- **Goal:** one policy, per-environment values.
- **Steps:**
  1. Create the `max-replicas` policy (step 4) and a second binding for `sandbox` using a different ConfigMap with `maxReplicas: "2"`.
  2. Patch a Deployment's `spec.replicas` to 4 in each namespace.
- **Done when:** `sandbox` rejects (or warns on) 4 replicas while `shop` allows it.

☐ **T4 — Kyverno mutate and validate** *(≈60 min, advanced)*

- **Goal:** enforce a label and inject a default.
- **Steps:**
  1. Install Kyverno and apply `team-label-and-defaults` (step 5).
  2. Create Deployments with and without `team`, and show `managed-by=platform` appear.
  3. Add a `generate` rule that creates a default-deny `NetworkPolicy` in every new namespace (see NetworkPolicies in Depth §16.2).
- **Done when:** a new namespace automatically receives the NetworkPolicy and an unlabelled Deployment is denied.

☐ **T5 — Break it: the webhook outage** *(≈30 min, advanced)*

- **Goal:** feel and fix a `failurePolicy: Fail` outage.
- **Steps:**
  1. Reproduce step 7.
  2. Recover three ways: delete the configuration, set `failurePolicy: Ignore`, or fix the Service.
  3. Write the three-line runbook entry, including exclusion of `kube-system`.
- **Done when:** you can create a ConfigMap in `webhook-test` again and your runbook names the failure mode, the quickest unblock and the structural fix.

☐ **T6 — Prove it: choose and justify** *(≈60 min, advanced)*

- **Goal:** implement "images only from `registry.example.com`" three ways.
- **Steps:**
  1. Write it as a ValidatingAdmissionPolicy, a Kyverno policy and a Gatekeeper constraint (CLI tests or a kind cluster).

2. Compare failure modes, rollout mode, and operational cost in a short table.
3. Recommend one for your team and explain why.

- **Done when:** all three reject `docker.io/nginx:1.27` and admit `registry.example.com/shop-api:1.4.2`, and your recommendation names a failure-mode trade-off.

10 Self-check questions

1. In what order do mutating admission, schema validation and validating admission run?
2. Why is a webhook with `failurePolicy: Fail` dangerous, and how do you mitigate it?
3. What two objects make up a ValidatingAdmissionPolicy deployment, and what does each do?
4. Which `validationActions` let you trial a policy without blocking anyone?
5. When would you choose Kyverno over VAP?
6. What admission-controller work already happens on every Pod you create, without any webhook?

ANSWERS

1. Mutating, then schema validation, then validating (validators in parallel).
2. If the webhook is down or slow, matching writes are rejected, possibly including those needed to fix it. Mitigate with HA replicas and PDB, namespace exclusions, short timeouts, `Ignore` for non-critical policies, and in-process VAP where possible.
3. The ValidatingAdmissionPolicy defines the CEL rules; the Binding selects where they apply, which parameters to use and which actions (`Deny` , `Warn` , `Audit`) to take.
4. `Warn` and/or `Audit` (without `Deny`).
5. When you need mutation, resource generation, image signature verification or background reports, and prefer YAML to Rego.
6. Built-in plugins such as `ServiceAccount` , `LimitRanger` , `ResourceQuota` , `DefaultStorageClass` and `PodSecurity` .

11 Further reading

- Kubernetes documentation: *Admission Controllers Reference*, *Dynamic Admission Control* and *Validating Admission Policy*.
- Kubernetes documentation: *Common Expression Language in Kubernetes*.

- Kyverno documentation (kyverno.io) and OPA Gatekeeper documentation (open-policy-agent.github.io/gatekeeper).
- Related: Pod Security §16.3, Policy as Code §18.7, DevSecOps §0.12.

16.7 HPA, VPA and Cluster Autoscaler

Topic 7 of 10 | ~6 hours | Difficulty: Advanced

1 What it is

Kubernetes can scale at two layers, and each layer has more than one tool:

- **Pod scaling** — change how many Pods run, or how big each Pod is.
  - **Horizontal Pod Autoscaler (HPA):** adds or removes Pod *replicas* based on metrics (CPU, memory, custom or external). You met the basics in Horizontal Pod Autoscaling §8.16.
  - **Vertical Pod Autoscaler (VPA):** adjusts a Pod's CPU and memory *requests* (and optionally limits) based on observed usage.
- **Node scaling** — change how many machines the cluster has.
  - **Cluster Autoscaler (CA):** grows and shrinks pre-defined *node groups* (for example AWS Auto Scaling groups) when Pods cannot be scheduled or nodes are idle.
  - **Karpenter:** a newer node provisioner that looks at pending Pods and launches *right-sized* instances directly (no node groups), then consolidates them when load drops.

Pod scaling decides *how much work is running*; node scaling decides *where there is room to run it*. They are separate control loops that react to each other with a delay, and most autoscaling incidents live in the gap between them.

2 Why it matters in DevOps

- **Cost and reliability in one knob.** Under-scaling means latency and errors during a traffic spike; over-scaling means paying for idle nodes (Autoscaling for Cost §20.4).
- **Requests are the currency.** The scheduler, HPA (CPU utilization is a percentage *of requests*), VPA and node autoscalers all key off `resources.requests`. Wrong requests make every autoscaler wrong (Resource Requests and Limits §8.14).
- **Autoscalers fight.** HPA and VPA on the same metric, a Cluster Autoscaler blocked by a PodDisruptionBudget, or an HPA that scales up faster than nodes can arrive all produce "autoscaling is on but the site is still down".

- **Lead time.** A new node can take one to several minutes to become Ready. Spikes shorter than that need headroom, not reaction.

3 Real-world system example

Amazon EKS with Karpenter, and GKE/AKS built-in autoscaling. AWS documents Karpenter as the recommended node autoscaler for EKS in its best-practices guide, using `NodePool` and `EC2NodeClass` objects to describe what capacity is allowed (instance families, architectures, Spot vs On-Demand, CPU limits). When Pods go `Pending`, Karpenter batches them, chooses suitable instance types from the cloud API and launches nodes directly; when nodes become empty or underutilized it consolidates workloads and terminates nodes, honoring PodDisruptionBudgets and `disruption.budgets`. EKS also supports Cluster Autoscaler with managed node groups, and EKS Auto Mode is a managed offering that runs node provisioning for you. GKE offers cluster autoscaling and node auto-provisioning, plus Autopilot, and AKS ships a cluster autoscaler and a Karpenter-based "node auto-provisioning" option. Karpenter is now a Kubernetes SIG Autoscaling project with a cloud-provider interface, with AWS the most mature provider. On the pod side, VPA is a Kubernetes autoscaler subproject often run in recommendation-only mode to right-size requests, and event-driven autoscaling with **KEDA** (a CNCF project) feeds HPA with queue-length metrics from systems like Kafka and SQS (Kafka §13.8).

4 Diagram

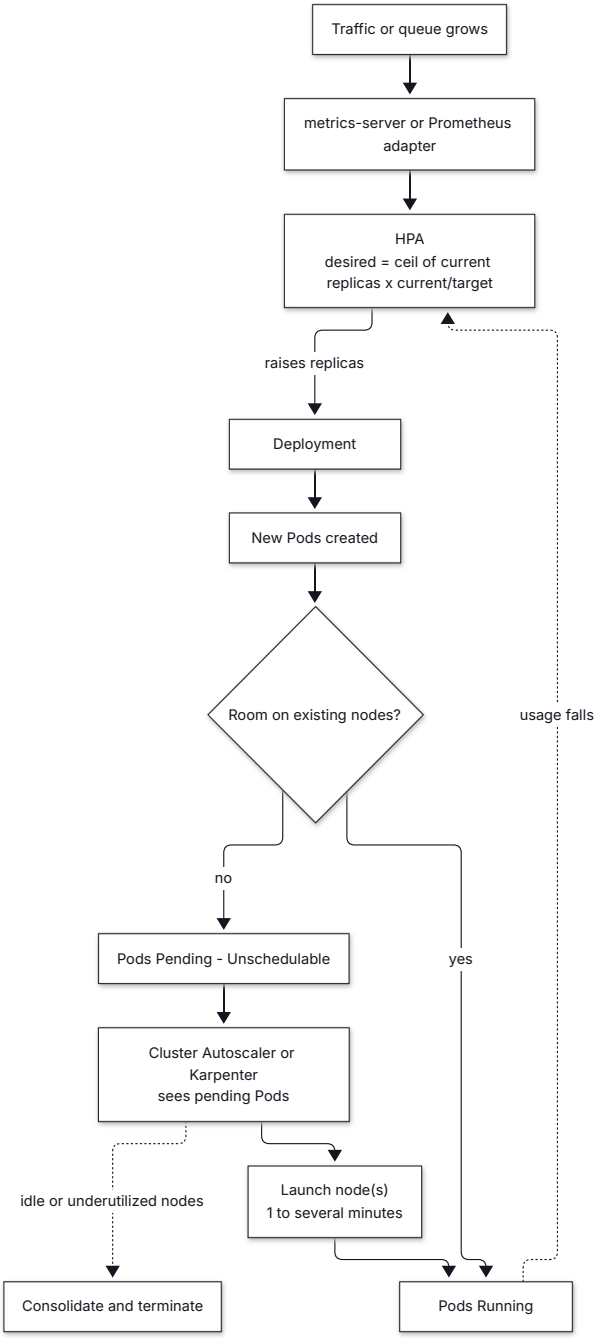


Figure 16.7-1

Why HPA and VPA on the same metric conflict:

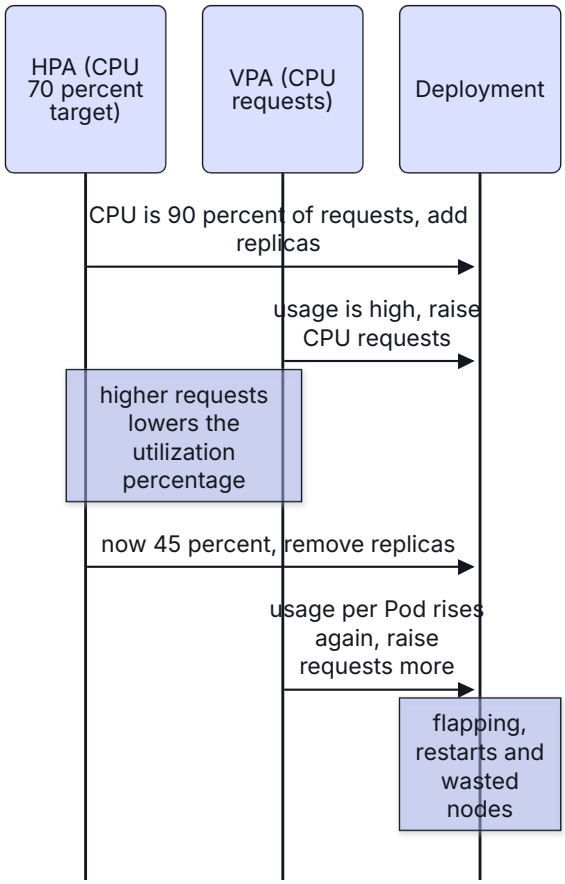


Figure 16.7-2

5 How it works

HPA (`autoscaling/v2`)

- Control loop (every 15 s by default): `desiredReplicas = ceil(currentReplicas × currentMetric / targetMetric)`, bounded by `minReplicas` and `maxReplicas`, and ignored inside a ±10% tolerance band.
- **Metric types:** `Resource` (CPU/memory from metrics-server), `ContainerResource` (per-container), `Pods` (per-Pod custom metric), `Object`, `External` (for example SQS queue depth through an adapter or KEDA).
- **Requires** `resources.requests` on the target containers for utilization-based targets, and metrics-server (or a custom metrics adapter).

- `behavior` tunes speed: `scaleUp` / `scaleDown` `policies` (for example +4 Pods or +100% per 60 s, whichever you choose with `selectPolicy`) and `stabilizationWindowSeconds` (default 300 s for scale-down, which stops flapping).
- Scaling to zero needs KEDA or the alpha `HPAScaleToZero` feature gate; stock HPA stops at 1.

VPA

Three components (Recommender, Updater, Admission Plugin). `updatePolicy.updateMode`:

Mode	Behaviour
`Off`	Only computes recommendations (read them with `kubectl describe vpa`). Safest, great for right-sizing.
`Initial`	Applies recommendations only when Pods are created
`Recreate` / `Auto`	Evicts Pods to apply new requests (Pod restart!)
`InPlaceOrRecreate` (newer versions)	Tries to resize without restart using Kubernetes in-place Pod resize, falling back to eviction; availability depends on your Kubernetes and VPA versions

Do not let HPA and VPA both act on CPU or memory for the same workload. Common pattern: HPA on CPU (or custom metrics) with VPA in `Off` mode giving advice, or VPA on memory only with HPA on a custom metric.

Cluster Autoscaler vs Karpenter

	Cluster Autoscaler	Karpenter
Unit of scaling	Node group (ASG / MIG / VMSS) you define in advance	Individual nodes chosen per pending-Pod set
Instance choice	Fixed per group (mixed instance policies help)	Picks from many types/families/zones given constraints
Scale-up trigger	Pending, unschedulable Pods	Pending, unschedulable Pods (batched)
Scale-down	Nodes under a utilization threshold (default 50%) for ~10 minutes, if Pods can move	Consolidation: delete empty nodes, replace/merge underutilized ones; `disruption.budgets` limit pace
Spot handling	Via mixed groups	First-class `capacity-type` requirement and interruption handling
Config	Flags on the autoscaler Deployment plus ASG tags	`NodePool` (what is allowed) + `EC2NodeClass` (how to build it)
Platforms	Many clouds	AWS most mature; other providers exist

Interactions and pitfalls

- **Missing or inflated requests:** no requests means HPA percentages are meaningless and nodes get overcommitted; inflated requests mean idle capacity you pay for.
- **PDBs and node drain:** a `PodDisruptionBudget` with `maxUnavailable: 0` makes a node undrainable, blocking both CA scale-down and Karpenter consolidation.
- **Scale-up latency:** node creation takes minutes. Use *overprovisioning* (low-priority "pause" Pods that reserve headroom and get preempted), or keep `minReplicas` above baseline before predictable peaks.
- **Local storage and anti-affinity:** Pods with `emptyDir`/local PVs, restrictive `topologySpreadConstraints` or hard anti-affinity can prevent bin-packing or scale-down. Annotate safe cases with `cluster-autoscaler.kubernetes.io/safe-to-evict: "true"` (CA) or `karpenter.sh/do-not-disrupt: "true"` for the opposite.
- **maxReplicas vs quotas:** `maxReplicas × requests` must fit in node limits (`NodePool.limits`, ASG max, cloud quotas), or Pods sit `Pending`.
- **Thrash:** short stabilization windows plus spiky metrics create scale-up/down loops; keep 300 s scale-down stabilization unless you have measured otherwise.

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`. Cost: none locally. Node autoscaling (CA/Karpenter) needs a cloud cluster and is shown as reviewed manifests, not run here. Outputs are illustrative.

1. **Create the cluster and install metrics-server (with the kind TLS tweak).**

```bash
kind create cluster --name scale-lab
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
  server/releases/latest/download/components.yaml
kubectl -n kube-system patch deployment metrics-server --type=json \
  -p '[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--
  kubelet-insecure-tls"}]'
kubectl -n kube-system rollout status deploy/metrics-server
kubectl top nodes
```

   `--kubelet-insecure-tls` is only for kind's self-signed kubelet certificates; never use it in production. `kubectl top nodes` prints CPU/memory after about a minute.

2. **Deploy a CPU-hungry sample app with proper requests.**

```bash
kubectl create namespace shop
```

```
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: php-apache}
spec:
  selector: {matchLabels: {app: php-apache}}
  replicas: 1
  template:
    metadata: {labels: {app: php-apache}}
    spec:
      containers:
        - name: php-apache
          image: registry.k8s.io/hpa-example
          ports: [{containerPort: 80}]
          resources:
            requests: {cpu: 200m, memory: 64Mi}
            limits: {cpu: 500m, memory: 128Mi}
---
apiVersion: v1
kind: Service
metadata: {name: php-apache}
spec:
  selector: {app: php-apache}
  ports: [{port: 80}]
EOF
```

3. **Create an HPA with a tuned `behavior`.**

BASH

```
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: php-apache}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: php-apache}
  minReplicas: 1
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 50}
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - {type: Pods, value: 4, periodSeconds: 60}
    scaleDown:
      stabilizationWindowSeconds: 120
      policies:
        - {type: Percent, value: 50, periodSeconds: 60}
```

```
EOF
kubectl -n shop get hpa php-apache
```

Expect `TARGETS cpu: 0%/50%` after a minute (if `<unknown>`, metrics-server is not ready or requests are missing).

4. **Generate load and watch it scale.** In one terminal:

```bash
kubectl -n shop run load --image=busybox:1.36 --restart=Never -- \
  /bin/sh -c "while true; do wget -q -O- http://php-apache; done"
```

In another:

```bash
kubectl -n shop get hpa php-apache -w
```

Similar to: `cpu: 250%/50%` then replicas stepping 1 → 5 → 8 within a few minutes. Compute it by hand: 1 replica at 250% of target 50% → `ceil(1 × 250/50) = 5`.

5. **Remove the load and observe slow scale-down.**

```bash
kubectl -n shop delete pod load
kubectl -n shop get hpa php-apache -w
```

CPU drops to 0% immediately but replicas shrink only after the 120 s stabilization window, in steps of at most 50% per minute.

6. **Install VPA in recommendation-only mode.** Install the components following the Kubernetes autoscaler repository's VPA documentation (it provides a `hack/vpa-up.sh`

script and a Helm chart; follow the version matching your cluster). Then:

```bash
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata: {name: php-apache}
spec:
  targetRef: {apiVersion: apps/v1, kind: Deployment, name: php-apache}
  updatePolicy: {updateMode: "Off"}
  resourcePolicy:
    containerPolicies:
      - containerName: "*"
        minAllowed: {cpu: 50m, memory: 32Mi}
        maxAllowed: {cpu: 1, memory: 512Mi}
EOF
kubectl -n shop describe vpa php-apache | sed -n '/Recommendation/,$p'
```

After a few minutes of traffic you see `Lower Bound`, `Target` and `Upper Bound` per container. Use `Target` to correct your requests. Note that setting `updateMode: "Off"` keeps HPA safe from VPA interference.

7. **Review the node-autoscaler manifests (EKS, not run here).** *Cost warning:* running nodes bills by the second; set an AWS Budget/billing alarm and delete the cluster afterwards. A Karpenter v1 setup on EKS defines allowed capacity and how nodes are built:

```yaml
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: default
spec:
  template:
    spec:
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: default
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot", "on-demand"]
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64"]
        - key: karpenter.k8s.aws/instance-category
          operator: In
          values: ["c", "m", "r"]
        - key: karpenter.k8s.aws/instance-generation
          operator: Gt
```

```
          values: ["2"]
      expireAfter: 720h
  limits:
    cpu: "100"
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 1m
    budgets:
      - nodes: "10%"
---
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass
metadata:
  name: default
spec:
  role: "KarpenterNodeRole-<CLUSTER_NAME>"
  amiSelectorTerms:
    - alias: al2023@latest
  subnetSelectorTerms:
    - tags:
        karpenter.sh/discovery: "<CLUSTER_NAME>"
  securityGroupSelectorTerms:
    - tags:
        karpenter.sh/discovery: "<CLUSTER_NAME>"
```

Read it line by line: `limits.cpu` caps total provisioned CPU (your cost fence), `capacity-type` allows Spot, `consolidationPolicy` lets Karpenter repack, `budgets` limits how many nodes are disrupted at once, and `expireAfter` rotates nodes for patching. Pin the AMI alias (for example a specific `al2023@v...` version) in production instead of `@latest`. Install steps for the controller (IAM roles, interruption queue, Helm chart) are in the Karpenter "Getting Started" guide.

8. **Add headroom with overprovisioning pods (works with CA or Karpenter).** Low-priority placeholder Pods hold space and are preempted when real Pods need it, triggering node scale-up *before* users feel it:

```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: overprovisioning
value: -10
globalDefault: false
description: placeholder pods that real workloads can preempt
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: overprovisioning
```

```
  namespace: kube-system
spec:
  replicas: 2
  selector:
    matchLabels: {app: overprovisioning}
  template:
    metadata:
      labels: {app: overprovisioning}
    spec:
      priorityClassName: overprovisioning
      containers:
        - name: pause
          image: registry.k8s.io/pause:3.9
          resources:
            requests: {cpu: "500m", memory: 512Mi}
```

9. **Clean up:** `kind delete cluster --name scale-lab`. For any cloud run: delete the workloads (so nodes drain), then the NodePools, then the cluster, and confirm no EC2 instances remain.

7 Real-world scenario

(Illustrative.) During a flash sale the `shop-api` HPA correctly scaled from 6 to 40 Pods in four minutes, but half of them sat `Pending` for another five: the Cluster Autoscaler's node group was capped at 10 nodes and the chosen instance type had no capacity in one zone. Checkout latency climbed and the team was paged. Investigation (`kubectl get pods --field-selector=status.phase=Pending`, `kubectl describe pod` showing `FailedScheduling`, and the autoscaler logs) found three issues: `maxSize` too low, a single instance type, and requests of `1 CPU` per Pod although usage was `150m` (so nodes looked full). The fix: VPA recommendations lowered requests to 250m, Karpenter replaced the fixed group with a multi-family NodePool allowing Spot and On-Demand with a CPU limit sized to budget, overprovisioning pods provided 10% headroom, and a load test before each sale verified time-to-capacity. **Lesson:** measure end-to-end time to capacity, not just whether HPA "works".

8 Common mistakes and best practices

Don't

- Run HPA on CPU while VPA also changes CPU requests for the same Deployment.
- Set `resources.requests` equal to limits "to be safe" without measuring; or omit them entirely.
- Set `maxUnavailable: 0` PDBs on everything, or `minAvailable` equal to replicas, which blocks node drains.

- Leave `maxReplicas` unbounded or `NodePool.limits` unset; a runaway loop is a surprise bill.
- Scale on a metric that does not correlate with load (CPU on an I/O-bound or queue-driven worker).

Do

- Right-size requests with VPA in `Off` mode and real load tests, then set HPA targets (50-70% CPU is typical).
- Use `behavior` to scale up fast and down slowly; keep a 5-minute scale-down window by default.
- Use custom or external metrics (requests per second, queue depth) or KEDA when CPU is a poor proxy.
- Spread across zones, keep Pods interruptible (readiness probes, graceful shutdown) when using Spot (Spot Instances §20.3), and set PDBs thoughtfully.
- Alert on `Pending` Pods, HPA `ScalingLimited` conditions and node provisioning errors; load test scale-up time.

9 Tasks

- [ ] **T1 — Scale on CPU** *(≈40 min, beginner)*
  - **Goal:** watch an HPA react to load.
  - **Steps:**
    1. Do guide steps 1-4.
    2. Record `TARGETS` and `REPLICAS` every 30 s for five minutes.
  - **Done when:** replicas rise above 1 under load and fall back after you stop it, and you can compute one of the scaling decisions by hand.
- [ ] **T2 — Tune the behavior** *(≈40 min, intermediate)*
  - **Goal:** control speed and stability.
  - **Steps:**
    1. Change `scaleDown.stabilizationWindowSeconds` to 30 and `scaleUp` to `+1 Pod per 60 s`.
    2. Repeat the load test and compare timelines with T1.
  - **Done when:** you can show the two timelines and explain which setting changed which part of the curve.
- [ ] **T3 — Right-size with VPA** *(≈60 min, intermediate)*

- **Goal:** use recommendations, not guesses.
- **Steps:**
  1. Install VPA and create the `Off` mode object (step 6).
  2. Apply steady load for 10 minutes and read `Target`.
  3. Update the Deployment's requests to the recommendation and rerun the HPA test.
- **Done when:** the HPA `TARGETS` percentage reflects realistic usage and you can state the before/after requests.

☐ **T4 — Break it: missing requests and pending Pods** *(≈40 min, advanced)*

- **Goal:** diagnose two classic autoscaling failures.
- **Steps:**
  1. Remove `resources.requests` from the Deployment and observe the HPA `TARGETS` become `<unknown>`; read `kubectl describe hpa` for the condition message.
  2. Restore requests, then set `maxReplicas: 100` with `requests.cpu: 2` and watch Pods go `Pending` on your small kind node; find the `FailedScheduling` event.
- **Done when:** you can state the exact message for each failure and the fix.

☐ **T5 — Plan a Karpenter NodePool** *(≈60 min, advanced)*

- **Goal:** design node autoscaling with a cost fence, for review rather than deployment.
- **Steps:**
  1. Adapt the step 7 manifests for `shop-api` in `us-east-1`: choose capacity types, instance categories, a CPU limit and a disruption budget, and add a second NodePool for system Pods that never uses Spot (with a taint).
  2. Write a short README: what happens on a spot interruption, on a Pod with `do-not-disrupt`, and when `limits.cpu` is hit.
  3. (Optional) try the Karpenter KWOK provider to simulate it locally, or a dev EKS cluster with a billing alarm and teardown.
- **Done when:** a reviewer can read your manifests and README and predict the behaviour in all three situations.

☐ **T6 — Prove it: end-to-end capacity test** *(≈90 min, advanced)*

- **Goal:** demonstrate and explain both scaling layers without the guide.
- **Steps:**
  1. On any cluster (kind for pods, a dev cloud cluster for nodes), create load that forces both HPA scale-up and node scale-up.
  2. Measure time from load start to all Pods Ready, and name which step dominated.

3. Explain how requests, PDBs, overprovisioning and HPA/VPA separation affect the result.

- **Done when:** you have a timeline with timestamps (load start, HPA decision, Pending, node Ready, Pods Ready) and a written explanation.

10 Self-check questions

1. Write the HPA formula and compute desired replicas for 4 Pods at 90% CPU with a 60% target.
2. Why must Pods have CPU requests for HPA utilization targets to work?
3. Why should HPA and VPA not both manage CPU for the same workload?
4. What triggers scale-up and scale-down in Cluster Autoscaler, and how does Karpenter differ?
5. How can a PodDisruptionBudget stop a node from being removed?
6. What is overprovisioning with pause Pods and what problem does it solve?

ANSWERS

1. `desired = ceil(current × currentMetric / target)` = ceil(4 × 90/60) = 6.
2. Utilization is computed as usage divided by requests; without requests there is no denominator, so the metric is `<unknown>`.
3. VPA raising requests lowers the HPA's utilization percentage, causing it to scale in, which raises per-Pod load and VPA recommendations again, so they oscillate.
4. CA scales up when Pods are unschedulable and down when nodes are underutilized, within pre-defined node groups. Karpenter provisions individual right-sized nodes directly from pending Pods and consolidates continuously, using NodePool constraints.
5. Draining evicts Pods; if evicting would violate the PDB (for example `maxUnavailable: 0`), the eviction is refused and the node cannot be removed.
6. Low-priority placeholder Pods reserve capacity; real Pods preempt them instantly while the autoscaler adds a node for the evicted placeholders, hiding node start-up latency.

11 Further reading

- Kubernetes documentation: *Horizontal Pod Autoscaling*, *Node Autoscaling* and *Resource Management for Pods and Containers*.
- Kubernetes autoscaler repository: Vertical Pod Autoscaler and Cluster Autoscaler FAQs.
- Karpenter documentation (karpenter.sh), the Amazon EKS Best Practices Guide (cluster autoscaling and cost sections), and KEDA documentation (keda.sh).

- Related: High Availability and Scalability §0.8, Auto Scaling §6.2, Kubernetes Infrastructure Costs §20.9.

LAB NOTES What I ran, what I saw, what it means

16.8 Stateful Workloads

Topic 8 of 10 | ~6 hours | Difficulty: Advanced

1 What it is

A **stateless** workload can be killed and replaced anywhere with no loss: the `shop-api` web tier. A **stateful** workload owns data that must survive restarts, rescheduling and node loss: PostgreSQL, Redis with persistence, Kafka, Elasticsearch. Kubernetes can run these, but the platform's default assumptions (Pods are disposable, interchangeable and can start anywhere) work against you until you add the right pieces:

- **Stable identity and ordered rollout:** StatefulSets §8.10 give Pods stable names (`db-0`, `db-1`), stable DNS through a headless Service, and one PersistentVolumeClaim per Pod via `volumeClaimTemplates`.
- **Durable storage:** PersistentVolumes and PVCs §8.9, provisioned dynamically through a **StorageClass** and a **CSI driver** (Container Storage Interface, the plugin API for storage).
- **Application-aware operations:** replication, leader election, failover, upgrades, backups. These are not Kubernetes features; an **operator** §16.4 or your own runbooks must supply them.

The honest question is not "can it run on Kubernetes?" but "should *we* run it, and who gets paged when it breaks?"

2 Why it matters in DevOps

- **Data loss is the one unrecoverable failure.** A deleted PVC with `reclaimPolicy: Delete`, an unverified backup, or a failover that promotes a lagging replica ends careers.
- **Storage is zonal.** An EBS volume lives in one Availability Zone; the Pod must be scheduled in the same zone, which interacts with autoscaling and node drains (HPA, VPA and Cluster Autoscaler §16.7).
- **Cost and toil trade-off.** Managed databases (RDS and Aurora §6.10) cost more per GB but remove patching, backups and failover from your plate.
- **Platform consistency.** Teams that already operate Kubernetes well often want databases in the same GitOps workflow, with the same observability and policy.

3 Real-world system example

PostgreSQL operators and Kafka on Kubernetes. Zalando open-sourced its Patroni-based `postgres-operator` and documents running it for its own applications; CloudNativePG (a CNCF project) models a Postgres cluster as a `Cluster` resource, with automated failover, rolling updates and backup to object storage with point-in-time recovery from archived WAL (write-ahead log) files; Crunchy Data's PGO and Percona's operators offer similar approaches. **Strimzi** (CNCF) runs Apache Kafka on Kubernetes with CRDs for clusters, topics and users. The Data on Kubernetes Community (DoKC) collects production reports and best practices, and the Kubernetes docs themselves include tutorials for replicated MySQL and ZooKeeper using StatefulSets. On the other side, cloud providers' own guidance generally recommends their managed database services for primary relational data unless you have a reason to self-manage. Both positions are mainstream; the right one depends on team size, compliance needs and failure tolerance.

4 Diagram

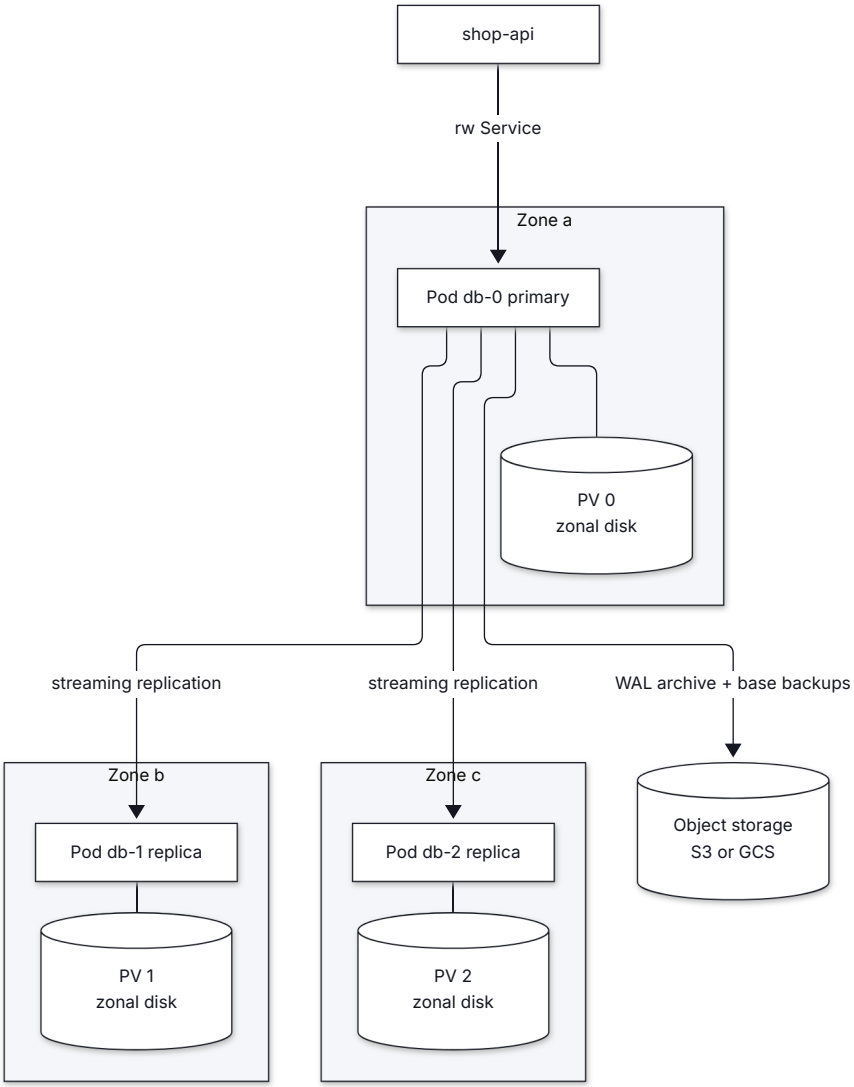


Figure 16.8-1

Failover as a state machine, driven by the operator:

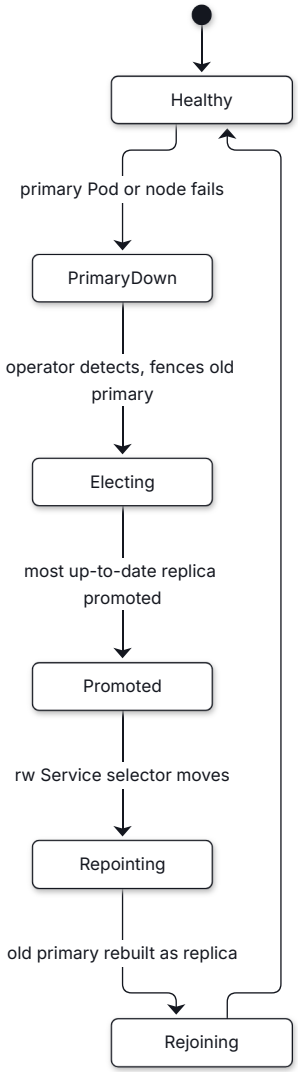


Figure 16.8-2

5 How it works

The storage chain

Object	Role	Settings that matter
StorageClass	Recipe for provisioning volumes	`provisioner` (CSI driver), `parameters` (disk type, IOPS), `reclaimPolicy` (`Delete` or `Retain`), `allowVolumeExpansion`, `volumeBindingMode`
PVC	A request for storage	access mode (`ReadWriteOnce` for a single node), size, class
PV	The actual volume	created dynamically; bound 1:1 to the PVC
VolumeSnapshot (`snapshot.storage.k8s.io`)	Point-in-time copy via the CSI driver	`VolumeSnapshotClass`; restore by creating a PVC with a `dataSource`

- Use `volumeBindingMode: WaitForFirstConsumer` so the volume is created in the zone where the Pod is scheduled, not before.
- Use `reclaimPolicy: Retain` for important data so that deleting a PVC does not delete the disk, and set `persistentVolumeClaimRetentionPolicy` on StatefulSets (`whenDeleted` / `whenScaled`: `Retain` or `Delete`) deliberately.
- Network block storage (EBS, Persistent Disk) survives node loss but has latency and IOPS limits; local NVMe is fast but is lost with the node, so replication must carry durability.

Running a database safely

- **Replication plus backups.** Replicas protect against node failure, *not* against `DROP TABLE` or corruption. Backups (base backup + continuous WAL archive for point-in-time recovery, see Backups and Point-in-Time Recovery §13.5) protect against both. **A backup is only real after a successful restore test** (Testing Backup Restoration §15.9).
- **Spread and protect.** Anti-affinity / topology spread so replicas land in different zones; PodDisruptionBudgets so drains cannot take out two members; dedicated node pools or taints so noisy neighbours do not starve the database.
- **Resources.** Set memory requests = limits for databases (avoid OOM surprises), `Guaranteed` QoS, and tune huge pages, `shared_buffers` etc. for your engine.
- **Upgrades.** Minor version upgrades via rolling restart replicas-first; major upgrades need a tested procedure (`pg_upgrade` or logical replication). Operators automate part of this; read what they do.
- **Connection management.** Put a pooler such as PgBouncer in front (Database Connection Pooling §13.6); many operators include one.

- **Velero** (and similar tools) back up Kubernetes *objects* and, with CSI snapshots or file-system backup, volume contents. It complements, not replaces, database-native backups, which are consistent at the transaction level.

When *not* to run it on Kubernetes

Situation	Better choice
Small team, no one with database operations experience	Managed service (RDS, Aurora, Cloud SQL)
Strict RPO/RTO and compliance that the provider already certifies	Managed service with multi-AZ and automated backups
Single huge database needing special hardware or kernel tuning	Dedicated VMs or bare metal
You cannot test restores and failover regularly	Do not self-run it yet
Cache or queue where loss is acceptable	Kubernetes is fine; persistence may not even be needed
Multi-tenant platform wanting self-service databases	Operator-based on Kubernetes can be a strong fit

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`, `helm`. Cost: none locally. Outputs are illustrative. kind uses a local-path provisioner whose volumes are node-local (good for learning, **not** a durability model).

1. **Create a cluster and inspect storage.**

```bash
kind create cluster --name stateful-lab
kubectl get storageclass
kubectl get storageclass standard -o jsonpath='{.provisioner}{"  "}
  {.volumeBindingMode}{"  "}{.reclaimPolicy}{"\n"}'
```

Expect a `standard (default)` class using `rancher.io/local-path`, `WaitForFirstConsumer` and `Delete`. On a real cloud you would define classes explicitly; for example:

YAML

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3-retain
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
reclaimPolicy: Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

2. **Run a persistent Redis with a StatefulSet and prove data survives a Pod restart.**

BASH

```bash
kubectl create namespace shop
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: v1
kind: Service
metadata: {name: redis}
spec:
  clusterIP: None
  selector: {app: redis}
  ports: [{port: 6379, name: redis}]
---
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: redis}
spec:
  serviceName: redis
  replicas: 1
  selector: {matchLabels: {app: redis}}
  persistentVolumeClaimRetentionPolicy:
    whenDeleted: Retain
    whenScaled: Retain
  template:
    metadata: {labels: {app: redis}}
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          args: ["--appendonly", "yes", "--dir", "/data"]
          ports: [{containerPort: 6379}]
          volumeMounts: [{name: data, mountPath: /data}]
  volumeClaimTemplates:
    - metadata: {name: data}
```

```
      spec:
        accessModes: [ReadWriteOnce]
        resources: {requests: {storage: 100Mi}}
EOF
kubectl -n shop rollout status sts/redis
kubectl -n shop exec redis-0 -- redis-cli set cart:42 "3 items"
kubectl -n shop delete pod redis-0
kubectl -n shop rollout status sts/redis
kubectl -n shop exec redis-0 -- redis-cli get cart:42
```

The last command prints `"3 items"`: the new Pod `redis-0` re-attached the same PVC `data-redis-0`. See it with `kubectl -n shop get pvc,pv`.

3. **Prove the PVC outlives the StatefulSet (retention policy).**

```bash
kubectl -n shop delete sts redis
kubectl -n shop get pvc            # data-redis-0 is still there
```

Re-applying the StatefulSet would re-attach it. Delete the PVC only when you mean to destroy the data.

4. **Install a Postgres operator and create a 3-instance cluster.**

```bash
helm repo add cnpg https://cloudnative-pg.github.io/charts
helm repo update
helm upgrade --install cnpg cnpg/cloudnative-pg -n cnpg-system --create-namespace
kubectl -n cnpg-system rollout status deploy -l app.kubernetes.io/name=cloudnative-pg
cat <<'EOF' | kubectl apply -f -
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: shop-db
  namespace: shop
spec:
  instances: 3
  storage:
    size: 1Gi
  resources:
    requests: {cpu: 250m, memory: 512Mi}
    limits: {memory: 512Mi}
EOF
kubectl -n shop get cluster shop-db -w
```

Wait for `Cluster in healthy state`. Services created: `shop-db-rw` (primary), `shop-db-ro` (replicas), `shop-db-r` (any).

5. **Write data, then fail the primary while writing.**

```bash
P=$(kubectl -n shop get cluster shop-db -o jsonpath='{.status.currentPrimary}')
kubectl -n shop exec "$P" -c postgres -- psql -U postgres -d app -c \
  "create table if not exists orders(id serial primary key, created timestamptz
  default now()); insert into orders default values;"
kubectl -n shop delete pod "$P" --wait=false
kubectl -n shop get cluster shop-db -w
```

   A replica is promoted. After the cluster is healthy again:

```bash
P2=$(kubectl -n shop get cluster shop-db -o jsonpath='{.status.currentPrimary}'); echo
  "new primary: $P2"
kubectl -n shop exec "$P2" -c postgres -- psql -U postgres -d app -c "select count(*)
  from orders;"
```

   The row survived. Applications connected through `shop-db-rw` reconnect to the new primary after a short interruption, so client retry logic matters (Idempotency §13.14).

6. **Take a logical backup and practice a restore.** This works anywhere and is independent of the operator's own backup mechanism:

```bash
kubectl -n shop exec "$P2" -c postgres -- pg_dump -U postgres -d app -Fc -f
  /tmp/app.dump
kubectl -n shop cp "$P2":/tmp/app.dump ./app.dump -c postgres
ls -lh app.dump
# restore into a scratch database
kubectl -n shop exec "$P2" -c postgres -- psql -U postgres -c "create database
  app_restore;"
kubectl -n shop exec "$P2" -c postgres -- pg_restore -U postgres -d app_restore
  /tmp/app.dump
kubectl -n shop exec "$P2" -c postgres -- psql -U postgres -d app_restore -c "select
  count(*) from orders;"
```

   The count should match. If `kubectl cp` complains that `tar` is missing, run the dump to stdout instead: `kubectl exec ... -- pg_dump -U postgres -d app -Fc > app.dump`.

7. **Review production backup configuration (object storage + PITR).** Shape of an operator-managed backup to S3-compatible storage (field names vary by operator version; recent CloudNativePG releases are moving object-store backups to a Barman Cloud plugin, so

follow the docs for your version):

```yaml
# illustrative shape, not applied in this lab
apiVersion: postgresql.cnpg.io/v1
kind: ScheduledBackup
metadata: {name: shop-db-nightly, namespace: shop}
spec:
  schedule: "0 0 2 * * *"      # six fields in CNPG: second minute hour day month weekday
  cluster: {name: shop-db}
  backupOwnerReference: self
```

The pieces every solution needs: credentials for the bucket (IAM role via IRSA / Pod Identity on EKS, not static keys), continuous WAL archiving, scheduled base backups, a retention policy, and a **recovery** definition (`bootstrap: recovery` with a target time) that you have actually exercised.

8. **Add scheduling protections to anything you self-run.** Spread members across zones and cap voluntary disruption (operators usually do this for you; plain StatefulSets need it written):

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: redis, namespace: shop}
spec:
  maxUnavailable: 1
  selector: {matchLabels: {app: redis}}
```

and in the Pod template: `topologySpreadConstraints: [{maxSkew: 1, topologyKey: topology.kubernetes.io/zone, whenUnsatisfiable: DoNotSchedule, labelSelector: {matchLabels: {app: redis}}}]`.

9. **Clean up:** `rm -f app.dump; kind delete cluster --name stateful-lab`. On a cloud cluster, retained PVs/volumes keep billing; delete the underlying disks and snapshots explicitly.

7 Real-world scenario

(Illustrative.) A team ran a three-node Postgres StatefulSet on a hand-written Helm chart, with replicas in two zones and nightly `pg_dump` backups they had never restored. A developer ran a migration containing `DELETE FROM payments;` without a `WHERE` clause. Replication faithfully copied the delete to both replicas within a second. The latest dump was 20 hours

old and, when restored for the first time, took 3 hours and was missing a table added after the dump's schema list was written. The postmortem produced: continuous WAL archiving with point-in-time recovery to just before the delete, a monthly restore drill in an isolated namespace measured against an RTO, migrations reviewed and run through a role without `DELETE` on critical tables, and a decision memo that, for a team of four, moving to a managed database with PITR was the better trade. **Lesson:** replication is availability, not backup; and whether to self-host is a staffing question as much as a technical one.

8 Common mistakes and best practices

Don't

- Treat replicas as backups.
- Use `reclaimPolicy: Delete` and `whenDeleted: Delete` on irreplaceable data without a tested restore path.
- Use `emptyDir` or node-local storage for data you cannot rebuild.
- Run a single-replica "production" database with no PDB and call it highly available.
- Upgrade the operator and the database engine at the same time.

Do

- Prefer an operator with a track record (and read its failure modes) over hand-rolled StatefulSets for databases.
- Archive WAL and take base backups to object storage in another account or region (Off-site Backups §15.8).
- Practice restore and failover on a schedule; measure RPO and RTO (RPO §15.6, RTO §15.7).
- Use `WaitForFirstConsumer`, zone spread, PDBs, dedicated node pools and Guaranteed QoS.
- Monitor replication lag, disk usage (alert before full; ensure `allowVolumeExpansion`), backup age and WAL archive failures.
- Decide explicitly, in writing, whether the database belongs on Kubernetes (Architecture Decision Records §19.11).

9 Tasks

- [ ] **T1 — Persistence across Pod death** *(≈30 min, beginner)*
  - **Goal:** prove a PVC outlives its Pod.
  - **Steps:**
    1. Run the Redis StatefulSet (step 2), write a key, delete `redis-0`.

2. Read the key again and run `kubectl get pvc,pv`.

- **Done when:** the value is still there and you can name the PVC that held it.

- [ ] **T2 — Retention and reclaim** *(≈30 min, intermediate)*
  - **Goal:** understand what really deletes data.
  - **Steps:**
    1. Delete the StatefulSet (PVC stays), then the PVC, then check the PV state.
    2. Switch the PV `persistentVolumeReclaimPolicy` to `Retain` with `kubectl patch pv ...` before deleting the PVC and repeat.
  - **Done when:** you can state, for each combination of `reclaimPolicy` and PVC deletion, whether the underlying volume survives, and demonstrate `Released` PV status.

- [ ] **T3 — Operator failover** *(≈45 min, advanced)*
  - **Goal:** observe automated failover and client impact.
  - **Steps:**
    1. Create `shop-db` (step 4).
    2. Run a loop inserting a row every second via `shop-db-rw` from a client Pod while you delete the primary.
    3. Count failed or missing inserts and the outage duration.
  - **Done when:** you have a measured outage in seconds and a statement of what the application must do (retry with backoff) to survive it.

- [ ] **T4 — Backup and restore drill** *(≈60 min, advanced)*
  - **Goal:** a restore you have actually performed.
  - **Steps:**
    1. Take a logical backup (step 6), then run `drop table orders;` in `app`.
    2. Restore from the dump into `app_restore`, and compare row counts with the dump's age.
    3. Write down the RPO this implies (time since last backup) and the measured RTO.
  - **Done when:** the table is back with correct data, and you have RPO and RTO numbers on paper.

- [ ] **T5 — Break it: drain and anti-affinity** *(≈45 min, advanced)*
  - **Goal:** see PDBs and spreading in action.
  - **Steps:**
    1. Create a kind cluster with 3 worker nodes, deploy a 3-replica StatefulSet with the step 8 PDB and zone/host spread.

2. `kubectl drain` one node, then try draining a second while the first Pod is not ready.
3. Observe the PDB refusing the second eviction.

- **Done when:** you capture the `Cannot evict pod as it would violate the pod's disruption budget` message and explain how it protects quorum.

- [ ] **T6 — Prove it: build-vs-buy decision** *(≈60 min, advanced)*
  - **Goal:** make and defend a decision for `shop-api`'s PostgreSQL.
  - **Steps:**
    1. Without the guide, write a one-page ADR comparing RDS/Aurora with CloudNativePG on Kubernetes across cost, operations, RPO/RTO, skills required and exit cost.
    2. Include the exact restore test you would run to accept either option.
    3. Present it to a teammate.
  - **Done when:** the ADR states a recommendation with the deciding factors, and the restore test is concrete enough to run.

10 Self-check questions

1. What do replicas protect against, and what do they not protect against?
2. Why use `volumeBindingMode: WaitForFirstConsumer` with zonal disks?
3. What happens to the PVCs when you delete a StatefulSet?
4. What makes a backup "real"?
5. Name three situations where you should prefer a managed database.
6. Why can a drain stall when a PodDisruptionBudget exists?

ANSWERS

1. They protect against node/Pod/zone failure (availability). They do not protect against logical errors (bad deletes, corruption, ransomware) because those replicate too.
2. It delays volume creation until the Pod is scheduled, so the disk is created in the zone where the Pod will run rather than a zone that cannot host it.
3. By default PVCs are retained (the StatefulSet's `persistentVolumeClaimRetentionPolicy` can change this); the Pods go away but the claims and data stay until you delete them.
4. A restore has been tested end-to-end, within the RTO, producing correct data as of the expected RPO.

5. Small team without DB expertise; strict compliance and SLAs the provider already meets; need for features like Aurora global databases or when failure of self-managed operations would be too costly.
6. The eviction API refuses evictions that would drop available Pods below the budget, so the drain waits or fails until enough Pods are ready elsewhere.

11 Further reading

- Kubernetes documentation: *StatefulSets*, *Storage Classes*, *Volume Snapshots* and *Run a Replicated Stateful Application*.
- CloudNativePG, Zalando postgres-operator, Strimzi and Velero project documentation.
- *Data on Kubernetes Community* (dok.community) reports and whitepapers.
- Related: Operators §16.4, Replication §13.4, Backups §15.1, Database Costs §20.8.

16.9 Service Mesh with Istio or Linkerd

Topic 9 of 10 | ~6 hours | Difficulty: Advanced

1 What it is

A **service mesh** is a dedicated infrastructure layer that handles service-to-service communication so that applications do not have to. Instead of every service re-implementing TLS, retries, timeouts, load balancing and metrics in its own language and library, the mesh puts a **proxy** next to (or near) every workload. All traffic flows through those proxies, which are configured centrally by a **control plane**.

A mesh is split into two planes:

- **Data plane:** the proxies that carry the traffic. In **Istio** classic mode each Pod gets an **Envoy** proxy as a *sidecar* container; in **Linkerd** each Pod gets `linkerd2-proxy`, a lightweight proxy written in Rust. Istio's newer **ambient mode** removes per-Pod sidecars: a per-node **ztunnel** handles identity and mTLS (L4), and optional **waypoint** proxies handle L7 features for the services that need them.
- **Control plane:** the brains. For Istio that is `istiod` (service discovery, config distribution, certificate authority); for Linkerd it is the `destination`, `identity` and `proxy-injector` components.

This topic covers the architecture, sidecar injection, and installing and onboarding an app. The next topic, Mesh traffic management §16.10, covers mTLS policy, routing, retries, timeouts, circuit breaking and observability. The general concept is in Service Mesh §12.9 and mTLS §12.12.

2 Why it matters in DevOps

- **Uniform security.** Automatic mutual TLS gives every connection encryption and a workload identity, without touching application code, a requirement in many zero-trust and compliance programs.
- **Uniform reliability and visibility.** Golden-signal metrics (rate, errors, duration), retries and timeouts apply to every service the same way, including legacy ones.

- **Progressive delivery.** Weighted routing makes canary releases a configuration change (Canary Deployments §5.12).
- **It is not free.** Proxies add CPU, memory and latency; the mesh is another critical system to upgrade and debug. Many teams are better off without one until they hit the problems it solves.

3 Real-world system example

Istio, Linkerd and their public adopters. Istio started as a joint project of Google, IBM and Lyft and is a CNCF graduated project; its data plane is Envoy, which Lyft created and also open-sourced. Linkerd was the project that coined the term "service mesh", is also a CNCF graduated project, and is built around its own Rust micro-proxy. Both projects publish public adopter lists on their websites, which is the place to check who runs them in production. Istio's ambient mode, which removes the per-Pod sidecar in favour of node-level ztunnels and optional waypoint proxies, reached general availability in the 1.24 release. Managed offerings include Google Cloud's Cloud Service Mesh (Istio-based); AWS has announced the end of support for its own App Mesh, so on EKS teams typically run open-source Istio, Linkerd or Cilium. Cloud-specific details change often, so check current documentation for your provider. On the open-source side, Cilium also offers a sidecar-free mesh built on eBPF and Envoy, and the Kubernetes **Gateway API** is becoming the common API for mesh traffic routing.

4 Diagram

Sidecar mode, with injection:

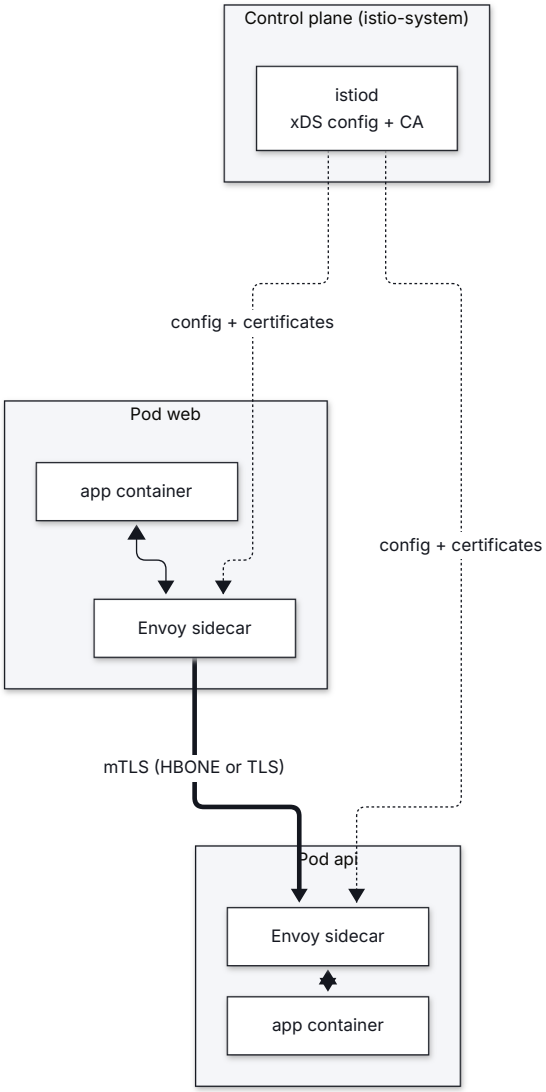


Figure 16.9-1

How a Pod gets its sidecar:

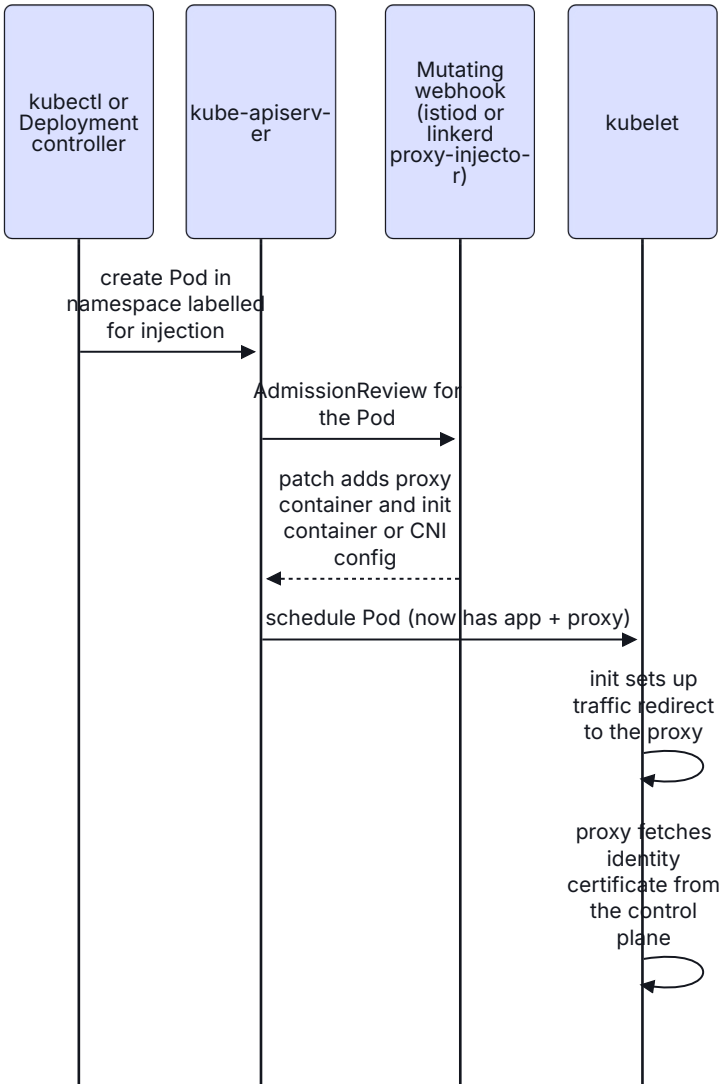


Figure 16.9-2

Ambient mode, sidecar-free:

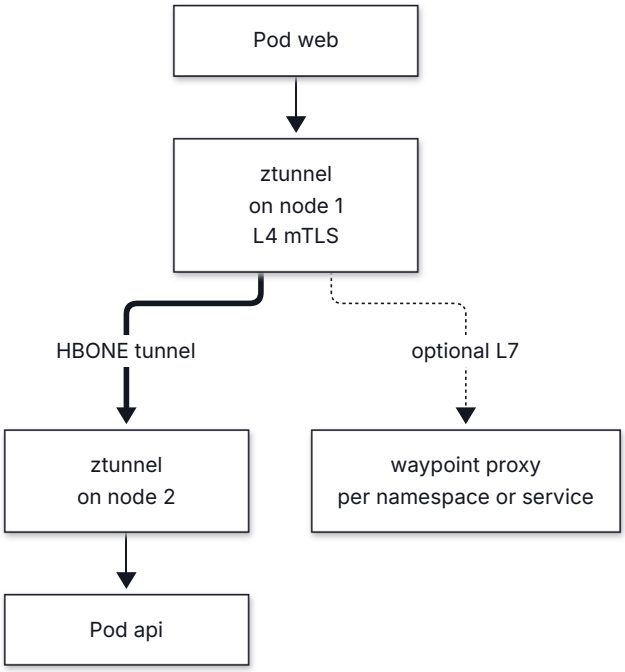


Figure 16.9-3

5 How it works

Comparison

	Istio (sidecar)	Istio (ambient)	Linkerd
Data plane	Envoy sidecar per Pod	ztunnel per node + optional waypoint (Envoy)	`linkerd2-proxy` sidecar per Pod
Opt in	Label namespace `istio-injection=enabled`	Label namespace `istio.io/dataplane-mode=ambient`	Annotate namespace/workload `linkerd.io/inject: enabled`
mTLS	Yes (permissive by default, strict by policy)	Yes, from ztunnel	Yes, on by default for meshed TCP traffic
L7 routing, retries	`VirtualService` / Gateway API `HTTPRoute`	Gateway API `HTTPRoute` via waypoint	Gateway API `HTTPRoute` and `ServiceProfile` (legacy)
Overhead	Highest, most features	Lower per-Pod, extra hop for L7	Lowest, smaller feature set

	Istio (sidecar)	Istio (ambient)	Linkerd
Learning curve	Steep	Moderate	Gentle
Ingress	Istio ingress gateway or Gateway API	same	bring your own ingress (meshed)

Licensing and release note: since early 2024 the Linkerd maintainer (Buoyant) publishes stable release artifacts through its own distribution with commercial terms for larger organizations, while open-source edge releases and the source remain available. Check the Linkerd site for the current installation path and terms before standardizing on it.

Key concepts

- **Workload identity:** each workload receives a short-lived X.509 certificate whose identity is tied to its ServiceAccount (Istio uses a SPIFFE ID such as `spiffe://cluster.local/ns/shop/sa/api`). This identity, not an IP address, is what policies match on.
- **Traffic interception:** an init container (or CNI plugin, or ambient node component) installs iptables/eBPF rules redirecting Pod traffic through the proxy. Applications keep calling `http://api` as before.
- **xDS:** the Envoy discovery APIs by which `istiod` pushes clusters, routes and endpoints to proxies.
- **Native sidecars:** Kubernetes can run sidecars as init containers with `restartPolicy: Always`, giving them proper start-up and shutdown ordering relative to the app; recent Istio and Linkerd versions can use this when the cluster supports it.
- **Protocol detection:** name Service ports with a protocol prefix (`http`, `grpc`, `tcp`) or set `appProtocol` so the proxy treats traffic correctly.
- **Upgrades:** control plane and data plane are versioned separately. Sidecars only update on Pod restart; Istio supports canary control planes using revision labels (`istio.io/rev`).

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`, about 4 CPUs and 8 GB RAM free for Istio. Cost: none. Commands follow the official Istio and Linkerd docs; outputs are illustrative. Use separate kind clusters for Istio and Linkerd so they do not interfere.

Part A: Istio (sidecar mode)

1. **Create a cluster and install Istio.**

```bash
kind create cluster --name mesh-lab
curl -L https://istio.io/downloadIstio | ISTIO_VERSION=<CURRENT_ISTIO_VERSION> sh -
export PATH="$PWD/istio-<CURRENT_ISTIO_VERSION>/bin:$PATH"
istioctl x precheck
istioctl install --set profile=default \
  --set meshConfig.accessLogFile=/dev/stdout -y
kubectl -n istio-system get pods
```

   Replace `<CURRENT_ISTIO_VERSION>` with the latest supported Istio release number from the release notes. Expect `istiod` and `istio-ingressgateway` Pods `Running`. (The `default` profile includes an ingress gateway; the `minimal` profile installs only `istiod`.)

2. **Create the `shop` namespace, enable injection and deploy the sample app.** The `http-echo` image prints a fixed text so we can later tell versions apart:

```bash
kubectl create namespace shop
kubectl label namespace shop istio-injection=enabled
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: v1
kind: ServiceAccount
metadata: {name: api}
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: api-v1}
spec:
  replicas: 2
  selector: {matchLabels: {app: api, version: v1}}
  template:
    metadata: {labels: {app: api, version: v1}}
    spec:
      serviceAccountName: api
      containers:
        - name: api
          image: hashicorp/http-echo:1.0
          args: ["-text=api v1", "-listen=:5678"]
          ports: [{containerPort: 5678}]
          resources:
            requests: {cpu: 50m, memory: 32Mi}
---
apiVersion: v1
kind: Service
metadata: {name: api}
spec:
```

```
  selector: {app: api}
  ports:
    - {name: http, port: 80, targetPort: 5678}
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: client}
spec:
  replicas: 1
  selector: {matchLabels: {app: client}}
  template:
    metadata: {labels: {app: client}}
    spec:
      containers:
        - name: client
          image: curlimages/curl:8.10.1
          command: ["sleep", "36000"]
EOF
kubectl -n shop get pods
```

Because the namespace is labelled, the mutating webhook injects a proxy: the `READY` column shows `2/2` (app plus `istio-proxy`). If you see `1/1`, you labelled after creating the Pods: run `kubectl -n shop rollout restart deploy`.

3. **Look at what injection added.**

```bash
kubectl -n shop get pod -l app=api -o jsonpath='{.items[0].spec.containers[*].name}
  {"\n"}'
kubectl -n shop get pod -l app=api -o
  jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

Expect `api istio-proxy` and `istio-init` (or `istio-proxy` in `initContainers` when native sidecars are in use; the exact layout depends on your versions).

4. **Call the service through the mesh and read the proxy's access log.**

```bash
kubectl -n shop exec deploy/client -c client -- curl -s http://api
kubectl -n shop logs deploy/client -c istio-proxy --tail=2
kubectl -n shop logs deploy/api-v1 -c istio-proxy --tail=2
```

You get `api v1`. Both proxies log the request (`"GET / HTTP/1.1" 200 ...`), proving the traffic went client proxy to server proxy.

5. **Check the mesh's own health and identity.**

```bash
istioctl proxy-status
istioctl analyze -n shop
istioctl proxy-config secret deploy/api-v1 -n shop
```

`proxy-status` lists each proxy as `SYNCED`; `analyze` should say `No validation issues found`; `proxy-config secret` shows a `default` workload certificate and the `ROOTCA`, which is the short-lived identity used for mTLS. By default Istio is in *permissive* mode: it uses mTLS between meshed workloads automatically but still accepts plain text from outside the mesh, which the next topic tightens.

6. **See that non-injected Pods are outside the mesh.**

```bash
kubectl create namespace legacy
kubectl -n legacy run curl --rm -it --restart=Never --image=curlimages/curl:8.10.1 --
  curl -s http://api.shop
```

In permissive mode this still works (plain text allowed). Keep this in mind for topic 10.

Part B: Ambient mode (optional, separate cluster)

7. **Install ambient and enrol a namespace** (on a fresh kind cluster):

```bash
istioctl install --set profile=ambient -y
kubectl -n istio-system get pods        # istiod, ztunnel (DaemonSet), istio-cni
kubectl label namespace shop istio.io/dataplane-mode=ambient
```

No Pod restart is needed and Pods stay `1/1`. mTLS between Pods in the namespace is now provided by the `ztunnel` DaemonSet. To use L7 features you add a waypoint (`istioctl waypoint apply -n shop --enroll-namespace`, see the Istio ambient docs for the current flags).

Part C: Linkerd (separate cluster)

8. **Install the Linkerd CLI and control plane** following the current official instructions at linkerd.io (the install path and the version to use depend on the release channel you

choose):

```bash
kind create cluster --name linkerd-lab
linkerd check --pre
linkerd install --crds | kubectl apply -f -
linkerd install | kubectl apply -f -
linkerd check
```

`linkerd check` prints a long list of green checks, ending in `Status check results are √`.

9. **Onboard the same app.** Annotate the namespace, deploy the manifests from step 2 (without the Istio label), restart if needed:

```bash
kubectl create namespace shop
kubectl annotate namespace shop linkerd.io/inject=enabled
# apply the same ServiceAccount/Deployments/Service from step 2 with: kubectl apply -n
  shop -f ...
kubectl -n shop get pods                        # READY 2/2 : app + linkerd-proxy
```

Install the viz extension and inspect the mesh:

```bash
linkerd viz install | kubectl apply -f -
linkerd viz check
kubectl -n shop exec deploy/client -c client -- curl -s http://api
linkerd viz -n shop stat deploy
linkerd viz -n shop edges deploy
```

`stat` prints success rate, requests per second and latency percentiles per Deployment with no instrumentation in your code; `edges` shows each connection with `SECURED` mTLS status.

10. **Clean up:** `kind delete cluster --name mesh-lab` (and `linkerd-lab`, or the ambient cluster).

7 Real-world scenario

(Illustrative.) A fintech team enabled sidecar injection on a production namespace on Monday and saw p99 latency rise 8 ms, which was acceptable, but two services began failing readiness probes after rollout. Investigation: one Job-based service finished its work, but the `istio-proxy` sidecar kept the Pod alive so the Job never completed; another service listened on a port named `db` so the mesh treated its protocol as unknown and applied HTTP routing

to a binary protocol. They fixed the first with the native sidecar mechanism (and `quitquitquit` in older setups), the second by naming the port `tcp-db`, added mesh resource requests to their capacity plans, and rolled out namespace by namespace with a rollback plan (removing the label and restarting). **Lesson:** a mesh changes Pod lifecycle, networking and capacity; onboard incrementally and watch both the app and the proxy.

8 Common mistakes and best practices

Don't

- Enable the mesh cluster-wide on day one.
- Forget that labelling a namespace does not change running Pods; they need a restart.
- Skip port naming / `appProtocol`, causing protocol mis-detection.
- Run mesh components with the default (often unbounded) resources in production, or ignore sidecar CPU and memory when sizing nodes.
- Mix control plane and data plane versions far apart.

Do

- Start with one low-risk namespace, verify `2/2` readiness, metrics and rollback, then expand.
- Pick the simplest tool that meets the need: Linkerd for mTLS and golden metrics with minimal fuss; Istio for rich traffic management, gateways and policy; ambient to reduce sidecar overhead (check current feature status for your use case).
- Use revision-based canary upgrades for Istio and test with `istioctl analyze` / `linkerd check` in CI.
- Treat the mesh as a platform component: monitor it, document ownership and runbooks.
- Combine it with NetworkPolicies (NetworkPolicies in Depth §16.2); the mesh authenticates workloads, NetworkPolicy blocks at L3/L4 even for non-mesh traffic.

9 Tasks

- [ ] **T1 — Install and onboard** *(≈60 min, intermediate)*
  - **Goal:** run an app inside the mesh.
  - **Steps:**
    1. Install Istio (step 1) and deploy `shop` (step 2).
    2. Show `2/2` Pods, `istioctl proxy-status` and `istioctl analyze`.

- **Done when:** the client reaches `api` and both proxies appear as `SYNCED` with no validation issues.

☐ **T2 — Read the access logs and the config** *(≈40 min, intermediate)*

- **Goal:** see the data plane.
- **Steps:**
  1. Send 20 requests from `client` to `api` and read the `istio-proxy` logs on both sides.
  2. Run `istioctl proxy-config clusters deploy/client -n shop | grep api` and `istioctl proxy-config endpoints deploy/client -n shop`.
- **Done when:** you can identify the client-side and server-side log lines of one request and the endpoint addresses of the `api` Pods.

☐ **T3 — Break it: forgotten restart and bad port name** *(≈40 min, advanced)*

- **Goal:** hit two real onboarding failures.
- **Steps:**
  1. Create a new Deployment in a freshly created namespace, then label the namespace afterward; show the Pod is `1/1`, then fix with a rollout restart.
  2. Rename a Service port from `http` to `db` and send HTTP traffic; look at the proxy behaviour (`istioctl proxy-config listeners`) and restore the name.
- **Done when:** you can state the cause of each symptom and the one-line fix.

☐ **T4 — Compare with Linkerd** *(≈75 min, advanced)*

- **Goal:** feel the trade-offs.
- **Steps:**
  1. Repeat the onboarding on a Linkerd cluster (Part C).
  2. Record the proxy container memory and CPU for each (`kubectl top pod --containers` with metrics-server) and the number of control-plane Pods.
  3. Fill a 5-row comparison (install steps, resource use, features you could do, debugging tools, docs).
- **Done when:** your comparison table is based on numbers you measured.

☐ **T5 — Try ambient mode** *(≈60 min, advanced)*

- **Goal:** onboard without sidecars.
- **Steps:**
  1. On a new cluster install the ambient profile and enrol `shop` by label.
  2. Show Pods are `1/1` and confirm traffic is secured by `ztunnel` (its logs, or `istioctl ztunnel-config workloads`).

3. Note which L7 feature would need a waypoint.

- **Done when:** you can explain, using your cluster output, where mTLS happens in ambient mode.

☐ **T6 — Prove it: mesh or no mesh?** *(≈45 min, advanced)*

- **Goal:** decide for `shop-api`.
- **Steps:**
  1. Without the guide, draw (by hand) the request path for `client` to `api` in sidecar, ambient and Linkerd modes.
  2. Write a half-page recommendation covering needs (mTLS, canary, observability), cost, team skills and a rollback plan.
- **Done when:** the diagrams are correct and the recommendation names at least two risks and how you would mitigate them.

10 Self-check questions

1. What are the control plane and data plane of Istio and of Linkerd?
2. How does sidecar injection work, and why do existing Pods not get a sidecar when you label the namespace?
3. What does ambient mode change, and what does a waypoint add?
4. Why must Service ports be named by protocol in Istio?
5. What is a workload identity and why is it better than IP-based policy?
6. Give two reasons not to adopt a service mesh yet.

ANSWERS

1. Istio: `istiod` and Envoy sidecars (or ztunnel/waypoints in ambient). Linkerd: `destination`, `identity` and `proxy-injector` plus `linkerd2-proxy` sidecars.
2. A mutating admission webhook patches the Pod spec at creation time; admission happens only on create/update, so running Pods must be restarted.
3. Ambient replaces per-Pod sidecars with a per-node ztunnel for L4 mTLS; waypoints add L7 routing and policy where needed.
4. So the proxy can pick correct protocol handling (HTTP vs TCP vs gRPC); wrong detection breaks routing or metrics.
5. A cryptographic identity (certificate tied to a ServiceAccount) that follows the workload; IPs change constantly in Kubernetes.

6. Operational overhead and resource cost, small number of services, no mTLS or traffic-management requirement, or no team capacity to operate it.

11 Further reading

- Istio documentation (istio.io): *Getting Started*, *Sidecar or Ambient?*, *Installation* and *Ambient Mode*.
- Linkerd documentation (linkerd.io): *Getting Started*, *Architecture* and *Automatic mTLS*.
- Envoy proxy documentation (envoyproxy.io) and the Kubernetes Gateway API (gateway-api.sigs.k8s.io).
- *Istio in Action* by Christian Posta and Rinor Maloku (Manning); *Linkerd: Up and Running* is also available.
- Related: Admission Controllers and Webhooks §16.6, Mesh traffic management §16.10.

16.10 Mesh: mTLS, Routing, Retries, Timeouts, Circuit Breaking and Observability

Topic 10 of 10 | ~7 hours | Difficulty: Advanced

1 What it is

Once workloads are in the mesh (Service Mesh with Istio or Linkerd §16.9), the proxies give you a set of policies you can apply **without changing application code**:

- **mTLS (mutual TLS):** both sides of a connection present certificates, so traffic is encrypted *and* each side knows the other's workload identity. You can then write **authorization policy** based on identity ("only `client` may call `api`").
- **Routing and traffic shifting:** send a percentage of requests, or requests with a given header, to a different version: the mechanism behind canary releases.
- **Resilience:** per-route **timeouts** and **retries**, and **circuit breaking** (limit concurrent connections/requests and eject unhealthy endpoints) so one slow dependency does not take down its callers.
- **Observability:** the proxies emit the **golden signals** (rate, errors, duration) for every service pair: you get dashboards for services that were never instrumented.

In Istio these are expressed with `PeerAuthentication`, `AuthorizationPolicy`, `VirtualService` and `DestinationRule`. In Linkerd mTLS is automatic, routing uses the Gateway API `HTTPRoute`, and retries, timeouts and failure accrual are configured with annotations and policy resources. The Kubernetes Gateway API is converging both ecosystems on one routing API.

2 Why it matters in DevOps

- **Safer releases.** Weighted routing turns "deploy and pray" into "send 5% and watch the error rate" (Canary Deployments §5.12, Deployment Strategies §0.7).
- **Containing failure.** Without timeouts, retries and circuit breakers, a slow database makes every upstream thread wait, and the failure cascades. With *bad* retry settings you can cause the outage yourself (a retry storm multiplies load) (Circuit Breakers §12.14).

- **Zero-trust networking.** Strict mTLS plus identity-based authorization replaces "anything inside the cluster is trusted" (mTLS §12.12).
- **SRE signals for free.** SLIs such as request success rate and p99 latency come straight from the mesh (SLIs §14.1, What to Monitor §10.11).

3 Real-world system example

Envoy-based meshes and progressive delivery tools. Envoy, the data-plane proxy behind Istio, was created at Lyft and exposes exactly these features (weighted clusters, retries, per-try timeouts, outlier detection, circuit-breaker thresholds) in its configuration; Istio translates `VirtualService` and `DestinationRule` objects into that configuration. Progressive-delivery controllers such as **Argo Rollouts** and **Flagger** (a CNCF project under the Flux umbrella) automate canaries on top of service meshes and the Gateway API: they shift traffic in steps, query Prometheus for mesh metrics such as success rate and latency, and roll back automatically if thresholds are breached (Canary Deployments §5.12). Linkerd's documentation describes retries and timeouts as per-route configuration plus a "retry budget" idea that bounds the extra load retries can add. The golden metrics (`istio_requests_total` , `istio_request_duration_milliseconds` for Istio; `response_total` , `response_latency_ms` for Linkerd) are the standard Prometheus series exported by these meshes.

4 Diagram

Weighted canary routing in the client's sidecar:

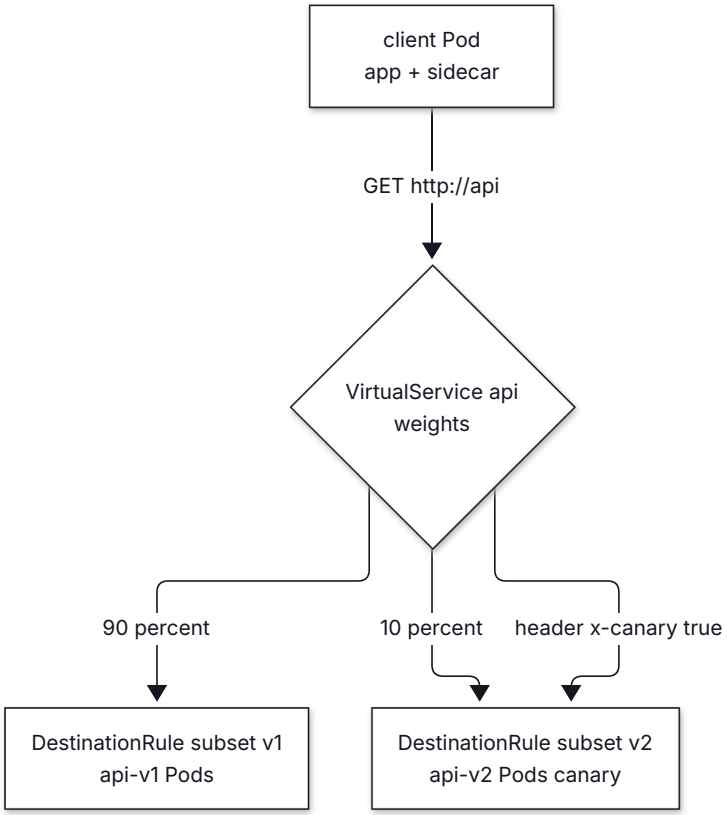


Figure 16.10-1

What retries, timeouts and ejection do on one request:

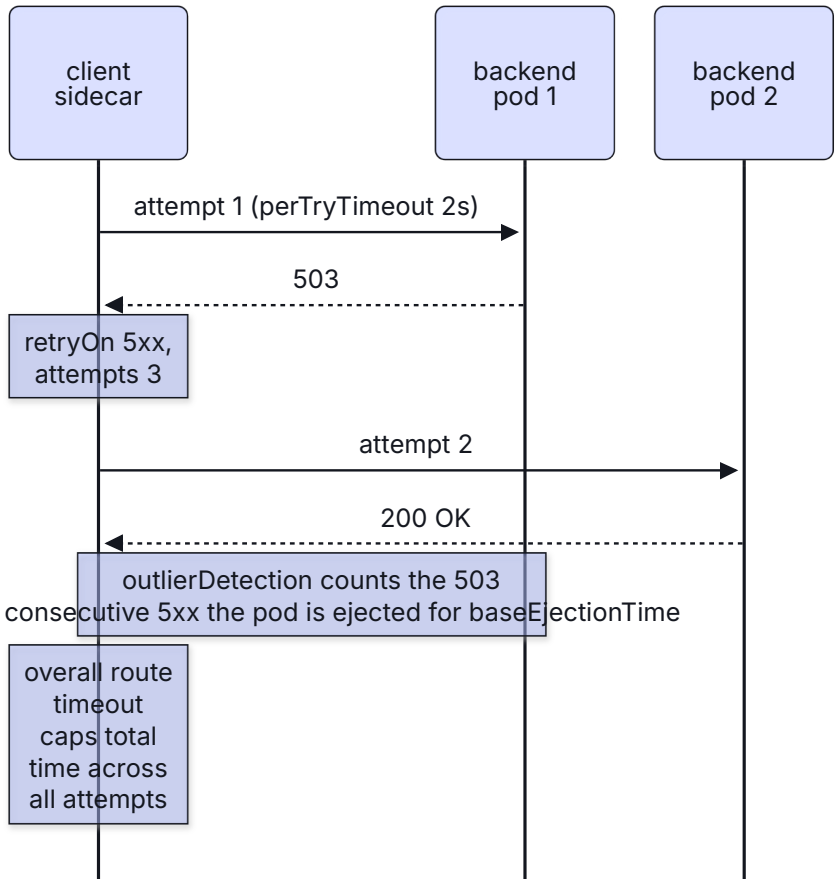


Figure 16.10-2

Circuit breaker states (Envoy outlier detection per endpoint):

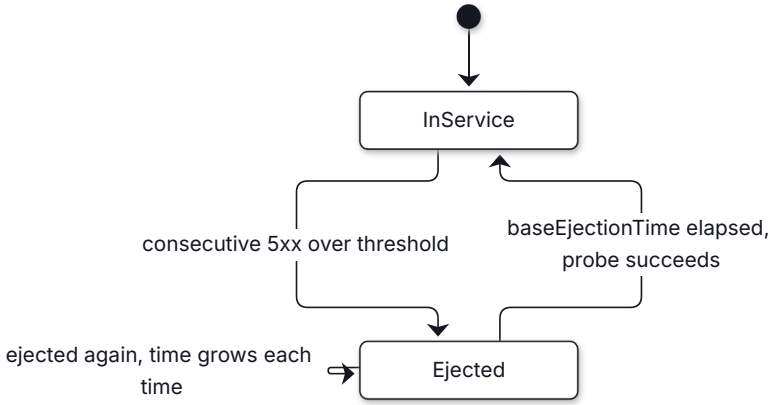


Figure 16.10-3

5 How it works

mTLS and authorization (Istio)

Resource	Purpose	Example effect
`PeerAuthentication`	Whether the workload accepts mTLS only (`STRICT`), both (`PERMISSIVE`, the default) or plain text	`STRICT` in namespace `shop`: a Pod outside the mesh gets connection reset
`AuthorizationPolicy`	Allow or deny by `principals` (SPIFFE identity), namespaces, IPs, methods, paths, headers	Only the `client` ServiceAccount may call `api`
`DestinationRule` `trafficPolicy.tls.mode`	Client-side TLS to the destination	`ISTIO_MUTUAL` (usually automatic with auto mTLS)

Rollout order for strict mTLS: install in `PERMISSIVE`, onboard all clients, check metrics to see no plain-text traffic remains (`connection_security_policy` label), then switch to `STRICT`.

Routing and shifting

- `DestinationRule` defines **subsets** (named label selectors such as `version: v1`) plus load balancing, connection pool and outlier-detection settings.
- `VirtualService` defines **routing rules**: matches (headers, paths, methods), `route` with `weight`s (must sum to 100), `timeout`, `retries`, `fault` injection, `mirror` (shadow traffic).
- A `VirtualService` hosts and rules apply in the **client's** sidecar, so the client must be in the mesh for them to take effect.

- Gateway API equivalent: an `HTTPRoute` whose `parentRefs` point at a Service (the "GAMMA" mesh pattern) and whose `backendRefs` carry weights. It is portable between Istio and Linkerd, but each version of each mesh supports a different subset; check the docs.

Resilience knobs

Concern	Istio setting	Linkerd equivalent (check docs for your version)
Timeout	`VirtualService.http[].timeout`	`timeout.linkerd.io/request` annotation or route `timeouts`
Retries	`http[].retries` (`attempts`, `perTryTimeout`, `retryOn`)	`retry.linkerd.io/http: 5xx` and `retry.linkerd.io/limit` annotations (route-level config for gRPC and HTTP)
Circuit breaking, concurrency limits	`DestinationRule.trafficPolicy.connectionPool`	Not concurrency limits; use rate limiting at the app or gateway
Ejecting bad endpoints	`trafficPolicy.outlierDetection`	Failure accrual (`balancer.linkerd.io/failure-accrual: consecutive` annotation)
Fault injection	`http[].fault` (`delay`, `abort`)	Usually done with a dedicated testing tool or an injector service

Retry guidance: retry only **idempotent** operations (Idempotency §13.14); keep `attempts` small (1-3); set `perTryTimeout` so total time (`attempts × perTryTimeout`) fits inside the route `timeout`; remember retries multiply load during an outage (retry budgets or outlier detection mitigate that).

Golden metrics from the mesh

Signal	Istio (Prometheus)	Linkerd
Rate	`istio_requests_total`	`request_total`, `linkerd viz stat` RPS column
Errors	`istio_requests_total{response_code=~"5.."}`	`response_total{classification="failure"}`
Duration	`istio_request_duration_milliseconds_bucket`	`response_latency_ms_bucket`
Saturation	Envoy stats (pending overflow), plus node/Pod CPU/memory	proxy metrics, plus cAdvisor

Use `reporter="destination"` for server-side numbers (and `source` for client-side) to avoid double counting.

6 Step-by-step guide

Prerequisites: the Istio lab from the previous topic (kind cluster `mesh-lab`, namespace `shop` with sidecar injection, `api-v1`, `api` Service and `client`), plus the `legacy` namespace. If you tore it down, redo steps 1 to 4 of that guide. Cost: none. YAML shown is for Istio's `networking.istio.io/v1` and `security.istio.io/v1` APIs (current in recent Istio; older clusters use `v1beta1`). Outputs are illustrative.

1. **Give the client its own identity.** The default ServiceAccount is shared by anything that does not set one, so use a dedicated one:

```bash
kubectl -n shop create serviceaccount client
kubectl -n shop set serviceaccount deploy/client client
kubectl -n shop rollout status deploy/client
```

2. **Turn on strict mTLS for `shop` and prove it.**

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: shop
spec:
  mtls:
    mode: STRICT
EOF
# inside the mesh: works
kubectl -n shop exec deploy/client -c client -- curl -s http://api
# outside the mesh: plain text is refused
kubectl -n legacy run probe --rm -it --restart=Never --image=curlimages/curl:8.10.1 --
  curl -sv --max-time 5 http://api.shop
```

   The first prints `api v1`. The second fails with something like `curl: (56) Recv failure: Connection reset by peer` (or `(52) Empty reply from server`). Flip `mode: PERMISSIVE` and it succeeds again, which is how you stage a migration.

3. **Restrict who may call `api` with identity.**

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: api-allow-client
```

```
  namespace: shop
spec:
  selector:
    matchLabels: {app: api}
  action: ALLOW
  rules:
    - from:
        - source:
            principals: ["cluster.local/ns/shop/sa/client"]
      to:
        - operation:
            methods: ["GET"]
EOF
kubectl -n shop exec deploy/client -c client -- curl -s -o /dev/null -w '%
  {http_code}\n' http://api
kubectl -n shop run other --rm -it --restart=Never --image=curlimages/curl:8.10.1 \
  --overrides='{"spec":{"serviceAccountName":"api"}}' -- curl -s -w '%{http_code}\n'
  http://api
```

The client gets `200`. The `other` Pod (a different identity, `api`) gets `403` with body `RBAC: access denied`. Note that the `other` Pod is injected because the namespace is labelled, so it has a proxy and an identity.

4. **Deploy `api-v2` and define subsets.**

```bash
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: api-v2}
spec:
  replicas: 1
  selector: {matchLabels: {app: api, version: v2}}
  template:
    metadata: {labels: {app: api, version: v2}}
    spec:
      serviceAccountName: api
      containers:
        - name: api
          image: hashicorp/http-echo:1.0
          args: ["-text=api v2", "-listen=:5678"]
          ports: [{containerPort: 5678}]
          resources:
            requests: {cpu: 50m, memory: 32Mi}
---
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata: {name: api}
spec:
  host: api
  subsets:
```

```
    - {name: v1, labels: {version: v1}}
    - {name: v2, labels: {version: v2}}
EOF
kubectl -n shop rollout status deploy/api-v2
```

Without a `VirtualService`, the `api` Service load-balances across v1 and v2 Pods in proportion to their numbers (2:1), which is *not* what you want for a canary.

5. **Canary: 90/10 split, then measure it.**

```
BASH
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: {name: api}
spec:
  hosts: [api]
  http:
    - match:
        - headers:
            x-canary: {exact: "true"}
      route:
        - destination: {host: api, subset: v2}
    - route:
        - destination: {host: api, subset: v1}
          weight: 90
        - destination: {host: api, subset: v2}
          weight: 10
EOF
kubectl -n shop exec deploy/client -c client -- sh -c 'for i in $(seq 1 100); do curl
  -s http://api; done' | sort | uniq -c
kubectl -n shop exec deploy/client -c client -- curl -s -H 'x-canary: true' http://api
```

Similar to `91 api v1` / `9 api v2` (it is random, so not exactly 90/10), and the header request always returns `api v2`, so testers can reach the canary deliberately. Now promote gradually by editing the weights:

```
BASH
for w in 50 100; do
  kubectl -n shop patch virtualservice api --type json -p "[
    {\"op\":\"replace\",\"path\":\"/spec/http/1/route/0/weight\",\"value\":$((100-
  w))},
    {\"op\":\"replace\",\"path\":\"/spec/http/1/route/1/weight\",\"value\":$w}]"
  kubectl -n shop exec deploy/client -c client -- sh -c 'for i in $(seq 1 100); do
  curl -s http://api; done' | sort | uniq -c
done
```

The split moves to about 50/50, then 0/100. Rollback is the same patch in reverse, taking effect in seconds without restarting any Pod. The portable Gateway API form of the same rule, using one Service per version (`api-v1` and `api-v2`), looks like this:

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: {name: api, namespace: shop}
spec:
  parentRefs:
    - {group: "", kind: Service, name: api, port: 80}
  rules:
    - backendRefs:
        - {name: api-v1, port: 80, weight: 90}
        - {name: api-v2, port: 80, weight: 10}
```

It needs the Gateway API CRDs installed and mesh support for it in your version; do not apply it together with the VirtualService for the same host.

6. **Add a flaky and slow backend for resilience experiments.**

```bash
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: httpbin}
spec:
  replicas: 2
  selector: {matchLabels: {app: httpbin}}
  template:
    metadata: {labels: {app: httpbin}}
    spec:
      containers:
        - name: httpbin
          image: kennethreitz/httpbin
          ports: [{containerPort: 80}]
          resources:
            requests: {cpu: 50m, memory: 64Mi}
---
apiVersion: v1
kind: Service
metadata: {name: httpbin}
spec:
  selector: {app: httpbin}
  ports: [{name: http, port: 80, targetPort: 80}]
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: fortio}
spec:
```

```
  replicas: 1
  selector: {matchLabels: {app: fortio}}
  template:
    metadata: {labels: {app: fortio}}
    spec:
      serviceAccountName: client
      containers:
        - name: fortio
          image: fortio/fortio
          args: ["server"]
EOF
kubectl -n shop rollout status deploy/httpbin deploy/fortio
```

`/status/200,503` returns one of the listed codes at random; `/delay/3` waits 3 seconds. (The AuthorizationPolicy from step 3 only applies to `api`, so these are unaffected.)

7. **Baseline failure rate, then add retries.**

```bash
kubectl -n shop exec deploy/client -c client -- sh -c \
  'for i in $(seq 1 60); do curl -s -o /dev/null -w "%{http_code}\n"
  http://httpbin/status/200,503; done' | sort | uniq -c
```

Expect roughly half `503`. Now:

```bash
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: {name: httpbin}
spec:
  hosts: [httpbin]
  http:
    - route:
        - destination: {host: httpbin}
      retries:
        attempts: 3
        perTryTimeout: 2s
        retryOn: 5xx
EOF
```

Re-run the loop. Similar to only a handful of `503` (about 0.5^4 of requests, roughly 6%, fail after one try plus three retries). The proxy hid most failures, which is useful for transient errors and dangerous for overloaded ones.

8. **Add a timeout.**

```bash
```

```
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: {name: httpbin}
spec:
  hosts: [httpbin]
  http:
    - route:
        - destination: {host: httpbin}
      timeout: 1s
      retries:
        attempts: 2
        perTryTimeout: 400ms
        retryOn: 5xx,reset,connect-failure
EOF
kubectl -n shop exec deploy/client -c client -- curl -s -o /dev/null -w '%{http_code}
  in %{time_total}s\n' http://httpbin/delay/3
```

Similar to `504 in 1.0s`: the caller is released after 1 second instead of waiting 3, with `upstream request timeout` as the body. Total time is bounded by the route `timeout` regardless of attempts.

9. **Circuit breaking: limit concurrency and eject bad endpoints.**

```
BASH
cat <<'EOF' | kubectl apply -n shop -f -
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata: {name: httpbin}
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      tcp: {maxConnections: 1}
      http: {http1MaxPendingRequests: 1, maxRequestsPerConnection: 1}
    outlierDetection:
      consecutive5xxErrors: 2
      interval: 5s
      baseEjectionTime: 30s
      maxEjectionPercent: 100
EOF
kubectl -n shop exec deploy/fortio -c fortio -- \
  fortio load -c 3 -qps 0 -n 30 -loglevel Warning http://httpbin/get
kubectl -n shop exec deploy/fortio -c istio-proxy -- pilot-agent request GET stats |
  grep httpbin | grep pending
```

With three concurrent connections against a limit of one, some requests are rejected: the load tool reports a share of `Code 503` and the stat `upstream_rq_pending_overflow` is non-zero. That is the circuit breaker shedding load instead of queueing it. Remove the strict

limits (or raise them) for real services based on measured capacity.

10. **Observe golden metrics.** Install the sample Prometheus add-on from the Istio release directory (it ships `samples/addons/`), generate traffic, and query:

```bash
kubectl apply -f istio-<CURRENT_ISTIO_VERSION>/samples/addons/prometheus.yaml
kubectl -n shop exec deploy/client -c client -- sh -c 'for i in $(seq 1 200); do curl
  -s -o /dev/null http://httpbin/status/200,503; done'
kubectl -n istio-system port-forward svc/prometheus 9090:9090 &
```

In the Prometheus UI at `http://localhost:9090` run:

```promql
# Rate: requests per second per destination workload
sum by (destination_workload) (rate(istio_requests_total{reporter="destination",
  destination_workload_namespace="shop"}[1m]))

# Errors: share of 5xx responses
sum by (destination_workload) (rate(istio_requests_total{reporter="destination",
  response_code=~"5.."}[1m]))
  / sum by (destination_workload) (rate(istio_requests_total{reporter="destination"}
  [1m]))

# Duration: p99 latency in ms
histogram_quantile(0.99, sum by (le, destination_workload)
  (rate(istio_request_duration_milliseconds_bucket{reporter="destination"}[1m])))
```

These are the SLIs for a canary: compare `destination_workload="api-v2"` with `api-v1`. Add `kubectl apply -f .../samples/addons/kiali.yaml` for a live service graph (`istioctl dashboard kiali`). On Linkerd the same view is `linkerd viz stat deploy -n shop` and `linkerd viz top`.

11. **Clean up:** `kill %1 2>/dev/null; kind delete cluster --name mesh-lab`.

7 Real-world scenario

(Illustrative.) A checkout service called a tax-calculation service that began to degrade: p50 latency stayed normal but 5% of calls took 30 seconds. With no timeout, checkout Pods exhausted their worker pools waiting; error rates climbed across the site. After mitigation (scaling tax replicas) the post-incident review added mesh policy: a 2-second route timeout on the tax call, one retry only for `connect-failure,reset` (the call is idempotent), and outlier detection ejecting any tax Pod with three consecutive 5xx for 30 seconds, with a connection-pool cap to shed excess load quickly. They also added a canary workflow: new tax versions receive 5% of traffic, with a Prometheus alert comparing the canary's error rate and p99 to

the stable version, and a one-line rollback (set the weight to 0). Later, a retry setting of `attempts: 5` on a different service turned a brief database slowdown into a retry storm. It was caught in staging by a fault-injection test and capped at 2 with a route timeout. **Lesson:** timeouts, retries and breakers are a set; configured together and tested with faults, they contain failure, and configured carelessly they amplify it.

8 Common mistakes and best practices

Don't

- Switch to `STRICT` before every client is meshed; you will see connection resets from jobs, health checkers and ingress.
- Retry non-idempotent calls (payment capture) or set `attempts` high; retries multiply load.
- Set the route `timeout` lower than `attempts × perTryTimeout` (retries never get to run) or leave no timeout at all.
- Forget that `VirtualService` rules run in the caller's sidecar: un-meshed callers bypass them.
- Test canaries with eyeballs; use metrics and an automated rollback criterion.
- Use `reporter` labels carelessly; mixing source and destination double counts.

Do

- Roll out mTLS as `PERMISSIVE` first, check for plain-text traffic, then `STRICT`; add identity-based `AuthorizationPolicy`, starting with a default deny per namespace once everything works.
- Keep canary weights in Git (GitOps) or drive them with Argo Rollouts / Flagger from metrics (Argo CD and Flux §17.2).
- Define timeouts for every external-facing and cross-service call; tune retries only for idempotent traffic with small budgets.
- Chaos-test: inject delay and aborts in staging to prove the settings do what you expect.
- Alert on SLIs from mesh metrics (success rate, p99), and dashboard per-version canary comparisons.
- Pair with NetworkPolicies §16.2 and Pod Security §16.3: the mesh does not protect traffic that bypasses it.

9 Tasks

- [ ] **T1 — Strict mTLS** *(≈40 min, intermediate)*

- **Goal:** prove the mesh enforces encryption.
- **Steps:**
  1. Apply the `STRICT` PeerAuthentication (step 2).
  2. Call `api` from the `legacy` namespace and from `client`.
  3. Switch to `PERMISSIVE` and call again.
- **Done when:** you have three outputs (blocked outside, allowed inside, allowed in permissive) and can explain each.

☐ **T2 — Identity-based authorization** *(≈45 min, intermediate)*

- **Goal:** allow only a named identity.
- **Steps:**
  1. Apply the `api-allow-client` policy (step 3).
  2. Show `200` for `client` and `403` for a Pod using another ServiceAccount.
  3. Add a path restriction (`paths: ["/health*"]`) and test allowed and denied paths.
- **Done when:** all four combinations behave as predicted.

☐ **T3 — Canary and header routing** *(≈45 min, intermediate)*

- **Goal:** shift traffic safely.
- **Steps:**
  1. Do step 5, measuring the split with 200 requests at 90/10, 50/50 and 0/100.
  2. Send `x-canary: true` and confirm it always reaches v2.
  3. Roll back to 100/0 and re-measure.
- **Done when:** your three measurements are near the configured weights and rollback takes effect without restarting any Pod.

☐ **T4 — Retries and timeouts, break it** *(≈60 min, advanced)*

- **Goal:** observe the benefit and the danger.
- **Steps:**
  1. Measure the failure rate of `/status/200,503` with no retries, `attempts: 3` and `attempts: 10`.
  2. Count requests received by `httpbin` Pods during each test (`kubectl logs` line counts or `istio_requests_total` at the destination).
  3. Show `/delay/3` with a `1s` timeout returning 504.
- **Done when:** you have a table of failure rate versus backend request count, showing retries amplify backend load, and a written rule for choosing `attempts`.

- [ ] **T5 — Circuit breaker and golden metrics** *(≈75 min, advanced)*
  - **Goal:** shed load and read the signals.
  - **Steps:**
    1. Apply the circuit-breaker `DestinationRule` (step 9) and run `fortio` at concurrency 3, then 1.
    2. Find `upstream_rq_pending_overflow` in the proxy stats.
    3. Install the Prometheus addon and build the rate, error and p99 queries; compare `api-v1` and `api-v2` during a 50/50 split.
  - **Done when:** overflow appears at concurrency 3 and not at 1, and your three queries return data for both versions.

- [ ] **T6 — Prove it: automated canary gate** *(≈2 hours, advanced)*
  - **Goal:** make traffic shifting depend on metrics, without the guide.
  - **Steps:**
    1. Write a script that increases `api-v2` weight 10 → 25 → 50 → 100, waiting 2 minutes per step.
    2. Before each step it queries Prometheus for the canary's 5xx ratio and p99, and aborts (weight back to 0) if the ratio exceeds 2% or p99 exceeds 1.5× the stable version's.
    3. Demonstrate both a successful promotion and an aborted one (make `api-v2` return errors, for example by pointing its Deployment at an image that returns 500).
    4. Explain how Argo Rollouts or Flagger would replace your script.
  - **Done when:** one run promotes to 100% and one run auto-rolls back to 0%, and your explanation names the metrics, thresholds and the failure it protects against.

10 Self-check questions

1. What is the difference between `PERMISSIVE` and `STRICT` PeerAuthentication, and how do you migrate safely?
2. Why is a `DestinationRule` needed for a version-based canary in Istio?
3. How can retries make an outage worse, and what settings limit that?
4. What does outlier detection do that a timeout does not?
5. Which Prometheus series give you rate, errors and duration in Istio, and what does the `reporter` label mean?
6. In which proxy are `VirtualService` rules applied, and what does that imply for clients outside the mesh?

ANSWERS

1. `PERMISSIVE` accepts both mTLS and plain text; `STRICT` accepts only mTLS. Start permissive, mesh all clients, verify no plain-text traffic remains, then switch to strict.
2. It defines the subsets (named label selectors) that the `VirtualService` weights point to.
3. Each failed request becomes several backend requests, multiplying load on an already struggling service (a retry storm). Limit `attempts`, set `perTryTimeout` and a route `timeout`, retry only idempotent and specific failure classes, and use outlier detection and connection-pool limits.
4. A timeout bounds one request's wait. Outlier detection removes consistently failing endpoints from load balancing for a period, so subsequent requests do not even try them.
5. `istio_requests_total` (rate and errors via `response_code`), `istio_request_duration_milliseconds_bucket` (duration). `reporter` says whether the metric was recorded by the source or destination proxy; filter on one to avoid double counting.
6. In the caller's sidecar (or waypoint in ambient mode). Callers outside the mesh bypass the rules and hit the Service as normal.

11 Further reading

- Istio documentation: *Traffic Management*, *Security* (PeerAuthentication, AuthorizationPolicy) and *Observability* (standard metrics).
- Linkerd documentation: *Automatic mTLS*, *Traffic Split / HTTPRoute*, *Retries and Timeouts* and *Failure accrual*.
- Kubernetes Gateway API documentation, including the GAMMA initiative for mesh routing (gateway-api.sigs.k8s.io).
- Argo Rollouts and Flagger documentation for metric-driven canaries.
- *Release It!* by Michael Nygard (timeouts, circuit breakers, bulkheads) and the Google SRE book chapters on handling overload and cascading failures.
- Related: Prometheus and PromQL §10.2, Rate Limiting §12.13.

LAB NOTES What I ran, what I saw, what it means

★ Harden a kind/EKS Cluster

Level 16 capstone ~16 hours

1 Goal and outcome

Take a plain cluster running the `shop-api` stand-in workloads and harden it the way a platform team would before letting several teams share it. At the end you will have, in one cluster you built and can tear down:

- **Least-privilege RBAC** for a CI pipeline and a controller, proven with `kubectl auth can-i` (L16.1).
- **Pod Security Standards** (`restricted`, enforced) on every application namespace (L16.3).
- **A ValidatingAdmissionPolicy** that rejects Deployments without memory limits or with `:latest` images (L16.6).
- **Default-deny NetworkPolicies** with only the flows you intend, enforced by Cilium and observed in Hubble (L16.2).
- **Autoscaling:** an HPA on the API, and a reviewed Karpenter NodePool for EKS (L16.7).
- **A stateful data tier** managed by a Postgres operator in its own locked-down namespace (L16.8, L16.4).
- **A service mesh** (Istio ambient mode) giving mTLS between workloads and a 90/10 **canary** route (L16.9, L16.10).
- **Your own CRD and a tiny controller** (`ShopRelease`) that drives the canary weight from a declarative object (L16.5, L16.4).

Success looks like: a bad Deployment is rejected, an intruder Pod cannot reach anything, the API scales under load, traffic is mTLS-encrypted, and `kubectl apply` of a `ShopRelease` shifts traffic from v1 to v2.

2 Architecture

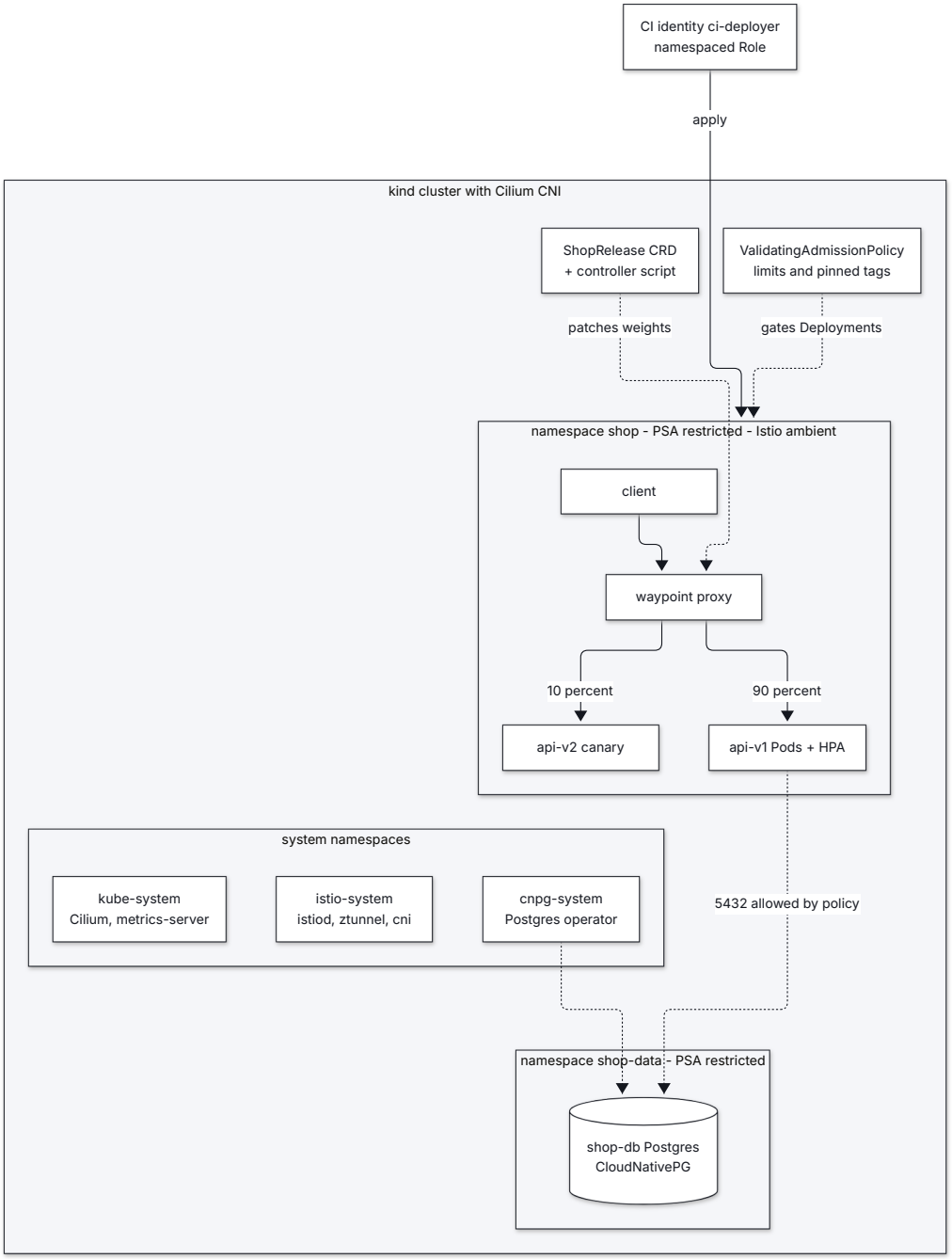


Figure 16-1

Request path and enforcement points:

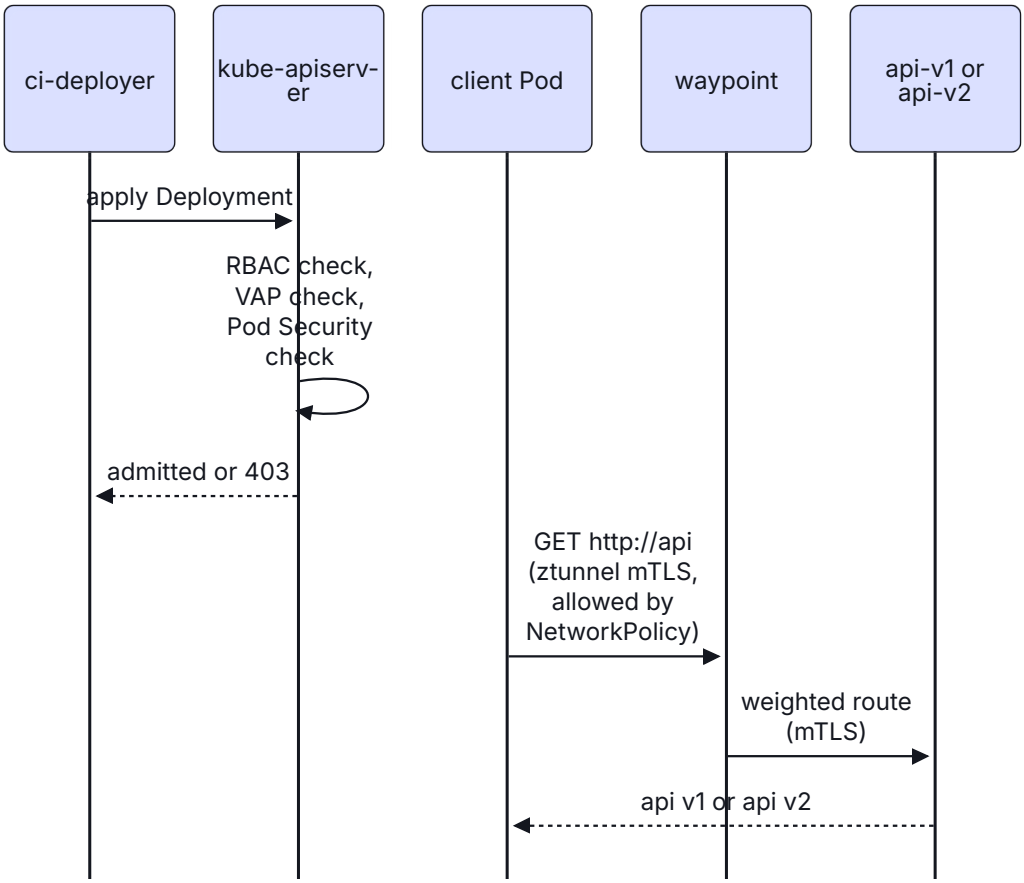


Figure 16-2

3 Prerequisites and cost

- **Machine:** Docker, `kind`, `kubectl`, `helm`, `jq`, the `cilium` CLI, `istioctl`; about 6 CPUs and 10 GB RAM free for a 3-node kind cluster running Cilium, Istio and Postgres. **Cost: none.**
- **Versions:** use current releases; pin them in your notes. The Istio and Gateway API commands below use `<ISTIO_VERSION>` and `<GATEWAY_API_VERSION>` placeholders: take compatible versions from the Istio ambient "getting started" page. Outputs shown are illustrative ("similar to").
- **EKS variant:** every milestone works on EKS with these differences: use the VPC CNI network policy support or install Cilium for NetworkPolicy; set Pod Security by namespace labels (you cannot edit the managed API server's admission config); install Karpenter

(see milestone 5) instead of relying on kind; the object-store backups for Postgres would use IRSA or Pod Identity. **Cost warning:** an EKS control plane plus nodes bills hourly; set an AWS Budget and billing alarm first, tag everything, and run the teardown section the same day.

- **Time plan:** milestones 0-3 about 4 h, 4-5 about 3 h, 6-8 about 8 h.

4 Build it step by step

Milestone 0 — Cluster, CNI, metrics

```bash
mkdir -p ~/l16-capstone && cd ~/l16-capstone
cat > kind.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
networking:
  disableDefaultCNI: true
nodes:
  - role: control-plane
  - role: worker
  - role: worker
EOF
kind create cluster --name hardened --config kind.yaml
cilium install --set cni.exclusive=false --set socketLB.hostNamespaceOnly=true
cilium status --wait
cilium hubble enable
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
  server/releases/latest/download/components.yaml
kubectl -n kube-system patch deployment metrics-server --type=json \
  -p '[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-
  insecure-tls"}]'
kubectl get nodes
```

Three `Ready` nodes. The two Cilium flags are what Istio's documentation asks for when Istio runs on a Cilium cluster (CNI chaining and socket load-balancing in the host namespace only); `--kubelet-insecure-tls` is for kind only.

Milestone 1 — Namespaces and least-privilege RBAC (L16.1)

```bash
kubectl create namespace shop
kubectl create namespace shop-data
kubectl create namespace sandbox       # a stand-in for another team's namespace
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: ServiceAccount
```

```
metadata: {name: ci-deployer, namespace: shop}
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: shop-deployer, namespace: shop}
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]
  - apiGroups: [""]
    resources: ["services", "serviceaccounts"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]
  - apiGroups: ["autoscaling"]
    resources: ["horizontalpodautoscalers"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]
  - apiGroups: ["policy"]
    resources: ["poddisruptionbudgets"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: ci-deployer, namespace: shop}
subjects: [{kind: ServiceAccount, name: ci-deployer, namespace: shop}]
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: Role, name: shop-deployer}
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: shop-devs-view, namespace: shop}
subjects: [{kind: Group, name: shop-devs, apiGroup: rbac.authorization.k8s.io}]
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: ClusterRole, name: view}
EOF
CI=system:serviceaccount:shop:ci-deployer
kubectl auth can-i create deployments -n shop --as=$CI          # yes
kubectl auth can-i get secrets        -n shop --as=$CI          # no
kubectl auth can-i create deployments -n sandbox --as=$CI       # no
kubectl auth can-i create rolebindings -n shop --as=$CI         # no
kubectl auth can-i list pods -n shop --as=anna --as-group=shop-devs   # yes (via view)
```

The pipeline can deploy only in `shop`, cannot read Secrets, cannot create RBAC objects and cannot mint tokens (no `serviceaccounts/token`).

Milestone 2 — Pod Security and an admission policy (L16.3, L16.6)

```bash
for ns in shop shop-data; do
kubectl label namespace $ns \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/enforce-version=latest \
```

```
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/audit=restricted
done
cat <<'EOF' | kubectl apply -f -
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata: {name: workload-hygiene}
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - apiGroups: ["apps"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["deployments"]
  variables:
    - name: containers
      expression: "object.spec.template.spec.containers"
  validations:
    - expression: "variables.containers.all(c, has(c.resources) && has(c.resources.limits)
  && 'memory' in c.resources.limits)"
      message: "every container must set resources.limits.memory"
    - expression: "variables.containers.all(c, c.image.contains(':') &&
  !c.image.endsWith(':latest'))"
      message: "images must be pinned to a tag other than latest"
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata: {name: workload-hygiene-shop}
spec:
  policyName: workload-hygiene
  validationActions: [Deny]
  matchResources:
    namespaceSelector:
      matchLabels: {kubernetes.io/metadata.name: shop}
    objectSelector:
      matchExpressions:
        - {key: gateway.istio.io/managed, operator: DoesNotExist}
EOF
```

The `objectSelector` exempts Deployments that Istio generates for waypoint proxies later (confirm the label with `--show-labels` in milestone 7; if your version differs, switch the binding to `[Warn, Audit]` for those). Test the guardrails as the CI identity:

```bash
CI=system:serviceaccount:shop:ci-deployer
kubectl -n shop create deployment bad --image=nginx:latest --as=$CI          # denied by
  the policy
kubectl -n shop create deployment root --image=nginx:1.27 --dry-run=client -o yaml \
  | kubectl set resources --local -f - --limits=memory=128Mi -o yaml \
  | kubectl apply --as=$CI -f -                                              # passes the
  policy (limit, pinned tag)
kubectl -n shop describe rs -l app=root | grep -i -m2 "violates PodSecurity"   # but its
  Pod is rejected
kubectl -n shop delete deployment root
```

Similar to `denied request: every container must set resources.limits.memory` and `violates PodSecurity "restricted:latest"`. The second shows that Pod Security rejects Pods, not Deployments, and the first that the admission policy rejects Deployments before they exist.

Milestone 3 — Hardened workloads (L16.3, L16.1)

Deploy `api-v1`, `api-v2` and a `client` as the CI identity so everything is checked by RBAC, the policy and PSA:

```bash
CI=system:serviceaccount:shop:ci-deployer
for sa in api client; do
cat <<EOF | kubectl apply -n shop --as=$CI -f -
apiVersion: v1
kind: ServiceAccount
metadata: {name: $sa}
automountServiceAccountToken: false
EOF
done
for v in v1 v2; do
r=2; [ $v = v2 ] && r=1
cat <<EOF | kubectl apply -n shop --as=$CI -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: api-$v}
spec:
  replicas: $r
  selector: {matchLabels: {app: api, version: $v}}
  template:
    metadata: {labels: {app: api, version: $v}}
    spec:
```

```
      serviceAccountName: api
      automountServiceAccountToken: false
      securityContext: {runAsNonRoot: true, runAsUser: 10001, runAsGroup: 10001,
  seccompProfile: {type: RuntimeDefault}}
      containers:
        - name: api
          image: hashicorp/http-echo:1.0
          args: ["-text=api $v", "-listen=:5678"]
          ports: [{containerPort: 5678}]
          securityContext: {allowPrivilegeEscalation: false, readOnlyRootFilesystem: true,
  capabilities: {drop: ["ALL"]}}
          resources: {requests: {cpu: 20m, memory: 32Mi}, limits: {memory: 64Mi}}
---
apiVersion: v1
kind: Service
metadata: {name: api-$v}
spec:
  selector: {app: api, version: $v}
  ports: [{name: http, port: 80, targetPort: 5678}]
EOF
done
cat <<'EOF' | kubectl apply -n shop --as=$CI -f -
apiVersion: v1
kind: Service
metadata: {name: api}
spec:
  selector: {app: api}
  ports: [{name: http, port: 80, targetPort: 5678}]
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: client}
spec:
  replicas: 1
  selector: {matchLabels: {app: client}}
  template:
    metadata: {labels: {app: client}}
    spec:
      serviceAccountName: client
      automountServiceAccountToken: false
      securityContext: {runAsNonRoot: true, runAsUser: 10001, runAsGroup: 10001,
  seccompProfile: {type: RuntimeDefault}}
      containers:
        - name: client
          image: curlimages/curl:8.10.1
          command: ["sleep", "36000"]
          securityContext: {allowPrivilegeEscalation: false, readOnlyRootFilesystem: true,
  capabilities: {drop: ["ALL"]}}
          resources: {requests: {cpu: 10m, memory: 16Mi}, limits: {memory: 64Mi}}
EOF
kubectl -n shop rollout status deploy/api-v1 deploy/api-v2 deploy/client
kubectl -n shop exec deploy/client -- curl -s http://api
```

Expect `api v1` or `api v2`. No warnings appeared, so the workloads satisfy `restricted`.

Milestone 4 — Default-deny NetworkPolicies (L16.2)

First prove the network is flat from another namespace, then close it:

```bash
kubectl -n sandbox run intruder --image=curlimages/curl:8.10.1 --restart=Never --command -
  - sleep 3600
kubectl -n sandbox wait --for=condition=Ready pod/intruder
kubectl -n sandbox exec intruder -- curl -s --max-time 3 http://api.shop        # works
  today
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: default-deny-all, namespace: shop}
spec: {podSelector: {}, policyTypes: [Ingress, Egress]}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: allow-dns, namespace: shop}
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: kube-system}}
          podSelector: {matchLabels: {k8s-app: kube-dns}}
      ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: client-egress-to-api, namespace: shop}
spec:
  podSelector: {matchLabels: {app: client}}
  policyTypes: [Egress]
  egress:
    - to: [{podSelector: {matchLabels: {app: api}}}]
      ports: [{protocol: TCP, port: 5678}]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: api-ingress-from-client, namespace: shop}
spec:
  podSelector: {matchLabels: {app: api}}
  policyTypes: [Ingress]
  ingress:
    - from: [{podSelector: {matchLabels: {app: client}}}]
      ports: [{protocol: TCP, port: 5678}]
EOF
kubectl -n shop exec deploy/client -- curl -s --max-time 3 http://api             # still
  works
```

```
kubectl -n sandbox exec intruder -- curl -s --max-time 3 http://api.shop; echo "exit=$?"
  # blocked
hubble observe --namespace shop --verdict DROPPED --last 5
```

(Run `cilium hubble port-forward &` first if `hubble observe` cannot connect.) The intruder times out (`exit=28`) and Hubble shows a `DROPPED` flow with `Policy denied` . Note the client reaches `api` by the Service name: the Service resolves, kube-proxy/Cilium translates to a Pod IP, and the policy is evaluated against the Pod.

Milestone 5 — Autoscaling (L16.7)

```bash
CI=system:serviceaccount:shop:ci-deployer
cat <<'EOF' | kubectl apply -n shop --as=$CI -f -
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: api-v1}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: api-v1}
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource: {name: cpu, target: {type: Utilization, averageUtilization: 50}}
  behavior:
    scaleDown: {stabilizationWindowSeconds: 120}
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: api-v1}
spec:
  maxUnavailable: 1
  selector: {matchLabels: {app: api, version: v1}}
EOF
kubectl -n shop exec deploy/client -- sh -c 'seq 1 8 | xargs -P8 -I{} sh -c "while true;
  do curl -s -o /dev/null http://api; done"' &
kubectl -n shop get hpa api-v1 -w
```

Watch `TARGETS` rise above 50% and `REPLICAS` climb (requests are only 20m CPU so a few parallel curl loops are enough; if it does not scale, add loops or lower the target). Stop the load with `kill %1; kubectl -n shop delete pod -l app=client` and watch the slow scale-down.

On EKS, add a reviewed Karpenter `NodePool` and `EC2NodeClass` (copy the manifests from L16.7 step 7, set `limits.cpu` as a cost fence), raise `maxReplicas` beyond node capacity and watch Pending Pods turn into new nodes. Local kind cannot add nodes.

Milestone 6 — Stateful data tier with an operator (L16.8, L16.4)

```bash
helm repo add cnpg https://cloudnative-pg.github.io/charts && helm repo update
helm upgrade --install cnpg cnpg/cloudnative-pg -n cnpg-system --create-namespace
kubectl -n cnpg-system rollout status deploy -l app.kubernetes.io/name=cloudnative-pg
cat <<'EOF' | kubectl apply -f -
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata: {name: shop-db, namespace: shop-data}
spec:
  instances: 2
  storage: {size: 1Gi}
  resources:
    requests: {cpu: 250m, memory: 512Mi}
    limits: {memory: 512Mi}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: db-ingress, namespace: shop-data}
spec:
  podSelector: {matchLabels: {cnpg.io/cluster: shop-db}}
  policyTypes: [Ingress]
  ingress:
    - from:   # replication and status between instances
        - podSelector: {matchLabels: {cnpg.io/cluster: shop-db}}
    - from:   # the operator
        - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: cnpg-system}}
      ports: [{protocol: TCP, port: 8000}, {protocol: TCP, port: 5432}]
    - from:   # only Pods explicitly labelled for database access, in shop
        - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: shop}}
          podSelector: {matchLabels: {db-access: "true"}}
      ports: [{protocol: TCP, port: 5432}]
EOF
kubectl -n shop-data get cluster shop-db -w
```

Wait for `Cluster in healthy state`. Ingress is default-denied for the database; egress is left open in this namespace because the Postgres instance manager must reach the Kubernetes API server, which has no stable Pod selector (a trade-off to record in your notes). Test the denied path from the `sandbox` intruder (milestone 4), which has no egress restrictions of its own:

```bash
kubectl -n sandbox exec intruder -- curl -sv --connect-timeout 3 --max-time 4
  telnet://shop-db-rw.shop-data:5432 2>&1 | grep -Ei "connected|timed out"
```

Expect `timed out`: the database admits only the operator, its own instances and `shop` Pods labelled `db-access=true`. Now the allowed path. `shop` is egress-restricted, so first allow egress for labelled Pods, then run a throwaway labelled Pod (a Pod, so the Deployment-only admission policy does not apply, but Pod Security still does):

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: db-clients-egress, namespace: shop}
spec:
  podSelector: {matchLabels: {db-access: "true"}}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: shop-data}}
          podSelector: {matchLabels: {cnpg.io/cluster: shop-db}}
      ports: [{protocol: TCP, port: 5432}]
EOF
kubectl -n shop run dbprobe --image=curlimages/curl:8.10.1 --restart=Never --labels=db-
  access=true \
  --overrides='{"spec":
  {"serviceAccountName":"client","automountServiceAccountToken":false,"securityContext":
  {"runAsNonRoot":true,"runAsUser":10001,"seccompProfile":
  {"type":"RuntimeDefault"}},"containers":
  [{"name":"dbprobe","image":"curlimages/curl:8.10.1","command":
  ["sleep","120"],"securityContext":{"allowPrivilegeEscalation":false,"capabilities":
  {"drop":["ALL"]}},"resources":{"limits":{"memory":"64Mi"}}}]}}'
kubectl -n shop wait --for=condition=Ready pod/dbprobe
kubectl -n shop exec dbprobe -- curl -sv --connect-timeout 3 --max-time 4 telnet://shop-
  db-rw.shop-data:5432 2>&1 | grep -Ei "connected|timed out"
```

Expect `Connected to shop-db-rw.shop-data` this time. Finally take a logical backup and restore it into a scratch database as in L16.8 step 6.

Milestone 7 — Service mesh: mTLS and canary (L16.9, L16.10)

Install Istio in ambient mode, which needs no sidecars, so the `restricted` Pods stay unprivileged (a sidecar's init container needs `NET_ADMIN`, which Pod Security forbids unless you use the Istio CNI plugin):

```bash
istioctl install --set profile=ambient -y
kubectl -n istio-system get pods                    # istiod, ztunnel on each node, istio-cni-node
kubectl get crd gateways.gateway.networking.k8s.io >/dev/null 2>&1 || \
  kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/<GATEWAY_API_VERSION>/standard-install.yaml
kubectl label namespace shop istio.io/dataplane-mode=ambient
istioctl waypoint apply -n shop --enroll-namespace
kubectl -n shop get pods --show-labels | grep -i waypoint
```

No restart is needed, and the `waypoint` Pod appears in `shop`. If it does not, `kubectl -n shop get events` shows whether the admission policy or Pod Security blocked it. Now (1) require mTLS, (2) route 90/10 and (3) fix the NetworkPolicies that the mesh now makes insufficient. In ambient mode, traffic between Pods is tunnelled over HBONE on TCP port 15008, so the policies must allow 15008 on the path client to waypoint to api, and the waypoint needs to reach `istiod` on 15012:

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata: {name: default, namespace: shop}
spec: {mtls: {mode: STRICT}}
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: {name: api, namespace: shop}
spec:
  parentRefs:
    - {group: "", kind: Service, name: api, port: 80}
  rules:
    - backendRefs:
        - {name: api-v1, port: 80, weight: 90}
        - {name: api-v2, port: 80, weight: 10}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: mesh-client-to-waypoint, namespace: shop}
spec:
  podSelector: {matchLabels: {app: client}}
  policyTypes: [Egress]
  egress:
```

```
    - to: [{podSelector: {matchLabels: {gateway.networking.k8s.io/gateway-name:
  waypoint}}}]
      ports: [{protocol: TCP, port: 15008}]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: mesh-waypoint, namespace: shop}
spec:
  podSelector: {matchLabels: {gateway.networking.k8s.io/gateway-name: waypoint}}
  policyTypes: [Ingress, Egress]
  ingress:
    - from: [{podSelector: {matchLabels: {app: client}}}]
      ports: [{protocol: TCP, port: 15008}]
  egress:
    - to: [{podSelector: {matchLabels: {app: api}}}]
      ports: [{protocol: TCP, port: 15008}]
    - to: [{namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: istio-
  system}}}]
      ports: [{protocol: TCP, port: 15012}]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: mesh-api-from-waypoint, namespace: shop}
spec:
  podSelector: {matchLabels: {app: api}}
  policyTypes: [Ingress]
  ingress:
    - from: [{podSelector: {matchLabels: {gateway.networking.k8s.io/gateway-name:
  waypoint}}}]
      ports: [{protocol: TCP, port: 15008}]
EOF
kubectl -n shop exec deploy/client -- sh -c 'for i in $(seq 1 100); do curl -s http://api;
  done' | sort | uniq -c
```

Similar to `91 api v1` and `9 api v2`. If requests hang after the policies, run `hubble observe --namespace shop --verdict DROPPED --last 20`: it names the exact source, destination and port to allow (the details of which hop uses which port can vary with CNI and Istio versions; Hubble is the source of truth). Verify encryption and identities:

```bash
istioctl ztunnel-config workloads | grep -E "NAMESPACE|shop"      # PROTOCOL column shows
  HBONE for meshed Pods
kubectl -n istio-system logs ds/ztunnel --tail=50 | grep -i "identity" | tail -3
```

Expect source and destination identities of the form `spiffe://cluster.local/ns/shop/sa/client`.

Milestone 8 — A CRD and a tiny controller drive the canary (L16.5, L16.4, L16.1)

BASH

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata: {name: shopreleases.shop.example.com}
spec:
  group: shop.example.com
  scope: Namespaced
  names: {plural: shopreleases, singular: shoprelease, kind: ShopRelease, shortNames:
  [rel]}
  versions:
    - name: v1alpha1
      served: true
      storage: true
      subresources: {status: {}}
      additionalPrinterColumns:
        - {name: Canary, type: integer, jsonPath: .spec.canaryWeight}
        - {name: Applied, type: integer, jsonPath: .status.appliedWeight}
      schema:
        openAPIV3Schema:
          type: object
          required: [spec]
          properties:
            spec:
              type: object
              properties:
                canaryWeight: {type: integer, minimum: 0, maximum: 100, default: 0}
            status:
              type: object
              properties:
                appliedWeight: {type: integer}
---
apiVersion: v1
kind: ServiceAccount
metadata: {name: release-controller, namespace: shop}
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: release-controller, namespace: shop}
rules:
  - {apiGroups: [shop.example.com], resources: [shopreleases], verbs: [get, list, watch]}
  - {apiGroups: [shop.example.com], resources: [shopreleases/status], verbs: [get, update,
  patch]}
  - {apiGroups: [gateway.networking.k8s.io], resources: [httproutes], resourceNames:
  [api], verbs: [get, patch]}
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: release-controller, namespace: shop}
subjects: [{kind: ServiceAccount, name: release-controller, namespace: shop}]
```

```
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: Role, name: release-controller}
---
apiVersion: shop.example.com/v1alpha1
kind: ShopRelease
metadata: {name: api, namespace: shop}
spec: {canaryWeight: 10}
EOF
cat > release-controller.sh <<'EOF'
#!/usr/bin/env bash
# Level-triggered reconcile loop: ShopRelease.spec.canaryWeight → HTTPRoute weights
K="kubectl -n shop --as=system:serviceaccount:shop:release-controller"
while true; do
  w=$($K get shoprelease api -o jsonpath='{.spec.canaryWeight}')
  have=$($K get httproute api -o jsonpath='{.spec.rules[0].backendRefs[1].weight}')
  if [ "$w" ≠ "$have" ]; then
    echo "$(date +%T) drift: route=$have want=$w → patching"
    $K patch httproute api --type json -p "[

  {\"op\":\"replace\",\"path\":\"/spec/rules/0/backendRefs/0/weight\",\"value\":$((100-
  w))},
      {\"op\":\"replace\",\"path\":\"/spec/rules/0/backendRefs/1/weight\",\"value\":$w}]"
  fi
  $K patch shoprelease api --subresource=status --type merge -p "{\"status\":
  {\"appliedWeight\":$w}}" >/dev/null
  sleep 5
done
EOF
chmod +x release-controller.sh && ./release-controller.sh &
kubectl -n shop patch shoprelease api --type merge -p '{"spec":{"canaryWeight":50}}'
sleep 10; kubectl -n shop get shoprelease
kubectl -n shop exec deploy/client -- sh -c 'for i in $(seq 1 100); do curl -s http://api;
  done' | sort | uniq -c
```

The loop impersonates the controller's ServiceAccount, so RBAC proves it can only touch `ShopRelease` objects, their status and the single `api` route. `kubectl patch shoprelease api ... canaryWeight=150` is rejected by the schema, and `--as=...:release-controller` cannot read Secrets.

5 Tasks

- [ ] **T1 — Build the hardened baseline** *(≈90 min, intermediate; milestones 0-3; topics L16.1, L16.3, L16.6)*
  - **Goal:** a cluster where the pipeline identity is least-privilege and bad workloads are rejected.
  - **Steps:**
    1. Complete milestones 0-3.

2. Capture the five `can-i` results, the VAP denial and the PSA rejection.

- **Done when:** `bad` is denied by the policy, `root`'s ReplicaSet reports `violates PodSecurity`, and the three hardened workloads run `1/1` with no warnings.

- [ ] **T2 — Default-deny proof** *(≈60 min, intermediate; milestone 4; topic L16.2)*
  - **Goal:** show the network is closed except for intended flows.
  - **Steps:**
    1. Run the intruder test before and after the policies.
    2. Add a new allowed flow: Prometheus-like Pods labelled `app=scraper` in `sandbox` may reach `api` on 5678, and prove an identically labelled Pod in another namespace is still denied.
    3. Capture a Hubble `DROPPED` line for a denied attempt.
  - **Done when:** client works, intruder times out, the scraper works only from `sandbox`, and you have the Hubble output.

- [ ] **T3 — Autoscale under load** *(≈75 min, intermediate; milestone 5; topic L16.7)*
  - **Goal:** pod scaling with sane behaviour; node scaling designed.
  - **Steps:**
    1. Generate load and record replicas and CPU every 30 s until the HPA stabilizes.
    2. Remove the load and record scale-down timing against `stabilizationWindowSeconds`.
    3. Write (do not apply) the Karpenter `NodePool` for EKS with a CPU limit and a disruption budget, and list what would happen at the limit.
  - **Done when:** you have a replicas-versus-time table and a reviewed NodePool manifest.

- [ ] **T4 — Operate the database** *(≈90 min, advanced; milestone 6; topics L16.8, L16.4)*
  - **Goal:** failover, access control and restore.
  - **Steps:**
    1. Delete the primary `shop-db` Pod and measure recovery.
    2. Prove the database is reachable only from Pods labelled `db-access=true` in `shop` (the `dbprobe` test connects, the `sandbox` intruder times out).
    3. Take a `pg_dump`, drop a table, restore into a scratch database, and state your RPO and RTO.
  - **Done when:** the cluster recovers with a new primary, the denied and allowed connection tests behave as predicted, and the restored table matches.

- [ ] **T5 — Mesh, mTLS and canary** *(≈2 hours, advanced; milestone 7; topics L16.9, L16.10)*

- **Goal:** encrypted workload traffic and a measured canary.
- **Steps:**
  1. Install ambient Istio, enrol `shop`, apply the waypoint, `STRICT` PeerAuthentication and the 90/10 HTTPRoute.
  2. Fix the NetworkPolicies using Hubble until traffic flows.
  3. Measure 100 requests at 90/10, 50/50 and 0/100, and show the `HBONE` protocol and SPIFFE identities.
- **Done when:** the distributions match the weights and `istioctl ztunnel-config workloads` shows `HBONE` for the `shop` Pods.

- [ ] **T6 — Extend the API with a CRD and controller** *(≈2 hours, advanced; milestone 8; topics L16.5, L16.4, L16.1)*
  - **Goal:** a declarative release object that changes live traffic with least privilege.
  - **Steps:**
    1. Apply the CRD, RBAC and the controller loop; patch `canaryWeight` to 50 then 0.
    2. Add a CEL rule that only allows `canaryWeight` to change by at most 25 points per update (`self.spec.canaryWeight - oldSelf.spec.canaryWeight`), then show the rejection when you jump from 0 to 100.
    3. Run the controller as the impersonated ServiceAccount and show it cannot patch a different HTTPRoute or read Secrets.
  - **Done when:** the route weights follow the object, a 0 to 100 jump is rejected, and the `can-i` checks for the controller are all as designed.

- [ ] **T7 — Break it and fix it** *(≈90 min, advanced; all milestones)*
  - **Goal:** practise diagnosing the failures this stack produces.
  - **Steps:**
    1. Delete the `allow-dns` policy and diagnose the resulting failure from `client` using only `kubectl` and Hubble.
    2. Change `api-v2` to `image: hashicorp/http-echo:latest` and observe the admission denial; change it to a root user and observe the Pod Security failure on the ReplicaSet.
    3. Remove the waypoint-related NetworkPolicy and diagnose the hung canary.
  - **Done when:** for each fault you can name the layer that failed (network, admission, Pod Security, mesh), the evidence, and the fix.

- [ ] **T8 — Prove it: harden a fresh cluster from memory** *(≈4 hours, advanced; all topics)*

- **Goal:** reproduce the whole hardening on a new cluster without these notes, and explain it.
- **Steps:**
  1. Create a new kind cluster and, from your own manifests in Git, apply RBAC, Pod Security labels, the admission policy, default-deny policies, an HPA and the mesh with a canary.
  2. Write a one-page security review: for each layer, one threat it stops, one thing it does not stop, and the command that proves it works.
  3. Present it to a teammate and answer "what if the CNI did not enforce NetworkPolicy?"
- **Done when:** the second cluster passes the verification checklist below and your review is understood by a teammate.

6 Verification checklist

- [ ] `kubectl auth can-i get secrets -n shop --as=system:serviceaccount:shop:ci-deployer` prints `no`; `create deployments` prints `yes`.
- [ ] `shop` and `shop-data` carry `pod-security.kubernetes.io/enforce=restricted`; `kubectl label --dry-run=server --overwrite ns shop pod-security.kubernetes.io/enforce=restricted` prints no warnings.
- [ ] Creating a Deployment with `nginx:latest` or without a memory limit in `shop` is denied by `workload-hygiene`.
- [ ] `kubectl get networkpolicy -n shop` shows `default-deny-all`; the intruder in `sandbox` cannot reach `api.shop`; Hubble shows the drop.
- [ ] `kubectl -n shop get hpa api-v1` shows a numeric target and replicas change under load.
- [ ] `kubectl -n shop-data get cluster shop-db` reports a healthy cluster; the `sandbox` intruder cannot connect to port 5432 while a `db-access=true` Pod in `shop` can.
- [ ] `istioctl ztunnel-config workloads` lists the `shop` Pods with protocol `HBONE`, and `PeerAuthentication default` is `STRICT`.
- [ ] 100 requests through `http://api` split according to the HTTPRoute weights; `kubectl -n shop get shoprelease` shows `Canary` equal to `Applied`.
- [ ] You can state, for each layer, what it protects against and what it does not.

7 Teardown

```bash
kill %1 %2 2>/dev/null            # stop load generators and the controller loop if still
  running
cilium hubble disable 2>/dev/null
kind delete cluster --name hardened
rm -rf ~/l16-capstone
```

Nothing else runs locally. **On EKS:** delete workloads first so Karpenter drains its nodes (`kubectl delete namespace shop shop-data`), then the NodePools and EC2NodeClasses, then the cluster and its node groups (`eksctl delete cluster` or `terraform destroy`), then confirm no EC2 instances, EBS volumes, load balancers or NAT gateways remain in the console, and check Cost Explorer the next day.

8 Stretch goals and what to learn next

- **Replace the policy engine:** reimplement `workload-hygiene` in Kyverno (add a `generate` rule creating a default-deny NetworkPolicy in every new namespace) and compare operational costs (L16.6).
- **Audit and detect:** enable API audit logging with the policy from L16.1 and write an alert when a `RoleBinding` to `cluster-admin` appears.
- **Per-service authorization:** add L7 `AuthorizationPolicy` through the waypoint so only the `client` identity may call `api`, and compare with Linkerd's policy model on a second cluster (L16.9).
- **Automated canary:** replace the shell loop with Argo Rollouts or Flagger driven by `istio_requests_total` success rate and latency (L16.10).
- **Port the controller to Go** with kubebuilder, adding a finalizer and status conditions (L16.4).
- **Add VPA recommendations** in `Off` mode and right-size the requests used here (L16.7).
- **Put it all in Git:** manage every manifest above with a GitOps controller. Next level: Level 17 — GitOps §17.1, then platform building blocks in Policy as Code §18.7.

LEVEL 17 · PART VIII

GitOps

17

Goal: Operate Kubernetes by declaring desired state in Git and letting a controller continuously reconcile it.

RECOMMENDED TIME
2-3 weeks

TOPICS
7

PRACTICE / CAPSTONE
Install Argo CD (or Flux), deploy the app from a GitOps repo, promote dev → staging → prod via Git, and roll back with git revert.

Topics

#	Topic	What you will learn
1	Desired State Management **p.3305**	GitOps principles, declarative config, pull vs push deploys, repo layout
2	Argo CD and Flux **p.3317**	architecture, install, Applications, Kustomization/HelmRelease, choosing between them
3	Continuous Reconciliation **p.3329**	drift correction, self-heal, prune, sync loops
4	Environment Promotion **p.3341**	overlays, image tag bumps, PR-based promotion, Image Updater/Flux automation
5	Application Synchronization **p.3351**	sync waves, hooks, sync policies, health assessment, ApplicationSets
6	Rollback Through Git **p.3365**	git revert as rollback, audit trail, emergency procedure
7	The Full Flow: CI to GitOps **p.3375**	Developer → Git → CI → Docker image → GitOps repo → Argo CD → Kubernetes, secrets in GitOps (Sealed Secrets/SOPS/ESO)
★	Level capstone project **p.3389**	Install Argo CD (or Flux), deploy the app from a GitOps repo, promote dev → staging → prod via Git, and roll back with git revert.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



17.1 Desired State Management

Topic 1 of 7 | ~4 hours | Difficulty: Intermediate

1 What it is

GitOps is a way of operating infrastructure and applications where a **Git repository is the single source of truth for the *desired state*** of the system, and a software agent running next to the system continuously makes the *actual state* match it. You do not run `kubectl apply` from your laptop or push a release from a CI server. You open a pull request that changes a file; once it is merged, an agent inside the cluster notices and converges the cluster.

The term was popularised by Weaveworks around 2017. Today the vendor-neutral definition is the **OpenGitOps principles** maintained by a CNCF (Cloud Native Computing Foundation) working group. A GitOps system is:

1. **Declarative** - the desired state is expressed as data ("3 replicas of image X"), not as a script of steps.
2. **Versioned and immutable** - the desired state is stored so every change is a versioned, auditable, revertible commit.
3. **Pulled automatically** - software agents automatically pull the desired state from the source.
4. **Continuously reconciled** - agents observe the actual state and keep trying to apply the desired state.

"Desired state management" is the underlying idea: describe *what* you want, store it somewhere durable, and let a control loop (the same idea as Kubernetes controllers) deal with *how*.

2 Why it matters in DevOps

- **Auditability:** "who changed production, when and why?" becomes `git log` plus a pull-request review, not archaeology in shell history.
- **Rollback is a commit:** returning to the last good state is `git revert`, which is fast, reviewed and itself recorded (see Rollback Through Git §17.6).

- **No cluster credentials in CI:** in a push model the CI system holds admin credentials to production. In a pull model only the in-cluster agent holds them, and it only needs read access to Git.
- **Drift disappears:** someone who hand-edits a Deployment with `kubectl edit` at 2 a.m. gets reverted (or at least flagged) instead of silently creating a snowflake environment.
- **Disaster recovery:** a cluster that dies can be rebuilt by pointing a fresh agent at the same repo. See Disaster Recovery §0.9.

3 Real-world system example

Argo CD and Flux, the two CNCF GitOps engines. Argo CD originated at Intuit and was open-sourced as part of the Argo project; Flux was created at Weaveworks. Both are CNCF projects that have reached the **graduated** maturity level, and both implement the pull model: a controller inside the cluster pulls from Git (or an OCI registry) and reconciles. Red Hat ships Argo CD as "OpenShift GitOps", and many cloud vendors offer managed or add-on versions of one of the two. Kubernetes itself is the original desired-state system: every controller, from the Deployment controller to the kubelet, compares `spec` (desired) with `status` (actual) and acts on the difference. GitOps extends the same loop one level up: the "spec" of your whole platform lives in Git.

4 Diagram

Push versus pull delivery:

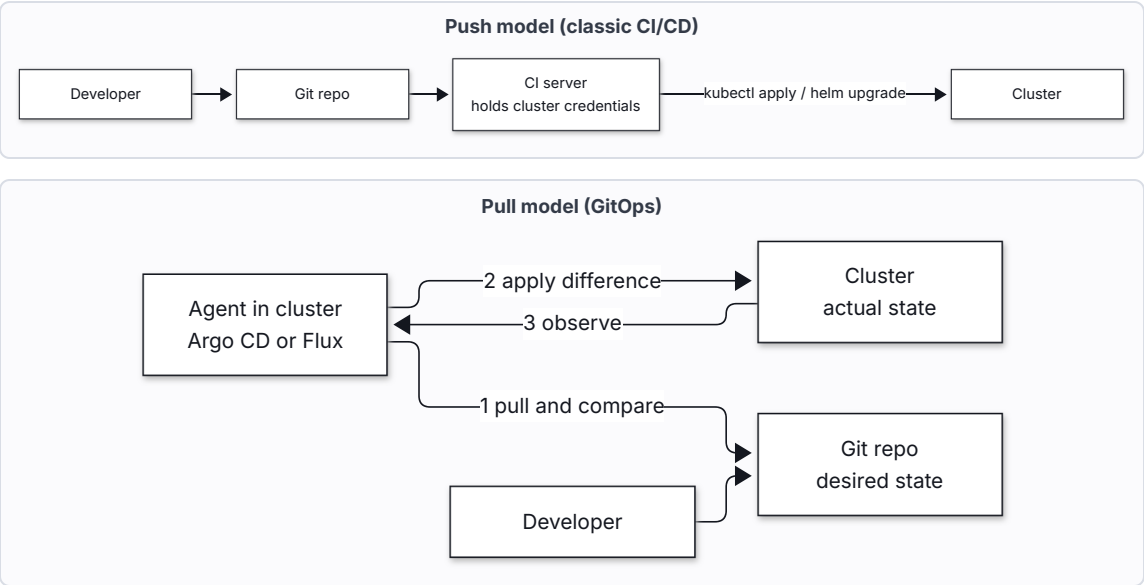


Figure 17.1-1

The reconcile loop, as a state machine:

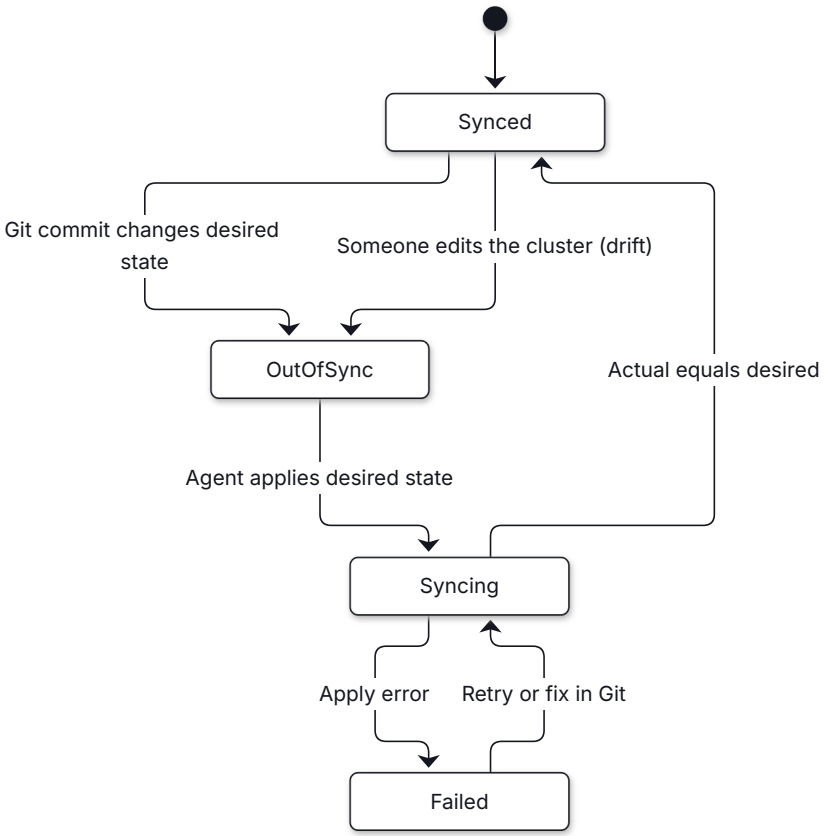


Figure 17.1-2

5 How it works

Declarative versus imperative

	Imperative	Declarative
You say	"run 3 more pods", "restart the service"	"there should be 5 pods running image `shop-api:1.4.2`"
Stored as	a runbook or shell history	YAML in Git
Re-running it	may duplicate or fail	idempotent - same input, same result
Detecting drift	impossible	diff desired against actual

Push versus pull

	Push (CI deploys)	Pull (GitOps agent)
Who holds prod credentials	the CI system, outside the cluster	the agent, inside the cluster
Drift handling	none until the next deploy	continuous detection and correction
Network	cluster API must be reachable from CI	cluster only needs outbound access to Git
Failure mode	a failed pipeline leaves a half-applied state	the agent keeps retrying toward the same target

Push is not "wrong": CI/CD pipelines §5.1 still build, test and publish the artifact. GitOps changes only the **last mile** - how the new version reaches the cluster.

Two repositories, not one

Most teams separate the **application repo** (source code, Dockerfile, CI) from the **GitOps / environment repo** (manifests that say which image version runs where). Reasons: different change cadence, different reviewers, no CI loop where a manifest commit triggers a rebuild, and a clean audit trail of deployments.

A repo layout with Kustomize overlays

Kustomize `(kustomize.io)` is built into `kubectl` (`kubectl apply -k`). A **base** holds the common manifests; an **overlay** per environment patches only what differs. This layout is used throughout the level:

```
shop-gitops/
├── apps/
│   └── shop-api/
│       ├── base/
│       │   ├── deployment.yaml
│       │   ├── service.yaml
│       │   └── kustomization.yaml
│       └── overlays/
│           ├── dev/
│           │   └── kustomization.yaml      # 1 replica, image tag = latest build
│           ├── staging/
│           │   └── kustomization.yaml      # 2 replicas, tag promoted from dev
│           └── prod/
│               └── kustomization.yaml      # 3 replicas, tag promoted from staging
├── clusters/
│   └── local/
│       └── root-app.yaml                   # the "app of apps" entry point (topic 5)
└── README.md
```

Key rules: **no environment-specific YAML copy-pasted into base**, **one directory per environment**, and the only thing a promotion changes is a few lines (usually the image tag) in an overlay. Alternatives: Helm charts with a `values-<env>.yaml` per environment (see Values §9.2), or a branch per environment (generally discouraged because merges between branches become the deployment process and diverge over time).

What belongs in the desired state

Everything you want reproducible: Deployments, Services, Ingresses, ConfigMaps, namespaces, RBAC, NetworkPolicies, CRDs and operator custom resources, and the GitOps agent's own configuration. What does **not** belong in plain text: secrets (covered in The Full Flow §17.7) and fast-changing runtime state such as the replica count that an HPA controls.

6 Step-by-step guide

Prerequisites: a Linux/macOS shell, `git`, and `kubectl` (version 1.14 or newer ships Kustomize). No cluster needed in this topic; we only build and render the repo layout. Cost: none.

1. **Create the repo skeleton.**

```bash
mkdir -p shop-gitops/apps/shop-api/{base,overlays/dev,overlays/staging,overlays/prod}
cd shop-gitops && git init -b main
```

2. **Write the base.**

```bash
cat > apps/shop-api/base/deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: shop-api
  template:
    metadata:
      labels:
        app: shop-api
    spec:
      containers:
        - name: shop-api
          image: ghcr.io/example/shop-api:1.0.0
```

```
          ports:
            - containerPort: 3000
EOF
cat > apps/shop-api/base/service.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: shop-api
spec:
  selector:
    app: shop-api
  ports:
    - port: 80
      targetPort: 3000
EOF
cat > apps/shop-api/base/kustomization.yaml <<'EOF'
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
EOF
```

3. **Write the `dev` and `prod` overlays.**

BASH

```
cat > apps/shop-api/overlays/dev/kustomization.yaml <<'EOF'
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-dev
resources:
  - ../../base
images:
  - name: ghcr.io/example/shop-api
    newTag: 1.1.0
EOF
cat > apps/shop-api/overlays/prod/kustomization.yaml <<'EOF'
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-prod
resources:
  - ../../base
replicas:
  - name: shop-api
    count: 3
images:
  - name: ghcr.io/example/shop-api
    newTag: 1.0.0
EOF
```

4. **Render each environment (this is what an agent does before it applies).**

```bash
kubectl kustomize apps/shop-api/overlays/dev  | grep -E 'namespace|replicas|image:'
kubectl kustomize apps/shop-api/overlays/prod | grep -E 'namespace|replicas|image:'
```

   Expected (similar to): dev shows `namespace: shop-dev`, `replicas: 1`, `image: ghcr.io/example/shop-api:1.1.0`; prod shows `namespace: shop-prod`, `replicas: 3`, `image: ghcr.io/example/shop-api:1.0.0`.

5. **Commit it - this commit *is* the deployment record.**

```bash
git add . && git commit -m "feat: shop-api base with dev and prod overlays"
```

6. **Diff desired state like an agent would.** If you have a cluster later, `kubectl diff -k apps/shop-api/overlays/dev` prints what would change. Practise reading that output; every GitOps tool shows you the same idea.

7 Real-world scenario

Illustrative scenario, not a documented incident. The `shop-api` team's production Deployment is edited by hand during a late-night incident: someone runs `kubectl scale --replicas=10` and `kubectl set env` to add a debug flag. The incident ends, nobody writes it down, and two weeks later a routine deploy from CI overwrites the replica count back to 3 and removes the flag - a second outage during Black Friday traffic. After moving to GitOps, the same hand edit would have shown the application as **OutOfSync** within minutes, and with self-heal enabled the cluster would have reverted it, forcing the engineer to put the change (with a reason) into a pull request. **Lesson:** when the repo is the only way to change prod, production stops containing undocumented state.

8 Common mistakes and best practices

Don't

- Keep application source and environment manifests in one repo where every manifest commit retriggers CI.
- Copy-paste whole manifests per environment; the copies drift apart.
- Commit plain-text secrets or kubeconfig files to the GitOps repo.
- Use long-lived branches per environment as your promotion mechanism.

- Let people keep `kubectl edit` access to production "just in case" without alerting on it.

Do

- Describe desired state declaratively and keep it small: base plus thin overlays.
- Protect the GitOps repo's main branch with required reviews (Protected Branches §3.8) - the merge button is now your deploy button.
- Pin images by explicit tag (ideally immutable tags or digests), never `latest`.
- Render and validate manifests in CI on every pull request (`kubectl kustomize`, `kubeconform`).
- Treat the repo layout as an API: document it in the README.

9 Tasks

- [ ] **T1 — Build the layout** *(≈30 min, beginner)*
  - **Goal:** create the `shop-gitops` base plus `dev` and `prod` overlays from section 6.
  - **Steps:**
    1. Follow the guide through step 5.
    2. Run `kubectl kustomize` on both overlays and save the output to `/tmp/dev.yaml` and `/tmp/prod.yaml`.
    3. Run `diff /tmp/dev.yaml /tmp/prod.yaml`.
  - **Done when:** the diff shows only namespace, replicas and image tag differences.
- [ ] **T2 — Add a staging overlay** *(≈20 min, beginner)*
  - **Goal:** extend the pattern without copying the base.
  - **Steps:**
    1. Create `overlays/staging/kustomization.yaml` with namespace `shop-staging`, 2 replicas and tag `1.0.0`.
    2. Render it and check the values.
  - **Done when:** `kubectl kustomize apps/shop-api/overlays/staging` shows 2 replicas in `shop-staging`.
- [ ] **T3 — Add a config difference with a patch** *(≈30 min, intermediate)*
  - **Goal:** use a strategic-merge patch for something other than image or replicas.
  - **Steps:**
    1. Add a `patches:` entry in the prod overlay that sets `resources.requests` (CPU `200m`, memory `256Mi`) on the container.
    2. Render prod and confirm only prod has the requests.

- **Done when:** dev renders without `resources:` and prod renders with them.

- [ ] **T4 — Break it and read the error** *(≈20 min, intermediate)*
  - **Goal:** learn what a failing render looks like before an agent shows it to you.
  - **Steps:**
    1. Misspell a resource path in `base/kustomization.yaml`.
    2. Run `kubectl kustomize apps/shop-api/overlays/dev` and read the error.
    3. Fix it and re-render.
  - **Done when:** you can explain, from the error text alone, which file and path was wrong.

- [ ] **T5 — Add a validation gate** *(≈45 min, advanced)*
  - **Goal:** automate "never merge unrenderable manifests".
  - **Steps:**
    1. Write a GitHub Actions workflow (see GitHub Actions workflows §3.11) that loops over `apps/*/overlays/*` and runs `kubectl kustomize` on each, failing on error.
    2. Push a deliberately broken overlay on a branch and confirm the check fails.
  - **Done when:** the pull request shows a red check for the broken overlay and green after your fix.

- [ ] **T6 — Prove it: push versus pull** *(≈30 min, advanced)*
  - **Goal:** explain GitOps to a teammate.
  - **Steps:**
    1. Without the guide, draw the push and pull flows and mark where cluster credentials live in each.
    2. List the four OpenGitOps principles and give one concrete example of each from the repo you built.
  - **Done when:** a teammate can say, from your drawing, why a compromised CI server is less dangerous in the pull model.

10 Self-check questions

1. What are the four OpenGitOps principles?
2. Why is a pull-based agent considered safer than a CI job that runs `kubectl apply`?
3. Why separate the application repository from the GitOps repository?
4. What is the difference between a Kustomize base and an overlay?
5. Why is "a branch per environment" a risky layout?

ANSWERS

1. Declarative; versioned and immutable; pulled automatically; continuously reconciled.
2. Cluster credentials stay inside the cluster; CI only needs to write to Git. The agent also corrects drift continuously rather than only at deploy time.
3. Different change cadence and reviewers, no rebuild loops from manifest commits, and a clean deployment audit trail.
4. The base holds the common manifests; an overlay references the base and patches only what differs for one environment.
5. Promotion becomes branch merging, which causes conflicts, hidden divergence and cherry-pick chaos; directories keep every environment visible side by side.

11 Further reading

- OpenGitOps (opengitops.dev) - the four principles, maintained under the CNCF.
- Kustomize documentation (kubectl.docs.kubernetes.io and kustomize.io) - bases, overlays, patches, images.
- Argo CD documentation, "Best Practices" - separating config repo from source repo.
- *GitOps and Kubernetes* by Billy Yuen, Alexander Matyushentsev, Todd Ekenstam and Jesse Suen (Manning).

LAB NOTES What I ran, what I saw, what it means

17.2 Argo CD and Flux

Topic 2 of 7 | ~5 hours | Difficulty: Intermediate

1 What it is

Argo CD and **Flux** are the two mainstream open-source **GitOps controllers** for Kubernetes. Both run *inside* the cluster, watch a Git repository (or Helm repository or OCI registry), render the manifests found there (plain YAML, Kustomize, Helm) and apply them. Both are CNCF (Cloud Native Computing Foundation) graduated projects.

- **Argo CD** is an application-centric tool with a web UI, a CLI (`argocd`) and one central object, the `Application` custom resource ("take *this path* of *this repo* and deploy it to *this cluster/namespace*").
- **Flux** (version 2, also called the "GitOps Toolkit") is a set of small, composable controllers driven entirely by Kubernetes custom resources and the `flux` CLI. There is no built-in UI; you compose `GitRepository`, `Kustomization` and `HelmRelease` objects.

They solve the same problem with different philosophies, covered in the comparison below.

2 Why it matters in DevOps

- Picking and operating the controller is the core of a GitOps platform: its availability, RBAC (role-based access control) and upgrade story are now part of your production path.
- Argo CD's UI shows health and diff to non-experts (support engineers, product teams); Flux's toolkit approach fits platform teams who want to build their own abstractions. Choosing wrongly costs months.
- Both read Git with credentials you create (deploy keys or tokens). Mis-scoped credentials are a real supply-chain risk.
- Knowing the mechanism (pull, render, diff, apply, health check) lets you debug "why is it OutOfSync?" quickly, which is the most common GitOps ticket.

3 Real-world system example

Intuit created Argo CD and uses the Argo projects internally; the project's public `USERS.md` lists hundreds of adopting organisations. **Red Hat OpenShift GitOps** packages Argo CD for OpenShift. **Flux** is used by many organisations and is the engine behind several vendors' managed GitOps features (for example, Azure's Kubernetes GitOps extension and the AKS/Arc offerings are built on Flux, according to Microsoft's documentation). Weaveworks, the company that created Flux, wound down its commercial operations in early 2024; Flux continued as a community-governed CNCF project, which is a reminder to evaluate a project's governance, not just its features. (Check the current project pages for the latest status before you decide.)

4 Diagram

Argo CD architecture and sync flow:

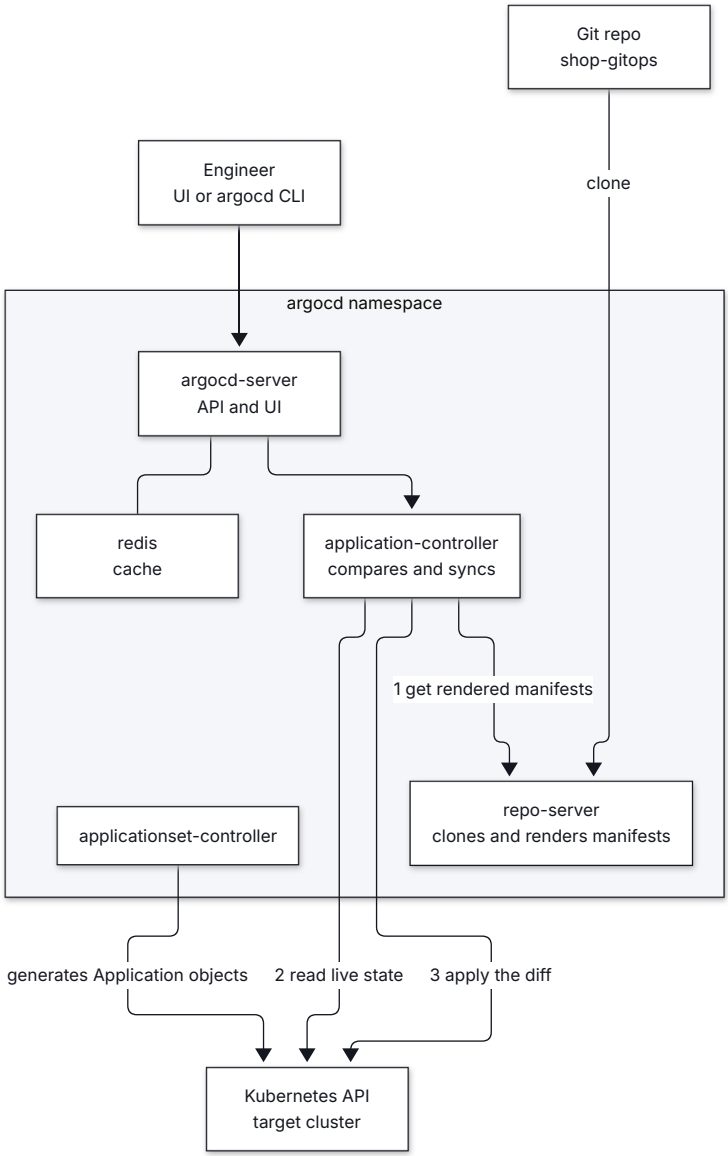


Figure 17.2-1

Flux controllers and their objects:

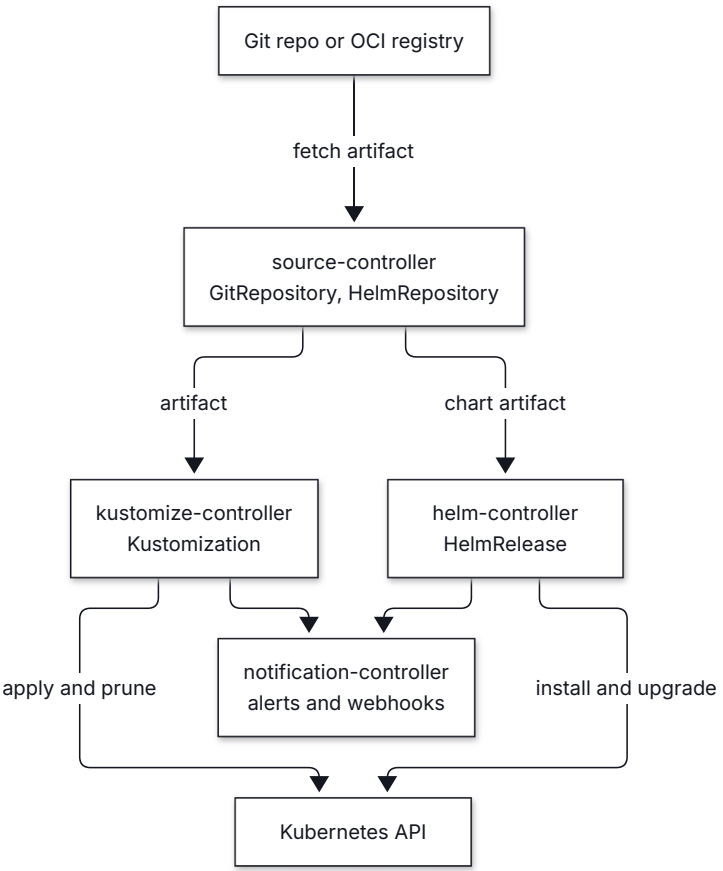


Figure 17.2-2

5 How it works

Argo CD

Component	Job
`argocd-server`	API server, web UI, auth (can use Dex for SSO - single sign-on)
`argocd-repo-server`	clones repos and renders manifests (plain, Kustomize, Helm, plugins)
`argocd-application-controller`	continuously compares **desired** (rendered) with **live** and syncs
`argocd-applicationset-controller`	generates many `Application`s from templates (topic 5)
`argocd-redis`	cache

Core objects: `Application` (what to deploy where), `AppProject` (guardrails: allowed repos, clusters, namespaces, resource kinds) and cluster/repo credentials stored as labelled Secrets. By default the controller refreshes Git roughly every **3 minutes** (configurable); webhooks from GitHub make it near-instant.

Flux

Controller	Objects	Job
source-controller	`GitRepository`, `HelmRepository`, `OCIRepository`, `Bucket`	fetch and package sources
kustomize-controller	`Kustomization` (Flux's, not Kustomize's `kustomization.yaml`)	build and apply manifests, prune, health-check
helm-controller	`HelmRelease`	install and upgrade Helm charts declaratively
notification-controller	`Provider`, `Alert`, `Receiver`	send alerts, receive webhooks
image-reflector / image-automation (optional)	`ImageRepository`, `ImagePolicy`, `ImageUpdateAutomation`	scan registries and commit new tags to Git

Flux bootstraps itself: `flux bootstrap` installs the controllers **and commits their own manifests to your repo**, so Flux manages Flux.

Choosing between them

	Argo CD	Flux
UI	rich built-in web UI (diff, tree, logs)	none built in (third-party UIs exist)
Core model	`Application` per deployable unit	`GitRepository` + `Kustomization` / `HelmRelease`
Multi-tenancy	`AppProject`, RBAC, SSO built in	Kubernetes RBAC and service-account impersonation
Multi-cluster	one control plane manages many clusters	typically one Flux per cluster, or remote kubeconfig
Helm	renders chart then applies it (not a Helm release in `helm ls`)	real Helm releases via helm-controller
Mass generation	`ApplicationSet`	Kustomization per directory, or templating tools
Image automation	Argo CD Image Updater (separate project)	built-in optional controllers
Drift correction	`selfHeal` option	on by default each reconcile interval

Rule of thumb: choose **Argo CD** when you want a UI, a central multi-cluster view and an app-centric model for many teams; choose **Flux** when you want a lightweight, Kubernetes-native toolkit with no extra API surface. Either is a sound choice; the concepts in the rest of this level apply to both. This level's examples use Argo CD, with Flux equivalents shown where they differ.

6 Step-by-step guide

Prerequisites: Docker, `kind` (`kind.sigs.k8s.io`), `kubectl`, `git`. Optional: `argocd` CLI. Cost: free (everything runs locally; allow ~4 GB RAM).

1. **Create a local cluster.**

```bash
kind create cluster --name gitops
kubectl cluster-info --context kind-gitops
```

   Expected: `Kubernetes control plane is running at https://127.0.0.1:...` (port varies).

2. **Install Argo CD.**

```bash
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server --timeout=300s
kubectl -n argocd get pods
```

   `--server-side` is used because some Argo CD CRDs are too large for client-side apply annotations. Expect about seven pods (server, repo-server, application-controller, applicationset-controller, redis, dex-server, notifications-controller), all `Running`. Pod names will differ (illustrative).

3. **Open the UI and log in.**

```bash
kubectl -n argocd get secret argocd-initial-admin-secret -o
  jsonpath='{.data.password}' | base64 -d; echo
kubectl -n argocd port-forward svc/argocd-server 8080:443
```

   Browse to `https://localhost:8080` (accept the self-signed certificate), user `admin`, paste the password. Change it and delete the initial secret once you have set up real auth.

4. **Create your first `Application`.** This deploys the public `guestbook` example from the Argo CD project; replace the repo with your own `shop-gitops` later.

```yaml
# guestbook-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: guestbook
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/argoproj/argocd-example-apps.git
    targetRevision: HEAD
    path: guestbook
  destination:
    server: https://kubernetes.default.svc
    namespace: guestbook
  syncPolicy:
    syncOptions:
      - CreateNamespace=true
```

```bash
kubectl apply -f guestbook-app.yaml
kubectl -n argocd get applications
```

   Expected (similar to): `guestbook OutOfSync Missing` at first, because no automated sync is configured.

5. **Sync it manually.**

```bash
kubectl -n argocd patch application guestbook --type merge \
  -p '{"operation":{"sync":{}}}'
kubectl -n argocd get applications
```

   After a few seconds expect `Synced Healthy`. (With the CLI: `argocd app sync guestbook`.)

6. **Flux equivalent (read, or try on a second kind cluster).** A `GitRepository` plus a `Kustomization`:

```yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: shop-gitops
```

```
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/example/shop-gitops
  ref:
    branch: main
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: shop-api-dev
  namespace: flux-system
spec:
  interval: 5m
  path: ./apps/shop-api/overlays/dev
  prune: true
  sourceRef:
    kind: GitRepository
    name: shop-gitops
  targetNamespace: shop-dev
```

Installing Flux itself is `flux bootstrap github --owner=<YOUR_USER> --repository=<YOUR_REPO> --path=clusters/local` (needs a GitHub token in `GITHUB_TOKEN`). A `HelmRelease` follows the same shape, with `spec.chart.spec` naming a chart and a `sourceRef` to a `HelmRepository`.

7. **Clean up** (or keep the cluster for the next topics).

```bash
kind delete cluster --name gitops
```

7 Real-world scenario

Illustrative scenario. A platform team of three supports 25 product teams. They first adopt Flux because it is "just controllers". Within a quarter, support tickets pile up: "is my deploy done?", "why is it red?". Developers cannot see anything without cluster access. The team adds Argo CD's UI for read-mostly visibility and `AppProject`s per team, keeping the same Git repo layout. Alternatively, a team that starts with Argo CD finds the central control plane is a single point of concern and moves to one lightweight controller per cluster. **Lesson:** the engine matters less than the repo structure, but the *audience* (who needs to see state, how many clusters) should drive the choice, and a pilot with real users beats a feature table.

8 Common mistakes and best practices

Don't

- Leave the default `admin` account and `argocd-initial-admin-secret` in a real cluster.
- Point production `Application`s at `targetRevision: HEAD` of an unprotected branch.
- Give the Argo CD controller cluster-admin on every cluster without `AppProject` restrictions.
- Install from `stable` manifests in production without pinning a version.
- Grant the agent write access to Git when it only needs to read.

Do

- Pin the Argo CD/Flux version (for example, a release tag in the install URL) and upgrade deliberately.
- Use read-only deploy keys or fine-grained tokens, and webhooks to cut sync latency.
- Define `AppProject`s that restrict source repos, destination namespaces and resource kinds per team.
- Back up the controller's configuration: with GitOps this means "it is in Git", including the controller's own manifests.
- Monitor the controllers (they export Prometheus metrics) - see Prometheus and PromQL §10.2.

9 Tasks

- [ ] **T1 — Install Argo CD on kind** *(≈45 min, beginner)*
  - **Goal:** a working Argo CD with UI access.
  - **Steps:**
    1. Follow section 6 steps 1-3.
    2. Log in at `https://localhost:8080`.
  - **Done when:** the UI loads and `kubectl -n argocd get pods` shows all pods `Running`.
- [ ] **T2 — Deploy the guestbook Application** *(≈30 min, beginner)*
  - **Goal:** see the Application custom resource drive a deployment.
  - **Steps:**
    1. Apply `guestbook-app.yaml` and sync it.
    2. Open the application tile in the UI and expand the resource tree.
    3. Run `kubectl -n guestbook get all`.
  - **Done when:** the app shows `Synced` and `Healthy` and the pods exist in namespace `guestbook`.

- [ ] **T3 — Point Argo CD at your own repo** *(≈45 min, intermediate)*
  - **Goal:** deploy the `shop-gitops` dev overlay from Desired State Management §17.1.
  - **Steps:**
    1. Push `shop-gitops` to a GitHub repository (public to start with).
    2. Write an `Application` with `path: apps/shop-api/overlays/dev` and `destination.namespace: shop-dev`.
    3. Apply and sync it.
  - **Done when:** `kubectl -n shop-dev get deploy shop-api` exists and Argo CD shows `Synced`.
- [ ] **T4 — Break it: bad path** *(≈20 min, intermediate)*
  - **Goal:** read an Argo CD error condition.
  - **Steps:**
    1. Change `path` to a directory that does not exist (`kubectl edit application ...`).
    2. Run `kubectl -n argocd describe application <name>` and look at `Conditions`.
    3. Fix the path.
  - **Done when:** you can quote the condition message and explain which component (repo-server) produced it.
- [ ] **T5 — Install Flux and compare** *(≈60 min, advanced)*
  - **Goal:** feel the difference between the tools.
  - **Steps:**
    1. Create a second kind cluster and run `flux check --pre`, then `flux bootstrap github` against a throwaway repo.
    2. Add a `GitRepository` and `Kustomization` for the dev overlay.
    3. Run `flux get kustomizations`.
  - **Done when:** `shop-api` runs in `shop-dev` and you have a 5-line written comparison (UI, objects, bootstrap, drift handling, image automation).
- [ ] **T6 — Prove it: choose and defend** *(≈30 min, advanced)*
  - **Goal:** justify a tool selection.
  - **Steps:**
    1. Without notes, draw the Argo CD component diagram and trace what happens between "git push" and "pods updated".
    2. For a fictional company (3 clusters, 12 teams, no platform staff), recommend Argo CD or Flux in one paragraph.

- **Done when:** your trace names repo-server, application-controller and the Kubernetes API in order, and your recommendation states two trade-offs.

10 Self-check questions

1. Which Argo CD component renders manifests and which one applies them?
2. What does `flux bootstrap` commit to your repository?
3. Name two situations where Argo CD is the better fit and two where Flux is.
4. Why is `--server-side` apply recommended for installing Argo CD?
5. What is an `AppProject` for?

ANSWERS

1. The repo-server renders; the application-controller compares live versus desired state and applies changes.
2. The Flux controllers' own manifests plus a sync configuration (a `GitRepository` and `Kustomization`), so Flux manages itself from Git.
3. Argo CD: need for a UI, central multi-cluster view, built-in SSO and project RBAC. Flux: Kubernetes-native toolkit, real Helm releases, built-in image automation, minimal footprint.
4. Some CRDs exceed the size limit of the annotation client-side apply uses to track state.
5. It restricts which repos, clusters, namespaces and resource kinds a group of Applications may use - multi-tenant guardrails.

11 Further reading

- Argo CD documentation (argo-cd.readthedocs.io) - "Getting Started", "Architecture", "Declarative Setup".
- Flux documentation (fluxcd.io) - "Get Started", "GitOps Toolkit components".
- OpenGitOps principles (opengitops.dev).
- See also: Helm charts §9.1 and Namespaces §8.8.

LAB NOTES What I ran, what I saw, what it means

17.3 Continuous Reconciliation

Topic 3 of 7 | ~4 hours | Difficulty: Intermediate

1 What it is

Reconciliation is the loop at the heart of GitOps: *observe* the actual state of the cluster, *compare* it with the desired state in Git, and *act* to remove any difference. **Continuous** reconciliation means this never stops - it runs on a timer and on events, not only when someone deploys.

Four ideas make up the topic:

- **Drift** - the cluster no longer matches Git (a manual `kubectl edit`, a controller mutating a field, a resource deleted by accident).
- **Self-heal** - the agent automatically reverts drift back to what Git says.
- **Prune** - the agent deletes cluster resources that were *removed from Git*, so deleting a file really deletes the object.
- **Sync loop** - the repeating cycle: fetch, render, diff, apply, check health, repeat.

2 Why it matters in DevOps

- Without self-heal, "GitOps" is just a nicer deploy button; production can still diverge silently.
- Without **prune**, deleted manifests leave orphan Deployments, Ingresses and Services behind, burning money and creating security holes.
- Over-aggressive reconcile can **fight other controllers** (HPA changing replicas, a mutating webhook injecting sidecars), producing endless "OutOfSync" noise or a sync loop that hammers the API server.
- Knowing which fields to ignore, and when to disable automation during an incident, separates a stable GitOps setup from a noisy one.

3 Real-world system example

Kubernetes' own controllers are the model: the Deployment controller keeps comparing `spec.replicas` with the number of ReplicaSet pods and corrects it - see Deployments and ReplicaSets §8.3. Argo CD and Flux extend it to Git. Argo CD's documentation describes automated sync with the `prune` and `selfHeal` options, and documents `ignoreDifferences` for fields managed by other controllers (a typical example is `spec.replicas` on a Deployment managed by an HPA). Flux's `Kustomization` has `prune: true` and reconciles on its `interval`, and supports `force` to recreate resources with immutable-field changes. Terraform's drift detection (see Drift Detection §7.9) is the same concept applied on demand; GitOps makes it continuous.

4 Diagram

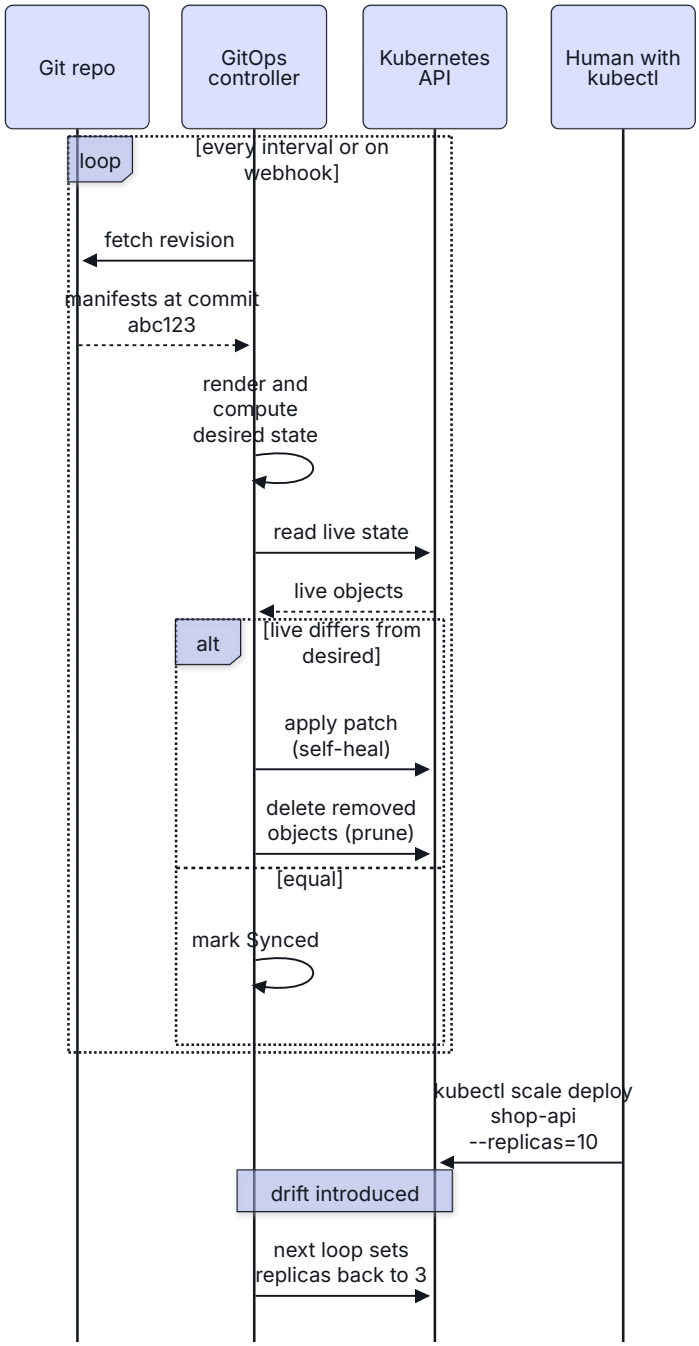


Figure 17.3-1

Sync status and health are two independent axes:

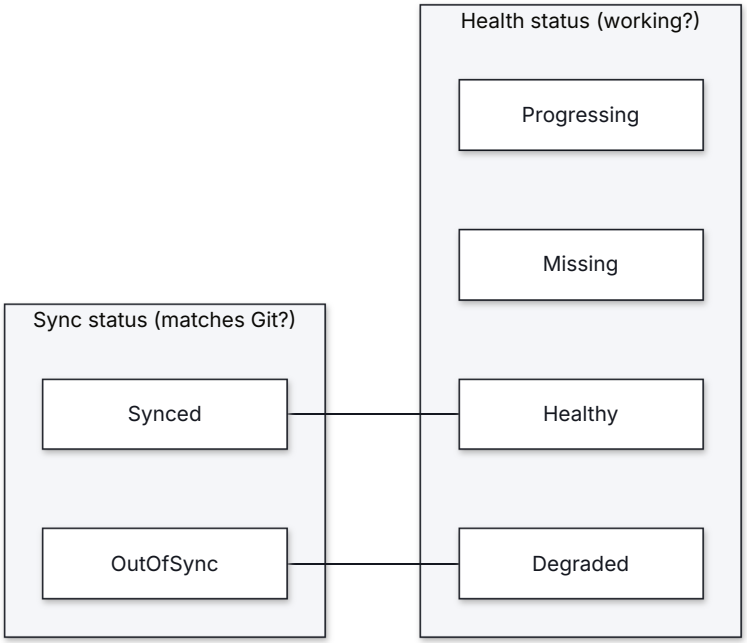


Figure 17.3-2

5 How it works

Argo CD sync policy

```yaml
spec:
  syncPolicy:
    automated:        # without this block, syncs are manual
      prune: true     # delete resources removed from Git
      selfHeal: true  # revert out-of-band changes
    syncOptions:
      - CreateNamespace=true
      - PruneLast=true      # prune only after new resources are healthy
    retry:
      limit: 5
      backoff:
        duration: 5s
        factor: 2
        maxDuration: 3m
```

Setting	What it does	Default
`automated`	sync automatically when Git changes	off (manual)
`automated.prune`	allow auto-sync to delete resources	off (safety)
`automated.selfHeal`	auto-sync when *live* state changes, not just Git	off
`PruneLast=true`	prune after other resources are healthy	off
`Prune=false` (annotation or option)	protect a specific resource from pruning	n/a
`ignoreDifferences`	tell the diff to skip fields owned by someone else	none
`retry`	retry failed syncs with back-off	no retry

Flux equivalents

Concept	Flux
Sync loop period	`spec.interval` on `GitRepository` and `Kustomization`
Self-heal	automatic: each reconcile re-applies desired state (server-side apply)
Prune	`spec.prune: true` on the `Kustomization`
Ignore a field or resource	annotate with `kustomize.toolkit.fluxcd.io/reconcile: disabled`, or use `spec.patches`
Pause everything	`flux suspend kustomization <name>` / `flux resume ...`
Force a reconcile now	`flux reconcile kustomization <name> --with-source`

Why an object can be OutOfSync forever

- **Defaulted fields:** the API server adds defaults the manifest lacks (normally handled by the diff, but some CRDs cause noise).
- **Field owned by another controller:** HPA changes `replicas`; a service mesh injects a sidecar; cert-manager patches a CA bundle. Fix with `ignoreDifferences` (Argo) or by removing the field from the manifest.
- **Non-deterministic rendering:** Helm templates using `randAlphaNum`, or `now`, produce a different output each render. Make them stable.

Refresh versus sync

A **refresh** means "re-read Git and recompute the diff". A **sync** means "apply the diff". Argo CD refreshes about every 3 minutes by default (or immediately via a Git webhook), and compares with the live cache continuously. Hard refresh also invalidates the manifest cache.

6 Step-by-step guide

Prerequisites: the kind cluster with Argo CD from Argo CD and Flux §17.2 and your `shop-gitops` repo on GitHub, with an `Application` named `shop-api-dev` pointing at `apps/shop-api/overlays/dev`. Cost: none.

1. **Turn on automated sync with prune and self-heal.**

```yaml
# apps/shop-api-dev-app.yaml  (apply with kubectl, or keep it in Git under
  clusters/local/)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-api-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
    targetRevision: main
    path: apps/shop-api/overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: shop-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

```bash
kubectl apply -f apps/shop-api-dev-app.yaml
kubectl -n argocd get application shop-api-dev
```

   Expect `Synced Healthy` within a minute or so.

2. **Create drift and watch it get reverted.**

```bash
kubectl -n shop-dev scale deploy/shop-api --replicas=5
kubectl -n shop-dev get deploy shop-api -w
```

   Within seconds the replica count returns to the value in Git (`1` for the dev overlay). Self-heal reacts to live-state changes quickly (it does not wait for the 3-minute Git poll). Press Ctrl-C to stop watching.

3. **Prove prune works.** Remove the Service from the repo and push.

```bash
cd shop-gitops
# remove service.yaml from the base kustomization resources list
sed -i '/service.yaml/d' apps/shop-api/base/kustomization.yaml
git commit -am "chore: drop shop-api service (prune test)" && git push
kubectl -n shop-dev get svc shop-api -w
```

   After the next refresh (use `argocd app get shop-api-dev --refresh` or a UI Refresh to skip the wait) the Service is deleted. Restore it with `git revert HEAD && git push`.

4. **Disable self-heal and see the difference.**

```bash
kubectl -n argocd patch application shop-api-dev --type merge \
  -p '{"spec":{"syncPolicy":{"automated":{"selfHeal":false}}}}'
kubectl -n shop-dev scale deploy/shop-api --replicas=5
kubectl -n argocd get application shop-api-dev
```

   Now the app shows `OutOfSync` and stays at 5 replicas until you sync. (Applying the patch to a live Application that is managed from Git will itself be reverted if the Application is also under GitOps - another reason to change policy in Git.) Re-enable `selfHeal: true` afterwards.

5. **Ignore a field owned by another controller.** Add an HPA-friendly ignore rule:

```yaml
spec:
  ignoreDifferences:
    - group: apps
      kind: Deployment
      jsonPointers:
        - /spec/replicas
```

   With this rule, scaling by hand or by an HPA no longer shows as drift (and self-heal will not fight it).

6. **Watch the controller log the loop.**

```bash
kubectl -n argocd logs deploy/argocd-repo-server --tail=5
kubectl -n argocd logs statefulset/argocd-application-controller --tail=10
```

Log wording varies by version; look for entries mentioning your application name and sync or comparison results. (In older releases the application controller is a Deployment rather than a StatefulSet.)

7 Real-world scenario

Illustrative scenario. After enabling HPA for `shop-api`, the on-call engineer sees the app flapping: HPA scales to 8 replicas for a traffic spike, Argo CD sees `replicas: 3` in Git and "heals" back to 3, the pods get overloaded, the HPA scales again - a tug of war visible as a sawtooth in the replica graph and rising latency. Investigation shows Git still has a hard-coded `replicas: 3`. Fix: remove `replicas` from the Deployment manifest (or add `ignoreDifferences` for `/spec/replicas`), so the HPA owns that field. **Lesson:** decide which controller owns each field; reconcilers that disagree will fight forever. See HPA, VPA and Cluster Autoscaler §16.7.

8 Common mistakes and best practices

Don't

- Enable `prune: true` on a first sync of a namespace that also contains hand-made resources, without checking what would be deleted.
- Fight HPAs, service-mesh injectors or cert-manager over the same field.
- "Fix" drift by disabling auto-sync permanently; find out why the drift exists.
- Set a 10-second poll interval on dozens of apps; use webhooks instead.
- Forget that suspending reconciliation during an incident must be re-enabled afterwards.

Do

- Start with manual sync, review the diff, then turn on automated sync, then prune, then self-heal, in that order.
- Protect critical resources from deletion (`argocd.argoproj.io/sync-options: Prune=false`, PVCs and namespaces in particular).
- Alert on `OutOfSync` longer than N minutes and on sync failures.
- Use `ignoreDifferences` narrowly (a specific kind and JSON pointer), never broadly.
- Keep an incident procedure: suspend reconcile, fix, commit the fix to Git, resume.

9 Tasks

- [ ] **T1 — Trigger self-heal** *(≈20 min, beginner)*

- **Goal:** watch drift correction happen.
- **Steps:**
  1. Enable `automated.selfHeal: true` for `shop-api-dev`.
  2. Scale the Deployment by hand to 5 replicas.
  3. Watch it return to the Git value.
- **Done when:** `kubectl get deploy shop-api -n shop-dev` shows the Git replica count again without you running a sync.

☐ **T2 — Observe a prune** *(≈30 min, beginner)*

- **Goal:** confirm that deleting a file deletes the object.
- **Steps:**
  1. Add a throwaway ConfigMap `scratch` to the base and let it sync.
  2. Delete the file and its entry in `kustomization.yaml`, commit, push.
  3. Refresh and watch the ConfigMap disappear.
- **Done when:** `kubectl -n shop-dev get cm scratch` returns NotFound and the Argo CD tree no longer shows it.

☐ **T3 — Turn self-heal off and compare** *(≈30 min, intermediate)*

- **Goal:** feel the behavioural difference.
- **Steps:**
  1. Set `selfHeal: false` (in Git, not by `kubectl patch`), wait for sync.
  2. Make a manual change; observe `OutOfSync`.
  3. Click Sync (or run `argocd app sync`) to correct it.
- **Done when:** you can show both states side by side and explain when each is appropriate.

☐ **T4 — Break it: the HPA tug of war** *(≈45 min, advanced)*

- **Goal:** reproduce and fix a reconciliation conflict.
- **Steps:**
  1. Install metrics-server on kind (or skip load) and create an HPA for `shop-api` with `minReplicas: 2`.
  2. Keep `replicas: 1` in the Deployment manifest and observe the conflict (`kubectl get hpa,deploy -w`).
  3. Fix it with `ignoreDifferences` on `/spec/replicas`, or remove the field from Git.
- **Done when:** the replica count stays at the HPA's value and the app is `Synced`.

- [ ] **T5 — Protect a resource and add alerting** *(≈45 min, advanced)*
  - **Goal:** guard against accidental prune.
  - **Steps:**
    1. Add the annotation `argocd.argoproj.io/sync-options: Prune=false` to a PVC or ConfigMap in your overlay.
    2. Remove it from Git and confirm it is **not** deleted.
    3. Write (or sketch) a Prometheus alert on Argo CD's `argocd_app_info` metric where `sync_status="OutOfSync"` for 15 minutes.
  - **Done when:** the protected object survives the Git removal and your alert rule parses with `promtool check rules`.

- [ ] **T6 — Prove it: explain the loop** *(≈30 min, advanced)*
  - **Goal:** teach reconciliation to a teammate.
  - **Steps:**
    1. Without the guide, draw the sequence: Git -> controller -> API -> live state, including where drift enters.
    2. Explain refresh versus sync, prune, selfHeal and one case where you would turn each off.
  - **Done when:** a teammate can predict what happens when they `kubectl delete deploy shop-api` in a self-healing namespace.

10 Self-check questions

1. What is the difference between `prune` and `selfHeal`?
2. A resource is OutOfSync immediately after every sync. Name three possible causes.
3. How do HPA and GitOps conflict, and how do you resolve it?
4. Why is prune disabled by default in Argo CD automated sync?
5. How do you pause reconciliation of one Flux Kustomization during an incident?

ANSWERS

1. Prune deletes live resources that no longer exist in Git; self-heal re-applies Git state when the live resource is changed out of band.
2. Another controller mutates a field (HPA, mesh injector, cert-manager), non-deterministic templating, or CRD defaulting differences.
3. HPA owns `replicas` while Git also declares it, so they fight. Remove `replicas` from Git or add `ignoreDifferences` for it.

4. Deleting is destructive; a mistaken path or empty render could delete a whole application, so it is an explicit opt-in.
5. `flux suspend kustomization <name>`, then `flux resume kustomization <name>` when done.

11 Further reading

- Argo CD documentation - "Automated Sync Policy", "Diffing Customization", "Sync Options".
- Flux documentation - "Kustomization" API reference (prune, interval, force) and `flux suspend`.
- Kubernetes documentation - "Controllers" and "Server-Side Apply" (field ownership).
- See also: Continuous Reconciliation of cloud infrastructure with Terraform drift detection §7.9.

LAB NOTES What I ran, what I saw, what it means

17.4 Environment Promotion

Topic 4 of 7 | ~4 hours | Difficulty: Intermediate

1 What it is

Promotion is moving a *known-good version* of an application from one environment to the next - typically **dev -> staging -> prod**. In GitOps, promotion is not a pipeline "deploy to prod" button; it is a **change to files in the GitOps repository** (usually an image tag in an overlay), merged by pull request, which the agent for that environment then reconciles.

The principle is **build once, promote the same artifact**: the container image built and tested for dev is the *identical* image (same digest) that reaches prod. Only configuration differs per environment, and that difference lives in the environment's overlay directory. See Docker builds and registries in CI §5.4.

2 Why it matters in DevOps

- **Confidence:** prod runs exactly what staging validated. Rebuilding for each environment can produce different bits.
- **Control:** a promotion PR is a natural place for approvals, change records and automated checks (smoke test results, security scans).
- **Speed:** automating dev and staging promotion while keeping prod gated gives fast feedback without risk.
- **Traceability:** `git log apps/shop-api/overlays/prod` is a complete deployment history for production.
- A sloppy promotion model (copy-paste YAML, branch-per-environment, `latest` tags) is the most common reason GitOps repos turn into a mess.

3 Real-world system example

Argo CD Image Updater and Flux image automation are the publicly documented tools for automatic tag bumps. Flux's `ImageRepository`, `ImagePolicy` and `ImageUpdateAutomation` objects scan a registry, choose the newest tag matching a policy (for example semantic version-

ing range or timestamp), and **commit the change to Git**, so Git stays the source of truth. Argo CD Image Updater does something similar for Argo CD applications, with a "write-back" method of either committing to Git or setting parameters on the Application (the Git write-back is the GitOps-clean one). Many teams instead use their CI pipeline (GitHub Actions, GitLab CI) to open a pull request against the environment repo. Kargo (an open-source promotion-orchestration tool from Akuity, a company founded by Argo creators) exists specifically because promotion between stages is a distinct problem from syncing. (Treat tool specifics as moving targets and check current docs.)

4 Diagram

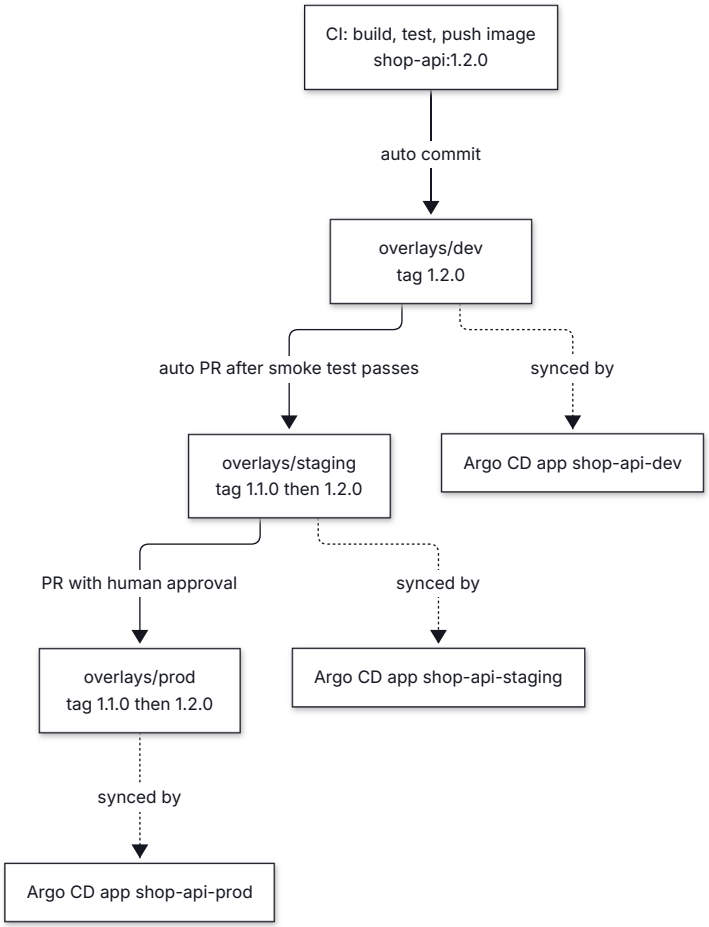


Figure 17.4-1

PR-based promotion in sequence:

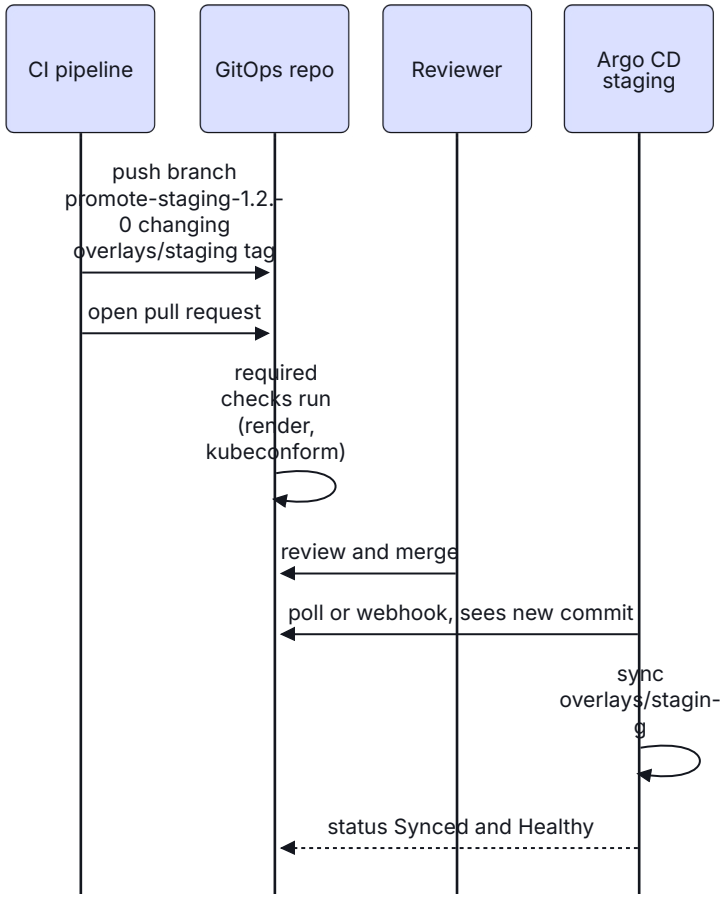


Figure 17.4-2

5 How it works

Three ways to hold per-environment differences

Approach	How	Verdict
Directories + Kustomize overlays	`overlays/dev`, `overlays/staging`, `overlays/prod` each with `images:` and patches	Recommended default: visible, diffable, no merge magic
Helm values per environment	`values-dev.yaml`, `values-prod.yaml` and the same chart	Good if you already use Helm
Branch per environment	`dev`, `staging`, `main` branches, promotion = merge	Avoid: drift, conflicts, hotfix confusion

What changes during promotion

Most promotions modify exactly one value: the image tag (or digest). Kustomize can do that without editing YAML by hand:

```bash
cd apps/shop-api/overlays/staging
kustomize edit set image ghcr.io/example/shop-api=ghcr.io/example/shop-api:1.2.0
```

(`kubectl kustomize` renders but cannot edit; `kustomize edit` needs the standalone `kustomize` binary.) Pin by **digest** (`...@sha256:...`) for stronger immutability: tags can be re-pointed, digests cannot.

Promotion strategies

Strategy	Mechanism	Typical use
Manual PR	engineer edits tag, opens PR	simplest, good for prod
CI-generated PR	pipeline step opens PR on successful dev verification	common and auditable
Direct commit by bot	CI or Image Updater commits to the dev overlay	dev only
Image automation	Flux `ImageUpdateAutomation` / Argo CD Image Updater	fast-moving dev/staging
Orchestrator	Kargo or similar models "stages" and promotion rules	many services, many stages

Gates between environments

A promotion PR should carry evidence: tests that passed in the previous environment, vulnerability scan results, and (for prod) a required approver. Protect the prod overlay path with CODEOWNERS so only a release owner can approve changes there. See Protected Branches and Pull Requests §3.8.

Argo CD Image Updater (concept)

An annotation-driven add-on that watches the registry and writes new tags back to Git. A typical configuration (annotation style used by classic Image Updater versions; newer releases are moving towards a CRD-based configuration, so check the current docs):

```yaml
metadata:
  annotations:
    argocd-image-updater.argoproj.io/image-list: shop=ghcr.io/example/shop-api
    argocd-image-updater.argoproj.io/shop.update-strategy: semver
    argocd-image-updater.argoproj.io/write-back-method: git
    argocd-image-updater.argoproj.io/git-branch: main
```

6 Step-by-step guide

Prerequisites: the `shop-gitops` repo from Desired State Management §17.1, kind cluster with Argo CD from Argo CD and Flux §17.2, `git`, and optionally the standalone `kustomize` and `gh` (GitHub CLI) binaries. Cost: none.

1. **Add the staging overlay (if you skipped T2 earlier).**

```bash
cat > apps/shop-api/overlays/staging/kustomization.yaml <<'EOF'
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-staging
resources:
  - ../../base
replicas:
  - name: shop-api
    count: 2
images:
  - name: ghcr.io/example/shop-api
    newTag: 1.0.0
EOF
```

2. **Create one Argo CD Application per environment.** Keep them in `clusters/local/`:

```yaml
# clusters/local/shop-api-staging.yaml  (dev and prod are identical except name, path
  and namespace)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-api-staging
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
    targetRevision: main
    path: apps/shop-api/overlays/staging
  destination:
    server: https://kubernetes.default.svc
    namespace: shop-staging
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

```bash
for e in dev staging prod; do sed "s/staging/$e/g" clusters/local/shop-api-
  staging.yaml | kubectl apply -f -; done
kubectl -n argocd get applications
```

Expect three applications. (For prod, consider leaving `automated` out so a human clicks Sync.)

3. **Promote dev to staging with a pull request.**

```bash
git switch -c promote-staging-1.1.0
cd apps/shop-api/overlays/staging
kustomize edit set image ghcr.io/example/shop-api=ghcr.io/example/shop-api:1.1.0
cd ../../../..
git diff
git commit -am "promote shop-api 1.1.0 to staging"
git push -u origin promote-staging-1.1.0
gh pr create --fill
```

Expect a diff of a single `newTag` line. Merge the PR; Argo CD picks it up within the poll interval.

4. **Verify what is running in each environment.**

```bash
for ns in shop-dev shop-staging shop-prod; do
  echo -n "$ns: "; kubectl -n $ns get deploy shop-api -o
  jsonpath='{.spec.template.spec.containers[0].image}'; echo
done
```

(Illustrative output: `shop-dev: ghcr.io/example/shop-api:1.1.0`, `shop-staging: ...:1.1.0`, `shop-prod: ...:1.0.0`. Note the images are fictional; use any public image such as `nginx` tags in your sandbox so pods actually start.)

5. **Automate the staging promotion in CI** (sketch, GitHub Actions in the *GitOps* repo, run after a dev smoke test):

```yaml
name: promote-to-staging
on:
  workflow_dispatch:
    inputs:
      tag:
        description: Image tag to promote
        required: true
jobs:
```

```
  promote:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      pull-requests: write
    steps:
      - uses: actions/checkout@v4
      - name: Bump tag
        run: |
          cd apps/shop-api/overlays/staging
          kustomize edit set image ghcr.io/example/shop-api=ghcr.io/example/shop-
  api:${{ inputs.tag }}
      - name: Open PR
        run: |
          git config user.name "promotion-bot"
          git config user.email "bot@example.com"
          git switch -c "promote-staging-${{ inputs.tag }}"
          git commit -am "promote shop-api ${{ inputs.tag }} to staging"
          git push -u origin HEAD
          gh pr create --fill
        env:
          GH_TOKEN: ${{ github.token }}
```

Pass `inputs.tag` through an environment variable rather than inline `${{ }}` in `run:` if the value could be attacker-influenced (script injection). The runner image includes `kustomize`; confirm for your runner version.

6. **Protect production with CODEOWNERS.**

```
# .github/CODEOWNERS
/apps/shop-api/overlays/prod/ @example/release-managers
```

Combine with a branch protection rule requiring code-owner review.

7 Real-world scenario

Illustrative scenario. A team rebuilds the image separately for each environment from the `release` branch. A base-image update lands between the staging and prod builds; prod gets a subtly different OpenSSL, and a TLS handshake bug appears only in production. After switching to build-once-promote-by-digest, the prod overlay pins `sha256:...` identical to what staging ran, and the release checklist becomes "open a PR that copies staging's digest into prod". **Lesson:** what you tested must be bit-for-bit what you ship.

8 Common mistakes and best practices

Don't

- Rebuild images per environment, or use `latest` /mutable tags in overlays.
- Promote by merging branches or copying whole directories.
- Let Image Updater write to prod automatically without a gate.
- Give the promotion bot more permissions than "open PR on the env repo".
- Put secrets or environment credentials inside the overlay as plain text.

Do

- Keep config differences minimal and explicit in each overlay.
- Promote through the same path every time: PR -> checks -> approval -> merge -> sync.
- Include evidence (links to test runs, scan results) in the promotion PR.
- Use digests where you can, and record the source tag in the commit message.
- Make the same promotion tooling work for hotfixes so nobody bypasses it at 3 a.m.

9 Tasks

- [ ] **T1 — Three environments from one base** *(≈45 min, beginner)*
  - **Goal:** run dev, staging and prod Applications side by side.
  - **Steps:**
    1. Follow steps 1-2 of the guide.
    2. Confirm three namespaces exist.
  - **Done when:** `kubectl get applications -n argocd` lists three Synced Applications and the Deployments differ in replicas.
- [ ] **T2 — Promote by pull request** *(≈30 min, beginner)*
  - **Goal:** move a version from dev to staging via PR.
  - **Steps:**
    1. Change the dev tag to `1.1.0` (use a real public image such as `nginx:1.27` so pods start).
    2. Promote it to staging with a PR as in step 3.
  - **Done when:** the merged PR is the only change and the staging Deployment runs the new tag.
- [ ] **T3 — Pin by digest** *(≈30 min, intermediate)*

- **Goal:** use an immutable reference.
- **Steps:**
  1. Find a digest (`docker buildx imagetools inspect nginx:1.27` or `crane digest nginx:1.27`).
  2. Set the overlay image `digest:` instead of `newTag:` (Kustomize `images` supports `digest`).
  3. Render and confirm `image: nginx@sha256:...`.
- **Done when:** the pod runs from the digest reference and `kubectl describe pod` shows it.

☐ **T4 — Break it: promote a bad image** *(≈30 min, intermediate)*

- **Goal:** see what a failed promotion looks like.
- **Steps:**
  1. Promote a non-existent tag to staging.
  2. Watch the app go `Progressing` / `Degraded` with `ImagePullBackOff`.
  3. Fix by reverting the PR (preview of Rollback Through Git §17.6).
- **Done when:** staging is Healthy again and the bad commit is visible in history next to its revert.

☐ **T5 — Automate promotion** *(≈60 min, advanced)*

- **Goal:** a workflow that opens the staging PR for you.
- **Steps:**
  1. Create the workflow from step 5 in your GitOps repo.
  2. Run it with a tag input.
  3. Add CODEOWNERS for the prod path and verify a PR touching prod requires the owner.
- **Done when:** the workflow creates a PR with a one-line diff and the prod path shows "review required from code owners".

☐ **T6 — Prove it: design a promotion policy** *(≈30 min, advanced)*

- **Goal:** write a short policy for a team.
- **Steps:**
  1. Without notes, write which promotions are automatic, which need approval, and who may approve prod.
  2. Explain why branch-per-environment is avoided and how a hotfix flows.

- **Done when:** your policy fits on half a page and names the gate, the evidence and the rollback path for each stage.

10 Self-check questions

1. What does "build once, promote the same artifact" mean, and why does it matter?
2. Why use overlay directories rather than branches per environment?
3. Where should automated tag bumps happen, and why must they be committed to Git?
4. Why pin a digest rather than a tag?
5. How do you require extra approval only for prod changes?

ANSWERS

1. The same image digest moves through all environments; only config changes. What you tested is what you run.
2. Directories are visible side by side, diffable and never need merging; branches diverge and hide differences.
3. Automatically in dev (and maybe staging), as commits or PRs to the GitOps repo, so Git remains the source of truth and the change is auditable.
4. A tag can be re-pushed to point to different content; a digest is content-addressed and cannot change.
5. CODEOWNERS for `overlays/prod/` plus branch protection requiring code-owner review.

11 Further reading

- Kustomize documentation - `images`, `replicas`, `kustomize edit set image`.
- Argo CD Image Updater documentation (argocd-image-updater.readthedocs.io) and Flux "Automate image updates to Git" guide (fluxcd.io).
- Kargo documentation (kargo.io) - stage-based promotion for Argo CD.
- See also: Deployment Strategies §0.7 and Canary Deployments §5.12.

17.5 Application Synchronization

Topic 5 of 7 | ~5 hours | Difficulty: Advanced

1 What it is

A **sync** is the act of making the cluster match the desired state for one application. Real applications are not a flat bag of YAML: a database migration must finish before the new API starts, a CRD must exist before the custom resources that use it, and a smoke test should run after everything is up. Argo CD gives you four tools to control this ordering and safety:

- **Sync waves** - numeric ordering of resources inside one sync (`argocd.argoproj.io/sync-wave`).
- **Resource hooks** - Jobs or Pods that run at a phase of the sync (`PreSync` , `Sync` , `PostSync` , `SyncFail`).
- **Sync policies and options** - when to sync automatically and how (`automated` , `retry` , `ServerSideApply` , `Replace` , ...).
- **Health assessment** - Argo CD's built-in (and custom) checks for whether a resource is actually working, not merely applied.

At fleet scale two more patterns matter: **App of Apps** (an Application whose contents are other Applications) and **ApplicationSets** (a controller that *generates* Applications from a template and a list, directory or cluster generator). Flux achieves ordering with `dependsOn` between Kustomizations and `healthChecks` .

2 Why it matters in DevOps

- Without ordering, first installs fail randomly: pods start before their ConfigMap, custom resources are rejected because the CRD is not there yet.
- Database migrations in the delivery flow need a safe, once-per-release home. A `PreSync` hook Job is the GitOps answer. See Database Migrations in Pipelines §5.9.
- "Applied" is not "working": an application that is `Synced` but `Degraded` must stop a promotion.

- A platform with 40 services and 3 environments is 120 Applications; hand-writing them does not scale, ApplicationSets do.

3 Real-world system example

Argo CD's own documentation uses **sync waves and hooks** for exactly these cases (CRD before CR, namespace before workloads, `PreSync` DB migration, `PostSync` notification). The widely used pattern of bootstrapping a whole cluster - ingress controller, cert-manager, monitoring stack, then business apps - with an **App of Apps** root is described in Argo CD's "Cluster Bootstrapping" documentation. **ApplicationSets** are part of Argo CD itself (formerly a separate project) and are used by platform teams to create one Application per directory in a repo, per cluster, or per pull request (the *pull request generator* creates preview environments). Flux users get similar results with a layered set of `Kustomization` objects linked by `dependsOn`.

4 Diagram

Order of a sync with hooks and waves:

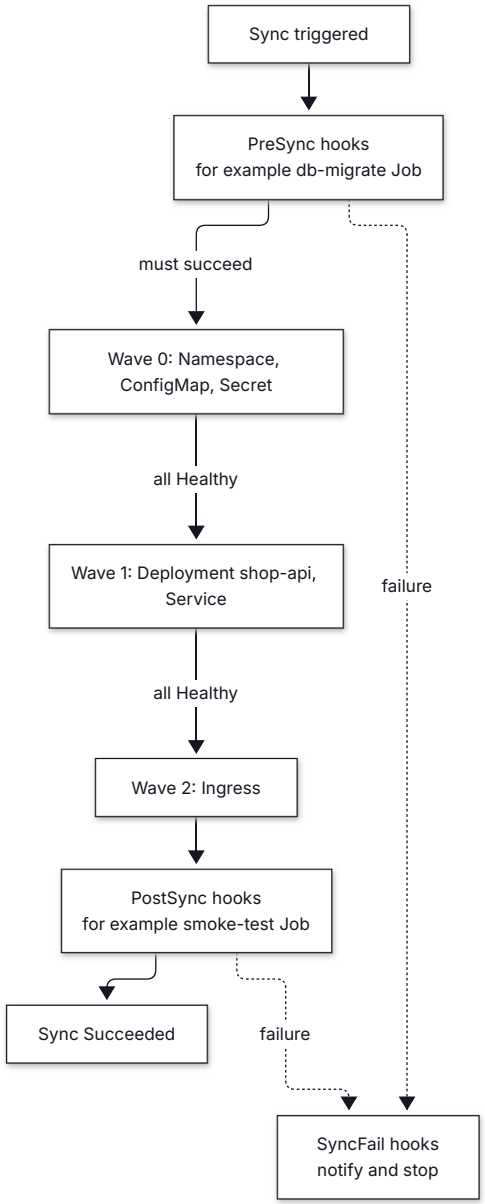


Figure 17.5-1

ApplicationSet generating Applications:

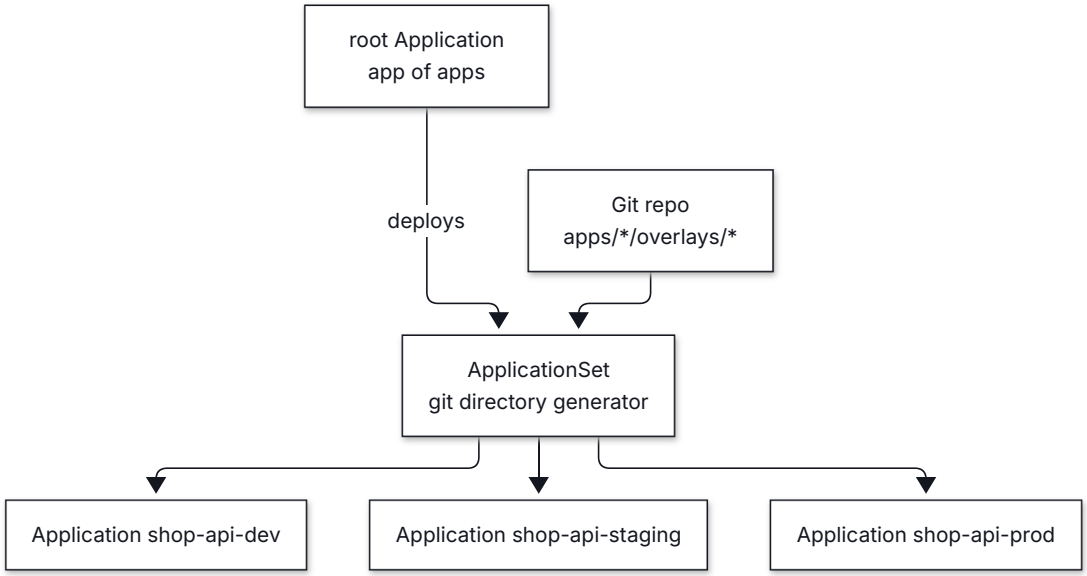


Figure 17.5-2

5 How it works

Sync waves

Resources carry an integer annotation; lower waves apply first and Argo CD waits for each wave to be **Healthy** before starting the next (default wave `0`, negative allowed). Within a wave, a built-in order applies (namespaces, then CRDs, then others).

```yaml
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "-1"   # before the default wave 0
```

Hooks

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
```

```
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: ghcr.io/example/shop-api:1.1.0
          command: ["node", "scripts/migrate.js"]
```

Hook	Runs	Typical use
`PreSync`	before manifests are applied	DB migrations, backup
`Sync`	together with the apply (ordered by waves)	complex install steps
`PostSync`	after all resources are Healthy	smoke tests, notifications
`SyncFail`	when the sync fails	cleanup, alerting
`PostDelete`	after the app is deleted (recent versions)	cleanup of external resources

Delete policies: `HookSucceeded`, `HookFailed`, `BeforeHookCreation` (recreate on every sync, the usual choice, avoids name clashes). Make migrations **idempotent and backward compatible**: a PreSync Job can run again on a retry or rollback.

Sync options worth knowing

Option	Purpose
`CreateNamespace=true`	create the destination namespace
`ServerSideApply=true`	use server-side apply (needed for huge CRDs, helps co-ownership of fields)
`Replace=true`	`kubectl replace` instead of `apply` (careful; can disrupt)
`PruneLast=true`	prune after everything else is healthy
`ApplyOutOfSyncOnly=true`	only touch resources that differ (big apps, faster syncs)
`Validate=false`	skip schema validation (use only to work around CRD ordering)
`SkipDryRunOnMissingResource=true`	allow CRs whose CRD is created in the same sync

Health assessment

Argo CD ships health checks for built-in kinds (Deployment is Healthy when its rollout is complete, a Service of type LoadBalancer when it has an address, a PVC when `Bound`, and so on). Statuses: `Healthy`, `Progressing`, `Degraded`, `Suspended`, `Missing`, `Unknown`. For custom

resources you can supply a **Lua health script** in the `argocd-cm` ConfigMap (`resource.customizations.health.<group>_<kind>`). Wave advancement depends on health, so a good check makes ordering trustworthy.

App of Apps

One "root" Application points to a directory containing other Application manifests. Deploying the root bootstraps everything. It is simple and visible; its drawback is that adding the 30th app means writing another YAML file.

```yaml
# clusters/local/root-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
    targetRevision: main
    path: clusters/local/apps        # contains Application manifests only
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

ApplicationSet with the Git directory generator

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: shop-api-envs
  namespace: argocd
spec:
  goTemplate: true
  goTemplateOptions: ["missingkey=error"]
  generators:
    - git:
        repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
        revision: main
        directories:
          - path: apps/shop-api/overlays/*
  template:
```

```
    metadata:
      name: 'shop-api-{{ .path.basename }}'
    spec:
      project: default
      source:
        repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
        targetRevision: main
        path: '{{ .path.path }}'
      destination:
        server: https://kubernetes.default.svc
        namespace: 'shop-{{ .path.basename }}'
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
```

Add a directory `overlays/qa` and an Application `shop-api-qa` appears without touching anything else. Other generators: `list`, `cluster` (one app per registered cluster), `matrix` and `merge` (combine generators), and `pullRequest` (preview environments).

Flux equivalent of ordering

YAML

```yaml
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: shop-api
  namespace: flux-system
spec:
  interval: 5m
  path: ./apps/shop-api/overlays/prod
  prune: true
  sourceRef:
    kind: GitRepository
    name: shop-gitops
  dependsOn:
    - name: infrastructure      # ingress, cert-manager, CRDs
  healthChecks:
    - apiVersion: apps/v1
      kind: Deployment
      name: shop-api
      namespace: shop-prod
  timeout: 3m
```

6 Step-by-step guide

Prerequisites: kind cluster with Argo CD (topic 2 §17.2) and your `shop-gitops` repo on GitHub with the three overlays from topic 4 §17.4. Use a real public image (for example `nginx`) in the overlays so pods start. Cost: none.

1. **Add a PreSync hook and a PostSync smoke test to the base.** For the sandbox, use a harmless `busybox` image.

```bash
cat > apps/shop-api/base/presync-job.yaml <<'EOF'
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: busybox:1.36
          command: ["sh", "-c", "echo running migrations && sleep 5"]
EOF
cat > apps/shop-api/base/postsync-job.yaml <<'EOF'
apiVersion: batch/v1
kind: Job
metadata:
  name: smoke-test
  annotations:
    argocd.argoproj.io/hook: PostSync
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: smoke
          image: busybox:1.36
          command: ["sh", "-c", "echo smoke test passed"]
EOF
```

   Add both files under `resources:` in `apps/shop-api/base/kustomization.yaml`, commit and push.

2. **Watch the ordered sync.**

```bash
kubectl -n shop-dev get jobs -w
```

   Expected (similar to): `db-migrate` appears and completes first, then the Deployment rolls out, then `smoke-test` runs. In the UI the sync operation view lists hooks with their phase. Because of `HookSucceeded`, completed Jobs disappear.

3. **Order two resources with waves.** Create a `Namespace` with wave `-1` and a ConfigMap with wave `0`, or annotate the Deployment with wave `"1"` and a ConfigMap with wave `"0"`; then trigger a sync and confirm ordering in the sync result.

4. **Replace the three hand-written Applications with an ApplicationSet.** Delete `shop-api-dev/staging/prod` Applications (`kubectl -n argocd delete application ...`), save the ApplicationSet from section 5 as `clusters/local/shop-api-appset.yaml`, apply it:

```bash
kubectl apply -f clusters/local/shop-api-appset.yaml
kubectl -n argocd get applications
```

   Expect `shop-api-dev`, `shop-api-staging`, `shop-api-prod` to reappear, generated. (If you apply the ApplicationSet while the old Applications exist, ownership conflicts occur; delete the old ones first.)

5. **Bootstrap with App of Apps.** Put the ApplicationSet manifest under `clusters/local/apps/`, apply only `root-app.yaml` from section 5, and confirm the root app creates the ApplicationSet which creates the three Applications.

```bash
kubectl apply -f clusters/local/root-app.yaml
kubectl -n argocd get applications,applicationsets
```

6. **Add an environment by adding a directory.** Copy `overlays/staging` to `overlays/qa`, edit the namespace, push, and wait. An Application `shop-api-qa` appears. Delete the directory to see it (and, with prune, its resources) go away. Production note: set `preserveResourcesOnDeletion` or use a `deletionPolicy` carefully, since removing a generator entry deletes the generated Application.

7 Real-world scenario

Illustrative scenario. A release of `shop-api` adds a `NOT NULL` column. The CI/CD pipeline used to run `migrate` before `kubectl apply`, but after the move to GitOps nobody owns that ordering: Argo CD rolls the new Deployment first, pods crash on the missing column, and a sync loop of failing pods ensues. The team moves the migration into a `PreSync` Job with `BeforeHookCreation`, makes the migration backward compatible (add nullable column first, enforce later), and adds a `PostSync` smoke test. A failing migration now blocks the sync and leaves the old version serving. **Lesson:** express ordering in the sync mechanism, and design migrations so old and new code both work for a release.

8 Common mistakes and best practices

Don't

- Put long-running or non-idempotent work in hooks without a `backoffLimit` and delete policy.
- Use waves as a substitute for readiness probes; waves wait on health, which depends on probes and health checks.
- Hand-write 100 Application YAMLs when a generator can produce them.
- Forget that deleting an ApplicationSet (or its generator entry) can delete the generated Applications and, via finalizers, their resources.
- Use `Replace=true` or `Validate=false` globally "to make errors go away".

Do

- Use `PreSync` for migrations and `PostSync` for verification, with `BeforeHookCreation` delete policy.
- Use `AppProject`s to restrict what generated Applications may deploy where.
- Write Lua health checks for custom resources that matter (databases, certificates).
- Keep the root app and ApplicationSets themselves in Git and reviewed.
- Roll out new generators on a test cluster first; a template bug fans out to every generated app.

9 Tasks

- [ ] **T1 — Observe hook ordering** *(≈30 min, beginner)*
  - **Goal:** see PreSync, apply and PostSync happen in order.
  - **Steps:**

1. Add the two Jobs from step 1 to the base.
2. Push and watch `kubectl -n shop-dev get jobs,pods -w`.

- **Done when:** the sync result in the UI lists `db-migrate` (PreSync) completing before the Deployment and `smoke-test` (PostSync) after.

☐ **T2 — Order resources with waves** *(≈30 min, intermediate)*

- **Goal:** control the apply order.
- **Steps:**
  1. Annotate a ConfigMap with wave `"0"` and the Deployment with wave `"1"`, and a Namespace with wave `"-1"`.
  2. Delete the namespace and resync with `argocd app sync` (or UI).
  3. Review the order in the sync operation details.
- **Done when:** the operation timeline shows wave -1 first, then 0, then 1.

☐ **T3 — Generate the environments** *(≈45 min, intermediate)*

- **Goal:** replace hand-written Applications with an ApplicationSet.
- **Steps:**
  1. Apply the ApplicationSet from section 5.
  2. Add `overlays/qa` and push.
  3. Observe `shop-api-qa` created automatically.
- **Done when:** `kubectl -n argocd get applications` shows four generated Applications with no manual Application YAML.

☐ **T4 — Break it: failing PreSync** *(≈30 min, intermediate)*

- **Goal:** confirm failed hooks block a release safely.
- **Steps:**
  1. Change the migrate command to `exit 1`.
  2. Push a new image tag to dev in the same commit.
  3. Observe the sync fail and the old Deployment keep serving.
  4. Fix the command (or `git revert`).
- **Done when:** the Application shows a failed operation with the hook error, the old pods are still running, and the fix restores sync.

☐ **T5 — Bootstrap with app of apps** *(≈45 min, advanced)*

- **Goal:** rebuild the whole environment from one command.
- **Steps:**

1. Delete all your Applications and the ApplicationSet.
2. Apply only `root-app.yaml`.
3. Time how long until all environments are Healthy.

- **Done when:** one `kubectl apply` restores everything, demonstrating cluster rebuild-from-Git.

☐ **T6 — Prove it: explain a sync** *(≈30 min, advanced)*

- **Goal:** describe exactly how a release is applied.
- **Steps:**
  1. Without notes, draw the sequence PreSync -> waves -> PostSync, marking where health gates apply.
  2. Say what happens if wave 1 never becomes Healthy, and what you would check first.
- **Done when:** your explanation names probes and health assessment as the reason the sync waits (or fails).

10 Self-check questions

1. What decides when Argo CD moves from one sync wave to the next?
2. Why use `BeforeHookCreation` as a hook delete policy?
3. When would you choose ApplicationSet over App of Apps?
4. What is the difference between sync status and health status?
5. How do you express ordering between two Flux Kustomizations?

ANSWERS

1. All resources in the current wave being applied and reporting Healthy.
2. It deletes the previous hook object before creating a new one, avoiding name collisions on re-sync.
3. When Applications follow a pattern (per directory, cluster, environment or pull request); App of Apps suits a small, hand-curated set and cluster bootstrap. They are often combined.
4. Sync status says whether live matches Git; health status says whether the resources are actually working.
5. `spec.dependsOn` on the dependent Kustomization, optionally with `healthChecks` on the dependency.

11 Further reading

- Argo CD documentation - "Sync Waves", "Resource Hooks", "Sync Options", "Resource Health", "Cluster Bootstrapping".
- Argo CD ApplicationSet documentation - generators (git, list, cluster, matrix, pullRequest).
- Flux documentation - Kustomization `dependsOn` and `healthChecks`.
- See also: Jobs and CronJobs §8.12 and Liveness and Readiness Probes §8.15.

LAB NOTES What I ran, what I saw, what it means

17.6 Rollback Through Git

Topic 6 of 7 | ~3 hours | Difficulty: Intermediate

1 What it is

In GitOps the way you roll back production is the way you changed it: **with a commit**. Because the repo is the source of truth, "go back to the last good version" means making the repo say the last good version again. The safest way is `git revert`: it creates a *new* commit that undoes an earlier commit, leaving history intact. The agent then reconciles the cluster to the reverted state, exactly as it would for any other change.

This contrasts with tool-level rollbacks (`kubectl rollout undo`, `helm rollback`, or Argo CD's UI **Rollback** button), which change the cluster *without* changing Git. Under GitOps those are emergency measures that create **drift**, because Git still says the bad version.

2 Why it matters in DevOps

- A rollback you can do in under five minutes, with a review trail, is the best protection against a bad release. See Rollback §5.7 in CI/CD.
- If the cluster is rolled back but Git is not, the next sync (or self-heal) re-deploys the broken version, often at the worst moment.
- Auditors and post-incident reviews need to know who reverted what and why; a revert commit with a message and a PR answers that.
- A clear **emergency procedure** keeps people from making things worse under pressure.

3 Real-world system example

Argo CD documents that its UI/CLI rollback (`argocd app rollback`) **requires automated sync to be disabled**, because with auto-sync on the controller would immediately re-apply Git's version. That is the tool telling you the intended model: Git is authoritative, and a cluster-only rollback is a temporary override. Flux has no dedicated rollback command for Kustomizations as of this writing - you revert in Git (its `HelmRelease` can also be configured to roll back automatically on failed upgrades via the `remediation` settings of the upgrade).

Teams that run GitOps at scale, as described in many conference talks and engineering blogs, treat "revert the PR" as the standard rollback and measure it as part of their mean time to recovery (MTTR). The Git side relies on the primitives from reset and revert §3.4.

4 Diagram

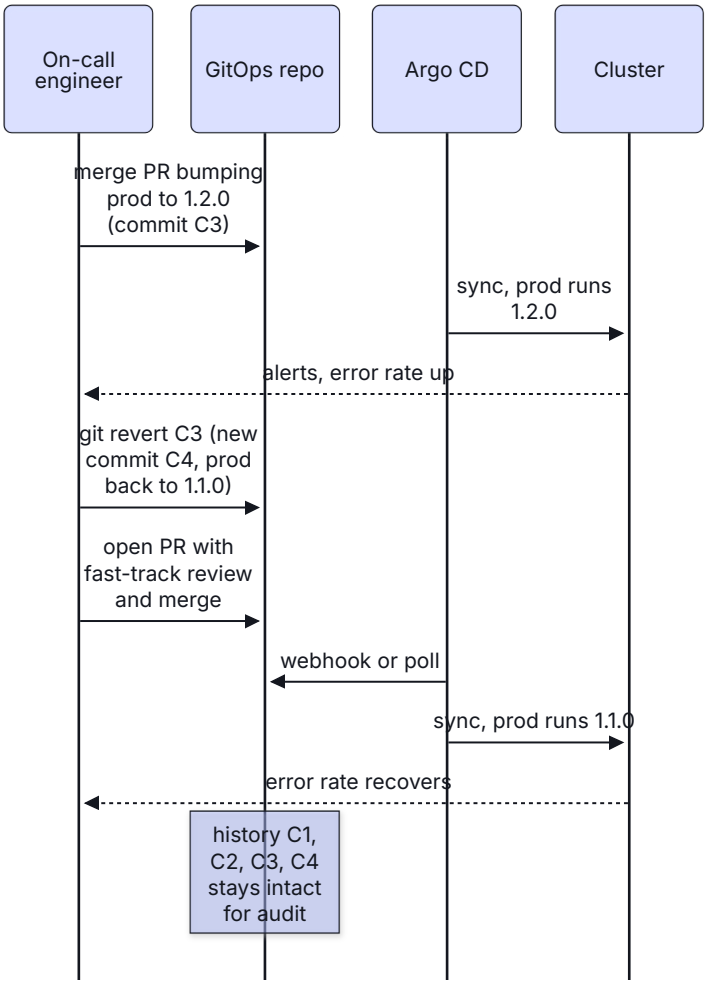


Figure 17.6-1

Choosing a rollback method:

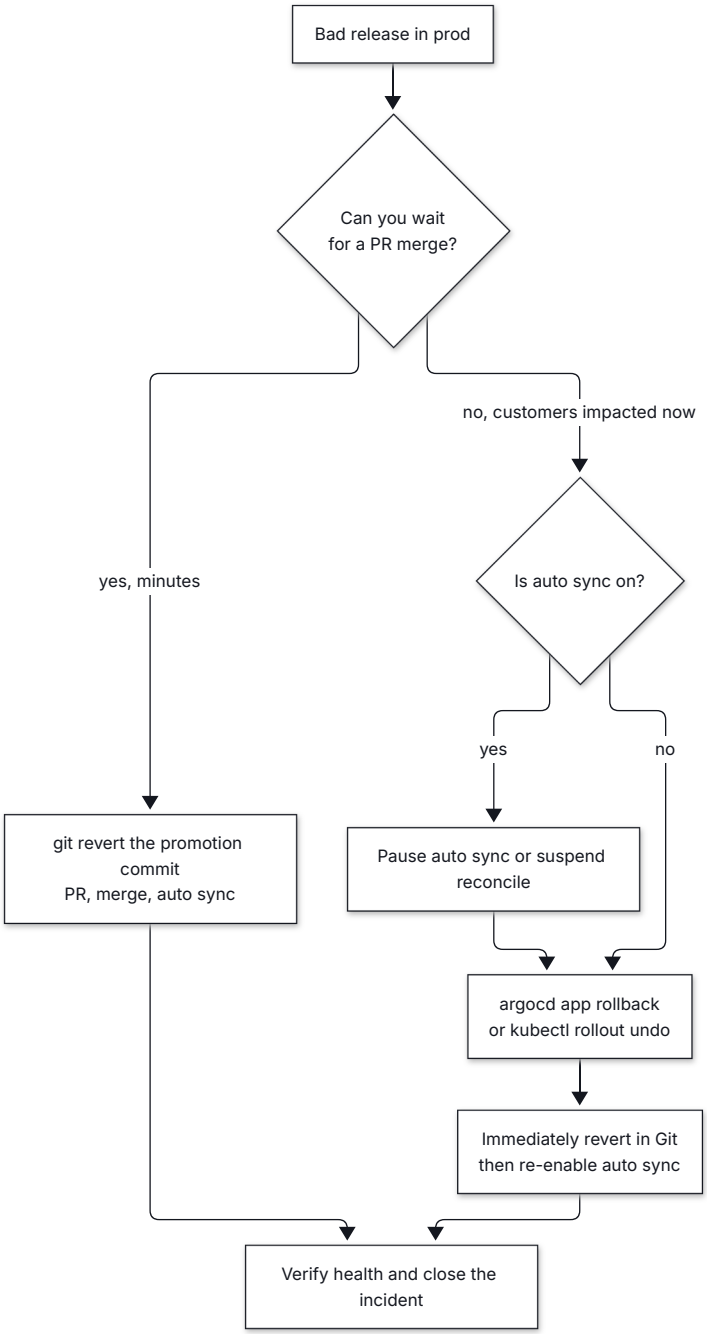


Figure 17.6-2

5 How it works

`git revert` versus `git reset`

	`git revert <sha>`	`git reset --hard <sha>` + force push
History	adds a new inverse commit	rewrites history, drops commits
Safe on shared `main`	yes	no, breaks everyone's clones and the audit trail
Works with protected branches	yes (it is a normal PR)	blocked, and should be
Audit	shows "Revert X" with reason	evidence disappears

Always revert on shared branches. For a merge commit use `git revert -m 1 <merge-sha>` (keep mainline parent 1). If the promotion was a squashed PR, there is a single commit to revert.

Revert one environment only

Because each environment is a directory, a revert of "promote 1.2.0 to prod" touches only `overlays/prod`. If a commit changed several environments at once, create the inverse commit manually for just the path you need:

```bash
# restore just prod's files to their state before commit <sha>
git checkout <sha>~1 -- apps/shop-api/overlays/prod
git commit -m "revert: roll prod back to 1.1.0 (incident INC-1234)"
```

This is still a normal forward commit (history is not rewritten), so it works with protected branches and reviews.

Emergency procedure (when the PR process is too slow)

1. **Stop the bleeding** in the cluster: pause auto-sync for the one app (`argocd app set shop-api-prod --sync-policy none`) or `flux suspend kustomization shop-api-prod`.
2. **Roll the cluster back**: `argocd app rollback shop-api-prod <ID>` (see `argocd app history`), or `kubectl rollout undo deploy/shop-api -n shop-prod`.
3. **Fix Git immediately**: revert the promotion commit and merge it, using a pre-approved break-glass path (an on-call admin may bypass review but the merge is still logged).
4. **Re-enable automation** so the cluster follows Git again: `argocd app set shop-api-prod --sync-policy automated --self-heal --auto-prune`, or `flux resume ...`.
5. **Write the post-mortem**: why the bad release got through and why rollback took as long as it did. See Postmortems §14.7.

Step 3 must happen before step 4, or re-enabling automation re-deploys the bad version.

Rolling back data and migrations

Git can restore *manifests*, not *data*. If a release ran a destructive migration, reverting the Deployment does not restore dropped columns. Design migrations as expand-then-contract and keep tested backups (Backups and PITR §13.5).

Roll forward is often better

If the fix is a one-line change and CI is quick, a **forward fix** (new commit, new image) can beat a revert. A revert is the right tool when you do not yet understand the problem; it has no unknowns.

6 Step-by-step guide

Prerequisites: the three Applications from earlier topics (from an ApplicationSet or by hand) with `shop-api-prod` using a real image such as `nginx`, and your `shop-gitops` repo on GitHub. Cost: none. Output below is illustrative.

1. **Record the known-good state.**

   ```bash
   git switch main && git pull
   git log --oneline -n 5 -- apps/shop-api/overlays/prod
   kubectl -n shop-prod get deploy shop-api -o
     jsonpath='{.spec.template.spec.containers[0].image}'; echo
   ```

   Note the current prod tag (for example `nginx:1.26`).

2. **Ship a "bad" release by promotion.** Promote a tag that cannot start:

   ```bash
   git switch -c promote-prod-bad
   cd apps/shop-api/overlays/prod
   kustomize edit set image ghcr.io/example/shop-api=nginx:does-not-exist
   cd ../../../.. && git commit -am "promote shop-api to prod: broken tag" && git push -u
     origin HEAD
   gh pr create --fill && gh pr merge --squash --admin
   ```

   (The `--admin` flag is only so you can self-merge in a sandbox. Substitute your own tag handling.)

3. **Observe the failure.**

```bash
kubectl -n shop-prod get pods
kubectl -n argocd get application shop-api-prod
```

Expect a new pod in `ErrImagePull` / `ImagePullBackOff`, the old pods still serving (a rolling update keeps the old ReplicaSet until the new one is ready), and the application `Degraded` or `Progressing`.

4. **Revert in Git.**

```bash
git switch main && git pull
git revert --no-edit HEAD        # HEAD is the squashed promotion commit
git switch -c revert-prod-bad && git push -u origin HEAD
gh pr create --fill --title "Revert: promote shop-api to prod (broken tag)" && gh pr
  merge --squash --admin
```

5. **Watch the cluster converge.**

```bash
kubectl -n argocd get application shop-api-prod -w
```

Expect `Synced` and `Healthy` once the sync runs (immediately if you use a webhook, otherwise within the poll interval; speed it with `argocd app get shop-api-prod --refresh`).

6. **Practise the emergency path.** Repeat steps 2-3, then without waiting for a PR:

```bash
argocd app set shop-api-prod --sync-policy none        # stop auto sync
argocd app history shop-api-prod                         # find the previous ID
argocd app rollback shop-api-prod <ID>
```

Now fix Git (step 4) and re-enable: `argocd app set shop-api-prod --sync-policy automated --self-heal --auto-prune`. Verify `git log` and the cluster agree.

7. **Show the audit trail.**

```bash
git log --oneline --decorate -n 6 -- apps/shop-api/overlays/prod
```

You should see the promotion, then `Revert "promote shop-api to prod: broken tag"`, each with an author and a PR number.

7 Real-world scenario

Illustrative scenario. At 14:05 the `shop-api` 1.2.0 promotion to production is merged. By 14:12 the checkout error rate alert fires. The on-call does not touch the cluster: they run `git revert` on the promotion commit, open the PR, a second engineer approves in two minutes, and Argo CD (with a webhook) syncs prod back to 1.1.0 at 14:19. The Slack incident channel links the revert PR as the record of the action. Contrast with last year's incident, when someone ran `kubectl rollout undo`, Argo CD's self-heal "fixed" it by redeploying the broken version ten minutes later, and the outage doubled. **Lesson:** do the rollback where the system is authoritative; if you must act in the cluster first, pause the reconciler.

8 Common mistakes and best practices

Don't

- Run `kubectl rollout undo` or click Rollback in the UI with auto-sync/self-heal enabled and leave Git unchanged.
- Force-push or `git reset` the GitOps `main` branch to "remove" a bad commit.
- Forget to re-enable auto-sync after an emergency.
- Assume a revert undoes data or schema changes.
- Wait for a perfect post-mortem before reverting; recover first, analyse after.

Do

- Make `git revert` of a promotion PR the documented default rollback.
- Keep promotions small and single-purpose (one environment, one service) so reverts are clean.
- Define a break-glass role that can bypass review on the GitOps repo, with alerting when it is used.
- Practise rollbacks regularly (game days) - see Failure Scenario Drills §15.10.
- Record the incident or ticket ID in the revert commit message.

9 Tasks

- [ ] **T1 — Revert a bad promotion** *(≈30 min, beginner)*
  - **Goal:** do the standard GitOps rollback.
  - **Steps:**
    1. Promote a broken tag to the dev overlay by PR (or direct commit in the sandbox).
    2. Confirm the app degrades.

3. `git revert` the commit and push.

- **Done when:** the app returns to `Synced` / `Healthy` and `git log` shows the original commit and its revert.

- [ ] **T2 — Revert only one environment** *(≈30 min, intermediate)*
  - **Goal:** practise path-scoped rollback.
  - **Steps:**
    1. Make a commit that changes both staging and prod tags.
    2. Roll back prod only using `git checkout <sha>~1 -- apps/shop-api/overlays/prod`.
    3. Commit and push.
  - **Done when:** staging still runs the new tag and prod runs the old tag, with a clear commit message.

- [ ] **T3 — Feel the drift trap** *(≈30 min, intermediate)*
  - **Goal:** see why cluster-only rollback fails with self-heal.
  - **Steps:**
    1. With self-heal on, run `kubectl rollout undo` on a Deployment whose Git tag is wrong.
    2. Watch Argo CD revert your undo.
  - **Done when:** you can show the sequence in `kubectl rollout history` and the Argo CD event list and explain it in one sentence.

- [ ] **T4 — Execute the emergency procedure** *(≈45 min, advanced)*
  - **Goal:** follow the five-step break-glass runbook.
  - **Steps:**
    1. Break prod again with a bad promotion.
    2. Pause auto-sync, `argocd app rollback`, revert in Git, re-enable auto-sync.
    3. Time each step.
  - **Done when:** cluster and Git agree, auto-sync is back on, and you have a timeline with durations.

- [ ] **T5 — Automate a revert helper** *(≈45 min, advanced)*
  - **Goal:** shorten the rollback.
  - **Steps:**
    1. Write a small script `rollback-env.sh <env>` that finds the last commit touching `overlays/<env>`, reverts it, pushes a branch and opens a PR.
    2. Test it on dev.

- **Done when:** running the script yields a ready-to-merge revert PR in under a minute.

- [ ] **T6 — Prove it: write the runbook** *(≈30 min, advanced)*
  - **Goal:** produce a rollback runbook for `shop-api`.
  - **Steps:**
    1. Without notes, write the decision tree: revert PR, forward fix, or emergency cluster rollback.
    2. Include what to do about database migrations and who may bypass review.
  - **Done when:** a teammate can execute your runbook on the sandbox without asking you questions.

10 Self-check questions

1. Why is `git revert` preferred over `git reset --hard` for rollbacks in a GitOps repo?
2. What happens if you `kubectl rollout undo` while self-heal is on?
3. Why must you fix Git before re-enabling auto-sync?
4. How do you roll back only production when one commit changed several environments?
5. What can Git not roll back?

ANSWERS

1. Revert adds a new commit, keeping shared history and the audit trail; reset rewrites history and is blocked on protected branches.
2. The controller sees live state differ from Git and re-applies Git's (bad) version.
3. Otherwise the controller redeploys whatever Git currently says, which is the broken version.
4. Restore only that directory from the previous commit (`git checkout <sha>~1 -- <path>`) and commit, or revert and remove unrelated changes.
5. Data and schema changes, and anything outside Git such as external resources not managed declaratively.

11 Further reading

- Argo CD documentation - "argocd app rollback" and "Automated Sync Policy".
- Git documentation - `git revert` (including `-m` for merge commits).
- Flux documentation - `HelmRelease` remediation and `flux suspend`.

- *Site Reliability Engineering* (Google) - chapters on release engineering and incident response.

17.7 The Full Flow: CI to GitOps

Topic 7 of 7 | ~5 hours | Difficulty: Advanced

1 What it is

This topic joins the pieces into one pipeline: **Developer -> Git -> CI -> Docker image -> GitOps repo -> Argo CD -> Kubernetes**.

- **CI (continuous integration)** builds, tests and scans the code and publishes an immutable container image to a registry.
- **CD (continuous delivery) becomes GitOps:** instead of CI running `kubectl apply`, CI's last step *edits the GitOps repo* (bumps an image tag, via direct commit or pull request). The in-cluster agent does the deploy.
- **Secrets** are the hard part: the desired state lives in Git, but passwords and tokens must not. Three well-known approaches keep Git safe: **Sealed Secrets** (encrypt with a cluster key, commit the ciphertext), **SOPS** (encrypt values in the file with age/KMS keys, decrypt at apply time) and **External Secrets Operator (ESO)** (commit only a *reference*; the operator fetches the value from AWS Secrets Manager, Vault, etc.).

2 Why it matters in DevOps

- A split of responsibilities - CI proves the artifact is good, GitOps decides where it runs - gives the cleanest audit trail and the smallest credential footprint.
- Getting the hand-off wrong causes loops (a manifest commit retriggers CI), races (two pipelines bump the same file) and un-deployable releases (the image tag in Git does not exist).
- A leaked secret in Git history is permanent in practice; rotating it is mandatory. Choosing a secrets strategy before the first commit is much cheaper than after. See Secrets Management and Encryption §11.4 and Secrets Scanning §11.11.
- This flow is what most "platform engineering" golden paths automate for product teams (see Platform Engineering §0.13).

3 Real-world system example

The pattern of **separate app and config repositories**, CI that pushes an image and then updates a manifest repo, and Argo CD deploying the result is the standard flow shown in Argo CD's documentation and in countless public engineering write-ups; the Argo CD "Best Practices" page explicitly recommends a separate config repository. **Sealed Secrets** (from Bitnami) is widely used in small and mid-size clusters; **SOPS** (originally from Mozilla, now a CNCF sandbox project) is supported natively by Flux's kustomize-controller; **External Secrets Operator** is a CNCF project that syncs from AWS Secrets Manager, Google Secret Manager, Azure Key Vault, HashiCorp Vault and others. Kubernetes `Secret` objects are only base64-encoded, not encrypted, which is why none of these approaches commit them as-is - see Kubernetes Secrets §8.7.

4 Diagram

End-to-end flow:

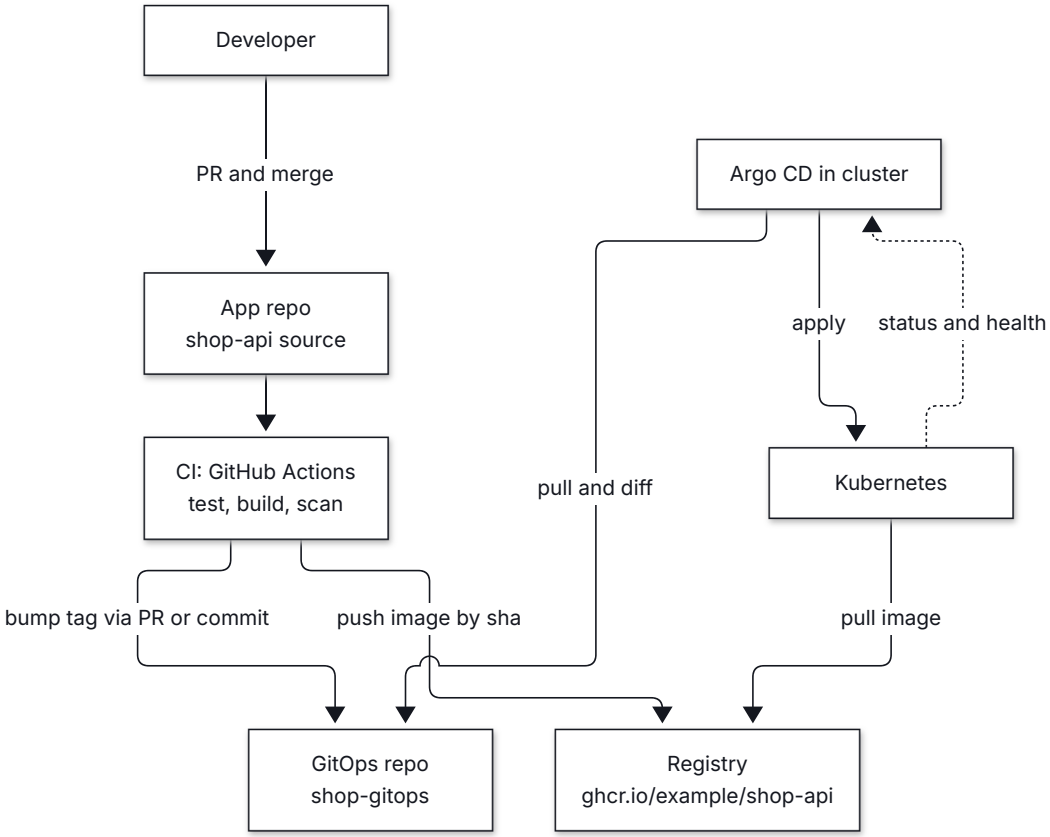


Figure 17.7-1

Three ways to keep secrets out of plain text in Git:

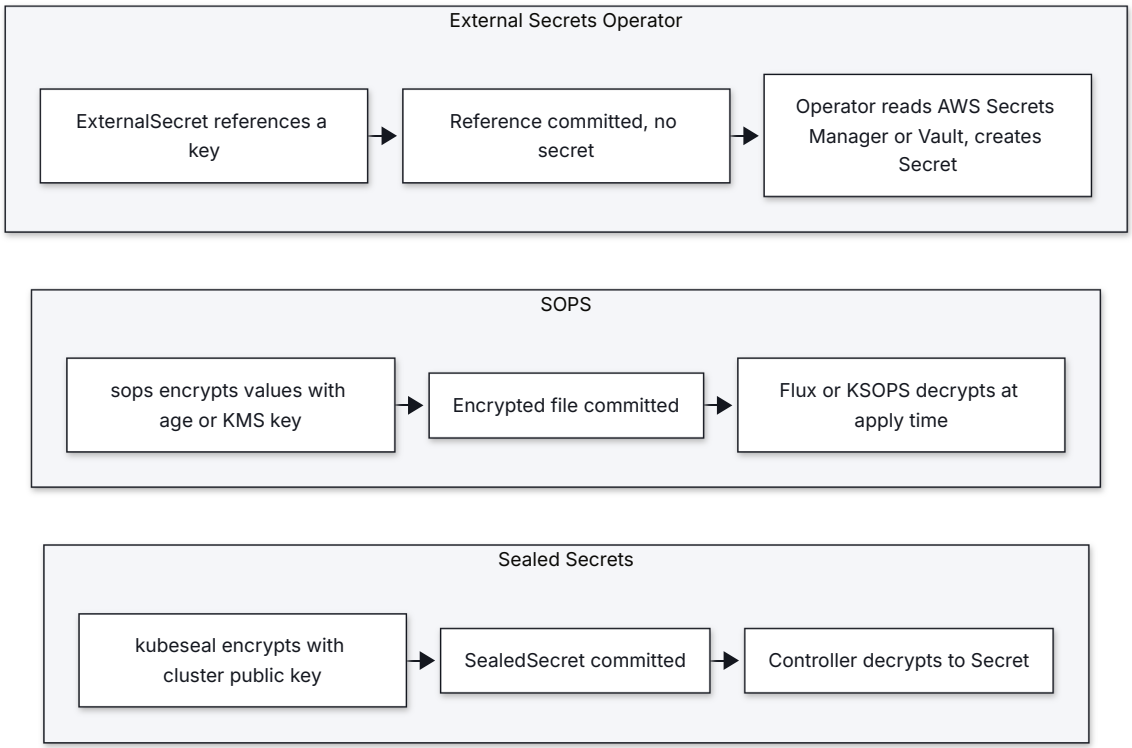


Figure 17.7-2

5 How it works

The CI side

1. Developer merges to `main` in the **app repo**.
2. CI runs tests (Automated Testing §5.3), builds the image, scans it (Image Scanning §11.9) and pushes it tagged with the **commit SHA** (and a semantic version for releases). SHA tags are unique and traceable.
3. The final CI job updates the **GitOps repo**: the dev overlay's tag, either by direct commit (dev only) or a PR (staging, prod). This job uses a token scoped to the GitOps repo only. The cluster never sees a CI credential.
4. Argo CD syncs. Optionally CI waits for `argocd app wait` or polls the Application status, then runs end-to-end tests.

Loop prevention: the GitOps repo has no build pipeline that triggers on manifest changes (or it uses path filters), so a tag bump does not retrigger an image build.

Secrets: choosing an approach

	Sealed Secrets	SOPS	External Secrets Operator
What is in Git	`SealedSecret` (ciphertext)	the Secret file with encrypted values	`ExternalSecret` (a reference only)
Source of truth for the value	Git (ciphertext)	Git (ciphertext)	external secret manager (AWS, Vault, ...)
Key management	controller's private key in the cluster (back it up!)	age key or cloud KMS key	operator's credentials to the store
Argo CD support	works natively (controller does the decrypt)	needs a plugin such as KSOPS or helm-secrets	works natively
Flux support	works natively	built into kustomize-controller	works natively
Rotation	re-seal and commit	re-encrypt and commit	rotate in the store, operator syncs (`refreshInterval`)
Best for	small teams, simple setups	teams wanting encrypted-in-Git with flexible keys	orgs with an existing secret manager and audit needs

Whichever you choose: Kubernetes does not encrypt Secrets at rest in etcd by default, so enable encryption at rest and restrict RBAC on Secrets (RBAC in Depth §16.1).

Example manifests (illustrative, for orientation; all values are placeholders):

YAML

```yaml
# Sealed Secrets: produced by kubeseal, safe to commit
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: shop-api-db
  namespace: shop-dev
spec:
  encryptedData:
    DATABASE_URL: AgBy3i4OJSWK...   # long ciphertext produced by kubeseal
  template:
    metadata:
      name: shop-api-db
      namespace: shop-dev
```

```yaml
# .sops.yaml: encrypt only the data of Secret manifests with an age public key
creation_rules:
  - path_regex: .*secret.*\.yaml$
    encrypted_regex: ^(data|stringData)$
    age: age1examplepublickeyexamplepublickeyexamplepublickeyexamplepub
```

```yaml
# External Secrets Operator: a reference to a value held elsewhere
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: shop-api-db
  namespace: shop-dev
spec:
  refreshInterval: 1h
  secretStoreRef:
    kind: SecretStore
    name: aws-secrets
  target:
    name: shop-api-db
  data:
    - secretKey: DATABASE_URL
      remoteRef:
        key: shop/dev/database-url
```

(ESO's API version was `external-secrets.io/v1beta1` in older releases; check your installed CRDs with `kubectl get crd externalsecrets.external-secrets.io -o yaml`.)

6 Step-by-step guide

Prerequisites: kind cluster with Argo CD (topic 2 §17.2), the `shop-gitops` repo with a dev Application, a GitHub account, `helm`, `kubectl`. Use a public image (e.g. `nginx`) if you do not build one. Cost: none (GitHub Actions free tier).

Part A: the CI hand-off

1. **Create a fine-grained token for the GitOps repo.** In GitHub: Settings -> Developer settings -> Fine-grained tokens, access to only `shop-gitops`, permission "Contents: read and write". Store it as an Actions secret named `GITOPS_TOKEN` in the **app** repo.
2. **Add the CI workflow to the app repo.**

```yaml
# .github/workflows/ci.yaml in shop-api
name: ci
```

```yaml
on:
  push:
    branches: [main]
permissions:
  contents: read
  packages: write
jobs:
  build-and-bump:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/${{ github.repository_owner }}/shop-api:${{ github.sha }}
      - name: Bump dev tag in GitOps repo
        env:
          GITOPS_TOKEN: ${{ secrets.GITOPS_TOKEN }}
          IMAGE_TAG: ${{ github.sha }}
          OWNER: ${{ github.repository_owner }}
        run: |
          git clone "https://x-access-token:${GITOPS_TOKEN}@github.com/${OWNER}/shop-
  gitops.git"
          cd shop-gitops/apps/shop-api/overlays/dev
          kustomize edit set image "ghcr.io/${OWNER}/shop-api=ghcr.io/${OWNER}/shop-
  api:${IMAGE_TAG}"
          cd ../../../..
          git config user.name "ci-bot"
          git config user.email "ci-bot@example.com"
          git commit -am "ci: shop-api dev → ${IMAGE_TAG}"
          git push
```

Staging and prod follow the PR-based promotion from Environment Promotion §17.4. Expected result: a green run, a new image in GitHub Packages and a commit by `ci-bot` in `shop-gitops`; Argo CD syncs dev within minutes. (Tag, action and runner versions evolve; check current releases.)

3. **Watch the hand-off end to end.**

```bash
kubectl -n shop-dev get deploy shop-api -o
  jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

Expect the new SHA tag. Make a trivial code change, push to `main`, and repeat.

Part B: secrets with Sealed Secrets (hands-on) and ESO (sandbox)

4. **Install the Sealed Secrets controller and CLI.**

```bash
helm repo add sealed-secrets https://bitnami-labs.github.io/sealed-secrets
helm repo update
helm install sealed-secrets sealed-secrets/sealed-secrets -n kube-system
kubectl -n kube-system rollout status deploy/sealed-secrets
```

   Install `kubeseal` from the project's releases page (or your package manager). The Deployment name can differ by chart version; check `kubectl -n kube-system get deploy | grep sealed`.

5. **Seal a secret and commit only the ciphertext.**

```bash
kubectl create secret generic shop-api-db -n shop-dev \
  --from-literal=DATABASE_URL='postgres://shop:CHANGE_ME@10.0.0.5:5432/shop' \
  --dry-run=client -o yaml \
| kubeseal --controller-name sealed-secrets --controller-namespace kube-system --
  format yaml \
> apps/shop-api/overlays/dev/sealed-db.yaml
head -12 apps/shop-api/overlays/dev/sealed-db.yaml
```

   Expect `kind: SealedSecret` and a long `encryptedData` value. Add the file under `resources:` in the dev overlay, commit, push. Argo CD applies it and the controller creates a normal `Secret`:

```bash
kubectl -n shop-dev get secret shop-api-db
```

   Back up the controller's sealing key (`kubectl -n kube-system get secret -l sealedsecrets.bitnami.com/sealed-secrets-key -o yaml > key-backup.yaml`, store it outside Git and the cluster); lose it and every SealedSecret is unrecoverable on a rebuilt cluster.

6. **Reference the secret from the Deployment.** In the base Deployment container, add:

```yaml
envFrom:
  - secretRef:
      name: shop-api-db
```

Commit, push, and confirm with `kubectl -n shop-dev exec deploy/shop-api -- printenv DATABASE_URL | cut -c1-12` (shows only the prefix).

7. **ESO in the sandbox (no cloud needed) using the `fake` provider.**

```bash
helm repo add external-secrets https://charts.external-secrets.io
helm install external-secrets external-secrets/external-secrets -n external-secrets --
  create-namespace --set installCRDs=true
```

```yaml
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: fake
  namespace: shop-dev
spec:
  provider:
    fake:
      data:
        - key: shop/dev/api-key
          value: not-a-real-key
---
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: shop-api-key
  namespace: shop-dev
spec:
  refreshInterval: 1m
  secretStoreRef:
    kind: SecretStore
    name: fake
  target:
    name: shop-api-key
  data:
    - secretKey: API_KEY
      remoteRef:
        key: shop/dev/api-key
```

Apply, then `kubectl -n shop-dev get externalsecret,secret shop-api-key`. Expect `SecretSynced` / `Ready`. In production you would swap the `fake` provider for `aws` (Secrets Manager, see Secrets Manager §6.16) using IRSA or similar - never static keys. (If `v1` is not served by your ESO version, use `external-secrets.io/v1beta1`.)

8. **SOPS (read-through).** `age-keygen -o key.txt`, put the public key in `.sops.yaml`, run `sops --encrypt --in-place secret.yaml`; with Flux, set `decryption: {provider: sops, secretRef: {name: sops-age}}` on the Kustomization. With Argo CD use KSOPS or helm-

secrets.

7 Real-world scenario

Illustrative scenario. A developer pastes the production database URL into `values-prod.yaml` "just to test" and merges it. The secret scanner (see Secrets Scanning §11.11) flags it within a minute; the team rotates the password immediately because the value is in Git history forever, then adopts External Secrets Operator so that the GitOps repo contains only references. A month later, a CI-token compromise attempt on the app repo can only write image tags, because the GitOps repo requires PRs with code-owner approval for prod, and the cluster credentials were never in CI. **Lesson:** the flow's value is in the *boundaries*: CI cannot reach the cluster, Git cannot hold secrets, prod needs a human.

8 Common mistakes and best practices

Don't

- Let CI run `kubectl apply` against prod "just for this one service".
- Commit a plain Kubernetes `Secret`, even base64-encoded, or rely on a private repo for secrecy.
- Tag images `latest` or overwrite tags; Argo CD will not notice a changed image behind the same tag.
- Put the CI that builds images on the GitOps repo's manifest changes (infinite loop).
- Lose the Sealed Secrets private key, or the SOPS age key, without a backup.

Do

- Tag images with the commit SHA; promote the same image through environments.
- Use a least-privilege, repo-scoped token (or a GitHub App) for the CI -> GitOps hand-off.
- Choose one secrets strategy per platform, document it and rotate regularly.
- Verify deploys end to end: have CI (or a PostSync hook) smoke-test the new version and report status.
- Sign images and verify them with an admission policy when you are ready (see Admission Controllers §16.6).

9 Tasks

- [ ] **T1 — Wire CI to the GitOps repo** *(≈60 min, intermediate)*
  - **Goal:** a push to the app repo ends with a new image running in dev.

- **Steps:**
  1. Create the token and secret, add the workflow from Part A.
  2. Push a code change to `main`.
  3. Watch Actions, the `ci-bot` commit, and the Argo CD sync.
- **Done when:** the image in `shop-dev` equals the commit SHA of your latest push.

☐ **T2 — Seal a secret** *(≈45 min, intermediate)*

- **Goal:** commit a secret safely.
- **Steps:**
  1. Install Sealed Secrets and `kubeseal`.
  2. Seal `DATABASE_URL`, add it to the dev overlay and push.
  3. Mount it with `envFrom`.
- **Done when:** the Secret exists in `shop-dev`, and `grep -r 'CHANGE_ME' .` in the repo finds nothing.

☐ **T3 — Try External Secrets** *(≈45 min, intermediate)*

- **Goal:** use a reference instead of a ciphertext.
- **Steps:**
  1. Install ESO and apply the `fake` provider example.
  2. Change the value in the SecretStore and wait for `refreshInterval`.
  3. Observe the Secret update.
- **Done when:** `kubectl get secret shop-api-key -o jsonpath='{.data.API_KEY}' | base64 -d` shows the new value.

☐ **T4 — Break it: lose the key** *(≈30 min, advanced)*

- **Goal:** feel why backing up the sealing key matters.
- **Steps:**
  1. Back up the sealing key, then delete the controller's key Secret and restart the controller (a new key is generated).
  2. Observe the old SealedSecret fail to decrypt (`kubectl -n shop-dev describe sealedsecret ...`).
  3. Restore the key backup and restart the controller.
- **Done when:** the SealedSecret decrypts again after restoring the backup.

☐ **T5 — Promote through all stages with a gate** *(≈75 min, advanced)*

- **Goal:** the full flow dev -> staging -> prod.

- **Steps:**
  1. Extend CI to open a PR for staging after the dev deploy is healthy (`argocd app wait` or a smoke test).
  2. Require CODEOWNERS approval for prod.
  3. Promote one release all the way to prod, then roll it back with `git revert` (Rollback Through Git §17.6).
- **Done when:** the commit history of the GitOps repo shows the promotion chain and the revert, all by PR.

☐ **T6 — Prove it: trace a change** *(≈30 min, advanced)*

- **Goal:** narrate the full flow.
- **Steps:**
  1. Without notes, draw Developer -> Git -> CI -> image -> GitOps repo -> Argo CD -> Kubernetes, marking which credentials each hop holds.
  2. Choose a secrets approach for a team using AWS and justify it in three sentences.
- **Done when:** you can state, for each credential, what damage its compromise allows and why it is the smallest possible.

10 Self-check questions

1. Why should CI edit a GitOps repo rather than run `kubectl apply`?
2. Why tag images with the commit SHA?
3. What do you commit with Sealed Secrets, SOPS and ESO respectively?
4. What is the biggest operational risk of Sealed Secrets?
5. How do you stop a manifest commit from retriggering the image build?

ANSWERS

1. No cluster credentials in CI, a reviewable audit trail, drift correction by the in-cluster agent, and a single source of truth.
2. SHAs are unique and immutable, traceable to source, and a changed tag triggers a visible Git diff that Argo CD syncs.
3. A `SealedSecret` ciphertext; a file with SOPS-encrypted values; an `ExternalSecret` reference with no secret value.
4. Losing the controller's private key (no backup) makes all SealedSecrets undecryptable on a rebuilt cluster.

5. Keep the GitOps repo separate from the app repo (or use path filters), so manifest commits do not trigger the app's build pipeline.

11 Further reading

- Argo CD documentation - "Best Practices" (separate config repository) and "Secret Management".
- Sealed Secrets (github.com/bitnami-labs/sealed-secrets), SOPS (getsops.io) and External Secrets Operator (external-secrets.io) documentation.
- Flux documentation - "Manage Kubernetes secrets with SOPS".
- *Continuous Delivery* by Jez Humble and David Farley; see also CI/CD Pipeline Design §5.1.

LAB NOTES What I ran, what I saw, what it means

★ GitOps Delivery for shop-api

Level 17 capstone ~10 hours

1 Goal and outcome

Build a complete GitOps delivery system on a laptop. By the end you will have:

- a local `kind` cluster running **Argo CD**;
- a **GitOps repository** (`shop-gitops`) with a Kustomize base and `dev` / `staging` / `prod` overlays;
- an **App of Apps** root and an **ApplicationSet** that generate one Argo CD Application per environment, with **automated sync, prune and self-heal**;
- a **PreSync / PostSync hook** pair and sync waves;
- **promotion dev -> staging -> prod by pull request**, gated by CODEOWNERS for prod;
- a **CI workflow** that builds an image and bumps the dev tag in the GitOps repo;
- a **sealed secret** (or External Secrets) instead of plain-text credentials;
- a rehearsed **rollback with** `git revert` and an emergency procedure.

This exercises every topic: Desired State Management §17.1, Argo CD and Flux §17.2, Continuous Reconciliation §17.3, Environment Promotion §17.4, Application Synchronization §17.5, Rollback Through Git §17.6 and The Full Flow: CI to GitOps §17.7.

2 Architecture

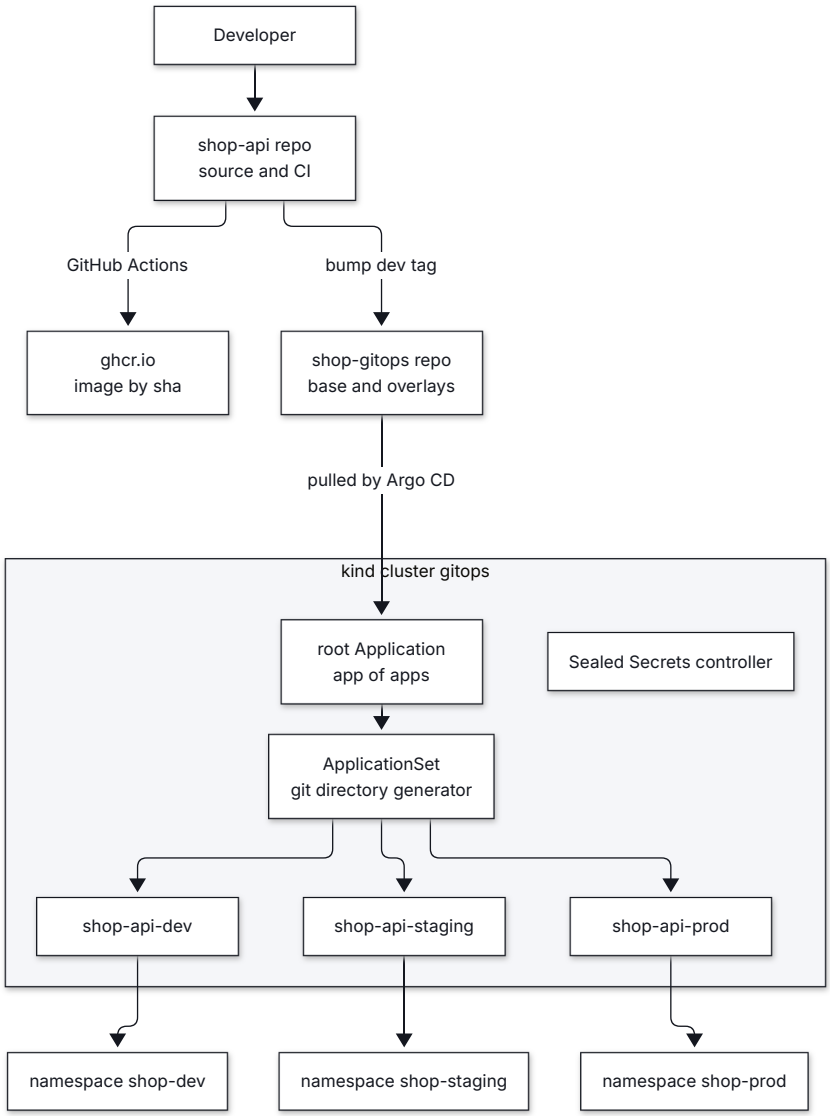


Figure 17-1

Promotion and rollback path:

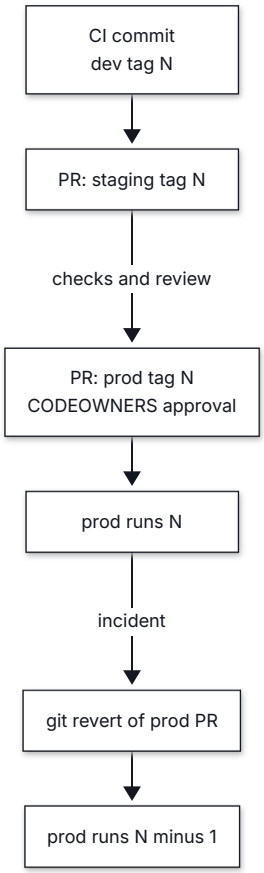


Figure 17-2

3 Prerequisites and cost

- Docker, `kind`, `kubectl`, `git`, `helm`, `kustomize` (standalone), `gh` (GitHub CLI), optional `argocd` CLI and `kubeseal`.
- A GitHub account, two empty repositories you create: `shop-gitops` (public is simplest) and, optionally, `shop-api` for the CI part.
- About 4 GB of free RAM and 10 GB of disk for the kind node and images.
- **Cost:** free. GitHub Actions free-tier minutes are enough. No cloud resources are created; teardown is `kind delete cluster`.
- To keep the sandbox simple the capstone deploys the public `nginx` image as a stand-in for `shop-api`. If you built `shop-api` in earlier levels, substitute your own image.

- Outputs shown below are illustrative; yours will differ in names, ages and timings.

4 Build it step by step

Milestone 1 - Cluster and Argo CD (topics 1-2)

```bash
kind create cluster --name gitops
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server --timeout=300s
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' |
  base64 -d; echo
kubectl -n argocd port-forward svc/argocd-server 8080:443 &
```

Browse to `https://localhost:8080`, log in as `admin`. (For anything beyond a sandbox, pin a specific Argo CD release instead of `stable`.)

Milestone 2 - The GitOps repo (topic 1)

```bash
git clone https://github.com/<YOUR_USER>/shop-gitops.git && cd shop-gitops
mkdir -p apps/shop-api/{base,overlays/{dev,staging,prod}} clusters/local/apps .github
```

Base manifests:

```yaml
# apps/shop-api/base/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
  labels:
    app: shop-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: shop-api
  template:
    metadata:
      labels:
        app: shop-api
    spec:
      containers:
        - name: shop-api
```

```
          image: nginx:1.26
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet:
              path: /
              port: 80
```

YAML

```yaml
# apps/shop-api/base/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: shop-api
spec:
  selector:
    app: shop-api
  ports:
    - port: 80
      targetPort: 80
```

YAML

```yaml
# apps/shop-api/base/presync-job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: busybox:1.36
          command: ["sh", "-c", "echo running migrations && sleep 3"]
```

YAML

```yaml
# apps/shop-api/base/postsync-job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: smoke-test
  annotations:
    argocd.argoproj.io/hook: PostSync
```

```
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: smoke
          image: busybox:1.36
          command: ["sh", "-c", "wget -qO- http://shop-api && echo smoke test passed"]
```

YAML

```yaml
# apps/shop-api/base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
  - presync-job.yaml
  - postsync-job.yaml
```

Overlays (shown for dev; staging uses namespace `shop-staging`, 2 replicas, tag `1.26`; prod uses `shop-prod`, 3 replicas, tag `1.26`):

YAML

```yaml
# apps/shop-api/overlays/dev/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-dev
resources:
  - ../../base
images:
  - name: nginx
    newTag: "1.27"
```

YAML

```yaml
# apps/shop-api/overlays/staging/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-staging
resources:
  - ../../base
replicas:
  - name: shop-api
    count: 2
images:
  - name: nginx
    newTag: "1.26"
```

YAML

```yaml
# apps/shop-api/overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-prod
resources:
  - ../../base
replicas:
  - name: shop-api
    count: 3
images:
  - name: nginx
    newTag: "1.26"
```

Render and commit:

BASH

```bash
for e in dev staging prod; do kubectl kustomize apps/shop-api/overlays/$e | grep -E
  'namespace:|replicas:|image: nginx'; done
git add . && git commit -m "feat: shop-api base and overlays" && git push
```

Milestone 3 - App of Apps and ApplicationSet (topics 2, 3, 5)

YAML

```yaml
# clusters/local/root-app.yaml  (applied once by hand)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
```

```
    targetRevision: main
    path: clusters/local/apps
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

YAML

```yaml
# clusters/local/apps/shop-api-appset.yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: shop-api-envs
  namespace: argocd
spec:
  goTemplate: true
  goTemplateOptions: ["missingkey=error"]
  generators:
    - git:
        repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
        revision: main
        directories:
          - path: apps/shop-api/overlays/*
  template:
    metadata:
      name: 'shop-api-{{ .path.basename }}'
    spec:
      project: default
      source:
        repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
        targetRevision: main
        path: '{{ .path.path }}'
      destination:
        server: https://kubernetes.default.svc
        namespace: 'shop-{{ .path.basename }}'
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
```

BASH

```bash
git add . && git commit -m "feat: root app and applicationset" && git push
kubectl apply -f clusters/local/root-app.yaml
kubectl -n argocd get applications
```

Expect (illustrative) `root`, `shop-api-dev`, `shop-api-staging`, `shop-api-prod`, all `Synced` and `Healthy` after a couple of minutes.

Milestone 4 - Reconciliation behaviour (topic 3)

```bash
kubectl -n shop-dev scale deploy/shop-api --replicas=5   # drift
kubectl -n shop-dev get deploy shop-api -w               # returns to 1
```

Milestone 5 - Promotion by pull request (topic 4)

```bash
git switch -c promote-staging-1.27
(cd apps/shop-api/overlays/staging && kustomize edit set image nginx=nginx:1.27)
git commit -am "promote shop-api 1.27 to staging" && git push -u origin HEAD
gh pr create --fill
```

Add the prod gate:

```
# .github/CODEOWNERS
/apps/shop-api/overlays/prod/ @<YOUR_USER>
```

and a validation workflow:

```yaml
# .github/workflows/validate.yaml
name: validate
on:
  pull_request:
jobs:
  render:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Render every overlay
        run: |
          for d in apps/*/overlays/*; do
            echo "== $d"
            kubectl kustomize "$d" > /dev/null
          done
```

Milestone 6 - Secrets and CI hand-off (topic 7)

```bash
helm repo add sealed-secrets https://bitnami-labs.github.io/sealed-secrets && helm repo
  update
helm install sealed-secrets sealed-secrets/sealed-secrets -n kube-system
kubectl create secret generic shop-api-db -n shop-dev \
  --from-literal=DATABASE_URL='postgres://shop:CHANGE_ME@10.0.0.5:5432/shop' --dry-
  run=client -o yaml \
 | kubeseal --controller-name sealed-secrets --controller-namespace kube-system --format
  yaml \
 > apps/shop-api/overlays/dev/sealed-db.yaml
```

Add `sealed-db.yaml` to the dev overlay's `resources:`. For CI, add the workflow from The Full Flow §17.7 to your `shop-api` repo (token secret `GITOPS_TOKEN`).

Milestone 7 - Rollback (topic 6)

Promote a broken tag to prod by PR (for example `nginx:does-not-exist`), watch it degrade, then:

```bash
git switch main && git pull
git revert --no-edit HEAD
git push        # or via a PR if main is protected
```

Then rehearse the emergency path: `argocd app set shop-api-prod --sync-policy none`, `argocd app rollback ...`, fix Git, re-enable automation.

5 Tasks

- [ ] **T1 — Stand up the platform** *(≈60 min, beginner; Milestone 1; topic 2)*
  - **Goal:** Argo CD running on kind.
  - **Steps:**
    1. Run Milestone 1.
    2. Log in to the UI and change the admin password.
  - **Done when:** `kubectl -n argocd get pods` is all `Running` and the UI loads.
- [ ] **T2 — Create the repo and render the overlays** *(≈60 min, beginner; Milestone 2; topic 1)*
  - **Goal:** a base plus three overlays that differ only in namespace, replicas and tag.
  - **Steps:**
    1. Create all files; run the render loop.

2. Diff dev against prod rendered output.

- **Done when:** the diff shows only namespace, replica count and image tag differences, and the repo is pushed.

- [ ] **T3 — Bootstrap with App of Apps and ApplicationSet** *(≈60 min, intermediate; Milestone 3; topics 2 and 5)*
  - **Goal:** one `kubectl apply` creates every environment.
  - **Steps:**
    1. Apply `root-app.yaml`.
    2. Wait for the three generated Applications.
    3. Add `overlays/qa` and watch `shop-api-qa` appear; then remove it.
  - **Done when:** four Applications existed at one point and the cluster ended with three, all without hand-writing an Application.

- [ ] **T4 — Prove reconciliation** *(≈45 min, intermediate; Milestone 4; topic 3)*
  - **Goal:** demonstrate drift correction and prune.
  - **Steps:**
    1. Scale the dev Deployment by hand and watch it revert.
    2. Add a ConfigMap to the base, let it sync, then delete it from Git and watch it be pruned.
    3. Add `ignoreDifferences` for `/spec/replicas` to the ApplicationSet template, then show manual scaling is no longer reverted.
  - **Done when:** you have a screenshot or terminal log of all three behaviours.

- [ ] **T5 — Observe hooks and waves** *(≈45 min, intermediate; Milestone 2-3; topic 5)*
  - **Goal:** see ordered syncs.
  - **Steps:**
    1. Bump the dev tag and push.
    2. Watch `kubectl -n shop-dev get jobs,pods -w` and the sync operation panel.
    3. Make the PreSync command `exit 1` and observe the sync fail without replacing the old pods; fix it.
  - **Done when:** the failed PreSync blocks the rollout and the fixed one lets it through.

- [ ] **T6 — Promote dev -> staging -> prod by PR** *(≈90 min, advanced; Milestone 5; topic 4)*
  - **Goal:** a full gated promotion.
  - **Steps:**
    1. Promote `nginx:1.27` to staging by PR; the `validate` check must pass.

2. Promote it to prod; add CODEOWNERS and enable branch protection requiring code-owner review.
3. Verify each namespace runs the intended tag.

- **Done when:** `git log` for each overlay shows one promotion PR and `kubectl` shows 1.27 in all three namespaces.

☐ **T7 — Secrets without plain text** *(≈60 min, advanced; Milestone 6; topic 7)*

- **Goal:** deliver a credential via Git safely.
- **Steps:**
  1. Seal `DATABASE_URL`, commit the SealedSecret, add `envFrom` to the Deployment.
  2. Search the repo for the plaintext value.
  3. Back up the sealing key outside Git.
- **Done when:** the Secret exists in `shop-dev`, the plain value is absent from the repo and history, and the key backup exists.

☐ **T8 — CI hand-off** *(≈75 min, advanced; Milestone 6; topic 7)*

- **Goal:** a push to the app repo updates the dev overlay.
- **Steps:**
  1. Create a fine-grained token limited to `shop-gitops` contents.
  2. Add the CI workflow to the app repo and push a change.
- **Done when:** a `ci-bot` commit appears in `shop-gitops` and Argo CD deploys the new SHA tag to dev.

☐ **T9 — Roll back with git revert** *(≈60 min, advanced; Milestone 7; topic 6)*

- **Goal:** recover from a bad prod promotion.
- **Steps:**
  1. Promote a broken tag to prod and watch it degrade.
  2. Recover with `git revert`, then repeat using the emergency path and re-enable automation.
- **Done when:** prod is `Synced` and `Healthy`, history shows promotion plus revert, and auto-sync is back on.

☐ **T10 — Prove it: rebuild from scratch** *(≈90 min, advanced; all milestones; all topics)*

- **Goal:** show the repo really is the source of truth.
- **Steps:**
  1. `kind delete cluster --name gitops`, create a new cluster, reinstall Argo CD, re-apply only `root-app.yaml` (and restore the sealing key).

2. Write a one-page explanation of what was restored automatically and what was not (secrets key, Argo CD admin password).

- **Done when:** all three environments return to Healthy with the same image tags as before, and your write-up lists the remaining manual steps.

6 Verification checklist

- [ ] `kubectl -n argocd get applications` shows `root` and three environment Applications, all `Synced` and `Healthy`.
- [ ] A manual `kubectl scale` on dev is reverted without human action.
- [ ] Deleting a manifest from Git deletes the live object (prune) and a `Prune=false` object survives.
- [ ] A bad PreSync hook blocks a release; a good one runs before the rollout and `smoke-test` after.
- [ ] Each promotion is a PR with a one-line diff; prod changes need a code-owner approval.
- [ ] No plain-text secret exists in the repo or its history (`git log -p | grep CHANGE_ME` returns nothing).
- [ ] CI pushes an image and a bot commit bumps the dev tag; no cluster credentials exist in CI.
- [ ] A `git revert` restores a previous prod version, and the audit trail shows who did what.
- [ ] The whole stack rebuilds from `root-app.yaml` on a new cluster.

7 Teardown

```bash
kill %1 2>/dev/null                    # stop the port-forward if it is still running
kind delete cluster --name gitops
```

Delete the `shop-gitops` / `shop-api` GitHub repositories and the fine-grained token if you no longer need them. Remove the Actions secret `GITOPS_TOKEN`. Keep the sealing-key backup only as long as you need it; it is a credential.

8 Stretch goals and what to learn next

- Replace the hand-built Applications with Flux on a second cluster and compare (Argo CD and Flux §17.2).
- Add Argo CD Image Updater or Flux image automation for dev, keeping staging and prod PR-gated.
- Use the ApplicationSet `pullRequest` generator to create a preview environment per app PR.
- Add Argo Rollouts for canary releases driven from Git (Canary Deployments §5.12).
- Restrict teams with `AppProject`s and Kubernetes RBAC (RBAC in Depth §16.1).
- Swap Sealed Secrets for External Secrets Operator backed by AWS Secrets Manager.
- Next level: Level 18 - Platform Engineering §18.1, where GitOps becomes the delivery engine for a self-service internal platform.

LEVEL 18 · PART VIII

Platform Engineering 18

Goal: Build internal platforms that let product teams self-serve infrastructure safely and quickly.

RECOMMENDED TIME
4-6 weeks

TOPICS
10

PRACTICE / CAPSTONE
Build a mini internal developer platform: Backstage catalog + software template that scaffolds a service repo, provisions infra via Crossplane/Terraform and deploys through Argo CD.

Topics

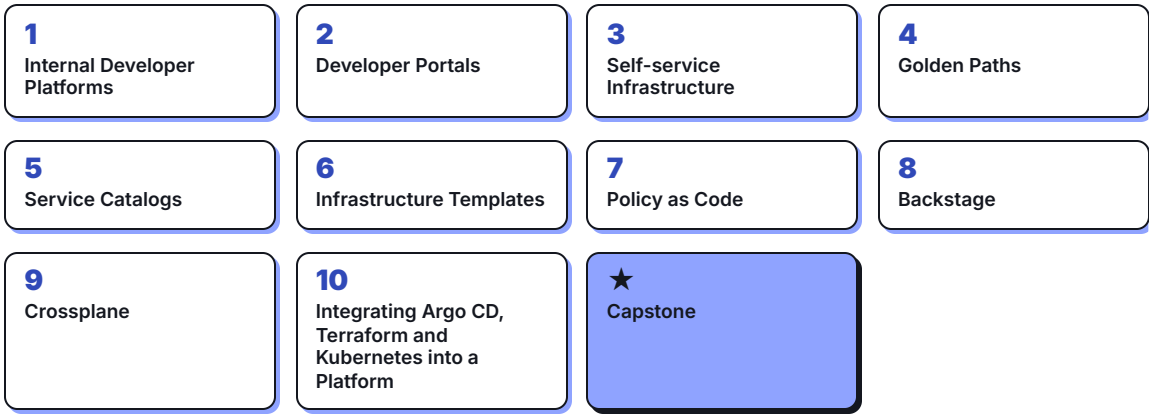
#	Topic	What you will learn
1	Internal Developer Platforms **p.3405**	IDP concept, platform as a product, thinnest viable platform
2	Developer Portals **p.3415**	what a portal offers, build vs buy, adoption
3	Self-service Infrastructure **p.3427**	request flows, guardrails, ticket-ops to API-ops
4	Golden Paths **p.3439**	paved roads vs mandates, templates, measuring adoption
5	Service Catalogs **p.3451**	ownership metadata, catalog-info.yaml, dependency graphs, scorecards
6	Infrastructure Templates **p.3463**	reusable modules/compositions, versioning, template testing
7	Policy as Code **p.3475**	OPA/Rego, Kyverno, Conftest, enforcing in CI and cluster
8	Backstage **p.3491**	install, catalog, software templates, plugins, TechDocs
9	Crossplane **p.3505**	providers, managed resources, compositions, XRDs, claims
10	Integrating Argo CD, Terraform and Kubernetes into a Platform **p.3521**	how the pieces compose: control planes, GitOps delivery, state ownership

#	Topic	What you will learn
★	Level capstone project **p.3535**	Build a mini internal developer platform: Backstage catalog + software template that scaffolds a service repo, provisions infra via Crossplane/Terraform and deploys through Argo CD.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



18.1 Internal Developer Platforms

Topic 1 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

An **Internal Developer Platform (IDP)** is the set of tools, workflows and documentation that a platform team builds so that product teams can create, deploy and operate software **without becoming experts in every underlying system**. It is the layer *between* developers and raw infrastructure (cloud accounts, Kubernetes, CI, observability, secrets).

An IDP is **not one product you buy**. It is a collection of capabilities - for example "create a new service from a template", "get a database", "deploy to staging", "see my service's logs and alerts" - exposed through a few consistent interfaces: a CLI, a Git workflow, APIs and usually a web portal (Developer Portals §18.2).

Two ideas make it work:

- **Platform as a product.** The platform team treats developers as *customers*: it researches their pain, has a roadmap, measures adoption and satisfaction, and supports its users. A platform nobody wants to use is a failed product, however elegant.
- **Thinnest Viable Platform (TVP).** The term comes from the book *Team Topologies* (Matthew Skelton and Manuel Pais): the smallest platform that removes the most friction - sometimes just a well-written wiki page and a template - and grows only when teams ask for more.

See Platform Engineering §0.13 for the introductory view.

2 Why it matters in DevOps

- **"You build it, you run it" has a cognitive-load limit.** Asking every product team to master Kubernetes, Terraform, IAM, networking and observability slows delivery and produces 20 slightly different, slightly insecure setups.
- **Consistency is a security and cost control.** One supported path for logging, secrets and deployment means one place to fix a vulnerability.

- **Toil moves to where it can be automated once.** Ticket queues ("please create a bucket") become APIs and templates.
- **Platform teams fail in predictable ways:** building a grand platform nobody asked for, mandating adoption without value, or becoming a ticket queue under a new name. Knowing the concept protects you from all three.

3 Real-world system example

Spotify and Backstage. Spotify built an internal portal to tame the sprawl of services, tools and docs, and open-sourced it as **Backstage** in 2020; it later became a CNCF project (incubating level at the time of writing). Backstage is the best-known building block for the *portal* part of an IDP, not an entire IDP by itself. **Netflix** has publicly described its "paved road": a set of well-supported tools and defaults that most teams use because it makes them faster, while teams may still choose to go off-road at their own cost. **The CNCF Platforms White Paper** and the *Team Topologies* book define platform teams as enabling teams that offer self-service capabilities; the community "Platform Engineering" movement and conferences grew around the same idea. Many vendors (Humanitec, Port, Cortex, OpsLevel and others) sell portal or orchestration components - pick the facts from each vendor's own documentation, since products change quickly.

4 Diagram

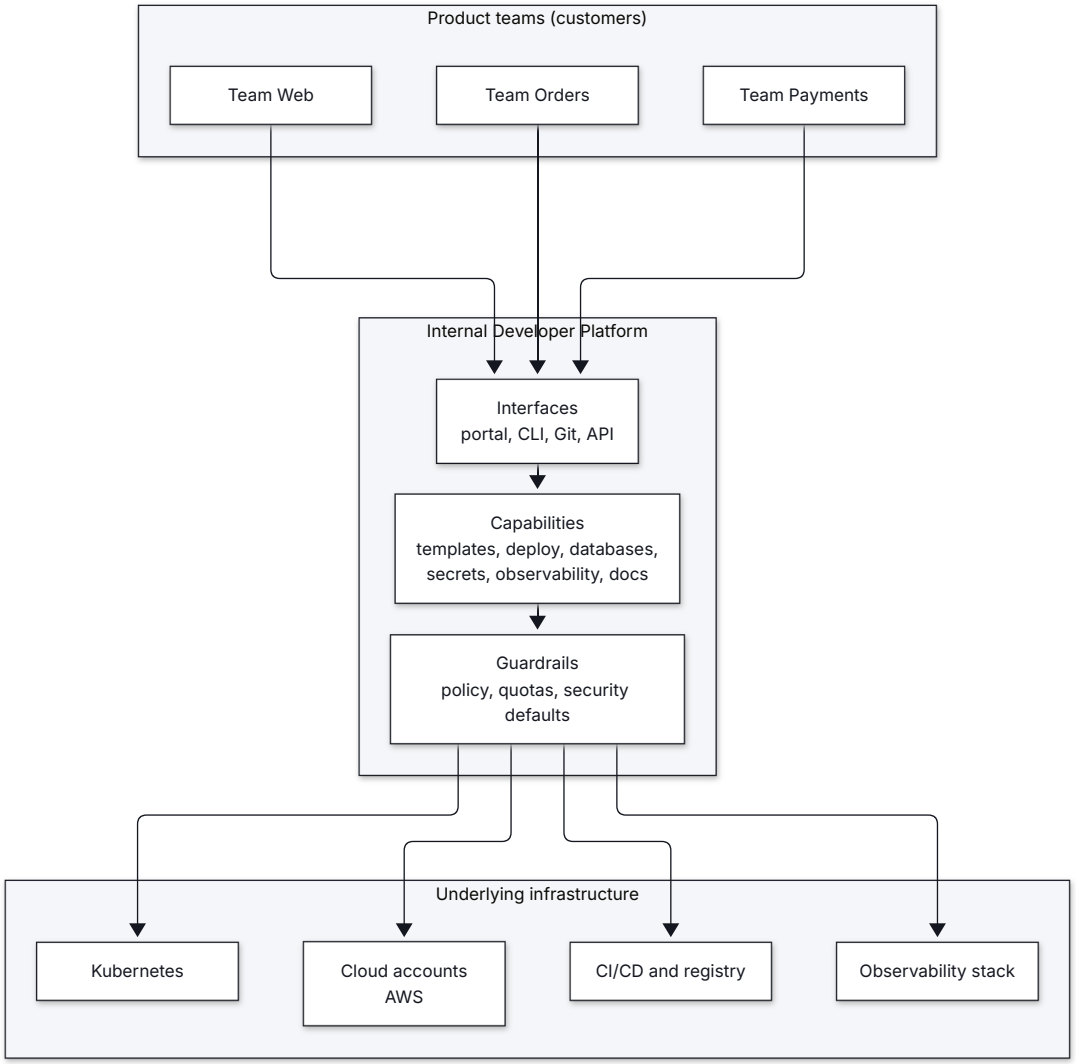


Figure 18.1-1

The product feedback loop that keeps a platform honest:

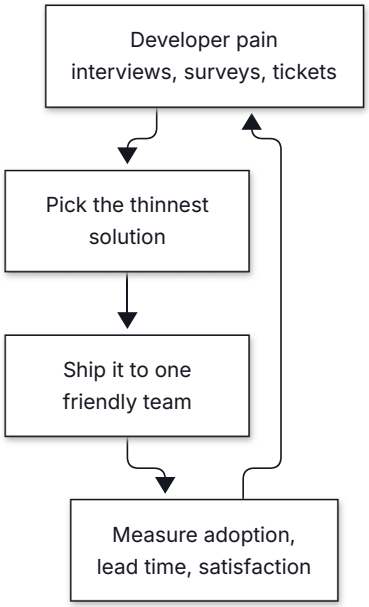


Figure 18.1-2

5 How it works

What an IDP typically contains

Layer	Examples	Question it answers
Interface	portal (Backstage), CLI, GitHub templates, chatops	"Where do I start?"
Orchestration	GitOps (Argo CD), CI workflows, Terraform/Crossplane	"How does it actually happen?"
Capabilities	service templates, database provisioning, secrets, preview environments	"What can I get?"
Guardrails	policy as code, quotas, default network policies	"What can't I accidentally break?"
Observability and feedback	dashboards per service, DORA metrics, platform usage stats	"Is it working, for them and for us?"

Platform as a product - the practices

- **Customers:** name them (the internal teams), interview them, keep a backlog based on their pain.

- **Roadmap and versioning:** announce changes, deprecate with notice, keep a changelog.
- **Support:** a channel, an on-call or office-hours rotation, an SLO for the platform itself (SLOs §14.2).
- **Metrics:** adoption (% of services on the paved path), time-to-first-deploy for a new service, DORA metrics (deployment frequency, lead time, change failure rate, time to restore), developer satisfaction.
- **Optional, not mandatory:** the best platforms win by being the easiest path (Golden Paths §18.4).

The Thinnest Viable Platform

Start with the highest-friction journey, usually "I need a new service in production". A TVP can be as small as:

1. a **template repo** with a Dockerfile, CI workflow, Kubernetes manifests and `catalog-info.yaml`;
2. a **README/wiki page** ("how we ship") and an owner list;
3. a **shared CI workflow** teams call instead of copying;
4. one **way to ask for help**.

Only add a portal, Crossplane or a fancy orchestrator when a measured problem justifies it. Platform projects fail more often from over-building than from under-building.

Maturity ladder

Stage	Looks like	Risk
0 Ad hoc	each team does its own thing	duplication, inconsistent security
1 Documented	wiki and runbooks	docs rot
2 Templated	template repos, shared CI	templates drift after creation
3 Self-service	APIs, portal, Git-driven requests	over-engineering, low adoption
4 Product	roadmap, metrics, SLOs for the platform	needs sustained staffing

6 Step-by-step guide

Prerequisites: a Linux/macOS shell with `bash`, `git`, `sed`. Cost: none. We build a miniature TVP: a template plus a scaffolding script.

1. **Create the template repository layout.**

```
BASH
```

```
mkdir -p tvp/template/.github/workflows tvp/template/k8s && cd tvp
cat > template/README.md <<'EOF'
# __NAME__

Owner: __OWNER__
EOF
cat > template/catalog-info.yaml <<'EOF'
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: __NAME__
spec:
  type: service
  lifecycle: experimental
  owner: __OWNER__
EOF
cat > template/Dockerfile <<'EOF'
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
USER node
CMD ["node", "server.js"]
EOF
cat > template/.github/workflows/ci.yaml <<'EOF'
name: ci
on: [push]
jobs:
  build:
    uses: example/platform-workflows/.github/workflows/node-service.yaml@v1
    with:
      service: __NAME__
EOF
```

The CI file calls a *shared* workflow owned by the platform team (a placeholder repo here); that is the leverage - fix it once, every service gets the fix.

2. **Write the scaffolding script - the first "platform API".**

BASH

```
cat > new-service.sh <<'EOF'
#!/usr/bin/env bash
# usage: ./new-service.sh <name> <owner-team>
set -euo pipefail
name="${1:?service name required}"
owner="${2:?owner team required}"
[[ "$name" =~ ^[a-z][a-z0-9-]{2,30}$ ]] || { echo "invalid name: $name" >&2; exit 1; }
dest="out/$name"
[[ -e "$dest" ]] && { echo "$dest already exists" >&2; exit 1; }
mkdir -p out
```

```
cp -r template "$dest"
grep -rl '__NAME__\|__OWNER__' "$dest" | xargs sed -i "s/__NAME__/$name/g;
  s/__OWNER__/$owner/g"
echo "created $dest"
find "$dest" -type f | sort
EOF
chmod +x new-service.sh
```

(On macOS use `sed -i ''` or install GNU sed.)

3. **Scaffold a service and inspect the result.**

```bash
./new-service.sh orders-api team-payments
cat out/orders-api/catalog-info.yaml
```

Expected output (verified): `created out/orders-api`, then the file list (`.github/workflows/ci.yaml`, `Dockerfile`, `README.md`, `catalog-info.yaml`), and the catalog file with `name: orders-api` and `owner: team-payments`.

4. **See the guardrail work.**

```bash
./new-service.sh Bad team-x; echo "exit code: $?"
```

Expect `invalid name: Bad` and exit code `1`. Small validation like this is the beginning of guardrails (Self-service Infrastructure §18.3).

5. **Write the platform's "front page" (one Markdown file).** Create `PLATFORM.md` with: what the platform offers today (one bullet per capability), how to ask for help, what is *not* supported, and the owner. This is your MVP documentation.

6. **Define success metrics before you build more.** Add to `PLATFORM.md`: "time from idea to first deploy of a new service" (measure it by timing T1 below), "% of services created from the template" and "developer satisfaction (1-5 survey)".

7 Real-world scenario

Illustrative scenario. A 60-engineer company forms a platform team of four. Their first plan is a year-long build: Backstage, Crossplane and a custom deployment UI. A staff engineer instead interviews eight teams. The top complaint: *"it takes two weeks and five tickets to get a new service to production"*. The team ships a template repo, a shared CI workflow and a Slack channel in three weeks; time-to-first-deploy falls to two days. Adoption is measured by the number of new services created from the template (11 of 12 in the next quarter). Only then do

they add a portal, because teams started asking "where is the list of all services and who owns them?". **Lesson:** solve the measured pain with the thinnest thing; earn the right to build more.

8 Common mistakes and best practices

Don't

- Build the platform for an imagined user without interviewing real ones.
- Mandate adoption of an immature platform; you will get workarounds and resentment.
- Let the platform team become a ticket-driven ops team under a new name.
- Hide everything: if developers cannot see or escape the abstraction when it fails, they stop trusting it.
- Start with the tool ("let's install Backstage") instead of the problem.

Do

- Name your customers, keep a public roadmap and measure adoption and time-to-first-deploy.
- Start with a template, a shared workflow and good docs.
- Offer an escape hatch and document the cost of leaving the paved path.
- Give the platform an SLO and an on-call, like any production service.
- Staff it with product thinking: a product manager or platform owner, not only engineers.

9 Tasks

- [ ] **T1 — Build the miniature TVP** *(≈45 min, beginner)*
  - **Goal:** a template plus a scaffolding script.
  - **Steps:**
    1. Follow steps 1-3 of the guide.
    2. Time how long it takes from `./new-service.sh` to a committed repo (`git init && git add . && git commit`) inside `out/orders-api`.
  - **Done when:** you have a scaffolded service in Git and a recorded "time to first commit".
- [ ] **T2 — Add a guardrail** *(≈30 min, beginner)*
  - **Goal:** make the script safer.
  - **Steps:**
    1. Make the script reject owners that do not start with `team-`.

2. Test both the success and failure cases.

- **Done when:** `./new-service.sh foo bar` fails with a clear message and `./new-service.sh foo team-bar` succeeds.

- [ ] **T3 — Interview a customer (or role-play one)** *(≈60 min, intermediate)*
  - **Goal:** discover real pain before building.
  - **Steps:**
    1. Ask two developers "walk me through the last time you started a new service" and list every wait or ticket.
    2. Rank the steps by time lost.
  - **Done when:** you have a ranked list of at least five friction points with times, and a proposed TVP for the top one.

- [ ] **T4 — Write the platform front page** *(≈45 min, intermediate)*
  - **Goal:** the documentation product.
  - **Steps:**
    1. Write `PLATFORM.md` as in step 5, including a "what we do NOT support" section and your three metrics.
    2. Have a teammate attempt to scaffold a service using only that page.
  - **Done when:** the teammate succeeds without asking you a question (note every question they did ask and fix the page).

- [ ] **T5 — Break it: drift after scaffolding** *(≈30 min, advanced)*
  - **Goal:** understand the limit of templates.
  - **Steps:**
    1. Scaffold two services, then change the template's Dockerfile (for example add a `HEALTHCHECK`).
    2. Compare the Dockerfiles in both services and in the template.
    3. Write down two ways to keep existing services current (shared workflows, Renovate/Dependabot PRs, a re-sync tool).
  - **Done when:** you can show the drift with `diff` and name which of your two ways you would choose and why.

- [ ] **T6 — Prove it: pitch the platform** *(≈30 min, advanced)*
  - **Goal:** explain platform-as-a-product.
  - **Steps:**

1. Without notes, explain in five minutes (to a teammate) what an IDP is, what a TVP is and why you would not start with a portal.
2. Name three metrics and one anti-pattern.

- **Done when:** the listener can restate the difference between a platform and a ticket queue.

10 Self-check questions

1. What problem does an IDP solve that "you build it, you run it" alone does not?
2. What is a Thinnest Viable Platform?
3. Name three signs a platform is being run as a product.
4. Why is mandating adoption early a risk?
5. Give two metrics you would use to judge a platform's success.

ANSWERS

1. It reduces cognitive load by packaging infrastructure, security and delivery into self-service capabilities, so teams do not each rebuild them.
2. The smallest set of capabilities and documentation that removes the most developer friction; it grows only as needs are proven.
3. Named customers and interviews, a roadmap and changelog, support channels and SLOs, and adoption and satisfaction metrics.
4. An immature platform that is forced on teams causes workarounds and resentment; adoption should be earned by being the easiest path.
5. For example time-to-first-deploy, % of services on the paved path, DORA metrics (lead time, deployment frequency), and developer satisfaction score.

11 Further reading

- *Team Topologies* by Matthew Skelton and Manuel Pais - platform teams and the Thinnest Viable Platform.
- CNCF Platforms White Paper (tag-app-delivery.cncf.io) - what a platform is and how to measure it.
- Backstage documentation (backstage.io) - the open-source portal framework.
- *Accelerate* by Nicole Forsgren, Jez Humble and Gene Kim - the research behind DORA metrics.
- See also: Developer Portals §18.2 and Golden Paths §18.4.

18.2 Developer Portals

Topic 2 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

A **developer portal** is the web front door to your internal developer platform (Internal Developer Platforms §18.1). It gives engineers one place to answer questions like: *What services exist? Who owns this one? How do I create a new service? Where are its docs, dashboards, on-call schedule and deployments?*

A portal is a **UI over metadata and actions**, not the platform itself. The metadata comes from the catalog (see Service Catalogs §18.5); the actions come from templates and automation behind the portal (Golden Paths §18.4, Self-service Infrastructure §18.3). Backstage, Port, Cortex and OpsLevel are well-known examples of portal products or frameworks.

2 Why it matters in DevOps

- **Findability:** in an organisation with hundreds of services, "who owns the payments queue and where is the runbook?" costs minutes during an incident - minutes that become money.
- **Onboarding:** a new engineer can discover, create and deploy a service without knowing five tools' URLs.
- **Consistency by default:** the portal is where templates, docs and standards become visible and easy to follow.
- **Risk:** a portal that is out of date, slow or hard to use is worse than none - it teaches people not to trust it. Adoption and data freshness are the real work.

3 Real-world system example

Backstage was created at Spotify to address "too many tools, too little discoverability" and was open-sourced in 2020; it is a CNCF project. Its core features, as documented on backstage.io, are the **Software Catalog**, **Software Templates** (the "scaffolder"), **TechDocs** (docs-as-code rendered in the portal) and **Search**, extended by a plugin ecosystem (for example

Kubernetes, CI and on-call plugins). Many companies run Backstage themselves; others use hosted versions or commercial portals such as Port, Cortex or OpsLevel, which offer catalogs, scorecards and self-service actions as a service. Which suits you depends on staffing, as the build-vs-buy section explains. Check each vendor's documentation for current features and pricing.

4 Diagram

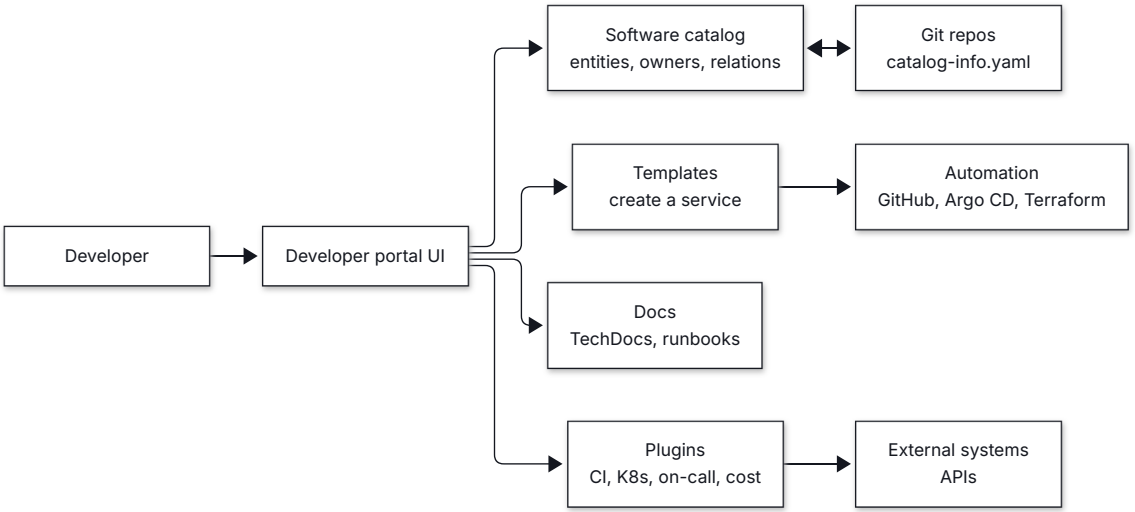


Figure 18.2-1

Where the portal sits in the "create a service" journey:

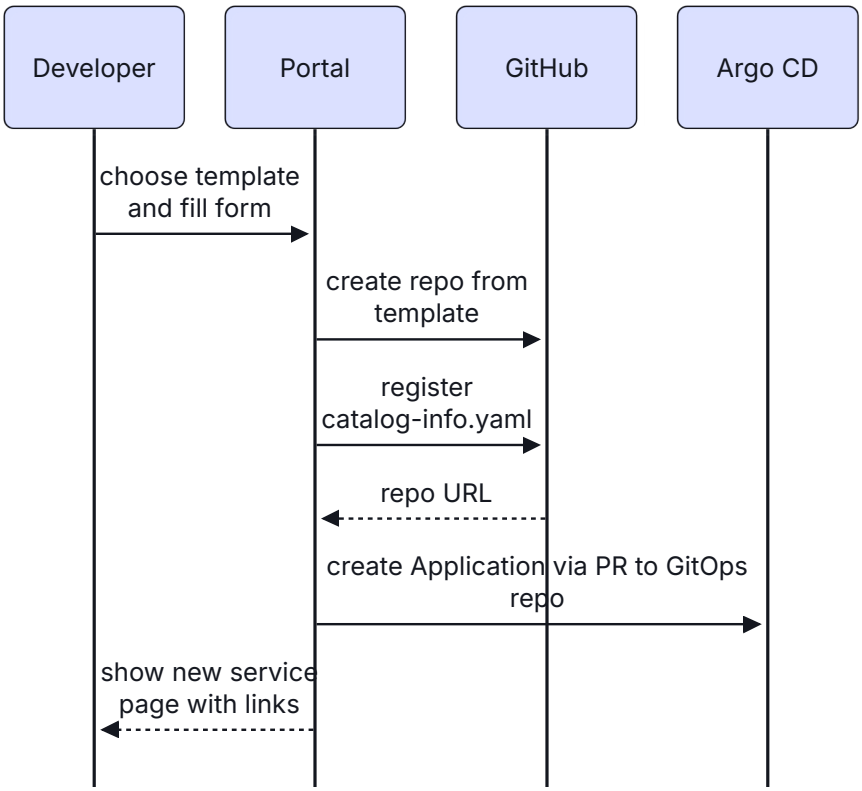


Figure 18.2-2

5 How it works

What a good portal offers

Capability	What it gives developers	Needs behind it
Catalog	searchable list of services, APIs, owners, lifecycle, dependencies	metadata in Git, ingestion
Templates / actions	"create service", "request database", "add pipeline"	automation and golden paths
Docs	docs-as-code, runbooks next to the service	Markdown in repos, a renderer
Scorecards	production-readiness and standards compliance	rules and data sources
Plugins / integrations	CI status, Kubernetes pods, alerts, costs, on-call in one page	APIs for each tool

Capability	What it gives developers	Needs behind it
Search	one search box across all of the above	indexing

Build versus buy

	Build on Backstage (self-hosted OSS)	Buy a portal product (SaaS)
Upfront effort	high: it is a framework you assemble and maintain (TypeScript, React)	low: configure and integrate
Flexibility	very high: write your own plugins	high within the product's model
Ongoing cost	engineers to upgrade, patch, host and write plugins	subscription fees
Time to first value	weeks to months	days to weeks
Lock-in	low (open source, your data in Git YAML)	medium (data model and workflows are vendor-specific)
Good fit	large orgs with a dedicated platform team	small and mid teams, or teams that want results fast

A common guideline from practitioners: unless you can dedicate roughly a small team to it, a framework-based portal will become a side project that decays. Hybrid choices exist (hosted Backstage, or a SaaS portal reading the same `catalog-info.yaml` files). Do not choose a portal first; choose the **problem** (discoverability? self-service? standards?) and evaluate against it.

Adoption is the hard part

- **Start with one high-value use case** (catalog with owners, or the "create service" template), not 30 plugins.
- **Keep data fresh automatically:** ingest from Git, CI and cloud APIs. Hand-edited portals rot.
- **Put it in the workflow:** link to it from CI output, alerts ("owner: team-shop, runbook") and onboarding docs.
- **Measure:** weekly active users, % of services with an owner, searches with no result, template usage.
- **Make it fast.** A portal that takes 10 seconds to load will be abandoned.
- **Own it like a product:** release notes, feedback channel, a named maintainer.

The minimal viable portal

The essence is small: collect metadata files from repos, render a list with owner and links. The guide builds exactly that in 40 lines of Python so you understand what Backstage's catalog is doing at its core.

6 Step-by-step guide

Prerequisites: `python3` with PyYAML (`pip install pyyaml`). Optional for step 6: Node.js 20+ and `yarn`. Cost: none.

1. **Create catalog entries (as if collected from three repos).**

```bash
mkdir -p portal/catalog && cd portal
cat > catalog/shop-api.yaml <<'EOF'
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: shop-api
  description: Node.js API for the shop
  annotations:
    github.com/project-slug: example/shop-api
spec:
  type: service
  lifecycle: production
  owner: team-shop
EOF
cat > catalog/shop-web.yaml <<'EOF'
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: shop-web
  description: Storefront frontend
spec:
  type: website
  lifecycle: production
  owner: team-web
EOF
cat > catalog/orders-worker.yaml <<'EOF'
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: orders-worker
spec:
  type: service
  lifecycle: experimental
  owner: team-shop
EOF
```

2. **Write the portal generator.**

```bash
cat > build_portal.py <<'EOF'
#!/usr/bin/env python3
"""Tiny static 'developer portal': reads catalog/*.yaml and writes site/index.html."""
import glob, html, pathlib, yaml

rows = []
for path in sorted(glob.glob("catalog/*.yaml")):
    doc = yaml.safe_load(open(path))
    meta, spec = doc["metadata"], doc.get("spec", {})
    slug = meta.get("annotations", {}).get("github.com/project-slug")
    rows.append({
        "name": meta["name"],
        "desc": meta.get("description", ""),
        "type": spec.get("type", "?"),
        "lifecycle": spec.get("lifecycle", "?"),
        "owner": spec.get("owner", "UNOWNED"),
        "repo": f"https://github.com/{slug}" if slug else "",
    })

def cell(v): return f"<td>{html.escape(v)}</td>"
body = "".join(
    "<tr>" + cell(r["name"]) + cell(r["type"]) + cell(r["lifecycle"]) +
  cell(r["owner"]) +
    (f'<td><a href="{html.escape(r["repo"])}">repo</a></td>' if r["repo"] else "<td>—
  </td>") + "</tr>"
    for r in rows)
pathlib.Path("site").mkdir(exist_ok=True)
pathlib.Path("site/index.html").write_text(
    "<!doctype html><meta charset=utf-8><title>Mini portal</title>"
    "<h1>Service catalog</h1><table border=1 cellpadding=6>"
    "<tr><th>Name</th><th>Type</th><th>Lifecycle</th><th>Owner</th><th>Source</th>
  </tr>"
    + body + "</table>")
print(f"wrote site/index.html with {len(rows)} entities")
for r in rows: print(f"  {r['name']:<14} {r['type']:<8} {r['lifecycle']:<13}
  {r['owner']}")
EOF
python3 build_portal.py
```

Expected output (verified):

```
wrote site/index.html with 3 entities
  orders-worker  service  experimental  team-shop
  shop-api       service  production    team-shop
  shop-web       website  production    team-web
```

3. **Serve and look at it.**

```bash
python3 -m http.server 8000 --directory site
```

   Open `http://localhost:8000`. You now have a (very) basic catalog UI built from YAML in Git: the same idea Backstage uses, minus search, auth, plugins and scale.

4. **Add a freshness check.** Append to the script (or a second script) a check that fails if any entity has `owner: UNOWNED`:

```bash
python3 - <<'EOF'
import glob, sys, yaml
bad = [p for p in glob.glob("catalog/*.yaml") if not
  yaml.safe_load(open(p)).get("spec", {}).get("owner")]
print("unowned:", bad)
sys.exit(1 if bad else 0)
EOF
```

   Run it in CI so ownerless services cannot be registered - a first "governance in the portal" rule.

5. **Try the real thing (optional, ~20-30 min).** Backstage's quickstart:

```bash
npx @backstage/create-app@latest      # prompts for an app name, e.g. dev-portal
cd dev-portal && yarn start           # some versions use: yarn dev
```

   Open `http://localhost:3000`, browse the sample catalog and "Create" page. Requirements and commands change between releases - follow the current "Getting Started" page on backstage.io. We go deeper in Backstage §18.8.

6. **Score a build-vs-buy decision.** Copy the table in section 5 into a doc and fill in your own numbers: people available, time to value required, number of services, existing integrations.

7 Real-world scenario

Illustrative scenario. A 120-engineer company launches a Backstage instance with 15 plugins after six months of work. Three months later weekly active users are 9% of engineers: the catalog is half empty (entries hand-registered), the "create service" template is a week behind real standards, and nobody remembers the URL. The platform team resets: they auto-ingest `catalog-info.yaml` from all repos with a GitHub discovery job, make PagerDuty-style

alerts link to the service page, add an owner field requirement to CI, and cut the plugin list to five. Active use passes 60% within two months. **Lesson:** a portal's value is the freshness and relevance of its data and its placement in daily workflows, not the number of features.

8 Common mistakes and best practices

Don't

- Start a portal project before you know which developer problem it solves.
- Underestimate the upkeep of a self-hosted framework (upgrades, plugin maintenance, auth, hosting).
- Rely on manual data entry for the catalog.
- Launch with dozens of plugins; each is a maintenance promise.
- Treat the portal as the platform: if there is no automation behind the buttons, it is a directory.

Do

- Ingest metadata from Git (`catalog-info.yaml`) and other sources automatically.
- Pick one or two use cases for launch and measure them.
- Integrate the portal into incident, CI and onboarding workflows.
- Require ownership metadata and enforce it in CI.
- Decide build versus buy with a written comparison and a review date.

9 Tasks

- [ ] **T1 — Build the mini portal** *(≈45 min, beginner)*
  - **Goal:** turn catalog YAML into a browsable page.
  - **Steps:**
    1. Follow steps 1-3.
    2. Add a fourth entity (`payments-db`, `type: database`).
  - **Done when:** the new entity appears after re-running the script, and the page loads at `localhost:8000`.
- [ ] **T2 — Add a column and a filter** *(≈45 min, intermediate)*
  - **Goal:** extend the portal's usefulness.
  - **Steps:**

    1. Add a `docs` link column from an annotation (`backstage.io/techdocs-ref` or your own).
    2. Add a simple filter: accept `--owner team-shop` on the command line and list only that team's services.
  - **Done when:** `python3 build_portal.py --owner team-shop` outputs two entities.

- [ ] **T3 — Enforce ownership in CI** *(≈30 min, intermediate)*
  - **Goal:** a governance rule that prevents rot.
  - **Steps:**
    1. Create the freshness script from step 4 as `check_owners.py`.
    2. Write a GitHub Actions job that runs it on pull requests.
    3. Open a PR that adds an entity without an owner.
  - **Done when:** the PR check fails with the unowned file listed, and passes after you add an owner.

- [ ] **T4 — Run Backstage locally** *(≈60 min, intermediate)*
  - **Goal:** compare the real thing with your mini portal.
  - **Steps:**
    1. Run the Backstage quickstart (step 5).
    2. Register your three YAML files by adding them as `catalog.locations` in `app-config.yaml` (`type: file`, `target: ../../portal/catalog/shop-api.yaml`, and so on).
    3. Find them in the catalog.
  - **Done when:** your entities show up in the Backstage catalog UI, and you can list three things Backstage does that the mini portal does not.

- [ ] **T5 — Write a build-vs-buy memo** *(≈60 min, advanced)*
  - **Goal:** make a defensible decision.
  - **Steps:**
    1. For a fictional company (80 engineers, 150 services, 3 platform engineers), fill in the section 5 table.
    2. Recommend self-hosted Backstage, hosted Backstage, or a SaaS portal, and list the first 90-day adoption plan with three metrics.
  - **Done when:** the memo states the decision, the two strongest arguments against it and a review date.

- [ ] **T6 — Prove it: pitch the portal** *(≈30 min, advanced)*

- **Goal:** explain what a portal is and is not.
- **Steps:**
  1. Without notes, explain to a teammate the difference between a portal, a catalog and a platform.
  2. Describe how you would keep its data fresh and how you would know if it is being used.
- **Done when:** the teammate can tell you which of the three is the UI, which is the data and which is the capability set.

10 Self-check questions

1. What is the difference between a developer portal and an internal developer platform?
2. Name four capabilities Backstage provides out of the box.
3. When does buying a portal product make more sense than building on Backstage?
4. Why do catalogs go stale and how do you prevent it?
5. Which metrics tell you whether a portal is succeeding?

ANSWERS

1. The portal is the user interface (one of several interfaces); the platform is the full set of capabilities, automation and guardrails behind it.
2. Software Catalog, Software Templates, TechDocs and Search (plus the plugin framework).
3. When there is no dedicated team to maintain a TypeScript framework, when time-to-value matters, or when the vendor's data model fits your needs.
4. Manual registration and no owner for the data; prevent it by ingesting from Git automatically and enforcing metadata in CI.
5. Weekly active users, % of services with an owner and docs, template usage, search success, and developer satisfaction.

11 Further reading

- Backstage documentation (backstage.io) - "What is Backstage?", Software Catalog, Software Templates, TechDocs.
- CNCF Platforms White Paper (tag-app-delivery.cncf.io) - capabilities and interfaces of a platform.
- Vendor documentation for Port, Cortex and OpsLevel if evaluating SaaS options.
- *Team Topologies* by Skelton and Pais - platform teams and their interfaces.

- See also: Service Catalogs §18.5 and Backstage §18.8.

LAB NOTES What I ran, what I saw, what it means

18.3 Self-service Infrastructure

Topic 3 of 10 | ~5 hours | Difficulty: Intermediate

1 What it is

Self-service infrastructure means a developer can obtain what they need - a namespace, a database, a queue, a DNS name, a preview environment - by *themselves*, in minutes, through an interface (Git pull request, portal form, CLI or API), **without filing a ticket and waiting for a human**. Crucially, it is **safe** self-service: requests are validated, sized, authorised and audited automatically by **guardrails**.

The shift is often described as **ticket-ops to API-ops**: from "email the infrastructure team and wait three days" to "submit a declarative request that the platform fulfils through automation". The request is data (a YAML file, a form submission, a custom resource); the platform reconciles it into real infrastructure, the same way GitOps reconciles manifests (Desired State Management §17.1).

2 Why it matters in DevOps

- **Wait time is the hidden cost.** A ticket that sits for two days to create a database is two days of a team's lead time, repeated hundreds of times a year.
- **Manual provisioning is inconsistent.** Different engineers create slightly different setups; nobody remembers which ones are encrypted or backed up.
- **Central teams don't scale.** If every request needs a human, the infrastructure team becomes the bottleneck - or developers bypass it ("shadow IT", personal cloud accounts).
- **But unguarded self-service is dangerous:** an unrestricted "create instance" button leads to 64-core instances left running over the weekend, public buckets and cost blowouts. Guardrails are what make self-service acceptable to security and finance.

3 Real-world system example

Several publicly documented tools exist to build this: **Backstage Software Templates** let a developer fill in a form that creates a repository and opens pull requests; **Crossplane** exposes cloud resources as Kubernetes custom resources and lets platform teams publish simplified "claims" (covered in Crossplane §18.9); **Terraform Cloud/Enterprise, Atlantis and similar tools** run `plan` / `apply` from pull requests; **AWS Service Catalog** offers pre-approved CloudFormation or Terraform products that users launch themselves under IAM and budget constraints. On Kubernetes, **namespace-per-team with ResourceQuota, LimitRange, NetworkPolicy and Pod Security Standards** is the standard minimal guardrail set, all documented in the Kubernetes docs. The hands-on below recreates that minimal pattern.

4 Diagram

From ticket-ops to API-ops:

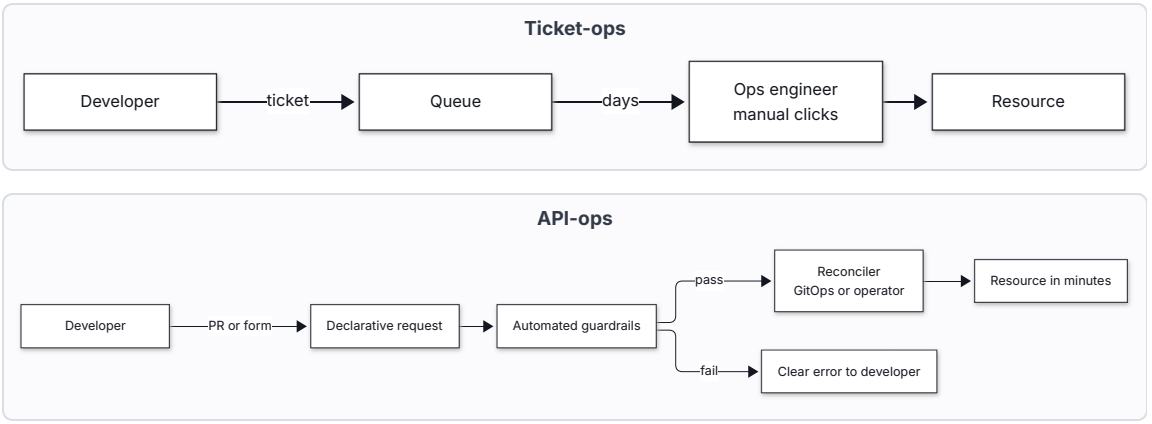


Figure 18.3-1

A request flow with approval only where risk demands it:

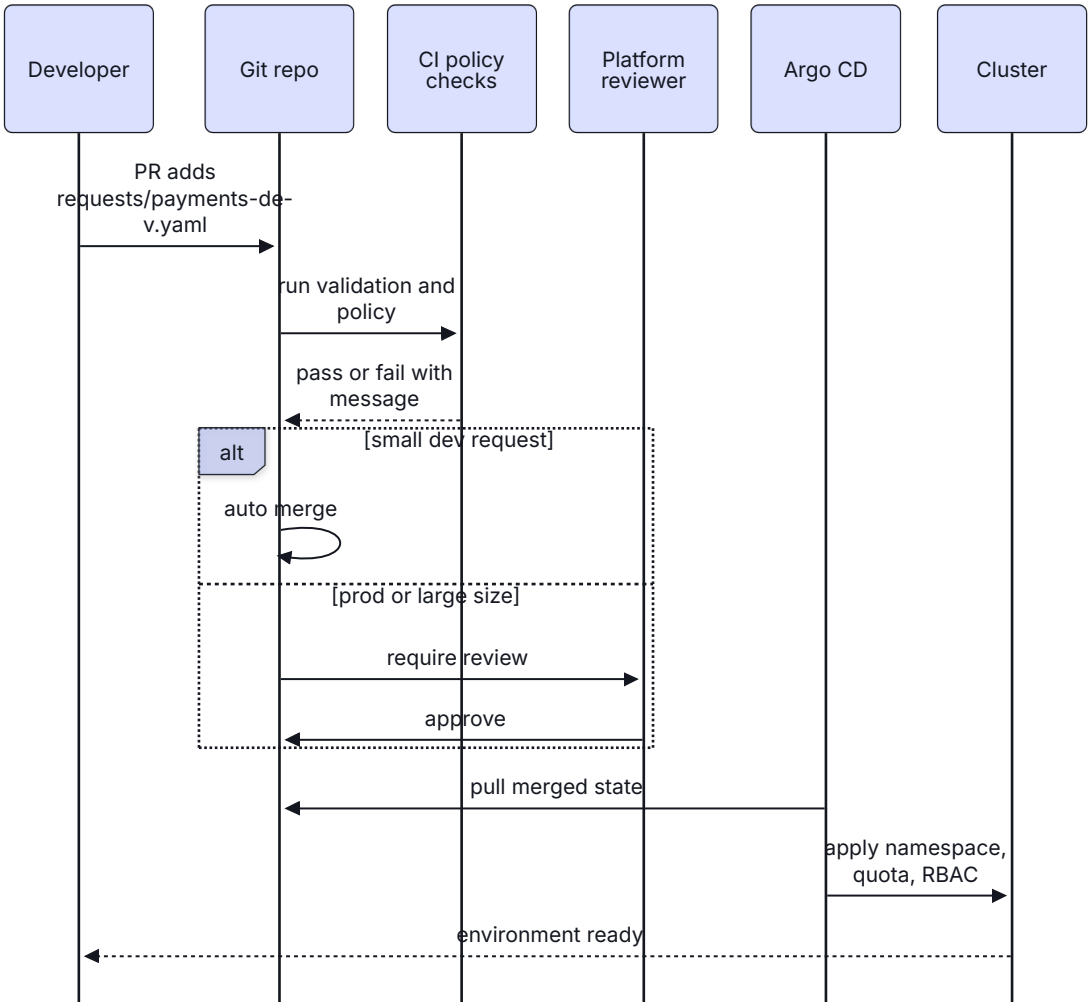


Figure 18.3-2

5 How it works

Anatomy of a self-service capability

1. **Interface:** how the developer asks (PR to a requests repo, portal form, `kubectl apply` of a custom resource, CLI).
2. **Request schema:** a small, opinionated, versioned contract (`size: small`, not 40 low-level parameters).

3. **Validation and policy:** reject bad input early with a helpful message (see Policy as Code §18.7).
4. **Fulfilment:** automation that creates the resource (GitOps, Terraform, Crossplane, an operator).
5. **Approval (when needed):** automatic for low risk, human for high risk or high cost.
6. **Feedback and lifecycle:** status visible to the requester, ownership recorded, expiry or cleanup for temporary resources.

Guardrails - the four families

Guardrail	Examples	Where enforced
Shape/size limits	only `small` / `medium` / `large`; instance types allow-list	request schema, CI validation
Security defaults	encryption on, no public buckets, Pod Security Standards, default-deny network policy	templates, admission control
Cost controls	namespace ResourceQuota, budgets and alerts, TTL on dev environments	quotas, tags, FinOps tooling (FinOps §20.1)
Authorisation and audit	RBAC per team, who-requested-what in Git history	RBAC (RBAC §8.18), Git

Interface choices

Interface	Strengths	Weaknesses
PR with a YAML request	auditable, reviewable, works with GitOps, no UI to build	needs Git fluency, slower feedback
Portal form	friendly, validates in the UI	UI is another system to build and keep in sync
Kubernetes custom resource (Crossplane claim)	one API for everything, native RBAC and reconciliation	need to run and understand the control plane
CLI or chatops	fast for power users	state and audit need extra work

Prefer *data* (declarative) over *scripts* for requests: you can validate, diff, review, reconcile and roll back data.

Measuring ticket-ops to API-ops

Track: median time from request to ready, % of requests fulfilled with no human touch, number of tickets per week for catalogued capabilities, and cost/security incidents attributable to self-service. If the first falls and the last stays flat, it is working.

Step-by-step guide

6

Prerequisites: `python3` with PyYAML; optional `kind` and `kubectl` for the final apply. Cost: none. We build a "namespace as a service" request pipeline.

1. **Define the request format** (the API contract).

```bash
mkdir -p selfsvc/requests && cd selfsvc
cat > requests/payments-dev.yaml <<'EOF'
apiVersion: platform.example.com/v1
kind: EnvironmentRequest
metadata:
  name: payments-dev
spec:
  team: team-payments
  environment: dev
  size: small
EOF
```

2. **Write the provisioner: validate then render.**

```bash
cat > provision.py <<'EOF'
#!/usr/bin/env python3
"""Validate an EnvironmentRequest and render the Kubernetes objects for it."""
import sys, yaml

SIZES = {  # guardrail: only these shapes exist
    "small":  {"cpu": "4",  "memory": "8Gi",  "pods": "20"},
    "medium": {"cpu": "8",  "memory": "16Gi", "pods": "40"},
}
ENVS = {"dev", "staging", "prod"}

def validate(req):
    errors = []
    spec = req.get("spec", {})
    if req.get("kind") ≠ "EnvironmentRequest":
        errors.append("kind must be EnvironmentRequest")
    if not str(spec.get("team", "")).startswith("team-"):
        errors.append("spec.team must start with 'team-'")
    if spec.get("environment") not in ENVS:
        errors.append(f"spec.environment must be one of {sorted(ENVS)}")
    if spec.get("size") not in SIZES:
        errors.append(f"spec.size must be one of {sorted(SIZES)} (larger sizes need a
  platform review)")
    if spec.get("environment") == "prod" and not spec.get("approvedBy"):
        errors.append("prod requires spec.approvedBy")
    return errors
```

```
def render(req):
    spec = req["spec"]
    ns = f"{spec['team']}-{spec['environment']}"
    q = SIZES[spec["size"]]
    return [
        {"apiVersion": "v1", "kind": "Namespace",
         "metadata": {"name": ns, "labels": {
             "platform.example.com/team": spec["team"],
             "pod-security.kubernetes.io/enforce": "baseline"}}},
        {"apiVersion": "v1", "kind": "ResourceQuota",
         "metadata": {"name": "default-quota", "namespace": ns},
         "spec": {"hard": {"requests.cpu": q["cpu"], "requests.memory": q["memory"],
  "pods": q["pods"]}}},
        {"apiVersion": "v1", "kind": "LimitRange",
         "metadata": {"name": "default-limits", "namespace": ns},
         "spec": {"limits": [{"type": "Container",
                              "defaultRequest": {"cpu": "100m", "memory": "128Mi"},
                              "default": {"cpu": "500m", "memory": "512Mi"}}]}},
        {"apiVersion": "networking.k8s.io/v1", "kind": "NetworkPolicy",
         "metadata": {"name": "default-deny-ingress", "namespace": ns},
         "spec": {"podSelector": {}, "policyTypes": ["Ingress"]}},
        {"apiVersion": "rbac.authorization.k8s.io/v1", "kind": "RoleBinding",
         "metadata": {"name": "team-edit", "namespace": ns},
         "subjects": [{"kind": "Group", "name": spec["team"], "apiGroup":
  "rbac.authorization.k8s.io"}],
         "roleRef": {"kind": "ClusterRole", "name": "edit", "apiGroup":
  "rbac.authorization.k8s.io"}},
    ]

if __name__ == "__main__":
    req = yaml.safe_load(open(sys.argv[1]))
    errs = validate(req)
    if errs:
        print("REJECTED:", *errs, sep="\n  - ", file=sys.stderr)
        sys.exit(1)
    print(yaml.safe_dump_all(render(req), sort_keys=False))
EOF
```

3. **Render a valid request.**

```
BASH
python3 provision.py requests/payments-dev.yaml
```

Expect (verified) five YAML documents: `Namespace team-payments-dev` with the Pod Security `baseline` label, a `ResourceQuota` (4 CPU, 8Gi, 20 pods), a `LimitRange` with defaults, a default-deny-ingress `NetworkPolicy` and a `RoleBinding` giving the team's group the `edit` role.

4. **Trigger the guardrails.**

```bash
sed 's/size: small/size: huge/; s/environment: dev/environment: prod/'
  requests/payments-dev.yaml > requests/bad.yaml
python3 provision.py requests/bad.yaml; echo "exit code: $?"
```

Expect (verified):

```
REJECTED:
  - spec.size must be one of ['medium', 'small'] (larger sizes need a platform review)
  - prod requires spec.approvedBy
exit code: 1
```

Note the message tells the requester exactly what to fix - good guardrails teach.

5. **Apply to a cluster (optional).** With `kind create cluster --name platform`:

```bash
python3 provision.py requests/payments-dev.yaml | kubectl apply -f -
kubectl get ns team-payments-dev --show-labels
kubectl -n team-payments-dev describe resourcequota default-quota
```

Expect the namespace with its labels and a quota table (output similar to the Kubernetes docs; numbers match the `small` size). Test enforcement: `kubectl -n team-payments-dev create deployment burst --image=nginx --replicas=30` creates the Deployment, but the ReplicaSet cannot create more than 20 pods, and `kubectl -n team-payments-dev get events` shows `exceeded quota` errors.

6. **Wire it into Git (the API-ops loop).** Put `requests/` and `provision.py` in a repo. Add a CI job that runs `provision.py` on every changed request (validation on PR) and a commit step that writes the rendered output to a GitOps directory that Argo CD syncs (Argo CD and Flux §17.2).

```yaml
# .github/workflows/validate-requests.yaml
name: validate-requests
on:
  pull_request:
    paths: ["requests/**"]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install pyyaml
```

```
      - name: Validate every request
        run: |
          for f in requests/*.yaml; do
            echo "== $f"
            python3 provision.py "$f" > /dev/null
          done
```

7. **Clean up** (if you created a cluster): `kind delete cluster --name platform`.

7 Real-world scenario

Illustrative scenario. Teams request staging environments by ticket; median wait is three working days and one in five tickets bounces back for missing information. The platform team introduces the PR-based `EnvironmentRequest`. Validation catches the missing fields in seconds, quotas stop a runaway test from starving the cluster, and a nightly job deletes environments labelled `ttl: 7d`. Median time-to-environment falls to under ten minutes for `small` requests; only `prod` and `large` still need a human. Two months later finance notices a 30% drop in "orphaned" dev spend, a benefit nobody had planned for. **Lesson:** guardrails and lifecycle management turn self-service from a risk into a cost control.

8 Common mistakes and best practices

Don't

- Expose raw cloud APIs ("here's an admin role, go create anything") and call it self-service.
- Ask developers for 40 parameters when 3 would do; hide complexity behind sensible defaults and sizes.
- Auto-approve everything including production and high-cost requests.
- Create resources without recording an owner and an expiry for temporary ones.
- Make validation errors cryptic.

Do

- Use declarative requests that live in Git, are reviewed where risk demands and are reconciled automatically.
- Provide a small menu of supported options and an escape hatch with a review process.
- Bake security defaults (encryption, network policy, Pod Security) into every template.
- Tag/label everything with team and cost centre; expire what is temporary.
- Measure time-to-ready and % of requests with no human touch.

9 Tasks

- [ ] **T1 — Run the provisioner** *(≈30 min, beginner)*
  - **Goal:** render and inspect a request.
  - **Steps:**
    1. Create the request and script (steps 1-2).
    2. Render `payments-dev` and list the object kinds.
  - **Done when:** `python3 provision.py requests/payments-dev.yaml | grep '^kind:'` prints five kinds.
- [ ] **T2 — Hit every guardrail** *(≈30 min, beginner)*
  - **Goal:** understand the validation rules.
  - **Steps:**
    1. Create requests that violate each rule (bad team prefix, unknown environment, unknown size, prod without approval).
    2. Record the error message for each.
  - **Done when:** each rule produces its own clear error and exit code 1.
- [ ] **T3 — Add a new guardrail and size** *(≈45 min, intermediate)*
  - **Goal:** modify the platform contract.
  - **Steps:**
    1. Add a `large` size that is allowed only when `spec.approvedBy` is set.
    2. Add an optional `ttlDays` field (max 30 for `dev`) that renders as a label `platform.example.com/ttl-days`.
  - **Done when:** a `large` request without approval is rejected, with approval it renders, and `ttlDays: 90` for dev is rejected.
- [ ] **T4 — Prove enforcement on a cluster** *(≈45 min, intermediate)*
  - **Goal:** guardrails that actually bite.
  - **Steps:**
    1. Apply the rendered namespace to a kind cluster.
    2. Try to exceed the pod quota (`kubectl create deployment burst --image=nginx --replicas=30 -n team-payments-dev`) and read the `FailedCreate` events.
    3. Try a privileged pod and observe the Pod Security rejection.
  - **Done when:** both attempts fail with quota and Pod Security messages that you can quote.

- [ ] **T5 — Close the GitOps loop** *(≈90 min, advanced)*
  - **Goal:** PR in, namespace out, no human.
  - **Steps:**
    1. Create a repo with `requests/` and the CI validation workflow.
    2. Extend CI (on merge to main) to commit rendered manifests to `generated/` which an Argo CD Application syncs.
    3. Merge a PR adding `requests/orders-dev.yaml`.
  - **Done when:** `team-orders-dev` exists in the cluster within minutes of the merge, with no manual `kubectl`.
- [ ] **T6 — Prove it: from tickets to APIs** *(≈30 min, advanced)*
  - **Goal:** present the case for self-service.
  - **Steps:**
    1. Without notes, list the six anatomy elements of a self-service capability and map each to your implementation.
    2. Propose three metrics and one risk you would monitor.
  - **Done when:** a teammate understands why the validation script is "the API" and what stops a developer from requesting a huge environment.

10 Self-check questions

1. What do "ticket-ops" and "API-ops" mean?
2. Name the four families of guardrails with an example each.
3. Why prefer declarative requests over scripts?
4. When should a self-service request require human approval?
5. How do ResourceQuota and LimitRange differ?

ANSWERS

1. Ticket-ops: humans fulfil requests from a queue. API-ops: requests are declarative data fulfilled by automation within minutes.
2. Shape limits (allowed sizes), security defaults (encryption, network policy), cost controls (quotas, TTLs) and authorisation/audit (RBAC, Git history).
3. Data can be validated, reviewed, diffed, reconciled and rolled back; scripts hide intent and are harder to audit.
4. When risk or cost is high: production, large sizes, public exposure, access to sensitive data.

5. ResourceQuota caps the aggregate resources for a namespace; LimitRange sets per-container defaults and min/max.

11 Further reading

- Kubernetes documentation - "Resource Quotas", "Limit Ranges", "Pod Security Standards", "Network Policies".
- Backstage documentation - Software Templates; Crossplane documentation - Composite Resources and Claims.
- CNCF Platforms White Paper - capabilities and self-service interfaces.
- See also: Namespaces §8.8, Pod Security §16.3 and Infrastructure as Code §0.3.

LAB NOTES What I ran, what I saw, what it means

18.4 Golden Paths

Topic 4 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

A **golden path** is an opinionated, well-supported, **self-service route** to accomplish a common task - most often "create a new service and get it to production" - in which the platform team has already made the boring decisions (language runtime, CI pipeline, container base image, Kubernetes manifests, logging, alerts, security scanning). A developer who follows the path gets a working, compliant, observable service in minutes, and does not need to know *why* each choice was made to be safe.

The idea is usually credited to Spotify, which describes "golden paths" as the supported way to build and ship software. Netflix uses the phrase **"paved road"** for the same concept. The key principle in both: **a path, not a cage**. Teams *can* leave it, but staying on it is so much easier that most choose to.

2 Why it matters in DevOps

- **Speed:** days of setup become a form submission and a few minutes of automation.
- **Safety by default:** security scanning, resource limits, health probes and non-root containers come built in (Non-root Containers §4.11).
- **Operability:** every service logs, exposes metrics and has an owner in the same way, so on-call engineers can debug an unfamiliar service quickly.
- **Maintenance leverage:** upgrade the path (a base image, a CI step) once and hundreds of services benefit - *if* services stay connected to the path.
- **The failure modes:** a path that is too rigid gets bypassed; one that is too loose provides no value; one that is mandated before it is good breeds hostility.

3 Real-world system example

Spotify's Backstage Software Templates are the open-source embodiment of golden paths: a template creates a repository from a skeleton, wires CI, registers the service in the catalog and links docs, so every new service starts the same way. **Netflix's "paved road"** is a publicly discussed philosophy: they provide a set of well-integrated tools and defaults and explicitly allow teams to deviate if they accept the extra operational burden. In the Kubernetes world, **Helm charts**, **Argo CD ApplicationSets** and **Crossplane compositions** are common building blocks behind a path. The *Team Topologies* book frames this as the platform team providing a "self-service, easy to consume" service to stream-aligned teams. (Details of any company's internal path change over time; rely on their published articles and talks.)

4 Diagram

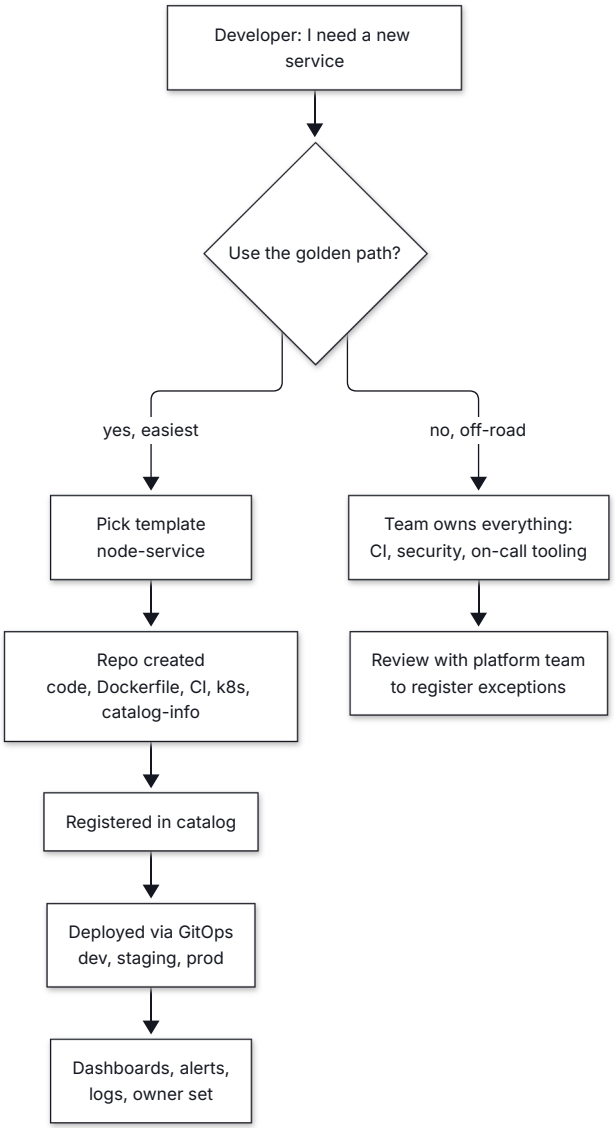


Figure 18.4-1

What a path contains (the layers a developer does not have to build):

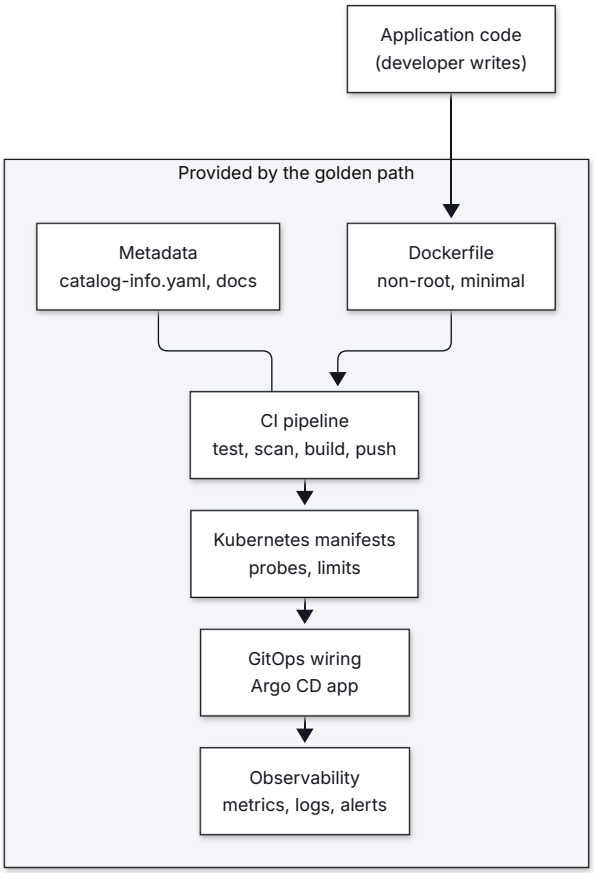


Figure 18.4-2

5 How it works

Paved road versus mandate

	Paved road (golden path)	Mandate
Adoption driver	value: it is faster and safer	policy: you must
Deviation	allowed, with accepted responsibility	blocked or requires exception
Feedback	teams tell you where it hurts, you improve it	teams hide workarounds

	Paved road (golden path)	Mandate
Best when	the platform is young and needs to earn trust	a hard compliance requirement exists (for example, scanning images)

A healthy pattern: **paved road for choices, mandate only for non-negotiables** (security baselines, mandatory ownership metadata), and enforce those via guardrails that are the same for everyone (Policy as Code §18.7).

What goes into a template

1. **Skeleton repo:** app starter, `Dockerfile`, tests, `README`.
2. **CI/CD:** a *thin* workflow that calls a shared, versioned workflow owned by the platform team (so improvements propagate).
3. **Deployment config:** Kubernetes manifests or Helm values plus the Argo CD `Application` or an entry in an `ApplicationSet` (Application Synchronization §17.5).
4. **Observability:** a metrics endpoint, structured logging, a default dashboard and alerts (Structured Logging §10.5).
5. **Metadata:** `catalog-info.yaml` with owner, lifecycle and a **label recording which path version it was created from** (enables adoption tracking).
6. **Docs:** a "getting started" page and runbook stub.

Template drift and keeping services current

A template is a *copy*; once copied, services drift. Mitigations:

- **Put logic in shared, versioned components** (reusable workflows, Helm library charts, base images) that services reference rather than copy.
- **Automated update PRs** (Renovate or Dependabot) when the shared component or path version changes.
- **Version the path** (`node-service-v1`, `v2`) and track who is on which version.

Measuring adoption

Metric	Meaning	Where it comes from
% of services created from a current path	reach	catalog labels
Time to first production deploy for a new service	the path's promise	repo creation to first deploy timestamps
Template usage per month	demand	portal/scaffolder logs
Deviation rate and reasons	where the path is wrong	exception log, survey

Metric	Meaning	Where it comes from
Developer satisfaction (CSAT or NPS)	perceived value	quarterly survey
Lead time, change failure rate (DORA)	outcome improvement	CI/CD data

Adoption is a **leading indicator**; DORA metrics and incident rates are outcomes. Watch both, and treat "nobody uses it" as feedback about the path, not about the users.

6 Step-by-step guide

Prerequisites: `bash`, `python3` with PyYAML. Optional: a Backstage instance from Developer Portals §18.2. Cost: none. We reuse the template idea from Internal Developer Platforms §18.1 and add versioning and adoption measurement.

1. **Make the template record its path version.** In the template's `catalog-info.yaml`, add a label:

   ```yaml
   apiVersion: backstage.io/v1alpha1
   kind: Component
   metadata:
     name: __NAME__
     labels:
       platform.example.com/golden-path: node-service-v2
   spec:
     type: service
     lifecycle: experimental
     owner: __OWNER__
   ```

   Every scaffolded service now carries its provenance.

2. **Simulate a fleet of services at different adoption levels.**

   ```bash
   mkdir -p repos/{shop-api,orders-worker,legacy-billing,shop-web,pricing}
   mk() { cat > repos/$1/catalog-info.yaml <<EOF
   apiVersion: backstage.io/v1alpha1
   kind: Component
   metadata:
     name: $1
   $2
   spec:
     type: service
     lifecycle: production
     owner: team-shop
   EOF
   }
   ```

```
mk shop-api "  labels:
    platform.example.com/golden-path: node-service-v2"
```

Repeat `mk` for the others: `orders-worker` on `node-service-v1`, `legacy-billing` with an empty second argument (no label), and `shop-web` and `pricing` on `node-service-v2`. Check one with `cat repos/shop-api/catalog-info.yaml`. (Alternatively, create the five files by hand; the point is the label values.)

3. **Write the adoption report.**

BASH

```
cat > adoption.py <<'EOF'
#!/usr/bin/env python3
"""Report golden path adoption from catalog-info.yaml files under repos/*/."""
import collections, glob, yaml

LABEL = "platform.example.com/golden-path"
CURRENT = "node-service-v2"
counts, laggards = collections.Counter(), []
for path in sorted(glob.glob("repos/*/catalog-info.yaml")):
    doc = yaml.safe_load(open(path))
    name = doc["metadata"]["name"]
    path_used = doc["metadata"].get("labels", {}).get(LABEL, "none")
    counts[path_used] += 1
    if path_used ≠ CURRENT:
        laggards.append((name, path_used))

total = sum(counts.values())
print(f"services: {total}")
for k, v in counts.most_common():
    print(f"  {k:<16} {v:>2}  {100*v/total:5.1f}%")
print(f"on current golden path ({CURRENT}): {100*counts[CURRENT]/total:.0f}%")
print("to migrate:", ", ".join(f"{n} ({p})" for n, p in laggards) or "nobody")
EOF
python3 adoption.py
```

Expected (verified for the five-service setup above):

```
services: 5
  node-service-v2   3   60.0%
  none              1   20.0%
  node-service-v1   1   20.0%
on current golden path (node-service-v2): 60%
to migrate: legacy-billing (none), orders-worker (node-service-v1)
```

This is the same query a catalog or scorecard tool would run (Service Catalogs §18.5).

4. **Express the path as a Backstage software template** (illustrative; Backstage's template syntax evolves, so check the current docs and see Backstage §18.8):

```yaml
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: node-service
  title: Node.js service (golden path v2)
  description: Creates a repo with CI, Kubernetes manifests, GitOps wiring and catalog
  registration
  tags: [nodejs, recommended]
spec:
  owner: group:platform-team
  type: service
  parameters:
    - title: Service details
      required: [name, owner]
      properties:
        name:
          title: Name
          type: string
          pattern: '^[a-z][a-z0-9-]{2,30}$'
        owner:
          title: Owner
          type: string
          ui:field: OwnerPicker
          ui:options:
            catalogFilter:
              kind: Group
  steps:
    - id: fetch
      name: Fetch skeleton
      action: fetch:template
      input:
        url: ./skeleton
        values:
          name: ${{ parameters.name }}
          owner: ${{ parameters.owner }}
    - id: publish
      name: Create repository
      action: publish:github
      input:
        repoUrl: github.com?owner=example&repo=${{ parameters.name }}
        description: Service ${{ parameters.name }}
    - id: register
      name: Register in catalog
      action: catalog:register
      input:
        repoContentsUrl: ${{ steps['publish'].output.repoContentsUrl }}
        catalogInfoPath: /catalog-info.yaml
```

5. **Define "done" for the path** (the contract you promise): write a short list of what every service gets on day one (CI green, image scanned, deployed to dev, dashboard, owner set, docs stub) and time a fresh scaffold against it. That timing is your headline metric.
6. **Plan an upgrade.** Create `node-service-v3` in your head (or a copy of the template with an added `HEALTHCHECK`) and decide how the 3 services on v2 will move: a bot PR per repo, a deadline, and an owner for stragglers. Update `CURRENT` in `adoption.py` and watch the percentage fall to 0% - the adoption chart always resets on a new version, which is why migration tooling matters.

7 Real-world scenario

Illustrative scenario. A platform team announces: "all new services must use the golden path from 1 March." Teams comply on paper but copy an old repo and rename it; the catalog shows "golden path adoption: 95%" because the label was copied too. A review of the metric against time-to-first-deploy and incident data shows the numbers do not match. The team switches approach: they make the path genuinely faster (scaffold to production in 20 minutes), add a real provenance check (the CI shared workflow version in use, not just a label), and talk to the three teams still off-road. Two of them have valid needs (GPU workloads) that become a second path (`ml-service`). **Lesson:** measure outcomes and real usage, not just labels, and let feedback from off-road teams shape new paths.

8 Common mistakes and best practices

Don't

- Mandate a path before it is better than the alternative.
- Copy logic into every service when it can live in a shared, versioned component.
- Build one path to rule them all; "web service", "batch job" and "data pipeline" have different needs.
- Judge success only by a label count.
- Forget day-2: upgrades, deprecations and migration help are part of the product.

Do

- Start with the most common journey and make it excellent.
- Version paths and keep a changelog and deprecation policy.
- Give opt-outs a clear process and learn from them.
- Make security and compliance defaults non-optional but invisible.
- Review adoption and satisfaction every quarter and retire paths nobody uses.

9 Tasks

- [ ] **T1 — Add provenance to a template** *(≈20 min, beginner)*
  - **Goal:** every scaffolded service records its path version.
  - **Steps:**
    1. Add the `golden-path` label to your template's `catalog-info.yaml` (step 1).
    2. Scaffold a service and check the label.
  - **Done when:** `grep golden-path out/<service>/catalog-info.yaml` shows `node-service-v2`.

- [ ] **T2 — Measure adoption** *(≈30 min, beginner)*
  - **Goal:** produce an adoption report.
  - **Steps:**
    1. Create the five sample repos and `adoption.py` (steps 2-3).
    2. Run it and note the percentages.
  - **Done when:** your output matches the expected report, and you can explain why `legacy-billing` is flagged.

- [ ] **T3 — Extend the report** *(≈45 min, intermediate)*
  - **Goal:** make the metric more useful.
  - **Steps:**
    1. Group adoption by `spec.owner` and print the team with the lowest adoption.
    2. Add a `--json` flag that outputs the data for a dashboard.
  - **Done when:** the script prints per-team percentages and valid JSON with `--json` (check with `python3 -m json.tool`).

- [ ] **T4 — Make the path deliver value (shared workflow)** *(≈60 min, intermediate)*
  - **Goal:** put logic in a shared component rather than the template.
  - **Steps:**
    1. Create a repo with a reusable GitHub Actions workflow (`on: workflow_call`) that runs tests and a Trivy scan (Container Image Scanning §11.9).
    2. Change the template's `ci.yaml` to `uses: <owner>/<repo>/.github/workflows/node-service.yaml@v1`.
    3. Add a scan step to the shared workflow only, and observe a service pick it up without changing its own files.

- **Done when:** a scaffolded service runs the new scan step after you changed only the shared repo.

- [ ] **T5 — Break it: the copy-paste cheat** *(≈45 min, advanced)*
  - **Goal:** see why labels alone are weak.
  - **Steps:**
    1. Copy `legacy-billing` and add the v2 label by hand without adopting anything else.
    2. Extend `adoption.py` to also check that `.github/workflows/ci.yaml` references the shared workflow and report "label but not real" cases.
  - **Done when:** the report distinguishes real adopters from label-only ones.

- [ ] **T6 — Prove it: design a path** *(≈45 min, advanced)*
  - **Goal:** specify a second golden path.
  - **Steps:**
    1. Without notes, write a one-page spec for a "scheduled batch job" path: what the developer provides, what the platform provides, success metrics, and the off-road policy.
    2. Explain how you would persuade a skeptical team to try it.
  - **Done when:** a teammate can tell what the path gives them, what it costs them, and how you will know it worked.

10 Self-check questions

1. What is the difference between a golden path and a mandate?
2. Why does a template alone not keep services up to date?
3. Name four metrics for golden path success.
4. Why version golden paths?
5. What should happen when a team goes off-road?

ANSWERS

1. A golden path wins by being the easiest, best-supported route and allows deviation; a mandate forces compliance regardless of value.
2. A template is copied once and then drifts; keep logic in shared versioned components and use automated update PRs.
3. For example: % of services on the current path, time to first production deploy, deviation rate and reasons, developer satisfaction, DORA metrics.

4. So improvements can ship without breaking existing services, adoption can be tracked per version and old versions can be deprecated.
5. They accept responsibility for the extra operational burden, register the exception, and the platform team learns why - the feedback may become a new path.

11 Further reading

- Spotify Engineering blog and Backstage documentation - "Golden Path" and Software Templates.
- Netflix Technology Blog - talks and posts on the "paved road".
- *Team Topologies* by Skelton and Pais - platform as a self-service product.
- CNCF Platforms White Paper - "Measuring your platform".
- See also: Internal Developer Platforms §18.1 and Service Catalogs §18.5.

18.5 Service Catalogs

Topic 5 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

A **service catalog** is a registry of the software your organisation runs - services, websites, libraries, APIs, databases, queues - together with **metadata** about each: who owns it, what lifecycle stage it is in, what it depends on, where its code, docs, dashboards and runbooks are. It is the "source of truth about our software" that the developer portal §18.2 displays and that automation can query.

In Backstage (the most widely used open-source catalog) the metadata lives **as code next to the service**, in a file called `catalog-info.yaml` at the repo root. A catalog ingests those files, builds a graph of entities and relationships, and exposes it through a UI and API. Because the data lives in Git, it is versioned, reviewed and owned by the team that knows it best.

2 Why it matters in DevOps

- **Ownership during incidents:** the first question at 3 a.m. is "who owns this service and how do I reach them?" A catalog answers in seconds; an outdated spreadsheet costs an hour.
- **Blast radius and impact analysis:** a **dependency graph** shows what breaks if a database goes down or an API changes.
- **Standards at scale: scorecards** measure each service against production-readiness criteria (runbook? on-call? tests? SLO?) and make gaps visible without nagging.
- **Compliance and cost:** auditors ask who owns what; FinOps allocates cost by owner (FinOps §20.1); security wants to find every service using a vulnerable library.
- **Automation:** CI, alerting and Argo CD can read owner and tier from the catalog instead of hard-coding it.

3 Real-world system example

Backstage's Software Catalog is documented with a formal model: entity **kinds** such as `Component`, `API`, `Resource`, `System`, `Domain`, `Group` and `User`, described in `catalog-info.yaml` with `apiVersion: backstage.io/v1alpha1`. Relations such as `ownedBy`, `dependsOn`, `providesApi` / `consumesApi` and `partOf` are derived from fields like `spec.owner`, `spec.dependsOn`, `spec.providesApis` and `spec.system`. The catalog was created at Spotify to answer "what do we own and who owns it?" at scale. Commercial portals (Port, Cortex, OpsLevel and others) provide catalogs and **scorecards** or "maturity" checks as core features, often ingesting the same `catalog-info.yaml` format or integrating with cloud and CI APIs. For the data model's details, see the official Backstage documentation, since it evolves.

4 Diagram

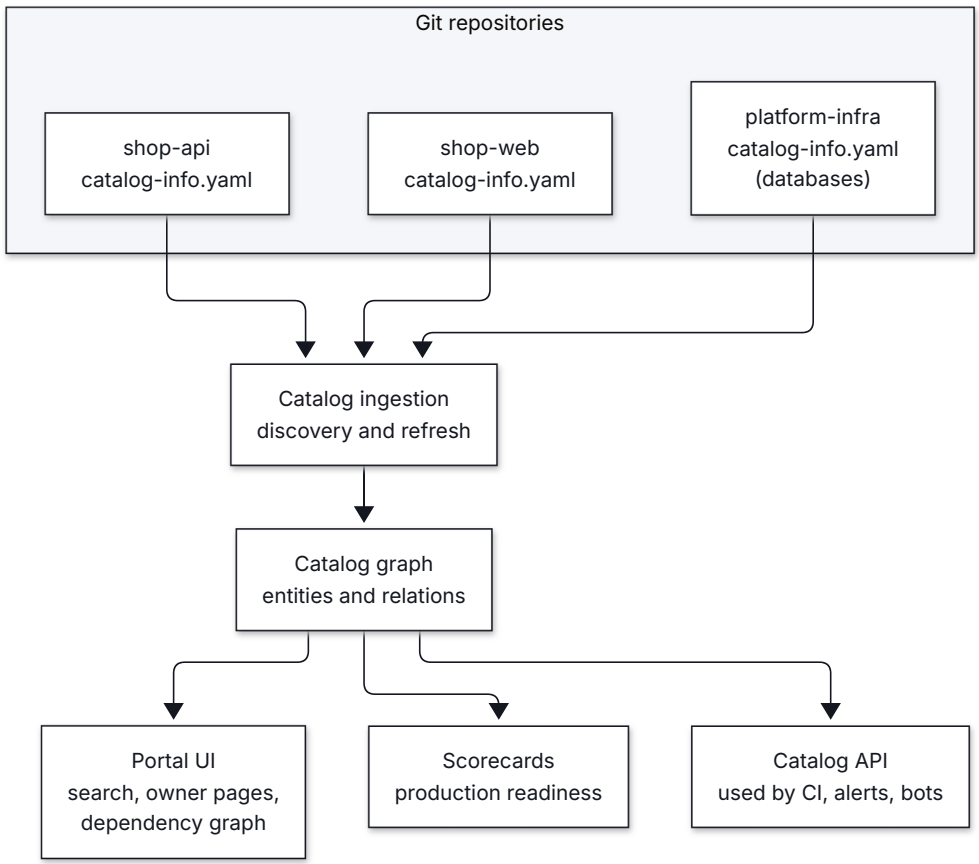


Figure 18.5-1

An example dependency graph for the running `shop` system:

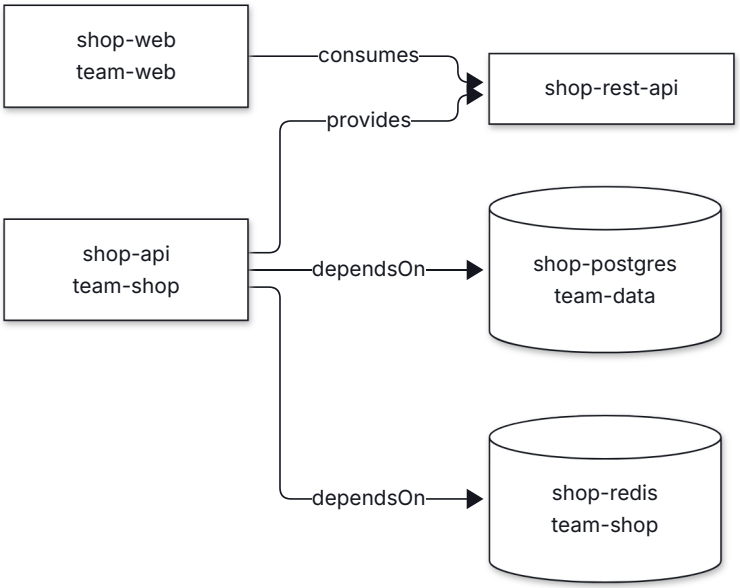


Figure 18.5-2

5 How it works

Entity kinds (Backstage model)

Kind	Represents	Example
`Component`	a piece of software (`type: service`, `website`, `library`)	`shop-api`
`API`	an interface a component exposes (OpenAPI, gRPC, AsyncAPI)	`shop-rest-api`
`Resource`	infrastructure a component needs	`shop-postgres`, an S3 bucket
`System`	a group of components and resources that make a product	`shop`
`Domain`	a group of systems in a business area	`commerce`
`Group` / `User`	teams and people (owners)	`team-shop`

Anatomy of `catalog-info.yaml`

```yaml
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: shop-api
```

```
  description: Node.js API for the shop
  annotations:
    github.com/project-slug: example/shop-api
    backstage.io/techdocs-ref: dir:.
  links:
    - url: https://example.com/runbooks/shop-api
      title: Runbook
  tags: [nodejs, tier-1]
spec:
  type: service
  lifecycle: production        # experimental | production | deprecated
  owner: team-shop             # a Group entity: the single accountable team
  system: shop
  providesApis:
    - shop-rest-api
  dependsOn:
    - resource:shop-postgres
    - resource:shop-redis
```

Key fields: `metadata.name` (unique), `spec.owner` (**mandatory in practice**), `spec.lifecycle`, `spec.system` (grouping), `dependsOn` (graph edges) and `annotations` (machine-readable hooks: the repo, docs location, Kubernetes label selector, on-call schedule identifier and so on, depending on installed plugins).

Ownership metadata - the rules that make it useful

1. **One owning team per entity**, a team (not a person) with a contact channel and an on-call rotation.
2. **Required in CI:** a repo without a valid `catalog-info.yaml` fails the pipeline.
3. **Reorg-proof:** owners reference `Group` entities, so renaming a team is one change.
4. **Lifecycle honesty:** mark `experimental` and `deprecated` accurately so people know what to rely on.

Getting data into the catalog

Method	Notes
Static `locations` in config	fine for a handful of repos
Auto-discovery of `catalog-info.yaml` across a GitHub org	scalable; the usual choice
Software template registers new services	new services appear automatically (Golden Paths §18.4)
Entity providers/importers	pull from cloud accounts, Kubernetes, CMDBs, Terraform state

Dependency graphs

Edges come from `dependsOn`, `consumesApis` / `providesApis` and `partOf`. Manually declared edges can drift from reality, so mature setups also **derive** edges from runtime data (service mesh, tracing - see Distributed Tracing §10.8 - or Kubernetes) and use the catalog as the human-curated layer on top.

Scorecards

A scorecard evaluates each entity against rules and reports pass/fail and a level (bronze/silver/gold, or "production ready").

Check	Why
Has an owner and an on-call route	someone gets paged
Has a runbook link	responders know what to do
Has docs (TechDocs)	knowledge outlives people
Has an SLO and dashboard	reliability is measured (SLOs §14.2)
CI passes, image scanned, no critical vulnerabilities	security baseline
Uses a supported golden path version	maintainability

Scorecards work best as **visibility plus gentle pressure** (team dashboards, quarterly goals), not as gates that block deploys on day one.

6 Step-by-step guide

Prerequisites: `python3` with PyYAML. Optional: your Backstage instance from Developer Portals §18.2 to ingest the same files. Cost: none.

1. **Write the catalog for the `shop` system.** Create `catalog/shop.yaml` containing several entities separated by `---`:

```yaml
apiVersion: backstage.io/v1alpha1
kind: System
metadata:
  name: shop
  description: The online shop
spec:
  owner: team-shop
---
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
```

```
  name: shop-api
  description: Node.js API for the shop
  annotations:
    github.com/project-slug: example/shop-api
    backstage.io/techdocs-ref: dir:.
  links:
    - url: https://example.com/runbooks/shop-api
      title: Runbook
spec:
  type: service
  lifecycle: production
  owner: team-shop
  system: shop
  providesApis:
    - shop-rest-api
  dependsOn:
    - resource:shop-postgres
    - resource:shop-redis
---
apiVersion: backstage.io/v1alpha1
kind: API
metadata:
  name: shop-rest-api
spec:
  type: openapi
  lifecycle: production
  owner: team-shop
  system: shop
  definition: |
    openapi: 3.0.0
    info: {title: shop-api, version: "1.0"}
    paths: {}
---
apiVersion: backstage.io/v1alpha1
kind: Resource
metadata:
  name: shop-postgres
spec:
  type: database
  owner: team-data
  system: shop
---
apiVersion: backstage.io/v1alpha1
kind: Resource
metadata:
  name: shop-redis
spec:
  type: cache
  owner: team-shop
  system: shop
---
apiVersion: backstage.io/v1alpha1
kind: Component
```

```
metadata:
  name: shop-web
  description: Storefront
  annotations:
    github.com/project-slug: example/shop-web
spec:
  type: website
  lifecycle: production
  owner: team-web
  system: shop
  consumesApis:
    - shop-rest-api
```

In real life each component's entry lives in its own repo; one file is simpler for practice.

2. **Generate the dependency graph and scorecard.**

BASH

```
cat > catalog_report.py <<'EOF'
#!/usr/bin/env python3
"""Read catalog/*.yaml; print a Mermaid dependency graph and a scorecard."""
import glob, yaml

entities = []
for path in sorted(glob.glob("catalog/*.yaml")):
    entities += [d for d in yaml.safe_load_all(open(path)) if d]

def ref(e): return f"{e['kind'].lower()}:{e['metadata']['name']}"
by_ref = {ref(e): e for e in entities}

print("```mermaid\nflowchart LR")
for e in entities:
    name = e["metadata"]["name"]
    node = name.replace("-", "_")
    for dep in e["spec"].get("dependsOn", []):
        print(f"    {node} ⟶|dependsOn| {dep.split(':')[1].replace('-', '_')}")
    for api in e["spec"].get("consumesApis", []):
        print(f"    {node} ⟶|consumes| {api.replace('-', '_')}")
print("```\n")

CHECKS = {
    "has owner":       lambda e: bool(e["spec"].get("owner")),
    "has description": lambda e: bool(e["metadata"].get("description")),
    "has repo link":   lambda e: "github.com/project-slug" in
  e["metadata"].get("annotations", {}),
    "has runbook":     lambda e: any("runbook" in l.get("title", "").lower() for l in
  e["metadata"].get("links", [])),
    "has docs":        lambda e: "backstage.io/techdocs-ref" in
  e["metadata"].get("annotations", {}),
}
print(f"{'component':<12}", *[f"{c:<16}" for c in CHECKS], "score", sep=" ")
for e in entities:
```

```
    if e["kind"] != "Component": continue
    res = [fn(e) for fn in CHECKS.values()]
    print(f"{e['metadata']['name']:<12}", *[f"{'PASS' if r else 'FAIL':<16}" for r in
  res], f"{sum(res)}/{len(res)}", sep=" ")

# broken references
for e in entities:
    for dep in e["spec"].get("dependsOn", []):
        if dep not in by_ref:
            print("BROKEN dependsOn:", ref(e), "→", dep)
EOF
python3 catalog_report.py
```

Expected output (verified):

```
```mermaid
flowchart LR
    shop_api ⟶|dependsOn| shop_postgres
    shop_api ⟶|dependsOn| shop_redis
    shop_web ⟶|consumes| shop_rest_api
```

component   has owner   has description has repo link   has runbook   has
  docs   score
shop-api   PASS   PASS   PASS   PASS   PASS
  5/5
shop-web   PASS   PASS   PASS   FAIL   FAIL
  3/5
```

(The first block is Mermaid text you can paste into any Mermaid renderer or a GitHub Markdown file; GitHub renders it.)

3. **Break a reference on purpose.** Change `resource:shop-postgres` to `resource:shop-postgress` and re-run. The script prints `BROKEN dependsOn: component:shop-api → resource:shop-postgress`. Real catalogs flag unresolved relations in the UI; you should also fail CI on them.
4. **Add one more rule to the scorecard** - for example "component with `lifecycle: production` must have a `tier-*` tag". Add the tag to `shop-api` only and confirm the table changes.

5. **Ingest it into Backstage (optional).** In your local Backstage `app-config.yaml`:

```yaml
catalog:
  locations:
    - type: file
      target: ../../catalog/shop.yaml
      rules:
        - allow: [Component, API, Resource, System]
```

   Restart and open **Catalog**; click `shop-api` and open the **Dependencies** tab / system diagram. Group entities (`team-shop`, `team-web`, `team-data`) must also be registered for owner links to resolve. Paths and plugin tabs depend on version; see the Backstage "Software Catalog" docs.

6. **Add an ownership check to CI** so every repo must carry a valid file:

```bash
python3 - <<'EOF'
import sys, yaml
docs = [d for d in yaml.safe_load_all(open("catalog-info.yaml")) if d]
problems = []
for d in docs:
    if not d.get("spec", {}).get("owner"):
        problems.append(f"{d['metadata']['name']}: missing spec.owner")
    if d["kind"] == "Component" and d["spec"].get("lifecycle") not in {"experimental",
  "production", "deprecated"}:
        problems.append(f"{d['metadata']['name']}: bad lifecycle")
print("\n".join(problems) or "catalog-info.yaml OK")
sys.exit(1 if problems else 0)
EOF
```

7 Real-world scenario

Illustrative scenario. At 02:10 an alert fires: `shop-redis` latency is high. The on-call engineer does not recognise the cache. They search the catalog, find `shop-redis`, see `owner: team-shop`, a dependency view showing `shop-api` and `orders-worker` depend on it, and a runbook link. They page the right team and warn the owners of the two dependants in one message. Without the catalog, they would have spent 40 minutes asking in chat. In the post-incident review the team notices that `orders-worker` was missing from the graph because its `dependsOn` was never declared; they add a scorecard check "declares its dependencies" and derive edges from tracing as a cross-check. **Lesson:** a catalog pays off in minutes saved during incidents, and only if the data is complete and fresh.

8 Common mistakes and best practices

Don't

- Maintain the catalog by hand in a wiki or spreadsheet.
- Assign ownership to individuals or to teams that no longer exist.
- Let scorecards become a punishment; metrics that block everyone get gamed or ignored.
- Model every library and script on day one; start with production services and their datastores.
- Trust hand-declared dependency edges as complete.

Do

- Keep `catalog-info.yaml` in each repo and auto-discover it.
- Require `owner`, `lifecycle` and `system` in CI, and validate references.
- Link owners to on-call, chat channels and runbooks.
- Start scorecards with 4-6 checks that matter during incidents, then grow.
- Cross-check declared dependencies with runtime data.

9 Tasks

- [ ] **T1 — Write a catalog entry** *(≈30 min, beginner)*
  - **Goal:** describe `shop-api` as a Component.
  - **Steps:**
    1. Create `catalog-info.yaml` with name, description, type, lifecycle, owner and a GitHub annotation.
    2. Validate that it parses (`python3 -c "import yaml; print(list(yaml.safe_load_all(open('catalog-info.yaml'))))"`).
  - **Done when:** the file parses and contains an owner and lifecycle.
- [ ] **T2 — Model the system** *(≈45 min, beginner)*
  - **Goal:** add Resources, an API and a System.
  - **Steps:**
    1. Create the entities from step 1, then add an `orders-worker` component that depends on `shop-redis` and `shop-postgres`.
    2. Run `catalog_report.py` and read the graph.
  - **Done when:** the Mermaid graph shows `orders_worker` with two `dependsOn` edges.

- [ ] **T3 — Extend the scorecard** *(≈45 min, intermediate)*
  - **Goal:** add two more checks and a letter grade.
  - **Steps:**
    1. Add checks for "has `tier-*` tag" and "has SLO link" (an annotation or link titled SLO).
    2. Print a grade (5/5 gold, 3-4 silver, otherwise bronze).
  - **Done when:** `shop-web` is bronze or silver and `shop-api` gold, with the reasons visible.
- [ ] **T4 — Break it: catch a broken reference** *(≈30 min, intermediate)*
  - **Goal:** make CI catch bad metadata.
  - **Steps:**
    1. Misspell a dependency and an owner.
    2. Extend the script to also report unknown owners (not in a `Group` list) and exit non-zero.
    3. Wire it into a GitHub Actions job on pull requests.
  - **Done when:** the PR check fails with both errors listed, and passes after you fix them.
- [ ] **T5 — Ingest into Backstage and use it in an incident drill** *(≈90 min, advanced)*
  - **Goal:** see the catalog help answer "who owns this?".
  - **Steps:**
    1. Register your YAML as a file location in a local Backstage instance (and add Group entities for the teams).
    2. Pretend `shop-redis` is failing: use only the portal to find the owner, dependants and runbook, and time yourself.
  - **Done when:** you identified owner, two dependants and the runbook link in under two minutes, and wrote down what was missing.
- [ ] **T6 — Prove it: design the metadata standard** *(≈45 min, advanced)*
  - **Goal:** specify what every service must declare.
  - **Steps:**
    1. Without notes, write a one-page "catalog-info.yaml standard": required fields, naming, lifecycle values, annotation conventions, and the five scorecard checks.
    2. Explain how you will keep the catalog fresh and how you will roll the standard out to 100 existing repos.
  - **Done when:** a teammate can add a correct entry for a new service from your page alone, and your rollout plan names the automation (discovery, CI check, bot PRs).

10 Self-check questions

1. What does `spec.owner` refer to, and why must it be a team?
2. Which fields create dependency edges in the Backstage catalog?
3. What is a scorecard and why not use it as a hard deploy gate at first?
4. Why store `catalog-info.yaml` in each repository?
5. How can you detect missing or stale dependency edges?

ANSWERS

1. A `Group` entity (team) that is accountable for the component; teams are stable and have on-call, while individuals leave or change roles.
2. `dependsOn`, `providesApis` / `consumesApis` and `system` (partOf) relations.
3. A scorecard checks entities against production-readiness rules; hard gates before the data is accurate create friction and gaming, so start as visibility.
4. The owning team keeps it current in the same PR as the change, it is versioned and reviewed, and discovery scales across the organisation.
5. Compare declared edges against runtime data (traces, mesh metrics, Kubernetes) and fail CI on unresolved references.

11 Further reading

- Backstage documentation - "Software Catalog": descriptor format, system model, well-known annotations and relations.
- CNCF Platforms White Paper - catalogs and metadata as platform capabilities.
- Vendor documentation for Port, Cortex or OpsLevel on scorecards and maturity models.
- *Team Topologies* by Skelton and Pais - ownership and team APIs.
- See also: Golden Paths §18.4 and Backstage §18.8.

18.6 Infrastructure Templates

Topic 6 of 10 | ~4 hours | Difficulty: Intermediate

1 What it is

An **infrastructure template** is a reusable, parameterised, versioned definition of "a thing the platform knows how to build": a database, a queue, a namespace with quotas, a whole service environment. Instead of every team copying 200 lines of Terraform from the last team that did it, they call the template with five inputs and get a result that already contains the organisation's defaults for naming, tagging, encryption, backups and networking.

Templates come in several shapes, and a platform usually uses more than one: **Terraform/OpenTofu modules**, **Crossplane Compositions** (see Crossplane §18.9), **Helm charts** (see Charts §9.1), and **scaffolding templates** that generate a repository (cookiecutter, Backstage software templates, see Backstage §18.8). They all share three problems that this topic is about: how to make them *reusable* without making them unusable, how to *version* them so a fix does not break every consumer at once, and how to *test* them so a change to the template is not first discovered in production.

2 Why it matters in DevOps

- **Consistency is the product.** If twenty teams each hand-roll an RDS instance, you get twenty slightly different backup windows, tag schemes and security groups. A template turns the policy into the default.
- **Fixes propagate.** When a vulnerability or a cost problem is found in the standard database setup, you change one template and roll it out, instead of auditing every repository.
- **Unversioned templates are outages waiting to happen.** A module consumed from `main` means any merge can change every consuming team's next `terraform apply` -- often a forced replacement of a stateful resource.
- **Untested templates ship bugs at platform scale.** One typo in a shared module breaks every team at once.
- **Cost and compliance.** Required tags (`owner`, `cost-center`) baked into a template are what make cost allocation work later (see FinOps §20.1).

See also: Modules §7.3 for module basics and Self-service Infrastructure §18.3 for how templates get exposed to developers.

3 Real-world system example

The public Terraform Registry and the `terraform-aws-modules` family. The Terraform Registry hosts community and vendor modules (for example the widely used `terraform-aws-modules/vpc/aws`). Every module there is versioned with Git tags following semantic versioning, and consumers pin with a constraint such as `version = "~> 5.0"`. That is exactly the model internal platforms copy: a module per repository, release tags, a documented input/output contract and a changelog.

Kubernetes-native examples. Crossplane's Compositions package "a database" as a Composite Resource Definition (XRD) plus one or more Compositions, and publish them as versioned **Configuration packages** (OCI images) that a cluster installs. Helm charts follow the same idea with `Chart.yaml` `version` and chart repositories/OCI registries. Backstage software templates (used at Spotify, which open-sourced Backstage, and by many adopters) scaffold a new repository and then register it in the catalog so the generated service is immediately discoverable.

4 Diagram

The life of a template, from change to consumer:

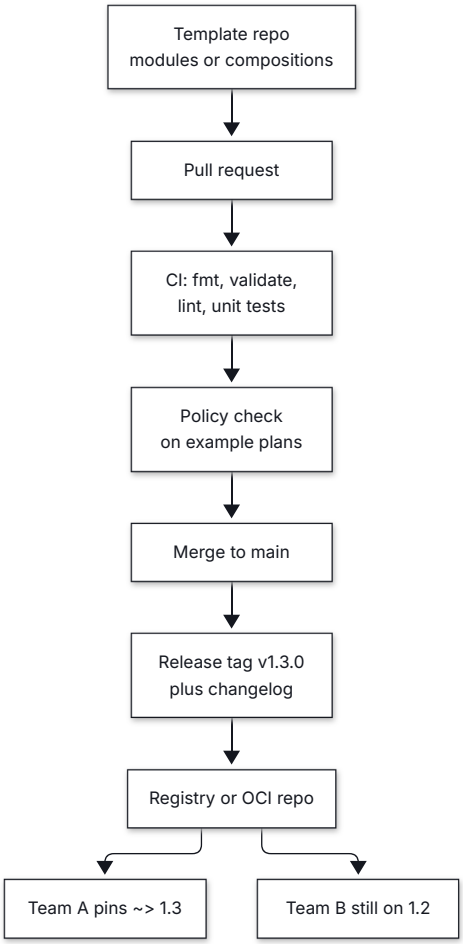


Figure 18.6-1

How a consumer is insulated from a breaking change:

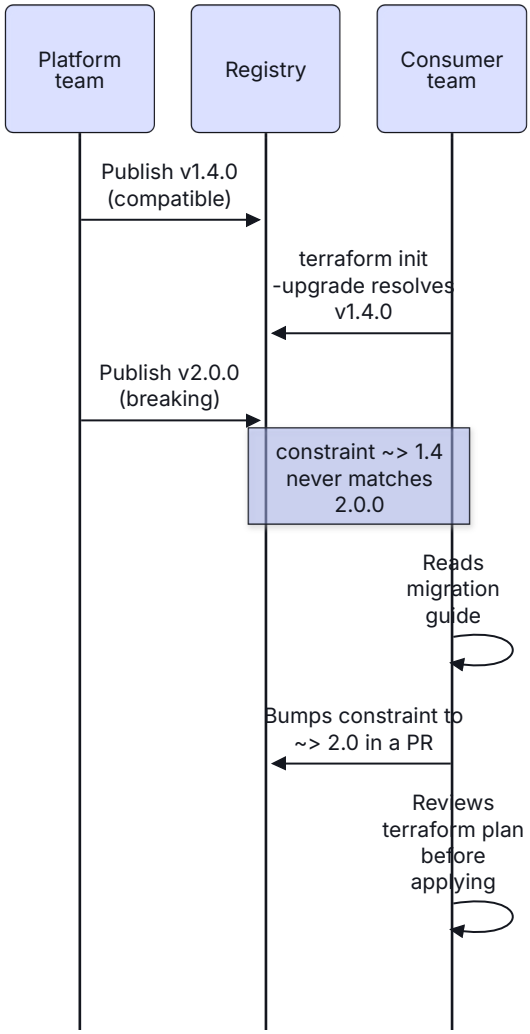


Figure 18.6-2

5 How it works

Reusable: the interface is the design

A good template exposes **few inputs with safe defaults** and hides the rest. A rule of thumb: inputs describe *what the team needs* (`size = "small"`), not *how it is built* (`instance_class = "db.t4g.medium"`). Teams that need more can escape with an explicit override variable, but the default path must stay short.

Template type	Interface	Versioned by	Typical tests
Terraform/OpenTofu module	`variables.tf` / `outputs.tf`	Git tag, registry `version`	`validate`, `tflint`, `terraform test`, Terratest
Crossplane Composition	XRD schema (the claim spec)	Configuration package tag	`crossplane render`, `crossplane beta validate`, e2e on kind
Helm chart	`values.yaml` + `values.schema.json`	`Chart.yaml` version	`helm lint`, `helm template`, `chart-testing`
Backstage/cookiecutter skeleton	template parameters form	Git ref of the template repo	scaffold into a temp dir and build it in CI

Versioning rules that keep consumers safe

- **Semantic versioning** (see Semantic Versioning §3.10): patch = bug fix, minor = new optional input or output, major = anything that can change or replace an existing resource (renamed input, changed default that alters a resource, removed output).
- **Consumers pin.** For Terraform modules: `version = "~> 1.3"` from a registry or `?ref=v1.3.2` on a Git source. Never `ref=main`.
- **Providers pin too.** The module declares `required_providers` with a lower bound; the root module owns the lock file.
- **Deprecate before removing.** Keep the old variable for one minor release with a `validation` or documentation warning.
- **Changelog and migration notes** are part of the release, because the consumer reads them before bumping.

Testing layers (cheapest first)

1. **Static:** `terraform fmt -check`, `terraform validate`, `tflint`, `helm lint`.
2. **Unit (plan-only):** `terraform test` with `command = plan` and assertions on outputs and computed values; the same file can use `mock_provider` blocks (Terraform 1.7+) so no cloud credentials are needed.
3. **Policy on the plan:** render an example plan to JSON and run Conftest against it (see Policy as Code §18.7).

4. **Integration:** apply the example into a throwaway account or kind cluster with Terratest or a CI job, assert, destroy.
5. **Upgrade test:** apply the *previous* release, then plan the new one -- a diff showing "forces replacement" on a stateful resource is a major-version change.

6 Step-by-step guide

Prerequisites: Terraform 1.6 or newer (or OpenTofu 1.6+) installed. No cloud account is needed: the sample module uses only built-in language features, so everything runs locally and costs nothing.

1. **Create the module skeleton.** This module is the "baseline" every `shop-api` resource calls: it computes a standard name and the mandatory tags.

```bash
mkdir -p ~/platform/tf-service-baseline/tests && cd ~/platform/tf-service-baseline
```

2. **Define the inputs with validation.**

```hcl
# variables.tf
variable "service" {
  type        = string
  description = "Service name, lowercase letters, digits and dashes."
  validation {
    condition     = can(regex("^[a-z][a-z0-9-]{2,30}$", var.service))
    error_message = "service must match ^[a-z][a-z0-9-]{2,30}$."
  }
}

variable "env" {
  type        = string
  description = "Deployment environment."
  validation {
    condition     = contains(["dev", "staging", "prod"], var.env)
    error_message = "env must be one of dev, staging, prod."
  }
}

variable "owner" {
  type        = string
  description = "Owning team (used for cost allocation)."
}

variable "extra_tags" {
  type    = map(string)
  default = {}
```

```
}
```

3. **Implement and expose outputs.**

```hcl
# main.tf
terraform {
  required_version = "≥ 1.6"
}

locals {
  name = "${var.service}-${var.env}"
  tags = merge(var.extra_tags, {
    service     = var.service
    env         = var.env
    owner       = var.owner
    cost-center = var.owner
    managed-by  = "terraform"
  })
}
```

```hcl
# outputs.tf
output "name" {
  value = local.name
}

output "tags" {
  value = local.tags
}
```

4. **Write the template tests.** Note `expect_failures` -- a template must also be tested for what it *rejects*.

```hcl
# tests/baseline.tftest.hcl
variables {
  service = "shop-api"
  env     = "dev"
  owner   = "team-shop"
}

run "builds_name_and_tags" {
  command = plan

  assert {
    condition     = output.name == "shop-api-dev"
    error_message = "name should be <service>-<env>"
```

```
  }

  assert {
    condition     = output.tags["managed-by"] == "terraform" && output.tags["owner"]
  == "team-shop"
    error_message = "standard tags missing"
  }
}

run "rejects_bad_env" {
  command = plan

  variables {
    env = "qa"
  }

  expect_failures = [var.env]
}
```

5. **Run the static and unit layers.**

```bash
terraform init -input=false
terraform fmt -check -recursive && echo "fmt ok"
terraform validate
terraform test
```

Expected (similar to):

```
Success! The configuration is valid.
tests/baseline.tftest.hcl... in progress
  run "builds_name_and_tags"... pass
  run "rejects_bad_env"... pass
tests/baseline.tftest.hcl... tearing down
tests/baseline.tftest.hcl... pass

Success! 2 passed, 0 failed.
```

6. **Release it with a version.** Put the module in its own Git repository and tag it.

```bash
git init && git add . && git commit -m "feat: service baseline module"
git tag v1.0.0
```

A consumer then references it by tag, never by branch:

```hcl
module "baseline" {
  source = "git::https://github.com/example-org/tf-service-baseline.git?ref=v1.0.0"
  service = "shop-api"
  env     = "prod"
  owner   = "team-shop"
}
```

(The URL is a placeholder -- for a local trial use `source = "../tf-service-baseline"`.)

7. **Wire it into CI** so every pull request runs the same checks.

```yaml
# .github/workflows/module-ci.yml
name: module-ci
on:
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform fmt -check -recursive
      - run: terraform init -input=false
      - run: terraform validate
      - run: terraform test
```

7 Real-world scenario

(Illustrative, not a documented incident.) A platform team ships a `postgres` module used by 14 teams, all referencing `ref=main`. Someone "tidies" the module by renaming the `aws_db_instance` resource from `this` to `primary` without a `moved` block. The next day three teams run their routine pipelines: the plan shows **destroy and create** of the production database. Two teams notice in review; the third has auto-apply on and loses a staging database.

The investigation shows no one could see the blast radius, because every consumer tracked `main` and the module had no test that planned an upgrade from the previous release. The fix is structural: the module is released with tags, the rename ships as `v2.0.0` with a `moved { from = aws_db_instance.this to = aws_db_instance.primary }` block and a migration note, consumers pin `~> 1.x` until they choose to move, and CI adds an upgrade test that plans the new code against state created by the last release. **Lesson:** a template is an API; breaking changes need versions, and "what does this do to existing state" is the most important test.

8 Common mistakes and best practices

Don't

- Consume modules from a moving branch (`ref=main`, no `version` constraint).
- Expose every underlying resource argument as an input -- you have just rebuilt the provider with extra steps.
- Hide security-relevant settings behind defaults that teams can silently turn off without review.
- Release a rename or default change as a patch or minor version.
- Skip negative tests: a module that accepts `env = "qa"` is silently broken.

Do

- Keep the interface small, outcome-oriented and validated; document it in a `README` generated from the code.
- Use semantic versioning, release notes and `moved` blocks; deprecate before removal.
- Test in layers: static, plan-level unit tests, policy on the plan, then occasional real applies.
- Provide an `examples/` directory that CI plans on every PR -- it doubles as documentation.
- Bake mandatory tags and encryption into the template so policy checks pass by default.

9 Tasks

- [ ] **T1 — Build and test the baseline module** *(≈30 min, beginner)*
  - **Goal:** follow the guide end to end locally.
  - **Steps:**
    1. Create the files from section 6 steps 1-4.
    2. Run `terraform init`, `terraform validate` and `terraform test`.
  - **Done when:** `terraform test` prints `Success! 2 passed, 0 failed.`
- [ ] **T2 — Consume it from a root module** *(≈30 min, beginner)*
  - **Goal:** see the template from the consumer side.
  - **Steps:**
    1. Create a sibling directory `consumer/` with a `main.tf` that calls the module via `source = "../tf-service-baseline"`.
    2. Add an `output "tags"` that forwards `module.baseline.tags`.
    3. Run `terraform init && terraform apply -auto-approve` (no resources, so it is safe) and `terraform output tags`.

- **Done when:** the output lists `managed-by = "terraform"` and your team's `owner`.

- [ ] **T3 — Add a feature as a minor version** *(≈30 min, intermediate)*
  - **Goal:** extend the interface without breaking consumers.
  - **Steps:**
    1. Add an optional `data_classification` variable (default `"internal"`, validated against a list) and include it in `tags`.
    2. Add a test run asserting the default and one asserting an invalid value fails.
    3. Tag `v1.1.0`.
  - **Done when:** `terraform test` passes, the consumer from T2 works unchanged, and `git tag` shows `v1.0.0` and `v1.1.0`.

- [ ] **T4 — Break it on purpose, then catch it** *(≈40 min, intermediate)*
  - **Goal:** prove the tests detect a regression.
  - **Steps:**
    1. Change the name format to `${var.env}-${var.service}`.
    2. Run `terraform test` and read the failure.
    3. Decide: is this a major version? Revert, or bump to `v2.0.0` with an updated test and a note in a `CHANGELOG.md`.
  - **Done when:** you can show a red test run and a green one, and justify the version number.

- [ ] **T5 — Run the template tests in CI** *(≈40 min, intermediate)*
  - **Goal:** make the checks mandatory for every change.
  - **Steps:**
    1. Push the module to a GitHub repository and add the workflow from section 6 step 7.
    2. Open a pull request that breaks a test and confirm the check fails.
    3. Enable branch protection requiring the `module-ci` check (see Protected Branches §3.8).
  - **Done when:** the failing PR cannot be merged and the fixed PR can.

- [ ] **T6 — Prove it: version policy for a template** *(≈45 min, advanced)*
  - **Goal:** be able to defend how a platform releases templates.
  - **Steps:**
    1. Without notes, write a one-page policy: which changes are patch, minor, major; how consumers pin; how deprecations work.

2. Give one Terraform example, one Crossplane example (XRD field removed) and one Helm example for each class.

- **Done when:** a teammate can classify five proposed changes using only your page.

10 Self-check questions

1. Why is consuming a module from `main` dangerous even if the module repository has code review?
2. What belongs in a template's input interface and what should be hidden?
3. Give two changes that must be a major version of a Terraform module.
4. What does `expect_failures` test, and why does a template need it?
5. Why is a plan-level test cheaper and often almost as valuable as an apply-level test?

ANSWERS

1. Review checks the diff, not every consumer's state. A merge changes what all consumers plan the next time they run, with no opt-in.
2. Inputs: outcomes the team cares about (size, environment, owner). Hidden: implementation details such as resource names, internal subnets, and security-sensitive defaults.
3. Renaming or removing an input or output; a change that forces replacement of an existing resource (renaming a resource without `moved`, changing an immutable argument default).
4. That invalid inputs are rejected by `validation` blocks. Without it a template can silently accept bad input.
5. It needs no credentials or cloud resources, runs in seconds, and still checks computed values, validations and policy results.

11 Further reading

- Terraform documentation: "Modules", "Module Testing" (`terraform test`) and the Terraform Registry publishing guide.
- OpenTofu documentation (compatible module and test syntax).
- Crossplane documentation: "Compositions" and "Configuration packages".
- Semantic Versioning specification (semver.org).
- *Terraform: Up & Running* by Yevgeniy Brikman -- chapters on reusable modules and testing.

18.7 Policy as Code

Topic 7 of 10 | ~5 hours | Difficulty: Intermediate

1 What it is

Policy as code means writing your organisation's rules ("every Deployment has an owner label", "no public S3 buckets", "images come only from our registry") as version-controlled, testable programs that machines enforce, instead of as a wiki page that humans are supposed to remember. A policy engine takes **input** (a Kubernetes manifest, a Terraform plan, a Dockerfile, an API request) plus a set of **rules**, and returns **allow/deny with a reason**.

Two engines dominate Kubernetes-centred platforms. **Open Policy Agent (OPA)** is a general-purpose engine whose rules are written in a declarative language called **Rego**; it also powers **Conftest** (a CLI that tests config files in CI) and **Gatekeeper** (a Kubernetes admission controller). **Kyverno** is Kubernetes-native: policies are ordinary YAML resources (`ClusterPolicy`), so teams that already know Kubernetes do not need a new language, and it can also *mutate* and *generate* resources, not just validate them.

2 Why it matters in DevOps

- **Guardrails instead of gates.** A platform that offers self-service (Self-service Infrastructure §18.3) must also make unsafe choices impossible, otherwise it needs a human approval for every request.
- **Shift left.** A rule that fails in a pull request costs a minute; the same rule failing at admission costs a failed deploy; discovered in an audit it costs a finding. Enforce in CI *and* in the cluster.
- **Auditable compliance.** "Show me that all production workloads have resource limits" becomes a query, not a spreadsheet.
- **Consistency across tools.** The same rule ("must have `owner` and `cost-center` tags") can protect Terraform plans, Kubernetes manifests and cloud accounts.
- **Risk:** policies are code that can block every deploy. Untested policies, or policies switched to enforce without an audit phase, cause platform-wide outages.

See also: Admission Controllers and Webhooks §16.6, Pod Security §16.3 and DevSecOps §0.12.

3 Real-world system example

OPA is a graduated project of the Cloud Native Computing Foundation (CNCF). Its Kubernetes integration, **Gatekeeper**, runs as a validating admission webhook and expresses policies as `ConstraintTemplate` resources containing Rego plus `Constraint` resources that instantiate them. **Conftest** applies the same Rego to Kubernetes YAML, Terraform plan JSON, Dockerfiles and other structured files in CI. **Kyverno** is also a CNCF project and is widely used for Kubernetes-only policy; its policies are YAML and it ships a CLI that evaluates policies offline in CI.

Kubernetes itself has a built-in, in-process option: **ValidatingAdmissionPolicy**, which uses the Common Expression Language (CEL) and is generally available in current Kubernetes releases (since v1.30). Many platforms use a built-in policy type for the simplest rules and an external engine for richer ones.

4 Diagram

Where a policy runs on the way from a developer's laptop to a running pod:

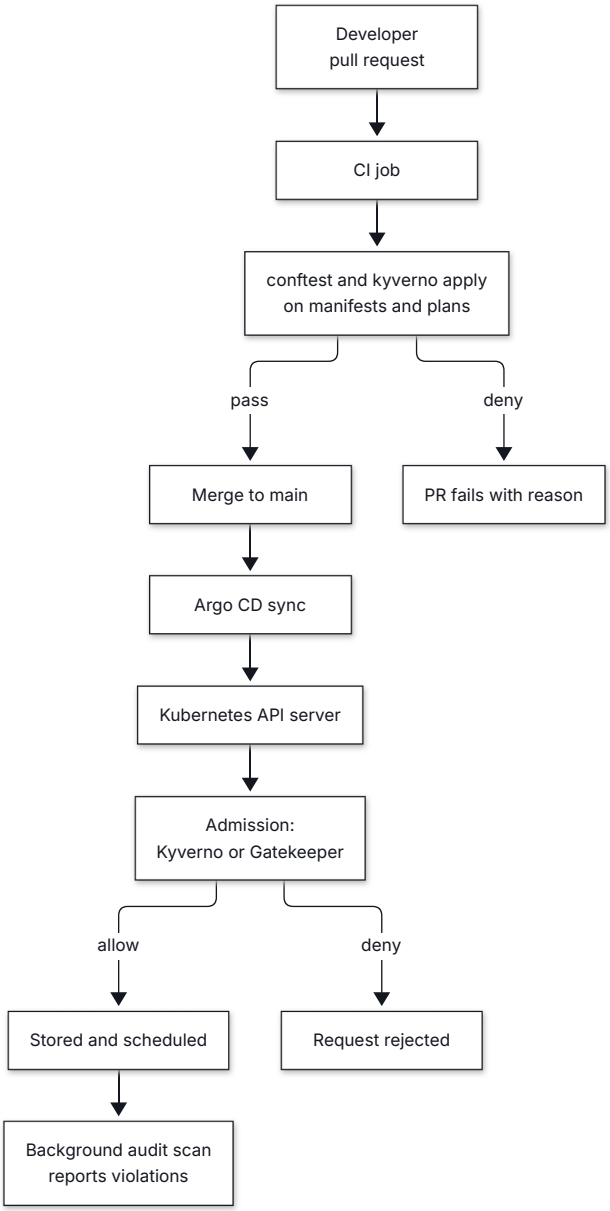


Figure 18.7-1

How a validating admission request is evaluated:

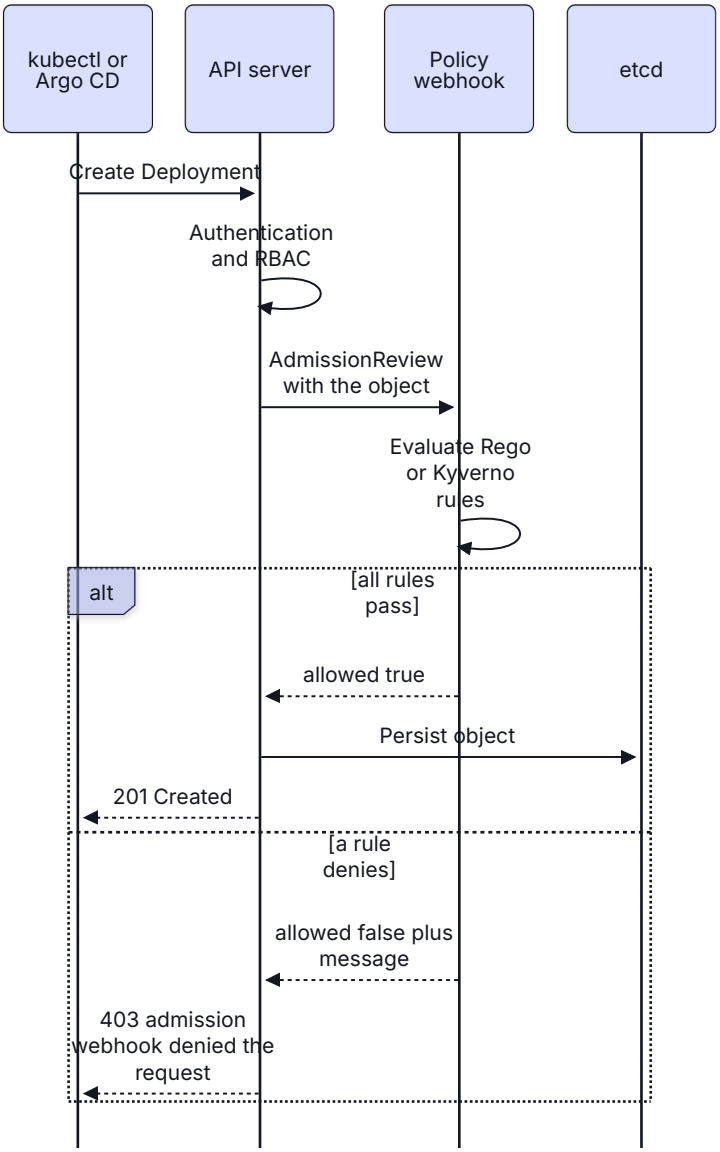


Figure 18.7-2

5 How it works

Rego in five minutes

Rego rules are **queries over data**. A rule like `deny contains msg if { ... }` adds `msg` to a set when *every* expression in the body is true. If any expression is undefined or false, the rule contributes nothing. That makes policies read as "this is bad when ...".

Concept	Meaning	Example
`input`	the document being checked	`input.kind == "Deployment"`
`data`	static data and other rules	`data.main.deny`
`some x in coll`	iterate	`some c in input.spec.template.spec.containers`
`not expr`	true if `expr` is undefined/false	`not c.resources.limits.memory`
`deny` / `warn` / `violation`	Conftest conventions: `deny` fails, `warn` warns	`deny contains msg if {...}`
`package main`	Conftest runs the `main` package by default; `-n` selects another	`conftest test -n terraform`

Rego policies are unit-tested with `_test.rego` files (`test_` rules) and `opa test` / `conftest verify` .

OPA/Gatekeeper versus Kyverno

	OPA / Gatekeeper / Conftest	Kyverno
Language	Rego (general purpose)	YAML with patterns, JMESPath and CEL
Scope	Anything JSON-shaped: K8s, Terraform, Dockerfiles, APIs	Kubernetes resources (plus image verification and CLI checks)
Mutate / generate	Mutation via Gatekeeper assign; no generate	Mutate and generate are first-class
Learning curve	Steeper, very expressive	Gentle for Kubernetes users
One policy for CI and cluster	Yes (same Rego)	Yes (same ClusterPolicy via the CLI)
Reporting	Gatekeeper audit results on constraints	`PolicyReport` / `ClusterPolicyReport` resources

Neither is "better": choose Kyverno when the scope is only Kubernetes and the team prefers YAML; choose OPA when the same engine must also guard Terraform plans, CI configs and service authorisation.

Rollout modes

Almost every engine supports an **audit/warn** mode and an **enforce/deny** mode. The safe order is: write the policy and unit tests, run it in audit mode and read the violation report, fix or exempt existing workloads, then switch to enforce. Always support **exceptions** (an annotated namespace or an explicit `PolicyException`) so one legitimate workload does not force you to disable a whole policy.

6 Step-by-step guide

Prerequisites: Linux or macOS shell. Install the OPA and Conftest binaries from their GitHub release pages (or run Conftest via Docker: `docker run --rm -v "$PWD":/project openpolicyagent/conftest test ...`). For the Kyverno part install the `kyverno` CLI from its release page. Nothing here needs a cluster or cloud account; all checks run offline on files. The outputs below were produced with OPA 1.0.0, Conftest 0.56.0 and Kyverno CLI 1.13.4; yours will be similar.

1. **Create a project and two test manifests.**

```bash
mkdir -p ~/policy-lab/{policy,manifests,tf} && cd ~/policy-lab
cat > manifests/deploy-bad.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
spec:
  replicas: 1
  selector:
    matchLabels: {app: shop-api}
  template:
    metadata:
      labels: {app: shop-api}
    spec:
      containers:
        - name: api
          image: shop-api:latest
EOF
cat > manifests/deploy-good.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
```

```
  labels: {owner: team-shop}
spec:
  replicas: 3
  selector:
    matchLabels: {app: shop-api}
  template:
    metadata:
      labels: {app: shop-api}
    spec:
      containers:
        - name: api
          image: ghcr.io/example/shop-api:1.4.2
          resources:
            limits: {memory: 256Mi}
EOF
```

2. **Write the Rego policy.** `import rego.v1` makes the file valid on both older and current OPA releases.

```rego
# policy/k8s.rego
package main

import rego.v1

is_deployment if input.kind == "Deployment"

containers contains c if {
  is_deployment
  some c in input.spec.template.spec.containers
}

deny contains msg if {
  some c in containers
  endswith(c.image, ":latest")
  msg := sprintf("container %q uses the mutable tag :latest", [c.name])
}

deny contains msg if {
  some c in containers
  not contains(c.image, ":")
  msg := sprintf("container %q has no image tag (defaults to :latest)", [c.name])
}

deny contains msg if {
  some c in containers
  not c.resources.limits.memory
  msg := sprintf("container %q must set resources.limits.memory", [c.name])
}

deny contains msg if {
```

```
  is_deployment
  not input.metadata.labels.owner
  msg := sprintf("Deployment %q must have an 'owner' label", [input.metadata.name])
}

warn contains msg if {
  is_deployment
  input.spec.replicas == 1
  msg := sprintf("Deployment %q runs a single replica", [input.metadata.name])
}
```

3. **Run Conftest against both files.**

```bash
conftest test manifests/deploy-bad.yaml; echo "exit=$?"
conftest test manifests/deploy-good.yaml; echo "exit=$?"
```

Expected for the bad file (exit code 1) and the good file (exit code 0):

```
WARN - manifests/deploy-bad.yaml - main - Deployment "shop-api" runs a single replica
FAIL - manifests/deploy-bad.yaml - main - Deployment "shop-api" must have an 'owner'
  label
FAIL - manifests/deploy-bad.yaml - main - container "api" must set
  resources.limits.memory
FAIL - manifests/deploy-bad.yaml - main - container "api" uses the mutable tag :latest

5 tests, 1 passed, 1 warning, 3 failures, 0 exceptions
...
5 tests, 5 passed, 0 warnings, 0 failures, 0 exceptions
```

The non-zero exit code is what makes a CI job fail.

4. **Unit-test the policy itself.** Policies are code; they need tests.

```rego
# policy/k8s_test.rego
package main

import rego.v1

good := {
  "kind": "Deployment",
  "metadata": {"name": "shop-api", "labels": {"owner": "team-shop"}},
  "spec": {"replicas": 3, "template": {"spec": {"containers": [{
          "name": "api",
          "image": "ghcr.io/example/shop-api:1.4.2",
          "resources": {"limits": {"memory": "256Mi"}},
  }]}}},
}
```

```
test_good_deployment_passes if {
  count(deny) == 0 with input as good
}

test_latest_tag_denied if {
  bad := json.patch(good, [{"op": "replace", "path":
  "/spec/template/spec/containers/0/image", "value": "shop-api:latest"}])
  count(deny) == 1 with input as bad
}

test_missing_owner_denied if {
  bad := json.patch(good, [{"op": "remove", "path": "/metadata/labels/owner"}])
  deny["Deployment \"shop-api\" must have an 'owner' label"] with input as bad
}
```

BASH

```
conftest verify          # or: opa test policy -v
```

Expect `3 tests, 3 passed`.

5. **Apply the same idea to a Terraform plan.** Conftest reads JSON, so policy can guard infrastructure before it exists. Save as `policy/tf.rego`:

REGO

```
package terraform

import rego.v1

required_tags := {"owner", "cost-center"}

changed contains rc if {
  some rc in input.resource_changes
  some action in rc.change.actions
  action in {"create", "update"}
}

deny contains msg if {
  some rc in changed
  rc.type == "aws_s3_bucket_public_access_block"
  rc.change.after.block_public_acls == false
  msg := sprintf("%s: block_public_acls must be true", [rc.address])
}

deny contains msg if {
  some rc in changed
  rc.type == "aws_instance"
  missing := required_tags - object.keys(object.get(rc.change.after, "tags", {}))
  count(missing) > 0
  msg := sprintf("%s: missing required tags %v", [rc.address, sort(missing)])
```

```
}
```

A real plan comes from `terraform plan -out=tfplan && terraform show -json tfplan > plan.json`. For a quick local test, a hand-written fragment is enough:

```bash
cat > tf/plan.json <<'EOF'
{"resource_changes": [
  {"address": "aws_instance.api", "type": "aws_instance",
   "change": {"actions": ["create"], "after": {"instance_type": "t3.micro", "tags":
  {"owner": "team-shop"}}}},
  {"address": "aws_s3_bucket_public_access_block.assets", "type":
  "aws_s3_bucket_public_access_block",
   "change": {"actions": ["create"], "after": {"block_public_acls": false}}}
]}
EOF
conftest test tf/plan.json -n terraform
```

```
FAIL - tf/plan.json - terraform - aws_instance.api: missing required tags ["cost-
  center"]
FAIL - tf/plan.json - terraform - aws_s3_bucket_public_access_block.assets:
  block_public_acls must be true

2 tests, 0 passed, 0 warnings, 2 failures, 0 exceptions
```

6. **Write the same rules in Kyverno.** Policies are plain Kubernetes YAML.

```yaml
# kyverno/require-limits.yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-memory-limits-and-owner
spec:
  validationFailureAction: Enforce   # use Audit while rolling out
  background: true
  rules:
    - name: memory-limit-required
      match:
        any:
          - resources:
              kinds: [Deployment]
      validate:
        message: "Every container must set resources.limits.memory."
        pattern:
          spec:
            template:
              spec:
```

```
              containers:
                - resources:
                    limits:
                      memory: "?*"
  - name: owner-label-required
    match:
      any:
        - resources:
            kinds: [Deployment]
    validate:
      message: "Deployments must have an 'owner' label."
      pattern:
        metadata:
          labels:
            owner: "?*"
  - name: no-latest-tag
    match:
      any:
        - resources:
            kinds: [Deployment]
    validate:
      message: "Images must not use the :latest tag."
      pattern:
        spec:
          template:
            spec:
              containers:
                - image: "!*:latest"
```

Evaluate it offline with the CLI, exactly as a CI job would:

BASH

```
kyverno apply kyverno/require-limits.yaml --resource manifests/deploy-bad.yaml
```

```
Applying 3 policy rule(s) to 1 resource(s)...
policy require-memory-limits-and-owner → resource default/Deployment/shop-api failed:
1 - memory-limit-required validation error: Every container must set
  resources.limits.memory. ...
2 - owner-label-required validation error: Deployments must have an 'owner' label. ...
3 - no-latest-tag validation error: Images must not use the :latest tag. ...

pass: 0, fail: 3, warn: 0, error: 0, skip: 0
```

The same command with `deploy-good.yaml` reports `pass: 3, fail: 0`.

7. **Enforce in CI.** One job runs both engines on every pull request.

YAML

```
# .github/workflows/policy.yml
```

```
name: policy
on:
  pull_request:
jobs:
  policy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install conftest
        run: |
          curl -sSL -o conftest.tgz https://github.com/open-policy-
  agent/conftest/releases/download/v0.56.0/conftest_0.56.0_Linux_x86_64.tar.gz
          tar xzf conftest.tgz conftest && sudo mv conftest /usr/local/bin/
      - run: conftest verify
      - run: conftest test manifests/ --all-namespaces
```

8. **Enforce in the cluster (optional, needs a kind cluster).** Install Kyverno from its Helm chart, apply the policy in `Audit` first, and read the report:

```bash
helm repo add kyverno https://kyverno.github.io/kyverno/ && helm repo update
helm install kyverno kyverno/kyverno -n kyverno --create-namespace
kubectl apply -f kyverno/require-limits.yaml
kubectl get clusterpolicyreport
```

   Switch `validationFailureAction` to `Enforce` only when the report is clean. Then `kubectl apply -f manifests/deploy-bad.yaml` is rejected with the policy message.

7 Real-world scenario

(Illustrative.) A platform team wants every workload to have memory limits. They write the policy and switch the cluster admission rule straight to enforce on a Friday. On Monday the cluster's own add-ons cannot upgrade: several system DaemonSets in `kube-system` have no limits, and the admission webhook rejects their rollout. Worse, the webhook itself has `failurePolicy: Fail`, and when its pods are evicted during a node drain nothing can be created.

The recovery is to exclude `kube-system` and the policy engine's own namespace, then re-run in audit mode. The durable fix: policies are unit-tested, new rules always ship in audit first, the report is reviewed for a week, and the webhook runs multiple replicas with a PodDisruptionBudget. CI also runs the same rules, so developers see failures in the pull request rather than as an admission error in Argo CD. **Lesson:** a policy engine is on the critical path of every deploy; roll out like any production change.

8 Common mistakes and best practices

Don't

- Turn on enforcement before reading an audit report of existing workloads.
- Enforce only at admission: developers then discover problems after merge, in the deploy pipeline.
- Write policies without unit tests, or with messages like "denied" that do not say how to fix it.
- Make the admission webhook a single replica with `failurePolicy: Fail` and no exclusions for system namespaces.
- Maintain two divergent copies of "the same rule" for CI and for the cluster.

Do

- Keep policies in Git, reviewed and released like application code (see Infrastructure Templates §18.6).
- Write messages that name the offender and the fix.
- Layer: IDE or pre-commit, CI (Conftest / Kyverno CLI), admission (Kyverno / Gatekeeper / ValidatingAdmissionPolicy), then background audit.
- Provide a documented exception process with an expiry date.
- Start with a short list of high-value rules: owner label, resource limits, trusted registries, no privileged containers, required cost tags.

9 Tasks

- [ ] **T1 — Run the Conftest lab** *(≈30 min, beginner)*
  - **Goal:** see a policy fail and pass on real YAML.
  - **Steps:**
    1. Create the files from section 6 steps 1-3.
    2. Run `conftest test` on the bad and the good manifest and check the exit codes with `echo $?`.
  - **Done when:** the bad manifest exits 1 with three `FAIL` lines and the good one exits 0.
- [ ] **T2 — Add a rule and its test** *(≈40 min, intermediate)*
  - **Goal:** extend a policy test-first.
  - **Steps:**
    1. Add a `deny` rule rejecting images that do not start with `ghcr.io/example/`.

2. Write `test_foreign_registry_denied` first and watch it fail with `conftest verify`.
3. Implement the rule until the tests pass, then run it against both manifests.

- **Done when:** `conftest verify` shows all tests passing and `deploy-bad.yaml` has an additional failure line.

- [ ] **T3 — Guard a Terraform plan** *(≈45 min, intermediate)*
  - **Goal:** block bad infrastructure before apply.
  - **Steps:**
    1. Use the policy in step 5. Generate a real plan JSON from any small Terraform configuration with `terraform show -json tfplan > plan.json` (a `terraform_data` or `local_file` resource is enough, adapt the `type` in the rule).
    2. Make the rule fail, fix the configuration, and re-run.
  - **Done when:** the pipeline step goes from exit 1 to exit 0 after you fix the configuration, not the policy.

- [ ] **T4 — Port a rule to Kyverno and test it** *(≈45 min, intermediate)*
  - **Goal:** compare the two engines on one rule.
  - **Steps:**
    1. Use the Kyverno policy from step 6 and run `kyverno apply` on both manifests.
    2. Add a `kyverno-test.yaml` (`apiVersion: cli.kyverno.io/v1alpha1`, `kind: Test`) with one expected `fail` and one `pass`, and run `kyverno test .`.
  - **Done when:** `kyverno test .` reports all tests passed.

- [ ] **T5 — Break it and enforce in a cluster** *(≈60 min, advanced)*
  - **Goal:** experience audit-then-enforce on a live API server.
  - **Steps:**
    1. Create a kind cluster, install Kyverno (step 8) and apply the policy with `validationFailureAction: Audit`.
    2. Deploy the bad manifest, read `kubectl get policyreport -A` and see the violations.
    3. Change to `Enforce` and retry the bad manifest. Then exclude one namespace using `exclude` in the rule.
  - **Done when:** the bad manifest is accepted in Audit, rejected in Enforce, and accepted again in the excluded namespace.

- [ ] **T6 — Prove it: policy rollout plan** *(≈45 min, advanced)*
  - **Goal:** be able to introduce a new mandatory policy safely.

- **Steps:**
  1. Without notes, write a rollout plan for "all production Deployments must have resource limits" covering CI, audit, exceptions and enforce.
  2. State which failures cause a platform outage and how your plan avoids them.
- **Done when:** a teammate can follow your plan and name the rollback action if enforcement breaks deploys.

10 Self-check questions

1. What does it mean that a Rego `deny` rule "contributes nothing" when an expression is undefined?
2. Why enforce policy in CI as well as in the cluster?
3. When would you choose Kyverno over OPA/Gatekeeper, and when the reverse?
4. Why should a new policy run in audit mode first?
5. What risks does a validating admission webhook introduce for cluster availability?

ANSWERS

1. The rule body is a conjunction. If any expression is undefined or false the whole body fails, so nothing is added to the `deny` set and the object is allowed by that rule.
2. CI gives fast feedback in the pull request and covers non-cluster inputs such as Terraform plans; the cluster is the last line of defence against anything that bypasses the pipeline (kubectl, other tools).
3. Kyverno for Kubernetes-only scope, YAML familiarity, and mutate/generate needs. OPA when the same engine and language must cover Terraform, CI configs, Dockerfiles or application authorisation.
4. To find existing violations and legitimate exceptions without blocking deploys; enforcement on day one can stall the whole delivery path.
5. If the webhook is down or slow and `failurePolicy` is `Fail`, resource creation can be blocked cluster-wide; mitigate with replicas, a PodDisruptionBudget, short timeouts and exclusions for system namespaces.

11 Further reading

- Open Policy Agent documentation: "Policy Language" (Rego), "Policy Testing" and the Conftest documentation.
- OPA Gatekeeper documentation (ConstraintTemplates and Constraints).
- Kyverno documentation: "Writing Policies", "Kyverno CLI" and policy reports.

- Kubernetes documentation: "Validating Admission Policy".
- CNCF Cloud Native Security Whitepaper (policy and admission sections).

18.8 Backstage

Topic 8 of 10 | ~6 hours | Difficulty: Intermediate

1 What it is

Backstage is an open-source framework for building a **developer portal**: one web application where engineers find every service, API, library and piece of infrastructure the company owns, see who owns it, read its docs, check its CI/CD and Kubernetes status, and create new things from approved templates. It was created at Spotify and is now a CNCF project. Backstage is not a finished product you install and forget -- it is a TypeScript/React codebase you configure, extend with plugins and deploy yourself (or buy as a hosted/commercial distribution).

It has four core pieces that map to this topic's checklist: the **Software Catalog** (a graph of entities described by `catalog-info.yaml` files kept next to the code), **Software Templates** (the "Scaffolder": forms that run steps such as "create a repository" and "register it"), **TechDocs** (docs-as-code: Markdown in the repo, rendered in the portal), and a **plugin system** for everything else (Kubernetes, Argo CD, PagerDuty, cost dashboards, and so on).

2 Why it matters in DevOps

- **Discoverability.** "Who owns the payments queue?" stops being a Slack archaeology exercise.
- **Self-service with guardrails.** A template turns "open a ticket, wait a week" into a form that creates a repo, a pipeline and an Argo CD application in minutes, using the golden path (see Golden Paths §18.4).
- **One front door** for the platform's capabilities, instead of ten bookmarks (see Developer Portals §18.2).
- **Ownership as data.** The catalog gives cost reports (FinOps §20.1), incident routing and policy checks a trustworthy `owner` field.
- **Cost of running it.** Backstage needs a team: upgrades, plugin maintenance, catalog data quality. A portal full of stale entities is worse than none, so ownership of the portal itself must be explicit.

3 Real-world system example

Spotify built Backstage internally to tame the sprawl of services and tooling across hundreds of teams, then open-sourced it in 2020; it entered the CNCF as a project afterwards. Many other companies publicly describe running it as the front end of their internal platform. The pattern those adopters describe is consistent: the catalog is the source of truth for ownership, a handful of software templates encode the golden paths for new services, and TechDocs keeps documentation in the same repo as the code so it gets reviewed with the code.

Backstage's own repository ships the reference app generator (`@backstage/create-app`), the plugin catalogue in the `backstage/backstage` monorepo, and community plugins (for example the Kubernetes plugin and an Argo CD plugin) that read live status from the systems of record rather than copying it.

4 Diagram

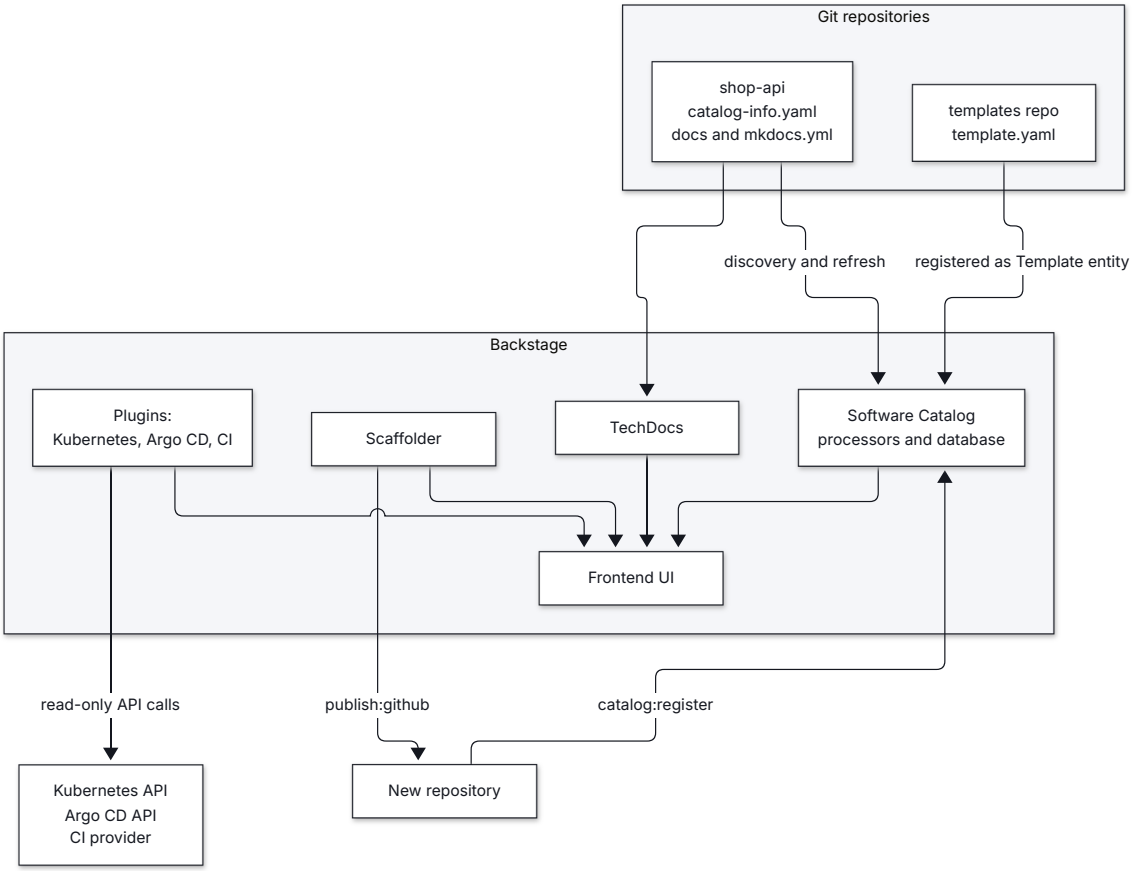


Figure 18.8-1

What happens when a developer uses a software template:

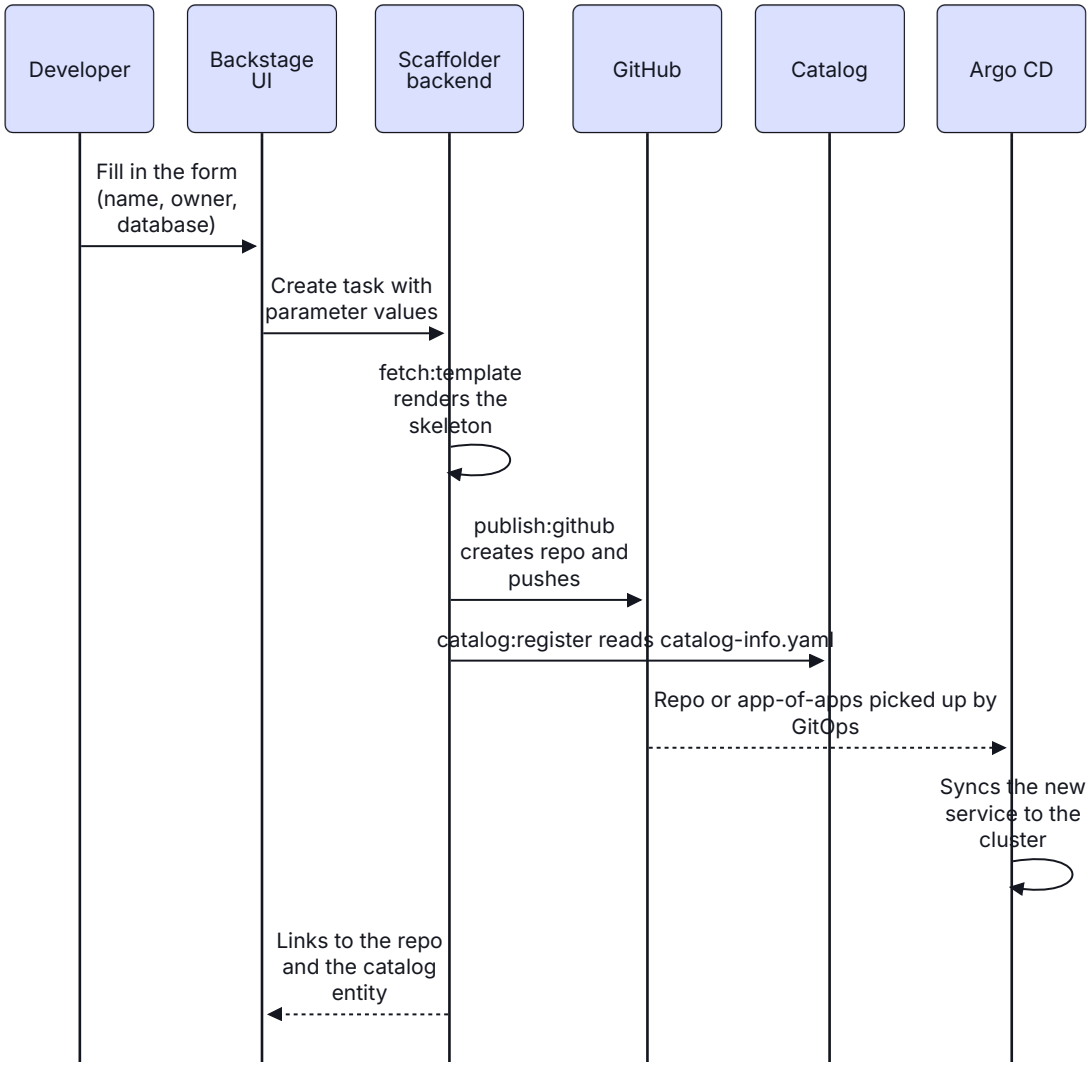


Figure 18.8-2

5 How it works

Catalog: entities and relations

Everything is an **entity** with `apiVersion`, `kind`, `metadata` and `spec`. Entities refer to each other with references like `group:default/team-shop`; Backstage builds a graph from those references.

Kind	Represents	Key fields
`Component`	a deployable thing: service, website, library	`spec.type`, `lifecycle`, `owner`, `system`, `dependsOn`
`API`	an interface a component provides or consumes	`spec.type` (openapi, grpc), `definition`
`Resource`	infrastructure a component needs (database, bucket)	`spec.type`, `owner`
`System` / `Domain`	grouping of components / of systems	`owner`, `domain`
`Group` / `User`	org structure, usually ingested from the IdP	`spec.members`, `parent`
`Template`	a scaffolder template	`spec.parameters`, `spec.steps`
`Location`	pointer to other catalog files	`spec.targets`

Catalog data enters through **entity providers/discovery** (scan a GitHub organisation for `catalog-info.yaml`), static `catalog.locations` in config, or manual registration in the UI. Annotations connect plugins to entities: `github.com/project-slug`, `backstage.io/techdocs-ref`, `backstage.io/kubernetes-id`, `argocd/app-name`.

Software Templates

A template has **parameters** (a JSON-Schema form), **steps** (actions such as `fetch:template`, `publish:github`, `catalog:register`) and **output** (links shown at the end). A skeleton directory contains files with `${{ values.name }}`-style placeholders. Platform teams can also write **custom actions** (a TypeScript backend module) to call internal APIs, such as "request a database from Crossplane" (see Crossplane §18.9).

Plugins and the app layout

A scaffolded app is a Yarn monorepo: `packages/app` is the React frontend, `packages/backend` the Node backend, and `plugins/` holds your own plugins. Backend plugins are added to the backend entry point (the "new backend system" registers them with `backend.add(import('@backstage/plugin-...'))`); frontend plugins are installed as packages

and mounted in the app. Because frontend plugin wiring has changed between Backstage releases, follow the documentation for the version `create-app` generated for you rather than copying old blog posts.

TechDocs

TechDocs is docs-as-code: a `docs/` folder and an `mkdocs.yml` (MkDocs) live in the component's repo and the `backstage.io/techdocs-ref: dir:.` annotation points at them. In the simplest "basic" setup Backstage builds the site on demand (with MkDocs installed locally or via Docker); in the recommended production setup CI builds with `techdocs-cli generate` and publishes to object storage (for example an S3 bucket), and Backstage only reads it.

6 Step-by-step guide

Prerequisites: a current Node.js LTS release (check the `engines` entry in the Backstage getting-started page for the supported versions), Yarn, Git, and about 4 GB of free disk. Docker is optional (used by TechDocs generation). The first install downloads many npm packages and takes several minutes. Everything runs on localhost; no cost. The UI addresses below are Backstage defaults; exact console output varies by version, so treat outputs as similar to what is shown.

1. **Create the app.**

   ```bash
   npx @backstage/create-app@latest
   # App name: shop-portal
   cd shop-portal
   ```

   The generator asks only for a name and creates `shop-portal/` with `app-config.yaml`, `packages/app`, `packages/backend` and an example catalog under `examples/`.

2. **Start it.**

   ```bash
   yarn start
   ```

   Wait for the frontend to compile. Open `http://localhost:3000`; the backend listens on `http://localhost:7007`. The default configuration uses an in-memory SQLite database and guest sign-in, which is fine for a lab and wrong for production (use PostgreSQL and a real identity provider).

3. **Describe `shop-api` in the catalog.** In a repo (or a local folder), create `catalog-info.yaml`:

   ```yaml
   ```

```yaml
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: shop-api
  description: Node.js API for the shop (PostgreSQL, Redis, Nginx in front)
  annotations:
    github.com/project-slug: example-org/shop-api
    backstage.io/techdocs-ref: dir:.
    argocd/app-name: shop-api-prod
  tags: [nodejs, api]
spec:
  type: service
  lifecycle: production
  owner: group:default/team-shop
  system: shop
  dependsOn:
    - resource:default/shop-db
  providesApis:
    - shop-api-http
---
apiVersion: backstage.io/v1alpha1
kind: Resource
metadata:
  name: shop-db
  description: PostgreSQL database for shop-api
spec:
  type: database
  owner: group:default/team-shop
  system: shop
---
apiVersion: backstage.io/v1alpha1
kind: API
metadata:
  name: shop-api-http
spec:
  type: openapi
  lifecycle: production
  owner: group:default/team-shop
  system: shop
  definition: |
    openapi: 3.0.3
    info: {title: shop-api, version: 1.0.0}
    paths: {}
```

4. **Register it through config.** Add a local file location to `app-config.yaml` (use the absolute path on your machine; file locations are read from the filesystem, so this is a lab-only

shortcut):

```yaml
catalog:
  rules:
    - allow: [Component, API, Resource, System, Domain, Group, User, Template,
  Location]
  locations:
    - type: file
      target: /home/you/lab/shop-api/catalog-info.yaml
```

Restart `yarn start`, open **Catalog**, and find `shop-api`. Its page shows the owner, the `shop-db` dependency and the API relation graph. In production you would instead use GitHub discovery so every repo with a `catalog-info.yaml` appears automatically.

5. **Add a software template.** Create `templates/nodejs-service/template.yaml`:

```yaml
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: nodejs-service
  title: Node.js service with PostgreSQL
  description: Scaffolds a repo with Dockerfile, CI, Helm values and an Argo CD
  Application
  tags: [nodejs, recommended]
spec:
  owner: group:default/platform-team
  type: service
  parameters:
    - title: Service details
      required: [name, owner]
      properties:
        name:
          title: Name
          type: string
          pattern: '^[a-z][a-z0-9-]{2,30}$'
          description: Lowercase letters, digits and dashes
        owner:
          title: Owner
          type: string
          ui:field: OwnerPicker
          ui:options:
            catalogFilter:
              kind: Group
    - title: Infrastructure
      properties:
        database:
          title: Needs a PostgreSQL database?
          type: boolean
```

```
        default: true
      size:
        title: Database size
        type: string
        enum: [small, medium]
        default: small
  steps:
    - id: fetch
      name: Fetch skeleton
      action: fetch:template
      input:
        url: ./skeleton
        values:
          name: ${{ parameters.name }}
          owner: ${{ parameters.owner }}
          database: ${{ parameters.database }}
          size: ${{ parameters.size }}
    - id: publish
      name: Create repository
      action: publish:github
      input:
        repoUrl: github.com?owner=example-org&repo=${{ parameters.name }}
        description: ${{ parameters.name }} service
        defaultBranch: main
    - id: register
      name: Register in catalog
      action: catalog:register
      input:
        repoContentsUrl: ${{ steps['publish'].output.repoContentsUrl }}
        catalogInfoPath: /catalog-info.yaml
  output:
    links:
      - title: Repository
        url: ${{ steps['publish'].output.remoteUrl }}
      - title: Open in catalog
        icon: catalog
        entityRef: ${{ steps['register'].output.entityRef }}
```

The skeleton directory (`templates/nodejs-service/skeleton/`) holds the files to generate, for example `catalog-info.yaml` with `name: ${{ values.name }}`, a `Dockerfile`, a CI workflow, and a `deploy/application.yaml` Argo CD `Application`. Register the template like any entity by adding `- type: file` / `target: .../template.yaml` under `catalog.locations`, then open **Create…** in the UI.

6. **Allow the `publish:github` step to work (optional).** It needs a GitHub integration with a

token or GitHub App:

```yaml
integrations:
  github:
    - host: github.com
      token: ${GITHUB_TOKEN}      # export GITHUB_TOKEN=<YOUR_TOKEN> before yarn start
```

Use a fine-grained token limited to a test organisation. Without it, the form renders but the publish step fails, which is still a useful experiment (T4).

7. **Turn on TechDocs.** In the `shop-api` repo add `mkdocs.yml` and `docs/index.md`:

```yaml
site_name: shop-api
nav:
  - Home: index.md
plugins:
  - techdocs-core
```

In `app-config.yaml` use the local builder for the lab:

```yaml
techdocs:
  builder: 'local'
  generator:
    runIn: 'docker'     # or 'local' if mkdocs and the techdocs-core plugin are
  installed
  publisher:
    type: 'local'
```

Open the component's **Docs** tab; the first load builds the site. For production, set `builder: 'external'`, build in CI with `npx @techdocs/cli generate` / `publish`, and use a cloud storage publisher.

8. **Add a plugin.** Follow the plugin's README for your Backstage version; the general shape is: install the frontend and backend packages with `yarn --cwd packages/app add ...` and `yarn --cwd packages/backend add ...`, register them in the app/backend, add configuration (for example the Kubernetes cluster list), and add the annotation (for example `backstage.io/kubernetes-id: shop-api`) to your `catalog-info.yaml`. Start with the Kubernetes plugin against a local kind cluster, which only needs read access.

7 Real-world scenario

(Illustrative.) A company launches its portal and a "new service" template. Adoption is slow: teams keep copying old repositories. Looking at the catalog, the platform team sees 40 percent of components have no owner and most `System` entities were typed by hand. A cost review cannot attribute a third of the bill to anyone.

The fix has three parts: groups and users are ingested from the identity provider so `owner` is a valid reference; the template now writes `catalog-info.yaml` with the owner chosen from a picker, and a CI check using Conftest fails any repo whose `catalog-info.yaml` lacks an owner or lifecycle (see Policy as Code §18.7); and the template also adds a TechDocs skeleton so documentation exists from day one. New services made by the template are fully catalogued; the old ones are migrated by a scripted pull request per repo. **Lesson:** a portal succeeds when the easiest way to create something is also the way that produces clean metadata.

8 Common mistakes and best practices

Don't

- Treat Backstage as a one-time installation; it needs an owning team, upgrades and a roadmap.
- Run production on the lab defaults (in-memory SQLite, guest login, static file locations).
- Let humans maintain catalog data by hand in a central repo; keep `catalog-info.yaml` next to the code.
- Copy plugin wiring from old tutorials without checking your Backstage version.
- Give the scaffolder a broad GitHub token; scope it to what templates need.

Do

- Start small: catalog plus one excellent template, then add plugins that remove a real daily pain.
- Ingest users and groups from your identity provider so ownership references resolve.
- Make templates the *only* easy path: they should register the component, add docs and wire GitOps (see Golden Paths §18.4).
- Validate `catalog-info.yaml` in CI and treat missing owners as build failures.
- Let plugins read from systems of record (Kubernetes, Argo CD) instead of duplicating state.

9 Tasks

- [ ] **T1 — Run Backstage locally** *(≈60 min, beginner)*
  - **Goal:** get the stock app running.
  - **Steps:**
    1. Run `npx @backstage/create-app@latest` and `yarn start` as in section 6.
    2. Open `http://localhost:3000` and browse the example catalog.
  - **Done when:** you can open the Catalog page and see at least one example component and its owner.
- [ ] **T2 — Catalog** `shop-api` *(≈30 min, beginner)*
  - **Goal:** model a service, its database and its API.
  - **Steps:**
    1. Create the three-entity `catalog-info.yaml` from step 3 and register it under `catalog.locations`.
    2. Restart and open the component page.
  - **Done when:** the page shows owner `team-shop` and a relation graph containing `shop-db` and `shop-api-http`.
- [ ] **T3 — Write a software template** *(≈60 min, intermediate)*
  - **Goal:** scaffold a service from a form.
  - **Steps:**
    1. Create the template and a small skeleton (Dockerfile, `catalog-info.yaml` using `${{ values.name }}`, a README).
    2. Register the template and open **Create…**.
    3. Replace `publish:github` with `publish:file` if your Backstage release lists it, or the dry-run feature, so you can test without GitHub.
  - **Done when:** running the template produces a directory (or repo) whose `catalog-info.yaml` has the name you typed in the form.
- [ ] **T4 — Break and fix the template** *(≈45 min, intermediate)*
  - **Goal:** learn to read scaffolder failures.
  - **Steps:**
    1. Remove the GitHub token and run the template; read the failing step in the task log.
    2. Introduce a typo in a `${{ parameters.xyz }}` reference and read the error.
    3. Fix both.

- **Done when:** the task log goes from failed to completed, and you can explain which step produced each error.

- [ ] **T5 — Add TechDocs** *(≈45 min, intermediate)*
  - **Goal:** docs rendered from the repo.
  - **Steps:**
    1. Add `mkdocs.yml`, `docs/index.md` and the annotation `backstage.io/techdocs-ref: dir:.`.
    2. Configure the local builder and open the Docs tab.
    3. Edit `index.md`, refresh and observe the change.
  - **Done when:** your Markdown text is visible in the Docs tab of the component.

- [ ] **T6 — Prove it: portal design review** *(≈60 min, advanced)*
  - **Goal:** justify a minimal production design.
  - **Steps:**
    1. On one page, draw the production deployment: database, auth provider, catalog ingestion, TechDocs storage and the GitOps link from the scaffolder.
    2. List three metrics that show the portal is succeeding (for example share of services with owners, template usage, time to first deploy).
  - **Done when:** a colleague can find where each lab shortcut (SQLite, guest login, file locations) is replaced.

10 Self-check questions

1. What are the four core capabilities of Backstage covered here?
2. Why should `catalog-info.yaml` live in the service's repository?
3. What do the `fetch:template`, `publish:github` and `catalog:register` actions do, in order?
4. What is the difference between TechDocs `builder: local` and `builder: external`?
5. Which parts of the default `create-app` setup must change before production?

ANSWERS

1. Software Catalog, Software Templates (Scaffolder), TechDocs, and the plugin system.
2. Ownership and metadata change with the code, get reviewed in the same pull request, and can be discovered automatically; a central hand-edited repo goes stale.
3. Render the skeleton with the form values; create the Git repository and push the rendered files; add the new repository's `catalog-info.yaml` to the catalog.

4. `local` builds the docs inside Backstage on demand (simple, slower, needs MkDocs or Docker there); `external` expects CI to build and publish to storage, and Backstage only reads it.
5. In-memory SQLite (use PostgreSQL), guest authentication (use a real identity provider), static file-based catalog locations (use provider-based discovery), and local TechDocs publishing.

11 Further reading

- Backstage documentation (backstage.io/docs): "Software Catalog", "Software Templates", "TechDocs" and "Getting Started".
- The `backstage/backstage` repository on GitHub, including the plugin directory and the app templates.
- CNCF project page for Backstage.
- Platform Engineering community material on measuring portal adoption (for example the CNCF Platforms White Paper).

18.9 Crossplane

Topic 9 of 10 | ~6 hours | Difficulty: Advanced

1 What it is

Crossplane is an open-source CNCF project that turns a Kubernetes cluster into a **control plane for infrastructure**. You describe cloud resources (an S3 bucket, an RDS database, a VPC) as Kubernetes objects; Crossplane **controllers** continuously reconcile the real cloud to match, the same way the Deployment controller keeps Pods running. Because the API is Kubernetes, you get RBAC, admission policy, GitOps delivery and `kubectl` for free.

Its distinctive idea is not "Terraform in YAML" but **building your own platform API**. Platform engineers define higher-level resource types ("an ObjectStorage for a team") and map each to a bundle of low-level cloud resources; product teams request the simple type and never see the 14 underlying objects. The vocabulary:

Term	Meaning
Provider	a package with controllers and CRDs for one external system (AWS S3, GCP, Kubernetes, Helm, Terraform)
Managed Resource (MR)	one Kubernetes object that maps 1:1 to one external resource (`s3.aws.upbound.io/Bucket`)
ProviderConfig	how a provider authenticates (credentials, IRSA/Pod Identity, region)
CompositeResourceDefinition (XRD)	the schema of your new platform API type (like a CRD factory)
Composite Resource (XR)	an instance of that type; the thing that owns the managed resources
Composition	the recipe: which managed resources an XR expands into, wired by a pipeline of **functions**
Claim	a namespaced, developer-facing handle on a cluster-scoped XR (the self-service surface)
Function	a small program that renders desired resources in a Composition pipeline

> **Version note.** Crossplane v2 (released in 2025) made composite resources namespaced by default, so the separate "claim" layer is no longer needed for new-style XRDs, and it removed the older built-in "patch and transform" composition mode in favour of function pipelines. Claims, cluster-scoped XRs and the manifests below remain the model used by the large amount of existing documentation and installations, and legacy XRDs are still supported. Read the upgrade notes for your Crossplane version before copying manifests; the concepts (provider, MR, XRD, Composition, function) are the same.

2 Why it matters in DevOps

- **Self-service without ticket queues.** A developer applies a 10-line claim; the platform provisions bucket, encryption, access block and tags exactly as the platform team decided.
- **Continuous reconciliation of infrastructure.** Terraform detects drift when you run `plan`; a Crossplane controller corrects drift continuously (see Drift Detection §7.9).
- **One control plane, one workflow.** Apps and infrastructure are both Kubernetes manifests delivered by Argo CD (see Argo CD and Flux §17.2).
- **Platform as product.** The XRD is a versioned API contract with your internal customers (see Infrastructure Templates §18.6 and Self-service Infrastructure §18.3).
- **Costs and risks.** The management cluster becomes critical infrastructure; deleting an MR can delete real data; credentials for the cloud now live in (or are reachable from) a cluster.

3 Real-world system example

Crossplane was created by Upbound and donated to the CNCF; it is a CNCF project with a public list of adopters in its GitHub repository. The widely used cloud providers come from the Upbound `provider-aws`, `provider-azure` and `provider-gcp` families, which are generated from Terraform provider schemas via the Upjet tool, so coverage of resources is broad. A public package marketplace (Upbound Marketplace) and OCI registries host providers, functions and **Configuration packages** (XRD plus Composition bundles) that organisations publish internally to distribute their platform APIs.

The same "custom resource plus controller" pattern is how Kubernetes itself works: see Custom Resource Definitions §16.5 and Operators §16.4. Crossplane is a general-purpose operator framework for infrastructure composition.

4 Diagram

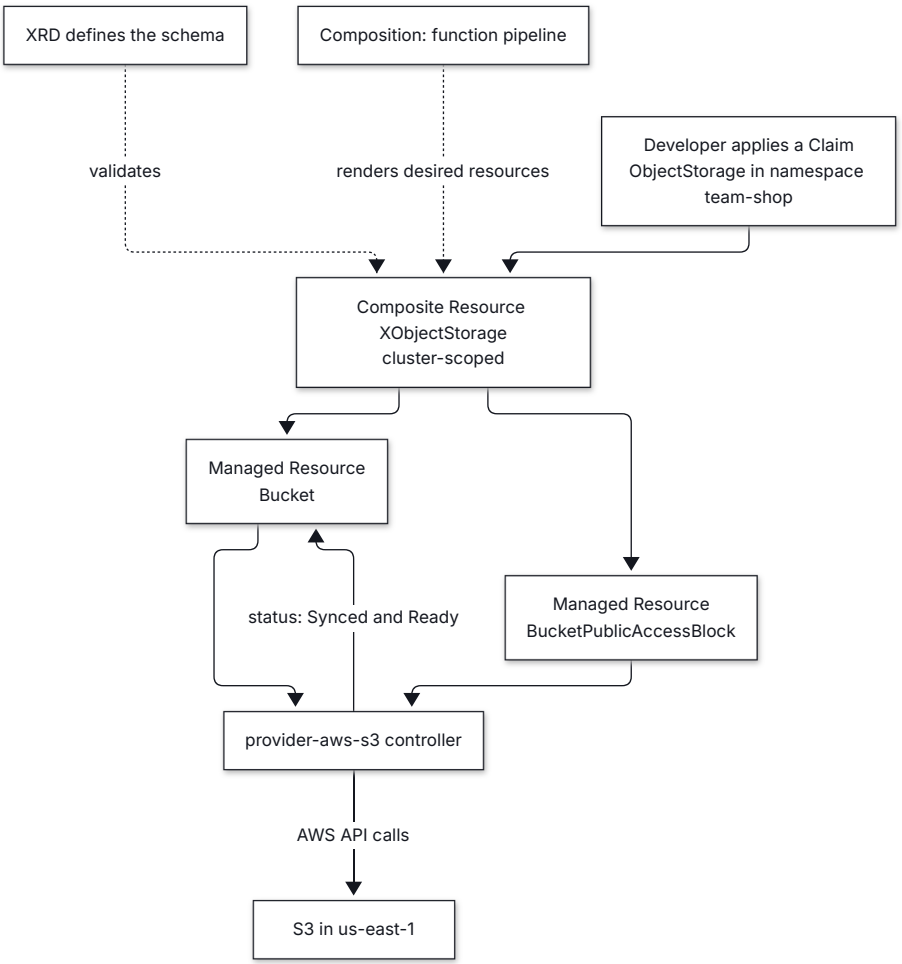


Figure 18.9-1

The reconcile loop for a managed resource:

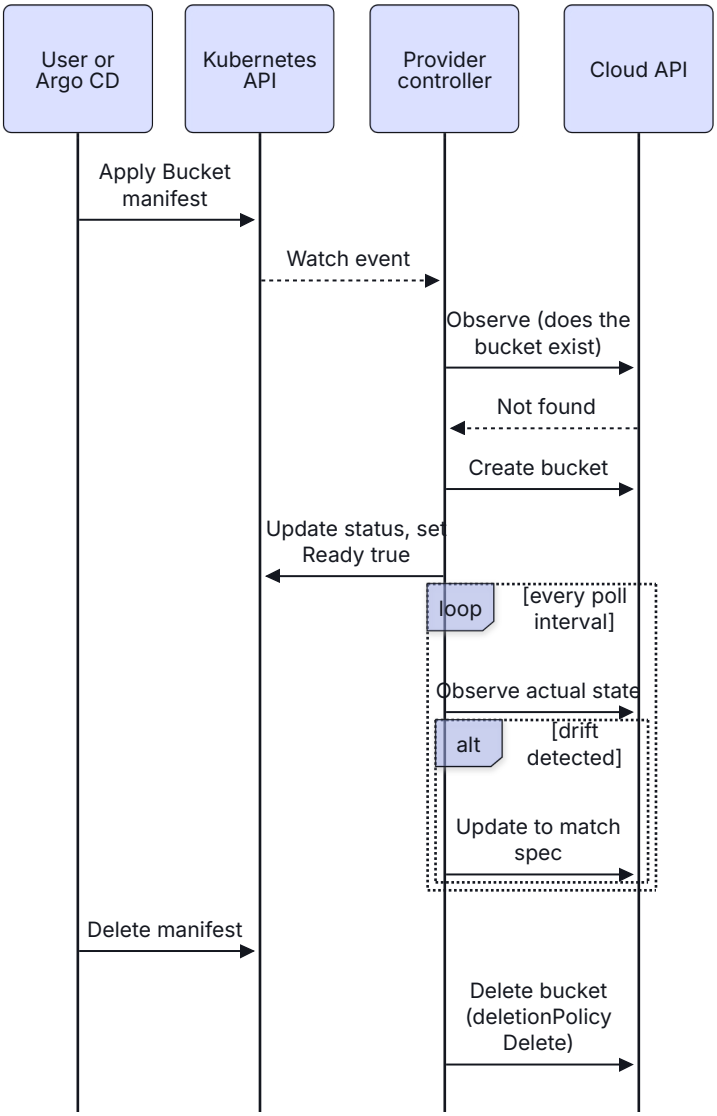


Figure 18.9-2

5 How it works

Layers

1. **Install Crossplane** (a controller in `crossplane-system`).

2. **Install providers and functions** as `Provider` / `Function` resources. Crossplane pulls the OCI package, installs its CRDs and starts the provider pod.
3. **Configure credentials** with a `ProviderConfig`. Prefer workload identity (IRSA or EKS Pod Identity) over static keys.
4. **Define your API** with an `XRD`: the OpenAPI schema defines exactly which fields developers can set, with defaults, enums and required fields, so invalid requests are rejected by the API server.
5. **Write a Composition** that references the XRD (`compositeTypeRef`) and lists the managed resources and how XR fields patch into them. In pipeline mode, each step calls a **function**; `function-patch-and-transform` handles the classic patching style, and others (Go templating, KCL, CEL) are available when you need logic.
6. **Developers create claims** (or XRs, in v2 style) in their namespace.

Useful behaviours

Topic	Detail
Status conditions	MRs have `Synced` (spec was applied) and `Ready` (external resource is usable). Read `kubectl describe` events first.
`deletionPolicy`	`Delete` (default) removes the cloud resource when the MR is deleted; `Orphan` leaves it. Choose deliberately for databases.
`managementPolicies`	restrict what Crossplane may do (for example `Observe` only, for importing existing resources).
Import	set the `crossplane.io/external-name` annotation to adopt an existing resource.
Composition revisions	updating a Composition creates a new revision; XRs can be pinned or follow the latest, controlling rollout of template changes.
Debugging	`crossplane beta trace <kind> <name>` shows the tree from claim to MRs and their conditions.
Testing	`crossplane render xr.yaml composition.yaml functions.yaml` shows what a Composition would produce without a cluster (it runs functions in containers).

Crossplane versus Terraform

	Crossplane	Terraform / OpenTofu
Execution	continuous controllers in a cluster	run on demand (CI, local)
State	the Kubernetes API (etcd)	a state file / backend
Drift	corrected continuously	detected at plan time
Developer API	custom types via XRD	module variables

	Crossplane	Terraform / OpenTofu
Ecosystem	growing, provider coverage varies	very large, mature
Complexity	needs a management cluster and Kubernetes skills	simpler to start

They compose well: see Integrating Argo CD, Terraform and Kubernetes into a Platform §18.10 for when each belongs in the platform.

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`, `helm`, and an **AWS sandbox account** with an IAM user or role limited to S3. **Cost:** the lab creates one empty S3 bucket, which costs effectively nothing, but set an AWS Budget/billing alarm first and run the cleanup step. Provider package versions below are examples; check the Upbound Marketplace for current ones. Output is similar to what is shown, not byte-for-byte.

1. **Create a cluster and install Crossplane.**

```bash
kind create cluster --name crossplane-lab
helm repo add crossplane-stable https://charts.crossplane.io/stable
helm repo update
helm install crossplane crossplane-stable/crossplane \
  --namespace crossplane-system --create-namespace
kubectl -n crossplane-system get pods
```

   Expect `crossplane` and `crossplane-rbac-manager` pods `Running`.

2. **Install the S3 provider and the patch-and-transform function.**

```yaml
# packages.yaml
apiVersion: pkg.crossplane.io/v1
kind: Provider
metadata:
  name: provider-aws-s3
spec:
  package: xpkg.upbound.io/upbound/provider-aws-s3:v1.20.0   # example version
---
apiVersion: pkg.crossplane.io/v1
kind: Function
metadata:
  name: function-patch-and-transform
spec:
  package: xpkg.upbound.io/crossplane-contrib/function-patch-and-transform:v0.8.2   #
  example version
```

```bash
kubectl apply -f packages.yaml
kubectl get providers,functions
```

Wait until `INSTALLED` and `HEALTHY` are `True` for both (a minute or two).

3. **Give the provider credentials (lab only).** Create a credentials file with a sandbox key and store it as a Secret. Never commit it. In production use IRSA / Pod Identity instead.

```bash
cat > aws-creds.conf <<'EOF'
[default]
aws_access_key_id = <YOUR_ACCESS_KEY_ID>
aws_secret_access_key = <YOUR_SECRET_ACCESS_KEY>
EOF
kubectl -n crossplane-system create secret generic aws-creds --from-file=creds=./aws-
  creds.conf
rm aws-creds.conf
```

```yaml
# providerconfig.yaml
apiVersion: aws.upbound.io/v1beta1
kind: ProviderConfig
metadata:
  name: default
spec:
  credentials:
    source: Secret
    secretRef:
      namespace: crossplane-system
      name: aws-creds
      key: creds
```

```bash
kubectl apply -f providerconfig.yaml
```

4. **Create one managed resource by hand** to see the lowest layer before abstracting it.

```yaml
# bucket-mr.yaml
apiVersion: s3.aws.upbound.io/v1beta1
kind: Bucket
metadata:
  name: crossplane-lab-example-12345      # S3 names are global: make this unique
spec:
  forProvider:
    region: us-east-1
    tags:
      owner: team-shop
  providerConfigRef:
    name: default
```

```bash
kubectl apply -f bucket-mr.yaml
kubectl get bucket
```

Expect `SYNCED True` and `READY True` shortly. Check with `aws s3 ls`. Delete it (`kubectl delete -f bucket-mr.yaml`) and confirm the bucket disappears from AWS.

5. **Define the platform API (XRD).** Developers will see only `owner` and `region`.

```yaml
# xrd.yaml
apiVersion: apiextensions.crossplane.io/v1
kind: CompositeResourceDefinition
metadata:
  name: xobjectstorages.platform.example.org
spec:
  group: platform.example.org
  names:
    kind: XObjectStorage
    plural: xobjectstorages
  claimNames:
    kind: ObjectStorage
    plural: objectstorages
  versions:
    - name: v1alpha1
      served: true
      referenceable: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
```

```
            properties:
              parameters:
                type: object
                properties:
                  owner:
                    type: string
                  region:
                    type: string
                    default: us-east-1
                    enum: [us-east-1, eu-west-1]
                required: [owner]
            required: [parameters]
```

6. **Write the Composition.** One request becomes a bucket plus a public-access block, with the owner tag patched in.

```yaml
# composition.yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
metadata:
  name: xobjectstorages.aws.platform.example.org
  labels:
    provider: aws
spec:
  compositeTypeRef:
    apiVersion: platform.example.org/v1alpha1
    kind: XObjectStorage
  mode: Pipeline
  pipeline:
    - step: patch-and-transform
      functionRef:
        name: function-patch-and-transform
      input:
        apiVersion: pt.fn.crossplane.io/v1beta1
        kind: Resources
        resources:
          - name: bucket
            base:
              apiVersion: s3.aws.upbound.io/v1beta1
              kind: Bucket
              spec:
                forProvider:
                  region: us-east-1
            patches:
              - type: FromCompositeFieldPath
                fromFieldPath: spec.parameters.region
                toFieldPath: spec.forProvider.region
              - type: FromCompositeFieldPath
                fromFieldPath: spec.parameters.owner
                toFieldPath: spec.forProvider.tags.owner
```

```
    - name: public-access-block
      base:
        apiVersion: s3.aws.upbound.io/v1beta1
        kind: BucketPublicAccessBlock
        spec:
          forProvider:
            region: us-east-1
            blockPublicAcls: true
            blockPublicPolicy: true
            ignorePublicAcls: true
            restrictPublicBuckets: true
            bucketSelector:
              matchControllerRef: true
      patches:
        - type: FromCompositeFieldPath
          fromFieldPath: spec.parameters.region
          toFieldPath: spec.forProvider.region
```

BASH

```
kubectl apply -f xrd.yaml
kubectl wait --for=condition=Established xrd/xobjectstorages.platform.example.org --
  timeout=60s
kubectl apply -f composition.yaml
```

7. Request storage as a developer (the claim).

YAML

```
# claim.yaml
apiVersion: platform.example.org/v1alpha1
kind: ObjectStorage
metadata:
  name: shop-assets
  namespace: team-shop
spec:
  parameters:
    owner: team-shop
    region: us-east-1
```

BASH

```
kubectl create namespace team-shop
kubectl apply -f claim.yaml
kubectl -n team-shop get objectstorage
kubectl get composite
kubectl get managed
```

Expect the claim to become `READY True`, one XR, and two managed resources (`bucket` and `bucketpublicaccessblock`). If something is stuck, run `kubectl -n team-shop describe objectstorage shop-assets` and read the events.

8. **Break the contract on purpose.** A region outside the enum is rejected by the API server before anything is created:

```bash
kubectl -n team-shop apply -f - <<'EOF'
apiVersion: platform.example.org/v1alpha1
kind: ObjectStorage
metadata:
  name: bad
spec:
  parameters:
    owner: team-shop
    region: ap-south-1
EOF
```

Expect an error similar to `Unsupported value: "ap-south-1": supported values: "us-east-1", "eu-west-1"`.

9. **Clean up (do not skip).**

```bash
kubectl -n team-shop delete objectstorage shop-assets      # deletes the S3 bucket
kubectl get managed                                          # wait until empty
kind delete cluster --name crossplane-lab
```

Finally verify in the AWS console or with `aws s3 ls` that no lab buckets remain, and delete the sandbox IAM access key.

7 Real-world scenario

(Illustrative.) A team deletes the namespace `team-shop` during a clean-up. Claims in that namespace go with it, and because the default `deletionPolicy` is `Delete`, Crossplane dutifully deletes the real S3 bucket holding customer uploads. The bucket had versioning off, so there is nothing to recover from the console; the data is restored from the nightly cross-region copy, with 14 hours of uploads lost.

The post-incident changes: the Composition now sets `deletionPolicy: Orphan` for data-bearing resources and enables versioning and a replication rule; Kyverno policy (see Policy as Code §18.7) denies deleting namespaces labelled `data=true`; Argo CD's sync policy for the in-

fra applications has `prune: false` with manual approval for deletes. **Lesson:** a control plane that reconciles also deletes; the safety rails (deletion policy, prune policy, RBAC on who may delete claims) are part of the platform design.

8 Common mistakes and best practices

Don't

- Expose every field of the underlying resources in the XRD; keep the API small and opinionated.
- Leave `deletionPolicy: Delete` on databases and buckets without backups and a deliberate review.
- Use long-lived static cloud keys in a Secret outside labs; use workload identity.
- Edit a Composition in production without understanding revisions; test with `crossplane render` and in a staging cluster.
- Treat the management cluster as disposable; it holds the only reconciling copy of your infrastructure intent (Git must hold it too).

Do

- Version your XRDs (`v1alpha1` to `v1`) and publish Compositions as Configuration packages.
- Put validation in the XRD schema (enum, default, required) so bad requests fail at admission.
- Deliver all Crossplane resources through Git and Argo CD; use sync waves so XRD, Composition and claims apply in order.
- Alert on MRs that are not `Synced` or not `Ready` for more than a few minutes.
- Combine with policy to enforce tags, regions and deletion protection.

9 Tasks

- [ ] **T1 — Install Crossplane and one provider** *(≈45 min, intermediate)*
  - **Goal:** reach a healthy control plane.
  - **Steps:**
    1. Follow steps 1-3 of the guide in a kind cluster.
    2. Run `kubectl get providers,functions`.
  - **Done when:** both packages show `INSTALLED=True` and `HEALTHY=True`.
- [ ] **T2 — Create and delete a managed resource** *(≈30 min, intermediate)*

- **Goal:** observe the 1:1 mapping and reconcile loop.
- **Steps:**
  1. Apply the `Bucket` MR from step 4 and confirm it exists in AWS.
  2. Change a tag in the manifest and re-apply; verify the tag changes in AWS.
  3. Delete the MR.
- **Done when:** the bucket appeared, the tag changed, and the bucket is gone after deletion (check with `aws s3 ls`).

☐ **T3 — Build the ObjectStorage API** *(≈60 min, advanced)*

- **Goal:** go from MR to platform API.
- **Steps:**
  1. Apply the XRD, Composition and claim from steps 5-7.
  2. Run `kubectl get managed` and `crossplane beta trace objectstorage shop-assets -n team-shop` (if the CLI is installed).
- **Done when:** the claim is `READY=True` and `kubectl get managed` lists a bucket and a public-access block.

☐ **T4 — Extend the Composition** *(≈45 min, advanced)*

- **Goal:** add a feature without changing the developer API.
- **Steps:**
  1. Add a third resource, `BucketVersioning` (or server-side encryption), to the Composition.
  2. Re-apply and watch the existing XR gain the new managed resource.
- **Done when:** `kubectl get managed` shows the new resource for the existing claim and AWS shows versioning enabled.

☐ **T5 — Break it and diagnose it** *(≈45 min, advanced)*

- **Goal:** learn the failure modes.
- **Steps:**
  1. Reuse a bucket name that is already taken, or break the credentials Secret.
  2. Find the cause using `kubectl describe` on the claim, the XR and the MR (events and conditions).
  3. Fix it and watch the claim become `Ready`.
- **Done when:** you can state which object first showed the error and the exact condition message.

☐ **T6 — Prove it: Crossplane versus Terraform decision** *(≈45 min, advanced)*

- **Goal:** make and defend a platform decision.
- **Steps:**
  1. Without notes, write a one-page comparison of using Crossplane versus Terraform modules for "team requests an S3 bucket" in your org.
  2. Recommend one, name three risks and one mitigation each (deletion policy, credentials, upgrade).
- **Done when:** a teammate can explain your recommendation back and identify the rollback plan.

10 Self-check questions

1. What is the difference between a Managed Resource and a Composite Resource?
2. What does an XRD do that a plain CRD does not?
3. Why can a developer not set `region: ap-south-1` in the example claim?
4. What does `deletionPolicy: Orphan` change?
5. How does Crossplane's handling of drift differ from Terraform's?

ANSWERS

1. An MR maps 1:1 to one external resource and is owned by a provider; an XR is your own abstraction that is composed into several MRs by a Composition.
2. It defines a new platform API type, generates the CRDs for the XR and the claim, and links them to Compositions that say how to implement it.
3. The XRD schema restricts `region` to an enum, and the API server rejects values outside it before any controller runs.
4. Deleting the Kubernetes object no longer deletes the external resource; it is left running and unmanaged.
5. Crossplane providers poll and reconcile continuously, correcting drift automatically; Terraform only detects and fixes drift when a plan/apply runs.

11 Further reading

- Crossplane documentation (docs.crossplane.io): "Concepts", "Composition" and "Get Started".
- Upbound Marketplace for provider and function packages and their API references.
- Crossplane GitHub repository (`crossplane/crossplane`) and release notes, especially the v2 upgrade notes.

- CNCF Platforms White Paper for control-plane based platform design.

LAB NOTES What I ran, what I saw, what it means

18.10 Integrating Argo CD, Terraform and Kubernetes into a Platform

Topic 10 of 10 | ~5 hours | Difficulty: Advanced

1 What it is

A platform is not one tool; it is a handful of tools that each own one job, wired together so a developer's single action (a form, a pull request) turns into running software and infrastructure. This topic is about the **composition**: which tool is the **control plane** for what, how **GitOps delivery** moves changes, and -- the part that breaks most platforms -- who owns which **state**.

The usual cast: **Git** holds desired state; **Argo CD** (see Argo CD and Flux §17.2) continuously applies Git to Kubernetes; **Kubernetes** is the runtime and, with **Crossplane**, a control plane for cloud resources (Crossplane §18.9); **Terraform/OpenTofu** builds the foundations that must exist before any cluster does (accounts, networks, the cluster itself) and anything not worth modelling as a Kubernetes API; **policy engines** (Policy as Code §18.7) guard every path; and **Backstage** (Backstage §18.8) is the human front door that writes to Git.

2 Why it matters in DevOps

- **Two tools managing one resource is the classic platform outage.** If Terraform and Crossplane (or Terraform and a controller, or a human in the console) all believe they own a database, one of them will "fix" the other's change.
- **Every tool has a state store, and the stores disagree.** Terraform has a state file, Kubernetes has etcd, Git has history, and the cloud has the truth. Knowing which is authoritative for what is the core skill.
- **Bootstrapping order.** You cannot run Crossplane on a cluster that Crossplane must create. Foundations come first, and the dependency direction must never loop.
- **Delivery safety.** A merge to `main` is now a production change for apps *and* infrastructure; sync order, pruning and approvals decide whether that is safe.
- **Team scaling.** A clear contract (developers touch claims and app manifests; platform team touches templates and foundations) is what lets a small platform team serve many product teams (see Internal Developer Platforms §18.1).

3 Real-world system example

The pattern of **GitOps for apps with Argo CD, a cloud control plane for infrastructure, and Terraform for the foundations** is described across public CNCF material, Argo CD and Crossplane documentation, and conference talks. The Argo CD documentation describes **app of apps** and **ApplicationSet** as the ways to manage many applications from one repository, and its **sync waves** and **hooks** for ordering. Crossplane's documentation explains running it alongside GitOps tools so that claims live in Git like any other manifest. Terraform's own documentation and many public architectures use it to create the Kubernetes cluster and bootstrap Argo CD, after which Argo CD takes over everything inside the cluster.

Kubernetes itself models this layering: the API server stores desired state, controllers (the built-in ones, Argo CD's application controller, Crossplane's provider controllers) reconcile independently, and each controller owns a clearly bounded set of objects, with ownership recorded in `ownerReferences` and field managers.

4 Diagram

The whole platform on one page:

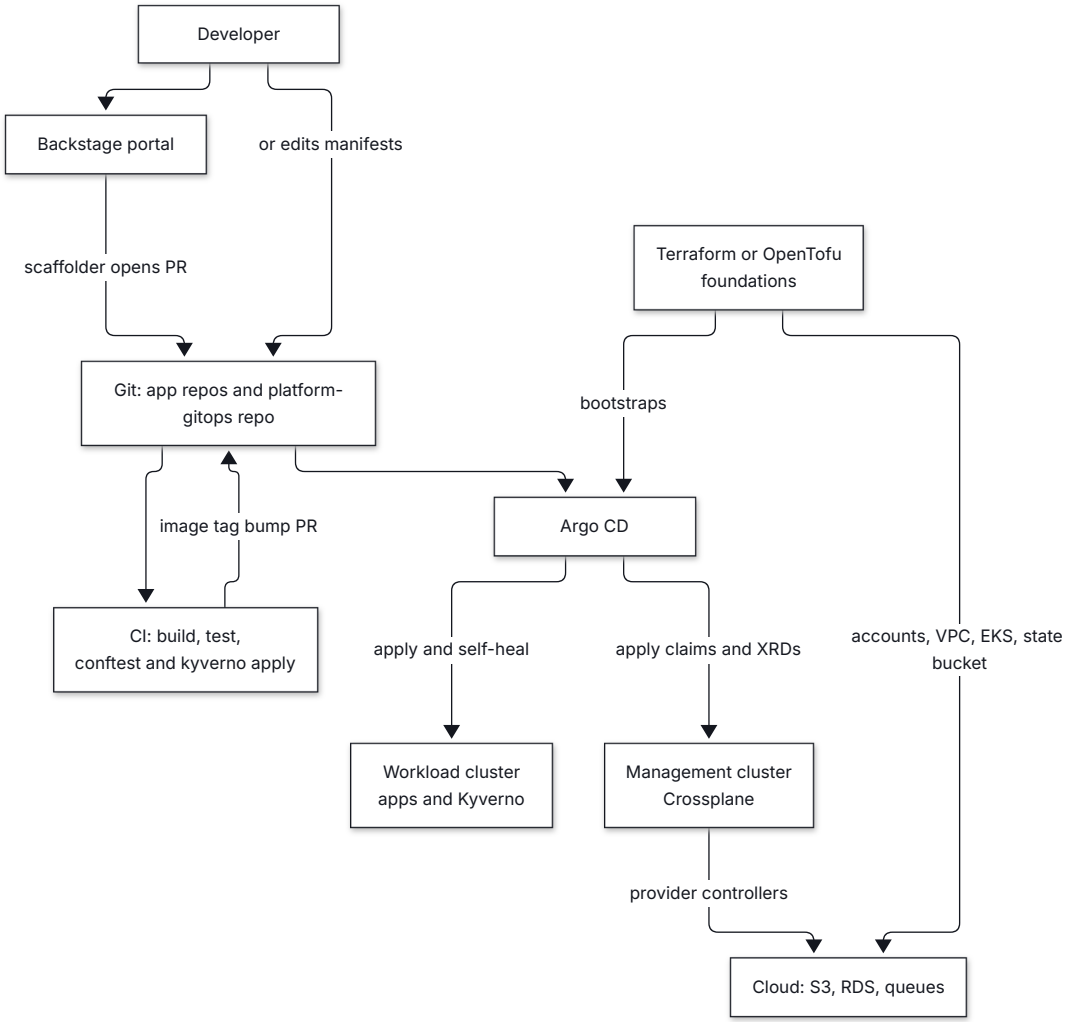


Figure 18.10-1

Ownership and the flow of state, for one `shop-api` database request:

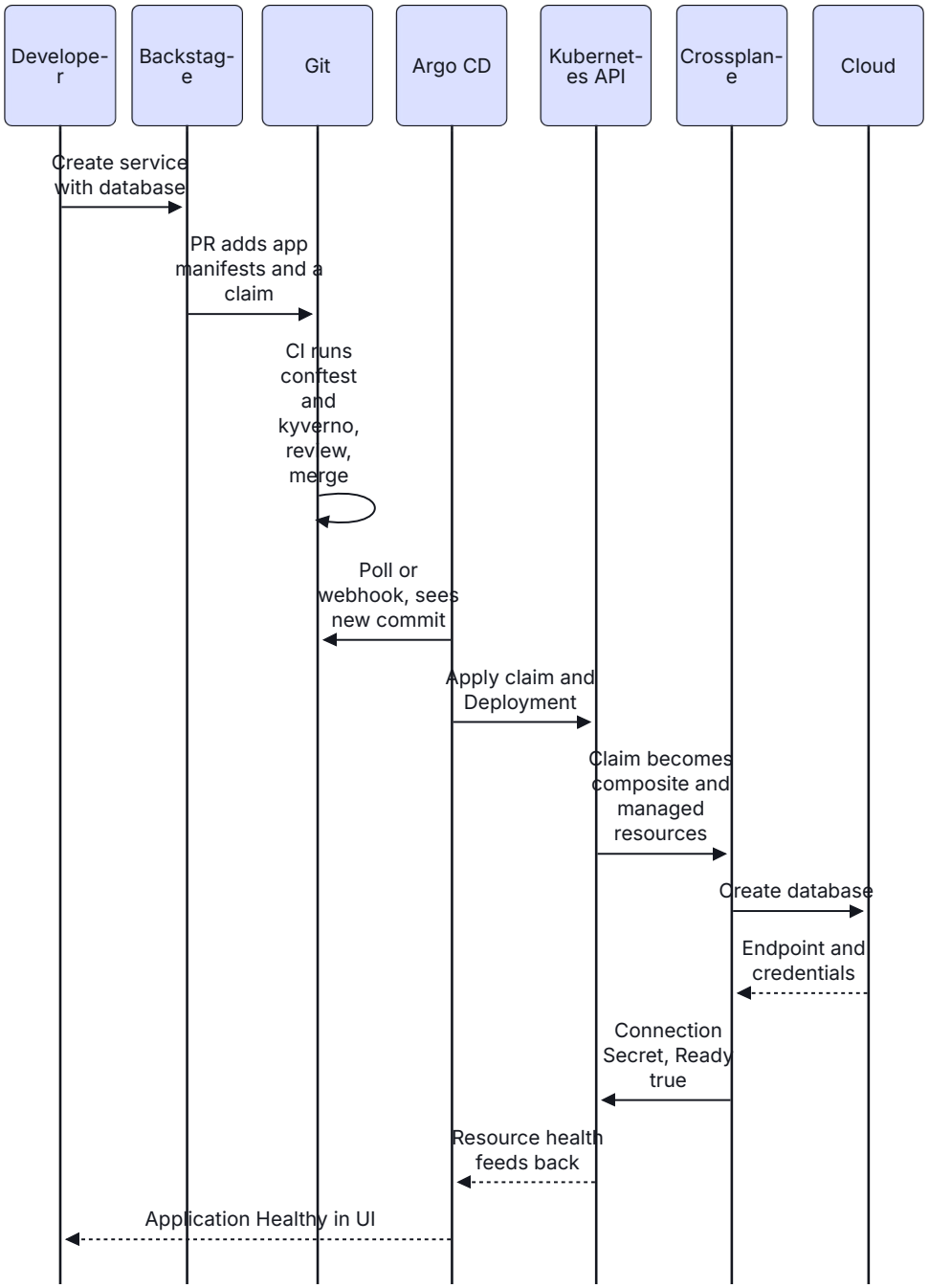


Figure 18.10-2

5 How it works

Who owns what (the state-ownership contract)

Concern	Tool (owner)	Desired state lives in	Actual state read from	Drift handling
Cloud accounts, org policy, networks, the clusters, Argo CD bootstrap	Terraform / OpenTofu	Git (HCL)	cloud, compared via the **state file**	`plan` in CI on a schedule, human approves apply
Per-team cloud resources (buckets, queues, databases)	Crossplane	Git (claims)	the cloud, observed by provider controllers	continuous, automatic
Applications and cluster add-ons	Argo CD	Git (manifests, Helm values)	Kubernetes API	continuous (`selfHeal`), prune optional
Security and compliance rules	Kyverno / OPA	Git (policies)	admission requests and background scans	audit reports, enforced at admission
Service metadata and ownership	Backstage catalog	`catalog-info.yaml` in each repo	rebuilt from Git	refresh from source

The rules that keep this sane:

1. **One resource, one owner.** If Terraform created it, Crossplane must not also manage it. If you move ownership, do it explicitly: remove from one state (`terraform state rm`) and import into the other with the external-name annotation, with the controller in `Observe` mode first.
2. **Git is the source of truth for intent, the system of record for actual state is the cloud.** State files and etcd are caches and bookkeeping. Back them up, but never edit them as a normal workflow.
3. **Dependencies point one way:** Terraform (foundations) → cluster → Argo CD → Crossplane and policy → team claims and apps. Nothing lower in the chain depends on something higher.
4. **Humans approve what is hard to undo** (accounts, networks, deletion), automation applies what is routine (claims, image bumps).

Delivery mechanics

- **App of apps / ApplicationSet:** one root Argo CD `Application` (or an `ApplicationSet` with a Git directory generator) points at a folder; each subfolder becomes an Application. Adding a team is adding a folder.

- **Sync waves:** `argocd.argoproj.io/sync-wave` annotations order the apply: policy engine and Crossplane first (negative waves), then XRDs and Compositions, then claims and apps.
- **Health of custom resources:** Argo CD does not know what "healthy" means for a claim; add a custom health check that looks at the `Ready` condition, otherwise dependent waves proceed too early.
- **Pruning:** `prune: true` deletes cloud resources when you delete a claim from Git. That is correct for dev and often wrong for production data; combine with `deletionPolicy: Orphan` or manual sync for stateful claims.
- **Where Terraform runs:** in CI (plan on PR, apply on merge with approval, see Terraform CI/CD §7.7), or in-cluster via a controller. Both keep the same rule: state in a locked remote backend (see State Locking §7.8).

Typical repository layout

Repo	Contains	Owned by
`infra-foundations`	Terraform for accounts, VPCs, clusters, bootstrap of Argo CD	platform team
`platform-gitops`	Argo CD root app, add-ons, Crossplane XRDs and Compositions, policies, `teams/<team>/<env>/` claims	platform team (platform parts), product teams (their `teams/` folder)
`shop-api` (app repo)	code, Dockerfile, Helm chart or Kustomize, `catalog-info.yaml`	product team

6 Step-by-step guide

Prerequisites: Git, Docker, `kind`, `kubectl`, and for the policy check the `conftest` binary. The first half runs offline on files; the optional second half installs Argo CD into a kind cluster and uses a GitHub repository you own (any Git host Argo CD can reach works). No cloud account or cost is required. Cluster outputs are similar to what is shown.

1. **Lay out the platform GitOps repository.**

```bash
mkdir -p platform-gitops/{bootstrap,platform/{policy,crossplane,addons},teams/team-shop/prod,teams/team-pay/prod,policy}
cd platform-gitops && git init -b main
```

`bootstrap/` holds the root Argo CD apps, `platform/` holds what the platform team owns, `teams/<team>/<env>/` holds what product teams own, and the top-level `policy/` is for CI checks (Conftest).

2. **Add two team claims, one wrong on purpose.** (These use the `ObjectStorage` API from Crossplane §18.9.)

```bash
cat > teams/team-shop/prod/assets.yaml <<'EOF'
apiVersion: platform.example.org/v1alpha1
kind: ObjectStorage
metadata:
  name: shop-assets
  namespace: team-shop
spec:
  parameters:
    owner: team-shop
    region: us-east-1
EOF
cat > teams/team-pay/prod/receipts.yaml <<'EOF'
apiVersion: platform.example.org/v1alpha1
kind: ObjectStorage
metadata:
  name: receipts
  namespace: team-pay
spec:
  parameters:
    owner: team-shop
EOF
```

3. **Add a CI-side guardrail: a team may only declare storage owned by itself.**

```rego
# policy/claims.rego
package main

import rego.v1

deny contains msg if {
  input.kind == "ObjectStorage"
  input.spec.parameters.owner ≠ input.metadata.namespace
  msg := sprintf("ObjectStorage %q: owner %q must equal its namespace %q",
  [input.metadata.name, input.spec.parameters.owner, input.metadata.namespace])
}
```

```bash
conftest test teams/*/*/*.yaml
```

Expected:

```
FAIL - teams/team-pay/prod/receipts.yaml - main - ObjectStorage "receipts": owner
  "team-shop" must equal its namespace "team-pay"

2 tests, 1 passed, 0 warnings, 1 failure, 0 exceptions
```

Fix `receipts.yaml` (`owner: team-pay`) and re-run to get exit code 0. This runs in CI before merge; the cluster-side Kyverno policy is the second layer.

4. **Define the ApplicationSet that turns folders into Applications.** Adding `teams/team-x/prod/` creates an app, no Argo CD change needed.

YAML

```yaml
# bootstrap/teams-appset.yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: team-resources
  namespace: argocd
spec:
  goTemplate: true
  goTemplateOptions: ["missingkey=error"]
  generators:
    - git:
        repoURL: https://github.com/example-org/platform-gitops.git   # use your own
  repo
        revision: main
        directories:
          - path: teams/*/*
  template:
    metadata:
      name: '{{ index .path.segments 1 }}-{{ index .path.segments 2 }}'
    spec:
      project: default
      source:
        repoURL: https://github.com/example-org/platform-gitops.git
        targetRevision: main
        path: '{{ .path.path }}'
      destination:
        server: https://kubernetes.default.svc
        namespace: '{{ index .path.segments 1 }}'
      syncPolicy:
        automated:
          prune: false        # deleting a claim from Git will not silently delete
  cloud data
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
```

5. **Order the platform components with sync waves.** Policy engine and Crossplane first, then packages, then the API definitions, then team claims. Each child `Application` (or resource) gets an annotation:

```
wave -2 : Kyverno and Crossplane installs (Helm-based Applications)
wave -1 : Provider and Function packages, ProviderConfig
wave  0 : XRDs and Compositions (the default wave)
wave  1 : team claims and applications
```

   For example, the Application that installs Crossplane:

```yaml
# bootstrap/crossplane-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: crossplane
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  project: default
  source:
    repoURL: https://charts.crossplane.io/stable
    chart: crossplane
    targetRevision: 1.20.0        # example version, pin the one you tested
  destination:
    server: https://kubernetes.default.svc
    namespace: crossplane-system
  syncPolicy:
    automated: {selfHeal: true}
    syncOptions: [CreateNamespace=true]
```

   Waves order resources inside one sync. When the waves live on child `Application` objects created by a root app-of-apps, the root application's sync honours them, which is how cross-application ordering is done.

6. **Teach Argo CD what "healthy" means for a claim.** Add a custom health check to the `argocd-cm` ConfigMap so waves wait for `Ready`:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-cm
  namespace: argocd
  labels:
    app.kubernetes.io/part-of: argocd
```

```
data:
  resource.customizations.health.platform.example.org_ObjectStorage: |
    hs = {}
    hs.status = "Progressing"
    hs.message = "Waiting for Ready condition"
    if obj.status ~= nil and obj.status.conditions ~= nil then
      for _, c in ipairs(obj.status.conditions) do
        if c.type == "Ready" and c.status == "True" then
          hs.status = "Healthy"
          hs.message = "Ready"
        end
      end
    end
    return hs
```

7. **(Optional) Bring up Argo CD locally and sync the repo.**

```bash
kind create cluster --name platform-lab
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-
  cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server
kubectl -n argocd get secret argocd-initial-admin-secret -o
  jsonpath='{.data.password}' | base64 -d; echo
kubectl -n argocd port-forward svc/argocd-server 8080:443
```

Open `https://localhost:8080` (accept the self-signed certificate), push your repo to GitHub, edit the `repoURL` values, and `kubectl apply -f bootstrap/teams-appset.yaml`. Without Crossplane installed, the claim Applications will show an error about the unknown kind -- that is the point of T4: ordering matters.

8. **Write the bootstrap story down.** The order that must hold on a fresh environment:

```
1. terraform apply        → VPC, cluster, state bucket, IAM, installs Argo CD (Helm)
  and ONE root Application
2. Argo CD syncs root     → Kyverno, Crossplane, providers, functions, XRDs,
  Compositions, policies
3. Argo CD syncs teams    → ApplicationSet creates one Application per
  teams/<team>/<env> folder
4. Crossplane reconciles  → claims become cloud resources; connection Secrets appear
  in team namespaces
```

7 Real-world scenario

(Illustrative.) A platform team provisions the production database for `shop-api` with Terraform in `infra-foundations`, and a few months later adds the Crossplane `PostgresInstance` claim so product teams can request their own databases. A well-meaning engineer migrates `shop-api`'s database to a claim by writing a new claim with the same identifier. Crossplane, seeing no existing managed resource, creates a *second* database; the next `terraform apply` plan wants to delete the first one because it was "renamed" in the HCL. The pipeline stops at the manual approval, which is the only reason production data survives.

The fix is the ownership rule: a migration runbook that first sets Terraform to stop managing the resource (`terraform state rm` after removing the block), then imports it into Crossplane with the external-name annotation and `managementPolicies: ["Observe"]`, verifies the XR reads it correctly, and only then enables full management. A Conftest rule now rejects a claim whose resource name collides with one in the Terraform state inventory. **Lesson:** integrating tools means defining handover procedures; the dangerous moment is always the transfer of ownership.

8 Common mistakes and best practices

Don't

- Let two controllers or tools manage the same resource, even "temporarily".
- Enable `prune: true` plus `deletionPolicy: Delete` for production data without human approval.
- Put Terraform state in the same account and bucket with no locking or backups.
- Let Argo CD proceed to claims before Crossplane's provider is healthy; use waves and health checks.
- Hide platform behaviour in the portal only; every action must be a Git change that can be reviewed and reverted.

Do

- Write a one-page ownership matrix (like section 5) and keep it next to the repositories.
- Keep the dependency chain acyclic: foundations, cluster, Argo CD, platform controllers, teams.
- Run policy checks in CI and at admission so the same rules apply on both paths.
- Treat the management cluster and its Git repo as the crown jewels: restrict who can merge, back up Terraform state, test a full rebuild from scratch.

- Measure the platform: lead time from "I need a service" to "serving traffic", and the share of requests that needed a human.

9 Tasks

- [ ] **T1 — Build the repo skeleton and guardrail** *(≈30 min, beginner)*
  - **Goal:** have a GitOps repo whose bad claim is caught in CI.
  - **Steps:**
    1. Follow steps 1-3 of the guide.
    2. Run `conftest test teams/*/*/*.yaml`, then fix the bad claim.
  - **Done when:** the command fails with one `FAIL` line for the bad claim and exits 0 after your fix.
- [ ] **T2 — Draw your ownership matrix** *(≈30 min, intermediate)*
  - **Goal:** make state ownership explicit for the `shop-api` stack.
  - **Steps:**
    1. List every resource `shop-api` needs (cluster, VPC, Postgres, Redis, bucket, Deployment, DNS record, alert rules).
    2. For each, choose the owner (Terraform, Crossplane, Argo CD) and where its desired state lives.
  - **Done when:** every resource has exactly one owner and a justification you can say aloud.
- [ ] **T3 — Generate Applications with an ApplicationSet** *(≈60 min, intermediate)*
  - **Goal:** onboard a team by adding a folder.
  - **Steps:**
    1. Do step 7: kind plus Argo CD, with your repository.
    2. Apply the ApplicationSet, then add `teams/team-new/prod/` with a simple ConfigMap and push.
  - **Done when:** a new Application named `team-new-prod` appears and syncs without any Argo CD configuration change.
- [ ] **T4 — Break the ordering and fix it** *(≈60 min, advanced)*
  - **Goal:** feel why sync waves and health checks exist.
  - **Steps:**
    1. Apply the claim before Crossplane and its XRD exist; read the Argo CD error.
    2. Add sync waves and the custom health check, then resync from empty.

- **Done when:** a fresh sync from a new kind cluster converges to Healthy with no manual retries.

- [ ] **T5 — Practise an ownership handover** *(≈60 min, advanced)*
  - **Goal:** move a resource from Terraform to Crossplane safely (on paper plus a local simulation).
  - **Steps:**
    1. Using a harmless resource (for example a `local_file` in Terraform), write the runbook: remove from state, adopt elsewhere, verify.
    2. Execute the Terraform half: `terraform state rm` the resource, confirm `terraform plan` shows no changes and the file still exists.
  - **Done when:** the plan is clean after `state rm` and the real resource was never destroyed.

- [ ] **T6 — Prove it: explain the platform in five minutes** *(≈45 min, advanced)*
  - **Goal:** be able to teach the composition to a new engineer.
  - **Steps:**
    1. Without notes, draw the two diagrams from section 4 and walk through "a developer needs a database for a new service".
    2. Name the owner of each piece of state and what happens if two owners collide.
  - **Done when:** a teammate can answer "where do I change X, and what will reconcile it?" for five different X.

10 Self-check questions

1. Why is Terraform usually the right tool for the cluster and Crossplane for per-team resources inside it?
2. What does the rule "one resource, one owner" prevent?
3. Why does the ApplicationSet in the guide use `prune: false`?
4. What breaks if Argo CD has no health check for a custom resource?
5. In what order must the platform be bootstrapped, and why can it not be circular?

ANSWERS

1. Terraform runs outside the cluster and can create the cluster and its network, which Crossplane would need to exist first; Crossplane gives developers a Kubernetes-native, continuously reconciled API for resources that teams request repeatedly.

2. Competing reconciliation: two tools "correcting" each other's changes, duplicate creates, or one tool destroying what the other created.
3. So removing a claim from Git cannot silently trigger deletion of real cloud data; deletes become an explicit, reviewed action.
4. Argo CD treats the resource as healthy as soon as it exists, so later sync waves start before the infrastructure is actually ready.
5. Terraform foundations and Argo CD first, then the platform controllers and policy, then teams. If a lower layer depended on a higher one (Crossplane creating the cluster it runs on) you could not rebuild from nothing.

11 Further reading

- Argo CD documentation: "ApplicationSet", "Sync Waves", "Resource Health" and "Declarative Setup".
- Crossplane documentation: "Composition", "Providers" and guidance on using it with GitOps tools.
- Terraform / OpenTofu documentation: "State", "Remote State" and `import`.
- CNCF Platforms White Paper and the CNCF Platform Engineering maturity model.
- *Platform Engineering on Kubernetes* by Mauricio Salatino.

★ A Mini Internal Developer Platform

Level 18 capstone ~14 hours

1 Goal and outcome

Build a small but real **internal developer platform (IDP)** on a laptop. A developer fills in one form in a portal; minutes later a new service repository exists, its infrastructure is requested through a Kubernetes API, policy has checked it, and Argo CD has deployed it. By the end you will have:

- a **Backstage** portal with a **catalog** (`shop-api` as a `Component`, its database as a `Resource`) and a **software template** that scaffolds a service repository;
- a **golden path** service skeleton: Dockerfile, CI workflow, Kubernetes manifests and `catalog-info.yaml`;
- a **platform GitOps repository** (`platform-gitops`) with an Argo CD **ApplicationSet** that turns every `teams/<team>/<env>` folder into an Application;
- a **Crossplane** `ObjectStorage` API (XRD + Composition) so a team requests a bucket with ten lines of YAML;
- **policy as code** in two layers: **Conftest/Rego** in CI and **Kyverno** at admission;
- **Terraform** for the foundations (the cluster and the Argo CD bootstrap), with a clear ownership contract.

This exercises every topic: Internal Developer Platforms §18.1, Developer Portals §18.2, Self-service Infrastructure §18.3, Golden Paths §18.4, Service Catalogs §18.5, Infrastructure Templates §18.6, Policy as Code §18.7, Backstage §18.8, Crossplane §18.9 and Integrating Argo CD, Terraform and Kubernetes into a Platform §18.10.

2 Architecture

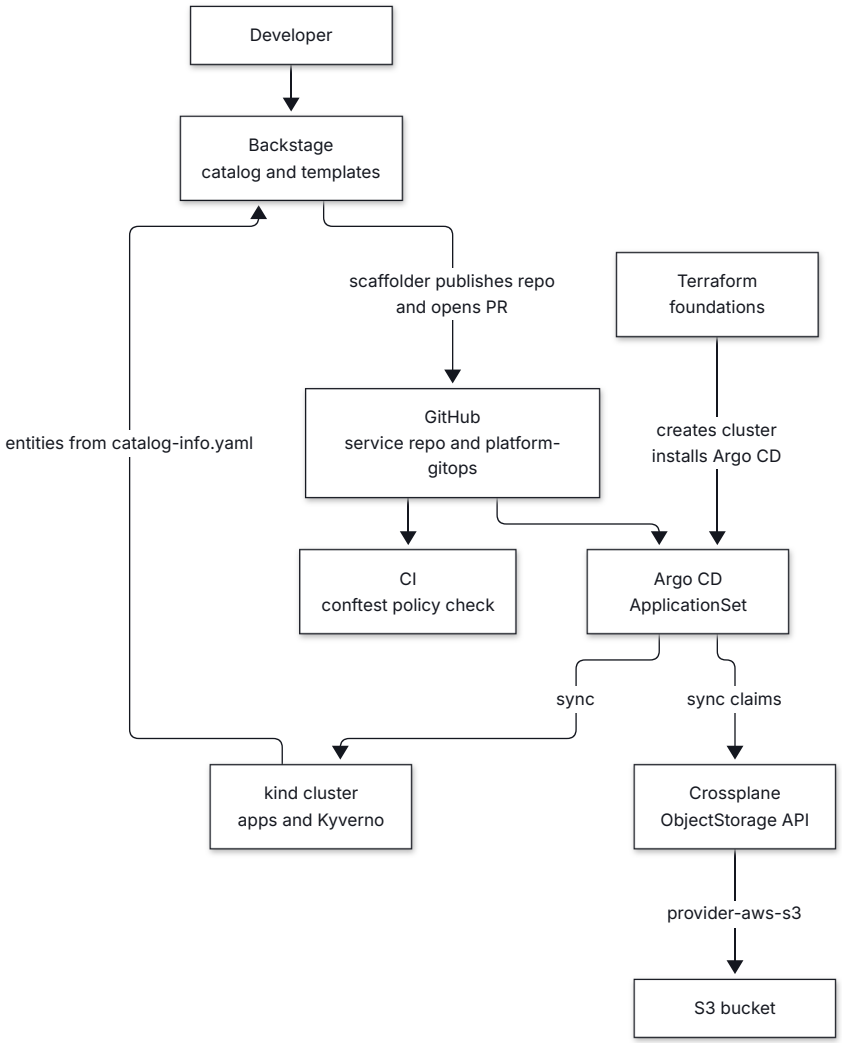


Figure 18-1

What happens when a developer clicks "Create":

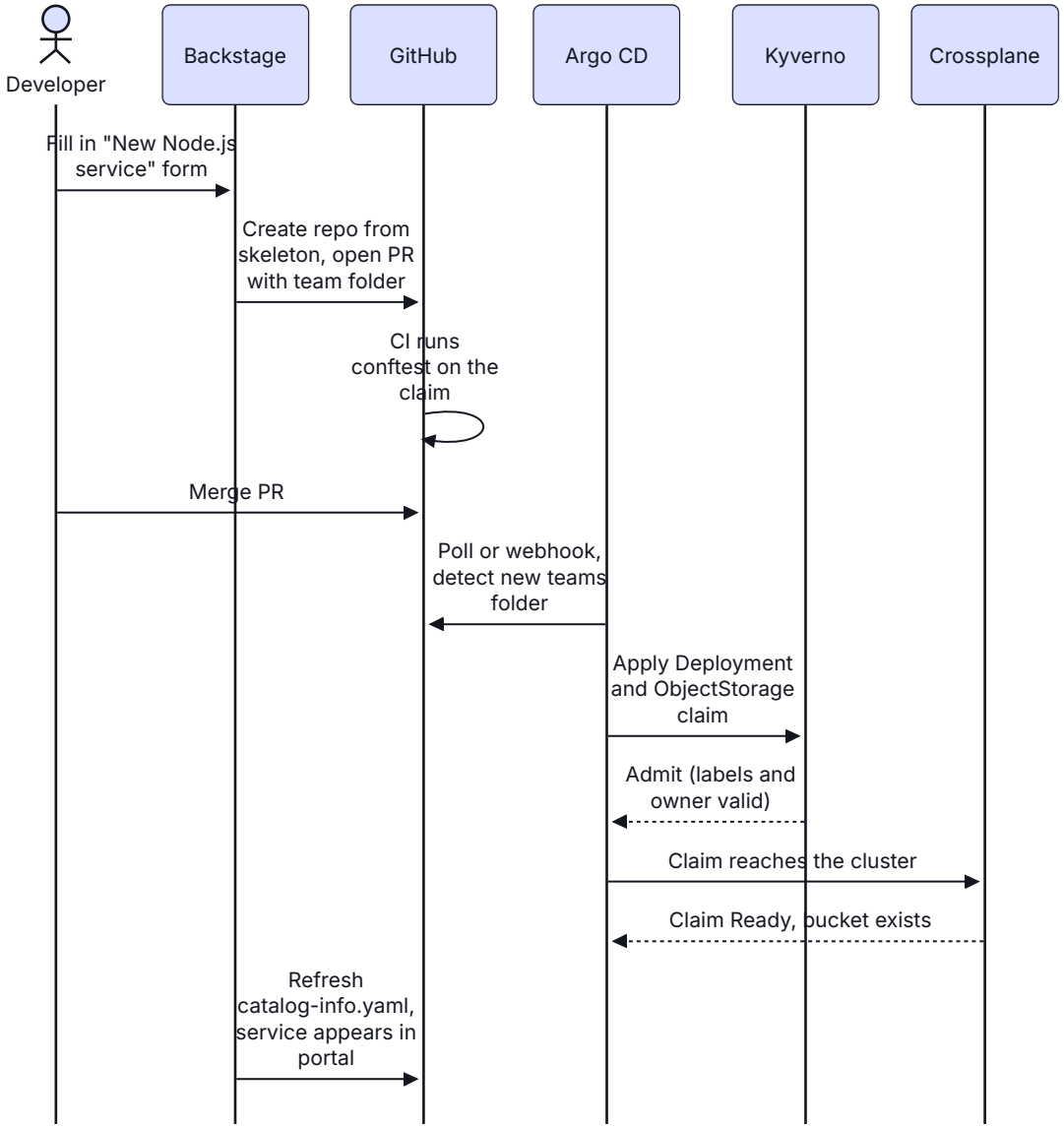


Figure 18-2

3 Prerequisites and cost

- Docker, `kind`, `kubectl`, `helm`, `git`, `terraform` (or `tofu`), `node` 20+ with `yarn` for Backstage, `gh` (GitHub CLI), and the `conftest` binary.

- A GitHub account where you can create two empty repositories: `platform-gitops` and (created by the template) a service repo. A personal access token is needed for the Backstage scaffolder (a fine-grained token limited to your lab repositories).
- About 8 GB of free RAM and 20 GB of disk (Backstage plus a kind cluster is the heavy part).
- **Cost:** the cluster and Backstage are free. Milestone 4 creates **one empty S3 bucket** in an AWS sandbox account, which costs effectively nothing, but set an AWS Budget or billing alarm first and run Teardown. You may skip the AWS part and still complete everything except the final "bucket is Ready" check.
- All versions below are examples; pin the versions you tested. Outputs are illustrative and will differ in names, ages and timings. Never commit credentials; use placeholders such as `<YOUR_TOKEN>`.

4 Build it step by step

Milestone 1 — Foundations with Terraform (topics 3, 6, 10)

Terraform owns what must exist before any cluster workload: here, the kind cluster and the Argo CD installation.

```hcl
# infra-foundations/main.tf
terraform {
  required_providers {
    kind = {
      source  = "tehcyx/kind"
      version = "~> 0.6"
    }
    helm = {
      source  = "hashicorp/helm"
      version = "~> 2.14"
    }
  }
}

resource "kind_cluster" "platform" {
  name           = "platform-lab"
  wait_for_ready = true
}

provider "helm" {
  kubernetes {
    config_path = kind_cluster.platform.kubeconfig_path
  }
}
```

```
resource "helm_release" "argocd" {
  name             = "argocd"
  namespace        = "argocd"
  create_namespace = true
  repository       = "https://argoproj.github.io/argo-helm"
  chart            = "argo-cd"
  version          = "7.6.0" # example version
}
```

BASH

```
cd infra-foundations
terraform init
terraform apply        # review the plan, then type yes
kind export kubeconfig --name platform-lab     # makes kubectl point at the new cluster
kubectl -n argocd get pods
```

Expect the `argocd-server`, `argocd-repo-server` and `argocd-application-controller` pods `Running`. If the `kind` provider is not available to you, create the cluster by hand with `kind create cluster --name platform-lab` and install Argo CD with `helm install`; the ownership idea is unchanged.

Milestone 2 — The platform GitOps repository (topics 3, 10)

BASH

```
mkdir -p platform-gitops/{bootstrap,platform/{policy,crossplane},teams/team-
  shop/dev,policy}
cd platform-gitops && git init -b main
```

The ApplicationSet turns any `teams/<team>/<env>` folder into an Application, so onboarding a team is adding a folder. Replace the repository URL with your own.

YAML

```
# bootstrap/teams-appset.yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: team-resources
  namespace: argocd
spec:
  goTemplate: true
  goTemplateOptions: ["missingkey=error"]
  generators:
    - git:
        repoURL: https://github.com/<YOUR_GITHUB_USER>/platform-gitops.git
        revision: main
        directories:
          - path: teams/*/*
```

```
  template:
    metadata:
      name: '{{ index .path.segments 1 }}-{{ index .path.segments 2 }}'
    spec:
      project: default
      source:
        repoURL: https://github.com/<YOUR_GITHUB_USER>/platform-gitops.git
        targetRevision: main
        path: '{{ .path.path }}'
      destination:
        server: https://kubernetes.default.svc
        namespace: '{{ index .path.segments 1 }}'
      syncPolicy:
        automated:
          prune: false
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
```

Add a root Application so platform components sync before team content (negative sync waves, see topic 10 §18.10):

YAML

```yaml
# bootstrap/kyverno-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: kyverno
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  project: default
  source:
    repoURL: https://kyverno.github.io/kyverno/
    chart: kyverno
    targetRevision: 3.3.0        # example version, pin the one you tested
  destination:
    server: https://kubernetes.default.svc
    namespace: kyverno
  syncPolicy:
    automated: {selfHeal: true}
    syncOptions: [CreateNamespace=true, ServerSideApply=true]
```

Commit and push `platform-gitops`, then `kubectl apply -f bootstrap/` (the Crossplane Application follows the same pattern with the chart from `https://charts.crossplane.io/stable`).

Milestone 3 — Policy as code, two layers (topic 7)

Layer one runs in CI on the claim files before merge:

REGO

```
# policy/claims.rego
package main

import rego.v1

deny contains msg if {
  input.kind == "ObjectStorage"
  input.spec.parameters.owner ≠ input.metadata.namespace
  msg := sprintf("ObjectStorage %q: owner %q must equal its namespace %q",
  [input.metadata.name, input.spec.parameters.owner, input.metadata.namespace])
}

deny contains msg if {
  input.kind == "ObjectStorage"
  not input.spec.parameters.region
  msg := sprintf("ObjectStorage %q: spec.parameters.region is required",
  [input.metadata.name])
}
```

YAML

```
# teams/team-shop/dev/assets.yaml
apiVersion: platform.example.org/v1alpha1
kind: ObjectStorage
metadata:
  name: shop-assets
  namespace: team-shop
spec:
  parameters:
    owner: team-shop
    region: us-east-1
```

BASH

```
conftest test teams/*/*/*.yaml
```

Expected: `1 test, 1 passed, 0 warnings, 0 failures, 0 exceptions` (similar to this). The same policy can be checked without Conftest using `opa eval` or `opa test`; the Rego is identical.

Layer two is a Kyverno `ClusterPolicy` that enforces the same convention at admission, so a `kubectl apply` that bypasses CI is still stopped. Start it in `Audit` mode and switch to `Enforce` once clean:

```yaml
# platform/policy/require-team-label.yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-team-label
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: deployments-need-team-label
      match:
        any:
          - resources:
              kinds: ["Deployment"]
      validate:
        message: "Deployments must carry the label 'team'."
        pattern:
          metadata:
            labels:
              team: "?*"
```

Milestone 4 — Crossplane `ObjectStorage` API (topics 3, 6, 9)

Install Crossplane (Helm or an Argo CD Application), then the S3 provider and function from Crossplane §18.9: `provider-aws-s3`, `function-patch-and-transform`, a `ProviderConfig` with sandbox credentials stored as a Secret (lab only; production uses IRSA or Pod Identity), then the XRD (`XObjectStorage` / claim `ObjectStorage`) and a Composition that creates one `Bucket` with versioning and public-access-block. Put the XRD and Composition in `platform/crossplane/` so Argo CD owns them.

```bash
kubectl get providers,functions
kubectl get xrd
```

Wait until providers show `INSTALLED` and `HEALTHY` `True`. Then copy `assets.yaml` into the cluster by letting Argo CD sync it (Milestone 2 already points at `teams/team-shop/dev`) and watch it:

```bash
kubectl -n team-shop get objectstorage
kubectl get managed
```

Expect `SYNCED` and `READY` to become `True` for the claim and a managed `Bucket` (names and ages will differ).

Milestone 5 — Backstage catalog and template (topics 1, 2, 4, 5, 8)

Create the portal and register a first component:

```bash
npx @backstage/create-app@latest      # app name: shop-portal
cd shop-portal && yarn start
```

Register the `shop-api` entities with a `catalog-info.yaml` in a repository (or `catalog.locations` in `app-config.yaml`), as described in Backstage §18.8:

```yaml
# catalog-info.yaml
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: shop-api
  description: Node.js API for the shop
  annotations:
    github.com/project-slug: <YOUR_GITHUB_USER>/shop-api
    argocd/app-name: team-shop-dev
spec:
  type: service
  lifecycle: production
  owner: group:default/team-shop
  dependsOn:
    - resource:default/shop-assets
---
apiVersion: backstage.io/v1alpha1
kind: Resource
metadata:
  name: shop-assets
  description: S3 bucket for static assets, provisioned by Crossplane
spec:
  type: s3-bucket
  owner: group:default/team-shop
```

Then add the golden-path template. It takes a name and team, copies a skeleton, publishes a repository, and opens a pull request against `platform-gitops` that adds the team folder:

```yaml
# templates/nodejs-service/template.yaml
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: nodejs-service
  title: New Node.js service
  description: Scaffolds a service repo on the golden path
spec:
```

```
  owner: group:default/platform-team
  type: service
  parameters:
    - title: Service details
      required: [name, team]
      properties:
        name:
          type: string
          pattern: '^[a-z][a-z0-9-]{2,30}$'
        team:
          type: string
          description: Owning team namespace, for example team-shop
  steps:
    - id: fetch
      name: Fetch skeleton
      action: fetch:template
      input:
        url: ./skeleton
        values:
          name: ${{ parameters.name }}
          team: ${{ parameters.team }}
    - id: publish
      name: Create repository
      action: publish:github
      input:
        repoUrl: github.com?owner=<YOUR_GITHUB_USER>&repo=${{ parameters.name }}
        defaultBranch: main
    - id: register
      name: Register in catalog
      action: catalog:register
      input:
        repoContentsUrl: ${{ steps['publish'].output.repoContentsUrl }}
        catalogInfoPath: /catalog-info.yaml
  output:
    links:
      - title: Repository
        url: ${{ steps['publish'].output.remoteUrl }}
```

The skeleton directory holds a `Dockerfile`, a `.github/workflows/ci.yaml`, a `k8s/` folder with a Deployment (carrying the `team` label), and a `catalog-info.yaml` using `${{ values.name }}`. Add `GITHUB_TOKEN` to the environment of the Backstage backend (`GITHUB_TOKEN=<YOUR_TOKEN> yarn start`) and an `integrations.github` entry in `app-config.local.yaml`.

Milestone 6 — End to end (topic 10)

1. In Backstage, open Create, choose **New Node.js service**, name it `orders-api`, team `team-shop`.
2. Confirm the repo exists, then copy its `k8s/` manifests into `platform-gitops/teams/team-shop/dev/orders-api/` (or add a second scaffolder step that opens this PR automatically).

3. Open a PR; CI runs `conftest test`; merge it.
4. Watch Argo CD:

```bash
kubectl -n argocd get applications
kubectl -n team-shop get deploy,objectstorage
```

5. Refresh the catalog and confirm `orders-api` appears with an owner, a repository link and an Argo CD annotation.

5 Tasks

- [ ] **T1 — Bootstrap the foundations** *(≈60 min, intermediate; Milestone 1; topics 3, 10)*
  - **Goal:** Terraform creates the cluster and installs Argo CD.
  - **Steps:**
    1. Write `main.tf`, run `terraform init` and `terraform apply`.
    2. Run `terraform plan` again and read the "no changes" result.
    3. Delete the `argocd` namespace by hand and run `terraform plan` to see the drift.
  - **Done when:** Argo CD pods are `Running`, a second plan shows no changes, and you can explain the drift plan.
- [ ] **T2 — Onboard a team by adding a folder** *(≈45 min, intermediate; Milestone 2; topics 3, 10)*
  - **Goal:** the ApplicationSet creates Applications from directories.
  - **Steps:**
    1. Push `platform-gitops` and apply `bootstrap/`.
    2. Add `teams/team-pay/dev/` with one ConfigMap and push.
    3. Observe the new Application, then remove the folder.
  - **Done when:** `kubectl -n argocd get applications` listed `team-pay-dev` while the folder existed, with no Argo CD config change.
- [ ] **T3 — Make the policy fail, then pass** *(≈45 min, intermediate; Milestone 3; topic 7)*
  - **Goal:** the CI guardrail blocks a bad claim.
  - **Steps:**
    1. Create a claim in namespace `team-pay` with `owner: team-shop`.
    2. Run `conftest test` and read the failure message.
    3. Fix it and re-run, then add a CI workflow step that runs it on pull requests.

- **Done when:** the failing run exits non-zero, the fixed run exits 0, and a PR shows the check.

- [ ] **T4 — Audit, then enforce, at admission** *(≈45 min, advanced; Milestone 3; topic 7)*
  - **Goal:** feel the safe rollout of a Kyverno rule.
  - **Steps:**
    1. Apply the `Audit` policy and create a Deployment without a `team` label.
    2. Read the policy report (`kubectl get policyreport -A`).
    3. Switch to `Enforce` and retry.
  - **Done when:** the audit run reports a violation without blocking, and the enforce run rejects the Deployment with your message.

- [ ] **T5 — Ship the `ObjectStorage` API** *(≈90 min, advanced; Milestone 4; topics 3, 6, 9)*
  - **Goal:** a claim becomes a real bucket.
  - **Steps:**
    1. Install Crossplane, the provider, the function, XRD and Composition.
    2. Let Argo CD sync `assets.yaml` and wait for `Ready`.
    3. Check the bucket exists with `aws s3api head-bucket` and its public-access block is on.
  - **Done when:** `kubectl get objectstorage -n team-shop` shows `READY True` and the AWS CLI confirms the bucket. Then delete the claim and confirm the bucket is removed.

- [ ] **T6 — Publish a golden-path template** *(≈120 min, advanced; Milestone 5; topics 1, 2, 4, 5, 8)*
  - **Goal:** a developer creates a working service from a form.
  - **Steps:**
    1. Run Backstage, register `shop-api` and `shop-assets` in the catalog.
    2. Add the template and skeleton, then create `orders-api`.
    3. Open the entity page and find its owner, repo link and dependency graph.
  - **Done when:** a new repository exists from the template and the `orders-api` entity is visible in the catalog with a correct owner.

- [ ] **T7 — Wire the whole path and break it** *(≈90 min, advanced; Milestone 6; topic 10)*
  - **Goal:** understand ordering and ownership failures.
  - **Steps:**
    1. Complete the end-to-end flow once.

2. Temporarily remove the sync wave from the Crossplane Application and sync a fresh cluster, noting the claim error about an unknown kind.
3. Restore the wave. Then try to delete a claim from Git and confirm `prune: false` leaves the bucket until you remove it deliberately.

- **Done when:** you have logs showing the ordering failure and its fix, and you can state which tool owns each resource.

- [ ] **T8 — Prove it** *(≈90 min, advanced; all milestones; all topics)*
  - **Goal:** rebuild the core of the platform from memory and explain it.
  - **Steps:**
    1. On a clean cluster, without this guide, stand up Argo CD, one policy, and one Crossplane claim delivered through an ApplicationSet.
    2. Draw the ownership table: who owns the cluster, the bucket, the app, and the policy.
    3. Write a half-page explanation of why a developer should not need to know any of these tools.
  - **Done when:** the pieces work end to end and your explanation names the golden path, the control plane and the guardrails.

6 Verification checklist

- [ ] `terraform plan` in `infra-foundations` shows no changes.
- [ ] `kubectl -n argocd get applications` lists the kyverno, crossplane and team applications as `Synced` and `Healthy`.
- [ ] `conftest test teams/*/*/*.yaml` exits 0 on `main` and non-zero on a deliberately bad claim.
- [ ] A Deployment without a `team` label is rejected once the Kyverno policy is in `Enforce`.
- [ ] `kubectl -n team-shop get objectstorage` shows `READY True` (if you used AWS).
- [ ] The Backstage catalog shows `shop-api`, `shop-assets` and the scaffolded service, each with an owner.
- [ ] No credentials are committed: `git log -p | grep -i 'secret_access'` finds nothing.

7 Teardown

```bash
# 1. Delete claims first so Crossplane removes the cloud resources
kubectl -n team-shop delete objectstorage --all
kubectl get managed          # wait until empty
aws s3 ls | grep -i shop     # confirm no lab bucket remains

# 2. Remove the cluster and foundations
cd infra-foundations && terraform destroy

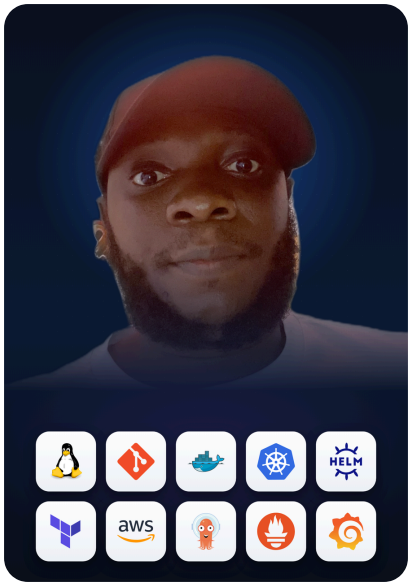
# 3. Remove local artefacts and revoke the token you created for Backstage
rm -rf ../shop-portal
```

Delete the lab GitHub repositories you no longer need, and delete the fine-grained token and any AWS access key you created. Check the AWS Billing console once more the next day.

8 Stretch goals and what to learn next

- Add a second template for a `PostgresInstance` claim and show it in the portal as a `Resource` with a dependency edge.
- Add Argo CD health checks for claims (see topic 10) so sync waves wait for `Ready`.
- Add a scheduled `terraform plan` in CI to detect drift in the foundations.
- Measure the platform: time from "Create" click to running service, and number of tickets avoided.
- Next: Level 19: Architecture §19.1 turns these building blocks into whole-system trade-offs, and Level 20: Cost Optimization §20.1 shows how to keep a platform's cloud bill under control.

About the Author



Ubong Eduok

Author and instructor, DevOps Roadmap: Beginner to Senior

Ubong Eduok created the DevOps Roadmap, the open-source, hands-on curriculum that this book sets in print, along with its companion **devops-deploy-toolkit** for deploying real applications with an AI pair-deployer.

The roadmap is free to read and released under the MIT License, so that anyone can use it, including in their own courses and projects.

GitHub: github.com/mascot200
Roadmap: github.com/mascot200/devops-roadmap
Toolkit: github.com/mascot200/devops-deploy-toolkit

COLOPHON

This book is set in Source Serif 4 for the text, Inter for headings, tables and diagrams, and JetBrains Mono for code, with DejaVu Sans and Noto Emoji supplying a few symbols. All are open-licensed typefaces. Trim size is 7.5 by 9.25 inches.

It was typeset directly from the Markdown of the repository at commit faf55fa, so the book and the repository can be kept in step. The 570 diagrams were drawn from their Mermaid sources as vector art. The staircase motif has one step for each of the twenty-one levels. The typesetting toolchain is in the `book/` folder of the repository, and you can rebuild the whole book from it with a single command.

Do not aim to know 50 tools.

Aim to take a software application and independently make it production-ready.

Secure · Observable · Scalable · Highly available · Automatically deployable · Recoverable · Cost-efficient

This is the full curriculum behind the open-source DevOps Roadmap, set down in print across six volumes. Twenty-one levels climb from Linux, networking and Git through Docker, CI/CD, AWS, Terraform, Kubernetes and Helm, on to observability, security, SRE, disaster recovery, GitOps, platform engineering, architecture and cost.

Every topic explains what a thing is and why it matters in production, shows it in a real-world system, draws the mechanism, walks you through it step by step, then hands you tasks you can verify and questions a senior engineer would expect you to answer. Nine projects chain it all into one evolving production system built around a single sample application.

Volume 5: Reliability and Platforms at Scale
Parts VII and VIII: SRE, Disaster Recovery, Advanced Kubernetes, GitOps, Platform Engineering



Ubong Eduok
Author and instructor of the DevOps Roadmap: Beginner to Senior.
github.com/mascot200/devops-roadmap

DEVOPS · CLOUD · PLATFORM ENGINEERING · BEGINNER TO SENIOR